

MiniZinc Handbook

Release 2.10.1

Peter J. Stuckey, Kim Marriott, Guido Tack

Aug 31, 2026

Contents

1 Overview **3**

1.1 Introduction . . . 5
1.1.1 Structure . . . 6
1.1.2 How to Read This . . . 6

1.2 Installation . . . 7
1.2.1 Microsoft Windows . . . 7
1.2.2 Linux . . . 8
1.2.2.1 Snap . . . 8
1.2.2.2 AppImage . . . 8
1.2.2.3 Archive . . . 9
1.2.3 Apple macOS . . . 9
1.2.4 Docker . . . 9
1.2.4.1 Adding MiniZinc to your own image . . . 10
1.2.4.2 Available images . . . 10
1.2.5 Adding Third-party Solvers . . . 10

1.3 First steps with MiniZinc . . . 11
1.3.1 The MiniZinc IDE . . . 11
1.3.2 The MiniZinc command line tool . . . 16

2 A MiniZinc Tutorial **19**

2.1 Basic Modelling in MiniZinc . . . 21
2.1.1 Our First Example . . . 21
2.1.2 An Arithmetic Optimisation Example . . . 26
2.1.3 Datafiles and Assertions . . . 27
2.1.4 Real Number Solving . . . 30
2.1.5 Basic structure of a model . . . 32

2.2 More Complex Models . . . 35
2.2.1 Arrays and Sets . . . 35
2.2.2 Global Constraints . . . 46
2.2.3 Conditional Expressions . . . 47
2.2.4 Enumerated Types . . . 50
2.2.5 Complex Constraints . . . 54

2.2.6 Array Access Constraints . . . 56
2.2.7 Partiality and Relational Semantics . . . 60
2.2.8 Set Constraints . . . 60
2.2.9 Putting it all together . . . 63

2.3 Predicates and Functions . . . 65
2.3.1 Global Constraints . . . 65
2.3.1.1 Alldifferent . . . 65
2.3.1.2 Cumulative . . . 66
2.3.1.3 Table . . . 67
2.3.1.4 Regular . . . 69
2.3.2 Defining Predicates . . . 72
2.3.3 Defining Functions . . . 74
2.3.4 Reflection Functions . . . 76
2.3.5 Local Variables . . . 77
2.3.6 Context . . . 79
2.3.7 Local Constraints and Partiality . . . 80
2.3.8 Domain Reflection Functions . . . 82
2.3.9 Scope . . . 84

2.4 Option Types . . . 85
2.4.1 Declaring and Using Option Types . . . 85
2.4.2 Option Types in Scheduling Problems . . . 86
2.4.3 Hidden Option Types . . . 87
2.4.4 Option Type Parameters . . . 88

2.5 Tuple and record types . . . 91
2.5.1 Declaring and using tuples and records . . . 92
2.5.2 Using type-inst synonyms . . . 93
2.5.3 Types with both var and par members . . . 94
2.5.4 Example: rectangle packing using records . . . 95

2.6 Search . . . 97
2.6.1 Finite Domain Search . . . 97
2.6.2 Search Annotations . . . 101
2.6.3 Annotations . . . 103
2.6.4 Restart . . . 105
2.6.5 Warm Starts . . . 106

2.7 Effective Modelling Practices in MiniZinc . . . 109
2.7.1 Variable Bounds . . . 109
2.7.2 Effective Generators . . . 110
2.7.3 Redundant Constraints . . . 112
2.7.4 Modelling Choices . . . 113
2.7.5 Multiple Modelling and Channels . . . 116
2.7.6 Symmetry . . . 117
2.7.6.1 Static Symmetry Breaking . . . 118
2.7.6.2 Other Examples of Symmetry . . . 120

2.8 Boolean Satisfiability Modelling in MiniZinc . . . 121
2.8.1 Modelling Integers . . . 121
2.8.2 Modelling Disequality . . . 122

2.8.3 Modelling Cardinality . . . 123

2.9 FlatZinc and Flattening . . . 131
2.9.1 Flattening Expressions . . . 131
2.9.1.1 Simplification and Evaluation . . . 132
2.9.1.2 Defining Subexpressions . . . 132
2.9.1.3 FlatZinc constraint form . . . 133
2.9.1.4 Bounds analysis . . . 133
2.9.1.5 Objectives . . . 134
2.9.2 Linear Expressions . . . 134
2.9.3 Unrolling Expressions . . . 135
2.9.4 Arrays . . . 136
2.9.5 Reification . . . 139
2.9.6 Predicates . . . 141
2.9.7 Let Expressions . . . 143

3 User Manual **149**

3.1 The MiniZinc Command Line Tool . . . 151
3.1.1 Basic Usage . . . 151
3.1.2 Adding Solvers . . . 152
3.1.2.1 Configuration files . . . 152
3.1.2.2 Configuration for MIP solvers . . . 153
3.1.3 Options . . . 153
3.1.3.1 General options . . . 153
3.1.3.2 Solving options . . . 154
3.1.3.3 Flattener input options . . . 155
3.1.3.4 Solution output options . . . 158
3.1.4 User Configuration Files . . . 159
3.1.5 Command-line Parameter Files . . . 161

3.2 The MiniZinc IDE . . . 163
3.2.1 Editing files . . . 163
3.2.1.1 Editing functions . . . 164
3.2.1.2 Fonts and dark mode . . . 164
3.2.2 Configuring and Running a Solver . . . 164
3.2.2.1 Running a model . . . 164
3.2.2.2 Solver configurations . . . 165
3.2.2.3 Automatic Solution Checking . . . 167
3.2.2.4 Compiling a model . . . 168
3.2.2.5 Profiling a model . . . 168
3.2.3 Working With Projects . . . 169
3.2.4 Submitting Solutions to Online Courses . . . 169
3.2.5 Configuration Options . . . 170
3.2.5.1 Locating the MiniZinc installation . . . 170
3.2.5.2 Adding Third-Party Solvers . . . 173
3.2.5.3 Editing options . . . 176
3.2.5.4 Output options . . . 177

3.3 Visualising solutions in the MiniZinc IDE . . . 179
3.3.1 Visualisation library . . . 179

3.3.2 Pre-defined visualisations . . . 179
3.3.3 Custom visualisations . . . 180
3.3.3.1 Example visualisation . . . 182
3.3.3.2 MiniZincIDE JavaScript API . . . 187

3.4 CP-Profiler . . . 191
3.4.1 Using the profiler . . . 191
3.4.2 Traditional Tree Visualisation . . . 192
3.4.3 Alternative Search Tree Visualisations . . . 195
3.4.4 Similar Subtree Analysis . . . 199
3.4.5 Execution Comparison by Merging . . . 200
3.4.6 Replaying Search . . . 201
3.4.7 Nogood Analysis . . . 202
3.4.8 The protocol (high level) . . . 203
3.4.9 The protocol (low level) . . . 205

3.5 Solving Technologies and Solver Backends . . . 207
3.5.1 Constraint Programming Solvers . . . 207
3.5.1.1 Gecode . . . 207
3.5.1.2 Chuffed . . . 208
3.5.1.3 OR-Tools . . . 208
3.5.2 Mixed-Integer Programming Solvers . . . 208
3.5.2.1 MIP-Aware Modeling (But Mostly Useful for All Backends) . . . 208
3.5.2.2 Useful Flattening Parameters . . . 209
3.5.2.3 Some Solver Options and Changed Default Values . . . 210
3.5.2.4 Subtour Elimination Constraints . . . 211
3.5.2.5 Unified Domains (MIPdomains) . . . 211
3.5.2.6 Indicator Constraints . . . 211
3.5.2.7 Quadratic Constraints and Objectives (MIQCP) . . . 211
3.5.2.8 Pools of User Cuts and Lazy Constraints . . . 212
3.5.2.9 Warm Starts . . . 212
3.5.3 Non-Linear Solvers via NL File Format . . . 212

3.6 Automatic Solution Checking, Model Validation, and Benchmarking . . . 215
3.6.1 Autonomous Automatic Solution Checking and Benchmarking . . . 215
3.6.2 Model Validation: Automatic Solution Checking with a Checker Model 216
3.6.2.1 Basic checker models . . . 217
3.6.2.2 Detailed feedback . . . 218
3.6.2.3 Instance data in checker models . . . 220
3.6.2.4 Checking optimisation problems . . . 221
3.6.2.5 Hidden variables . . . 221

3.7 Assumptions (experimental) . . . 225
3.7.1 Using `assume` . . . 225
3.7.2 Interpreting the core . . . 226

3.8 Black-Box Propagators (experimental) . . . 227
3.8.1 Defining a black-box propagator in MiniZinc . . . 228
3.8.1.1 Value propagators . . . 228
3.8.1.2 Bounds propagators . . . 229
3.8.2 Selecting the external implementation . . . 229
3.8.2.1 Locating the executable or library . . . 230

3.8.3 The external interface . . . 230
3.8.3.1 Dynamically loaded library (DLL) mode . . . 231
3.8.3.2 Executable (subprocess) mode . . . 232

3.9 Globalizer . . . 235
3.9.1 Basic Usage . . . 235
3.9.2 Caveats . . . 237
3.9.3 Supported Constraints . . . 237
3.9.4 Using Globalizer in the MiniZinc IDE . . . 238
3.9.5 How it works . . . 239
3.9.6 Performance tips . . . 240
3.9.7 Limitations / Future work . . . 241

3.10 FindMUS . . . 243
3.10.1 Basic Usage . . . 243
3.10.1.1 Commandline arguments . . . 243
3.10.1.2 Example . . . 246
3.10.2 Using FindMUS in the MiniZinc IDE . . . 249
3.10.3 How it works . . . 252
3.10.4 Performance tips . . . 253
3.10.5 Limitations / Future work . . . 253

3.11 Mzn-Analyse . . . 255
3.11.1 Basic Usage . . . 255
3.11.1.1 Passes: read/write models & json . . . 255
3.11.1.2 Passes: Manipulate Models . . . 256
3.11.1.3 Passes: Special Passes . . . 256
3.11.2 Examples . . . 257
3.11.3 Limitations / Future work . . . 258

3.12 Using MiniZinc in Jupyter Notebooks . . . 259
3.12.1 Installation . . . 259
3.12.2 Basic usage . . . 259

3.13 Python Interface . . . 263

3.14 JavaScript Interface . . . 265

3.15 Installation from Source Code . . . 267

4 Reference Manual 269

4.1 Specification of MiniZinc . . . 271
4.1.1 Introduction . . . 271
4.1.1.1 Original authors. . . . 272
4.1.2 Notation . . . 272
4.1.3 Overview of a Model . . . 273
4.1.3.1 Evaluation Phases . . . 273
4.1.3.2 Run-time Outcomes . . . 274
4.1.3.3 Output . . . 274
4.1.4 Syntax Overview . . . 275
4.1.4.1 Character Set . . . 275

4.1.4.2 Comments . . . 275
4.1.4.3 Identifiers . . . 275
4.1.5 High-level Model Structure . . . 276
4.1.5.1 Model Instance Files . . . 277
4.1.5.2 Namespaces . . . 277
4.1.5.3 Scopes . . . 278
4.1.6 Types and Type-insts . . . 278
4.1.6.1 Properties of Types . . . 278
4.1.6.2 Instantiations . . . 279
4.1.6.3 Type-insts . . . 279
4.1.6.4 Type-inst expression overview . . . 279
4.1.6.5 Built-in Scalar Types and Type-insts . . . 281
4.1.6.6 Built-in Compound Types and Type-insts . . . 282
4.1.6.7 User-defined Types and Type-insts . . . 289
4.1.6.8 Constrained Type-insts . . . 290
4.1.7 Expressions . . . 291
4.1.7.1 Expressions Overview . . . 291
4.1.7.2 Operators . . . 293
4.1.7.3 Expression Atoms . . . 295
4.1.8 Items . . . 310
4.1.8.1 Include Items . . . 310
4.1.8.2 Variable Declaration Items . . . 310
4.1.8.3 Enum Items . . . 311
4.1.8.4 Type-inst Synonym Items . . . 313
4.1.8.5 Assignment Items . . . 314
4.1.8.6 Constraint Items . . . 314
4.1.8.7 Solve Items . . . 314
4.1.8.8 Output Items . . . 315
4.1.8.9 Annotation Items . . . 316
4.1.8.10 User-defined Operations . . . 316
4.1.9 Annotations . . . 320
4.1.10 Partiality . . . 321
4.1.10.1 Partial Assignments . . . 322
4.1.10.2 Partial Predicate/Function and Annotation Arguments . . . 323
4.1.10.3 Partial Array Accesses . . . 323
4.1.11 Built-in Operations . . . 324
4.1.11.1 Comparison Operations . . . 324
4.1.11.2 Arithmetic Operations . . . 324
4.1.11.3 Logical Operations . . . 326
4.1.11.4 Set Operations . . . 327
4.1.11.5 Array Operations . . . 328
4.1.11.6 Coercion Operations . . . 329
4.1.11.7 String Operations . . . 329
4.1.11.8 Bound and Domain Operations . . . 330
4.1.11.9 Option Type Operations . . . 331
4.1.11.10 Other Operations . . . 332
4.1.12 Content-types . . . 333
4.1.13 JSON support . . . 333
4.1.14 Full grammar . . . 335
4.1.14.1 Items . . . 335
4.1.14.2 Type-Inst Expressions . . . 337

4.1.14.3 Expressions . . . 338
4.1.14.4 Miscellaneous Elements . . . 341

4.2 The MiniZinc library . . . 343
4.2.1 Standard Library . . . 343
4.2.1.1 Built-in functions and operators . . . 343
4.2.1.2 Annotations . . . 394
4.2.1.3 Option type support . . . 409
4.2.1.4 Array sorting operations . . . 412
4.2.1.5 Language information . . . 413
4.2.1.6 Compiler options . . . 413
4.2.1.7 Conditionals . . . 415
4.2.1.8 Reflection operations . . . 417
4.2.1.9 Assertions and debugging functions . . . 420
4.2.1.10 Random Number Generator builtins . . . 424
4.2.1.11 Structured Output . . . 426
4.2.2 Global constraints . . . 426
4.2.2.1 All-Different and related constraints . . . 427
4.2.2.2 Lexicographic constraints . . . 428
4.2.2.3 Sorting constraints . . . 432
4.2.2.4 Channeling constraints . . . 434
4.2.2.5 Counting constraints . . . 435
4.2.2.6 Array-related constraints . . . 443
4.2.2.7 Set-related constraints . . . 445
4.2.2.8 Mathematical constraints . . . 445
4.2.2.9 Packing constraints . . . 450
4.2.2.10 Scheduling constraints . . . 456
4.2.2.11 Graph constraints . . . 459
4.2.2.12 Extensional constraints (table, regular etc.) . . . 472
4.2.2.13 Machine learning constraints . . . 476
4.2.2.14 Deprecated constraints . . . 477
4.2.3 FlatZinc builtins . . . 478
4.2.3.1 Integer FlatZinc builtins . . . 478
4.2.3.2 Bool FlatZinc builtins . . . 482
4.2.3.3 Set FlatZinc builtins . . . 484
4.2.3.4 Float FlatZinc builtins . . . 487
4.2.3.5 FlatZinc builtins added in MiniZinc 2.0.0. . . . 492
4.2.3.6 FlatZinc builtins added in MiniZinc 2.0.2. . . . 493
4.2.3.7 FlatZinc builtins added in MiniZinc 2.1.0. . . . 494
4.2.3.8 FlatZinc builtins added in MiniZinc 2.1.1. . . . 494
4.2.3.9 FlatZinc builtins added in MiniZinc 2.2.1. . . . 495
4.2.3.10 FlatZinc builtins added in MiniZinc 2.3.3. . . . 495
4.2.3.11 FlatZinc builtins added in MiniZinc 2.5.2. . . . 495
4.2.3.12 FlatZinc builtins added in MiniZinc 2.7.1. . . . 496
4.2.3.13 Deprecated FlatZinc builtins . . . 497
4.2.4 Additional declarations for Gecode . . . 497
4.2.4.1 Additional Gecode search annotations . . . 497
4.2.4.2 Additional Gecode constraints . . . 499
4.2.5 Additional declarations for Chuffed . . . 501
4.2.5.1 Additional Chuffed search annotations . . . 501
4.2.5.2 Other declarations . . . 501
4.2.6 MiniZincIDE tools . . . 502

4.2.6.1 MiniZincIDE solution visualisation tools . . . 502
4.2.7 MiniZinc definitions for the MiniZinc diversity extension. . . . 513
4.2.7.1 Annotations provided to guide the MiniZinc diversity extension. . 513
4.2.7.2 Distance functions provided in the MiniZinc diversity extension. . 515
4.2.8 Experimental Features . . . 515
4.2.8.1 Assumptions . . . 515
4.2.8.2 On Restart . . . 516

4.3 Interfacing Solvers to Flatzinc . . . 519
4.3.1 Specification of FlatZinc . . . 519
4.3.1.1 Comments . . . 520
4.3.1.2 Types . . . 520
4.3.1.3 Values and expressions . . . 522
4.3.1.4 FlatZinc models . . . 522
4.3.1.5 Predicate declarations . . . 523
4.3.1.6 Parameter declarations . . . 523
4.3.1.7 Variable declarations . . . 524
4.3.1.8 Constraints . . . 524
4.3.1.9 Solve item . . . 525
4.3.1.10 Annotations . . . 525
4.3.2 Specification of FlatZinc-JSON . . . 529
4.3.3 Output . . . 534
4.3.3.1 Solution output . . . 534
4.3.3.2 Statistics output . . . 537
4.3.3.3 Unsatisfiable core output (experimental) . . . 538
4.3.3.4 Error and warning output . . . 539
4.3.4 Solver-specific Libraries . . . 539
4.3.4.1 Standard predicates . . . 539
4.3.4.2 Solver-specific predicates . . . 540
4.3.4.3 Parameter names in solver-specific libraries . . . 541
4.3.4.4 Reified and half-reified predicates . . . 541
4.3.5 Command-Line Interface and Standard Options . . . 542
4.3.6 Solver Configuration Files . . . 543
4.3.7 Grammar . . . 546

4.4 Machine-readable JSON output format . . . 551
4.4.1 Solution messages . . . 551
4.4.2 Checker messages . . . 552
4.4.3 Status messages . . . 552
4.4.4 Statistics messages . . . 553
4.4.5 Timestamp messages . . . 553
4.4.6 Comment messages . . . 553
4.4.7 Trace messages . . . 554
4.4.8 Profiling messages . . . 554
4.4.9 Paths messages . . . 555
4.4.10 Error messages . . . 555
4.4.10.1 Locations . . . 556
4.4.10.2 Stack traces . . . 556
4.4.11 Warning messages . . . 556
4.4.12 Core messages . . . 557

MiniZinc is a free and open-source constraint modeling language. You can use MiniZinc to model constraint satisfaction and optimization problems in a high-level, solver-independent way, taking advantage of a large library of pre-defined constraints.

This handbook consists of four parts: Section 1 covers installation and basic steps; Section 2 is a tutorial-style introduction into modelling with MiniZinc; Section 3 is a user manual for the individual tools in the MiniZinc tool chain; and Section 4 is a reference to the language.

This documentation is licensed under a Creative Commons Attribution-NoDerivatives 4.0 International License[5]. This means that you are free to copy and redistribute the material in any medium or format for any purpose, even commercially. However, you must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use. If you remix, transform, or build upon the material, you may **not** distribute the modified material.

[5] http://creativecommons.org/licenses/by-nd/4.0/

Part 1

Overview

CHAPTER 1.1

Introduction

MiniZinc is a language for specifying constrained optimization and decision problems over integers and real numbers. A MiniZinc model does not dictate *how* to solve the problem - the MiniZinc compiler can translate it into different forms suitable for a wide range of *solvers*, such as Constraint Programming (CP), Mixed Integer Linear Programming (MIP) or Boolean Satisfiability (SAT) solvers.

The MiniZinc language lets users write models in a way that is close to a mathematical formulation of the problem, using familiar notation such as existential and universal quantifiers, sums over index sets, or logical connectives like implications and if-then-else statements. Furthermore, MiniZinc supports defining predicates and functions that let users structure their models (similar to procedures and functions in regular programming languages).

MiniZinc models are usually *parametric*, i.e., they describe a whole *class* of problems rather than an individual problem instance. That way, a model of, say, a vehicle routing problem could be reused to generate weekly plans, by instantiating it with the updated customer demands for the upcoming week.

MiniZinc is designed to interface easily to different backend solvers. It does this by transforming an input MiniZinc model and data file into a FlatZinc model. FlatZinc models consist of variable declarations and constraint definitions as well as a definition of the objective function if the problem is an optimization problem. The translation from MiniZinc to FlatZinc makes use of a library of function and predicate definitions for the particular target solver, which allows the MiniZinc compiler to produce specialised FlatZinc that only contains the types of variables and constraints that are supported by the target. In particular, MiniZinc allows the specification of *global constraints* by *decomposition*. Furthermore, *annotations* of the model let the user fine tune the behaviour of the solver, independent of the declarative meaning of the model.

1.1.1 Structure

This documentation consists of four parts. *The first part* (page 5) includes this introduction and then describes how to download and install MiniZinc and how to make your first steps using the MiniZinc IDE and the command line tools. *The second part* (page 21) is a tutorial introduction to modelling with MiniZinc, from basic syntax and simple modelling techniques to more advanced topics. It also explains how MiniZinc is compiled to FlatZinc. *The third part* (page 151) is a user manual for the tools that make up the MiniZinc tool chain. Finally, *The fourth part* (page 271) contains the reference documentation for MiniZinc, including a definition of the MiniZinc language, documentation on how to interface a solver to FlatZinc, and an annotated list of all predicates and functions in the MiniZinc standard library.

1.1.2 How to Read This

If you are new to MiniZinc, follow the installation instructions and the introduction to the IDE and then work your way through the tutorial. Most example code can be downloaded, but it is sometimes more useful to type it in yourself to get the language into your muscle memory! If you need help, visit the MiniZinc web site at http://www.minizinc.org where you find a discussion forum.

Some of the code examples are shown in boxes like the one below. If a code box has a heading, it usually lists the name of a file that can be downloaded from http://minizinc.org/doc-latest/en/downloads/index.html.

Listing 1.1: A code example (`dummy.mzn`)

```
% Just an example
var int: x;
solve satisfy;
```

Throughout the documentation, some concepts are defined slightly more formally in special sections like this one.

More details

These sections can be skipped over if you just want to work through the tutorial for the first time, but they contain important information for any serious MiniZinc user!

Finally, if you find a mistake in this documentation, please report it through our GitHub issue tracker at https://github.com/MiniZinc/libminizinc/issues.

CHAPTER 1.2

Installation

A complete installation of the MiniZinc system comprises the MiniZinc *compiler tool chain*, one or more *solvers*, and (optionally) the *MiniZinc IDE*.

The easiest way to get a full, working MiniZinc system is to use the **bundled binary packages**, available from https://www.minizinc.org/downloads/.

The bundled binary packages contain the compiler and IDE, as well as the following solvers: Gecode, Chuffed, COIN-OR CBC, HiGHS, and OR-Tools CP-SAT. Interfaces to Gurobi, CPLEX, SCIP and XPress are also included and can be used if the corresponding solvers are installed.

Alternatively, it is possible to compile all components from source code, as described in *Installation from Source Code* (page 267).

1.2.1 Microsoft Windows

To install the bundled binary packages, simply download the installer, double-click to execute it, and follow the prompts. **Note:** 32 bit Windows installations are no longer supported in the bundled binary packages.

After installation is complete, you can find the MiniZinc IDE installed as a Windows application. The file extensions `.mzn`, `.dzn` and `.fzn` are linked to the IDE, so double-clicking any MiniZinc file should open it in the IDE.

If you want to use MiniZinc from a command prompt, you need to add the installation directory to the PATH environment variable. In a Windows command prompt you could use the following command:

```
C:\>setx PATH "%PATH%;C:\Program Files\MiniZinc 2.10.1 (bundled)\"
```

1.2.2 Linux

The MiniZinc bundled binary distribution for Linux is provided using three different distribution methods: *Snap*, *AppImage*, and an *archive* containing all necessary files. The capabilities of each package is the same, but it depends on your linux distribution which method is the easiest to use. **Note:** linux bundles are compiled to be compatible with many linux distributions using a *x86_64* architecture, but some dependencies on low level libraries cannot be avoided. Make use of the MiniZinc issue trackers if you encounter any problems.

1.2.2.1 Snap

The MiniZinc Snap is the easiest way to install the MiniZinc bundle if Snap (https://snapcraft.io/) is available on your system. The installation will provide the `minizinc` command within terminal environments and will add an application entry for the MiniZinc IDE. The IDE can be started from the command line using `minizinc.ide`. The following command will commence the installation the MiniZinc snap:

```
$ snap install minizinc --classic
```

Note: the MiniZinc snap requires the *classic* permission model to access solvers that are installed somewhere else on your system.

An advantage of using the MiniZinc snap is that it will be automatically updated whenever a new version of the snap is available. Using snap channels it is also possible to automatically follow the developmental versions of MiniZinc. More information about the MiniZinc snap can be found in the Snap Store.

1.2.2.2 AppImage

The MiniZinc AppImage provides a way of installing the MiniZinc bundles without any added requirements. AppImages (https://appimage.org/) provide self-contained versions of applications with all their required dependencies. To use the MiniZinc AppImage you simply download the AppImage, make it executable, and run:

```
$ chmod +x MiniZincIDE-2.10.1-x86_64.AppImage
$ ./MiniZincIDE-2.10.1-x86_64.AppImage
```

AppImages have no standard integration with the desktop environment. A script has been added to the MiniZinc AppImage to integrate the application into both the terminal and the desktop environment. The one requirement for this script is that the AppImage *CANNOT be moved* after the install script has been run. A suggestion would be to store the AppImages in `~/Applications/` or `/opt/Applications/`. The following commands move the AppImage to the second location and install `minizinc` and `MiniZincIDE` to `/usr/local/bin` and will add a desktop entry and its corresponding icon to `$HOME/.local/share/{applications,icons}`:

```
$ mv ./MiniZincIDE-2.10.1-x86_64.AppImage /opt/Applications/
$ /opt/Applications/MiniZincIDE-2.10.1-x86_64.AppImage install
```

Note: The installation locations can be changed using the environmental variables `$BIN_LOCATION` and `$DESKTOP_LOCATION`. The default behaviour can be achieved using the following command:

```
$ BIN_LOCATION=/usr/local/bin DESKTOP_LOCATION=$HOME/.local/share ./MiniZincIDE-2.
↪10.1-x86_64.AppImage install
```

1.2.2.3 Archive

The MiniZinc archive bundle is straightforward, but doesn't offer any automatic integrations with the desktop environment. After downloading, uncompress the archive, for example in your home directory or any other location where you want to install it:

```
$ tar xf MiniZincIDE-2.10.1-bundle-linux-x86_64.tgz
```

This will unpack MiniZinc into a directory that is called the same as the archive file (without the `.tgz`). You can start the MiniZinc IDE using the `MiniZincIDE.sh` shell script, which sets up a number of paths and environment variables.

In order to use the command line tools, after unpacking, make the following changes to some environment variables (assuming a bash-like shell):

```
$ export PATH=MiniZincIDE-2.10.1-bundle-linux-x86_64/bin:$PATH
$ export LD_LIBRARY_PATH=MiniZincIDE-2.10.1-bundle-linux-x86_64/lib:$LD_LIBRARY_
↪PATH
$ export QT_PLUGIN_PATH=MiniZincIDE-2.10.1-bundle-linux-x86_64/plugins:$QT_PLUGIN_
↪PATH
```

1.2.3 Apple macOS

The macOS bundled binary distribution works with any version of OS X starting from 10.9. After downloading the disk image (`.dmg`) file, double click it if it doesn't open automatically. You will see an icon for the MiniZinc IDE that you can drag into your Applications folder (or anywhere else you want to install MiniZinc).

In order to use the MiniZinc tools from a terminal, you need to add the path to the MiniZinc installation to the PATH environment variable. If you installed the MiniZinc IDE in the standard Applications folder, the following command will add the correct path:

```
$ export PATH=/Applications/MiniZincIDE.app/Contents/Resources:$PATH
```

1.2.4 Docker

Container images are published to the GitHub Container Registry at `ghcr.io/minizinc/minizinc`. They contain the compiler tool chain and the bundled solvers, but not the IDE.

To solve a model with the image directly:

```
docker run --rm -v "$PWD:/work" -w /work ghcr.io/minizinc/minizinc model.mzn␣
↪data.dzn
```

1.2.4.1 Adding MiniZinc to your own image

The `-dist` images contain nothing but the MiniZinc installation: no operating system, no C library, no shell. They are not runnable on their own, and exist to be copied into an image you already maintain:

```
FROM python:3.12-slim
COPY --from=ghcr.io/minizinc/minizinc:2.10.1-dist /opt/minizinc /opt/minizinc
ENV PATH=/opt/minizinc/bin:$PATH LD_LIBRARY_PATH=/opt/minizinc/lib
```

This is the recommended way to use MiniZinc in a larger image. Because the layer carries no operating system of its own, it never needs security updates: the base image remains yours, and you update it on your own schedule.

Use `-musl-dist` instead if your base uses musl rather than glibc, for example Alpine Linux, which also needs `apk add libstdc++`.

1.2.4.2 Available images

Every image is available for `linux/amd64` and `linux/arm64`.

Tag suffix	Contents
(none)	Runnable, glibc. Based on Google's `distroless/cc`
`-musl`	Runnable, musl
`-dist`	MiniZinc only, for `COPY --from`, glibc build
`-musl-dist`	MiniZinc only, for `COPY --from`, musl build

Versions are tagged as `2.10.1`, `2.10`, `2` and `latest`, each pointing at the newest release in its series, plus `edge` for the current development build. Combine the two, for example `ghcr.io/minizinc/minizinc:2.10-musl-dist`.

The runnable images are rebuilt weekly so that fixes to the underlying base image are picked up; the MiniZinc version they contain does not change, but the image digest does. If you need a byte-for-byte reproducible build, refer to an image by digest rather than by tag:

```
docker pull ghcr.io/minizinc/minizinc@sha256:...
```

1.2.5 Adding Third-party Solvers

Third party solvers for MiniZinc typically consist of two parts: a solver *executable*, and a solver-specific MiniZinc *library*. MiniZinc must be aware of the location of both the executable and the library in order to compile and run a model with that solver. Each solver therefore needs to provide a *configuration file* in a location where the MiniZinc toolchain can find it.

The easiest way to add a solver to the MiniZinc system is via the MiniZinc IDE. This is explained in Section 3.2.5.2. You can also add configuration files manually, as explained in Section 4.3.6.

CHAPTER 1.3

First steps with MiniZinc

We recommend using the bundled binary distribution of MiniZinc introduced in Section 1.2. It contains the MiniZinc IDE, the MiniZinc compiler, and several pre-configured solvers so you can get started straight away.

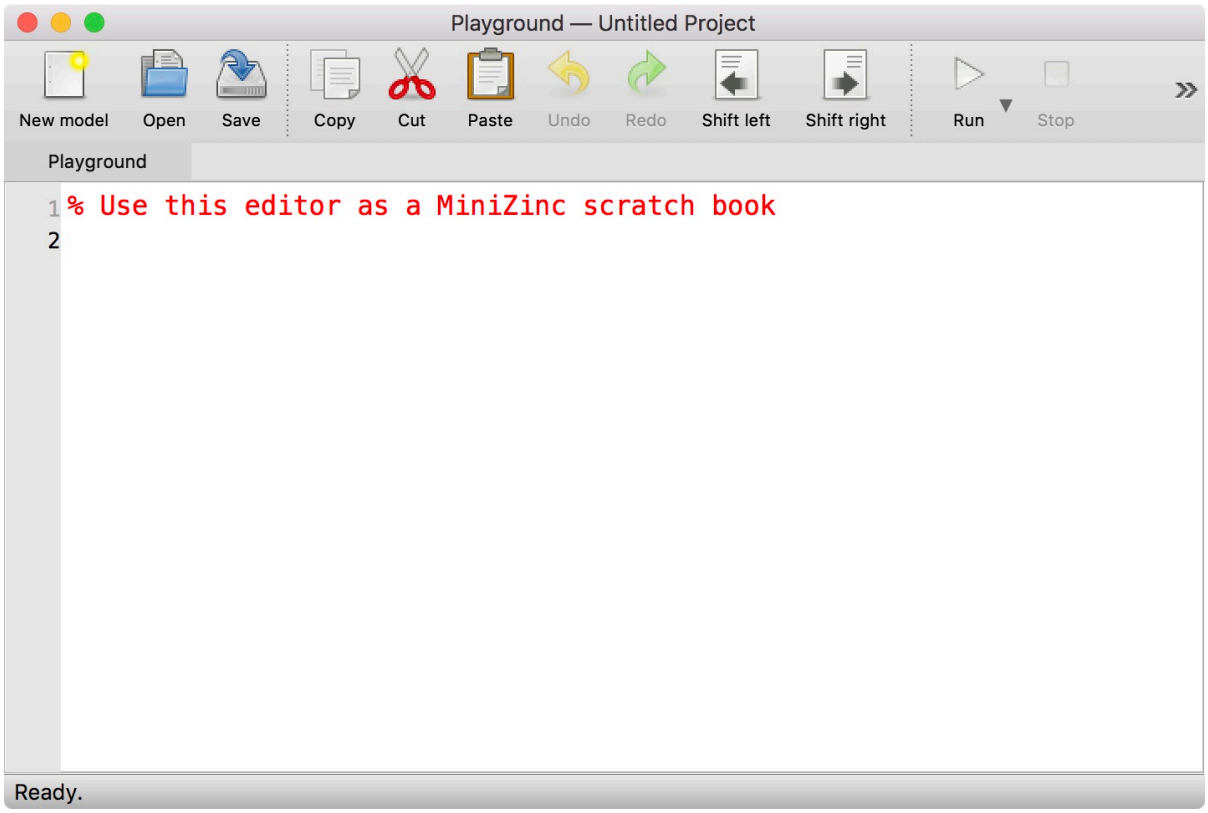
This section introduces the MiniZinc IDE and the command line tool `minizinc` using some very basic examples. This should be enough to get you started with the MiniZinc tutorial in Section 2 (in fact, you only need to be able to use one of the two, for instance just stick to the IDE if you are not comfortable with command line tools, or just use the `minizinc` command if you suffer from fear of mice).

1.3.1 The MiniZinc IDE

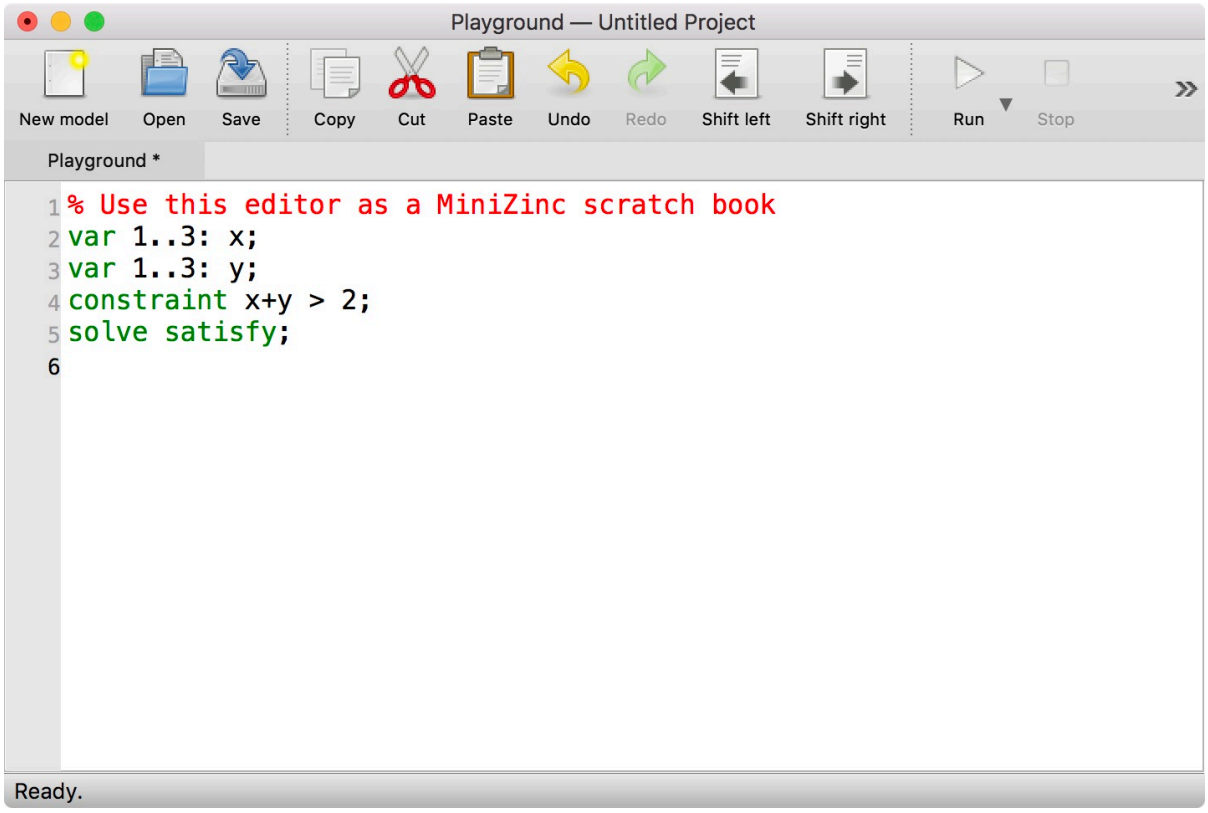
The MiniZinc IDE provides a simple interface to most of MiniZinc's functionality. It lets you edit model and data files, solve them with any of the solvers supported by MiniZinc, run debugging and profiling tools, and submit solutions to online courses (such as the MiniZinc Coursera courses).

When you open the MiniZinc IDE for the first time, it will ask you whether you want to be notified when an update is available. If you installed the IDE from sources, it may next ask you to locate your installation of the MiniZinc compiler. Please refer to Section 3.2.5 for more details on this.

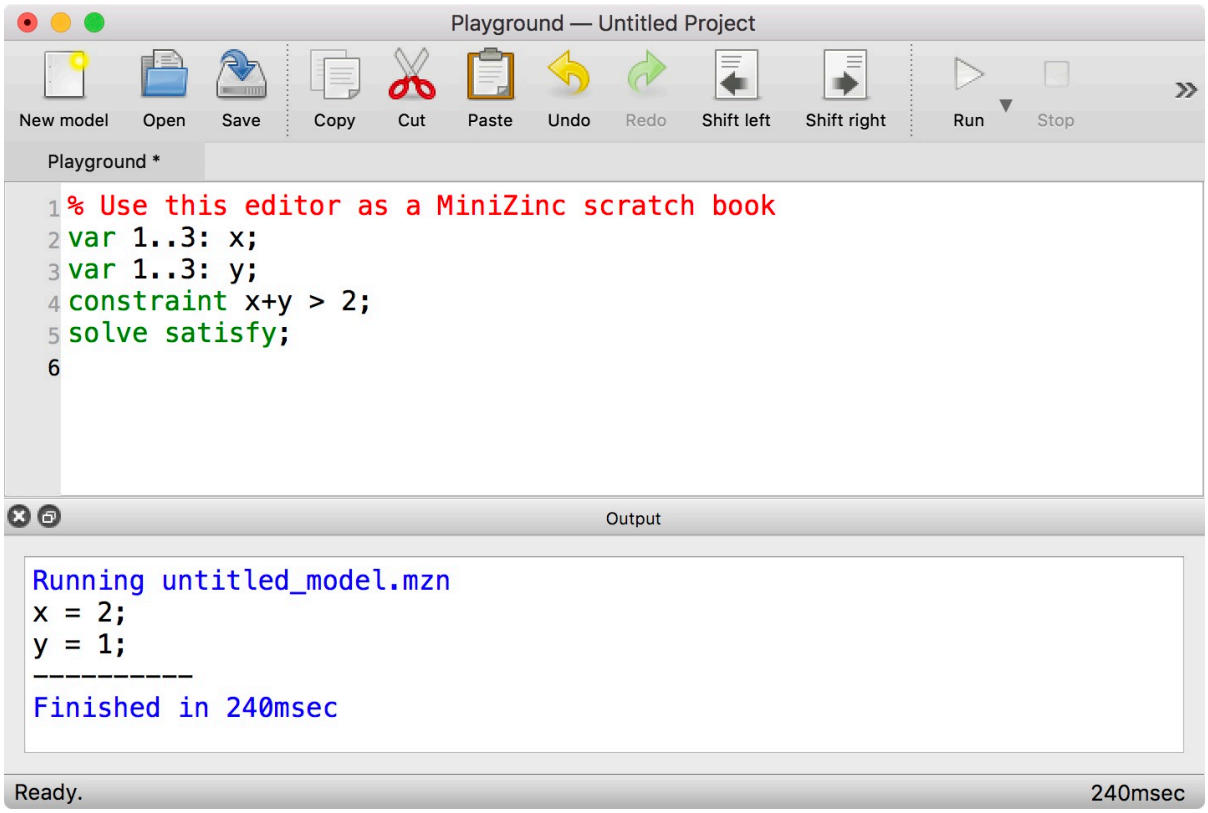
The IDE will then greet you with the *MiniZinc Playground*, a window that will look like this:



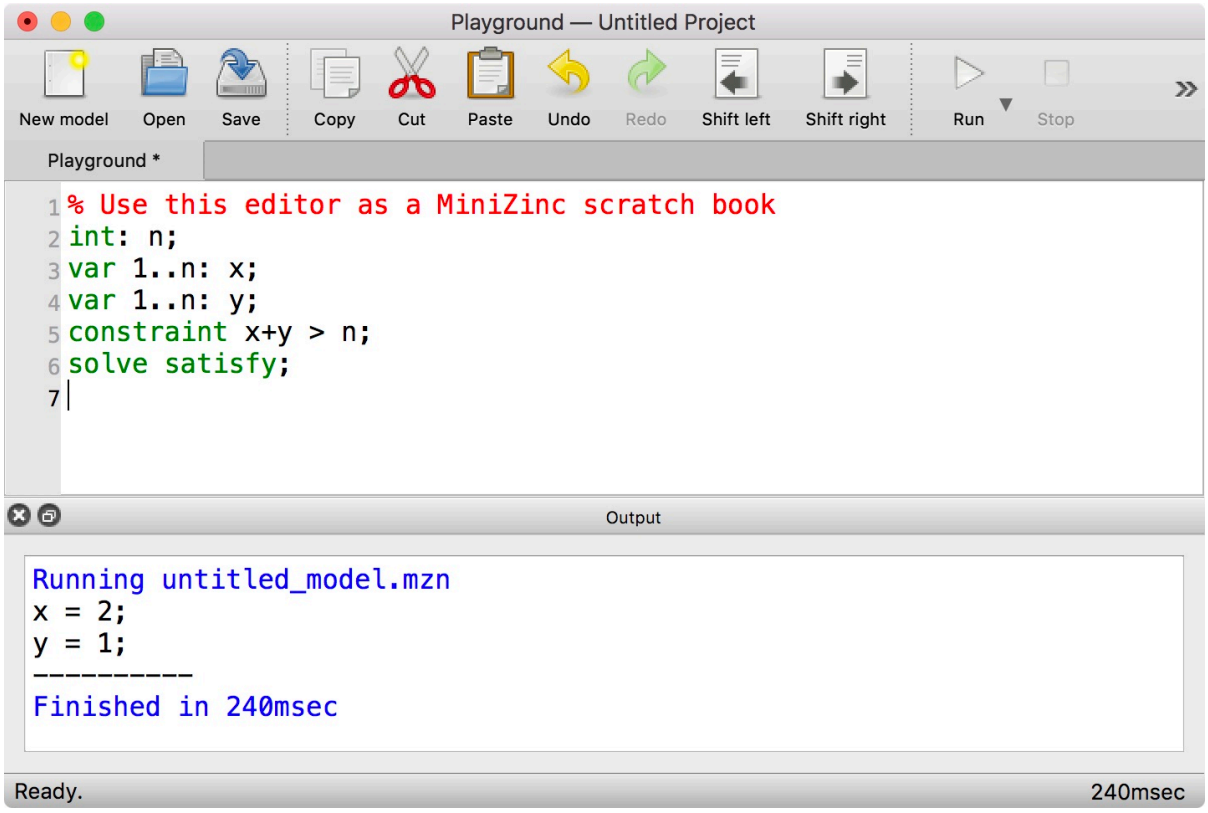
You can start writing your first MiniZinc model! Let's try something very simple:



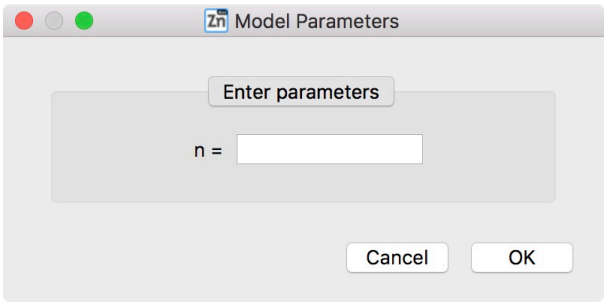
In order to solve the model, you click on the *Run* button in the toolbar, or use the keyboard shortcut *Ctrl+R* (or *command+R* on macOS):



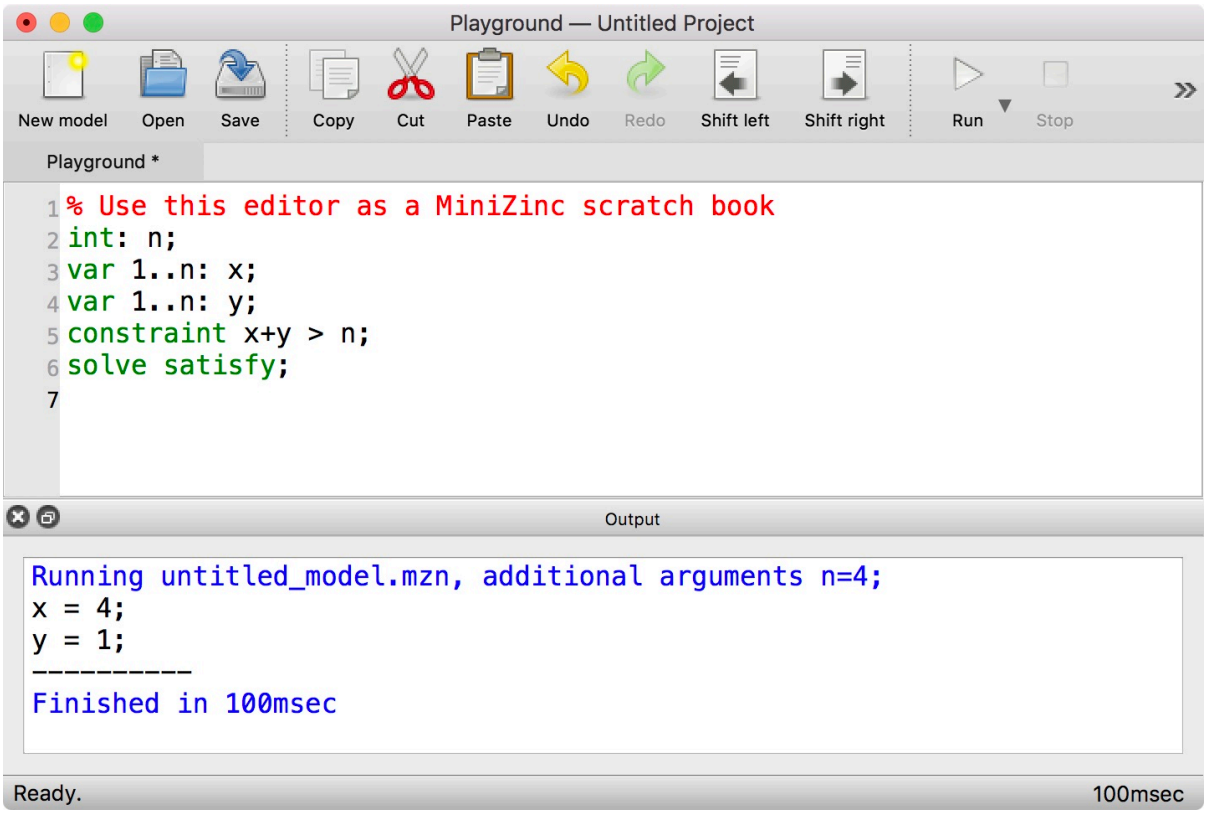
As you can see, an output window pops up that displays a solution to the problem you entered. Let us now try a model that requires some additional data.



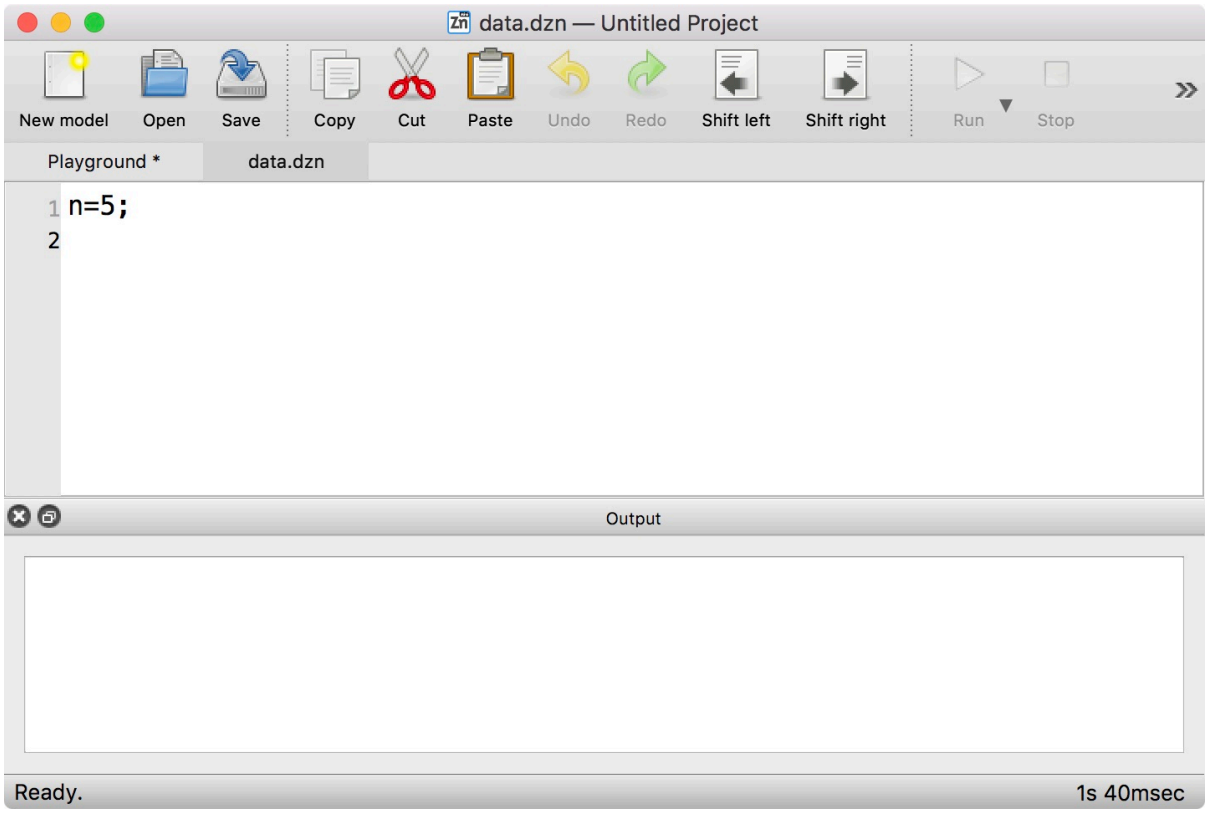
When you run this model, the IDE will ask you to enter a value for the parameter *n*:



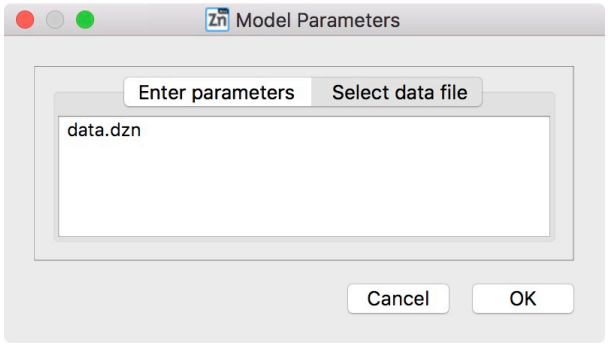
After entering, for example, the value 4 and clicking *Ok*, the solver will execute the model for *n=4*:



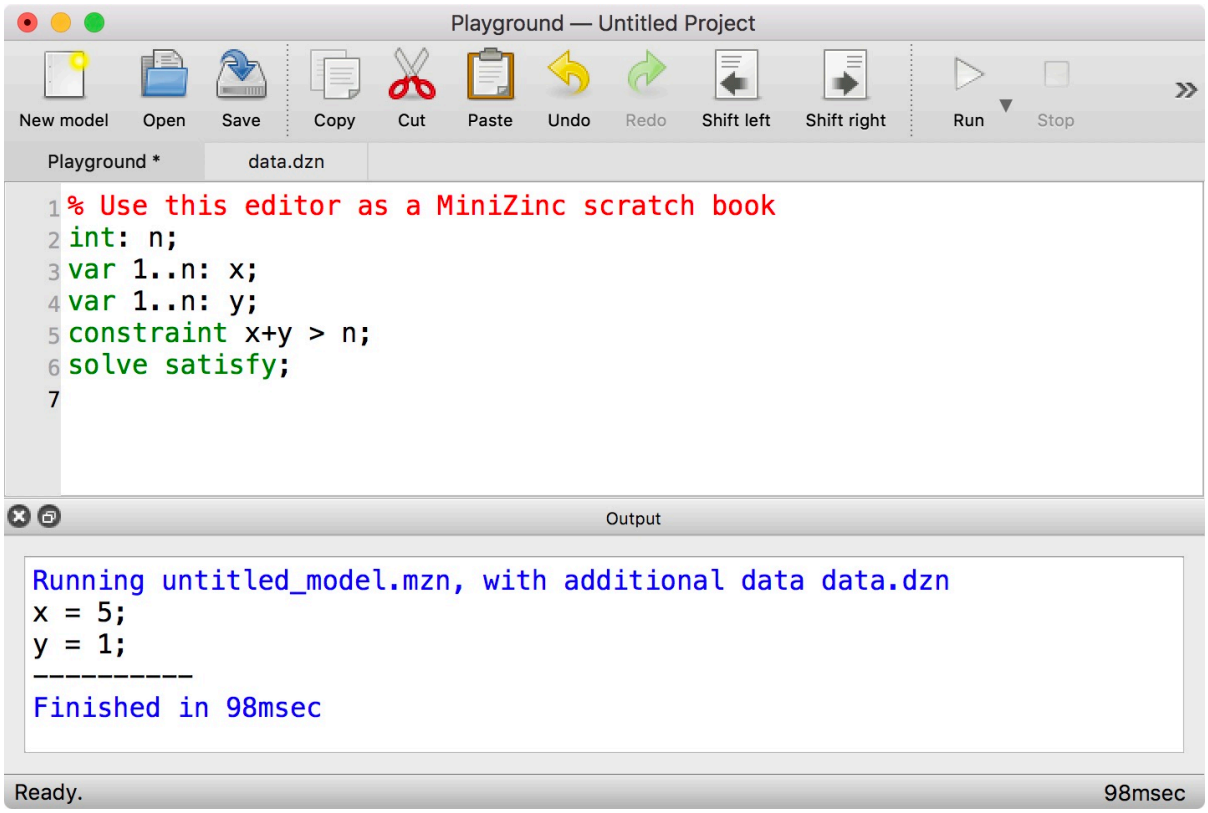
Alternatively, data can also come from a file. Let's create a new file with the data and save it as `data.dzn`:



When you now go back to the *Playground* tab and click *Run*, the IDE will give you the option to select a data file:



Click on the `data.dzn` entry, then on *Ok*, and the model will be run with the given data file:



Of course you can save your model to a file, and load it from a file, and the editor supports the usual functionality.

If you want to know more about the MiniZinc IDE, continue reading from Section 3.2.

1.3.2 The MiniZinc command line tool

The MiniZinc command line tool, `minizinc`, combines the functionality of the MiniZinc compiler, different solver interfaces, and the MiniZinc output processor. After installing MiniZinc from the bundled binary distribution, you may have to set up your `PATH` in order to use the command line tool (see Section 1.2).

Let's assume we have a file `model.mzn` with the following contents:

```
var 1..3: x;
var 1..3: y;
constraint x+y > 3;
solve satisfy;
```

You can simply invoke `minizinc` on that file to solve the model and produce some output:

```
$ minizinc model.mzn
x = 3;
y = 1;
----------
$
```

If you have a model that requires a data file (like the one we used in the IDE example above), you pass both files to `minizinc`:

```
$ minizinc model.mzn data.dzn
x = 5;
y = 1;
----------
$
```

The `minizinc` tool supports numerous command line options. One of the most useful options is `-a`, which switches between *one solution* mode and *all solutions* mode. For example, for the first model above, it would result in the following output:

```
$ minizinc -a model.mzn
x = 3;
y = 1;
----------
x = 2;
y = 2;
----------
x = 3;
y = 2;
----------
x = 1;
y = 3;
----------
x = 2;
y = 3;
----------
x = 3;
y = 3;
----------
==========
$
```

To learn more about the `minizinc` command, explore the output of `minizinc --help` or continue reading in Section 3.1.

Part 2

A MiniZinc Tutorial

CHAPTER 2.1

Basic Modelling in MiniZinc

In this section we introduce the basic structure of a MiniZinc model using two simple examples.

2.1.1 Our First Example



Fig. 1.1: Australian states

As our first example, imagine that we wish to colour a map of Australia as shown in Fig. 1.1. It is made up of seven different states and territories each of which must be given a colour so that adjacent regions have different colours.

Listing 1.1: A MiniZinc model `aust.mzn` for colouring the states and territories in Australia

```
% Colouring Australia using nc colours
int: nc = 3;

var 1..nc: wa;   var 1..nc: nt;  var 1..nc: sa;   var 1..nc: q;
var 1..nc: nsw;  var 1..nc: v;   var 1..nc: t;

constraint wa != nt;
constraint wa != sa;
constraint nt != sa;
constraint nt != q;
constraint sa != q;
constraint sa != nsw;
constraint sa != v;
constraint q != nsw;
constraint nsw != v;
solve satisfy;

output ["wa=\(wa)\t nt=\(nt)\t sa=\(sa)\n",
        "q=\(q)\t nsw=\(nsw)\t v=\(v)\n",
         "t=", show(t),  "\n"];
```

We can model this problem very easily in MiniZinc. The model is shown in Listing 1.1. The first line in the model is a comment. A comment starts with a `%` which indicates that the rest of the line is a comment. MiniZinc also has C-style block comments, which start with `/*` and end with `*/`.

The next part of the model declares the variables in the model. The line

```
int: nc = 3;
```

specifies a parameter in the problem which is the number of colours to be used. Parameters are similar to (constant) variables in most programming languages. They must be declared and given a type. In this case the type is `int`. They are given a value by an assignment. MiniZinc allows the assignment to be included as part of the declaration (as in the line above) or to be a separate assignment statement. Thus the following is equivalent to the single line above

```
int: nc;
nc = 3;
```

Unlike variables in many programming languages a parameter can only be given a *single* value, in that sense they are named constants. It is an error for a parameter to occur in more than one assignment.

The basic parameter types are integers (`int`), floating point numbers (`float`), Booleans (`bool`) and strings (`string`). Arrays and sets are also supported.

MiniZinc models can also contain another kind of variable called a *decision variable*. Decision variables are variables in the sense of mathematical or logical variables. Unlike parameters and variables in a standard programming language, the modeller does not need to give them a value. Rather the value of a decision variable is unknown and it is only when the MiniZinc model is executed that the solving system determines if the decision variable can be assigned a value that satisfies the constraints in the model and if so what this is.

In our example model we associate a *decision variable* with each region, `wa`, `nt`, `sa`, `q`, `nsw`, `v` and `t`, which stands for the (unknown) colour to be used to fill the region.

For each decision variable we need to give the set of possible values the variable can take. This is called the variable's *domain*. This can be given as part of the variable declaration and the type of the decision variable is inferred from the type of the values in the domain.

In MiniZinc decision variables can be Booleans, integers, floating point numbers, or sets. Also supported are arrays whose elements are decision variables. In our MiniZinc model we use integers to model the different colours. Thus each of our decision variables is declared to have the domain `1..nc` which is an integer range expression indicating the set $\{1, 2, \ldots, nc\}$ using the `var` declaration. The type of the values is integer so all of the variables in the model are integer decision variables.

Identifiers

Identifiers, which are used to name parameters and variables, are sequences of lower and uppercase alphabetic characters, digits and the underscore _ character. They must start with an alphabetic character. Thus `myName_2` is a valid identifier. MiniZinc *keywords* are not allowed to be used as identifier names, they are listed in *Identifiers* (page 275). Neither are MiniZinc *operators* allowed to be used as identifier names; they are listed in *Operators* (page 293).

MiniZinc carefully distinguishes between the two kinds of model variables: parameters and decision variables. The kinds of expressions that can be constructed using decision variables are more restricted than those that can be built from parameters. However, in any place that a decision variable can be used, so can a parameter of the same type.

Integer Variable Declarations

An integer parameter variable is declared as either:

```
int : <var-name>
<l> .. <u> : <var-name>
```

where `<l>` and `<u>` are fixed integer expressions.
An integer decision variable is declared as either:

```
var int : <var-name>
var <l>..<u> : <var-name>
```

where `<l>` and `<u>` are fixed integer expressions.

Formally the distinction between parameters and decision variables is called the *instantiation* of the variable. The combination of variable instantiation and type is called a

type-inst. As you start to use MiniZinc you will undoubtedly see examples of *type-inst* errors.

The next component of the model are the *constraints*. These specify the Boolean expressions that the decision variables must satisfy to be a valid solution to the model. In this case we have a number of not equal constraints between the decision variables enforcing that if two states are adjacent then they must have different colours.

Relational Operators

MiniZinc provides the relational operators:
equal (= or ==), not equal (!=), strictly less than (<), strictly greater than (>), less than or equal to (<=), and greater than or equal to (>=).

The next line in the model:

```
solve satisfy;
```

indicates the kind of problem it is. In this case it is a satisfaction problem: we wish to find a value for the decision variables that satisfies the constraints but we do not care which one.

The final part of the model is the *output* statement. This tells MiniZinc what to print when the model has been run and a solution is found.

Output and Strings

An output statement is followed by a *list* of strings. These are typically either string literals which are written between double quotes and use a C like notation for special characters, or an expression of the form `show(e)` where `e` is a MiniZinc expression. In the example `\n` represents the newline character and `\t` a tab.
There are also formatted varieties of `show` for numbers: `show_int(n,X)` outputs the value of integer `X` in at least $|n|$ characters, right justified if $n > 0$ and left justified otherwise; `show_float(n,d,X)` outputs the value of float `X` in at least $|n|$ characters, right justified if $n > 0$ and left justified otherwise, with d characters after the decimal point.
String literals must fit on a single line. Longer string literals can be split across multiple lines using the string concatenation operator `++`. For example, the string literal

```
"Invalid datafile: Amount of flour must be non-negative"
```

is equivalent to the string literal expression

```
"Invalid datafile: " ++
"Amount of flour must be non-negative"
```

MiniZinc supports interpolated strings. Expressions can be embedded directly in string literals, where a sub string of the form `"\(e)"` is replaced by the result of `show(e)`. For example `"t=\(t)\n"` produces the same string as `"t=" ++ show(t) ++ "\n"`.
A model can contain multiple output statements. In that case, all outputs are concatenated in the order they appear in the model.
Additionally, output statements can be annotated using a string literal giving a section name. These section names can be then used to filter the output using the `--only-sections` and `--not-sections` options.

We can evaluate our model by clicking the *Run* button in the MiniZinc IDE, or by typing

```
$ minizinc --solver gecode aust.mzn
```

where `aust.mzn` is the name of the file containing our MiniZinc model. We must use the file extension `.mzn` to indicate a MiniZinc model. The command `minizinc` with the option `--solver gecode` uses the Gecode finite domain solver to evaluate our model. If you use the MiniZinc binary distribution, this solver is in fact the default, so you can just run `minizinc aust.mzn` instead.

When we run this we obtain the result:

```
wa=3   nt=2    sa=1
q=3   nsw=2    v=3
t=1
----------
```

The line of 10 dashes `----------` is automatically added by the MiniZinc output to indi-

cate a solution has been found.

2.1.2 An Arithmetic Optimisation Example

Our second example is motivated by the need to bake some cakes for a fete at our local school. We know how to make two sorts of cakes (WARNING: please don't use these recipes at home). A banana cake which takes 250g of self-raising flour, 2 mashed bananas, 75g sugar and 100g of butter, and a chocolate cake which takes 200g of self-raising flour, 75g of cocoa, 150g sugar and 150g of butter. We can sell a chocolate cake for $4.50 and a banana cake for $4.00. And we have 4kg self-raising flour, 6 bananas, 2kg of sugar, 500g of butter and 500g of cocoa. The question is how many of each sort of cake should we bake for the fete to maximise the profit. A possible MiniZinc model is shown in Listing 1.2.

Listing 1.2: Model for determining how many banana and chocolate cakes to bake for the school fete (`cakes.mzn`)

```
% Baking cakes for the school fete

var 0..100: b; % no. of banana cakes
var 0..100: c; % no. of chocolate cakes

% flour
constraint 250*b + 200*c <= 4000;
% bananas
constraint 2*b  <= 6;
% sugar
constraint 75*b + 150*c <= 2000;
% butter
constraint 100*b + 150*c <= 500;
% cocoa
constraint 75*c <= 500;

% maximize our profit
solve maximize 400*b + 450*c;

output ["no. of banana cakes = \(b)\n",
        "no. of chocolate cakes = \(c)\n"];
```

The first new feature is the use of *arithmetic expressions*.

Integer Arithmetic Operators

MiniZinc provides the standard integer arithmetic operators. Addition (`+`), subtraction (`-`), multiplication (`*`), integer division (`div`) and integer modulus (`mod`). It also provides `+` and `-` as unary operators.
Integer modulus is defined to give a result a `mod` b that has the same sign as the dividend a. Integer division is defined so that $a = b * (a \texttt{ div } b) + (a \texttt{ mod } b)$.
MiniZinc provides standard integer functions for absolute value (`abs`) and power function (`pow`). For example `abs(-4)` and `pow(2,5)` evaluate to `4` and `32` respectively.
The syntax for arithmetic literals is reasonably standard. Integer literals can be decimal, hexadecimal or octal. For instance `0`, `5`, `123`, `0x1b7`, `0o777`.

The second new feature shown in the example is optimisation. The line

```
solve maximize 400 * b + 450 * c;
```

specifies that we want to find a solution that maximises the expression in the solve statement called the *objective*. The objective can be any kind of arithmetic expression. One can replace the keyword `maximize` by `minimize` to specify a minimisation problem.

When we run this we obtain the result:

```
no. of banana cakes = 2
no. of chocolate cakes = 2
----------
==========
```

The line `==========` is output automatically for optimisation problems when the system has proved that a solution is optimal.

2.1.3 Datafiles and Assertions

A drawback of this model is that if we wish to solve a similar problem the next time we need to bake cakes for the school (which is often) we need to modify the constraints in the model to reflect the ingredients that we have in the pantry. If we want to reuse the model then we would be better off to make the amount of each ingredient a parameter of the model and then set their values at the top of the model.

Even better would be to set the value of these parameters in a separate *data file*. MiniZinc (like most other modelling languages) allows the use of data files to set the value of parameters declared in the original model. This allows the same model to be easily used with different data by running it with different data files.

Data files must have the file extension `.dzn` to indicate a MiniZinc data file and a model can be run with any number of data files (though a variable/parameter can only be assigned a value in one file).

Listing 1.3: Data-independent model for determining how many banana and chocolate cakes to bake for the school fete (cakes2.mzn)

```
% Baking cakes for the school fete (with data file)

int: flour;  %no. grams of flour available
int: banana; %no. of bananas available
int: sugar;  %no. grams of sugar available
int: butter; %no. grams of butter available
int: cocoa;  %no. grams of cocoa available

constraint assert(flour >= 0,"Invalid datafile: " ++
                  "Amount of flour should be non-negative");
constraint assert(banana >= 0,"Invalid datafile: " ++
                  "Amount of banana should be non-negative");
constraint assert(sugar >= 0,"Invalid datafile: " ++
                  "Amount of sugar should be non-negative");
constraint assert(butter >= 0,"Invalid datafile: " ++
                  "Amount of butter should be non-negative");
constraint assert(cocoa >= 0,"Invalid datafile: " ++
                  "Amount of cocoa should be non-negative");

var 0..100: b; % no. of banana cakes
var 0..100: c; % no. of chocolate cakes

% flour
constraint 250*b + 200*c <= flour;
% bananas
constraint 2*b  <= banana;
% sugar
constraint 75*b + 150*c <= sugar;
% butter
constraint 100*b + 150*c <= butter;
% cocoa
constraint 75*c <= cocoa;

% maximize our profit
solve maximize 400*b + 450*c;

output ["no. of banana cakes = \(b)\n",
        "no. of chocolate cakes = \(c)\n"];
```

Our new model is shown in Listing 1.3. We can run it using the command

```
$ minizinc cakes2.mzn pantry.dzn
```

where the data file `pantry.dzn` is defined in Listing 1.4. This gives the same result as `cakes.mzn`. The output from running the command

```
$ minizinc cakes2.mzn pantry2.dzn
```

with an alternate data set defined in Listing 1.5 is

```
no. of banana cakes = 3
no. of chocolate cakes = 8
----------
==========
```

If we remove the output statement from `cakes.mzn` then MiniZinc will use a default output. In this case the resulting output will be

```
b = 3;
c = 8;
----------
==========
```

Default Output

A MiniZinc model with no output will output a line for each decision variable with its value, unless it is assigned an expression on its declaration. Note how the output is in the form of a correct datafile.
Declarations can be annotated as `::output` or `::no_output` to override the default behaviour.

Listing 1.4: Example data file for `cakes2.mzn` (`pantry.dzn`)

```
1 flour = 4000;
2 banana = 6;
3 sugar = 2000;
4 butter = 500;
5 cocoa = 500;
```

Listing 1.5: Example data file for `cakes2.mzn` (`pantry2.dzn`)

```
1 flour = 8000;
2 banana = 11;
3 sugar = 3000;
4 butter = 1500;
5 cocoa = 800;
```

Small data files can be entered without directly creating a `.dzn` file, using the command line flag `-D` *string*, where *string* is the contents of the data file. For example the command

```
$ minizinc cakes2.mzn -D \
    "flour=4000;banana=6;sugar=2000;butter=500;cocoa=500;"
```

will give identical results to

```
$ minizinc cakes2.mzn pantry.dzn
```

Data files can only contain assignment statements for decision variables and parameters in the model(s) for which they are intended.

Defensive programming suggests that we should check that the values in the data file are reasonable. For our example it is sensible to check that the quantity of all ingredients is non-negative and generate a run-time error if this is not true. MiniZinc provides a built-in Boolean operator for checking parameter values. The form is `assert(B,S)`. The Boolean expression `B` is evaluated and if it is false execution aborts and the string expression `S` is evaluated and printed as an error message. To check and generate an appropriate error message if the amount of flour is negative we can simply add the line

```
constraint assert(flour >= 0,"Amount of flour must be non-negative");
```

to our model. Notice that the `assert` expression is a Boolean expression and so is regarded as a type of constraint. We can add similar lines to check that the quantity of the other ingredients is non-negative.

2.1.4 Real Number Solving

MiniZinc also supports "real number" constraint solving using floating point variables and constraints. Consider a problem of taking out a short loan for one year to be repaid in 4 quarterly instalments. A model for this is shown in Listing 1.6. It uses a simple interest calculation to calculate the balance after each quarter.

Listing 1.6: Model for determining relationships between a 1 year loan repaying every quarter (`loan.mzn`)

```
% variables
var float: R;        % quarterly repayment
var float: P;        % principal initially borrowed
var 0.0 .. 10.0: I;  % interest rate (per quarter)

% intermediate variables
var float: B1; % balance after one quarter
var float: B2; % balance after two quarters
var float: B3; % balance after three quarters
var float: B4; % balance owing at end

constraint B1 = P * (1.0 + I) - R;
constraint B2 = B1 * (1.0 + I) - R;
constraint B3 = B2 * (1.0 + I) - R;
constraint B4 = B3 * (1.0 + I) - R;

solve satisfy;

output [
 "Borrowing ", show_float(0, 2, P), " at ", show(I*100.0),
```

```
  "% interest, and repaying ", show_float(0, 2, R),
  "\nper quarter for 1 year leaves ", show_float(0, 2, B4), " owing\n"
];
```

Note that we declare a float variable `f` similar to an integer variable using the keyword `float` instead of `int`.

We can use the same model to answer a number of different questions. The first question is: if I borrow $1000 at 4% quarterly interest and repay $260 per quarter, how much do I end up owing? This question is encoded by the data file `loan1.dzn`.

Since we wish to use real number variables and constraint we need to use a solver that supports this type of problem. While Gecode (the default solver in the MiniZinc bundled binary distribution) does support floating point variables, a mixed integer linear programming (MIP) solver may be better suited to this particular type of problem. The MiniZinc distribution contains such a solver. We can invoke it by selecting `COIN-BC` from the solver configuration menu in the IDE, or on the command line using the command `minizinc --solver cbc`. Note: When solving this problem with the MiniZinc IDE, you will have to open the tab with the data file *loan1.dzn* and then click the Run button. If you just click Run in the original model file, the IDE won't know that any data is missing (since all inputs were declared `var`), and not pop up the dialog for selecting a data file.

```
$ minizinc --solver cbc loan.mzn loan1.dzn
```

The output is

```
Borrowing 1000.00 at 4.0% interest, and repaying 260.00
per quarter for 1 year leaves 65.78 owing
----------
```

The second question is if I want to borrow $1000 at 4% and owe nothing at the end, how much do I need to repay? This question is encoded by the data file `loan2.dzn`. The output from running the command

```
$ minizinc --solver cbc loan.mzn loan2.dzn
```

is

```
Borrowing 1000.00 at 4.0% interest, and repaying 275.49
per quarter for 1 year leaves 0.00 owing
----------
```

The third question is if I can repay $250 a quarter, how much can I borrow at 4% to end up owing nothing? This question is encoded by the data file `loan3.dzn`. The output from running the command

```
$ minizinc --solver cbc loan.mzn loan3.dzn
```

is

```
Borrowing 907.47 at 4.0% interest, and repaying 250.00
per quarter for 1 year leaves 0.00 owing
----------
```

Listing 1.7: Example data file for `loan.mzn` (`loan1.dzn`)

```
I = 0.04;
P = 1000.0;
R = 260.0;
```

Listing 1.8: Example data file for `loan.mzn` (`loan2.dzn`)

```
I = 0.04;
P = 1000.0;
B4 = 0.0;
```

Listing 1.9: Example data file for `loan.mzn` (`loan3.dzn`)

```
I = 0.04;
R = 250.0;
B4 = 0.0;
```

Float Arithmetic Operators

MiniZinc provides the standard floating point arithmetic operators: addition (`+`), subtraction (`-`), multiplication (`*`) and floating point division (`/`). It also provides `+` and `-` as unary operators.
MiniZinc can automatically coerce integers to floating point numbers. But to make the coercion explicit, the built-in function `int2float` can be used. Note that one consequence of the automatic coercion is that an expression `a / b` is always considered a floating point division. If you need an integer division, make sure to use the `div` operator!
MiniZinc provides in addition the following floating point functions: absolute value (`abs`), square root (`sqrt`), natural logarithm (`ln`), logarithm base 2 (`log2`), logarithm base 10 (`log10`), exponentiation of e (`exp`), sine (`sin`), cosine (`cos`), tangent (`tan`), arcsine (`asin`), arccosine (`acos`), arctangent (`atan`), hyperbolic sine (`sinh`), hyperbolic cosine (`cosh`), hyperbolic tangent (`tanh`), hyperbolic arcsine (`asinh`), hyperbolic arccosine (`acosh`), hyperbolic arctangent (`atanh`), and power (`pow`) which is the only binary function, the rest are unary.
The syntax for arithmetic literals is reasonably standard. Example float literals are `1.05`, `1.3e-5` and `1.3E+5`.

2.1.5 Basic structure of a model

We are now in a position to summarise the basic structure of a MiniZinc model. It consists of multiple *items* each of which has a semicolon `;` at its end. Items can occur in any order. For example, identifiers need not be declared before they are used.

There are 8 kinds of items.

- Include items allow the contents of another file to be inserted into the model. They have the form:

  ```
  include <filename>;
  ```

  where `<filename>` is a string literal. They allow large models to be split into smaller sub-models and also the inclusion of constraints defined in library files. We shall see an example in Listing 2.4.

- Variable declarations declare new variables. Such variables are global variables and can be referred to from anywhere in the model. Variables come in two kinds. Parameters which are assigned a fixed value in the model or in a data file and decision variables whose value is found only when the model is solved. We say that parameters are fixed and decision variables are unfixed. The variable can be optionally assigned a value as part of the declaration. The form is:

  ```
  <type inst expr>: <variable> [ = <expression> ] ;
  ```

  The `<type-inst expr>` gives the instantiation and type of the variable. These are one of the more complex aspects of MiniZinc. Instantiations are declared using `par` for parameters and `var` for decision variables. If there is no explicit instantiation declaration then the variable is a parameter. The type can be a base type, an integer or float range or an array or a set. The base types are `float`, `int`, `string`, `bool`, `ann` of which only `float`, `int` and `bool` can be used for decision variables. The base type `ann` is an annotation – we shall discuss annotations in *Search* (page 97). Integer range expressions can be used instead of the type `int`. Similarly float range expressions can be used instead of type `float`. These are typically used to give the domain of a decision variable but can also be used to restrict the range of a parameter. Another use of variable declarations is to define enumerated types, which we discuss in *Enumerated Types* (page 50).

- Assignment items assign a value to a variable. They have the form:

  ```
  <variable> = <expression>;
  ```

  Values can be assigned to decision variables in which case the assignment is equivalent to writing `constraint <variable> = <expression>`.

- Constraint items form the heart of the model. They have the form:

  ```
  constraint <Boolean expression>;
  ```

  We have already seen examples of simple constraints using arithmetic comparison and the built-in `assert` operator. In the next section we shall see examples of more complex constraints.

- Solve items specify exactly what kind of solution is being looked for. As we have seen they have one of three forms:

```
solve satisfy;
solve maximize <arithmetic expression>;
solve minimize <arithmetic expression>;
```

A model is required to have at most one solve item. If it's omitted, it is treated as `solve satisfy`.

- Output items are for nicely presenting the results of the model execution. They have the form:

```
output [ <string expression>, ..., <string expression> ];
```

If there is no output item, MiniZinc will by default print out the values of all the decision variables which are not optionally assigned a value in the format of assignment items. The `output` and `no_output` annotations can be used to control this on a per-variable basis.

- Enumerated type declarations. We discuss these in *Arrays and Sets* (page 35) and *Enumerated Types* (page 50).
- Predicate, function and test items are for defining new constraints, functions and Boolean tests. We discuss these in *Predicates and Functions* (page 65).
- The annotation item is used to define a new annotation. We discuss these in *Search* (page 97).

CHAPTER 2.2

More Complex Models

In the last section we introduced the basic structure of a MiniZinc model. In this section we introduce the array and set data structures, enumerated types and more complex constraints.

2.2.1 Arrays and Sets

Almost always we are interested in building models where the number of constraints and variables is dependent on the input data. In order to do so we will usually use arrays.

Consider a simple finite element model for modelling temperatures on a rectangular sheet of metal. We approximate the temperatures across the sheet by breaking the sheet into a finite number of elements in a two-dimensional matrix. A model is shown in Listing 2.1. It declares the width `w` and height `h` of the finite element model. The declaration

```
set of int: HEIGHT = 0..h;
set of int: CHEIGHT = 1..h-1;
set of int: WIDTH = 0..w;
set of int: CWIDTH = 1..w-1;
array[HEIGHT,WIDTH] of var float: t; % temperature at point (i,j)
```

declares four fixed sets of integers describing the dimensions of the finite element model: `HEIGHT` is the whole height of the model, while `CHEIGHT` is the centre of the height omitting the top and bottom, `WIDTH` is the whole width of the model, while `CWIDTH` is the centre of the width omitting the left and right sides. Finally a two dimensional array of float variables `t` with rows numbered 0 to h (`HEIGHT`) and columns 0 to w (`WIDTH`), to represent the temperatures at each point in the metal plate. We can access the element of the array in the ith row and j^{th} column using an expression `t[i,j]`.

Laplace's equation states that when the plate reaches a steady state the temperature at each internal point is the average of its orthogonal neighbours. The constraint

```
% Laplace equation: each internal temp. is average of its neighbours
constraint forall(i in CHEIGHT, j in CWIDTH)(
              4.0*t[i,j] = t[i-1,j] + t[i,j-1] + t[i+1,j] + t[i,j+1]);
```

ensures that each internal point (i, j) is the average of its four orthogonal neighbours. The constraints

```
% edge constraints
constraint forall(i in CHEIGHT)(t[i,0] = left);
constraint forall(i in CHEIGHT)(t[i,w] = right);
constraint forall(j in CWIDTH)(t[0,j] = top);
constraint forall(j in CWIDTH)(t[h,j] = bottom);
```

restrict the temperatures on each edge to be equal, and gives these temperatures names: `left`, `right`, `top` and `bottom`. While the constraints

```
% corner constraints
constraint t[0,0]=0.0;
constraint t[0,w]=0.0;
constraint t[h,0]=0.0;
constraint t[h,w]=0.0;
```

ensure that the corners (which are irrelevant) are set to 0.0. We can determine the temperatures in a plate broken into 5×5 elements with left, right and bottom temperature 0 and top temperature 100 with the model shown in Listing 2.1.

Listing 2.1: Finite element plate model for determining steady state temperatures (`laplace.mzn`).

```
int: w = 4;
int: h = 4;

% arraydec
set of int: HEIGHT = 0..h;
set of int: CHEIGHT = 1..h-1;
set of int: WIDTH = 0..w;
set of int: CWIDTH = 1..w-1;
array[HEIGHT,WIDTH] of var float: t; % temperature at point (i,j)
var float: left;   % left   edge temperature
var float: right;  % right  edge temperature
var float: top;    % top    edge temperature
var float: bottom; % bottom edge temperature

% equation
% Laplace equation: each internal temp. is average of its neighbours
constraint forall(i in CHEIGHT, j in CWIDTH)(
              4.0*t[i,j] = t[i-1,j] + t[i,j-1] + t[i+1,j] + t[i,j+1]);
% sides
% edge constraints
```

```
constraint forall(i in CHEIGHT)(t[i,0] = left);
constraint forall(i in CHEIGHT)(t[i,w] = right);
constraint forall(j in CWIDTH)(t[0,j] = top);
constraint forall(j in CWIDTH)(t[h,j] = bottom);
% corners
% corner constraints
constraint t[0,0]=0.0;
constraint t[0,w]=0.0;
constraint t[h,0]=0.0;
constraint t[h,w]=0.0;
left = 0.0;
right = 0.0;
top = 100.0;
bottom = 0.0;

solve satisfy;

output [ show_float(6, 2, t[i,j]) ++
         if j == w then "\n" else " " endif |
         i in HEIGHT, j in WIDTH
];
```

Running the command

```
$ minizinc --solver cbc laplace.mzn
```

gives the output

```
 -0.00 100.00 100.00 100.00  -0.00
 -0.00  42.86  52.68  42.86  -0.00
 -0.00  18.75  25.00  18.75  -0.00
 -0.00   7.14   9.82   7.14  -0.00
 -0.00  -0.00  -0.00  -0.00  -0.00
----------
```

Sets

Set variables are declared with a declaration of the form

```
set of <type-inst> : <var-name> ;
```

where sets of integers, enums (see later), floats or Booleans are allowed. The only type allowed for decision variable sets are variable sets of integers or enums. Set literals are of the form

```
{ <expr-1>, ..., <expr-n> }
```

or are range expressions over either integers, enums or floats of the form

```
<expr-1> .. <expr-2>
```

The standard set operations are provided: element membership (`in`), (non-strict) subset relationship (`subset`), (non-strict) superset relationship (`superset`), union (`union`), intersection (`intersect`), set difference (`diff`), symmetric set difference (`symdiff`) and the number of elements in the set (`card`).
As we have seen set variables and set literals (including ranges) can be used as an implicit type in variable declarations in which case the variable has the type of the elements in the set and the variable is implicitly constrained to be a member of the set.
A set declaration may also include a *cardinality expression* between the `set` keyword and `of`:

```
<var-par> set "(" <expr> ")" of <type-inst> : <var-name> ;
```

This is a shorthand for declaring the set as usual together with an implicit constraint on its cardinality. When the cardinality expression is an integer, the cardinality is constrained to be equal to it; when it is a set of integers, the cardinality is constrained to lie in that set. For example, `var set(2) of 1..5: a;` declares a variable set whose cardinality is exactly two, and is equivalent to `var set of 1..5: a; constraint card(a) = 2;`. The form `var set(1..3) of 1..10: b;` declares a variable set whose cardinality is between one and three. See spec-set-cardinality for the precise rules.

Our cake baking problem is an example of a very simple kind of production planning problem. In this kind of problem we wish to determine how much of each kind of product to make to maximise the profit where manufacturing a product consumes varying amounts of some fixed resources. We can generalise the MiniZinc model in Listing 1.3 to handle this kind of problem with a model that is generic in the kinds of resources and products. The model is shown in Listing 2.2 and a sample data file (for the cake baking example) is shown in Listing 2.3.

Listing 2.2: Model for simple production planning (prod-planning.mzn).

```
% Products to be produced
enum Products;
% profit per unit for each product
array[Products] of int: profit;
% Resources to be used
enum Resources;
% amount of each resource available
array[Resources] of int: capacity;

% units of each resource required to produce 1 unit of product
array[Products, Resources] of int: consumption;
constraint assert(forall (r in Resources, p in Products)
           (consumption[p,r] >= 0), "Error: negative consumption");

% bound on number of Products
int: mproducts = max (p in Products)
                     (min (r in Resources where consumption[p,r] > 0)
                          (capacity[r] div consumption[p,r]));

% Variables: how much should we make of each product
array[Products] of var 0..mproducts: produce;
array[Resources] of var 0..max(capacity): used;

% Production cannot use more than the available Resources:
constraint forall (r in Resources) (
      used[r] = sum (p in Products)(consumption[p, r] * produce[p])
);
constraint forall (r in Resources) (
      used[r] <= capacity[r]
);

% Maximize profit
solve maximize sum (p in Products) (profit[p]*produce[p]);

output [ "\(p) = \(produce[p]);\n" | p in Products ] ++
       [ "\(r) = \(used[r]);\n" | r in Resources ];
```

Listing 2.3: Example data file for the simple production planning problem (prod-planning-data.dzn).

```
% Data file for simple production planning model
Products = { BananaCake, ChocolateCake };
profit = [400, 450]; % in cents

Resources = { Flour, Banana, Sugar, Butter, Cocoa };
capacity = [4000, 6, 2000, 500, 500];
```

```
consumption= [| 250, 2, 75, 100, 0,
              | 200, 0, 150, 150, 75 |];
```

The new feature in this model is the use of enumerated types. These allow us to treat the choice of resources and products as parameters to the model. The first item in the model

```
enum Products;
```

declares `Products` as an *unknown* set of products.

Enumerated Types

Enumerated types, which we shall refer to as `enums`, are declared with a declaration of the form

```
enum <var-name> ;
```

An enumerated type is defined by an assignment of the form

```
enum <var-name> = { <var-name-1>, ..., <var-name-n> } ;
```

where `<var-name-1>`, ..., `<var-name-n>` are the elements of the enumerated type, with name `<var-name>`. Each of the elements of the enumerated type is also effectively declared by this definition as a new constant of that type. The declaration and definition can be combined into one line as usual.

The second item declares an array of integers:

```
array[Products] of int: profit;
```

The index set of the array `profit` is `Products`. This means that only elements of the set `Products` can be used to index the array.

The elements of an enumerated type of n elements act very similar to the integers $1 \ldots n$. They can be compared, they are ordered, by the order they appear in the enumerated type definition, they can be iterated over, they can appear as indices of arrays, in fact they can appear anywhere an integer can appear.

In the example data file we have initialized the array using a list of integers

```
Products = { BananaCake, ChocolateCake };
profit = [400,450];
```

meaning the profit for a banana cake is 400, while for a chocolate cake it is 450. Internally `BananaCake` will be treated like the integer 1, while `ChocolateCake` will be treated like the integer 2. The expression `[400,500]` represents a literal one-dimensional array. In MiniZinc, the index set of literal arrays always starts at 1 (this is similar to other mathematically inspired languages such as MATLAB, Julia or Mathematica). While MiniZinc

does not provide an explicit list type, one-dimensional arrays with an index set `1..n` behave like lists, and we will sometimes refer to them as lists.

In a similar fashion, in the next two items we declare a set of resources `Resources`, and an array `capacity` which gives the amount of each resource that is available.

More interestingly, the item

```
array[Products, Resources] of int: consumption;
```

declares a 2-D array `consumption`. The value of `consumption[p,r]` is the amount of resource `r` required to produce one unit of product `p`. Note that the first index is the row and the second is the column.

The data file contains an example initialization of a 2-D array:

```
consumption= [| 250, 2, 75,  100, 0,
              | 200, 0, 150, 150, 75 |];
```

Notice how the delimiter `|` is used to separate rows. As for one-dimensional array literals, indexing of two-dimensional array literals also starts at 1.

Arrays

Thus, MiniZinc provides one- and multi-dimensional arrays which are declared using the type:

```
array [ <index-set-1>, ..., <index-set-n> ] of <type-inst>
```

MiniZinc requires that the array declaration contains the index set of each dimension and that the index set is either an integer range, a set variable initialised to an integer range, or an enumeration type. Arrays can contain any of the base types: integers, enums, Booleans, floats or strings. These can be fixed or unfixed except for strings which can only be parameters. Arrays can also contain sets but they cannot contain arrays.
One-dimensional array literals are of form

```
[ <expr-1>, ..., <expr-n> ]
```

and their index sets are `1..n`, while two-dimensional array literals are of form

```
[| <expr-1-1>, ..., <expr-1-n> |
   ...                         |
   <expr-m-1>, ..., <expr-m-n> |]
```

where the array has `m` rows and `n` columns, with index sets `1..m` for the first and `1..n` for the second dimension.
The family of built-in functions `array1d`, `array2d`, etc, can be used to initialise an array of any dimension from a list (or more exactly a one-dimensional array). The call:

```
array<n>d(<index-set-1>, ..., <index-set-n>, <list>)
```

returns an `n` dimensional array with index sets given by the first `n` arguments and the last argument contains the elements of the array. For instance, `array2d(1..3, 1..2, [1, 2, 3, 4, 5, 6])` is equivalent to `[|1, 2 |3, 4 |5, 6|]`.
Array elements are accessed using bracket syntax: `a[i,j]` gives the element at row index i and column index j.
The concatenation operator `++` can be used to concatenate two one-dimensional arrays together. The result is a list, i.e. a one-dimensional array whose elements are indexed from 1. For instance `[4000, 6] ++ [2000, 500, 500]` evaluates to `[4000, 6, 2000, 500, 500]`. The built-in function `length` returns the length of a one-dimensional array.
An index list entry of an array type can also take the form `<ident> in <index-set>`, which binds the identifier to the element's index and makes it available in the element type. This is useful when the domain of each element depends on its position, e.g. to give each decision variable a domain that depends on per-element data:

```
enum Task;
array[Task] of int: deadline;
array[t in Task] of var 0..deadline[t]: end;
```

Here `end[t]` is declared with its own upper bound, taken from the corresponding entry of `deadline`. Multiple dimensions may each introduce their own binder. See *Index-dependent array declarations* (page 286) for the full rules.

The next item in the model defines the parameter `mproducts`. This is set to an upper-bound on the number of products of any type that can be produced. This is quite a complex example of nested array comprehensions and aggregation operators. We shall introduce these before we try to understand this item and the rest of the model.

First, MiniZinc provides list comprehensions similar to those provided in many functional programming languages, or Python. For example, the list comprehension `[i + j | i, j in 1..3 where j < i]` evaluates to `[2 + 1, 3 + 1, 3 + 2]` which is `[3, 4, 5]`. Of course `[3, 4, 5]` is simply an array with index set `1..3`.

MiniZinc also provides set comprehensions which have a similar syntax: for instance, `{i + j | i, j in 1..3 where j < i}` evaluates to the set `{3, 4, 5}`.

List and Set Comprehensions

The generic form of a list comprehension is

```
[ <expr> | <generator-exp> ]
```

The expression `<expr>` specifies how to construct elements in the output list from the elements generated by `<generator-exp>`. The generator `<generator-exp>` consists of a comma separated sequence of generator expressions optionally followed by a Boolean expression. The two forms are

```
<generator>
<generator> where <bool-exp>
```

The optional `<bool-exp>` in the second form acts as a filter on the generator expression: only elements satisfying the Boolean expression are used to construct elements in the output list. A generator `<generator>` has the form

```
<identifier>, ..., <identifier> in <array-exp>
```

Each identifier is an *iterator* which takes the values of the array expression in turn, with the last identifier varying most rapidly.
The generators of a list comprehension and `<bool-exp>` usually do not involve decision variables. If they do involve decision variables then the list produced is a list of `var opt <T>` where `<T>` is the type of the `<expr>`. See the discussion of option types in *Option Types* (page 85) for more details.
Set comprehensions are almost identical to list comprehensions: the only difference is the use of `{` and `}` to enclose the expression rather than `[` and `]`. The elements generated by a set comprehension must be fixed, i.e. free of decision variables. Similarly the generators and optional `<bool-exp>` for set comprehensions must be fixed.

Second, MiniZinc provides a number of built-in functions that take a one-dimensional array and aggregate the elements. Probably the most useful of these is `forall`. This takes an array of Boolean expressions (that is, constraints) and returns a single Boolean expression which is the logical conjunction of the Boolean expressions in the array.

For example, consider the expression

```
forall( [a[i] != a[j] | i,j in 1..3 where i < j])
```

where `a` is an arithmetic array with index set `1..3`. This constrains the elements in `a` to be pairwise different. The list comprehension evaluates to `[ a[1] != a[2], a[1] != a[3], a[2] != a[3] ]` and so the `forall` function returns the logical conjunction `a[1] != a[2] /\ a[1] != a[3] /\ a[2] != a[3]`.

Aggregation functions

The *aggregation functions* for arithmetic arrays are: `sum` which adds the elements, `product` which multiplies them together, and `min` and `max` which respectively return the least and greatest element in the array. When applied to an empty array, `min` and `max` give a run-time error, `sum` returns 0 and `product` returns 1.
MiniZinc provides four aggregation functions for arrays containing Boolean expressions. As we have seen, the first of these, `forall`, returns a single constraint which is the logical conjunction of the constraints. The second function, `exists`, returns the logical disjunction of the constraints. Thus, `forall` enforces that all constraints in the array hold, while `exists` ensures that at least one of the constraints holds. The third function, `xorall`, ensures that an odd number of constraints hold. The fourth function, `iffall`, ensures that an even number of constraints holds.

The third, and final, piece in the puzzle is that MiniZinc allows a special syntax for aggregation functions when used with an array comprehension. Instead of writing

```
forall( [a[i] != a[j] | i,j in 1..3 where i < j])
```

the modeller can instead write the more mathematical looking

```
forall (i,j in 1..3 where i < j) (a[i] != a[j])
```

The two expressions are completely equivalent: the modeller is free to use whichever they feel looks most natural.

Generator call expressions

A *generator call expression* has form

```
<agg-func> ( <generator-exp> ) ( <exp> )
```

The round brackets around the generator expression `<generator-exp>` and the constructor expression `<exp>` are not optional: they must be there. This is equivalent to writing

```
<agg-func> ( [ <exp> | <generator-exp> ] )
```

The aggregation function `<agg-func>` is any MiniZinc function expecting a single array as argument.

We are now in a position to understand the rest of the simple production planning model

shown in Listing 2.2. For the moment ignore the item defining `mproducts`. The item afterwards:

```
array[Products] of var 0..mproducts: produce;
```

defines a one-dimensional array `produce` of decision variables. The value of `produce[p]` will be set to the amount of product `p` in the optimal solution. The next item

```
array[Resources] of var 0..max(capacity): used;
```

defines a set of auxiliary variables that record how much of each resource is used. The next two constraints

```
constraint forall (r in Resources)
           (used[r] = sum (p in Products) (consumption[p, r] * produce[p]));
constraint forall (r in Resources)(used[r] <= capacity[r] );
```

compute in `used[r]` the total consumption of the resource `r` and ensure it is less than the available amount. Finally, the item

```
solve maximize sum (p in Products) (profit[p]*produce[p]);
```

indicates that this is a maximisation problem and that the objective to be maximised is the total profit.

We now return to the definition of `mproducts`. For each product `p` the expression

```
(min (r in Resources where consumption[p,r] > 0)
     (capacity[r] div consumption[p,r]))
```

determines the maximum amount of `p` that can be produced taking into account the amount of each resource `r` and how much of `r` is required to produce the product. Notice the use of the filter `where consumption[p,r] > 0` to ensure that only resources required to make the product are considered so as to avoid a division by zero error. Thus, the complete expression

```
int: mproducts = max (p in Products)
                     (min (r in Resources where consumption[p,r] > 0)
                          (capacity[r] div consumption[p,r]));
```

computes the maximum amount of *any* product that can be produced, and so this can be used as an upper bound on the domain of the decision variables in `produce`.

Finally notice the output item is more complex, and uses list comprehensions to create an understandable output. Running

```
$ minizinc --solver gecode prod-planning.mzn prod-planning-data.dzn
```

results in the output

```
BananaCake = 2;
ChocolateCake = 2;
Flour = 900;
Banana = 4;
Sugar = 450;
Butter = 500;
Cocoa = 150;
----------
==========
```

2.2.2 Global Constraints

MiniZinc includes a library of global constraints which can also be used to define models. An example is the `alldifferent` constraint which requires all the variables appearing in its argument to be pairwise different.

Listing 2.4: Model for the cryptarithmetic problem SEND+MORE=MONEY (`send-more-money.mzn`)

```
include "alldifferent.mzn";

var 1..9: S;
var 0..9: E;
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;

constraint           1000 * S + 100 * E + 10 * N + D
                   + 1000 * M + 100 * O + 10 * R + E
       = 10000 * M + 1000 * O + 100 * N + 10 * E + Y;

constraint alldifferent([S,E,N,D,M,O,R,Y]);

solve satisfy;

output ["   \(S)\(E)\(N)\(D)\n",
        "+  \(M)\(O)\(R)\(E)\n",
        "= \(M)\(O)\(N)\(E)\(Y)\n"];
```

The SEND+MORE=MONEY problem requires assigning a different digit to each letter so that the arithmetic constraint holds. The model shown in Listing 2.4 uses the constraint expression `alldifferent([S,E,N,D,M,O,R,Y])` to ensure that each letter takes a different digit value. The global constraint is made available in the model using include item

```
include "alldifferent.mzn";
```

which makes the global constraint `alldifferent` usable by the model. One could replace this line by

```
include "globals.mzn";
```

which includes all globals.

A list of all the global constraints defined for MiniZinc is included in the release documentation. See *Global Constraints* (page 65) for a description of some important global constraints.

2.2.3 Conditional Expressions

MiniZinc provides a conditional *if-then-else-endif* expression. An example of its use is

```
int: r = if y != 0 then x div y else 0 endif;
```

which sets `r` to `x` divided by `y` unless `y` is zero in which case it sets it to zero.

Conditional expressions

The form of a conditional expression is

```
if <bool-exp> then <exp-1> else <exp-2> endif
```

It is an actual expression rather than a control flow statement and so can be used in other expressions. It evaluates to `<exp-1>` if the Boolean expression `<bool-exp>` is true and `<exp-2>` otherwise. The type of the conditional expression is that of `<exp-1>` and `<exp-2>` which must have the same type.
If the `<bool-exp>` contains decision variables, then the type-inst of the expression is `var <T>` where `<T>` is the type of `<exp-1>` and `<exp-2>` even if both expressions are fixed.

Listing 2.5: Model for generalized Sudoku problem (`sudoku.mzn`)

```
include "alldifferent.mzn";

int: S;
int: N = S * S;
int: digs = ceil(log(10.0,int2float(N))); % digits for output

set of int: PuzzleRange = 1..N;
set of int: SubSquareRange = 1..S;

array[1..N,1..N] of 0..N: start; %% initial board 0 = empty
array[1..N,1..N] of var PuzzleRange: puzzle;

% fill initial board
constraint forall(i,j in PuzzleRange)(
```

```
    if start[i,j] > 0 then puzzle[i,j] = start[i,j] else true endif );

% All different in rows
constraint forall (i in PuzzleRange) (
                   alldifferent( [ puzzle[i,j] | j in PuzzleRange ]) );

% All different in columns.
constraint forall (j in PuzzleRange) (
                   alldifferent( [ puzzle[i,j] | i in PuzzleRange ]) );

% All different in sub-squares:
constraint
        forall (a, o in SubSquareRange)(
                alldifferent( [ puzzle[(a-1) *S + a1, (o-1)*S + o1] |
                                    a1, o1 in SubSquareRange ] ) );

solve satisfy;

output  [ show_int(digs,puzzle[i,j]) ++ " " ++
         if j mod S == 0 then " " else "" endif ++
         if j == N then
             if i != N then
                if i mod S == 0 then "\n\n" else "\n" endif
         else "" endif else "" endif
         | i,j in PuzzleRange ] ++ ["\n"];
```

Listing 2.6: Example data file for generalised Sudoku problem (sudoku.dzn)

```
S=3;
start=[|
0, 0, 0, 0, 0, 0, 0, 0, 0|
0, 6, 8, 4, 0, 1, 0, 7, 0|
0, 0, 0, 0, 8, 5, 0, 3, 0|
0, 2, 6, 8, 0, 9, 0, 4, 0|
0, 0, 7, 0, 0, 0, 9, 0, 0|
0, 5, 0, 1, 0, 6, 3, 2, 0|
0, 4, 0, 6, 1, 0, 0, 0, 0|
0, 3, 0, 2, 0, 7, 6, 9, 0|
0, 0, 0, 0, 0, 0, 0, 0, 0|];
```

Conditional expressions are very useful in building complex models, or complex output. Consider the model of Sudoku problems shown in Listing 2.5. The initial board positions are given by the `start` parameter where 0 represents an empty board position. This is converted to constraints on the decision variables `puzzle` using the conditional expression

```
constraint forall(i,j in PuzzleRange)(
     if start[i,j] > 0 then puzzle[i,j] = start[i,j] else true endif );
```

Conditional expressions are also very useful for defining complex output. In the Sudoku

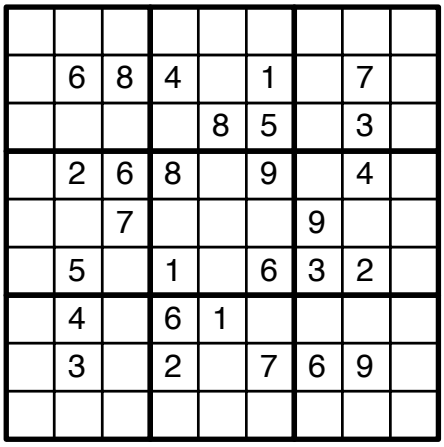


Fig. 2.1: The problem represented by data file `sudoku.dzn`

model of Listing 2.5 the expression

```
if j mod S == 0 then " " else "" endif
```

inserts an extra space between groups of size `S`. The output expression also uses conditional expressions to add blank lines after each `S` lines. The resulting output is highly readable.

The remaining constraints ensure that the numbers appearing in each row and column and $S \times S$ subsquare are all different.

One can use MiniZinc to search for all solutions to a satisfaction problem (`solve satisfy`). In the MiniZinc IDE, this can be achieved by unticking the "Stop after this many solutions" option in the "User-defined behavior" part of the solver configuration pane (see Section 3.2.2.2). On the command line, one can use the flag `-a` or `--all-solutions`. Running

```
$ minizinc --solver gecode --all-solutions sudoku.mzn sudoku.dzn
```

results in

```
 5 9 3  7 6 2  8 1 4
 2 6 8  4 3 1  5 7 9
 7 1 4  9 8 5  2 3 6

 3 2 6  8 5 9  1 4 7
 1 8 7  3 2 4  9 6 5
 4 5 9  1 7 6  3 2 8

 9 4 2  6 1 8  7 5 3
 8 3 5  2 4 7  6 9 1
 6 7 1  5 9 3  4 8 2
----------
```

```
==========
```

The line `==========` is output when the system has output all possible solutions, here verifying that there is exactly one.

2.2.4 Enumerated Types

Enumerated types allows us to build models that depend on a set of objects which are part of the data, or are named in the model, and hence make models easier to understand and debug. We have introduced enumerated types or enums briefly, in this subsection we will explore how we can use them more fully, and show some of the built in functions for dealing with enumerated types.

Let's revisit the problem of coloring the graph of Australia from *Basic Modelling in MiniZinc* (page 21).

Listing 2.7: Model for coloring Australia using enumerated types (`aust-enum.mzn`).

```
enum Color;
var Color: wa;
var Color: nt;
var Color: sa;
var Color: q;
var Color: nsw;
var Color: v;
var Color: t;
constraint wa != nt /\ wa != sa /\ nt != sa /\ nt != q /\ sa != q;
constraint sa != nsw /\ sa != v /\ q != nsw /\ nsw != v;
solve satisfy;
```

The model shown in Listing 2.7 declares an enumerated type `Color` which must be defined in the data file. Each of the state variables is declared to take a value from this enumerated type. Running this program using

```
$ minizinc --solver gecode -D"Color = { red, yellow, blue };" aust-enum.mzn
```

might result in output

```
wa = blue;
nt = yellow;
sa = red;
q = blue;
nsw = yellow;
v = blue;
t = red;
```

Enumerated Type Variable Declarations

An enumerated type parameter is declared as either:

```
<enum-name> : <var-name>
<l>..<u> : <var-name>
```

where <enum-name> is the name of an enumerated type, and <l> and <u> are fixed enumerated type expressions of the same enumerated type.
An enumerated type decision variable is declared as either:

```
var <enum-name> : <var-name>
var <l>..<u> : <var-name>
```

where <enum-name> is the name of an enumerated type, and <l> and <u> are fixed enumerated type expressions of the same enumerated type.

A key behaviour of enumerated types is that they are automatically coerced to integers when they are used in a position expecting an integer. For example, this allows us to use global constraints defined on integers, such as

```
global_cardinality([wa,nt,sa,q,nsw,v,t],
                   [red,yellow,blue],[2,2,2],[2,2,3]);
```

This requires at least two states to be colored each color and three to be colored blue.

Enumerated Type Operations

There are a number of built in operations on enumerated types:

- `enum_next(X,x)`: returns the next value after `x` in the enumerated type `X`. This is a partial function, if `x` is the last value in the enumerated type `X` then the function returns $\bot$ causing the Boolean expression containing the expression to evaluate to `false`.
- `enum_prev(X,x)`: returns the previous value before `x` in the enumerated type `X`. Similarly `enum_prev` is a partial function.
- `enum2int(x)`: coerces the enumerated type value `x` into an integer.
- `to_enum(X,i)`: maps an integer expression `i` to an enumerated type value in type `X` or evaluates to $\bot$ if `i` is less than or equal to 0 or greater than the number of elements in `X`.

Note also that a number of standard functions are applicable to enumerated types:

- `card(X)`: returns the cardinality of an enumerated type `X`.
- `min(X)`: returns the minimum element of of an enumerated type `X`.
- `max(X)`: returns the maximum element of of an enumerated type `X`.

It is often useful to define an enumerated type by *extending* another enumerated type. A good example for this is adding a "don't care" value to a type. For instance, consider a typical assignment problem, where we have to assign chores to flatmates. The names of the flatmates are given as an enumerated type `Flatmates`, and the chores as an enumerated type `Chores`.

In the case where not every flatmate needs to be assigned a job, and not every chore needs to be assigned to a flatmate, we need to introduce a value that signals "no assignment". We can then use the `alldifferent_except` constraint to constrain the assignment.

Listing 2.8: Model for an assignment problem using extended enumerated types (`assignment-extend-enum.mzn`).

```
include "alldifferent_except.mzn";

enum Flatmates = { Anne, Bert, Ceci, Dave };
enum Chores = { Cooking, Vacuuming, Bathroom, Kitchen, Rubbish };

enum ChoreOrNothing = C(Chores) ++ { Nothing };

array[Flatmates] of var ChoreOrNothing: assignment;

constraint alldifferent_except(assignment, {Nothing});

output [ show(f)++":\t"++
         if fix(assignment[f])=Nothing then "Nothing\n"
         else show(C^-1(assignment[f]))++"\n" endif
       | f in Flatmates];
```

The model is shown in Listing 2.8. It introduces a new enumerated type `ChoreOrNothing`, which is a combination of the `Chores` type and a new constructor `Nothing`. The `Chores` type is *injected* into the new type via a *constructor function*. In this case, we chose the name `C` for the constructor function, but you can use any valid identifier you like. The constructor function can then be used to coerce a value of type `Chores` into a value of the new type `ChoreOrNothing`. For example, we could write a constraint such as

```
constraint assignment[Bert] = C(Vacuuming);
```

This is required because the type `ChoreOrNothing` is considered different from the type `Chores` by the compiler, in order to avoid any ambiguities or accidental type conversions. The compiler also generates inverses of all constructor functions, to map back from the extended types into the base types. This can be seen in the output item, where `C^-1` is used to map from `ChoreOrNothing` back to `Chores`. The inverse function can be written as ASCII using the `^-1` notation, or using the unicode symbols ⁻¹, as in `C ⁻¹`.

In the example above, we could have used option types, instead of adding a new value that acts essentially like the absent value <>. However, extended enum types have some advantages in this scenario. For example, we can extend a type by multiple extra values (say, different reasons for not being assigned any chore). We can also combine multiple enumerated types into one. Consider the scenario in Listing 2.9 where we can assign chores to flatmates and/or robots. Flatmates have preferences for certain chores, while robots have integer benefits. The model also shows how to test if a value of the extended enum in fact belongs to the underlying enum: In the output item, the test `w in F(Flatmates)` uses the `F` constructor to translate the set `Flatmates` into the `Worker` type, which enables the use of the `in` operator.

Listing 2.9: Model for an assignment problem using combined enumerated types (assignment-combine-enums.mzn).

```
include "alldifferent_except.mzn";

enum Flatmates = { Anne, Bert, Ceci, Dave };
enum Robots = { R2D2, C3PO, Marvin };

% Higher number = higher preference
array[Flatmates,Chores] of int: preference =
  [| 1, 2, 3, 4, 5
   | 2, 1, 3, 4, 5
   | 3, 5, 4, 1, 2
   | 1, 5, 4, 2, 3 |];

array[Robots,Chores] of int: benefit =
  [| 20, 100, 20, 100, 30
   | 10, 120, 40, 40,  60
   | 50, 500, 30, 10,  70 |];

enum Chores = { Cooking, Vacuuming, Bathroom, Kitchen, Rubbish };

enum Workers = F(Flatmates) ++ R(Robots);

enum ChoreOrNothing = C(Chores) ++ { Nothing };

% Create preference and benefit arrays with added values for Nothing

array[Flatmates,ChoreOrNothing] of int: prefWithNothing =
  array2d(Flatmates,ChoreOrNothing,
          [ if c=Nothing then 0 else preference[f,C^-1(c)] endif
          | f in Flatmates, c in ChoreOrNothing]);

array[Robots,ChoreOrNothing] of int: benefitWithNothing =
  array2d(Robots,ChoreOrNothing,
          [ if c=Nothing then 0 else benefit[r,C^-1(c)] endif
          | r in Robots, c in ChoreOrNothing]);

array[Workers] of var ChoreOrNothing: assignment;

constraint alldifferent_except(assignment, {Nothing});

var int: overall_preferences =
  sum(f in Flatmates)(prefWithNothing[f, assignment[F(f)]]);

var int: overall_benefit =
  sum(r in Robots)(benefitWithNothing[r, assignment[R(r)]]);

solve maximize overall_benefit + overall_preferences;
```

```
output [ if w in F(Flatmates) then show(F^-1(w)) else show(R^-1(w)) endif++
↪":\t"++
         if fix(assignment[w])=Nothing then "Nothing\n"
         else show(C^-1(assignment[w]))++"\n" endif
       | w in Workers];
```

2.2.5 Complex Constraints

Constraints are the core of the MiniZinc model. We have seen simple relational expressions but constraints can be considerably more powerful than this. A constraint is allowed to be any Boolean expression. Imagine a scheduling problem in which we have two tasks that cannot overlap in time. If `s1` and `s2` are the corresponding start times and `d1` and `d2` are the corresponding durations we can express this as:

```
constraint s1 + d1 <= s2  \/ s2 + d2 <= s1;
```

which ensures that the tasks do not overlap.

Booleans

Boolean expressions in MiniZinc can be written using a standard mathematical syntax. The Boolean literals are `true` and `false` and the Boolean operators are conjunction, i.e. and (`/\`), disjunction, i.e. or (`\/`), only-if (`<-`), implies (`->`), if-and-only-if (`<->`) and negation (`not`). Booleans can be automatically coerced to integers, but to make this coercion explicit the built-in function `bool2int` can be used: it returns 1 if its argument is true and 0 otherwise.

Listing 2.10: Model for job-shop scheduling problems (`jobshop.mzn`).

```
enum JOB;
enum TASK;
TASK: last = max(TASK);
array [JOB,TASK] of int: d;                    % task durations
int: total = sum(i in JOB, j in TASK)(d[i,j]);% total duration
int: digs = ceil(log(10.0,int2float(total))); % digits for output
array [JOB,TASK] of var 0..total: s;           % start times
var 0..total: end;                             % total end time


constraint %% ensure the tasks occur in sequence
    forall(i in JOB) (
        forall(j in TASK where j < last)
            (s[i,j] + d[i,j] <= s[i,enum_next(j)]) /\
        s[i,last] + d[i,last] <= end
    );

constraint %% ensure no overlap of tasks
```

```
    forall(j in TASK) (
        forall(i,k in JOB where i < k) (
            s[i,j] + d[i,j] <= s[k,j] \/
            s[k,j] + d[k,j] <= s[i,j]
        )
    );

solve minimize end;

output ["end = \(end)\n"] ++
       [ show_int(digs,s[i,j]) ++ " " ++
         if j == last then "\n" else "" endif |
         i in JOB, j in TASK ];
```

Listing 2.11: Data for job-shop scheduling problems (`jdata.dzn`).

```
JOB = Job(1..5);
TASK = Task(1..5);
d = [| 1, 4, 5, 3, 6
     | 3, 2, 7, 1, 2
     | 4, 4, 4, 4, 4
     | 1, 1, 1, 6, 8
     | 7, 3, 2, 2, 1 |];
```

The job shop scheduling model given in Listing 2.10 gives a realistic example of the use of this disjunctive modelling capability. In job shop scheduling we have a set of jobs, each consisting of a sequence of tasks on separate machines: so task `[i,j]` is the task in the ith job performed on the jth machine. Each sequence of tasks must be completed in order, and no two tasks on the same machine can overlap in time. Even for small instances of this problem it can be quite challenging to find optimal solutions.

The command

```
$ minizinc --solver gecode --intermediate jobshop.mzn jdata.dzn
```

solves a small job shop scheduling problem, and illustrates the behaviour of `--intermediate` for optimisation problems (note that when running this in the MiniZinc IDE, `--intermediate` is the default behaviour for optimisation problems). Here the solver outputs increasingly better solutions as it finds them. This flag has no effect for satisfaction problems. The output from this command is:

```
end = 39
 5  9 13 22 30
 6 13 18 25 36
 0  4  8 12 16
 4  8 12 16 22
 9 16 25 27 38
----------
end = 37
```

```
 4  8 12 17 20
 5 13 18 26 34
 0  4  8 12 16
 8 12 17 20 26
 9 16 25 27 36
----------
end = 34
 0  1  5 10 13
 6 10 15 23 31
 2  6 11 19 27
 1  5 10 13 19
 9 16 22 24 33
----------
end = 30
 5  9 13 18 21
 6 13 18 25 27
 1  5  9 13 17
 0  1  2  3  9
 9 16 25 27 29
----------
==========
```

indicating an optimal solution with end time 30 is finally found, and proved optimal. We can generate all *optimal solutions* by adding a constraint that `end = 30` and changing the solve item to `solve satisfy` and then executing

```
$ minizinc --solver gecode --all-solutions jobshop.mzn jdata.dzn
```

For this problem there are 3,444,375 optimal solutions. In the MiniZinc IDE, you can select "User-defined behavior" in the configuration editor and uncheck the option "Stop after this many solutions" in order to display all solutions of this satisfaction problem.

2.2.6 Array Access Constraints

Another powerful modelling feature in MiniZinc is that decision variables can be used for array access. As an example, consider the (old-fashioned) *stable marriage problem*. We have `n` (straight) women and `n` (straight) men. Each man has a ranked list of women and vice versa. We want to find a husband/wife for each women/man so that all marriages are *stable* in the sense that:

- whenever `m` prefers another women `o` to his wife `w`, `o` prefers her husband to `m`, and
- whenever `w` prefers another man `o` to her husband `m`, `o` prefers his wife to `w`.

This can be elegantly modelled in MiniZinc. The model and sample data is shown in Listing 2.12 and Listing 2.13.

Listing 2.12: Model for the stable marriage problem (`stable-marriage.mzn`).

```
int: n;

enum Men = _(1..n);
enum Women = _(1..n);

array[Women, Men] of int: rankWomen;
array[Men, Women] of int: rankMen;

array[Men] of var Women: wife;
array[Women] of var Men: husband;

% assignment
constraint forall (m in Men) (husband[wife[m]]=m);
constraint forall (w in Women) (wife[husband[w]]=w);
% ranking
constraint forall (m in Men, o in Women) (
     rankMen[m,o] < rankMen[m,wife[m]] ->
         rankWomen[o,husband[o]] < rankWomen[o,m] );

constraint forall (w in Women, o in Men) (
     rankWomen[w,o] < rankWomen[w,husband[w]] ->
         rankMen[o,wife[o]] < rankMen[o,w] );
solve satisfy;

output ["wives= \(wife)\nhusbands= \(husband)\n"];
```

Listing 2.13: Example data for the stable marriage problem (`stable-marriage.dzn`).

```
n = 5;
rankWomen =
 [| 1, 2, 4, 3, 5,
  | 3, 5, 1, 2, 4,
  | 5, 4, 2, 1, 3,
  | 1, 3, 5, 4, 2,
  | 4, 2, 3, 5, 1 |];

rankMen =
 [| 5, 1, 2, 4, 3,
  | 4, 1, 3, 2, 5,
  | 5, 3, 2, 4, 1,
  | 1, 5, 4, 3, 2,
  | 4, 3, 2, 1, 5 |];
```

The first three items in the model declare the number of men/women and the set of men and women. Here we introduce the use of *anonymous enumerated types*. Both `Men` and `Women` are sets of size `n`, but we do not wish to mix them up so we use an anonymous enumerated type. This allows MiniZinc to detect modelling errors where we use `Men` for

`Women` or vice versa.

The matrices `rankWomen` and `rankMen`, respectively, give the women's ranking of the men and the men's ranking of the women. Thus, the entry `rankWomen[w,m]` gives the ranking by woman `w` of man `m`. The lower the number in the ranking, the more the man or women is preferred.

There are two arrays of decision variables: `wife` and `husband`. These, respectively, contain the wife of each man and the husband of each women.

The first two constraints

```
constraint forall (m in Men) (husband[wife[m]]=m);
constraint forall (w in Women) (wife[husband[w]]=w);
```

ensure that the assignment of husbands and wives is consistent: `w` is the wife of `m` implies `m` is the husband of `w` and vice versa. Notice how in `husband[wife[m]]` the index expression `wife[m]` is a decision variable, not a parameter.

The next two constraints are a direct encoding of the stability condition:

```
constraint forall (m in Men, o in Women) (
     rankMen[m,o] < rankMen[m,wife[m]] ->
         rankWomen[o,husband[o]] < rankWomen[o,m] );

constraint forall (w in Women, o in Men) (
     rankWomen[w,o] < rankWomen[w,husband[w]] ->
         rankMen[o,wife[o]] < rankMen[o,w] );
```

This natural modelling of the stable marriage problem is made possible by the ability to use decision variables as array indices and to construct constraints using the standard Boolean connectives. The alert reader may be wondering at this stage, what happens if the array index variable takes a value that is outside the index set of the array. MiniZinc treats this as failure: an array access `a[e]` implicitly adds the constraint `e in index_set(a)` to the closest surrounding Boolean context where `index_set(a)` gives the index set of `a`.

Anonymous Enumerated Types

An *anonymous enumerated type* is of the form `_(<integer set>)` where `<integer set>` is a fixed integer set expression such as `1..6` which defines the size of the enumerated type.
An anonymous enumerated type is just like any other enumerated type except that we have no names for its elements. When printed out, they are given unique names based on the enumerated type name.

Thus for example, consider the variable declarations

```
array[1..2] of int: a= [2,3];
var 0..2: x;
var 2..3: y;
```

The constraint `a[x] = y` will succeed with $x = 1 \wedge y = 2$ and $x = 2 \wedge y = 3$. And the constraint `not (a[x] = y)` will succeed with $x = 0 \wedge y = 2$, $x = 0 \wedge y = 3$, $x = 1 \wedge y = 3$ and $x = 2 \wedge y = 2$.

In the case of invalid array accesses by a parameter, the formal semantics of MiniZinc treats this as failure so as to ensure that the treatment of parameters and decision variables is consistent, but a warning is issued since it is almost always an error.

Listing 2.14: Model solving the magic series problem (`magic-series.mzn`).

```
int: n;
array[0..n-1] of var 0..n: s;

constraint forall(i in 0..n-1) (
   s[i] = (sum(j in 0..n-1)(bool2int(s[j]=i))));

solve satisfy;

output [ "s = \(s);\n" ] ;
```

The coercion function `bool2int` can be called with any Boolean expression. This allows the MiniZinc modeller to use so called *higher order constraints*. As a simple example consider the *magic series problem*: find a list of numbers $s = [s_0, \ldots, s_{n-1}]$ such that s_i is the number of occurrences of i in s. An example is $s = [1, 2, 1, 0]$.

A MiniZinc model for this problem is shown in Listing 2.14. The use of `bool2int` allows us to sum up the number of times the constraint `s[j]=i` is satisfied. Executing the command

```
$ minizinc --solver gecode --all-solutions magic-series.mzn -D "n=4;"
```

leads to the output

```
s = [1, 2, 1, 0];
----------
s = [2, 0, 2, 0];
----------
==========
```

indicating exactly two solutions to the problem (the effect of `--all-solutions` can be achieved in the MiniZinc IDE using the "User-defined behavior" option in the solver configuration pane).

Note that MiniZinc will automatically coerce Booleans to integers and integers to floats when required. We could replace the the constraint item in Listing 2.14 with

```
constraint forall(i in 0..n-1) (
   s[i] = (sum(j in 0..n-1)(s[j]=i)));
```

and get identical results, since the Boolean expression `s[j] = i` will be automatically coerced to an integer, effectively by the MiniZinc system automatically adding the missing `bool2int`.

Coercion

In MiniZinc one can *coerce* a Boolean value to an integer value using the `bool2int` function. Similarly one can coerce an integer value to a float value using `int2float`. The instantiation of the coerced value is the same as the argument, e.g. `par bool` is coerced to `par int`, while `var bool` is coerced to `var int`.
MiniZinc automatically coerces Boolean expressions to integer expressions and integer expressions to float expressions, by inserting `bool2int` and `int2float` in the model appropriately. Note that it will also coerce Booleans to floats using two steps.

2.2.7 Partiality and Relational Semantics

Since MiniZinc supports complex constraints, it is important to understand how partial functions are handled. For example, we would expect the following model to have the solution `b=true; x=1`, but also `b=false; x=0` and `b=false; x=1`:

```
var bool: b;
var 0..10: x;
constraint b -> 10 div x = 10;
```

However, division is a partial function, and `10 div 0` is undefined. So how exactly does MiniZinc handle partiality?

MiniZinc implements the *relational semantics*, which means that any undefinedness "bubbles up" to the closest Boolean context and becomes `false` there. In the case above, the closest Boolean context of the division is the `=` operator, so the right hand side of the implication becomes false. This forces the left hand side to be false as well.

MiniZinc has a number of built-in partial functions. The most common one is probably array access, since an expression `x[i]` is undefined if `i` is not in the index set of `x`.

In order to "guard" against partiality, we can use conditional statements like `if y=0 then 0 else x div y endif`, or `if i in index_set(x) then x[i] else 0 endif`, which define a default value for the cases where the expression is undefined.

A shorthand notation for this is the `default` operator, which allows you to write `(x div y) default 0` or `x[i] default 0` instead.

2.2.8 Set Constraints

Another powerful modelling feature of MiniZinc is that it allows sets containing integers to be decision variables: this means that when the model is evaluated the solver will find which elements are in the set.

As a simple example, consider the *0/1 knapsack problem*. This is a restricted form of the knapsack problem in which we can either choose to place the item in the knapsack or not. Each item has a weight and a profit and we want to find which choice of items leads to the maximum profit subject to the knapsack not being too full.

It is natural to model this in MiniZinc with a single decision variable: `var set of ITEM: knapsack` where `ITEM` is the set of possible items. If the arrays

`weight[i]` and `profit[i]` respectively give the weight and profit of item `i`, and the maximum weight the knapsack can carry is given by `capacity` then a natural model is given in Listing 2.15.

Listing 2.15: Model for the 0/1 knapsack problem (`knapsack.mzn`).

```
enum ITEM;
int: capacity;

array[ITEM] of int: profits;
array[ITEM] of int: weights;

var set of ITEM: knapsack;

constraint sum (i in knapsack) (weights[i]) <= capacity;

solve maximize sum (i in knapsack) (profits[i]) ;

output ["knapsack = \(knapsack)\n"];
```

Notice that the `var` keyword comes before the `set` declaration indicating that the set itself is the decision variable. This contrasts with an array in which the `var` keyword qualifies the elements in the array rather than the array itself since the basic structure of the array is fixed, i.e. its index set.

Listing 2.16: Model for the social golfers problems (`social-golfers.mzn`).

```
include "partition_set.mzn";
int: weeks;    set of int: WEEK = 1..weeks;
int: groups;   set of int: GROUP = 1..groups;
int: size;     set of int: SIZE = 1..size;
int: ngolfers = groups*size;
set of int: GOLFER = 1..ngolfers;

array[WEEK,GROUP] of var set of GOLFER: Sched;

% constraints
constraint
    forall (i in WEEK, j in GROUP) (
           card(Sched[i,j]) = size
        /\ forall (k in j+1..groups) (
                Sched[i,j] intersect Sched[i,k] = {}
           )
    ) /\
    forall (i in WEEK) (
          partition_set([Sched[i,j] | j in GROUP], GOLFER)
    ) /\
    forall (i in 1..weeks-1, j in i+1..weeks) (
        forall (x,y in GROUP) (
            card(Sched[i,x] intersect Sched[j,y]) <= 1
        )
```

```
    );
% symmetry
  constraint
    % Fix the first week %
    forall (i in GROUP, j in SIZE) (
        ((i-1)*size + j) in Sched[1,i]
    ) /\
    % Fix first group of second week %
    forall (i in SIZE) (
        ((i-1)*size + 1) in Sched[2,1]
    ) /\
    % Fix first 'size' players
    forall (w in 2..weeks, p in SIZE) (
        p in Sched[w,p]
    );

solve satisfy;

output [ show(Sched[i,j]) ++ " " ++
         if j == groups then "\n" else "" endif |
         i in WEEK, j in GROUP ];
```

Listing 2.17: Example data file for the social golfers problems (`social-golfers.dzn`).

```
weeks = 4;
groups = 4;
size =3 ;
```

As a more complex example of set constraint consider the social golfers problem shown in Listing 2.16. The aim is to schedule a golf tournament over `weeks` using `groups` × `size` golfers. Each week we have to schedule `groups` different groups each of size `size`. No two pairs of golfers should ever play in two groups.

The variables in the model are sets of golfers `Sched[i,j]` for the ith week and j^{th} group.

The constraints shown in lines 11-32 first enforces an ordering on the first set in each week to remove symmetry in swapping weeks. Next they enforce an ordering on the sets in each week, and make each set have a cardinality of `size`. They then ensure that each week is a partition of the set of golfers using the global constraint `partition_set`. Finally the last constraint ensures that no two players play in two groups together (since the cardinality of the intersection of any two groups is at most 1).

There are also symmetry breaking initialisation constraints shown in lines 34-46: the first week is fixed to have all players in order; the second week is made up of the first players of each of the first groups in the first week; finally the model forces the first `size` players to appear in their corresponding group number for the remaining weeks.

Executing the command

```
$ minizinc --solver gecode social-golfers.mzn social-golfers.dzn
```

where the data file defines a problem with 4 weeks, with 4 groups of size 3 leads to the output

```
1..3 4..6 7..9 10..12
{1,4,7} {2,9,12} {3,5,10} {6,8,11}
{1,6,9} {2,8,10} {3,4,11} {5,7,12}
{1,5,8} {2,7,11} {3,6,12} {4,9,10}
----------
```

Notice hows sets which are ranges may be output in range format.

2.2.9 Putting it all together

We finish this section with a complex example illustrating most of the features introduced in this chapter including enumerated types, complex constraints, global constraints, and complex output.

Listing 2.18: Planning wedding seating using enumerated types (`wedding.mzn`).

```
enum Guests = { bride, groom, bestman, bridesmaid, bob, carol,
  ted, alice, ron, rona, ed, clara};
set of int: Seats = 1..12;
set of int: Hatreds = 1..5;
array[Hatreds] of Guests: h1 = [groom, carol, ed, bride, ted];
array[Hatreds] of Guests: h2 = [clara, bestman, ted, alice, ron];
set of Guests: Males = {groom, bestman, bob, ted, ron,ed};
set of Guests: Females = {bride,bridesmaid,carol,alice,rona,clara};

array[Guests] of var Seats: pos; % seat of guest
array[Hatreds] of var Seats: p1;  % seat of guest 1 in hatred
array[Hatreds] of var Seats: p2;  % seat of guest 2 in hatred
array[Hatreds] of var 0..1: sameside;  % seats of hatred on same side
array[Hatreds] of var Seats: cost;  % penalty of hatred

include "alldifferent.mzn";
constraint alldifferent(pos);
constraint forall(g in Males)( pos[g] mod 2 == 1 );
constraint forall(g in Females)( pos[g] mod 2 == 0 );
constraint not (pos[ed] in {1,6,7,12});
constraint abs(pos[bride] - pos[groom]) <= 1 /\
           (pos[bride] <= 6 <-> pos[groom] <= 6);
constraint forall(h in Hatreds)(
               p1[h] = pos[h1[h]] /\
               p2[h] = pos[h2[h]] /\
               sameside[h] = bool2int(p1[h] <= 6 <-> p2[h] <= 6) /\
               cost[h] = sameside[h] * abs(p1[h] - p2[h]) +
                  (1 - sameside[h]) * (abs(13 - p1[h] - p2[h]) + 1) );

solve maximize sum(h in Hatreds)(cost[h]);
```

```
output [ show(g)++" " | s in Seats,g in Guests where fix(pos[g]) == s]
       ++ ["\n"];
```

The model of Listing 2.18 arranges seats at the wedding table. The table has 12 numbered seats in order around the table, 6 on each side. Males must sit in odd numbered seats, and females in even. Ed cannot sit at the end of the table because of a phobia, and the bride and groom must sit next to each other. The aim is to maximize the distance between known hatreds. The distance between seats is the difference in seat number if on the same side, otherwise its the distance to the opposite seat + 1.

Note that in the output statement we consider each seat `s` and search for a guest `g` who is assigned to that seat. We make use of the built in function `fix` which checks if a decision variable is fixed and returns its fixed value, and otherwise aborts. This is always safe to use in output statements, since by the time the output statement is run all decision variables should be fixed.

Running

```
$ minizinc --solver gecode wedding.mzn
```

Results in the output

```
ted bride groom rona ed carol ron alice bob bridesmaid bestman clara
----------
==========
```

The resulting table placement is illustrated in Fig. 2.2 where the lines indicate hatreds. The total distance is 22.

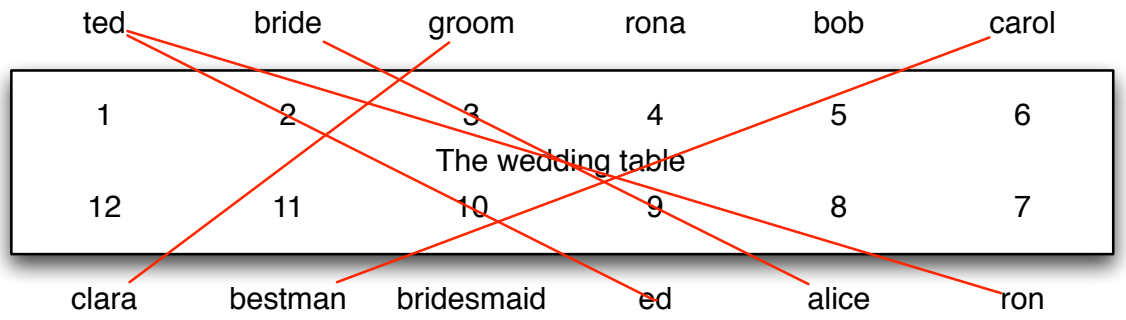


Fig. 2.2: Seating arrangement at the wedding table

Fix

In output items the built-in function `fix` checks that the value of a decision variable is fixed and coerces the instantiation from decision variable to parameter.

CHAPTER 2.3

Predicates and Functions

Predicates in MiniZinc allow us to capture complex constraints of our model in a succinct way. Predicates in MiniZinc are used to model with both predefined global constraints, and to capture and define new complex constraints by the modeller. Functions are used in MiniZinc to capture common structures of models. Indeed a predicate is just a function with output type `var bool`.

2.3.1 Global Constraints

There are many global constraints defined in MiniZinc for use in modelling. The definitive list is to be found in the documentation for the release, as the list is slowly growing. Below we discuss some of the most important global constraints.

2.3.1.1 Alldifferent

The `alldifferent` constraint takes an array of variables and constrains them to take different values. A use of the `alldifferent` has the form

```
alldifferent(array[int] of var int: x)
```

The argument is an array of integer variables.

The `alldifferent` constraint is one of the most studied and used global constraints in constraint programming. It is used to define assignment subproblems, and efficient global propagators for `alldifferent` exist. The models `send-more-money.mzn` (Listing 2.4) and `sudoku.mzn` (Listing 2.5) are examples of models using `alldifferent`.

2.3.1.2 Cumulative

The `cumulative` constraint is used for describing cumulative resource usage.

```
cumulative(array[int] of var int: s, array[int] of var int: d,
           array[int] of var int: r, var int: b)
```

It requires that a set of tasks given by start times `s`, durations `d`, and resource requirements `r`, never require more than a global resource bound `b` at any one time.

Listing 3.1: Model for moving furniture using `cumulative` (`moving.mzn`).

```
include "cumulative.mzn";

enum OBJECTS;
array[OBJECTS] of int: duration; % duration to move
array[OBJECTS] of int: handlers; % number of handlers required
array[OBJECTS] of int: trolleys; % number of trolleys required

int: available_handlers;
int: available_trolleys;
int: available_time;

array[OBJECTS] of var 0..available_time: start;
var 0..available_time: end;

constraint cumulative(start, duration, handlers, available_handlers);
constraint cumulative(start, duration, trolleys, available_trolleys);

constraint forall(o in OBJECTS)(start[o] +duration[o] <= end);

solve minimize end;

output [ "start = \(start)\nend = \(end)\n"];
```

Listing 3.2: Data for moving furniture using `cumulative` (`moving.dzn`).

```
OBJECTS = { piano, fridge, doublebed, singlebed,
            wardrobe, chair1, chair2, table };

duration = [60, 45, 30, 30, 20, 15, 15, 15];
handlers = [3, 2, 2, 1, 2, 1, 1, 2];
trolleys = [2, 1, 2, 2, 2, 0, 0, 1];

available_time = 180;
available_handlers = 4;
available_trolleys = 3;
```

The model in Listing 3.1 finds a schedule for moving furniture so that each piece of furniture has enough handlers (people) and enough trolleys available during the move. The available time, handlers and trolleys are given, and the data gives for each object

the move duration, the number of handlers and the number of trolleys required. Using the data shown in `ex-movingd`, the command

```
$ minizinc moving.mzn moving.dzn
```

may result in the output

```
start = [0, 60, 60, 90, 120, 0, 15, 105]
end = 140
----------
==========
```

Fig. 3.1 and Fig. 3.2 show the requirements for handlers and trolleys at each time in the move for this solution.

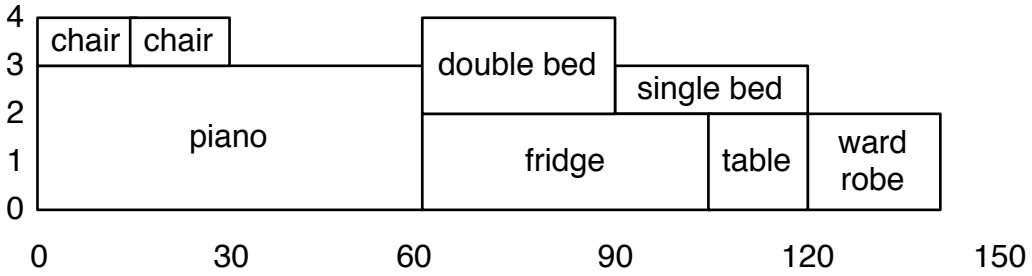


Fig. 3.1: Histogram of usage of handlers in the move.

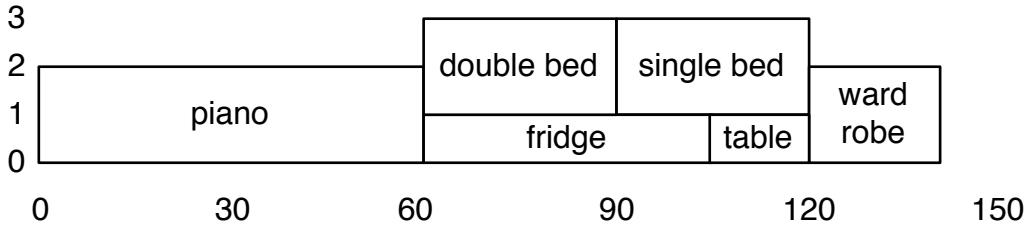


Fig. 3.2: Histogram of usage of trolleys in the move.

2.3.1.3 Table

The `table` constraint enforces that a tuple of variables takes a value from a set of tuples. Since there are no tuples in MiniZinc this is encoded using arrays. The usage of `table` has one of the forms

```
table(array[int] of var bool: x, array[int, int] of bool: t)
table(array[int] of var int:  x, array[int, int] of int:  t)
```

depending on whether the tuples are Boolean or integer. The constraint enforces $x \in t$ where we consider x and each row in t to be a tuple, and t to be a set of tuples.

Listing 3.3: Model for meal planning using `table` constraint (`meal.mzn`).

```
% Planning a balanced meal
include "table.mzn";
int: min_energy;
int: min_protein;
int: max_salt;
int: max_fat;
set of FOOD: desserts;
set of FOOD: mains;
set of FOOD: sides;
enum FEATURE = { name, energy, protein, salt, fat, cost};
enum FOOD;
array[FOOD,FEATURE] of int: dd; % food database

array[FEATURE] of var int: main;
array[FEATURE] of var int: side;
array[FEATURE] of var int: dessert;
var int: budget;

constraint main[name] in mains;
constraint side[name] in sides;
constraint dessert[name] in desserts;
constraint table(main, dd);
constraint table(side, dd);
constraint table(dessert, dd);
constraint main[energy] + side[energy] + dessert[energy] >=min_energy;
constraint main[protein]+side[protein]+dessert[protein] >=min_protein;
constraint main[salt] + side[salt] + dessert[salt] <= max_salt;
constraint main[fat] + side[fat] + dessert[fat] <= max_fat;
constraint budget = main[cost] + side[cost] + dessert[cost];

solve minimize budget;

output ["main = ",show(to_enum(FOOD,main[name])),
        ", side = ",show(to_enum(FOOD,side[name])),
        ", dessert = ",show(to_enum(FOOD,dessert[name])),
        ", cost = ",show(budget), "\n"];
```

Listing 3.4: Data for meal planning defining the `table` used (`meal.dzn`).

```
FOOD = { icecream, banana, chocolatecake, lasagna,
          steak, rice, chips, brocolli, beans} ;

dd = [| icecream,      1200,  50,  10, 120,  400     % icecream
      | banana,         800, 120,   5,  20,  120     % banana
      | chocolatecake, 2500, 400,  20, 100,  600     % chocolate cake
      | lasagna,       3000, 200, 100, 250,  450     % lasagna
      | steak,         1800, 800,  50, 100, 1200     % steak
```

```
      | rice,          1200,  50,   5,  20,  100     % rice
      | chips,         2000,  50, 200, 200,  250     % chips
      | brocolli,       700, 100,  10,  10,  125     % brocolli
      | beans,         1900, 250,  60,  90,  150 |]; % beans

min_energy = 3300;
min_protein = 500;
max_salt = 180;
max_fat = 320;
desserts = { icecream, banana, chocolatecake };
mains = { lasagna, steak, rice };
sides = { chips, brocolli, beans };
```

The model in Listing 3.3 searches for balanced meals. Each meal item has a name (encoded as an integer), a kilojoule count, protein in grams, salt in milligrams, and fat in grams, as well as cost in cents. The relationship between these items is encoded using a `table` constraint. The model searches for a minimal cost meal which has a minimum kilojoule count `min_energy`, a minimum amount of protein `min_protein`, maximum amount of salt `max_salt` and fat `max_fat`.

2.3.1.4 Regular

The `regular` constraint is used to enforce that a sequence of variables takes a value defined by a deterministic finite automaton (DFA). The usage of `regular` has the form

```
regular(array[int] of var $$Val: x,
        array[$$State,$$Val] of opt $$State: d,
        $$State: q0, set of $$State: F)
```

We use the `$$` prefix (here, `$$Val` and `$$State`) to indicate that these arguments can be of any enumerated type (including simple integers).

The `regular` constraint ensures that the sequence of values in array `x` is accepted by the DFA with transition function `d`, initial state `q0` and accepting states `F`. The transition function maps a state and a value to another state. Let's illustrate how to define the transition function using an example.

Consider a nurse rostering problem. Each nurse is scheduled for each day as either: (d) on day shift, (n) on night shift, or (o) off. In each four day period a nurse must have at least one day off, and no nurse can be scheduled for 3 night shifts in a row. This can be encoded using the incomplete DFA shown in Fig. 3.3.

We can encode this DFA by introducing an enumerated type for the states

```
enum State = {S1, S2, S3, S4, S5, S6};
```

The start state is `S1`, all states are final states, and the transition function is given by the matrix

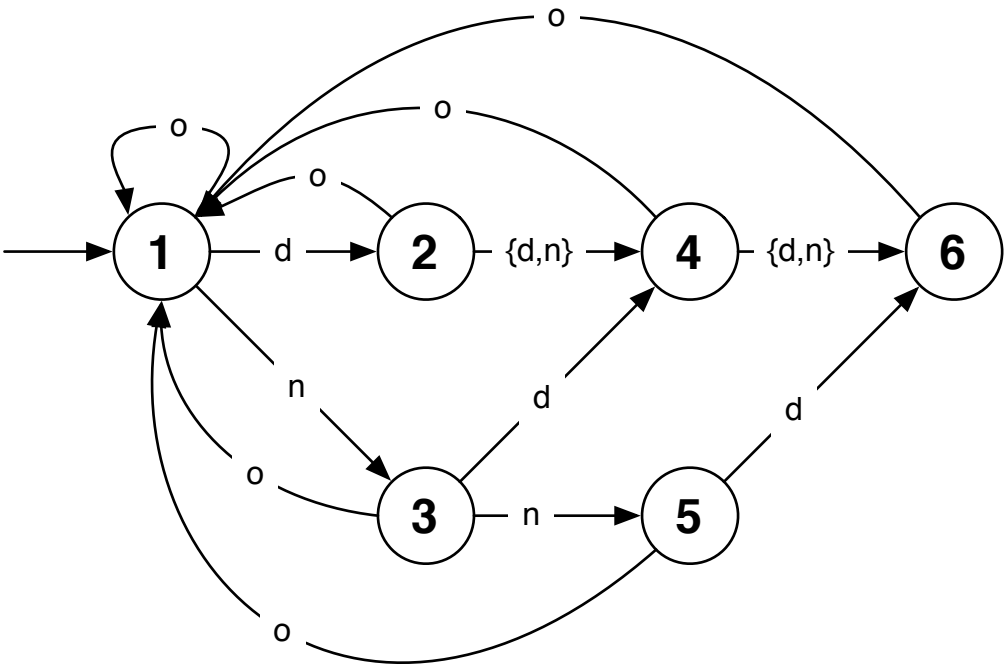


Fig. 3.3: A DFA determining correct rosters.

	d	n	o
S1	S2	S3	S1
S2	S4	S4	S1
S3	S4	S5	S1
S4	S6	S6	S1
S5	S6	<>	S1
S6	<>	<>	S1

Note that value <> in the table indicates an error state, i.e., an illegal transition. This is an example of an option type, which will be discussed in more detail in Section 2.4.

The model shown in Listing 3.5 finds a schedule for the nurses defined by enum `NURSE` over the days defined by enum `DAYS`, where we require `req_day` nurses on day shift each day, and `req_night` nurses on night shift, and that each nurse takes at least `min_night` night shifts.

Listing 3.5: Model for nurse rostering using `regular` constraint (`nurse.mzn`)

```
% Simple nurse rostering
include "regular.mzn";
enum NURSE;
enum DAY;
int: req_day;
int: req_night;
int: min_night;

enum SHIFT = { d, n, o };

enum STATE = { S1, S2, S3, S4, S5, S6 };

array[STATE,SHIFT] of opt STATE: t =
     [|      d:  n:  o:
      | S1:  S2, S3, S1
      | S2:  S4, S4, S1
      | S3:  S4, S5, S1
      | S4:  S6, S6, S1
      | S5:  S6, <>, S1
      | S6:  <>, <>, S1|];

array[NURSE,DAY] of var SHIFT: roster;

constraint forall(j in DAY)(
             sum(i in NURSE)(roster[i,j] == d) == req_day /\
             sum(i in NURSE)(roster[i,j] == n) == req_night
           );
constraint forall(i in NURSE)(
             regular([roster[i,j] | j in DAY], t, S1, STATE) /\
             sum(j in DAY)(roster[i,j] == n) >= min_night
           );
```

```
solve satisfy;
```

Listing 3.6: Example data for nurse rostering using `regular` constraint (`nurse.dzn`)

```
NURSE = Nurse(1..7);
DAY = D(1..10);

req_day = 3;
req_night = 2;
min_night = 2;
```

Running the command

```
$ minizinc nurse.mzn nurse.dzn
```

finds a 10 day schedule for 7 nurses, requiring 3 on each day shift and 2 on each night shift, with a minimum 2 night shifts per nurse. A possible output is

```
roster =
[|           D(1): D(2): D(3): D(4): D(5): D(6): D(7): D(8): D(9): D(10):
 | Nurse(1):    d,    o,    n,    o,    n,    o,    n,    n,    o,     o
 | Nurse(2):    d,    o,    d,    n,    n,    o,    d,    d,    n,     o
 | Nurse(3):    o,    d,    d,    n,    o,    n,    d,    n,    o,     n
 | Nurse(4):    d,    d,    d,    o,    d,    n,    d,    o,    n,     n
 | Nurse(5):    o,    d,    n,    d,    o,    d,    n,    o,    d,     d
 | Nurse(6):    n,    n,    o,    d,    d,    d,    o,    d,    d,     d
 | Nurse(7):    n,    n,    o,    d,    d,    d,    o,    d,    d,     d
 |];
----------
```

There is an alternate form of the regular constraint `regular_nfa` which specifies the regular expression using an NFA (without ϵ arcs). This constraint has the form

```
regular_nfa(array[int] of var $$Val: x,
        array[$$State,$$Val] of set of $$State: d, $$State: q0, set of $
↪$State: F)
```

It constrains that the sequence of values in array `x` is accepted by the NFA with transition function `d`, initial state `q0` and accepting states `F`. There is no need for option types in this case, since an impossible transition can be represented using the empty set.

2.3.2 Defining Predicates

One of the most powerful modelling features of MiniZinc is the ability for the modeller to define their own high-level constraints. This allows them to abstract and modularise their model. It also allows re-use of constraints in different models and allows the development of application specific libraries defining the standard constraints and types.

Listing 3.7: Model for job shop scheduling using predicates (jobshop2.mzn)

```
enum JOB;
enum TASK;
TASK: last = max(TASK);
array [JOB,TASK] of int: d;                    % task durations
int: total = sum(i in JOB, j in TASK)(d[i,j]);% total duration
int: digs = ceil(log(10.0,total));             % digits for output
array [JOB,TASK] of var 0..total: s;           % start times
var 0..total: end;                             % total end time

% nooverlap
predicate no_overlap(var int:s1, int:d1, var int:s2, int:d2) =
    s1 + d1 <= s2 \/ s2 + d2 <= s1;

constraint %% ensure the tasks occur in sequence
    forall(i in JOB) (
        forall(j in TASK where j < last)
            (s[i,j] + d[i,j] <= s[i,enum_next(j)]) /\
        s[i,last] + d[i,last] <= end
    );

constraint %% ensure no overlap of tasks
    forall(j in TASK) (
        forall(i,k in JOB where i < k) (
            no_overlap(s[i,j], d[i,j], s[k,j], d[k,j])
        )
    );

solve minimize end;

output ["end = \(end)\n"] ++
       [ show_int(digs,s[i,j]) ++ " " ++
         if j == last then "\n" else "" endif |
         i in JOB, j in TASK ];
```

We start with a simple example, revisiting the job shop scheduling problem from the previous section. The model is shown in Listing 3.7. The item of interest is the `predicate` item:

```
    s1 + d1 <= s2 \/ s2 + d2 <= s1;
```

This defines a new constraint that enforces that a task with start time `s1` and duration `d1` does not overlap with a task with start time `s2` and duration `d2`. This can now be used inside the model anywhere any other Boolean expression (involving decision variables) can be used.

As well as predicates the modeller can define new constraints that only involve parameters. These are useful to write fixed tests for a conditional expression. These are defined

using the keyword `test`. For example

```
test even(int:x) = x mod 2 = 0;
```

Predicate definitions

Predicates are defined by a statement of the form

```
predicate <pred-name> ( <arg-def>, ..., <arg-def> ) = <bool-exp>
```

The `<pred-name>` must be a valid MiniZinc identifier, and each `<arg-def>` is a valid MiniZinc type declaration.
One relaxation of argument definitions is that the index types for arrays can be unbounded, written `int`.

```
test <pred-name> ( <arg-def>, ..., <arg-def> ) = <bool-exp>
```

The `<bool-exp>` of the body must be fixed.
We also introduce a new form of the `assert` command for use in predicates.

```
assert ( <bool-exp>, <string-exp>, <exp> )
```

The type of the `assert` expression is the same as the type of the last argument. The `assert` expression checks whether the first argument is false, and if so prints the second argument string. If the first argument is true it returns the third argument.

Note that assert expressions are lazy in the third argument, that is if the first argument is false they will not be evaluated. Hence, they can be used for checking:

```
predicate lookup(array[int] of var int:x, int: i, var int: y) =
    assert(i in index_set(x), "index out of range in lookup",
           y = x[i]
    );
```

This code will not evaluate `x[i]` if `i` is out of the range of the array `x`.

2.3.3 Defining Functions

Functions are defined in MiniZinc similarly to predicates, but with a more general return type.

The function below defines the index in a Sudoku matrix of the $a1^{th}$ row (or column) of the ath subsquare.

```
function int: posn(int: a, int: a1) = (a-1) * S + a1;
```

With this definition we can replace the last constraint in the Sudoku problem shown in Listing 2.5 by

```
constraint forall(a, o in SubSquareRange)(
                  alldifferent([ puzzle [ posn(a,a1), posn(o,o1) ] |
                                          a1, o1 in SubSquareRange ] ) );
```

Functions are useful for encoding complex expressions that are used frequently in the model. For example, imagine placing the numbers 1 to n in different positions in an $n \times n$ grid such that the Manhattan distance between any two numbers i and j is greater than the maximum of the two numbers minus 1. The aim is to minimize the total of the Manhattan distances between the pairs. The Manhattan distance function can be expressed as:

```
function var int: manhattan(var int: x1, var int: y1,
                            var int: x2, var int: y2) =
         abs(x1 - x2) + abs(y1 - y2);
```

The complete model is shown in Listing 3.8.

Listing 3.8: Model for a number placement problem illustrating the use of functions (`manhattan.mzn`).

```
int: n;
set of int: NUM = 1..n;

array[NUM] of var NUM: x;
array[NUM] of var NUM: y;
array[NUM,NUM] of var 0..2*n-2: dist =
     array2d(NUM,NUM,[
     if i < j then manhattan(x[i],y[i],x[j],y[j]) else 0 endif
                     | i,j in NUM ]);

% manf
function var int: manhattan(var int: x1, var int: y1,
                            var int: x2, var int: y2) =
         abs(x1 - x2) + abs(y1 - y2);

constraint forall(i,j in NUM where i < j)
                 (dist[i,j] >= max(i,j)-1);

var int: obj = sum(i,j in NUM where i < j)(dist[i,j]);
solve minimize obj;

% simply to display result
include "alldifferent_except_0.mzn";
array[NUM,NUM] of var 0..n: grid;
constraint forall(i in NUM)(grid[x[i],y[i]] = i);
constraint alldifferent_except_0([grid[i,j] | i,j in NUM]);

output ["obj = \(obj);\n"] ++
       [ if fix(grid[i,j]) > 0 then show(grid[i,j]) else "." endif
```

```
        ++ if j = n  then "\n" else "" endif
      | i,j in NUM ];
```

Function definitions

Functions are defined by a statement of the form

```
function <ret-type> : <func-name> ( <arg-def>, ..., <arg-def> ) = <exp>
```

The <func-name> must be a valid MiniZinc identifier, and each <arg-def> is a valid MiniZinc type declaration. The <ret-type> is the return type of the function which must be the type of <exp>. Arguments have the same restrictions as in predicate definitions.

Functions in MiniZinc can have any return type, not just fixed return types. Functions are useful for defining and documenting complex expressions that are used multiple times in a model.

2.3.4 Reflection Functions

To help write generic tests and predicates, various reflection functions return information about array index sets, var set domains and decision variable ranges. Those for index sets are `index_set(<1-D array>)`, `index_set_1of2(<2-D array>)`, `index_set_2of2(<2-D array>)`, and so on for higher dimensional arrays.

A better model of the job shop conjoins all the non-overlap constraints for a single machine into a single disjunctive constraint. An advantage of this approach is that while we may initially model this simply as a conjunction of `non-overlap` constraints, if the underlying solver has a better approach to solving disjunctive constraints we can use that instead, with minimal changes to our model. The model is shown in Listing 3.9.

Listing 3.9: Model for job shop scheduling using `disjunctive` predicate (`jobshop3.mzn`).

```
include "disjunctive.mzn";

enum JOB;
enum TASK;
TASK: last = max(TASK);
array [JOB,TASK] of int: d;                    % task durations
int: total = sum(i in JOB, j in TASK)(d[i,j]);% total duration
int: digs = ceil(log(10.0,total));             % digits for output
array [JOB,TASK] of var 0..total: s;           % start times
var 0..total: end;                             % total end time

constraint %% ensure the tasks occur in sequence
    forall(i in JOB) (
        forall(j in TASK where j < last)
            (s[i,j] + d[i,j] <= s[i,enum_next(j)]) /\
        s[i,last] + d[i,last] <= end
```

```
    );

constraint %% ensure no overlap of tasks
    forall(j in TASK) (
        disjunctive([s[i,j] | i in JOB], [d[i,j] | i in JOB])
    );

solve minimize end;

output ["end = \(end)\n"] ++
       [ show_int(digs,s[i,j]) ++ " " ++
         if j == last then "\n" else "" endif |
         i in JOB, j in TASK ];
```

The `disjunctive` constraint takes an array of start times for each task and an array of their durations and makes sure that only one task is active at any one time. We define the disjunctive constraint as a predicate with signature

```
predicate disjunctive(array[int] of var int:s, array[int] of int:d);
```

We can use the disjunctive constraint to define the non-overlap of tasks as shown in Listing 3.9. We assume a definition for the `disjunctive` predicate is given by the file `disjunctive.mzn` which is included in the model. If the underlying system supports `disjunctive` directly, it will include a file `disjunctive.mzn` in its globals directory (with contents just the signature definition above). If the system we are using does not support disjunctive directly we can give our own definition by creating the file `disjunctive.mzn`. The simplest implementation simply makes use of the `no_overlap` predicate defined above. A better implementation is to make use of a global `cumulative` constraint assuming it is supported by the underlying solver. Listing 3.10 shows an implementation of `disjunctive`. Note how we use the `index_set` reflection function to (a) check that the arguments to `disjunctive` make sense, and (b) construct the array of resource utilisations of the appropriate size for `cumulative`. Note also that we use a ternary version of `assert` here.

Listing 3.10: Defining a `disjunctive` predicate using `cumulative` (`disjunctive.mzn`).

```
include "cumulative.mzn";

predicate disjunctive(array[int] of var int:s, array[int] of int:d) =
          assert(index_set(s) == index_set(d), "disjunctive: " ++
          "first and second arguments must have the same index set",
            cumulative(s, d, [ 1 | i in index_set(s) ], 1)
          );
```

2.3.5 Local Variables

It is often useful to introduce *local variables* in a predicate, function or test. The `let` expression allows you to do so. It can be used to introduce both decision variables and parameters, but parameters must be initialised. For example:

```
var s..e: x;
let {int: l = s div 2; int: u = e div 2; var l .. u: y;} in x = 2*y
```

introduces parameters `l` and `u` and variable `y`. While most useful in predicate, function and test definitions, `let` expressions can also be used in other expressions, for example for eliminating common subexpressions:

```
constraint let { var int: s = x1 + x2 + x3 + x4 } in
           l <= s /\ s <= u;
```

Local variables can be used anywhere and can be quite useful for simplifying complex expressions. Listing 3.11 gives a revised version of the wedding model, using local variables to define the objective function, rather than adding lots of variables to the model explicitly.

Listing 3.11: Using local variables to define a complex objective function (`wedding2.mzn`).

```
enum Guests = { bride, groom, bestman, bridesmaid, bob, carol,
  ted, alice, ron, rona, ed, clara};
set of int: Seats = 1..12;
set of int: Hatreds = 1..5;
array[Hatreds] of Guests: h1 = [groom, carol, ed, bride, ted];
array[Hatreds] of Guests: h2 = [clara, bestman, ted, alice, ron];
set of Guests: Males = {groom, bestman, bob, ted, ron,ed};
set of Guests: Females = {bride,bridesmaid,carol,alice,rona,clara};

array[Guests] of var Seats: pos; % seat of guest

include "alldifferent.mzn";
constraint alldifferent(pos);

constraint forall(g in Males)( pos[g] mod 2 == 1 );
constraint forall(g in Females)( pos[g] mod 2 == 0 );

constraint not (pos[ed] in {1,6,7,12});
constraint abs(pos[bride] - pos[groom]) <= 1 /\
           (pos[bride] <= 6 <-> pos[groom] <= 6);

solve maximize sum(h in Hatreds)(
          let {  var Seats: p1 = pos[h1[h]];
                 var Seats: p2 = pos[h2[h]];
                 var 0..1: same = bool2int(p1 <= 6 <-> p2 <= 6); } in
          same * abs(p1 - p2) + (1-same) * (abs(13 - p1 - p2) + 1));

output [ show(g)++" " | s in Seats,g in Guests where fix(pos[g]) == s]
       ++ ["\n"];
```

Using `let` expressions, it is possible to define a function whose result is not well-defined. For example, we could write the following:

```
function var int: x_or_x_plus_1(var int: x) = let {
  var 0..1: y;
} in x+y; % result is not well-defined!
```

The result, `x+y`, is indeed not functionally defined by the argument of the function, `x`. The MiniZinc compiler does not detect this, and **the behaviour of the resulting model is undefined**. In particular, calling this function twice with the same argument may or may not return the same result. It is therefore important to make sure that any function you define is indeed a function! **If you need non-deterministic behaviour, use a predicate:**

```
predicate x_or_x_plus_1(var int: x, var int: z) = let {
  var 0..1: y;
} in z=x+y;
```

2.3.6 Context

One limitation is that predicates and functions containing decision variables that are not initialised in the declaration cannot be used inside a negative context. The following is illegal:

```
predicate even(var int:x) =
          let { var int: y } in x = 2 * y;

constraint not even(z);
```

The reason for this is that solvers only solve existentially constrained problems, and if we introduce a local variable in a negative context, then the variable is *universally quantified* and hence out of scope of the underlying solvers. For example the $\neg even(z)$ is equivalent to $\neg\exists y.z = 2y$ which is equivalent to $\forall y.z \neq 2y$.

If local variables are given values, then they can be used in negative contexts. The following is legal

```
predicate even(var int:x) =
          let { var int: y = x div 2; } in x = 2 * y;

constraint not even(z);
```

Note that the meaning of `even` is correct, since if `x` is even then $x = 2 * (x \texttt{ div } 2)$. Note that for this definition $\neg even(z)$ is equivalent to $\neg\exists y.y = z \texttt{ div } 2 \wedge z = 2y$ which is equivalent to $\exists y.y = z \texttt{ div } 2 \wedge \neg z \neq 2y$, because y is functionally defined by z.

Every expression in MiniZinc appears in one of the four *contexts*: root, positive, negative, or mixed. The context of a non-Boolean expression is simply the context of its nearest enclosing Boolean expression. The one exception is that the objective expression appears in a root context (since it has no enclosing Boolean expression).

For the purposes of defining contexts we assume implication expressions `e1 -> e2` are rewritten equivalently as `not e1 \/ e2`, and similarly `e1 <- e2` is rewritten as

`e1 \/ not e2`.

The context for a Boolean expression is given by:

root
: root context is the context for any expression `e` appearing as the argument of `constraint` or as an assignment item, or appearing as a sub expression `e1` or `e2` in an expression `e1 /\ e2` occurring in a root context.

  Root context Boolean expressions must hold in any model of the problem.

positive
: positive context is the context for any expression appearing as a sub expression `e1` or `e2` in an expression `e1 \/ e2` occurring in a root or positive context, appearing as a sub expression `e1` or `e2` in an expression `e1 /\ e2` occurring in a positive context, or appearing as a sub expression `e` in an expression `not e` appearing in a negative context.

  Positive context Boolean expressions need not hold in a model, but making them hold will only increase the possibility that the enclosing constraint holds. A positive context expression has an even number of negations in the path from the enclosing root context to the expression.

negative
: negative context is the context for any expression appearing as a sub expression `e1` or `e2` in an expression `e1 \/ e2` or `e1 /\ e2` occurring in a negative context, or appearing as a sub expression `e` in an expression `not e` appearing in a positive context.

  Negative context Boolean expressions need not hold in a model, but making them false will increase the possibility that the enclosing constraint holds. A negative context expression has an odd number of negations in the path from the enclosing root context to the expression.

mixed
: mixed context is the context for any Boolean expression appearing as a subexpression `e1` or `e2` in `e1 <-> e2`, `e1 = e2`, or `bool2int(e)`.

  Mixed context expression are effectively both positive and negative. This can be seen from the fact that `e1 <-> e2` is equivalent to `(e1 /\ e2) \/ (not e1 /\ not e2)` and `x = bool2int(e)` is equivalent to `(e /\ x=1) \/ (not e /\ x=0)`.

Consider the code fragment

```
constraint x > 0 /\ (i <= 4 -> x + bool2int(x > i) = 5);
```

then `x > 0` is in the root context, `i <= 4}` is in a negative context, `x + bool2int(x > i) = 5` is in a positive context, and `x > i` is in a mixed context.

2.3.7 Local Constraints and Partiality

Let expressions can also be used to include local constraints, usually to constrain the behaviour of local variables. For example, consider defining an integer square root function making use of only multiplication:

```
function var int: mysqrt(var int:x) =
         let { var 0..infinity: y;
               constraint x = y * y; } in y;
```

The local constraints ensure `y` takes the correct value; which is then returned by the function.

Local constraints can be used in any let expression, though the most common usage is in defining functions. A function with local constraints is often *partial*, which means that it is not defined for all possible inputs. For example, the `mysqrt` function above constrains its argument `x` to take a value that is in fact the square of an integer. The MiniZinc compiler handles these cases according to the *relational semantics*, which means that the result of applying a partial function may become false in its enclosing Boolean context. For example, consider the following model:

```
var 1..9: x;
var 0..9: y;
constraint (x=3 /\ y=0) \/ y = mysqrt(x);
```

Clearly, the intention of the modeller is that `x=3, y=0` should be a solution. This requires the compiler to take care not to "lift" the constraint `x=y*y` out of the context of the function, because that would prevent it from finding any solution with `x=3`. You can verify that the set of solutions contains `x=3, y=0` as well as the expected `x=1, y=1`, `x=4, y=2` and `x=9, y=3`.

If you define a *total* function using local constraints, you can give the compiler a hint that allows it to produce more efficient code. For example, you could write the square function (not square root, note the subtle difference) as follows:

```
function var int: mysqr(var int:x) ::promise_total =
         let { var 0..infinity: y;
               constraint y = x * x; } in y;
```

This function is indeed defined for any input value `x`. The `::promise_total` annotation tells the compiler that it can safely lift all local constraints out of the context of the function call.

Let expressions

Local variables can be introduced in any expression with a *let expression* of the form:

```
let { <dec>; ... <dec> ; } in <exp>
```

The declarations `<dec>` can be declarations of decision variables and parameters (which must be initialised) or constraint items. No declaration can make use of a newly declared variable before it is introduced.
Note that local variables and constraints cannot occur in tests. Local variables cannot occur in predicates or functions that appear in a negative or mixed context, unless the variable is defined by an expression.

2.3.8 Domain Reflection Functions

Other important reflection functions are those that allow us to access the domains of variables. The expression `lb(x)` returns a value that is lower than or equal to any value that `x` may take in a solution of the problem. Usually it will just be the declared lower bound of `x`. If `x` is declared as a non-finite type, e.g. simply `var int` then it is an error. Similarly the expression `dom(x)` returns a (non-strict) superset of the possible values of `x` in any solution of the problem. Again it is usually the declared values, and again if it is not declared as finite then there is an error.

Listing 3.12: Using reflection predicates (`reflection.mzn`).

```
var -10..10: x;
constraint x in 0..4;
int: y = lb(x);
set of int: D = dom(x);
solve satisfy;
output ["y = ", show(y), "\nD = ", show(D), "\n"];
```

For example, the model show in Listing 3.12 may output

```
y = -10
D = -10..10
----------
```

or

```
y = 0
D = {0, 1, 2, 3, 4}
----------
```

or any answer with $-10 \leq y \leq 0$ and $\{0,\ldots,4\} \subseteq D \subseteq \{-10,\ldots,10\}$.

Variable domain reflection expressions should be used in a manner where they are correct for any safe approximations, but note this is not checked! For example the additional code

```
var -10..10: z;
constraint z <= y;
```

is not a safe usage of the domain information. Since using the tighter (correct) approximation leads to more solutions than the weaker initial approximation.

Domain reflection

There are reflection functions to interrogate the possible values of expressions containing variables:

- `dom(<exp>)` returns a safe approximation to the possible values of the expression.
- `lb(<exp>)` returns a safe approximation to the lower bound value of the expression.
- `ub(<exp>)` returns a safe approximation to the upper bound value of the expression.

The expressions for `lb` and `ub` can only be of types `int`, `bool`, `float` or `set of int`. For `dom` the type cannot be `float`. If one of the variables appearing in `<exp>` has a non-finite declared type (e.g. `var int` or `var float`) then an error can occur.
There are also versions that work directly on arrays of expressions (with similar restrictions):

- `dom_array(<array-exp>)`: returns a safe approximation to the union of all possible values of the expressions appearing in the array.
- `lb_array(<array-exp>)` returns a safe approximation to the lower bound of all expressions appearing in the array.
- `ub_array(<array-exp>)` returns a safe approximation to the upper bound of all expressions appearing in the array.

The combinations of predicates, local variables and domain reflection allows the definition of complex global constraints by decomposition. We can define the time based decomposition of the `cumulative` constraint using the code shown in Listing 3.13.

Listing 3.13: Defining a `cumulative` predicate by decomposition (`cumulative.mzn`).

```
%-----------------------------------------------------------------------%
% Requires that a set of tasks given by start times 's',
% durations 'd', and resource requirements 'r',
% never require more than a global
% resource bound 'b' at any one time.
% Assumptions:
% - forall i, d[i] >= 0 and r[i] >= 0
%-----------------------------------------------------------------------%
predicate cumulative(array[int] of var int: s,
                     array[int] of var int: d,
                     array[int] of var int: r, var int: b) =
   assert(index_set(s) == index_set(d) /\
          index_set(s) == index_set(r),
     "cumulative: the array arguments must have identical index sets",
   assert(lb_array(d) >= 0 /\ lb_array(r) >= 0,
     "cumulative: durations and resource usages must be non-negative",
          let {
              set of int: times =
                lb_array(s) ..
                max([ ub(s[i]) + ub(d[i]) | i in index_set(s) ])
              }
          in
             forall( t in times ) (
```

```
                b >= sum( i in index_set(s) ) (
                   bool2int( s[i] <= t /\ t < s[i] + d[i] ) * r[i]
                )
             )
       )
   );
```

The decomposition uses `lb` and `ub` to determine the set of times `times` over which tasks could range. It then asserts for each time `t` in `times` that the sum of resources for the active tasks at time `t` is less than the bound `b`.

2.3.9 Scope

It is worth briefly mentioning the scope of declarations in MiniZinc. MiniZinc has a single namespace, so all variables appearing in declarations are visible in every expression in the model. MiniZinc introduces locally scoped variables in a number of ways:

- as iterator variables in comprehension expressions
- using `let` expressions
- as predicate and function arguments

Any local scoped variable overshadows the outer scoped variables of the same name.

Listing 3.14: A model for illustrating scopes of variables (`scope.mzn`).

```
int: x = 3;
int: y = 4;
predicate smallx(var int:y) = -x <= y /\ y <= x;
predicate p(int: u, var bool: y) =
      exists(x in 1..u)(y \/ smallx(x));
constraint p(x,false);
solve satisfy;
```

For example, in the model shown in Listing 3.14 the `x` in `-x <= y` is the global `x`, the `x` in `smallx(x)` is the iterator `x in 1..u`, while the `y` in the disjunction is the second argument of the predicate.

CHAPTER 2.4

Option Types

Option types are a powerful abstraction that allows for concise modelling. An option type decision variable represents a decision that has another possibility $\top$, represented in MiniZinc as <> indicating the variable is *absent*. Option type decisions are useful for modelling problems where a decision is not meaningful unless other decisions are made first.

2.4.1 Declaring and Using Option Types

Option type Variables

An option type variable is declared as:

```
var opt <type> : <var-name>
```

where `<type>` is one of `int`, `float` or `bool` or a fixed range expression. Option type variables can be parameters.
An option type variable can take the additional value <> indicating *absent*.

An option type variable behaves like a normal, non-optional variable, as long as it is not absent, i.e., as long as it is not equal to <>. An absent option type variable should behave as if the object it represents does not exist. This means different things for different functions and relations.

For example, when adding <> to another variable `x`, the result is just `x`. Similarly, `all_different([x,y,z])` should behave exactly like `all_different([x,y])` in case `z=<>`, i.e., `z` is absent.

Option type variables can be used like their non-optional versions with most operators on Booleans, integers and floats. Many functions and predicates in the MiniZinc library are also defined for option type variables. Here are a few examples of how different functions and operators treat option types:

```
% Expression:                Equivalent to:          Simplified:
<> + a                  =    0 + a                =  a
a * <>                  =    a * 1                =  a
sum([x1,<>,x2])         =    sum([x1,x2])
sum([<>,<>])            =    sum([])              = 0
product([x1,x2,<>])     =    product([x1,x2])
product([<>,<>,<>])     =    product([])          = 1
exists([b1,<>])         =    exists([b1])         = b1
exists([<>,<>])         =    exists([])           = false
forall([<>,b1,b2])      =    forall([b1,b2])
forall([<>,<>])         =    forall([])           = true
all_different([x,<>,y]) =    all_different([x,y])
```

Comparison operators return `true` if any of their arguments is absent. For instance, `3 <= <>` is `true`, as is `<> <= 3`. However, note that equality between option type expressions is only `true` if both expressions have the same optionality: `<> = <>` is `true`, but `3 = <>` is `false`. If you need the "weaker" version of equality, MiniZinc provides the `~=` operator: `3 ~= <>` is `true`. The `~!=` operator is the weak version of disequality, it is true if either side is absent or they are not equal. Note that `a ~!= b` is different from `not (a ~= b)`, because `<> ~!= <>` is true, while `not (<> ~= <>)` is false.

Similarly, it can sometimes be useful to have "weak" versions of the arithmetic operators that return `<>` if any of their arguments is absent. MiniZinc provides the `~+`, `~-`, `~*`, `~/` and `~div` operators for this purpose (e.g., `3 + <> =3`, but `3 ~+ <> = <>`).

Operations on option type variables

Two builtin functions and a binary operator are provided for option type variables: `absent(v)` returns `true` iff option type variable `v` takes the value `<>`, `occurs(v)` returns `true` iff option type variable `v` does *not* take the value `<>`, and `x default y` returns `x` if `x` occurs, and `y` otherwise.
In addition, the function `deopt(v)` returns the normal value of `v` or returns undefined if it takes the value `<>`. This function should not be used in normal models, but is required to implement predicates over option type variables.

2.4.2 Option Types in Scheduling Problems

A common use of option types is for optional tasks in scheduling. In the flexible job shop scheduling problem we have `n` tasks to perform on `k` machines, and the time to complete each task on each machine may be different. The aim is to minimize the completion time of all tasks. A model using option types to encode the problem is given in Listing 4.1. We model the problem using $n \times k$ optional tasks representing the possibility of each task run on each machine. We require that start time of the task and its duration spans the optional tasks that make it up, and require only one actually runs using the `alternative` global constraint. We require that at most one task runs on any machine using the `disjunctive` global constraint extended to optional tasks. Finally we constrain that at most `k` tasks run at any time, a redundant constraint that holds on the actual (not optional) tasks.

Listing 4.1: Model for flexible job shop scheduling using option types (`flexible-js.mzn`).

```
include "globals.mzn";

int: horizon;                  % time horizon
set of int: Time = 0..horizon;
enum Task;
enum Machine;

array[Task,Machine] of int: d; % duration on each machine
int: maxd = max([ d[t,m] | t in Task, m in Machine ]);
int: mind = min([ d[t,m] | t in Task, m in Machine ]);

array[Task] of var Time: S;              % start time
array[Task] of var mind..maxd: D;        % duration
array[Task,Machine] of var opt Time: O; % optional task start

constraint forall(t in Task)(alternative(S[t],D[t],
                 [O[t,m]|m in Machine],[d[t,m]|m in Machine]));
constraint forall(m in Machine)
                 (disjunctive([O[t,m]|t in Task],[d[t,m]|t in Task]));
constraint cumulative(S, D, [i: 1 | i in Task], card(Machine));

solve minimize max(t in Task)(S[t] + D[t]);
```

2.4.3 Hidden Option Types

Option type variables arise implicitly when list comprehensions are constructed with iteration over variable sets, or where the expressions in `where` clauses are not fixed.

For example the model fragment

```
var set of 1..n: x;
constraint sum(i in x)(i) <= limit;
```

is syntactic sugar for

```
var set of 1..n: x;
constraint sum(i in 1..n)(if i in x then i else <> endif) <= limit;
```

The `sum` builtin function actually operates on a list of type-inst `var opt int`. Since the `<>` acts as the identity 0 for + this gives the expected results.

Similarly the model fragment

```
array[1..n] of var int: x;
constraint forall(i in 1..n where x[i] >= 0)(x[i] <= limit);
```

is syntactic sugar for

```
array[1..n] of var int: x;
constraint forall(i in 1..n)(if x[i] >= 0 then x[i] <= limit else <> endif);
```

Again the `forall` function actually operates on a list of type-inst `var opt bool`. Since `<>` acts as identity `true` for `/\` this gives the expected results.

The hidden uses can lead to unexpected behaviour though so care is warranted. Consider

```
var set of 1..9: x;
constraint card(x) <= 4;
constraint length([ i | i in x]) > 5;
solve satisfy;
```

which would appear to be unsatisfiable. It returns `x = {1,2,3,4}` as example answer. This is correct since the second constraint is equivalent to

```
constraint length([ if i in x then i else <> endif | i in 1..9 ]) > 5;
```

and the length of the list of optional integers is always 9 so the constraint always holds!

One can avoid hidden option types by not constructing iteration over variable sets or using unfixed `where` clauses. For example the above two examples could be rewritten without option types as

```
var set of 1..n: x;
constraint sum(i in 1..n)(bool2int(i in x)*i) <= limit;
```

and

```
array[1..n] of var int: x;
constraint forall(i in 1..n)(x[i] >= 0 -> x[i] <= limit);
```

2.4.4 Option Type Parameters

Option type variables of fixed parameter type can be used to define optional model parameters. For example, a variable defined like this:

```
opt bool: enable_feature;
```

could be used to enable or disable a certain optional feature of a model. In contrast to other parameter variables, these optional parameters do not have to be assigned a value

(e.g., the data file may omit an assignment to `enable_feature`). In that case, the variable is automatically assigned the value <>. In the model, such an optional parameter could be used like this:

```
bool: enable_feature_enabled = enable_feature default false;

constraint if enable_feature_enabled then ... endif;
```

CHAPTER 2.5

Tuple and record types

In MiniZinc models we often deal with multiple points of data and multiple decisions that all concern the same thing, an "object". There are multiple ways of modelling this. It is common practice in MiniZinc to have different arrays for different types of data points or decisions. These arrays will then share a common index mapping to which object each data point belongs.

In this section we present an alternative, "object-oriented", approach in MiniZinc in the form of tuple and record types. Using these types, different types of data points and decisions can be combined into collections for the modeller's convenience.

2.5.1 Declaring and using tuples and records

Tuple types and literals

A tuple type variable is declared as:

```
<var-par> tuple(<ti-expr>, ...): <var-name>
```

where one or more type instantiations, `<ti-expr>`, are placed between the parentheses to declare the types contained in the tuple variable.
For convenience the `var` keyword can be used to varify all the member types of the tuple (i.e., `var tuple(int, bool)` is the same type as `tuple(var int, var bool)`).
A tuple literal is created using the following syntax:

```
(<expr>, [ <expr>, ... ])
```

For example, `(1, true)` is a tuple literal of type `tuple(int,bool)`. Note that when a tuple contains multiple members, then adding a trailing comma is optional, but when it contains a single member it is **required** to distinguish the literal from a normal parenthesized expression.

Tuples provide a very simple way to create a collection that contains values of different types. In a tuple variable, the values contained in the tuple can be accessed using a number representing the place in the tuple. For example, in the tuple `any: x = (1, true, 2.0)` the first member, `1`, can be retrieved using `x.1`, and the final member, `2.0`, can be retrieved using `x.3`. Note that the MiniZinc compiler will raise a **Type Error**, when an integer is used that is lower than one or higher than the number of members in the tuple.

Although tuple types can be useful, it can often be confusing which member represents what. Record types improve on this by associating a name with each member of the type.

Record types and literals

A record type variable is declared as:

```
<var-par> record(<ti-expr-and-id>, ...): <var-name>
```

where one or more type instantiations with a corresponding identifier, `<ti-expr-and-id>`, are placed between the parentheses to declare the types contained in the record variable.
For convenience the `var` keyword can be used to varify all the member types of the record (i.e., `var record(int: i, bool: b)` is the same type as `record(var int: i, var bool: b)`).
A record literal is created using the following syntax:

```
(<ident>: <expr>[, ... ])
```

For example, `(i: 1, b: true)` is a record literal of type `record(int: i, bool: b)`. Different from tuples, record literals with only a single member do not require a trailing comma.

The syntax for accessing a member of a record is very similar to accessing a member of a tuple. The difference is that instead of a number, we use the name given to the member. For example, given the record `any: x = (i: 1, b: true)`, the integer value `1` that is named by identifier `i` is retrieved using `x.i`. Using identifiers that do not name a member (or any number) will once more result in a **Type Error**.

2.5.2 Using type-inst synonyms

When using records and tuples, writing the types in many places can quickly become tedious and confusing. Additionally, it might often make sense to give a name to such a type-inst to describe its meaning. For this purpose, you can use type-inst synonyms in MiniZinc.

Type-inst synonyms

A type-inst synonym is declared as:

```
type <ident> <annotations> = <ti-expr>;
```

where the identifier `<ident>` can be used instead of the type-inst `<ti-expr>` where required.

For example, in the following MiniZinc fragment we declare two synonyms, `Coord` and `Number`

```
type Coord = var record(int: x, int: y, int: z);
type Number = int;
```

In a model that contains these definitions, we can now de-

clare a variable `array[1..10] of Coord: placement;` or a function `function Number: add(Number: x, Number: y) = x + y;`

Similar to record and tuple types, the `var` keyword can be used before the identifier of a type-inst synonym to varify the type-inst. For instance, given the synonym `type OnOff = bool;` and the variable declaration `var OnOff: choice;`, the type-inst of `choice` would be `var bool`. Different from records and tuple, the reverse is also possible using the `par` keyword. For instance, the given the synonym `type Choice = var bool;` and the variable declaration `par Choice: check = fix(choice);` the type-inst of `check` would be `bool`.

2.5.3 Types with both var and par members

Tuples and records can be used to collect both data and decisions about an object, and it can be natural to mix data and decisions that concern the same object. However, when data and decisions are contained within the same tuple or record type, initialization of these types can be complex. To avoid the usage of anonymous variables or let-expressions and allow the assignment from data files, it is recommended to split data and decisions into separate types.

For example, in a rostering problem it would be natural to combine information about the employees, both data and decisions. It might thus be natural to define the following synonyms and variable declarations.

```
enum EmpId;
type Employee = record(
  string: name,
  array[Timespan] of bool: available,
  set of Capability: capacities,
  array[Timespan] of var Shift: shifts,
  var 0..infinity: hours,
);

array[EmpId] of Employee: employee;
```

However, it is not possible to assign `employees` from a data file. Instead, the data and the decisions could be split as follows.

```
enum EmpId;
type EmployeeData = record(
  string: name,
  array[Timespan] of bool: available,
  set of Capability: capacities,
);
type EmployeeVar = record(
  array[Timespan] of var Shift: shifts,
  var 0..infinity: hours,
);

array[EmpId] of EmployeeData: employee_data;
```

```
array[EmpId] of EmployeeVar: employee_var;
```

Now it is possible to initialize `employee_data` from a data file. Depending on the model, it might be easiest to use these synonyms and variable declarations separately, but it might be useful to combine them again. For instance, when your model contains functions that should operate on the complete record. The `++` operator can be used to easily combine these synonyms and values, as shown in the following fragment.

```
type Employee = EmployeeData ++ EmployeeVar;

array[EmpId] of Employee: employee = [employee_data[id] ++ employee_var[id]
↪| id in EmpId];
```

`++` operator for tuples and records

The `++` operator can be used to combine expressions and type-inst expressions that both have tuple types or both have record types.

```
<expr> ++ <expr>
<ti-expr> ++ <ti-expr>
```

When both (type-inst) expressions have a tuple type, this operation is very similar to array concatenation. The result of evaluating the expression will be a new tuple that contains all members of left-hand side followed by all the members of the right-hand side.
When both (type-inst) expressions have a record type, then the result of evaluating the expression is a record that merges the fields in both records. Note that if both records contain a field with the same name, then the compiler will reject this as a Type Error.

2.5.4 Example: rectangle packing using records

Consider a packing problem where a collection of rectangular objects have to be packed into a rectangular space. Importantly, none of the rectangles being packed can overlap. To keep the model simple, we don't allow the rectangles to be turned. When we create a model to decide whether the packing is possible, we can use records to help us make the model easier to read.

The data for the model comes in two parts: the total area of the space and the sizes of the rectangles being packed. We can represent both using a "dimensions" record that includes the width and the height of a rectangle. We therefore define the following type-inst synonym.

```
type Dimensions = record(int: width, int: height);
```

To decide whether the problem is satisfiable or not, we will have to find a placement for the rectangles being packed. We can thus use the coordinates of their left bottom corner as our decision variables, for which a type-inst synonym can be defined as follows.

```
type Coordinates = record(var 0..infinity: x, var 0..infinity: y);
```

To enforce that no rectangles overlap, it can be easy to start by writing a predicate that ensures that two rectangles do not overlap. This predicate will require both the information about the dimensions of the rectangle and the coordinate decision variable. As such, it can be valuable to first combine these records into a new record type. The following fragment shows the additional type-inst synonym, and a definition for the described predicate.

```
type Rectangle = Dimensions ++ Coordinates;

% No overlap predicate
predicate no_overlap(Rectangle: rectA, Rectangle: rectB) =
  rectA.x + rectA.width <= rectB.x
  \/ rectB.x + rectB.width <= rectA.x
  \/ rectA.y + rectA.height <= rectB.y
  \/ rectB.y + rectB.height <= rectA.y;
```

Using these definitions the remainder of the model is now easy to define as shown below.

Listing 5.1: Partial rectangle packing model using record types (full model: `rect_packing.mzn` example data: `rect_packing.json`).

```
% Instance data
array[_] of Dimensions: rectDim;
Dimensions: area;

% Decision variables
array[index_set(rectDim)] of Coordinates: rectCoord ::no_output;

array[_] of Rectangle: rectangles ::output = [ rectDim[i] ++ rectCoord[i] |␣
↪i in index_set(rectDim)];

% Constraint: rectangles must be placed within the area
constraint forall(rect in rectangles) (
  rect.x + rect.width <= area.width
  /\ rect.y + rect.height <= area.height
);

% Constraint: no rectangles can overlap
constraint forall(i, j in index_set(rectangles) where i < j) (
  no_overlap(rectangles[i], rectangles[j])
);
```

CHAPTER 2.6

Search

By default in MiniZinc there is no declaration of how we want to search for solutions. This leaves the search completely up to the underlying solver. But sometimes, particularly for combinatorial integer problems, we may want to specify how the search should be undertaken. This requires us to communicate to the solver a search strategy. Note that the search strategy is *not* really part of the model. Indeed it is not required that each solver implements all possible search strategies. MiniZinc uses a consistent approach to communicating extra information to the constraint solver using *annotations*.

2.6.1 Finite Domain Search

Search in a finite domain solver involves examining the remaining possible values of variables and choosing to constrain some variables further. The search then adds a new constraint that restricts the remaining values of the variable (in effect guessing where the solution might lie), and then applies propagation to determine what other values are still possible in solutions. In order to guarantee completeness, the search leaves another choice which is the negation of the new constraint. The search ends either when the finite domain solver detects that all constraints are satisfied, and hence a solution has been found, or that the constraints are unsatisfiable. When unsatisfiability is detected the search must proceed down a different set of choices. Typically finite domain solvers use depth first search where they undo the last choice made and then try to make a new choice.

Listing 6.1: Model for n-queens (`nqueens.mzn`).

```
int: n;
array [1..n] of var 1..n: q; % queen in column i is in row q[i]

include "alldifferent.mzn";

constraint alldifferent(q);                        % distinct rows
constraint alldifferent([ q[i] + i | i in 1..n]); % distinct diagonals
```

```
constraint alldifferent([ q[i] - i | i in 1..n]); % upwards+downwards

% search
solve :: int_search(q, first_fail, indomain_min)
      satisfy;
output [ if fix(q[j]) == i then "Q" else "." endif ++
         if j == n then "\n" else "" endif | i,j in 1..n];
```

A simple example of a finite domain problem is the n queens problem which requires that we place n queens on an $n \times n$ chessboard so that none can attack another. The variable `q[i]` records in which row the queen in column `i` is placed. The `alldifferent` constraints ensure that no two queens are on the same row, or diagonal. A typical (partial) search tree for `n = 9` is illustrated in Fig. 6.1. We first set `q[1] = 1`, this removes values from the domains of other variables, so that e.g. `q[2]` cannot take the values 1 or 2. We then set `q[2] = 3`, this further removes values from the domains of other variables. We set `q[3] = 5` (its earliest possible value). The state of the chess board after these three decisions is shown in Fig. 6.2 where the queens indicate the position of the queens fixed already and the stars indicate positions where we cannot place a queen since it would be able to take an already placed queen.

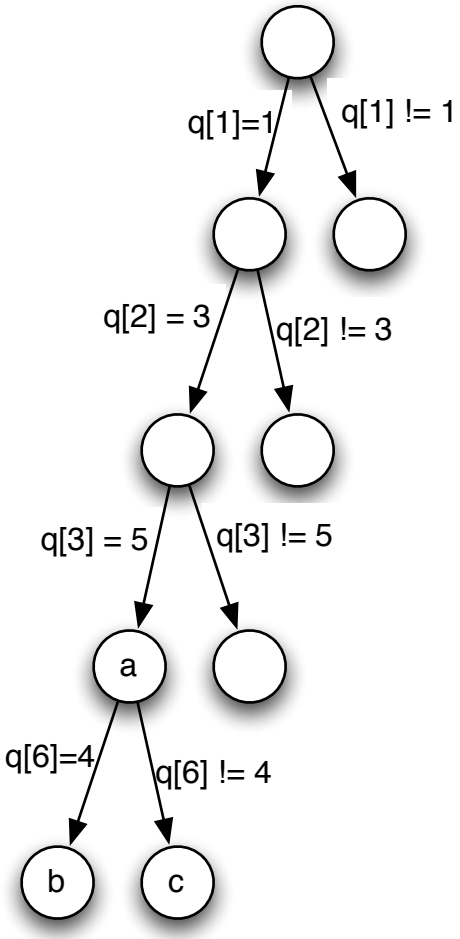


Fig. 6.1: Partial search trees for 9 queens

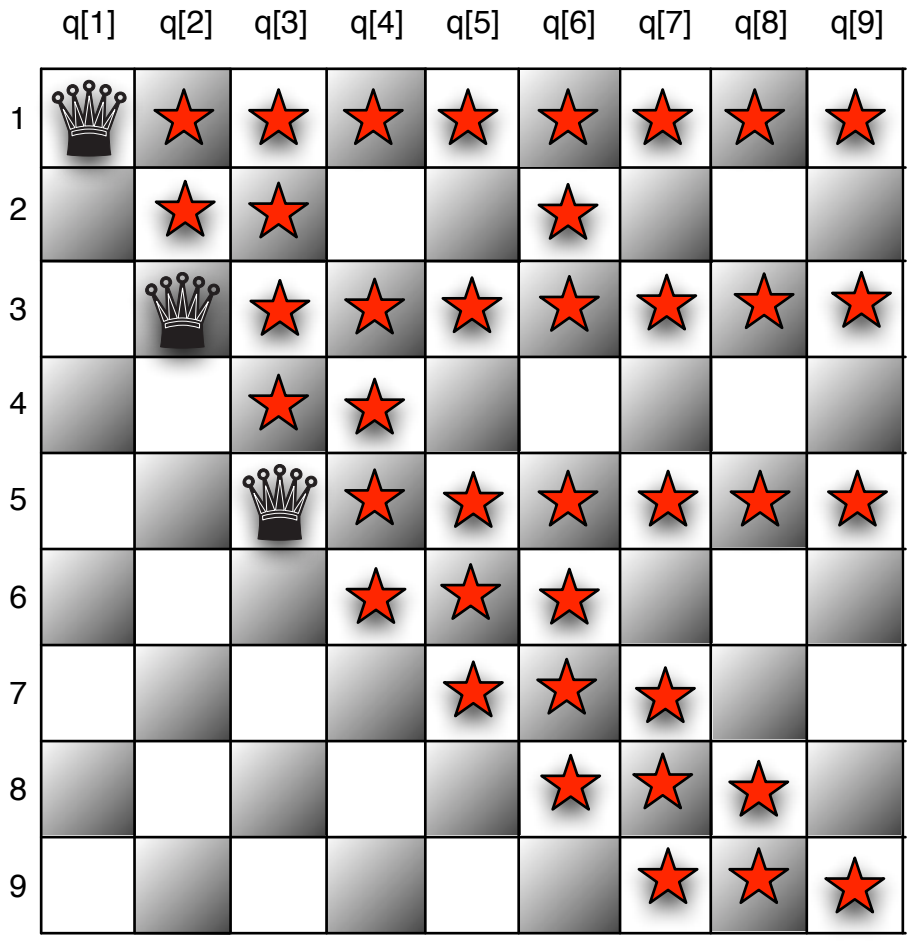


Fig. 6.2: The state after the addition of `q[1] = 1`, `q[2] = 4`, `q[3] = 5`

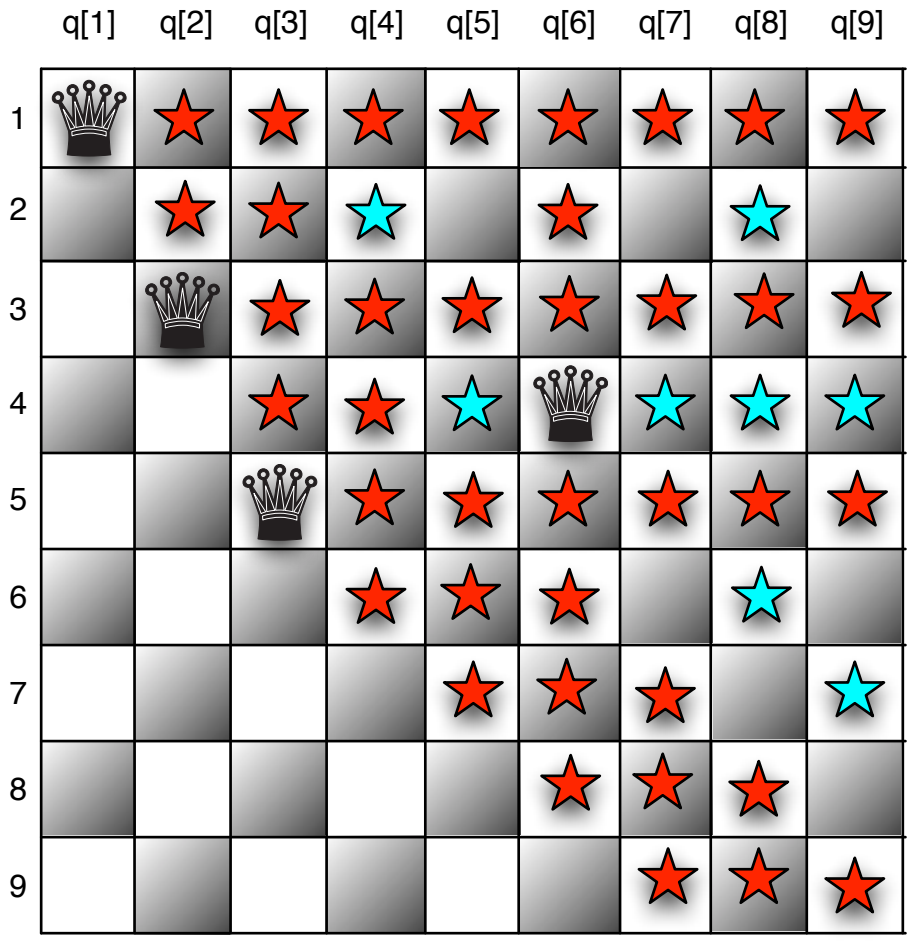


Fig. 6.3: The initial propagation on adding further `q[6] = 4`

A search strategy determines which choices to make. The decisions we have made so far follow the simple strategy of picking the first variable which is not fixed yet, and try to set it to its least possible value. Following this strategy the next decision would be `q[4] = 2`. An alternate strategy for variable selection is to choose the variable whose current set of possible values (*domain*) is smallest. Under this so called *first-fail* variable selection strategy the next decision would be `q[6] = 4`. If we make this decision, then initially propagation removes the additional values shown in Fig. 6.3. But this leaves only one value for `q[8]`, `q[8] = 7`, so this is forced, but then this leaves only one possible value for `q[7]` and `q[9]`, that is 2. Hence a constraint must be violated. We have detected unsatisfiability, and the solver must backtrack undoing the last decision `q[6] = 4` and adding its negation `q[6] != 4` (leading us to state (c) in the tree in Fig. 6.1) which forces `q[6] = 9`. This removes some values from the domain and then we again reinvoke the search strategy to decide what to do.

Many finite domain searches are defined in this way: choose a variable to constrain further, and then choose how to constrain it further.

2.6.2 Search Annotations

Search annotations in MiniZinc specify how to search in order to find a solution to the problem. The annotation is attached to the solve item, after the keyword `solve`. The search annotation

```
solve :: int_search(q, first_fail, indomain_min)
      satisfy;
```

appears on the solve item. Annotations are attached to parts of the model using the connector `::`. This search annotation means that we should search by selecting from the array of integer variables `q`, the variable with the smallest current domain (this is the `first_fail` rule), and try setting it to its smallest possible value (`indomain_min` value selection).

Basic search annotations

There are three basic search annotations corresponding to different basic variable types:

- `int_search( <variables>, <varchoice>, <constrainchoice> )` where `<variables>` is a one dimensional array of `var int`, `<varchoice>` is a variable choice annotation discussed below, `<constrainchoice>` is a choice of how to constrain a variable, discussed below.
- `bool_search( <variables>, <varchoice>, <constrainchoice> )` where `<variables>` is a one dimensional array of `var bool` and the rest are as above.
- `set_search( <variables>, <varchoice>, <constrainchoice> )` where `<variables>` is a one dimensional array of `var set of int` and the rest are as above.
- `float_search( <variables>, <precision>, <varchoice>, <constrainchoice> )` where `<variables>` is a one dimensional array of `var float`, `<precision>` is a fixed float specifying the ϵ below which two float values are considered equal, and the rest are as above.

Example variable choice annotations are:

- `input_order`: choose in order from the array,
- `first_fail`: choose the variable with the smallest domain size,
- `smallest`: choose the variable with the smallest value in its domain, and
- `dom_w_deg`: choose the variable with the smallest value of domain size divided by weighted degree, which is the number of times it has been in a constraint that caused failure earlier in the search.

Example ways to constrain a variable are:

- `indomain_min`: assign the variable its smallest domain value,
- `indomain_median`: assign the variable its median domain value (or the smaller of the two middle values in case of an even number of elements in the domain),
- `indomain_random`: assign the variable a random value from its domain, and
- `indomain_split` bisect the variables domain excluding the upper half.

For backwards compatibility with older version of MiniZinc, the search annotations can be called with an additional argument that represents the search strategy to use. The only such strategy that is currently supported is `complete`, meaning an exhaustive exploration of the search space. With the additional argument, an annotation might then look like this: `::int_search(x, input_order, indomain_min, complete)`.

For a complete list of variable and constraint choice annotations see the FlatZinc specification in the MiniZinc reference documentation.

We can construct more complex search strategies using search constructor annotations. There is only one such annotation at present:

```
seq_search([ <search-ann>, ..., <search-ann> ])
```

The sequential search constructor first undertakes the search given by the first annotation in its list, when all variables in this annotation are fixed it undertakes the second search annotation, etc. until all search annotations are complete.

Consider the jobshop scheduling model shown in Listing 3.9. We could replace the solve

item with

```
solve :: seq_search([
            int_search(s, smallest, indomain_min),
            int_search([end], input_order, indomain_min)])
      minimize end
```

which tries to set start times `s` by choosing the job that can start earliest and setting it to that time. When all start times are complete the end time `end` may not be fixed. Hence we set it to its minimal possible value.

2.6.3 Annotations

Annotations are a first class object in MiniZinc. We can declare new annotations in a model, and declare and assign to annotation variables.

Annotations

Annotations have a type `ann`. You can declare an annotation parameter (with optional assignment):

```
ann : <ident>;
ann : <ident> = <ann-expr> ;
```

and assign to an annotation variable just as any other parameter.
Expressions, variable declarations, and `solve` items can all be annotated using the `::` operator.
We can declare a new annotation using the `annotation` item:

```
annotation <annotation-name> ( <arg-def>, ..., <arg-def> ) ;
```

Listing 6.2: Annotated model for n-queens (`nqueens-ann.mzn`).

```
annotation bitdomain(int:nwords);

include "alldifferent.mzn";

int: n;
array [1..n] of var 1..n: q :: bitdomain(n div 32 + 1);

constraint alldifferent(q) :: domain_propagation;
constraint alldifferent([ q[i] + i | i in 1..n]) :: domain_propagation;
constraint alldifferent([ q[i] - i | i in 1..n]) :: domain_propagation;

ann: search_ann;

solve :: search_ann satisfy;
```

```
output [ if fix(q[j]) == i then "Q" else "." endif ++
         if j == n then "\n" else "" endif | i,j in 1..n];
```

The program in Listing 6.2 illustrates the use of annotation declarations, annotations and annotation variables. We declare a new annotation `bitdomain` which is meant to tell the solver that variables domains should be represented via bit arrays of size `nwords`. The annotation is attached to the declarations of the variables `q`. Each of the `alldifferent` constraints is annotated with the built in annotation `domain_propagation` which instructs the solver to use the domain propagating version of `alldifferent` if it has one. An annotation variable `search_ann` is declared and used to define the search strategy. We can give the value to the search strategy in a separate data file.

Example search annotations might be the following (where we imagine each line is in a separate data file)

```
search_ann = int_search(q, input_order, indomain_min);
search_ann = int_search(q, input_order, indomain_median);
search_ann = int_search(q, first_fail, indomain_min);
search_ann = int_search(q, first_fail, indomain_median);
search_ann = int_search(q, input_order, indomain_random);
```

The first just tries the queens in order, setting them to the minimum value, the second tries the queens variables in order, but sets them to their median value, the third tries the queen variable with smallest domain and sets it to the minimum value, the fourth strategy tries the queens variable with smallest domain, setting it to its median value, anbd the last strategy tries the queens in order, setting them to a random value in the domain.

Different search strategies can make a significant difference in how easy it is to find solutions. A small comparison of the number of failures made to find the first solution of the n-queens problems using the 5 different search strategies is shown in the table below (where — means more than 100,000 failures). Clearly the right search strategy can make a significant difference, and variables selection is more important than value selection, except that for this problem random value selection is very powerful.

n	input-min	input-median	ff-min	ff-median	input-random
10	22	2	5	0	6
15	191	4	4	12	39
20	20511	32	27	16	2
25	2212	345	51	25	2
30	—	137	22	66	9
35	—	1722	52	12	12
40	—	—	16	44	2
45	—	—	41	18	6

2.6.4 Restart

Any kind of depth first search for solving optimization problems suffers from the problem that wrong decisions made at the top of the search tree can take an exponential amount of search to undo. One common way to ameliorate this problem is to restart the search from the top thus having a chance to make different decisions.

MiniZinc includes annotations to control restart behaviour. These annotations, like other search annotations, are attached to the solve item of the model.

Restart search annotations

The different restart annotations control how frequently a restart occurs. Restarts occur when a limit in nodes is reached, where search returns to the top of the search tree and begins again. The possibilities are

- `restart_constant(<scale>)` where `<scale>` is an integer defining after how many nodes to restart.
- `restart_linear(<scale>)` where `<scale>` is an integer defining the initial number of nodes before the first restart. The second restart gets twice as many nodes, the third gets three times, etc.
- `restart_geometric(<base>,<scale>)` where `<base>` is a float and `<scale>` is an integer. The `k` th restart has a node limit of `<scale> * <base>^k`.
- `restart_luby(<scale>)` where `<scale>` is an integer. The `k` th restart gets `<scale>*L[k]` where :mzn`L[k]` is the `k` th number in the Luby sequence. The Luby sequence looks like 1 1 2 1 1 2 4 1 1 2 1 1 2 4 8 ..., that is it repeats two copies of the sequence ending in `2^i` before adding the number `2^{i+1}`.
- `restart_none` don't apply any restart (useful for setting a `ann` parameter that controls restart).

If two or more restart annotations are used, the solver's behaviour is undefined.

Restart search is much more robust in finding solutions, since it can avoid getting stuck in a non-productive area of the search. Note that restart search does not make much sense if the underlying search strategy does not do something different the next time it starts at the top. For example the search annotation

```
solve :: int_search(q, input_order, indomain_min);
      :: restart_linear(1000)
      satisfy
```

does not make very much sense since the underlying search is deterministic and each restart will just redo the same search as the previous search. Some solvers record the parts of the search tree that have already been searched and avoid them. This will mean deterministic restarts will simply effectively continue the search from the previous position. This gives no benefit to restarts, whose aim is to change decisions high in the search tree.

The simplest way to ensure that something is different in each restart is to use some randomization, either in variable choice or value choice. Alternatively some variable selection strategies make use of information gathered from earlier search and hence will give different behaviour, for example `dom_w_deg`.

2.6.5 Warm Starts

In many cases when solving an optimization or satisfaction problem we may have solved a previous version of the problem which is very similar. In this case it can be advantageous to use the previous solution found when searching for solution to the new problem. This is currently supported by some MIP backends.

The warm start annotations are attached to the solve item, just like other search annotations.

Warm start search annotations

Warm start annotations are used to indicate a (potentially partial or even invalid) starting point for solvers which support them.

- `warm_start(<vars>,<vals>)` where `<vars>` is a one dimensional array of integer variables, and `<vals>` is a one dimensional array of integers of the same length giving the warm start values for each integer variable in `<vars>`.
- `warm_start(<vars>,<vals>)` where `<vars>` is a one dimensional array of float variables, and `<vals>` is a one dimensional array of floats of the same length giving the warm start values for each float variable in `<vars>`.
- `warm_start(<vars>,<vals>)` where `<vars>` is a one dimensional array of Boolean variables, and `<vals>` is a one dimensional array of Booleans of the same length giving the warm start values for each Boolean variable in `<vars>`.
- `warm_start(<vars>,<vals>)` where `<vars>` is a one dimensional array of set variables, and `<vals>` is a one dimensional array of sets of integers of the same length giving the warm start values for each set variable in `<vars>`.

The `warm_start_array` annotation takes a list of `warm_start` annotations and can be used to specify an ordering of the warm starts which is maintained in the FlatZinc.

The warm start annotation can be used by the solver as part of value selection. For example, if the selected variable `v[i]` has in its current domain the warm start value `w[i]` then this is the value selected for the variable. If not the solver uses the existing value selection rule applicable to that variable. The order of warm_starts, relative to other search annotations, can be important (especially for CP), so they all might need to be put into a `seq_search` as below:

```
array[1..3] of var 0..10: x;
array[1..3] of var 0.0..10.5: xf;
var bool: b;
array[1..3] of var set of 5..9: xs;
constraint b+sum(x)==1;
constraint b+sum(xf)==2.4;
constraint 5==sum( [ card(xs[i]) | i in index_set(xs) ] );
solve
  :: warm_start_array( [                    %%% Can be on the upper level
    warm_start( x, [<>,8,4] ),              %%% Use <> for missing values
    warm_start( xf, array1d(-5..-3, [5.6,<>,4.7] ) ),
    warm_start( xs, array1d( -3..-2, [ 6..8, 5..7 ] ) )
  ] )
  :: seq_search( [
    warm_start_array( [                     %%% Now included in seq_search␣
```

```
↪to keep order
      warm_start( x, [<>,5,2] ),              %%% Repeated warm_starts␣
↪allowed but not specified
      warm_start( xf, array1d(-5..-3, [5.6,<>,4.7] ) ),
      warm_start( xs, array1d( -3..-2, [ 6..8, 5..7 ] ) )
    ] ),
    warm_start( [b], [true] ),
    int_search(x, first_fail, indomain_min)
  ] )
  minimize x[1] + b + xf[2] + card( xs[1] intersect xs[3] );
```

If you'd like to provide a most complete warm start information, please provide values for all variables which are output when there is no output item or when compiled with `--output-mode dzn`.

CHAPTER 2.7

Effective Modelling Practices in MiniZinc

There are almost always multiple ways to model the same problem, some of which generate models which are efficient to solve, and some of which are not. In general it is very hard to tell a priori which models are the most efficient for solving a particular problem, and indeed it may critically depend on the underlying solver used, and search strategy. In this chapter we concentrate on modelling practices that avoid inefficiency in generating models and generated models. For solver-specific recommendations see User Manual/Solver Backends.

2.7.1 Variable Bounds

Finite domain propagation engines, which are the principle type of solver targeted by MiniZinc, are more effective the tighter the bounds on the variables involved. They can also behave badly with problems which have subexpressions that take large integer values, since they may implicitly limit the size of integer variables.

Listing 7.1: A model with unbounded variables (`grocery.mzn`).

```
var int: item1;
var int: item2;
var int: item3;
var int: item4;

constraint item1 + item2 + item3 + item4 == 711;
constraint item1 * item2 * item3 * item4 == 711 * 100 * 100 * 100;

constraint         0 < item1 /\ item1 <= item2
           /\ item2 <= item3 /\ item3 <= item4;

solve satisfy;
```

```
output ["{", show(item1), ",", show(item2), ",", show(item3), ",",
        show(item4),"}\n"];
```

The grocery problem shown in Listing 7.1 finds 4 items whose prices in dollars add up to 7.11 and multiply up to 7.11. The variables are declared unbounded. Running

```
$ minizinc --solver chuffed grocery.mzn
```

does not yield a solution because the intermediate expressions in the multiplication are also `var int` and are given default bounds in the solver $-1,000,000 \ldots 1,000,000$, and these ranges are too small to hold the values that the intermediate expressions may need to take.

Modifying the model so that the items are declared with tight bounds

```
var 1..711: item1;
var 1..711: item2;
var 1..711: item3;
var 1..711: item4;
```

results in a better model, since now MiniZinc can infer bounds on the intermediate expressions and use these rather than the default bounds. With this modification, executing the model gives

```
{120,125,150,316}
----------
```

Note however that even the improved model may be too difficult for some solvers.

Bounding variables

Always try to use bounded variables in models. When using `let` declarations to introduce new variables, always try to define them with correct and tight bounds. This will make your model more efficient, and avoid the possibility of unexpected overflows. One exception is when you introduce a new variable which is immediately defined as equal to an expression. Usually MiniZinc will be able to infer effective bounds from the expression.

2.7.2 Effective Generators

Imagine we want to count the number of triangles (K_3 subgraphs) appearing in a graph. Suppose the graph is defined by an adjacency matrix: `adj[i,j]` is true if nodes `i` and `j` are adjacent. We might write

```
int: count = sum ([ 1 | i,j,k in NODES where i < j  /\ j < k
                      /\ adj[i,j] /\ adj[i,k] /\ adj[j,k]]);
```

which is certainly correct, but it examines all triples of nodes. If the graph is sparse we can do better by realising that some tests can be applied as soon as we select `i` and `j`.

```
int: count = sum ([ 1 | i,j in NODES where i < j  /\ adj[i,j],
                        k in NODES where j < k /\ adj[i,k] /\ adj[j,k]]);
```

You can use the builtin `trace` function to help determine what is happening inside generators.

Tracing

The function `trace(s,e)` prints the string `s` before evaluating the expression `e` and returning its value. It can be used in any context.

For example, we can see how many times the test is performed in the inner loop for both versions of the calculation.

```
int:count=sum([ 1 | i,j,k in NODES where
          trace("+", i<j /\j<k /\ adj[i,j] /\ adj[i,k] /\ adj[j,k]) ]);
adj = [| false, true,  true,  false
       | true,  false, true,  false
       | true,  true,  false, true
       | false, false, true,  false |];
constraint trace("\n",true);
solve satisfy;
```

Produces the output:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
----------
```

indicating the inner loop is evaluated 64 times while

```
  int: count = sum ([ 1 | i,j in NODES where i < j  /\ adj[i,j],
                          k in NODES where trace("+", j < k /\ adj[i,k] /\␣
↪adj[j,k])]);
```

Produces the output:

```
++++++++++++++++
----------
```

indicating the inner loop is evaluated 16 times.

Note that you can use the dependent strings in `trace` to understand what is happening during model creation.

```
int: count = sum( i,j in NODES where i < j /\ adj[i,j])(
       sum([trace("("++show(i)++","++show(j)++","++show(k)++")",1) |
            k in NODES where  j < k /\ adj[i,k] /\ adj[j,k]]));
```

will print out each of triangles that is found in the calculation. It produces the output

```
(1,2,3)
----------
```

We have to admit that we cheated a bit here: In certain circumstances, the MiniZinc compiler is in fact able to re-order the arguments in a `where` clause automatically, so that they are evaluated as early as possible. In this case, adding the `trace` function in fact *prevented* this optimisation. In general, it is however a good idea to help the compiler get it right, by splitting the `where` clauses and placing them as close to the generators as possible.

2.7.3 Redundant Constraints

The form of a model will affect how well the constraint solver can solve it. In many cases adding constraints which are redundant, i.e. are logically implied by the existing model, may improve the search for solutions by making more information available to the solver earlier.

Consider the magic series problem from *Complex Constraints* (page 54). Running this for `n = 16` as follows:

```
$ minizinc --solver gecode --all-solutions --statistics magic-series.mzn -D
↪"n=16;"
```

might result in output

```
s = [12, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0];
----------
==========
```

and the statistics showing 117 failures required.

We can add redundant constraints to the model to improve the performance. Since each number in the sequence counts the number of occurrences of a number, we know that they sum up to `n`. Similarly we know that the sum of `s[i] * i` must also add up to `n` because the sequence is magic.

These constraints do not actually constrain the model further: any solution of the original model is still a solution of the new model that includes the redundant constraints. It is good practice to make the MiniZinc compiler aware that a constraint is redundant. That way, the compiler can decide to ignore the constraint for particular solvers, since not all solvers benefit from redundant constraints. We therefore wrap redundant constraints in a call to the library predicate `redundant_constraint`.

The magic sequence model with these redundant constraints is shown in Listing 7.2.

Listing 7.2: Model solving the magic series problem with redundant constraints (magic-series2.mzn).

```
int: n;
array[0..n-1] of var 0..n: s;

constraint forall(i in 0..n-1) (
   s[i] = (sum(j in 0..n-1)(bool2int(s[j]=i))));

constraint redundant_constraint(sum(i in 0..n-1)(s[i]) = n);
constraint redundant_constraint(sum(i in 0..n-1)(s[i] * i) = n);

solve satisfy;

output [ "s = ", show(s), ";\n" ] ;
```

Running the same problem as before

```
$ minizinc --solver gecode --all-solutions --statistics magic-series2.mzn -D
↪"n=16;"
```

results in the same output, but with statistics showing just 14 failures explored. The redundant constraints have allowed the solver to prune the search much earlier.

Because we wrapped the redundant constraints in the special predicate, we can now switch them off easily on the command line by setting the flag `mzn_ignore_redundant_constraints=true`:

```
$ minizinc --solver gecode --all-solutions --statistics magic-series2.mzn -D
↪"n=16;mzn_ignore_redundant_constraints=true;"
```

2.7.4 Modelling Choices

There are many ways to model the same problem in MiniZinc, although some may be more natural than others. Different models may have very different efficiency of solving, and worse yet, different models may be better or worse for different solving backends. There are however some guidelines for usually producing better models:

Choosing between models

The better model is likely to have some of the following features
- smaller number of variables, or at least those that are not functionally defined by other variables
- smaller domain sizes of variables
- more succinct, or direct, definition of the constraints of the model
- uses global constraints as much as possible

In reality all this has to be tempered by how effective the search is for the model. Usually the effectiveness of search is hard to judge except by experimentation.

Consider the problem of finding permutations of n numbers from 1 to n such that the differences between adjacent numbers also form a permutation of numbers 1 to $n-1$. Note that the `u` variables are functionally defined by the `x` variables so the raw search space is n^n. The obvious way to model this problem is shown in Listing 7.3.

Listing 7.3: A natural model for the all interval series problem `prob007` in CSPlib (`allinterval.mzn`).

```
include "alldifferent.mzn";

int: n;

array[1..n] of var 1..n: x;       % sequence of numbers
array[1..n-1] of var 1..n-1: u;   % sequence of differences

constraint alldifferent(x);
constraint alldifferent(u);
constraint forall(i in 1..n-1)(u[i] = abs(x[i+1] - x[i]));

solve :: int_search(x, first_fail, indomain_min)
      satisfy;
output ["x = \(x);\n"];
```

In this model the array `x` represents the permutation of the `n` numbers and the constraints are naturally represented using `alldifferent`.

Running the model

```
$ minizinc --solver gecode --all-solutions --statistics allinterval.mzn -D
↪"n=10;"
```

finds all solutions in 16077 nodes and 71ms

An alternate model uses array `y` where `y[i]` gives the position of the number `i` in the sequence. We also model the positions of the differences using variables `v`. `v[i]` is the position in the sequence where the absolute difference `i` occurs. If the values of `y[i]` and `y[j]` differ by one where `j > i`, meaning the positions are adjacent, then `v[j-i]` is constrained to be the earliest of these positions. We can add two redundant constraints to this model: since we know that a difference of `n-1` must result, we know that the positions of 1 and `n` must be adjacent (`abs( y[1] - y[n] ) = 1`), which also tell us that the

position of difference `n-1` is the earlier of `y[1]` and `y[n]`, i.e. `v[n-1] = min(y[1], y[n])`. With this we can model the problem as shown in Listing 7.4. The output statement recreates the original sequence `x` from the array of positions `y`.

Listing 7.4: An inverse model for the all interval series problem `prob007` in CSPlib (`allinterval2.mzn`).

```
include "alldifferent.mzn";

int: n;

array[1..n] of var 1..n: y;  % position of each number
array[1..n-1] of var 1..n-1: v; % position of difference i

constraint alldifferent(y);
constraint alldifferent(v);
constraint forall(i,j in 1..n where i < j)(
                    (y[i] - y[j] = 1 -> v[j-i] = y[j]) /\
                  (y[j] - y[i] = 1 -> v[j-i] = y[i])
           );

constraint abs(y[1] - y[n]) = 1 /\ v[n-1] = min(y[1], y[n]);

solve :: int_search(y, first_fail, indomain_min)
      satisfy;

array[1..n] of 1..n: x :: output_only
  = [ sum(i in 1..n)(i*(fix(y[j]) = i)) | j in 1..n ];
output [ "x = \(x);\n" ]
```

The inverse model has the same size as the original model, in terms of number of variables and domain sizes. But the inverse model has a much more indirect way of modelling the relationship between the `y` and `v` variables as opposed to the relationship between `x` and `u` variables. Hence we might expect the original model to be better.

The command

```
$ minizinc --solver gecode --all-solutions --statistics allinterval2.mzn -D
↪"n=10;"
```

finds all the solutions in 98343 nodes and 640 ms. So the more direct modelling of the constraints is clearly paying off.

Note that the inverse model prints out the answers using the same `x` view of the solution. The way this is managed is using `output_only` annotations. The array `x` is defined as a fixed array and annotated as `output_only`. This means it will only be evaluated, and can only be used in output statements. Once a solution for `y` is discovered the value of `x` is calculated during output processing, and hence can be displayed in the output.

Output_only annotation

The `output_only` annotation can be applied to variable definitions. The variable defined must not be a `var` type, it can only be `par`. The variable must also have a right hand side definition giving its value. This right hand side definition can make use of `fix` functions to access the values of decision variables, since it is evaluated at solution processing time

2.7.5 Multiple Modelling and Channels

When we have two models for the same problem it may be useful to use both models together by tying the variables in the two models together, since each can give different information to the solver.

Listing 7.5: A dual model for the all interval series problem `prob007` in CSPlib (`allinterval3.mzn`).

```
include "inverse.mzn";

int: n;

array[1..n] of var 1..n: x;  % sequence of numbers
array[1..n-1] of var 1..n-1: u;  % sequence of differences

constraint forall(i in 1..n-1)(u[i] = abs(x[i+1] - x[i]));

array[1..n] of var 1..n: y;  % position of each number
array[1..n-1] of var 1..n-1: v; % position of difference i

constraint inverse(x,y);
constraint inverse(u,v);

constraint abs(y[1] - y[n]) = 1 /\ v[n-1] = min(y[1], y[n]);

solve :: int_search(y, first_fail, indomain_min)
      satisfy;

output ["x = ",show(x),"\n"];
```

Listing 7.5 gives a dual model combining features of `allinterval.mzn` and `allinterval2.mzn`. The beginning of the model is taken from `allinterval.mzn`. We then introduce the `y` and `v` variables from `allinterval2.mzn`. We tie the variables together using the global `inverse` constraint: `inverse(x,y)` holds if `y` is the inverse function of `x` (and vice versa), that is `x[i] = j <-> y[j] = i`. A definition is shown in Listing 7.6. The model does not include the constraints relating the `y` and `v` variables, they are redundant (and indeed propagation redundant) so they do not add information for a propagation solver. The `alldifferent` constraints are also missing since they are made redundant (and propagation redundant) by the inverse constraints. The only constraints are the relationships of the `x` and `u` variables and the redundant constraints on `y` and `v`.

Listing 7.6: A definition of the `inverse` global constraint (`inverse.mzn`).

```
predicate inverse(array[int] of var int: f,
                  array[int] of var int: invf) =
    forall(j in index_set(invf))(invf[j] in index_set(f)) /\
    forall(i in index_set(f))(
        f[i] in index_set(invf) /\
        forall(j in index_set(invf))(j == f[i] <-> i == invf[j])
    );
```

One of the benefits of the dual model is that there is more scope for defining different search strategies. Running the dual model,

```
$ minizinc --solver gecode --all-solutions --statistics allinterval3.mzn -D
↪"n=10;"
```

which uses the search strategy of the inverse model, labelling the `y` variables, finds all solutions in 126031 nodes and 250ms. Note that running the same model with labelling on the `x` variables requires 35245 nodes and 100ms.

2.7.6 Symmetry

Symmetry is very common in constraint satisfaction and optimisation problems. To illustrate this, let us look again at the n-queens problem from Listing 6.1. The top left chess board in Fig. 7.1 shows a solution to the 8-queens problems (labeled "original"). The remaining chess boards show seven symmetric variants of the same solution: rotated by 90, 180 and 270 degrees, and flipped vertically.

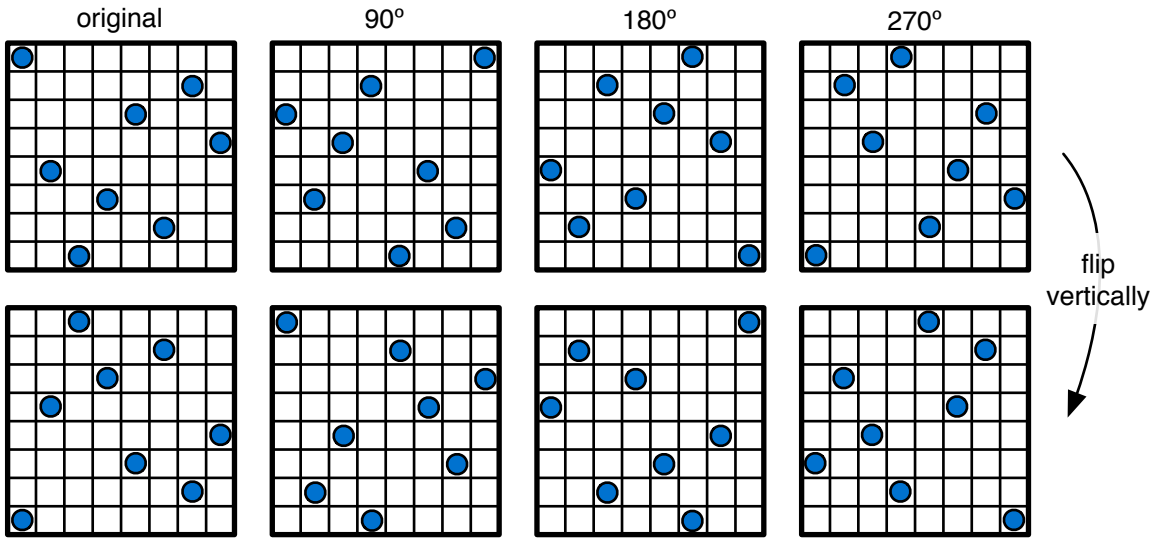


Fig. 7.1: Symmetric variants of an 8-queens solution

If we wanted to enumerate *all* solutions to the 8-queens problem, we could obviously save the solver some work by only enumerating *non-symmetric* solutions, and then generating the symmetric variants ourselves. This is one reason why we want to get rid

of symmetry in constraint models. The other, much more important reason, is that the solver may also **explore symmetric variants of non-solution states!**

For example, a typical constraint solver may try to place the queen in column 1 into row 1 (which is fine), and then try to put the column 2 queen into row 3, which, at first sight, does not violate any of the constraints. However, this configuration cannot be completed to a full solution (which the solver finds out after a little search). Fig. 7.2 shows this configuration on the top left chess board. Now nothing prevents the solver from trying, e.g., the second configuration from the left in the bottom row of Fig. 7.2, where the queen in column 1 is still in row 1, and the queen in column 3 is placed in row 2. Therefore, even when only searching for a single solution, the solver may explore many symmetric states that it has already seen and proven unsatisfiable before!

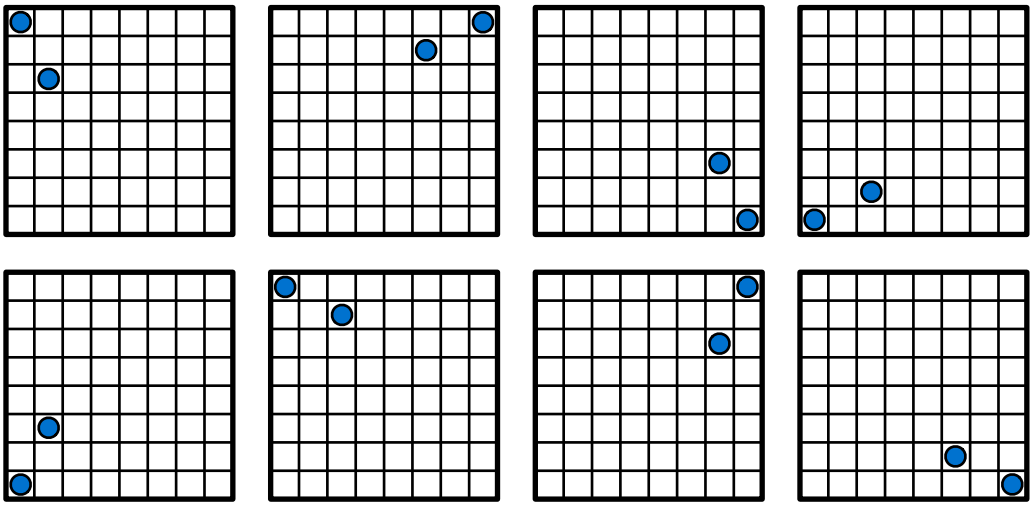


Fig. 7.2: Symmetric variants of an 8-queens unsatisfiable partial assignment

2.7.6.1 Static Symmetry Breaking

The modelling technique for dealing with symmetry is called *symmetry breaking*, and in its simplest form, involves adding constraints to the model that rule out all symmetric variants of a (partial) assignment to the variables except one. These constraints are called *static symmetry breaking constraints*.

The basic idea behind symmetry breaking is to impose an *order*. For example, to rule out any vertical flips of the chess board, we could simply add the constraint that the queen in the first column must be in the top half of the board:

```
constraint q[1] <= n div 2;
```

Convince yourself that this would remove exactly half of the symmetric variants in Fig. 7.1. You can tell the compiler that a particular constraint is a symmetry breaking constraint by wrapping it in a special call:

```
constraint symmetry_breaking_constraint(q[1] <= n div 2);
```

This has the advantage that solvers can ignore the constraint or treat it differently. It also means that you can easily switch off symmetry breaking by setting

`mzn_ignore_symmetry_breaking_constraints=true`, either in your model, or a data file, or as extra data using the `-D` command line option.

In order to remove *all* symmetry in the n-queens problem, we need to work a bit harder. Whenever we can express all symmetries as permutations of the array of variables, a set of *lexicographic ordering constraints* can be used to break all symmetry. For example, if the array of variables is called `x`, and reversing the array is a symmetry of the problem, then the following constraint will break that symmetry:

```
constraint symmetry_breaking_constraint(lex_lesseq(x, reverse(x)));
```

How about two-dimensional arrays? Lexicographic ordering works just the same, we only have to coerce the arrays into one dimension. For example, the following breaks the symmetry of flipping the array along one of the diagonals (note the swapped indices in the second comprehension):

```
array[1..n,1..n] of var int: x;
constraint symmetry_breaking_constraint(
  lex_lesseq([ x[i,j] | i,j in 1..n ], [ x[j,i] | i,j in 1..n ])
);
```

The great thing about using lexicographic ordering constraints is that we can add multiple ones (to break several symmetries simultaneously), without them interfering with each other, as long as we keep the order in the first argument the same.

For the n-queens problem, unfortunately this technique does not immediately apply, because some of its symmetries cannot be described as permutations of the `q` array. The trick to overcome this is to express the n-queens problem in terms of Boolean variables that model, for each field of the board, whether it contains a queen or not. Now all the symmetries can be modeled as permutations of this array. Since the main constraints of the n-queens problem are much easier to express with the integer `q` array, we simply use both models together and add channeling constraints between them, as explained in *Multiple Modelling and Channels* (page 116).

The full model, with added Boolean variables, channeling constraints and symmetry breaking constraints is shown in Listing 7.7. We can conduct a little experiment to check whether it successfully breaks all the symmetry. Try running the model with increasing values for `n`, e.g. from 1 to 10, counting the number of solutions (e.g., by using the `-s` flag with the Gecode solver, or selecting "Print all solutions" as well as "Statistics for solving" in the IDE). You should get the following sequence of numbers of solutions: 1, 0, 0, 1, 2, 1, 6, 12, 46, 92. To verify the sequence, you can search for it in the *On-Line Encyclopedia of Integer Sequences* (http://oeis.org).

Listing 7.7: Partial model for n-queens with symmetry breaking (full model: `nqueens_sym.mzn`).

```
constraint symmetry_breaking_constraint(
let {
  % Alternative Boolean model:
  % Map each position i,j to a Boolean telling us whether there is a queen␣
↪at i,j
```

```
  array[1..n,1..n] of var bool: qb;
} in
    % Channeling constraint
    forall (i,j in 1..n) ( qb[i,j] <-> (q[i]=j) )
    % Lexicographic symmetry breaking constraints
/\  lex_lesseq(array1d(qb), [ qb[j,i] | i,j in 1..n ])
/\  lex_lesseq(array1d(qb), [ qb[i,j] | i in reverse(1..n), j in 1..n ])
/\  lex_lesseq(array1d(qb), [ qb[j,i] | i in 1..n, j in reverse(1..n) ])
/\  lex_lesseq(array1d(qb), [ qb[i,j] | i in 1..n, j in reverse(1..n) ])
/\  lex_lesseq(array1d(qb), [ qb[j,i] | i in reverse(1..n), j in 1..n ])
/\  lex_lesseq(array1d(qb), [ qb[i,j] | i,j in reverse(1..n) ])
/\  lex_lesseq(array1d(qb), [ qb[j,i] | i,j in reverse(1..n) ])
);
```

2.7.6.2 Other Examples of Symmetry

Many other problems have inherent symmetries, and breaking these can often make a significant difference in solving performance. Here is a list of some common cases:

- Bin packing: when trying to pack items into bins, any two bins that have the same capacity are symmetric.
- Graph colouring: When trying to assign colours to nodes in a graph such that adjacent nodes must have different colours, we typically model colours as integer numbers. However, any permutation of colours is again a valid graph colouring.
- Vehicle routing: if the task is to assign customers to certain vehicles, any two vehicles with the same capacity may be symmetric (this is similar to the bin packing example).
- Rostering/time tabling: two staff members with the same skill set may be interchangeable, just like two rooms with the same capacity or technical equipment.

CHAPTER 2.8

Boolean Satisfiability Modelling in MiniZinc

MiniZinc can be used to model Boolean satisfiability problems where the variables are restricted to be Boolean (`bool`). MiniZinc can be used with efficient Boolean satisfiability solvers to solve the resulting models efficiently.

2.8.1 Modelling Integers

Many times although we wish to use a Boolean satisfiability solver we may need to model some integer parts of our problem.

There are three common ways of modelling an integer variables I in the range $0 \ldots m$ where $m = 2^k - 1$ using Boolean variables.

- Binary: I is represented by k binary variables $i_0, \ldots, i_{k-1}$ where $I = 2^{k-1}i_{k-1} + 2^{k-2}i_{k-2} + \cdots + 2i_1 + i_0$. This can be represented in MiniZinc as

```
array[0..k-1]  of var bool: i;
var 0..pow(2,k)-1: I = sum(j in 0..k-1)(bool2int(i[j])*pow(2,j));
```

- Unary: where I is represented by m binary variables $i_1, \ldots, i_m$ and $i = \sum_{j=1}^{m} \texttt{bool2int}(i_j)$. Since there is massive redundancy in the unary representation we usually require that $i_j \rightarrow i_{j-1}, 1 < j \leq m$. This can be represented in MiniZinc as

```
array[1..m]  of var bool: i;
constraint forall(j in 2..m)(i[j] -> i[j-1]);
var 0..m: I = sum(j in 1..m)(bool2int(i[j]));
```

- Value: where I is represented by $m+1$ binary variables $i_0, \ldots, i_m$ where $I = k \Leftrightarrow i_k$, and at most one of $i_0, \ldots, i_m$ is true. This can be represented in MiniZinc as

```
array[0..m]  of var bool: i;
constraint sum(j in 0..m)(bool2int(i[j]) == 1);
var 0..m: I;
constraint forall(j in 0..m)(I == j <-> i[j]);
```

There are advantages and disadvantages to each representation. It depends on what operations on integers are to required in the model as to which is preferable.

2.8.2 Modelling Disequality

Let us consider modelling a latin squares problem. A latin square is an $n \times n$ grid of numbers from $1..n$ such that each number appears exactly once in every row and column. An integer model for latin squares is shown in Listing 8.1.

Listing 8.1: Integer model for Latin Squares (`latin.mzn`).

```
int: n; % size of latin square
array[1..n,1..n] of var 1..n: a;

include "alldifferent.mzn";
constraint forall(i in 1..n)(
             alldifferent(j in 1..n)(a[i,j]) /\
             alldifferent(j in 1..n)(a[j,i])
           );
solve satisfy;
output [ show(a[i,j]) ++ if j == n then "\n" else " " endif |
         i in 1..n, j in 1..n ];
```

The only constraint on the integers is in fact disequality, which is encoded in the `alldifferent` constraint. The value representation is the best way of representing disequality. A Boolean only model for latin squares is shown in Listing 8.2. Note each integer array element `a[i,j]` is replaced by an array of Booleans. We use the `exactlyone` predicate to encode that each value is used exactly once in every row and every column, as well as to encode that exactly one of the Booleans corresponding to integer array element `a[i,j]` is true.

Listing 8.2: Boolean model for Latin Squares (`latinbool.mzn`).

```
int: n; % size of latin square
array[1..n,1..n,1..n] of var bool: a;

predicate atmostone(array[int] of var bool:x) =
          forall(i,j in index_set(x) where i < j)(
            (not x[i] \/ not x[j]));
predicate exactlyone(array[int] of var bool:x) =
          atmostone(x) /\ exists(x);

constraint forall(i,j in 1..n)(
             exactlyone(k in 1..n)(a[i,j,k]) /\
```

```
            exactlyone(k in 1..n)(a[i,k,j]) /\
            exactlyone(k in 1..n)(a[k,i,j])
          );
solve satisfy;
output [ if fix(a[i,j,k]) then
           show(k) ++ if j == n then "\n" else " " endif
         else "" endif | i,j,k in 1..n ];
```

2.8.3 Modelling Cardinality

Let us consider modelling the Light Up puzzle. The puzzle consists of a rectangular grid of squares which are blank, or filled. Every filled square may contain a number from 1 to 4, or may have no number. The aim is to place lights in the blank squares so that

- Each blank square is "illuminated", that is can see a light through an uninterrupted line of blank squares
- No two lights can see each other
- The number of lights adjacent to a numbered filled square is exactly the number in the filled square.

An example of a Light Up puzzle is shown in Fig. 8.1 with its solution in Fig. 8.2.

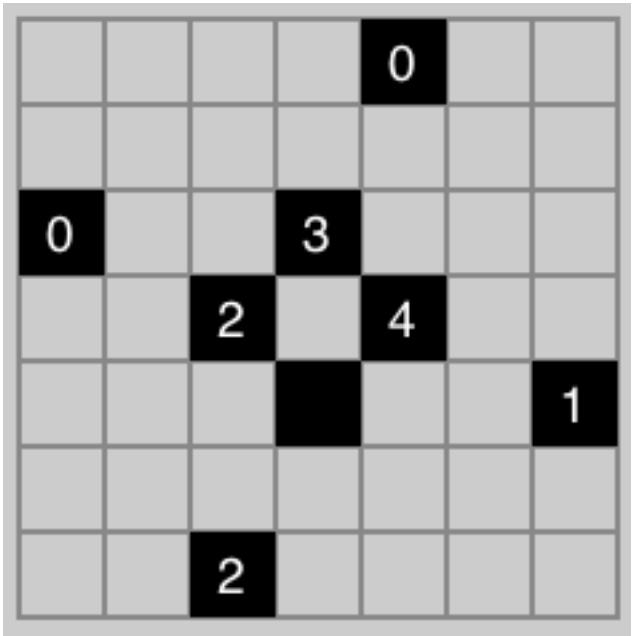


Fig. 8.1: An example of a Light Up puzzle

It is natural to model this problem using Boolean variables to determine which squares contain a light and which do not, but there is some integer arithmetic to consider for the filled squares.

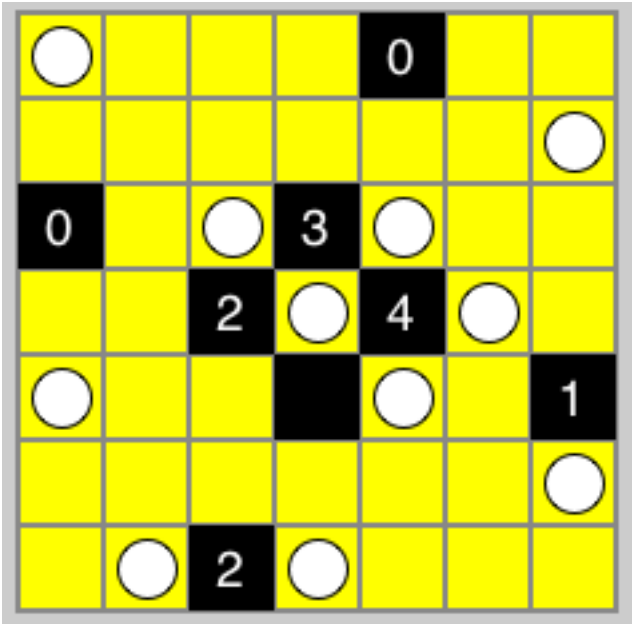


Fig. 8.2: The completed solution of the Light Up puzzle

Listing 8.3: SAT Model for the Light Up puzzle (`lightup.mzn`).

```
int: h; set of int: H = 1..h; % board height
int: w; set of int: W = 1..w; % board width
array[H,W] of -1..5: b;       % board
int: E = -1;                  % empty square
set of int: N = 0..4;         % filled and numbered square
int: F =  5;                  % filled unnumbered square

% position (i1,j1) is visible to (i2,j2)
test visible(int: i1, int: j1, int: i2, int: j2) =
    ((i1 == i2) /\ forall(j in min(j1,j2)..max(j1,j2))(b[i1,j] == E))
 \/ ((j1 == j2) /\ forall(i in min(i1,i2)..max(i1,i2))(b[i,j1] == E));

array[H,W] of var bool: l; % is there a light

% filled squares have no lights
constraint forall(i in H, j in W where b[i,j] != E)(l[i,j] == false);
% lights next to filled numbered square agree

constraint forall(i in H, j in W where b[i,j] in N)(
    bool_sum_eq([ l[i1,j1] | i1 in i-1..i+1, j1 in j-1..j+1 where
                             abs(i1 - i) + abs(j1 - j) == 1 /\
                             i1 in H /\ j1 in W ], b[i,j]));
% each blank square is illuminated
constraint forall(i in H, j in W where b[i,j] == E)(
             exists(j1 in W where visible(i,j,i,j1))(l[i,j1]) \/
             exists(i1 in H where visible(i,j,i1,j))(l[i1,j])
           );
```

```
% no two lights see each other
constraint forall(i1,i2 in H, j1,j2 in W where
                  (i1 != i2 \/ j1 != j2) /\ b[i1,j1] == E
                  /\ b[i2,j2] == E /\ visible(i1,j1,i2,j2))(
             not l[i1,j1] \/ not l[i2,j2]
           );

solve satisfy;
output [ if b[i,j] != E then show(b[i,j])
         else if fix(l[i,j]) then "L" else "." endif
         endif ++ if j == w then "\n" else " " endif |
         i in H, j in W];
```

A model for the problem is given in Listing 8.3. A data file for the problem shown in Fig. 8.1 is shown in Listing 8.4.

Listing 8.4: Datafile for the Light Up puzzle instance shown in Fig. 8.1 (`lightup.dzn`).

```
h = 7;
w = 7;
b = [| -1,-1,-1,-1, 0,-1,-1
     | -1,-1,-1,-1,-1,-1,-1
     |  0,-1,-1, 3,-1,-1,-1
     | -1,-1, 2,-1, 4,-1,-1
     | -1,-1,-1, 5,-1,-1, 1
     | -1,-1,-1,-1,-1,-1,-1
     |  1,-1, 2,-1,-1,-1,-1 |];
```

The model makes use of a Boolean sum predicate

```
predicate bool_sum_eq(array[int] of var bool:x, int:s);
```

which requires that the sum of an array of Boolean equals some fixed integer. There are a number of ways of modelling such *cardinality* constraints using Booleans.

- Adder networks: we can use a network of adders to build a binary Boolean representation of the sum of the Booleans
- Sorting networks: we can use a sorting network to sort the array of Booleans to create a unary representation of the sum of the Booleans
- Binary decision diagrams: we can create a binary decision diagram (BDD) that encodes the cardinality constraint.

Listing 8.5: Cardinality constraints by binary adder networks (`bboolsum.mzn`).

```
% the sum of booleans x = s
predicate bool_sum_eq(array[int] of var bool:x, int:s) =
     let { int: c = length(x) } in
     if s < 0 then false
```

```
    elseif s == 0 then
        forall(i in 1..c)(x[i] == false)
    elseif s < c then
        let { % cp = number of bits required for representing 0..c
              int: cp = floor(log2(int2float(c))),
              % z is sum of x in binary
              array[0..cp] of var bool:z  } in
        binary_sum(x, z) /\
        % z == s
        forall(i in 0..cp)(z[i] == ((s div pow(2,i)) mod 2 == 1))
    elseif s == c then
        forall(i in 1..c)(x[i] == true)
    else false endif;

include "binarysum.mzn";
```

Listing 8.6: Code for building binary addition networks (`binarysum.mzn`).

```
% the sum of bits x = s in binary.
%           s[0], s[1], ..., s[k] where 2^k >= length(x) > 2^(k-1)
predicate binary_sum(array[int] of var bool:x,
                     array[int] of var bool:s)=
    let { int: l = length(x) } in
    if l == 1 then s[0] = x[1]
    elseif l == 2 then
           s[0] = (x[1] xor x[2]) /\ s[1] = (x[1] /\ x[2])
    else let { int: ll = (l div 2),
               array[1..ll] of var bool: f = [ x[i] | i in 1..ll ],
               array[1..ll] of var bool: t = [x[i]| i in ll+1..2*ll],
               var bool: b = if ll*2 == l then false else x[l] endif,
               int: cp = floor(log2(int2float(ll))),
               array[0..cp] of var bool: fs,
               array[0..cp] of var bool: ts } in
               binary_sum(f, fs) /\ binary_sum(t, ts) /\
               binary_add(fs, ts, b, s)
    endif;

% add two binary numbers x, and y and carry in bit ci to get binary s
predicate binary_add(array[int] of var bool: x,
                     array[int] of var bool: y,
                     var bool: ci,
                     array[int] of var bool: s) =
    let { int:l = length(x),
          int:n = length(s), } in
    assert(l == length(y),
          "length of binary_add input args must be same",
    assert(n == l \/ n == l+1, "length of binary_add output " ++
                        "must be equal or one more than inputs",
    let { array[0..l] of var bool: c } in
    full_adder(x[0], y[0], ci, s[0], c[0]) /\
```

```
    forall(i in 1..l)(full_adder(x[i], y[i], c[i-1], s[i], c[i])) /\
    if n > l then s[n] = c[l] else c[l] == false endif ));

predicate full_adder(var bool: x, var bool: y, var bool: ci,
                     var bool: s, var bool: co) =
          let { var bool: xy = x xor y } in
          s = (xy xor ci) /\ co = ((x /\ y) \/ (ci /\ xy));
```

We can implement `bool_sum_eq` using binary adder networks using the code shown in Listing 8.5. The predicate `binary_sum` defined in Listing 8.6 creates a binary representation of the sum of `x` by splitting the list into two, summing up each half to create a binary representation and then summing these two binary numbers using `binary_add`. If the list `x` is odd the last bit is saved to use as a carry in to the binary addition.

Listing 8.7: Cardinality constraints by sorting networks (`uboolsum.mzn`).

```
% the sum of booleans x = s
predicate bool_sum_eq(array[int] of var bool:x, int:s) =
     let { int: c = length(x) } in
     if s < 0 then false
     elseif s == 0 then forall(i in 1..c)(x[i] == false)
     elseif s < c then
          let { % cp = nearest power of 2 >= c
                int: cp = pow(2,ceil(log2(int2float(c)))),
                array[1..cp] of var bool:y, % y is padded version of x
                array[1..cp] of var bool:z  } in
          forall(i in 1..c)(y[i] == x[i]) /\
          forall(i in c+1..cp)(y[i] == false) /\
          oesort(y, z) /\ z[s] == true /\ z[s+1] == false
     elseif s == c then forall(i in 1..c)(x[i] == true)
     else false endif;

include "oesort.mzn";
```

Listing 8.8: Odd-even merge sorting networks (`oesort.mzn`).

```
%% odd-even sort
%% y is the sorted version of x, all trues before falses
predicate oesort(array[int] of var bool:x, array[int] of var bool:y)=
  let { int: c = card(index_set(x)) } in
  if c == 1 then x[1] == y[1]
  elseif c == 2 then comparator(x[1],x[2],y[1],y[2])
  else
    let {
      array[1..c div 2] of var bool:xf = [x[i] | i in 1..c div 2],
      array[1..c div 2] of var bool:xl = [x[i] | i in c div 2 +1..c],
      array[1..c div 2] of var bool:tf,
      array[1..c div 2] of var bool:tl } in
    oesort(xf,tf) /\ oesort(xl,tl) /\ oemerge(tf ++ tl, y)
  endif;
```

```
%% odd-even merge
%% y is the sorted version of x, all trues before falses
%% assumes first half of x is sorted, and second half of x
predicate oemerge(array[int] of var bool:x, array[int] of var bool:y)=
  let { int: c = card(index_set(x)) } in
  if c == 1 then x[1] == y[1]
  elseif c == 2 then comparator(x[1],x[2],y[1],y[2])
  else
    let { array[1..c div 2] of var bool:xo =
            [ x[i] | i in 1..c where i mod 2 == 1],
          array[1..c div 2] of var bool:xe =
            [ x[i] | i in 1..c where i mod 2 == 0],
          array[1..c div 2] of var bool:to,
          array[1..c div 2] of var bool:te } in
      oemerge(xo,to) /\ oemerge(xe,te) /\
      y[1] = to[1] /\
      forall(i in 1..c div 2 -1)(
        comparator(te[i],to[i+1],y[2*i],y[2*i+1])) /\
      y[c] = te[c div 2]
  endif;

% comparator o1 = max(i1,i2), o2 = min(i1,i2)
predicate comparator(var bool:i1,var bool:i2,var bool:o1,var bool:o2)=
          (o1 = (i1 \/ i2)) /\ (o2 = (i1 /\ i2));
```

We can implement `bool_sum_eq` using unary sorting networks using the code shown in Listing 8.7. The cardinality constraint is defined by expanding the input `x` to have length a power of 2, and sorting the resulting bits using an odd-even merge sorting network. The odd-even merge sorter shown in Listing 8.8 works recursively by splitting the input list in 2, sorting each list and merging the two sorted lists.

Listing 8.9: Cardinality constraints by binary decision diagrams (`bddsum.mzn`).

```
% the sum of booleans x = s
predicate bool_sum_eq(array[int] of var bool:x, int:s) =
     let { int: c = length(x),
           array[1..c] of var bool: y = [x[i] | i in index_set(x)]
     } in
     rec_bool_sum_eq(y, 1, s);

predicate rec_bool_sum_eq(array[int] of var bool:x, int: f, int:s) =
     let { int: c = length(x) } in
     if s < 0 then false
     elseif s == 0 then
          forall(i in f..c)(x[i] == false)
     elseif s < c - f + 1 then
          (x[f] == true /\ rec_bool_sum_eq(x,f+1,s-1)) \/
          (x[f] == false /\ rec_bool_sum_eq(x,f+1,s))
     elseif s == c - f + 1 then
```

```
        forall(i in f..c)(x[i] == true)
    else false endif;
```

We can implement `bool_sum_eq` using binary decision diagrams using the code shown in Listing 8.9. The cardinality constraint is broken into two cases: either the first element `x[1]` is `true`, and the sum of the remaining bits is `s-1`, or `x[1]` is `false` and the sum of the remaining bits is `s`. For efficiency this relies on common subexpression elimination to avoid creating many equivalent constraints.

CHAPTER 2.9

FlatZinc and Flattening

Constraint solvers do not directly support MiniZinc models, rather in order to run a MiniZinc model, it is translated into a simple subset of MiniZinc called FlatZinc. FlatZinc reflects the fact that most constraint solvers only solve satisfaction problems of the form $exists c_1 \wedge \cdots \wedge c_m$ or optimization problems of the form minimize z subject to $c_1 \wedge \cdots \wedge c_m$, where c_i are primitive constraints and z is an integer or float expression in a restricted form.

The `minizinc` tool includes the MiniZinc *compiler*, which takes a MiniZinc model and data files and creates a flattened FlatZinc model which is equivalent to the MiniZinc model with the given data, and that appears in the restricted form discussed above. Normally the construction of a FlatZinc model which is sent to a solver is hidden from the user but you can view the result of flattening a model `model.mzn` with its data `data.dzn` as follows:

```
minizinc -c model.mzn data.dzn
```

which creates a FlatZinc model called `model.fzn`.

In this chapter we explore the process of translation from MiniZinc to FlatZinc.

2.9.1 Flattening Expressions

The restrictions of the underlying solver mean that complex expressions in MiniZinc need to be *flattened* to only use conjunctions of primitive constraints which do not themselves contain structured terms.

Consider the following model for ensuring that two circles in a rectangular box do not overlap:

Listing 9.1: Modelling non overlap of two circles (cnonoverlap.mzn).

```
float: width;          % width of rectangle to hold circles
float: height;         % height of rectangle to hold circles
float: r1;
var r1..width-r1:  x1; % (x1,y1) is center of circle of radius r1
var r1..height-r1: y1;
float: r2;
var r2..width-r2:  x2; % (x2,y2) is center of circle of radius r2
var r2..height-r2: y2;
                       % centers are at least r1 + r2 apart
constraint (x1-x2)*(x1-x2) + (y1-y2)*(y1-y2) >= (r1+r2)*(r1+r2);
solve satisfy;
```

2.9.1.1 Simplification and Evaluation

Given the data file

```
width = 10.0;
height = 8.0;
r1 = 2.0;
r2 = 3.0;
```

the translation to FlatZinc first simplifies the model by replacing all the parameters by their values, and evaluating any fixed expression. After this simplification the values of parameters are not longer needed. An exception to this is large arrays of parametric values. If they are used more than once, then the parameter is retained to avoid duplicating the large expression.

After simplification the variable and parameter declarations parts of the model of Listing 9.1 become

```
var 2.0 .. 8.0: x1;
var 2.0 .. 6.0: y1;
var 3.0 .. 7.0: x2;
var 3.0 .. 5.0: y2;
```

2.9.1.2 Defining Subexpressions

Now no constraint solver directly handles complex constraint expressions like the one in Listing 9.1. Instead, each subexpression in the expression is named, and we create a constraint to construct the value of each expression. Let's examine the subexpressions of the constraint expression. `(x1 - x2)` is a subexpression, if we name if `FLOAT01` we can define it as `constraint FLOAT01 = x1 - x2;` Notice that this expression occurs twice in the model. We only need to construct the value once, we can then reuse it. This is called *common subexpression elimination*. The subexpression `(x1 - x2)*(x1 - x2)` can be named `FLOAT02` and we can define it as `constraint FLOAT02 = FLOAT01 * FLOAT01;` We can similarly name

`constraint FLOAT03 = y1 - y2;` and `constraint FLOAT04 = FLOAT03 * FLOAT03;` and finally `constraint FLOAT05 = FLOAT02 * FLOAT04;`. The inequality constraint itself becomes `constraint FLOAT05 >= 25.0;` since `(r1+r2)*(r1 + r2)` is calculated as `25.0`. The flattened constraint is hence

```
constraint FLOAT01 = x1 - x2;
constraint FLOAT02 = FLOAT01 * FLOAT01;
constraint FLOAT03 = y1 - y2;
constraint FLOAT04 = FLOAT03 * FLOAT03;
constraint FLOAT05 = FLOAT02 * FLOAT04;
constraint FLOAT05 >= 25.0
```

2.9.1.3 FlatZinc constraint form

Flattening as its final step converts the form of the constraint to a standard FlatZinc form which is always $p(a_1, \ldots, a_n)$ where `p` is the name of the primitive constraint and $a_1, \ldots, a_n$ are the arguments. FlatZinc tries to use a minimum of different constraint forms so for example the constraint `FLOAT01 = x1 - x2` is first rewritten to `FLOAT01 + x2 = x1` and then output using the `float_plus` primitive constraint. The resulting constraint form is as follows:

```
constraint float_plus(FLOAT01, x2, x1);
constraint float_plus(FLOAT03, y2, y1);
constraint float_plus(FLOAT02, FLOAT04, FLOAT05);
constraint float_times(FLOAT01, FLOAT01, FLOAT02);
constraint float_times(FLOAT03, FLOAT03, FLOAT04);
```

2.9.1.4 Bounds analysis

We are still missing one thing, the declarations for the introduced variables `FLOAT01`, ..., `FLOAT05`. While these could just be declared as `var float`, in order to make the solver's task easier MiniZinc tries to determine upper and lower bounds on newly introduced variables, by a simple bounds analysis. For example since `FLOAT01 = x1 - x2` and $2.0 \leq$ `x1` ≤ 8.0 and $3.0 \leq$ `x2` ≤ 7.0 then we can see that $-5.0 \leq$ `FLOAT0` ≤ 5.0. Given this information we can see that $-25.0 \leq$ `FLOAT02` ≤ 25.0 (although note that if we recognized that the multiplication was in fact a squaring we could give the much more accurate bounds $0.0 \leq$ `FLOAT02` ≤ 25.0).

The alert reader may have noticed a discrepancy between the flattened form of the constraints in *Defining Subexpressions* (page 132) and *FlatZinc constraint form* (page 133). In the latter there is no inequality constraint. Since unary inequalities can be fully represented by the bounds of a variable, the inequality forces the lower bound of `FLOAT05` to be `25.0` and is then redundant. The final flattened form of the model of Listing 9.1 is:

```
% Variables
var 2.0 .. 8.0: x1;
var 2.0 .. 6.0: y1;
var 3.0 .. 7.0: x2;
var 3.0 .. 5.0: y2;
```

```
%
var -5.0..5.0:   FLOAT01;
var -25.0..25.0: FLOAT02;
var -3.0..3.0:   FLOAT03;
var -9.0..9.0:   FLOAT04;
var 25.0..34.0:  FLOAT05;
% Constraints
constraint float_plus(FLOAT01, x2, x1);
constraint float_plus(FLOAT03, y2, y1);
constraint float_plus(FLOAT02, FLOAT04, FLOAT05);
constraint float_times(FLOAT01, FLOAT01, FLOAT02);
constraint float_times(FLOAT03, FLOAT03, FLOAT04);
%
solve satisfy;
```

2.9.1.5 Objectives

MiniZinc flattens minimization or maximization objectives just like constraints. The objective expression is flattened and a variable is created for it, just as for other expressions. In the FlatZinc output the solve item is always on a single variable. See *Let Expressions* (page 143) for an example.

2.9.2 Linear Expressions

One of the most important form of constraints, widely used for modelling, are linear constraints of the form

$$a_1x_1 + \cdots + a_nx_n \begin{matrix} = \\ \leq \\ < \end{matrix} a_0$$

where a_i are integer or floating point constants, and x_i are integer or floating point variables. They are highly expressive, and are the only class of constraint supported by (integer) linear programming constraint solvers. The translator from MiniZinc to FlatZinc tries to create linear constraints, rather than break up linear constraints into many subexpressions.

Listing 9.2: A MiniZinc model to illustrate linear constraint flattening (`linear.mzn`).

```
int:       d = -1;
var 0..10: x;
var -3..6: y;
var 3..8:  z;
constraint 3*x - y + x * z <= 19 + d * (x + y + z) - 4*d;
solve satisfy;
```

Consider the model shown in Listing 9.2. Rather than create variables for all the subexpressions $3 * x$, $3 * x - y$, $x * z$, $3 * x - y + x * z$, $x + y + z$, $d * (x + y + z)$, $19 + d * (x + y + z)$,

and $19 + d * (x + y + z) - 4 * d$ translation will attempt to create a large linear constraint which captures as much as possible of the constraint in a single FlatZinc constraint.

Flattening creates linear expressions as a single unit rather than building intermediate variables for each subexpression. It also simplifies the linear expression created. Extracting the linear expression from the constraints leads to

```
var 0..80: INT01;
constraint 4*x + z + INT01 <= 23;
constraint INT01 = x * z;
```

Notice how the *nonlinear expression* $x \times z$ is extracted as a new subexpression and given a name, while the remaining terms are collected together so that each variable appears exactly once (and indeed variable y whose terms cancel is eliminated).

Finally each constraint is written to FlatZinc form obtaining:

```
var 0..80: INT01;
constraint int_lin_le([1,4,1],[INT01,x,z],23);
constraint int_times(x,z,INT01);
```

2.9.3 Unrolling Expressions

Most models require creating a number of constraints which is dependent on the input data. MiniZinc supports these models with array types, list and set comprehensions, and aggregation functions.

Consider the following aggregate expression from the production scheduling example of Listing 2.2.

```
int: mproducts = max (p in Products)
                     (min (r in Resources where consumption[p,r] > 0)
                                   (capacity[r] div consumption[p,r]));
```

Since this uses generator call syntax we can rewrite it to equivalent form which is processed by the compiler:

```
int: mproducts = max([ min [ capacity[r] div consumption[p,r]
                           | r in Resources where consumption[p,r] > 0])
                     | p in Products]);
```

Given the data

```
nproducts = 2;
nresources = 5;
capacity = [4000, 6, 2000, 500, 500];
consumption= [| 250, 2, 75, 100, 0,
              | 200, 0, 150, 150, 75 |];
```

this first builds the array for `p = 1`

```
[ capacity[r] div consumption[p,r]
                          | r in 1..5 where consumption[p,r] > 0]
```

which is `[16, 3, 26, 5]` and then calculates the minimum as 3. It then builds the same array for `p = 2` which is `[20, 13, 3, 6]` and calculates the minimum as 3. It then constructs the array `[3, 3]` and calculates the maximum as 3. There is no representation of `mproducts` in the output FlatZinc, this evaluation is simply used to replace `mproducts` by the calculated value 3.

The most common form of aggregate expression in a constraint model is `forall`. Forall expressions are unrolled into multiple constraints.

Consider the MiniZinc fragment

```
array[1..8] of var 0..9: v = [S,E,N,D,M,O,R,Y];
constraint forall(i,j in 1..8 where i < j)(v[i] != v[j])
```

which arises from the SEND-MORE-MONEY example of Listing 2.4 using a default decomposition for `alldifferent`. The `forall` expression creates a constraint for each i, j pair which meet the requirement $i < j$, thus creating

```
constraint v[1] != v[2]; % S != E
constraint v[1] != v[3]; % S != N
...
constraint v[1] != v[8]; % S != Y
constraint v[2] != v[3]; % E != N
...
constraint v[7] != v[8]; % R != Y
```

In FlatZinc form this is

```
constraint int_neq(S,E);
constraint int_neq(S,N);
...
constraint int_neq(S,Y);
constraint int_neq(E,N);
...
constraint int_neq(R,Y);
```

Notice how the temporary array variables `v[i]` are replaced by the original variables in the output FlatZinc.

2.9.4 Arrays

One dimensional arrays in MiniZinc can have arbitrary indices as long as they are contiguous integers. In FlatZinc all arrays are indexed from `1..l` where `l` is the length of

the array. This means that array lookups need to be translated to the FlatZinc view of indices.

Consider the following MiniZinc model for balancing a seesaw of length `2 * l2`, with a child of weight `cw` kg using exactly `m` 1kg weights.

```
int: cw;                                % child weight
int: l2;                                % half seesaw length
int: m;                                 % number of 1kg weight
array[-l2..l2] of var 0..max(m,cw): w; % weight at each point
var -l2..l2: p;                         % position of child
constraint sum(i in -l2..l2)(i * w[i]) = 0; % balance
constraint sum(i in -l2..l2)(w[i]) = m + cw; % all weights used
constraint w[p] = cw;                   % child is at position p
solve satisfy;
```

Given `cw = 2`, `l2 = 2`, and `m = 3` the unrolling produces the constraints

```
array[-2..2] of var 0..3: w;
var -2..2: p
constraint -2*w[-2] + -1*w[-1] + 0*w[0] + 1*w[1] + 2*w[2] = 0;
constraint w[-2] + w[-1] + w[0] + w[1] + w[2] = 5;
constraint w[p] = 2;
```

But FlatZinc insists that the `w` array starts at index 1. This means we need to rewrite all the array accesses to use the new index value. For fixed array lookups this is easy, for variable array lookups we may need to create a new variable. The result for the equations above is

```
array[1..5] of var 0..3: w;
var -2..2: p
var 1..5: INT01;
constraint -2*w[1] + -1*w[2] + 0*w[3] + 1*w[4] + 2*w[5] = 0;
constraint w[1] + w[2] + w[3] + w[4] + w[5] = 5;
constraint w[INT01] = 2;
constraint INT01 = p + 3;
```

Finally we rewrite the constraints into FlatZinc form. Note how the variable array index lookup is mapped to `array_var_int_element`.

```
array[1..5] of var 0..3: w;
var -2..2: p
var 1..5: INT01;
constraint int_lin_eq([2, 1, -1, -2], [w[1], w[2], w[4], w[5]], 0);
constraint int_lin_eq([1, 1, 1, 1, 1], [w[1],w[2],w[3],w[4],w[5]], 5);
constraint array_var_int_element(INT01, w, 2);
constraint int_lin_eq([-1, 1], [INT01, p], -3);
```

Multidimensional arrays are supported by MiniZinc, but only single dimension arrays

are supported by FlatZinc (at present). This means that multidimensional arrays must be mapped to single dimension arrays, and multidimensional array access must be mapped to single dimension array access.

Consider the Laplace equation constraints defined for a finite element plate model in Listing 2.1:

```
set of int: HEIGHT = 0..h;
set of int: CHEIGHT = 1..h-1;
set of int: WIDTH = 0..w;
set of int: CWIDTH = 1..w-1;
array[HEIGHT,WIDTH] of var float: t; % temperature at point (i,j)
var float: left;   % left   edge temperature
var float: right;  % right  edge temperature
var float: top;    % top    edge temperature
var float: bottom; % bottom edge temperature

% equation
% Laplace equation: each internal temp. is average of its neighbours
constraint forall(i in CHEIGHT, j in CWIDTH)(
              4.0*t[i,j] = t[i-1,j] + t[i,j-1] + t[i+1,j] + t[i,j+1]);
```

Assuming `w = 4` and `h = 4` this creates the constraints

```
array[0..4,0..4] of var float: t; % temperature at point (i,j)
constraint 4.0*t[1,1] = t[0,1] + t[1,0] + t[2,1] + t[1,2];
constraint 4.0*t[1,2] = t[0,2] + t[1,1] + t[2,2] + t[1,3];
constraint 4.0*t[1,3] = t[0,3] + t[1,2] + t[2,3] + t[1,4];
constraint 4.0*t[2,1] = t[1,1] + t[2,0] + t[3,1] + t[2,2];
constraint 4.0*t[2,2] = t[1,2] + t[2,1] + t[3,2] + t[2,3];
constraint 4.0*t[2,3] = t[1,3] + t[2,2] + t[3,3] + t[2,4];
constraint 4.0*t[3,1] = t[2,1] + t[3,0] + t[4,1] + t[3,2];
constraint 4.0*t[3,2] = t[2,2] + t[3,1] + t[4,2] + t[3,3];
constraint 4.0*t[3,3] = t[2,3] + t[3,2] + t[4,3] + t[3,4];
```

The 2 dimensional array of 25 elements is converted to a one dimensional array and the indices are changed accordingly: so index `[i,j]` becomes `[i * 5 + j + 1]`.

```
array [1..25] of var float: t;
constraint 4.0*t[7] = t[2] + t[6] + t[12] + t[8];
constraint 4.0*t[8] = t[3] + t[7] + t[13] + t[9];
constraint 4.0*t[9] = t[4] + t[8] + t[14] + t[10];
constraint 4.0*t[12] = t[7] + t[11] + t[17] + t[13];
constraint 4.0*t[13] = t[8] + t[12] + t[18] + t[14];
constraint 4.0*t[14] = t[9] + t[13] + t[19] + t[15];
constraint 4.0*t[17] = t[12] + t[16] + t[22] + t[18];
constraint 4.0*t[18] = t[13] + t[17] + t[23] + t[19];
constraint 4.0*t[19] = t[14] + t[18] + t[24] + t[20];
```

2.9.5 Reification

FlatZinc models involve only variables and parameter declarations and a series of primitive constraints. Hence when we model in MiniZinc with Boolean connectives other than conjunction, something has to be done. The core approach to handling complex formulae that use connectives other than conjunction is by *reification*. Reifying a constraint c creates a new constraint equivalent to $b \leftrightarrow c$ where the Boolean variable b is `true` if the constraint holds and `false` if it doesn't hold.

Once we have the capability to *reify* constraints, the treatment of complex formulae is no different from arithmetic expressions. We create a name for each subexpression and a flat constraint to constrain the name to take the value of its subexpression.

Consider the following constraint expression that occurs in the jobshop scheduling example of Listing 2.10.

```
constraint %% ensure no overlap of tasks
    forall(j in 1..tasks) (
        forall(i,k in 1..jobs where i < k) (
            s[i,j] + d[i,j] <= s[k,j] \/
            s[k,j] + d[k,j] <= s[i,j]
    ) );
```

Given the data file

```
jobs = 2;
tasks = 3;
d = [| 5, 3, 4 | 2, 6, 3 |]
```

then the unrolling creates

```
constraint s[1,1] + 5 <= s[2,1] \/ s[2,1] + 2 <= s[1,1];
constraint s[1,2] + 3 <= s[2,2] \/ s[2,2] + 6 <= s[1,2];
constraint s[1,3] + 4 <= s[2,3] \/ s[2,3] + 3 <= s[1,3];
```

Reification of the constraints that appear in the disjunction creates new Boolean variables to define the values of each expression.

```
array[1..2,1..3] of var 0..23: s;
constraint BOOL01 <-> s[1,1] + 5 <= s[2,1];
constraint BOOL02 <-> s[2,1] + 2 <= s[1,1];
constraint BOOL03 <-> s[1,2] + 3 <= s[2,2];
constraint BOOL04 <-> s[2,2] + 6 <= s[1,2];
constraint BOOL05 <-> s[1,3] + 4 <= s[2,3];
constraint BOOL06 <-> s[2,3] + 3 <= s[1,3];
constraint BOOL01 \/ BOOL02;
constraint BOOL03 \/ BOOL04;
constraint BOOL05 \/ BOOL06;
```

Each primitive constraint can now be mapped to the FlatZinc form. Note how the two dimensional array `s` is mapped to a one dimensional form.

```
array[1..6] of var 0..23: s;
constraint int_lin_le_reif([1, -1], [s[1], s[4]], -5, BOOL01);
constraint int_lin_le_reif([-1, 1], [s[1], s[4]], -2, BOOL02);
constraint int_lin_le_reif([1, -1], [s[2], s[5]], -3, BOOL03);
constraint int_lin_le_reif([-1, 1], [s[2], s[5]], -6, BOOL04);
constraint int_lin_le_reif([1, -1], [s[3], s[6]], -4, BOOL05);
constraint int_lin_le_reif([-1, 1], [s[3], s[6]], -3, BOOL06);
constraint bool_clause([BOOL01, BOOL02], []);
constraint bool_clause([BOOL03, BOOL04], []);
constraint bool_clause([BOOL05, BOOL06], []);
```

The `int_lin_le_reif` is the reified form of the linear constraint `int_lin_le`.

Most FlatZinc primitive constraints $p(\bar{x})$ have a reified form *p_reif*$(\bar{x}, b)$ which takes an additional final argument b and defines the constraint $b \leftrightarrow p(\bar{x})$. FlatZinc primitive constraints which define functional relationships, like `int_plus` and `int_plus`, do not need to support reification. Instead, the equality with the result of the function is reified.

Another important use of reification arises when we use the coercion function `bool2int` (either explicitly, or implicitly by using a Boolean expression as an integer expression). Flattening creates a Boolean variable to hold the value of the Boolean expression argument, as well as an integer variable (restricted to `0..1`) to hold this value.

Consider the magic series problem of Listing 2.14.

```
int: n;
array[0..n-1] of var 0..n: s;

constraint forall(i in 0..n-1) (
   s[i] = (sum(j in 0..n-1)(bool2int(s[j]=i))));
```

Given `n = 2` the unrolling creates

```
constraint s[0] = bool2int(s[0] = 0) + bool2int(s[1] = 0);
constraint s[1] = bool2int(s[0] = 1) + bool2int(s[1] = 1);
```

and flattening creates

```
constraint BOOL01 <-> s[0] = 0;
constraint BOOL03 <-> s[1] = 0;
constraint BOOL05 <-> s[0] = 1;
constraint BOOL07 <-> s[1] = 1;
constraint INT02 = bool2int(BOOL01);
constraint INT04 = bool2int(BOOL03);
constraint INT06 = bool2int(BOOL05);
constraint INT08 = bool2int(BOOL07);
constraint s[0] = INT02 + INT04;
```

```
constraint s[1] = INT06 + INT08;
```

The final FlatZinc form is

```
var bool: BOOL01;
var bool: BOOL03;
var bool: BOOL05;
var bool: BOOL07;
var 0..1: INT02;
var 0..1: INT04;
var 0..1: INT06;
var 0..1: INT08;
array [1..2] of var 0..2: s;
constraint int_eq_reif(s[1], 0, BOOL01);
constraint int_eq_reif(s[2], 0, BOOL03);
constraint int_eq_reif(s[1], 1, BOOL05);
constraint int_eq_reif(s[2], 1, BOOL07);
constraint bool2int(BOOL01, INT02);
constraint bool2int(BOOL03, INT04);
constraint bool2int(BOOL05, INT06);
constraint bool2int(BOOL07, INT08);
constraint int_lin_eq([-1, -1, 1], [INT02, INT04, s[1]], 0);
constraint int_lin_eq([-1, -1, 1], [INT06, INT08, s[2]], 0);
solve satisfy;
```

2.9.6 Predicates

An important factor in the support for MiniZinc by many different solvers is that global constraints (and indeed FlatZinc constraints) can be specialized for the particular solver.

Each solver will either specify a predicate without a definition, or with a definition. For example a solver that has a builtin global `alldifferent` predicate, will include the definition

```
predicate alldifferent(array[int] of var int:x);
```

in its globals library, while a solver using the default decomposition will have the definition

```
predicate alldifferent(array[int] of var int:x) =
    forall(i,j in index_set(x) where i < j)(x[i] != x[j]);
```

Predicate calls $p(\bar{t})$ are flattened by first constructing variables v_i for each argument terms t_i. If the predicate has no definition we simply use a call to the predicate with the constructed arguments: $p(\bar{v})$. If the predicate has a definition $p(\bar{x}) = \phi(\bar{x})$ then we replace the predicate call $p(\bar{t})$ with the body of the predicate with the formal arguments replaced by the argument variables, that is $\phi(\bar{v})$. Note that if a predicate call $p(\bar{t})$ appears in a reified position and it has no definition, we check for the existence of a reified

version of the predicate *p_reif*$(\bar{x}, b)$ in which case we use that.

Consider the `alldifferent` constraint in the SEND-MORE-MONEY example of Listing 2.4

```
constraint alldifferent([S,E,N,D,M,O,R,Y]);
```

If the solver has a builtin `alldifferent` we simply construct a new variable for the argument, and replace it in the call.

```
array[1..8] of var 0..9: v = [S,E,N,D,M,O,R,Y];
constraint alldifferent(v);
```

Notice that bounds analysis attempts to find tight bounds on the new array variable. The reason for constructing the array argument is if we use the same array twice the FlatZinc solver does not have to construct it twice. In this case since it is not used twice a later stage of the translation will replace `v` by its definition.

What if the solver uses the default definition of `alldifferent`? Then the variable `v` is defined as usual, and the predicate call is replaced by a renamed copy where `v` replaces the formal argument `x`. The resulting code is

```
array[1..8] of var 0..9: v = [S,E,N,D,M,O,R,Y];
constraint forall(i,j in 1..8 where i < j)(v[i] != v[j])
```

which we examined in *Unrolling Expressions* (page 135).

Consider the following constraint, where `alldifferent` appears in a reified position.

```
constraint alldifferent([A,B,C]) \/ alldifferent([B,C,D]);
```

If the solver has a reified form of `alldifferent` this will be flattened to

```
constraint alldifferent_reif([A,B,C],BOOL01);
constraint alldifferent_reif([B,C,D],BOOL02);
constraint bool_clause([BOOL01,BOOL02],[]);
```

Using the default decomposition, the predicate replacement will first create

```
array[1..3] of var int: v1 = [A,B,C];
array[1..3] of var int: v2 = [B,C,D];
constraint forall(i,j in 1..3 where i<j)(v1[i] != v1[j]) \/
           forall(i,j in 1..3 where i<j)(v2[i] != v2[j]);
```

which will eventually be flattened to the FlatZinc form

```
constraint int_neq_reif(A,B,BOOL01);
constraint int_neq_reif(A,C,BOOL02);
constraint int_neq_reif(B,C,BOOL03);
constraint array_bool_and([BOOL01,BOOL02,BOOL03],BOOL04);
```

```
constraint int_neq_reif(B,D,BOOL05);
constraint int_neq_reif(C,D,BOOL06);
constraint array_bool_and([BOOL03,BOOL05,BOOL06],BOOL07);
constraint bool_clause([BOOL04,BOOL07],[]);
```

Note how common subexpression elimination reuses the reified inequality `B != C` (although there is a better translation which lifts the common constraint to the top level conjunction).

2.9.7 Let Expressions

Let expressions are a powerful facility of MiniZinc to introduce new variables. This is useful for creating common sub expressions, and for defining local variables for predicates. During flattening let expressions are translated to variable and constraint declarations. The relational semantics of MiniZinc means that these constraints must appear as if conjoined in the first enclosing Boolean expression.

A key feature of let expressions is that each time they are used they create new variables.

Consider the flattening of the code

```
constraint even(u) \/ even(v);
predicate even(var int: x) =
          let { var int: y } in x = 2 * y;
```

First the predicate calls are replaced by their definition.

```
constraint (let { var int: y} in u = 2 * y) \/
           (let { var int: y} in v = 2 * y);
```

Next let variables are renamed apart

```
constraint (let { var int: y1} in u = 2 * y1) \/
           (let { var int: y2} in v = 2 * y2);
```

Finally variable declarations are extracted to the top level

```
var int: y1;
var int: y2;
constraint u = 2 * y1 \/ v = 2 * y2;
```

Once the let expression is removed we can flatten as usual.

Remember that let expressions can define values for newly introduced variables (and indeed must do so for parameters). These implicitly define constraints that must also be flattened.

Consider the complex objective function for wedding seating problem of Listing 3.11.

```
solve maximize sum(h in Hatreds)(
      let {  var Seats: p1 = pos[h1[h]],
             var Seats: p2 = pos[h2[h]],
             var 0..1: same = bool2int(p1 <= 6 <-> p2 <= 6) } in
      same * abs(p1 - p2) + (1-same) * (abs(13 - p1 - p2) + 1));
```

For conciseness we assume only the first two Hatreds, so

```
set of int: Hatreds = 1..2;
array[Hatreds] of Guests: h1 = [groom, carol];
array[Hatreds] of Guests: h2 = [clara, bestman];
```

The first step of flattening is to unroll the `sum` expression, giving (we keep the guest names and parameter `Seats` for clarity only, in reality they would be replaced by their definition):

```
solve maximize
      (let { var Seats: p1 = pos[groom],
             var Seats: p2 = pos[clara],
             var 0..1: same = bool2int(p1 <= 6 <-> p2 <= 6) } in
       same * abs(p1 - p2) + (1-same) * (abs(13 - p1 - p2) + 1))
      +
      (let { var Seats: p1 = pos[carol],
             var Seats: p2 = pos[bestman],
             var 0..1: same = bool2int(p1 <= 6 <-> p2 <= 6) } in
       same * abs(p1 - p2) + (1-same) * (abs(13 - p1 - p2) + 1));
```

Next each new variable in a let expression is renamed to be distinct

```
solve maximize
      (let { var Seats: p11 = pos[groom],
             var Seats: p21 = pos[clara],
             var 0..1: same1 = bool2int(p11 <= 6 <-> p21 <= 6) } in
       same1 * abs(p11 - p21) + (1-same1) * (abs(13 - p11 - p21) + 1))
      +
      (let { var Seats: p12 = pos[carol],
             var Seats: p22 = pos[bestman],
             var 0..1: same2 = bool2int(p12 <= 6 <-> p22 <= 6) } in
       same2 * abs(p12 - p22) + (1-same2) * (abs(13 - p12 - p22) + 1));
```

Variables in the let expression are extracted to the top level and defining constraints are extracted to the correct level (which in this case is also the top level).

```
var Seats: p11;
var Seats: p21;
var 0..1: same1;
constraint p12 = pos[clara];
constraint p11 = pos[groom];
```

```
constraint same1 = bool2int(p11 <= 6 <-> p21 <= 6);
var Seats p12;
var Seats p22;
var 0..1: same2;
constraint p12 = pos[carol];
constraint p22 = pos[bestman];
constraint same2 = bool2int(p12 <= 6 <-> p22 <= 6) } in
solve maximize
      same1 * abs(p11 - p21) + (1-same1) * (abs(13 - p11 - p21) + 1))
      +
      same2 * abs(p12 - p22) + (1-same2) * (abs(13 - p12 - p22) + 1));
```

Now we have constructed equivalent MiniZinc code without the use of let expressions and the flattening can proceed as usual.

As an illustration of let expressions that do not appear at the top level consider the following model

```
var 0..9: x;
constraint x >= 1 -> let { var 2..9: y = x - 1 } in
                     y + (let { var int: z = x * y } in z * z) < 14;
```

We extract the variable definitions to the top level and the constraints to the first enclosing Boolean context, which here is the right hand side of the implication.

```
var 0..9: x;
var 2..9: y;
var int: z;
constraint x >= 1 -> (y = x - 1 /\ z = x * y /\ y + z * z < 14);
```

Note that if we know that the equation defining a variable definition cannot fail we can extract it to the top level. This will usually make solving substantially faster.

For the example above the constraint `y = x - 1` can fail since the domain of `y` is not big enough for all possible values of `x - 1`. But the constraint `z = x * y` cannot (indeed bounds analysis will give `z` bounds big enough to hold all possible values of `x * y`). A better flattening will give

```
var 0..9: x;
var 2..9: y;
var int: z;
constraint z = x * y;
constraint x >= 1 -> (y = x - 1 /\ y + z * z < 14);
```

Currently the MiniZinc compiler does this by always defining the declared bounds of an introduced variable to be big enough for its defining equation to always hold and then adding bounds constraints in the correct context for the let expression. On the example above this results in

```
var 0..9: x;
var -1..8: y;
var -9..72: z;
constraint y = x - 1;
constraint z = x * y;
constraint x >= 1 -> (y >= 2 /\ y + z * z < 14);
```

This translation leads to more efficient solving since the possibly complex calculation of the let variable is not reified.

Another reason for this approach is that it also works when introduced variables appear in negative contexts (as long as they have a definition). Consider the following example similar to the previous one

```
var 0..9: x;
constraint (let { var 2..9: y = x - 1 } in
           y + (let { var int: z = x * y } in z * z) > 14) -> x >= 5;
```

The let expressions appear in a negated context, but each introduced variable is defined. The flattened code is

```
var 0..9: x;
var -1..8: y;
var -9..72: z;
constraint y = x - 1;
constraint z = x * y;
constraint (y >= 2 /\ y + z * z > 14) -> x >= 5;
```

Note the analog to the simple approach to let elimination does not give a correct translation:

```
var 0..9: x;
var 2..9: y;
var int: z;
constraint (y = x - 1 /\ z = x * y /\ y + z * z > 14) -> x >= 5;
```

gives answers for all possible values of `x`, whereas the original constraint removes the possibility that `x = 4`.

The treatment of *constraint items* in let expressions is analogous to defined variables. One can think of a constraint as equivalent to defining a new Boolean variable. The definitions of the new Boolean variables are extracted to the top level, and the Boolean remains in the correct context.

```
constraint z > 1 -> let { var int: y,
                          constraint (x >= 0) -> y = x,
                          constraint (x < 0)  -> y = -x
                        } in y * (y - 2) >= z;
```

is treated like

```
constraint z > 1 -> let { var int: y,
                          var bool: b1 = ((x >= 0) -> y = x),
                          var bool: b2 = ((x < 0)  -> y = -x),
                          constraint b1 /\ b2
                        } in y * (y - 2) >= z;
```

and flattens to

```
constraint b1 = ((x >= 0) -> y = x);
constraint b2 = ((x < 0)  -> y = -x);
constraint z > 1 -> (b1 /\ b2 /\ y * (y - 2) >= z);
```

Part 3

User Manual

CHAPTER 3.1

The MiniZinc Command Line Tool

The core of the MiniZinc constraint modelling system is the `minizinc` tool. You can use it directly from the command line, through the MiniZinc IDE, or through a programmatic interface (API). This chapter summarises the options accepted by the tool, and explains how it interacts with target solvers.

3.1.1 Basic Usage

The `minizinc` tool performs three basic functions: it *compiles* a MiniZinc model (plus instance data), it *runs* an external solver, and it *translates solver output* into the form specified in the model. Most users would use all three functions at the same time. For example, let us assume that we want to solve the following simple problem, given as two files (`model.mzn` and `data.dzn`):

```
int: n;
array[1..n] of var 1..2*n: x;
include "alldifferent.mzn";
constraint alldifferent(x);
solve maximize sum(x);
output ["The resulting values are \(x).\n"];
```

```
n = 5;
```

To run the model file `model.mzn` with data file `data.dzn` using the Gecode solver, you can use the following command line:

```
$ minizinc --solver gecode model.mzn data.dzn
```

This would result in the output

```
The resulting values are [10, 9, 8, 7, 6].
----------
==========
```

However, each of the three functions can also be accessed individually. For example, to compile the same model and data, use the `-c` option:

```
$ minizinc -c --solver gecode model.mzn data.dzn
```

This will result in two new files, `model.fzn` and `model.ozn`, being output in the same directory as `model.mzn`. You could then run a target solver directly on the `model.fzn` file, or use `minizinc`:

```
$ minizinc --solver gecode model.fzn
```

You will see that the solver produces output in a standardised form, but not the output prescribed by the output item in the model:

```
x = array1d(1..5 ,[10, 9, 8, 7, 6]);
----------
==========
```

The translation from this output to the form specified in the model is encoded in the `model.ozn` file. You can use `minizinc` to execute the `.ozn` file. In this mode, it reads a stream of solutions from standard input, so we need to pipe the solver output into `minizinc`:

```
$ minizinc --solver gecode model.fzn | minizinc --ozn-file model.ozn
```

These are the most basic command line options that you need in order to compile and run models and translate their output. The next section lists all available command line options in detail. Section 4.3.6 explains how new solvers can be added to the system.

3.1.2 Adding Solvers

Solvers that support MiniZinc typically consist of two parts: a solver *executable*, which can be run on the FlatZinc output of the MiniZinc compiler, and a *solver library*, which consists of a set of MiniZinc files that define the constraints that the solver supports natively. This section deals with making existing solvers available to the MiniZinc tool chain. For information on how to add FlatZinc support to a solver, refer to Section 4.3.

3.1.2.1 Configuration files

In order for MiniZinc to be able to find both the solver library and the executable, the solver needs to be described in a *solver configuration file* (see Section 4.3.6 for details). If the solver you want to install comes with a configuration file (which has the file extension `.msc` for MiniZinc Solver Configuration), it has to be in one of the following locations:

- In the `minizinc/solvers/` directory of the MiniZinc installation. If you install MiniZinc from the binary distribution, this directory can be found at `/usr/share/minizinc/solvers` on Linux systems, inside the MiniZincIDE application on macOS system, and in the `Program Files\\MiniZinc IDE (bundled)` folder on Windows.
- In the directory `$HOME/.minizinc/solvers` on Linux and macOS systems, and the Application Data directory on Windows systems.
- In any directory listed on the `MZN_SOLVER_PATH` environment variable (directories are separated by `:` on Linux and macOS, and by `;` on Windows systems).
- In any directory listed in the `mzn_solver_path` option of the global or user-specific configuration file (see Section 3.1.4)
- Alternatively, you can use the MiniZinc IDE to create solver configuration files, see Section 3.2.5.2 for details.

After adding a solver, it will be listed in the output of the `minizinc --solvers` command.

3.1.2.2 Configuration for MIP solvers

Some solvers require additional configuration flags before they can be used. For example, the binary bundle of MiniZinc comes with interfaces to the CPLEX and Gurobi Mixed Integer Programming solvers. However, due to licensing restrictions, the solvers themselves are not part of the bundled release. Depending on where CPLEX or Gurobi is installed on your system, MiniZinc may be able to find the solvers automatically, or it may require an additional option to point it to the shared library.

In case the libraries cannot be found automatically, you can use one of the following:

- CPLEX: Specify the location of the shared library using the `--cplex-dll` command line option. On Windows, the library is called `cplexXXXX.dll` and typically found in same directory as the `cplex` executable. On Linux it is `libcplexXXX.so`, and on macOS `libcplexXXXX.jnilib`, where `XXX` and `XXXX` stand for the version number.
- Gurobi: The command line option for Gurobi is `--gurobi-dll`. On Windows, the library is called `gurobiXX.dll` (in the same directory as the `gurobi` executable), and on Linux and macOS is it `libgurobiXX.so` (in the `lib` directory of your Gurobi installation).
- You can define these paths as defaults in your user configuration file, see Section 3.1.4.

3.1.3 Options

You can get a list of all the options supported by the `minizinc` tool using the `--help` flag.

3.1.3.1 General options

These options control the general behaviour of the `minizinc` tool.

`--help, -h`
: Print a help message.

`--version`
: Print version information.

`--solvers`
: Print list of available solvers.

`--solver <id>, --solver <solver configuration file>.msc`
: Select solver to use. The first form of the command selects one of the solvers known to MiniZinc (that appear in the list of the `--solvers` command). You can select a solver by name, id, or tag, and add a specific version. For example, to select a mixed-integer programming solver, identified by the `mip` tag, you can use `--solver mip`. To select a specific version of Gecode (in case you have two versions installed), use `--solver Gecode@6.3.0`. Instead of the name you can also use the solver's identifier, e.g. `--solver org.gecode.gecode`.

  The second form of the command selects the solver from the given configuration file (see Section 4.3.6).

`--help <id>`
: Print help for a particular solver. The scheme for selecting a solver is the same as for the `--solver` option.

`-v, -l, --verbose`
: Print progress/log statements (for both compilation and solver). Note that some solvers may log to stdout.

`--verbose-compilation`
: Print progress/log statements for compilation only.

`-s, --statistics`
: Print statistics (for both compilation and solving).

`--compiler-statistics`
: Print statistics for compilation.

`-c, --compile`
: Compile only (do not run solver).

`--config-dirs`
: Output configuration directories.

`--solvers-json`
: Print configurations of available solvers as a JSON array.

`--param-file <file>`
: Read command line options from the given JSON file. See Section 3.1.5.

`--json-stream`
: Generate output as line-break delimited JSON objects.

3.1.3.2 Solving options

Each solver may support specific command line options for controlling its behaviour. These can be queried using the `--help <id>` flag, where `<id>` is the name or identifier of a particular solver. Most solvers will support some or all of the following options.

`-a, --all-solutions`
: Report *all* solutions in the case of satisfaction problems, or print *intermediate* solutions of increasing quality in the case of optimisation problems.

`-n <i>, --num-solutions <i>`
: Stop after reporting `i` solutions (only used with satisfaction problems).

`-i, --intermediate`
: Turns on printing of intermediate solutions for optimisation problems. Has no effect for satisfaction problems.

`-n-i, --no-intermediate`
: Turns off printing of intermediate solutions for optimisation problems. Has no effect for satisfaction problems.

`--all-satisfaction`
: Turns on printing of all solutions for satisfaction problems. Has no effect for optimisation problems.

`--disable-all-satisfaction`
: Turns off printing of all solutions for satisfaction problems. Has no effect for optimisation problems.

`-f, --free-search`
: Instructs the solver to conduct a "free search", i.e., ignore any search annotations. The solver is not *required* to ignore the annotations, but it is *allowed* to do so.

`--solver-statistics`
: Print statistics during and/or after the search for solutions.

`--verbose-solving`
: Print log messages (verbose solving) to the standard error stream.

`-p <i>, --parallel <i>`
: Run with `i` parallel threads (for multi-threaded solvers).

`-r <i>, --random-seed <i>`
: Use `i` as the random seed (for any random number generators the solver may be using).

3.1.3.3 Flattener input options

These options control aspects of the MiniZinc compiler.

`--ignore-stdlib`
: Ignore the standard libraries stdlib.mzn and builtins.mzn

`--instance-check-only`
: Check the model instance (including data) for errors, but do not convert to FlatZinc.

`-e, --model-check-only`
: Check the model (without requiring data) for errors, but do not convert to FlatZinc.

`--model-interface-only`
: Only extract parameters and output variables.

`--model-types-only`
: Only output variable (enum) type information.

`--no-optimize`
: Do not optimize the FlatZinc

`-m <file>, --model <file>`
: File named <file> contains the model.

`-d <file>, --data <file>`
: File named <file> contains data used by the model.

`--checker <file>, --solution-checker <file>`
: File named <file> contains the solution checker model.

`-D <data>, --cmdline-data <data>`
: Include the given data assignment in the model.

`--cmdline-json-data <data>`
: Use the given JSON data in the model.

`--stdlib-dir <dir>`
: Path to MiniZinc standard library directory

`-G --globals-dir --mzn-globals-dir <dir>`
: Search for included globals in <stdlib>/<dir>.

`- --input-from-stdin`
: Read problem from standard input. Combine with `--input-is-flatzinc` when passing FlatZinc code.

`-I --search-dir`
: Additionally search for included files in <dir>.

`-D "fMIPdomains=false"`
: No domain unification for MIP

`--MIPDMaxIntvEE <n>`
: Max integer domain subinterval length to enforce equality encoding, default 0

`--MIPDMaxDensEE <n>`
: Max domain cardinality to N subintervals ratio to enforce equality encoding, default 0, either condition triggers

`--only-range-domains`
: When no MIPdomains: all domains contiguous, holes replaced by inequalities

`--allow-multiple-assignments`
: Allow multiple assignments to the same variable (e.g. in dzn)

`--compile-solution-checker <file>.mzc.mzn`
: Compile solution checker model.

Flattener two-pass options

Two-pass compilation means that the MiniZinc compiler will first compile the model in order to collect some global information about it, which it can then use in a second pass to improve the resulting FlatZinc. For some combinations of model and target solver, this can lead to substantial improvements in solving time. However, the additional time spent on the first compilation pass does not always pay off.

`--two-pass`
: Flatten twice to make better flattening decisions for the target

`--use-gecode`
: Perform root-node-propagation with Gecode (adds –two-pass)

`--shave`
: Probe bounds of all variables at the root node (adds –use-gecode)

`--sac`
: Probe values of all variables at the root node (adds –use-gecode)

`--pre-passes <n>`
: Number of times to apply shave/sac pass (0 = fixed-point, 1 = default)

`-O<n>`
: Two-pass optimisation levels:

  -O0: Disable optimize (–no-optimize) -O1: Single pass (default) -O2: Same as: –two-pass -O3: Same as: –use-gecode -O4: Same as: –shave -O5: Same as: –sac

Flattener output options

These options control how the MiniZinc compiler produces the resulting FlatZinc output. If you run the solver directly through the `minizinc` command or the MiniZinc IDE, you do not need to use any of these options.

`--no-output-ozn, -O-`
: Do not output ozn file

`--output-base <name>`
: Base name for output files

`--fzn <file>, --output-fzn-to-file <file>`
: Filename for generated FlatZinc output

`-O, --ozn, --output-ozn-to-file <file>`
: Filename for model output specification (-O- for none)

`--keep-paths`
: Don’t remove path annotations from FlatZinc

`--output-paths`
: Output a symbol table (.paths file)

`--output-paths-to-file <file>`
: Output a symbol table (.paths file) to <file>

`--output-to-stdout, --output-fzn-to-stdout`
: Print generated FlatZinc to standard output

`--output-ozn-to-stdout`
: Print model output specification to standard output

`--output-paths-to-stdout`
: Output symbol table to standard output

`--output-mode <item|dzn|json>`
: Create output according to output item (default), or output compatible with dzn or json format

`--output-objective`
: Print value of objective function in dzn or json output

`--only-sections <section1[,section2,...]>`
: Include only the given sections in output

`--not-sections <section1[,section2,...]>`
: Exclude the given sections from output

`-Werror`
: Turn warnings into errors

`--warn-non-authoritative-names`
: Warn when a library's predicate or function parameter names differ from the canonical names fixed by the standard library's body-less (native) declaration for the same signature. Off by default; intended for solver-library implementers auditing their libraries (see *Parameter names in solver-specific libraries* (page 541)).

3.1.3.4 Solution output options

These options control how solutions are output. Some of these options only apply if `minizinc` is used to translate a stream of solutions coming from a solver into readable output (using a .ozn file generated by the compiler).

`--ozn-file <file>`
: Read output specification from ozn file.

`-o <file>, --output-to-file <file>`
: Filename for generated output.

`-i <n>, --ignore-lines <n>, --ignore-leading-lines <n>`
: Ignore the first <n> lines in the FlatZinc solution stream.

`--soln-sep <s>, --soln-separator <s>, --solution-separator <s>`
: Specify the string printed after each solution (as a separate line). The default is to use the same as FlatZinc, "----------".

`--soln-comma <s>, --solution-comma <s>`
: Specify the string used to separate solutions. The default is the empty string.

`--unsat-msg (--unsatisfiable-msg)`
: Specify status message for unsatisfiable problems (default: `"=====UNSATISFIABLE====="`)

`--unbounded-msg`
: Specify status message for unbounded problems (default: `"=====UNBOUNDED====="`)

`--unsatorunbnd-msg`
: Specify status message for unsatisfiable or unbounded problems (default: `"=====UNSATorUNBOUNDED====="`)

`--unknown-msg`
: Specify status message if search finished before determining status (default: `"=====UNKNOWN====="`)

`--error-msg`
: Specify status message if search resulted in an error (default: `"=====ERROR====="`)

`--search-complete-msg <msg>`
: Specify status message if when search exhausted the entire search space (default: `"=========="`)

`--non-unique`
: Allow duplicate solutions.

`-c, --canonicalize`
: Canonicalize the output solution stream (i.e., buffer and sort).

`--output-non-canonical <file>`
: Non-buffered solution output file in case of canonicalization.

`--output-raw <file>`
: File to dump the solver's raw output (not for hard-linked solvers)

`--no-output-comments`
: Do not print comments in the FlatZinc solution stream.

`--output-time`
: Print timing information in the FlatZinc solution stream.

`--no-flush-output`
: Don't flush output stream after every line.

3.1.4 User Configuration Files

The `minizinc` tool reads a system-wide and a user-specific configuration file to determine default paths, solvers and solver options. The files are called `Preferences.json`, and you can find out the locations for your platform using the option `--config-dirs`:

```
$ minizinc --config-dirs
{
  "globalConfigFile" : "/
↪Applications/MiniZincIDE.app/Contents/Resources/share/minizinc/Preferences.json",
↪
```

```
  "userConfigFile" : "/Users/Joe/.minizinc/Preferences.json",
  "userSolverConfigDir" : "/Users/Joe/.minizinc/solvers",
  "mznStdlibDir" : "/
↪Applications/MiniZincIDE.app/Contents/Resources/share/minizinc"
}
```

The configuration files are simple JSON files that can contain the following configuration options:

- `mzn_solver_path` (list of strings): Additional directories to scan for solver configuration files.
- `mzn_lib_dir` (string): Location of the MiniZinc standard library.
- `tagDefaults` (list of lists of strings): Each entry maps a tag to the default solver for that tag. For example, `[["cp","org.chuffed.chuffed"],["mip","org.minizinc.gurobi"]]` would declare that whenever a solver with tag `"cp"` is requested, Chuffed should be used, and for the `"mip"` tag, Gurobi is the default. The empty tag (`""`) can be used to define the system-wide default solver (i.e., the solver that is chosen when running `minizinc` without the `--solver` argument).
- `solverDefaults` (list of lists of strings): Each entry consists of a list of three strings: a solver identifier, a command line option, and a value for that command line option.For example, `[["org.minizinc.gurobi","--gurobi-dll", "/Library/gurobi752/mac64/lib/libgurobi75.so"]]` would specify the Gurobi shared library to use (on a macOS system with Gurobi 7.5.2). For command line options that don't take a value, you have to specify an empty string, e.g. `[["org.minizinc.gurobi","--uniform-search",""]]`.

Here is a sample configuration file:

```
{
  "mzn_solver_path": ["/usr/share/choco"],
  "tagDefaults": [["cp","org.choco-solver.choco"],["mip",
↪"org.minizinc.mip.cplex"],["","org.gecode.gecode"]],
  "solverDefaults": [["org.minizinc.mip.cplex","--cplex-dll","/
↪opt/CPLEX_Studio128/cplex/bin/x86-64_sles10_4.1/libcplex128.so"]]
}
```

Configuration values in the user-specific configuration file override the global values, except for solver default arguments, which are only overridden if the name of the option is the same, and otherwise get added to the command line.

Note: Due to current limitations in MiniZinc's JSON parser, we use lists of strings rather than objects for the default mappings. This may change in a future release, but the current syntax will remain valid. The location of the global configuration is currently the `share/minizinc` directory of the MiniZinc installation. This may change in future versions to be more consistent with file system standards (e.g., to use `/etc` on Linux and `/Library/Preferences` on macOS).

3.1.5 Command-line Parameter Files

The `minizinc` tool is able to read command line options from one or more JSON files through use of the `--param-file` option. This option may be specified multiple times, and substitutes the options given in the JSON file in its place, meaning it can be combined with normal command-line options, as well as other `--param-file` configurations. MiniZinc also recognises the `.mpc` file extension for command line parameter files without requiring the `--param-file` option.

As most `minizinc` options which take a single value are given the final one they are assigned, later specifications of command-line parameter files will generally override options contained in previous ones, and later command-line options will override previous options.

The given file must be in JSON format and contain a root object whose keys and values correspond to command line options and their values respectively. For long-form options which start with two dashes, the leading dashes may be omitted from the JSON object key name. For short-form options which start with a single dash, the leading dash is required. Command line options which take multiple values through being specified multiple times can be represented using a JSON array. Options which take the JSON boolean value `true` are treated as flags without associated parameters (`false` omits the flag).

Consider the configuration file `config.mpc`:

```
{
  "solver": "gecode",
  "cmdline-data": ["x = 1", "y = 2", "z = 3"],
  "-p": 2,
  "output-paths": true
}
```

Running `minizinc --verbose config.mpc -p 4 model.mzn` (or `minizinc --verbose --param-file config.mpc -p 4 model.mzn`) is equivalent to running

```
$ minizinc --verbose \
    --solver gecode \
    --cmdline-data "x = 1" \
    --cmdline-data "y = 2" \
    --cmdline-data "z = 3" \
    -p 2 \
    --output-paths \
    -p 4 \
    model.mzn
```

Note that since `-p 4` comes after `-p 2`, its value overrides the previous one and so `-p 4` is used. However, the multiple `--cmdline-data` options do not override each other as `--cmdline-data` allows for specification of multiple strings of data.

Configuration files may also be used as a convenient way to specify instances.

For a configuration file `instance.mpc`

```
{
  "model": "model.mzn",
  "data": "data.dzn"
}
```

And another configuration file `gecode.mpc`

```
{
  "solver": "gecode",
  "intermediate-solutions": true
}
```

We can run the instance with `minizinc gecode.mpc instance.mpc`. Note that the files `model.dzn` and `data.dzn` will be resolved relative to the location of the configuration file which specified them.

In this way, multiple configuration files can be set up for different instances and solver configurations, then combined as required.

CHAPTER 3.2

The MiniZinc IDE

The MiniZinc IDE lets you edit and run MiniZinc models. It requires a working installation of the MiniZinc tool chain (which is included when you download the *bundled version* of the IDE). For installation instructions, see Section 1.2. This document assumes that you have installed the IDE from the bundled binary distribution, which means that it will already be configured correctly when you start it for the first time. The configuration options are described in Section 3.2.5.

Section 1.3 contains an introduction to the absolute basics of working with the IDE. This document goes into a bit more detail.

3.2.1 Editing files

The basic editor provides the usual functionality of a simple text editor. You can edit MiniZinc models (file extension `.mzn`) and data files (`.dzn`). When you first open the MiniZinc IDE, you are presented with an empty *Playground* editor, which lets you quickly try out simple MiniZinc models (it does not have to be saved to a file before running a solver on it).

Each file will be opened in a separate tab. To switch between files, click on the tab, select the file from the *Window* menu, or use the *Previous tab*/*Next tab* options from the *View* menu.

When saving a file, make sure that you select the correct file type (represented by the file extension). Models should be saved as `.mzn` files, while data files should have a `.dzn` extension. This is important because the IDE will only let you invoke the solver on model files.

3.2.1.1 Editing functions

The *Edit* menu contains the usual functions for editing text files, such as undo/redo, copy/cut/paste, and find/replace. It also allows you to jump to a particular line number (*Go to line*), and to shift the currently selected text (or the current line if nothing is selected) right or left by two spaces. The *(Un)comment* option will turn the current selection (or current line) into comments, or remove the comment symbols if it is already commented.

3.2.1.2 Fonts and dark mode

You can select the font and font size in the *View* menu. We recommend to use a fixed-width font (the IDE should pick such a font by default).

Depending on your platform, the *View* menu also lets you activate "dark mode", which switches the colour scheme to a dark background. On macOS 10.14 and later, the operating systems supports a global dark mode, and the MiniZinc IDE will follow that setting.

3.2.2 Configuring and Running a Solver

The MiniZinc IDE automatically detects which solvers are available to MiniZinc. You can select the solver to use from the solver selection drop-down menu next to the *Run* icon in the tool bar:



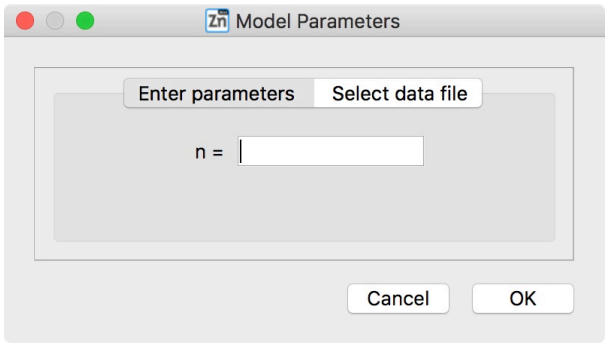
3.2.2.1 Running a model

MiniZinc models can be compiled and run by clicking the *Run* icon, selecting *Run* from the *MiniZinc* menu, or using the keyboard shortcut `Ctrl+R` (`Cmd+R` on macOS). The IDE will use the currently selected solver for compiling and running the model.

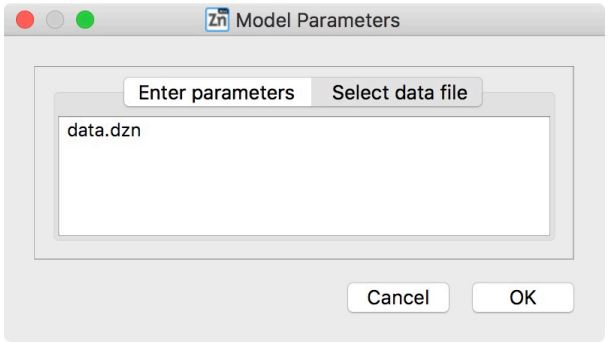
Running a model will open the *Output* window, usually located at the bottom of the IDE's main window. MiniZinc displays progress messages as well as any output received from the solver there. If compilation resulted in an error message, clicking on the error will jump to the corresponding location in the model or data file.

The current run can be aborted by clicking the *Stop* icon, selecting *Stop* from the *MiniZinc* menu, or using the keyboard shortcut `Ctrl+E` (`Cmd+E` on macOS).

If the selected model requires some parameters to be set before it can be run, the MiniZinc IDE will open a parameter dialog. It has two tabs. The left tab lets you enter the parameters manually:



The second tab lets you select one or several of the data files that are currently open:



3.2.2.2 Solver configurations

Selecting one of the built-in solvers from the drop-down menu activates its default configuration. In order to change the solver's behaviour, open the solver configuration editor by clicking on the icon in the tool bar, selecting *Show configuration editor* from the *MiniZinc/Solver configurations* menu, or using the keyboard shortcut `Ctrl+Shift+C` (`Cmd+Shift+C` on macOS).

Fig. 2.1 shows the configuration window. The first section (marked with a `1` in a red circle) contains a drop-down menu to select the *solver configuration*. In this case, a built-in configuration for the Gecode solver was selected. You can make this configuration the default (the MiniZinc IDE will remember this setting), you can reset all values to the defaults, and you can make a clone of the configuration. Cloning a configuration is useful if you want to be able to quickly switch between different sets of options. Solver configurations can be saved once cloned/modified as `.mpc` files to be reused later. This is done by selecting *Save solver configuration from the *File* menu. The concrete solver used is also shown in this section.

The *Options* section allows you configure common settings which can remain persistent as you switch between configurations. This behaviour can be switched off by unticking the *Maintain these options across solver configurations checkbox*. This area contains two subsections described below.

The *Solving* subsection contains a number of general options. You can set a time limit, after which the execution will be stopped. The behaviour selector controls which kinds of solutions a solver should produce. The "default behaviour" (marked with a `2`) is to print all intermediate solutions for optimisation problems, and stop after the first found solution for satisfaction problems. To change this, you can select *User-defined behavior* instead (marked with a `3`).

The *Output* subsection (marked with a `4`) allows you to control what extra information,

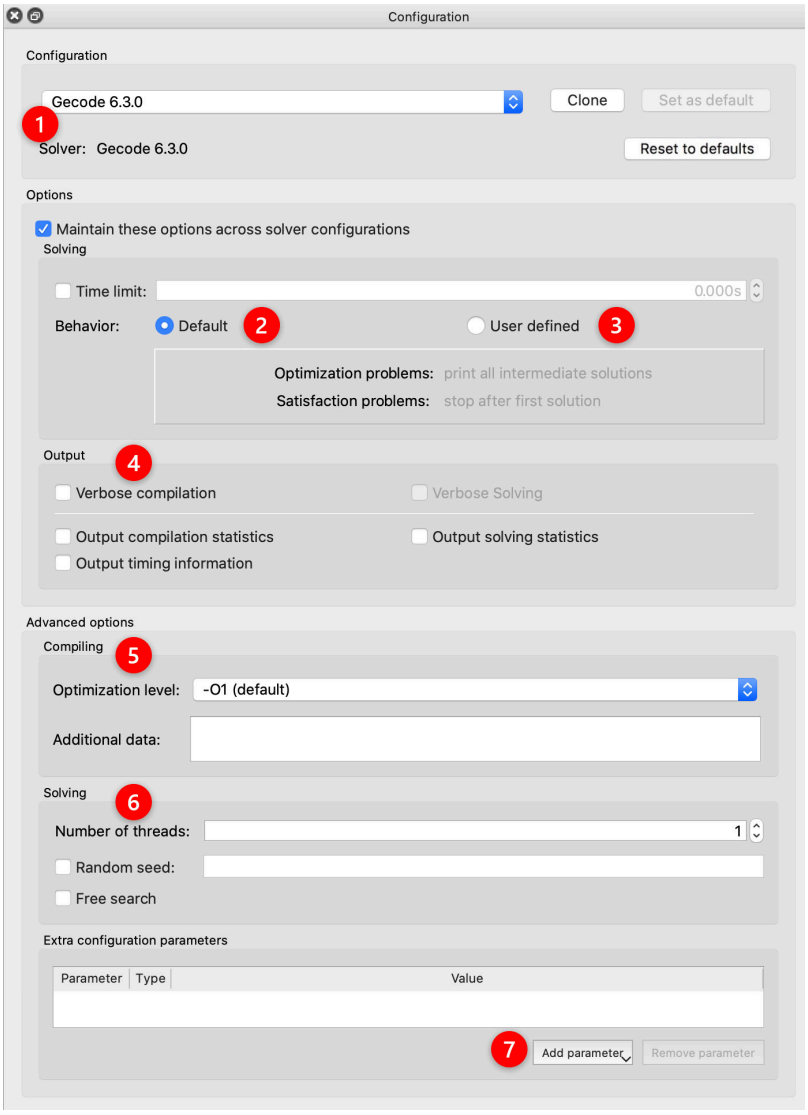


Fig. 2.1: The solver configuration window

if any, the compiler/solver should produce. This includes verbosity options, statistics options and extra timing information where supported.

The *Advanced Options* section contains options for more fine-grained control of the compilation and solving of your instance. These options are less commonly used, and as such the settings are unique to each solver configuration. It contains three subsections described below.

The *Compiling* subsection (marked with a 5) controls different aspects of the compilation from MiniZinc to FlatZinc for the selected solver. The drop-down controls the optimisation level of the compiler, i.e., how much effort it should spend on creating the best possible FlatZinc representation for the given model. The input field below allows you to specify additional data (passed into the compilation as if it was part of a `.dzn` file).

The *Solving* subsection (marked with a 6) contains configuration options for the selected solver, such as the number of threads to use for solvers which support parallelism, the seed to use for random number generation, and whether or not to enable free search (i.e. permitting the solver to ignore search annotations). Only options that are supported by the solver will be available, others will be greyed out.

Finally, the *Extra configuration parameters* subsection allows you to enter any extra options supported by a solver, as well as custom ones where necessary. These options are passed to the `minizinc` command in the same manner as described in Section 3.1.5. The *Add parameter* button (marked with a 7) opens a drop-down presenting any known options supported by a solver (with a convenience option to add all of them), as well as an option to add a custom parameter. The values associated with each added parameter can be edited by double clicking on the corresponding *Value* cell in the table. Adding a custom parameter requires you to specify the corresponding command-line flag in the first column, the data-type in the second, and the value in the third. The *Remove parameter* button can be used to remove the currently highlighted parameters.

3.2.2.3 Automatic Solution Checking

MiniZinc can automatically run the output of a model through a *solution checker*, another MiniZinc model that verifies that the solution satisfies a given set of rules. This can be useful for teaching constraint modelling, if the solution checker is given to students. Another use case is to use a simple checker while working on a more complex model, to ensure that the complex model still meets the specification.

The default behaviour of the MiniZinc IDE is to run a solution checker if one is present. For a model `abc.mzn`, a solution checker must be called `abc.mzc` or `abc.mzc.mzn`. If a checker is present, the *Run* icon will turn into a *Run + check* icon instead. The output of the solution checker is displayed together with the normal solution output in the *Output* window.

You can disable solution checkers by deselecting the *Check solutions* option in the output tab of the preferences window.

You can find a detailed description of MiniZinc solution checkers in *Automatic Solution Checking, Model Validation, and Benchmarking* (page 215).

3.2.2.4 Compiling a model

It can sometimes be useful to look at the FlatZinc code generated by the MiniZinc compiler for a particular model. You can use the *Compile* option from the *MiniZinc* menu to compile the model without solving. The generated FlatZinc will be opened in a new tab. You can edit and save the FlatZinc to a file, and run it directly (without recompiling).

3.2.2.5 Profiling a model

MiniZinc can produce statistics about the translation of a model to FlatZinc, which can be helpful in order to better understand the behaviour of the model. In the IDE, this feature can be accessed from the *MiniZinc* menu: Select *Profile compilation*, which will run the compiler with the current settings (such as selected solver etc.) and output compilation statistics next to each line of code.

For the Magic Series puzzle from Listing 7.2, profiling using the Gurobi solver backend could produce the statistics shown in Fig. 2.2. The first column ("Cons") represents the number of FlatZinc constraints generated from each line of code, the second column ("Vars") the number of variables, and the third column ("Time") the time it took to compile the line. In this example, for `n=10`, the constraint in lines 4 and 5 creates 100 variables (one 0/1 variable for each of the `bool2int` calls), and the constraints in lines 7 and 8 each result in a single FlatZinc constraint and no additional variables.

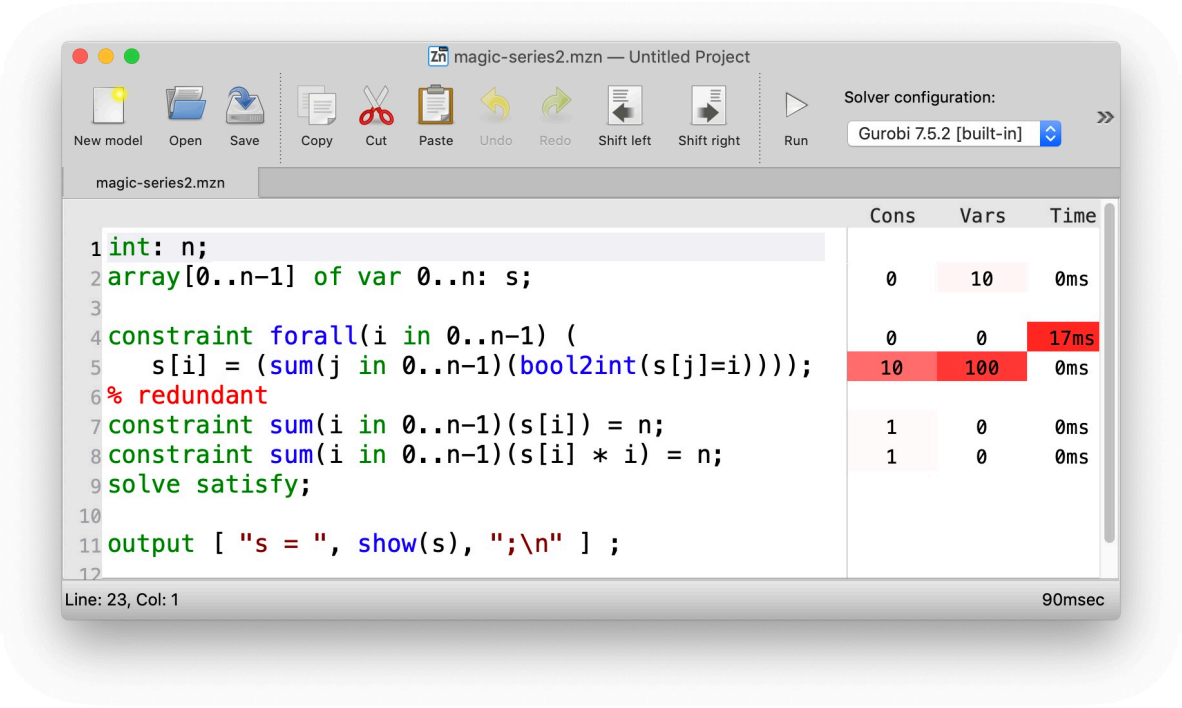


Fig. 2.2: Profiling results for the Magic Series problem (`n=10`)

In addition to the simple statistics output, the IDE also ships an experimental integration of the CP-Profiler tool. This tool provides profiling information of the actual solving process. Since this requires integration with the backend solver, this is only available for a small number of solvers at the moment. See Section 3.4 for more details.

3.2.3 Working With Projects

Each main window of the MiniZinc IDE corresponds to a *project*, a collection of files and settings that belong together. A project can be saved to and loaded from a file.

You can open a new project by selecting the *New project* option from the *File* menu, or using the `Ctrl+Shift+N` keyboard shortcut (`Cmd+Shift+N` on macOS).

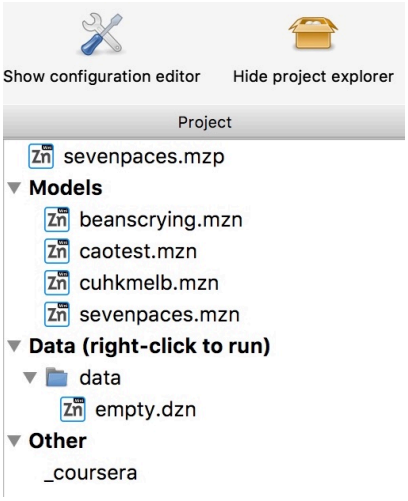


Fig. 2.3: Project explorer

All the files that belong to the current project are shown in the *Project explorer* (see Fig. 2.3), which can be opened using the tool bar icon, or using the *Show project explorer* option in the *View* menu. The project explorer lets you run the model in the currently active tab with any of the data files by right-clicking on a `.dzn` file and selecting *Run model with this data*. Right-clicking any file presents a number of options: opening it, removing it from the project, renaming it, running it, and adding new files to the project.

A saved project contains the following pieces of information:

- The names of all files in the project. These are stored as relative paths (relative to the project file).
- Which files were open in tabs (and in which order) at the time the project was saved.
- The active solver configuration.
- The state of any cloned solver configuration.

The following will *not* be saved as part of the project:

- The contents of the *Output* window.
- The state of the built-in solver configurations.

3.2.4 Submitting Solutions to Online Courses

The MiniZinc IDE has built-in support for submitting solutions and models to online courses, including the Coursera courses that introduce modelling in MiniZinc:

- Basic Modeling for Discrete Optimization[6]
- Advanced Modeling for Discrete Optimization[7]

The submission system is controlled by a file called `_mooc`, which is typically part of the projects that you can download for workshops and assignments. When a project contains this file, a new submission icon will appear in the tool bar, together with an option in the *MiniZinc* menu.

Clicking the icon (or selecting the menu option) opens the submission dialog (see Fig. 2.4). It lets you select the problems that you would like to run on your machine, after which the solutions will be sent to the online course auto-grading system. Some projects may also contain model submissions, which are not run on your machine, but are evaluated by the online auto-grader on new data that was not available to you for testing.

You will have to enter the assignment-specific login details. By clicking the *Run and submit* button, you start the solving process. When it finishes, the MiniZinc IDE will upload the solutions to the auto-grading platform.

3.2.5 Configuration Options

The MiniZinc IDE can be configured through the *Preferences* dialog in the *MiniZinc* menu (on Windows and Linux) or the *MiniZincIDE* menu (on macOS), as shown in Fig. 2.5.

There are four tabs which allow configuration of different aspects of the IDE:

- **MiniZinc**. Options for configuring compiler search paths. Shown in Fig. 2.6.
- **Solvers**. Options for built-in and user solvers. Shown in Fig. 2.7.
- **Editing**. Options for code editing and appearance. Shown in Fig. 2.9.
- **Output**. Options for solution output. Shown in Fig. 2.10.

3.2.5.1 Locating the MiniZinc installation

The most important configuration option is the path to the `minizinc` executable. In the example in Fig. 2.5, this field has been left empty, in which case `minizinc` is assumed to be on the standard search path (usually the `PATH` environment variable). Typically, in a bundled binary installation of MiniZinc, this field can therefore be left empty.

If you installed MiniZinc from sources, or want to switch between different versions of the compiler, you can add the path to the directory that contains the `minizinc` executable here. You can select a directory from a file dialog using the *Select* button, or enter it manually. Clicking the *Check* button will check that `minizinc` can in fact be run, and has the right version. The version of `minizinc` that was found is displayed below the path input field. Fig. 2.6 below shows an example where MiniZinc is located at `/home/me/minizinc-2.6.0/bin`.

You can have the MiniZinc IDE check once a day whether a new version of MiniZinc is available.

[6] https://www.coursera.org/learn/basic-modeling
[7] https://www.coursera.org/learn/advanced-modeling

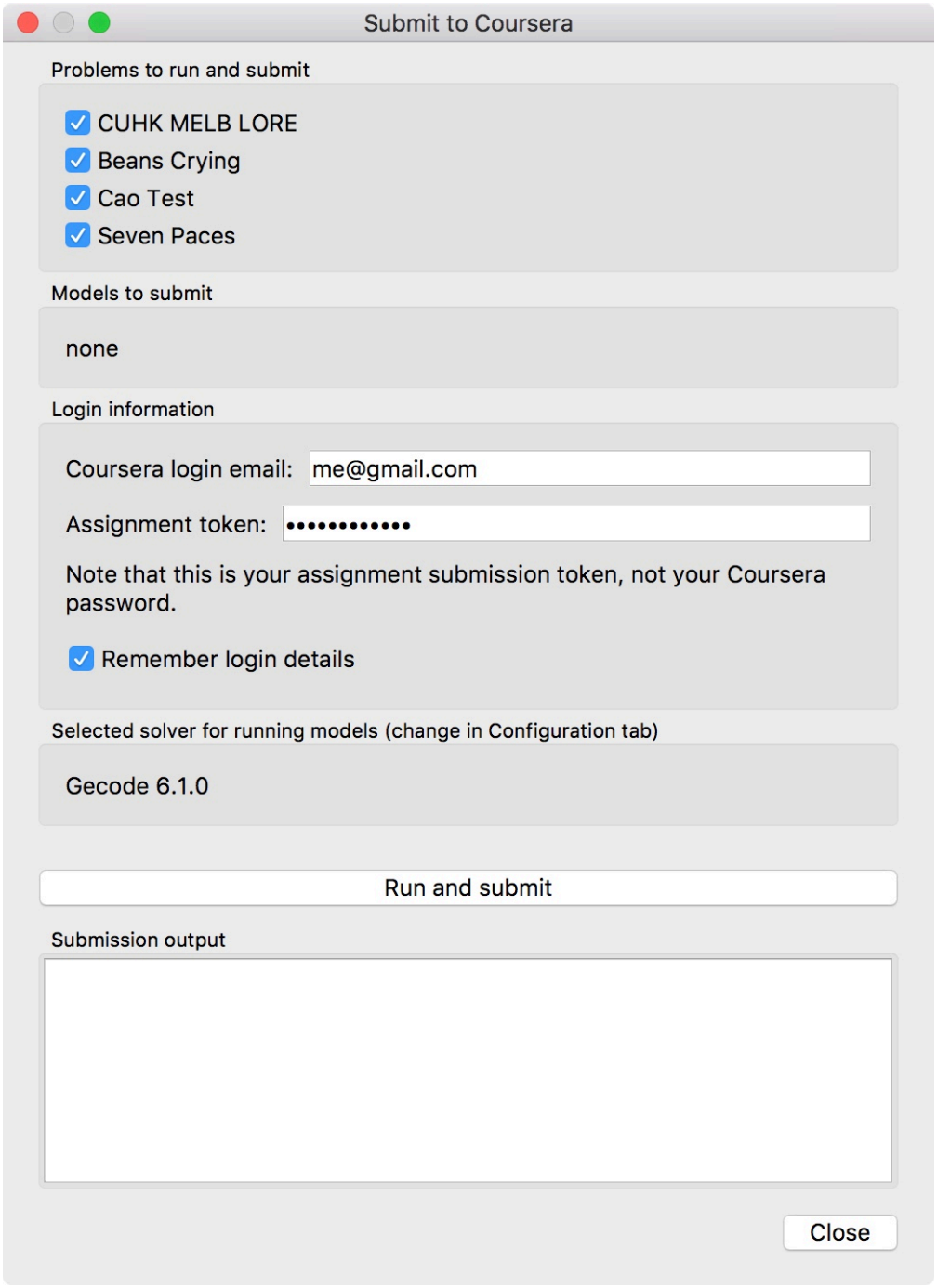


Fig. 2.4: Submitting to Coursera

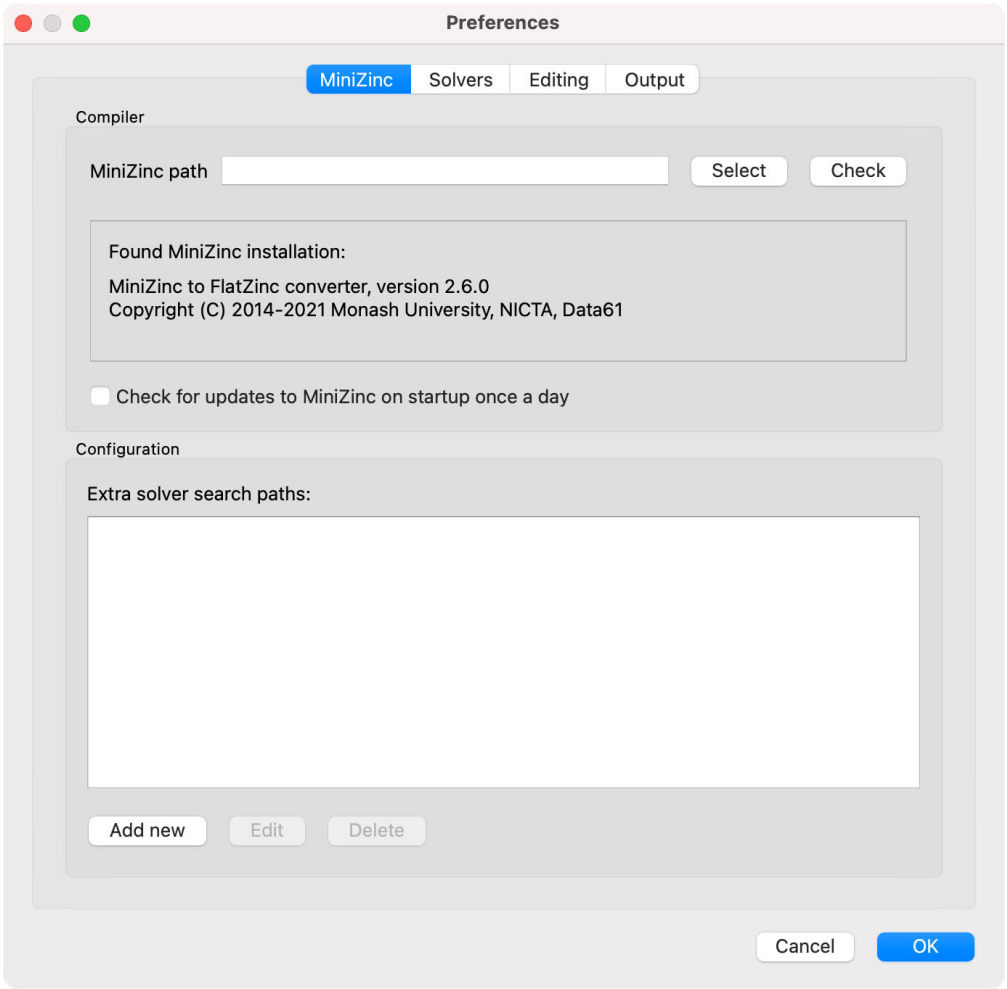


Fig. 2.5: The MiniZincIDE preferences window

3.2.5.2 Adding Third-Party Solvers

Solvers which distribute a **.msc** solver configuration file (see Section 4.3.6) can be added to the MiniZinc IDE by adding the directory containing the solver configuration file to the **Extra solver search paths** as shown in Fig. 2.6.

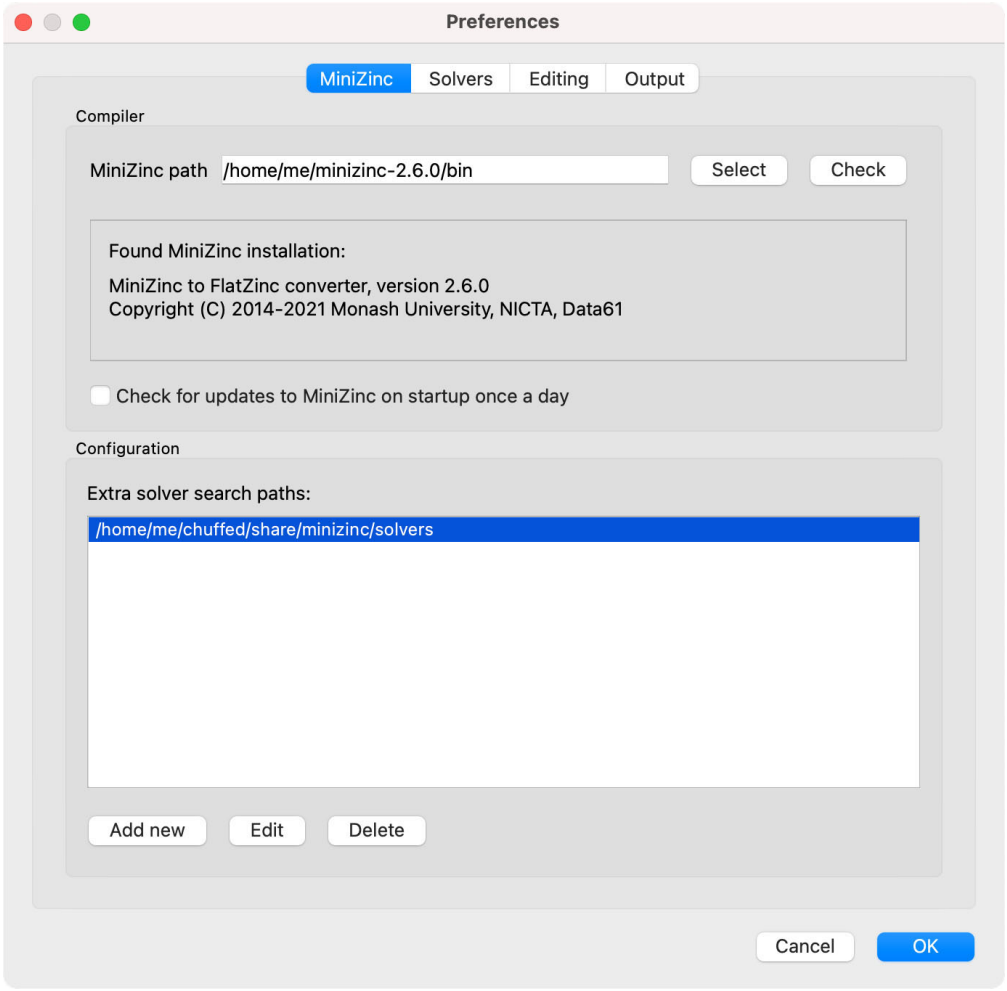


Fig. 2.6: Configuring MiniZinc search paths

The *Solvers* tab of the configuration dialog can be used to inspect the solvers that are currently available to MiniZinc, and to add new solvers which don't have an existing solver configuration file to the system.

Configuring existing solvers

You can use the configuration dialog to set up defaults for the installed solvers. In the current version of the MiniZinc IDE, this is limited to configuring the CPLEX and Gurobi backends. The bundled binary version of MiniZinc comes with support for loading CPLEX and Gurobi as *plugins*, i.e., MiniZinc does not ship with the code for these solvers but can load them dynamically if they are installed.

For example, Fig. 2.7 shows a potential configuration for Gurobi. On Windows, the library is called `gurobiXX.dll` (in the same directory as the `gurobi` executable), and on Linux and macOS is it `libgurobiXX.so` (in the `lib` directory of your Gurobi installation), where `XX` stands for the version number of Gurobi.

If you select the CPLEX solver, a similar option appears (`--cplex-dll`). On Windows, the CPLEX library is called `cplexXXXX.dll` and typically found in same directory as the `cplex` executable. On Linux it is `libcplexXXX.so`, and on macOS `libcplexXXXX.jnilib`, where `XXX` and `XXXX` stand for the version number of CPLEX.

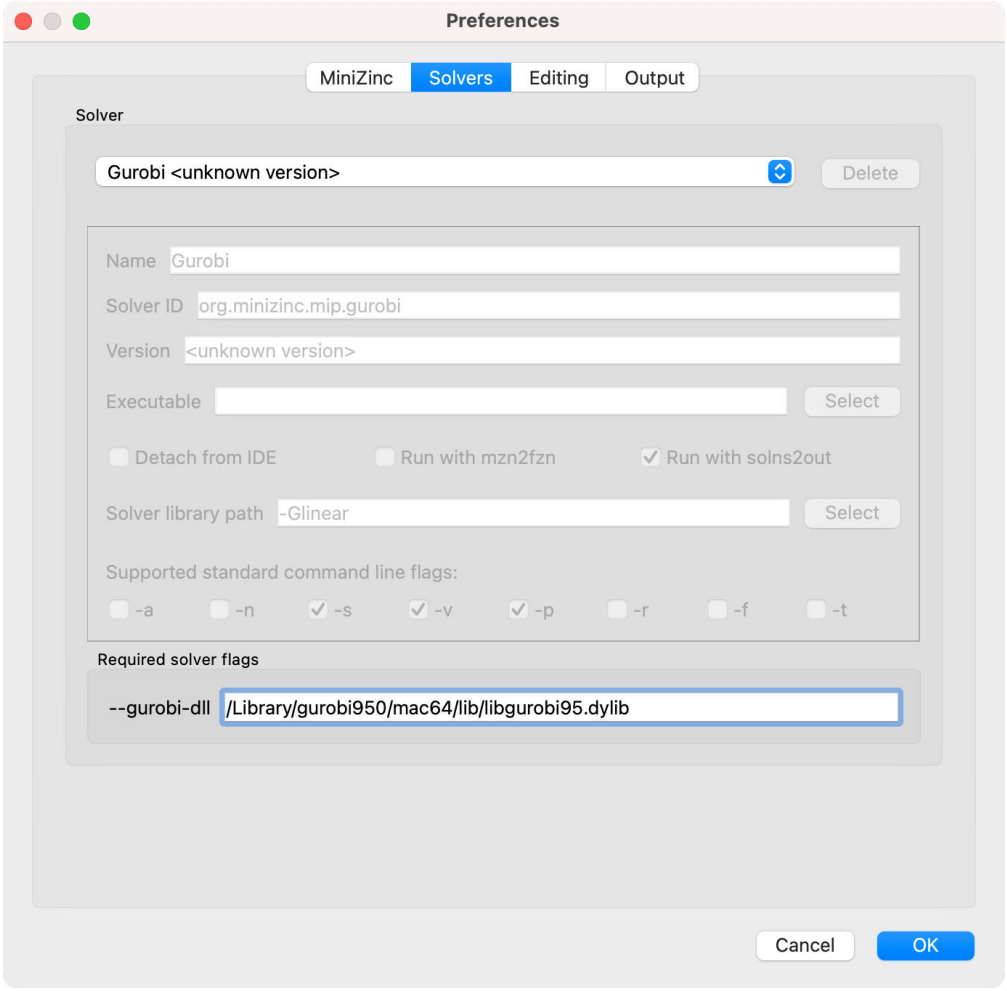


Fig. 2.7: Configuring the Gurobi DLL path

Adding new solvers

The example in Fig. 2.8 shows a potential configuration for the or-tools solver, which was installed in `/Users/me/or-tools`.

Each solver needs to be given

- a name;
- a unique identifier (usually in reverse domain name notation);
- a version string; and
- the executable that can run FlatZinc.

In addition, you can specify the location of a solver-specific MiniZinc library (see Section 4.3.4 for details). If you leave this field empty, the MiniZinc standard library will be used. The path entered into this field should be an absolute path in the file system, without extra quoting, and without any command line arguments (such as `-I`).

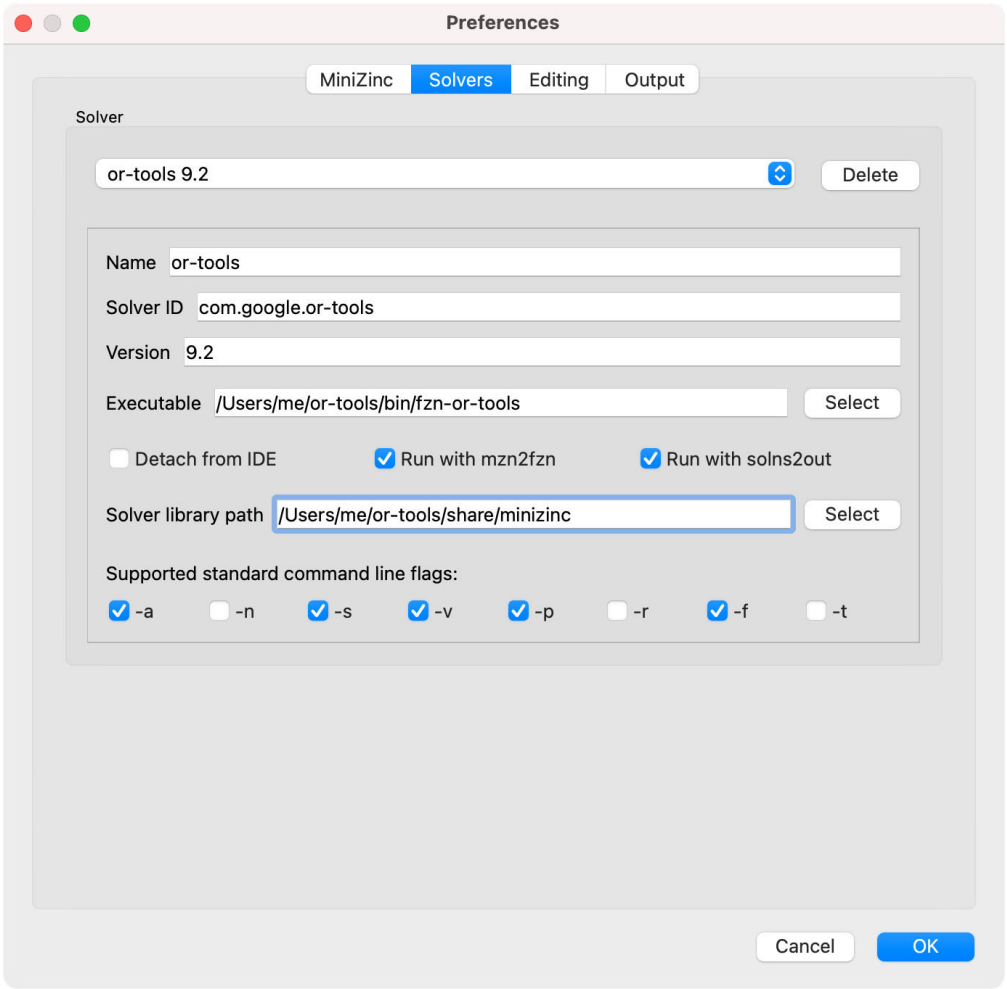


Fig. 2.8: Adding a new solver

Most solvers will require compilation and output processing, since they only deal with FlatZinc files. For these solvers, the *Run with mzn2fzn and solns2out* option must be selected. For solvers that can deal with MiniZinc models natively, this option can be deselected.

Some solvers open an independent application with its own graphical user interface. One such example is the *Gecode (Gist)* solver that comes with the bundled version of the IDE. For these solvers, select the *Run solver detached from IDE* option, so that the IDE does not wait for solver output.

Finally, you can select which command line flags are supported by the solver. This controls which options will be available in the solver configuration window.

Solver configurations that are edited or created through the IDE are saved in a configuration file in a standard location. These solvers are therefore available the next time the IDE is started, as well as through the `minizinc` command line tool.

3.2.5.3 Editing options

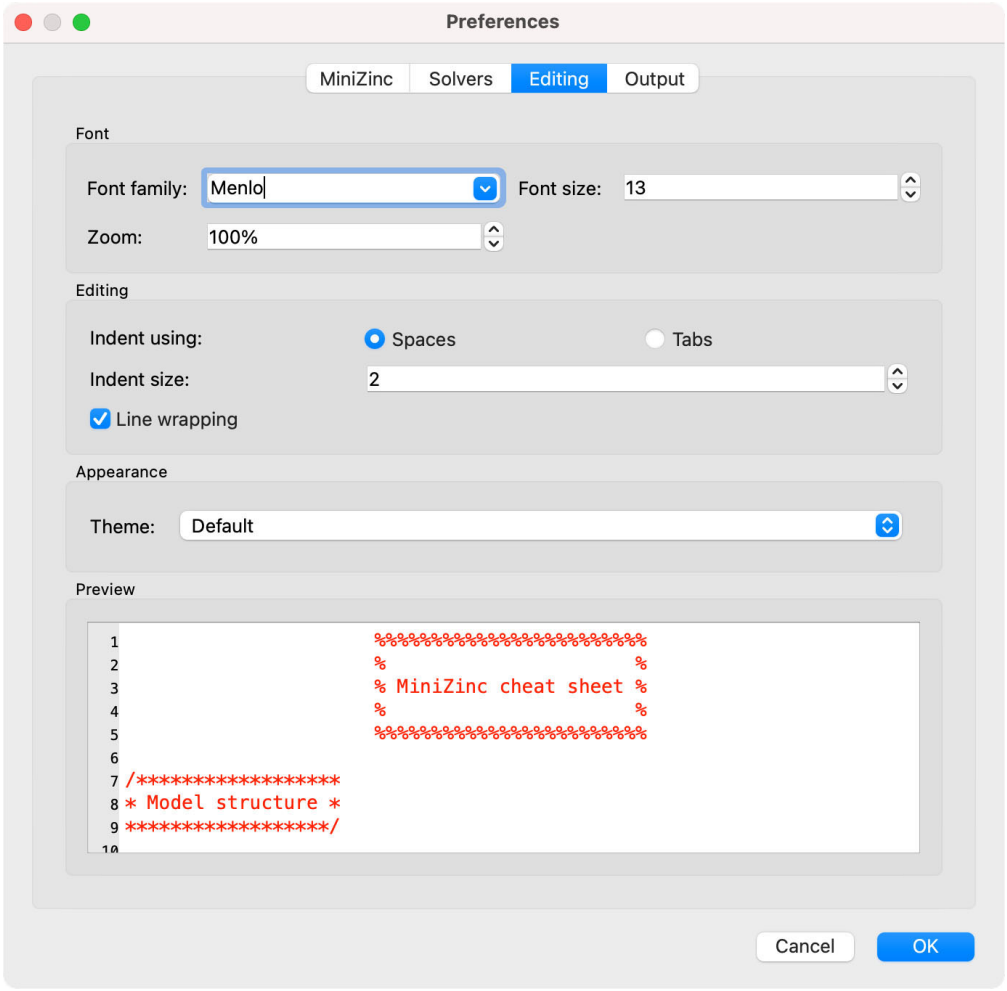


Fig. 2.9: Configuring code editor preferences

The **Editing** tab contains options for the code editor, such as fonts, indentation, and theming.

3.2.5.4 Output options

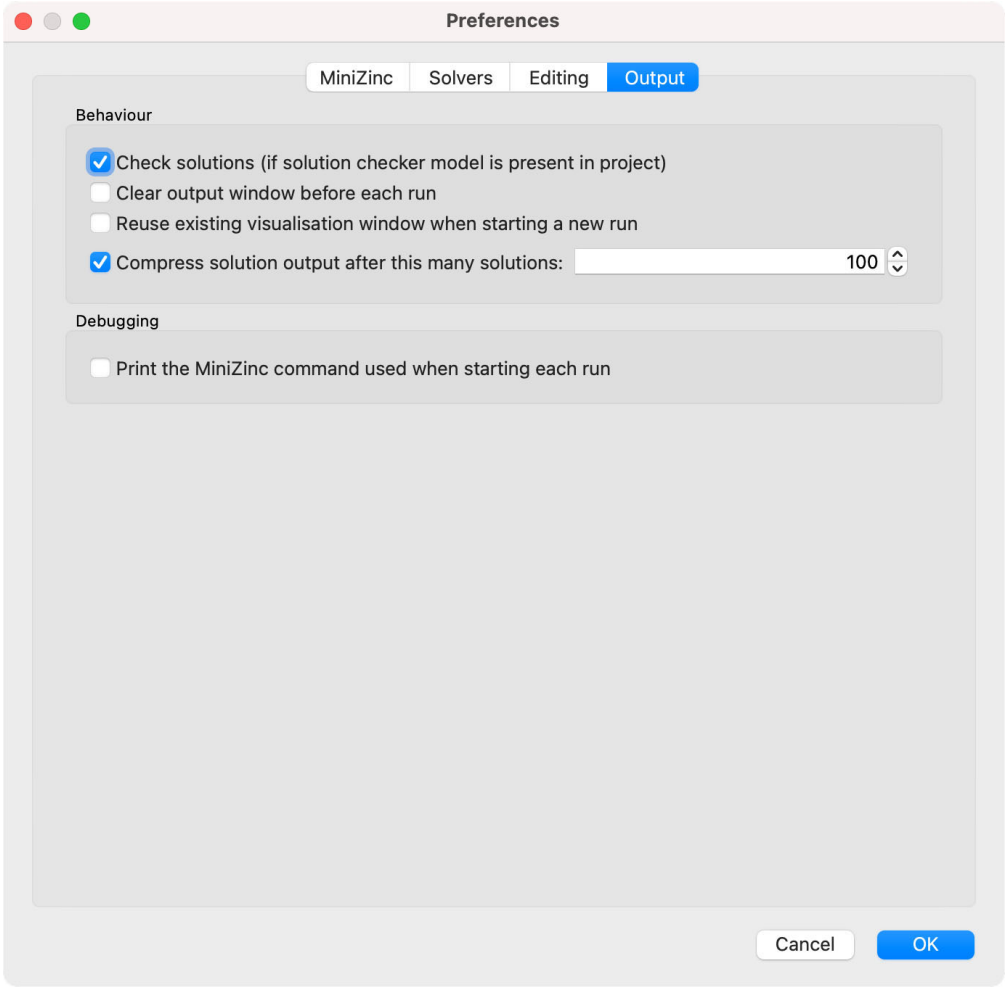


Fig. 2.10: Configuring output settings

There are also a number of output-related options. The first tick box (*Check solutions*) enables checking as described in Section 3.2.2.3. The second tick box enables you to clear the *Output* window automatically every time you run a model. The third *Compress solution output* option is useful for problems that produce a lot (read: thousands) of solutions, which can slow down and clutter the output window. The compression works by printing just the number of solutions rather than the solutions themselves. For example, the following model would produce 1000 solutions when run with *User-defined behavior* and solution limit disabled:

```
var 1..1000: x;
solve satisfy;
```

When running with compression set to 100, MiniZinc will output the first 100 solutions, and then a sequence of output like this:

```
  [ 99 more solutions ]
x = 200;
----------
```

```
  [ 199 more solutions ]
x = 400;
----------
  [ 399 more solutions ]
x = 800;
----------
  [ 199 more solutions ]
x = 1000;
----------
==========
```

The number of solutions captured by one of the `... more solutions` lines is doubled each time, in order to keep the overall output low. The last solution produced by the solver will always be printed (since, in the case of optimisation problems, the last solution is the best one found). The hidden solutions can be shown by clicking the symbol, but this may take some time where a large number of solutions have been truncated.

CHAPTER 3.3

Visualising solutions in the MiniZinc IDE

The MiniZinc IDE provides a web-browser based system for visualising solutions generated by MiniZinc.

3.3.1 Visualisation library

In order to use these features, the library file `ide/vis.mzn` must be included:

```
include "ide/vis.mzn";
```

Running a model with a visualisation will automatically launch a web browser page containing the visualisation. If this page is closed, it can be reopened by clicking the hyperlink on the right side of the run's output.

3.3.2 Pre-defined visualisations

There are a number of pre-defined visualisations to suit common use cases, such as line graphs, scatter plots, bar charts, graph visualisation, and 2D packing visualisation. Multiple visualisations can be shown concurrently.

These come in the form of functions which used as the value of an `output` item.

Documentation for these is available in *MiniZincIDE solution visualisation tools* (page 502).

An example model with a visualisation for a graph along with a time series line graph of the objective value is shown in Listing 3.1.

Listing 3.1: Minimum spanning tree data visualisation (`vis_mst.mzn`).

```
include "globals.mzn";
include "ide/vis.mzn";

% Graph definition
enum NODE = {A, B, C, D, E, F, G, H, I, J};
array [int] of NODE: from = [A, A, A, B, B, B, B, C, C, D, D, E,  E, F, F, F,
↪ G,  G, H, H, I];
array [int] of NODE:   to = [B, C, E, C, D, E, G, D, F, F, G, G,  J, G, H, I,
↪ I,  J, I, J, J];
array [int] of int:     w = [3, 6, 9, 4, 2, 9, 9, 2, 9, 9, 8, 8, 18, 7, 4, 5,
↪ 9, 10, 1, 4, 3];

% Find the minimum spanning tree of the graph
var 0..sum(w): total_weight;
array [index_set(from)] of var bool: es;
constraint weighted_spanning_tree(
  card(NODE),   % Number of nodes
  length(from), % Number of edges
  from,         % Edge from node
  to,           % Edge to node
  w,            % Weight of edge
  es,           % Whether edge is in spanningtree
  total_weight  % Total weight of spanning tree
);
solve minimize total_weight;

% Graph visualisation
array [int] of string: edge_labels = [i: show(w[i]) | i in index_set(w)];
output vis_graph_highlight(
  from,                  % Edge from node
  to,                    % Edge to node
  edge_labels,           % Edges are labelled with their weights
  [i: true | i in NODE], % All nodes are present in a spanning tree
  es                     % Whether edge is in spanningtree
);

% Objective visualisation
output vis_line(total_weight, "Spanning tree weight");
```

3.3.3 Custom visualisations

A user-defined visualisation can be created by making use of the visualisation JavaScript API from an HTML page.

The visualisation is setup using an `output` item annotated with `vis_server`:

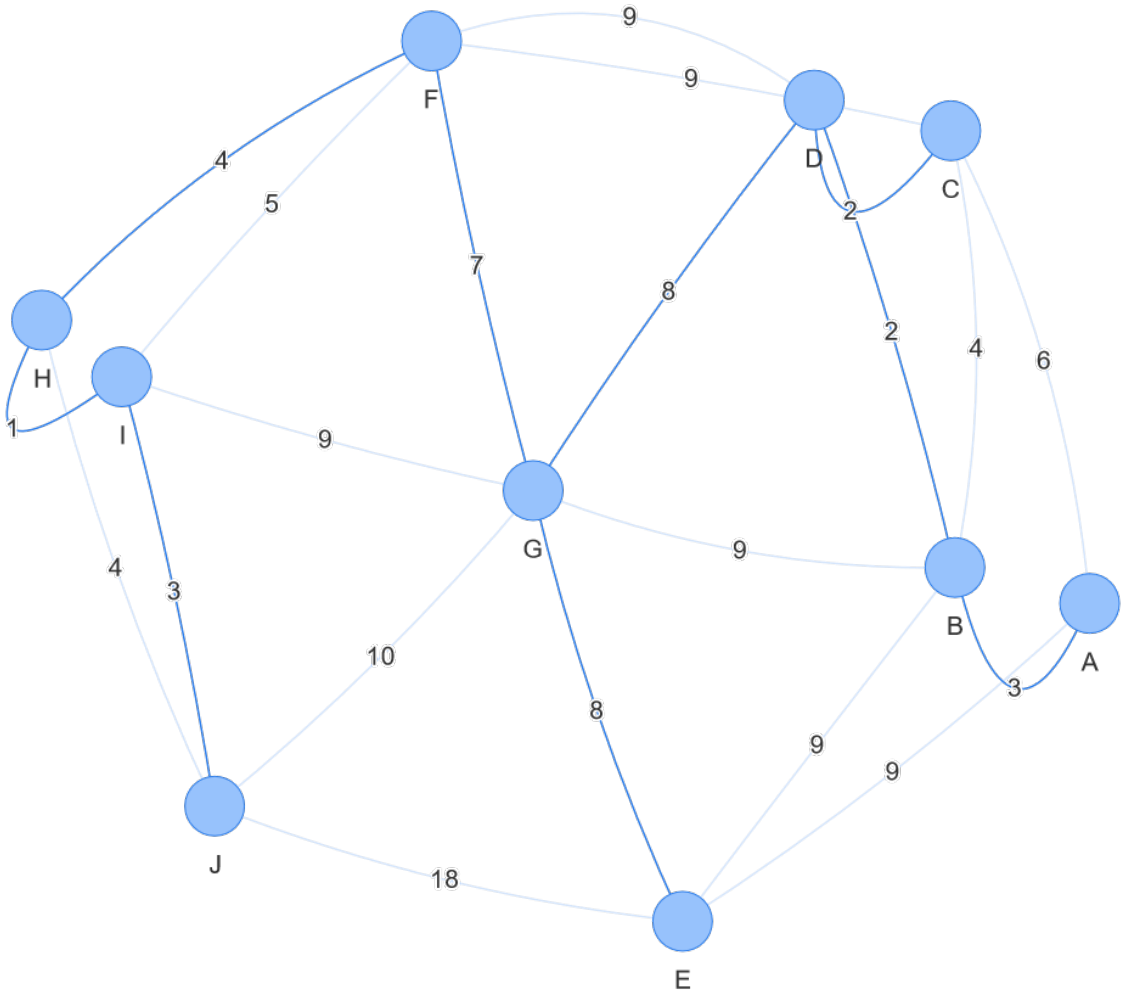


Fig. 3.1: Minimum spanning tree visualisation result

```
any: init_data = (
  size: 10,
  label: "My custom visualisation"
);
var 1..3: x;
var 1..3: y;
any: solution_data = (
  x: x,
  y: y
);
output :: vis_server("my-vis.html", init_data) solution_data;
```

The call to `vis_server` takes a path to an HTML file to load (relative to the model), and can optionally be given some initialisation user data (`init_data`) that is sent upon loading the page. Then, on each solution, the output expression (`solution_data`) is sent to the page.

Note that the initial user data and the solution data are sent as JSON objects, so it is generally convenient to use records (see *Tuple and record types* (page 91)), and you may wish to do some processing either in

Multiple such output items can be used to start concurrent visualisations.

The HTML page is loaded in an embedded web server and has access to the MiniZincIDE JavaScript API, served at `/minizinc-ide.js`. The HTML page can use this API to subscribe to events from the IDE and update its visualisation accordingly.

3.3.3.1 Example visualisation

In this example, we will create a visualisation for a version of the colouring problem introduced in *Basic Modelling in MiniZinc* (page 21).

All files should be placed in the same directory so that they are properly served by the IDE's embedded web server.

Listing 3.2: Australia colouring data visualisation model (`vis_aust.mzn`).

```
include "ide/vis.mzn";

int: n_colors;

enum PALETTE = { red, lime, blue, cyan, magenta, yellow, orange };
set of PALETTE: COLOR = to_enum(PALETTE, 1..n_colors);

% Color for each region
var COLOR: wa;
var COLOR: nt;
var COLOR: sa;
var COLOR: q;
var COLOR: nsw;
var COLOR: v;
var COLOR: t;
```

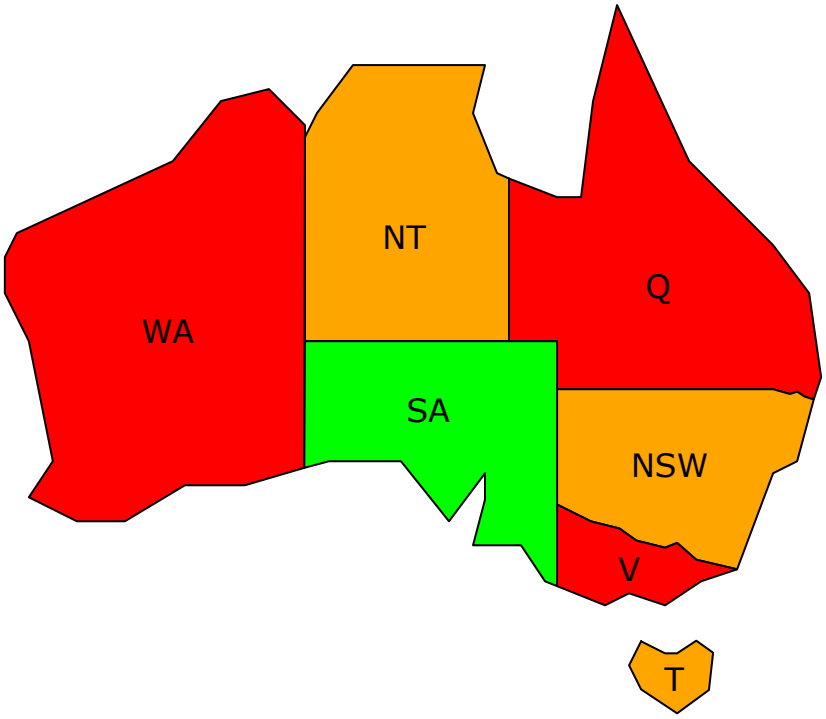


Fig. 3.2: Australia colouring visualisation result

```
% Neighboring regions have different colours
constraint wa != nt;
constraint wa != sa;
constraint nt != sa;
constraint nt != q;
constraint sa != q;
constraint sa != nsw;
constraint sa != v;
constraint q != nsw;
constraint nsw != v;

% Data to send during initialisation of the visualisation
any: initial_data = (n: n_colors);

% Data to send on each solution
% Note the use of :: output_only
any: solution_data :: output_only = (
  wa: show(wa), % Get colors as strings
  nt: show(nt),
  sa: show(sa),
  q: show(q),
  nsw: show(nsw),
  v: show(v),
  t: show(t)
);
% Launch the visualisation
output :: vis_server("vis_aust.html", initial_data) solution_data;
```

The MiniZinc model contains an output item annotated with *vis_server* (page 512). This initialises the visualisation, telling the IDE what page to load, and the user data to pass to it. In this case, we are passing the input number of colours so it can be shown on the visualisation.

The solution data is given as a record mapping regions to their string color names. If we didn't convert these to strings, we would have to extract their names from the JSON enum output format in the visualisation script.

Passing JSON data to the visualisation

It should be noted that `showJSON()` is used on the user/solution data to convert them into JSON for the visualisation script. While this supports all MiniZinc values including sets and enums, the format used may need special handling in the visualisation script.
In many cases, it may be easier to convert sets into arrays and enums into strings to make them easier to deal with. For more information about the JSON format generated, see *JSON support* (page 333).
Another important thing to note is that when you place the solution data in its own declaration, you will likely need annotate it with `::output_only` to ensure that does not get evaluated during compilation.

Listing 3.3: Australia colouring data visualisation HTML page (`vis_aust.html`).

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Australia Visualisation</title>
    <script src="/minizinc-ide.js"></script>
    <style>
      html, body {
        margin: 0;
        padding: 0;
        overflow: hidden;
        font-family: Helvetica, sans-serif;
      }
      * {
        box-sizing: border-box;
      }
      main {
        width: 100vw;
        height: 100vh;
        padding: 50px;
      }
      svg {
        width: 100%;
        height: 100%;
      }
      .shapes {
        fill: none;
```

```
        stroke: #000;
        stroke-linejoin: round;
        stroke-linecap: round;
        stroke-width: 1;
      }
      .labels {
        font-family: sans-serif;
        font-size: 16px;
        text-anchor: middle;
        fill: #000;
      }
    </style>
  </head>
  <body>
    <main>
      <h3>Map coloring with <span id="count">n</span> colors</h3>
      <svg viewBox="0 0 412 357.33">
        <g class="shapes">
          <path id="wa" d="m152 67.333v-6l-18-18-24 6-24 30-78 36-6 12v18l12␣
↪24 12 60-12 18 24 12h24l30-18h30l29.68-8.72z" />
          <path id="nt" d="m152 67.333 6-12 18-24h66l-6 24 12 30 6␣
↪2.72v81.28h-102v-102" />
          <path id="q" d="m254 88.053 24 9.28h12l6-48 12-48 36 78 42 42 18␣
↪24 6 42-3.6 10.8h-1.2l-3.6-1.2-3.6-2.4-3.6␣
↪1.2-8.4-2.4h-108v-24h-24v-81.28" />
          <path id="nsw" d="m406.4 198.13-8.4 31.2-12 6-18␣
↪48-20.4-4.8-9.6-8.4-6 2.4-14.4-3.6-8.4-6-14.4-3.6-16.8-8.4v-57.6h108l8.4␣
↪2.4 3.6-1.2 3.6 2.4 3.6 1.2z" />
          <path id="v" d="m368 283.33-20.4-4.8-9.6-8.4-6␣
↪2.4-14.4-3.6-8.4-6-14.4-3.6-16.8-8.4v40.8l24 9.6 12-6 18 6 18-12z" />
          <path id="sa" d="m278 169.33h-126l-0.32 63.28 12.32-3.28h36l24 30␣
↪18-24v13.2l-6 22.8h24l12 18 6 2.4v-122.4" />
          <path id="t" d="m320 319.33 12 6h6l9.6-6.32 8.4 6-2.08 18.64-15.92␣
↪11.68-18-12-6-12 6-12" />
        </g>
        <g class="labels">
          <text x="83.533119" y="170.0192">WA</text>
          <text x="201.66409" y="123.36276">NT</text>
          <text x="328.74969" y="147.87715">Q</text>
          <text x="334.34839" y="237.23608">NSW</text>
          <text x="314.19815" y="289.03265">V</text>
          <text x="336.71103" y="344.3877">T</text>
          <text x="213.65285" y="209.55853">SA</text>
        </g>
      </svg>
    </main>
    <script src="vis_aust.js"></script>
  </body>
</html>
```

The visualisation page includes the script `/minizinc-ide.js`, which is provided by the embedded web server. This allows access to the MiniZinc IDE JavaScript API using the global `MiniZincIDE` object. The page contains an SVG element containing the basic parts of the visualisation, ready to be coloured when a solution is produced. Another script, `vis_aust.js` is loaded with the visualisation logic.

Listing 3.4: Australia colouring data visualisation script (`vis_aust.js`).

```
const HTML_COLORS = ['red', 'yellow', 'blue', 'lime', 'magenta', 'cyan',
↪'orange'];

(async function() {
  // getUserData can be used to retrieve the JSON data passed to vis_server()
  const userData = await MiniZincIDE.getUserData();
  document.getElementById('count').textContent = userData.n;

  // Handler to set the colors for the solution
  function setSolution(data) {
    for (const r in data) {
      document.getElementById(r).setAttribute('fill', data[r]);
    }
  }

  // Visualise last solution on startup
  const numSols = await MiniZincIDE.getNumSolutions();
  if (numSols > 0) {
    const solution = await MiniZincIDE.getSolution(numSols - 1);
    setSolution(solution.data);
  }

  // Show new solutions if we're following the latest solution
  let followLatest = true;
  MiniZincIDE.on('solution', (solution) => {
    if (followLatest) {
      setSolution(solution.data);
    }
  });

  MiniZincIDE.on('goToSolution', async (index) => {
    // Requesting index -1 turns on following latest solution
    // Otherwise, we stop showing the latest solution and show the requested␣
↪one
    followLatest = index === -1;
    const solution = await MiniZincIDE.getSolution(index);
    setSolution(solution.data);
  })
})();
```

The script uses an asynchronous immediately invoked function expression, allowing us to easily use the asynchronous functions in the MiniZincIDE JavaScript API. `MiniZincIDE.getUserData()` allows us to retrieve the data we passed to `vis_server`. The

function `setSolution(data)` is defined to handle the solution data and update the visualisation. We load the latest solution using `MiniZincIDE.getSolution(idx)` if one exists on startup, in case the visualisation page is refreshed or open after solving has begun. The `solution` and `goToSolution` events are subscribed to, so we know when to update the visualisation.

3.3.3.2 MiniZincIDE JavaScript API

The MiniZincIDE JavaScript API provides an asynchronous interface between the IDE and visualisations.

```
MiniZincIDE.on(event, callback)
```

Subscribes to the event `event` and call `callback` whenever it occurs. The valid events are:

- `solution`: triggered when a new solution is produced. The callback function is passed an object containing the `time` of the solution in milliseconds and the `data` from the associated `vis_json` output statement.

  ```
  MiniZincIDE.on('solution', function(solution) {
    console.log(solution.time);
    console.log(solution.data);
  });
  ```

- `goToSolution`: triggered when the user has selected a particular solution to visualise. The callback function is passed the solution index to display, or `-1` if the user has enabled following the latest solution. Visualisations which support showing a particular solution should subscribe to this event.

  ```
  MiniZincIDE.on('goToSolution', async function(index) {
    const solution = await MiniZincIDE.getSolution(index);
    visualiseSolution(solution);
  });
  ```

- `status`: triggered when a final status line is received (such as `==========`). The callback function is passed an object containing the `status` and `time` in milliseconds. The `status` value can be one of
  - `ALL_SOLUTIONS`
  - `OPTIMAL_SOLUTION`
  - `UNSATISFIABLE`
  - `UNSAT_OR_UNBOUNDED`
  - `UNBOUNDED`
  - `UNKNOWN`
  - `ERROR`
- `finish`: triggered when solving finishes (or is stopped). The callback function is passed the time when solving finished in milliseconds.

```
MiniZincIDE.off(event, callback)
```

Unsubscribes from an event given the `event` name and `callback` function.

```
MiniZincIDE.getUserData()
```

Returns a `Promise` which resolves to the user data passed to `ide_launch_server()`.

```
MiniZincIDE.goToSolution(idx)
```

Requests that the solution with the given index `idx` is shown. This causes the `goToSolution` event to be triggered for all visualisations being shown. Visualisations which allow multiple solutions to displayed simultaneously can call this function when the user interacts on a particular solution. This allows other visualisations to then also show the solution of interest.

The index `-1` is used to request that the latest solution be tracked by the visualisations.

```
MiniZincIDE.solve(modelFile, dataFiles, options)
```

Requests that the IDE stop solving the current instance and start solving with the given `modelFile`, `dataFiles`, and parameter configuration `options` `modelFile` and `dataFiles` may be set to `null` to re-solve using the current model and data files. Otherwise, `modelFile` is the path to the `.mzn` file to solve and `dataFiles` is an array of paths to the `.dzn` or `.json` files. These must be in the same directory or a child directory of the current model. `options` is an optional object in the parameter configuration file format (see *Command-line Parameter Files* (page 161)) to be passed to MiniZinc (in addition to the current solver configuration).

```
MiniZincIDE.getNumSolutions()
```

Returns a `Promise` which resolves to the number of solutions which have been found.

```
MiniZincIDE.getSolution(index)
```

Returns a `Promise` which resolves to the solution object (containing `time` and `data`) for the given index. Negative indexes refer to indexes from the end (e.g. `-1` refers to the latest solution).

```
MiniZincIDE.getAllSolutions()
```

Returns a `Promise` which resolves to an array of all solution objects (each containing `time` and `data`).

```
MiniZincIDE.getStatus()
```

Returns a `Promise` which resolves to an object containing the `time` and final `status`, or `null` if there is no final status.

```
MiniZincIDE.getFinishTime()
```

Returns a `Promise` which resolves to the finishing time, or `null` if it has not finished.

CHAPTER 3.4

CP-Profiler

CP-Profiler provides search tree visualisations for executions of constraint programming solvers from within the MiniZinc IDE.

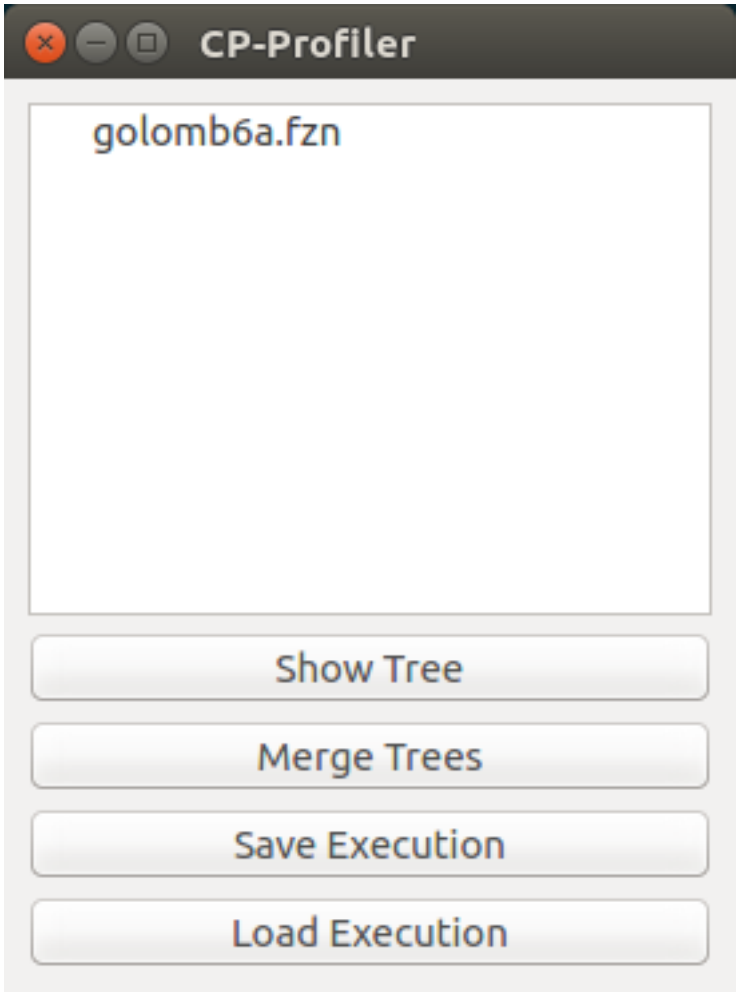
This integration is currently experimental.

3.4.1 Using the profiler

The profiler can be activated by selecting *Show search profiler* from the *View* menu, or by starting a solver with profiling enabled through the *Profile search option in the *Run* dropdown button menu. This starts a local TCP server listening on one of the available ports (6565 by default) which can be seen in at the bottom of the profiler window.

The solver must implement the profiling protocol (see below). Integration libraries are available if you wish to extend your solver to work with CP-Profiler.

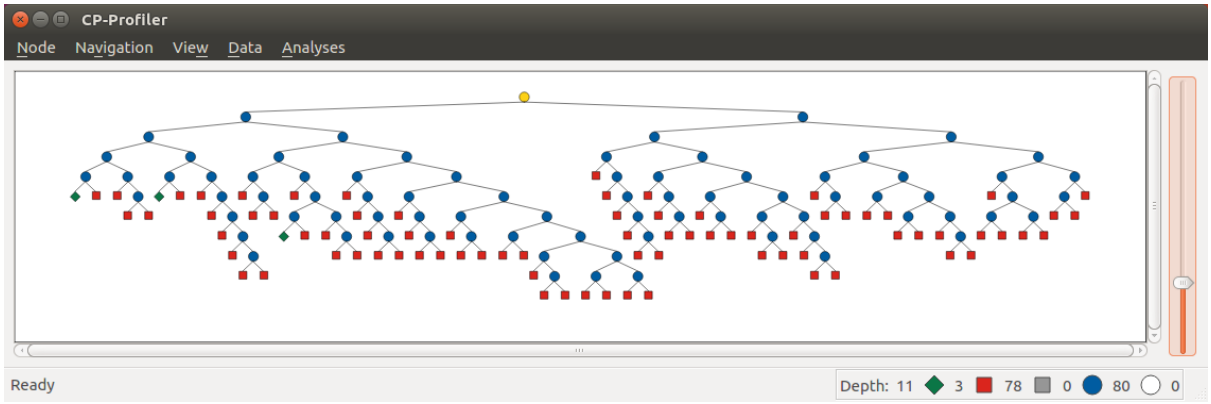
When a new execution is connected to the profiler it will be added to the list of executions displayed at the top of the profiler window. For example, in the image below execution *golomb6a.fzn* is shown to be added to the profiler. To display the execution (traditional visualisation), select its name from the list and click the *Show Tree* button. Note that the solver can still be running the execution, in which case the profiler will draw the search tree in real time.



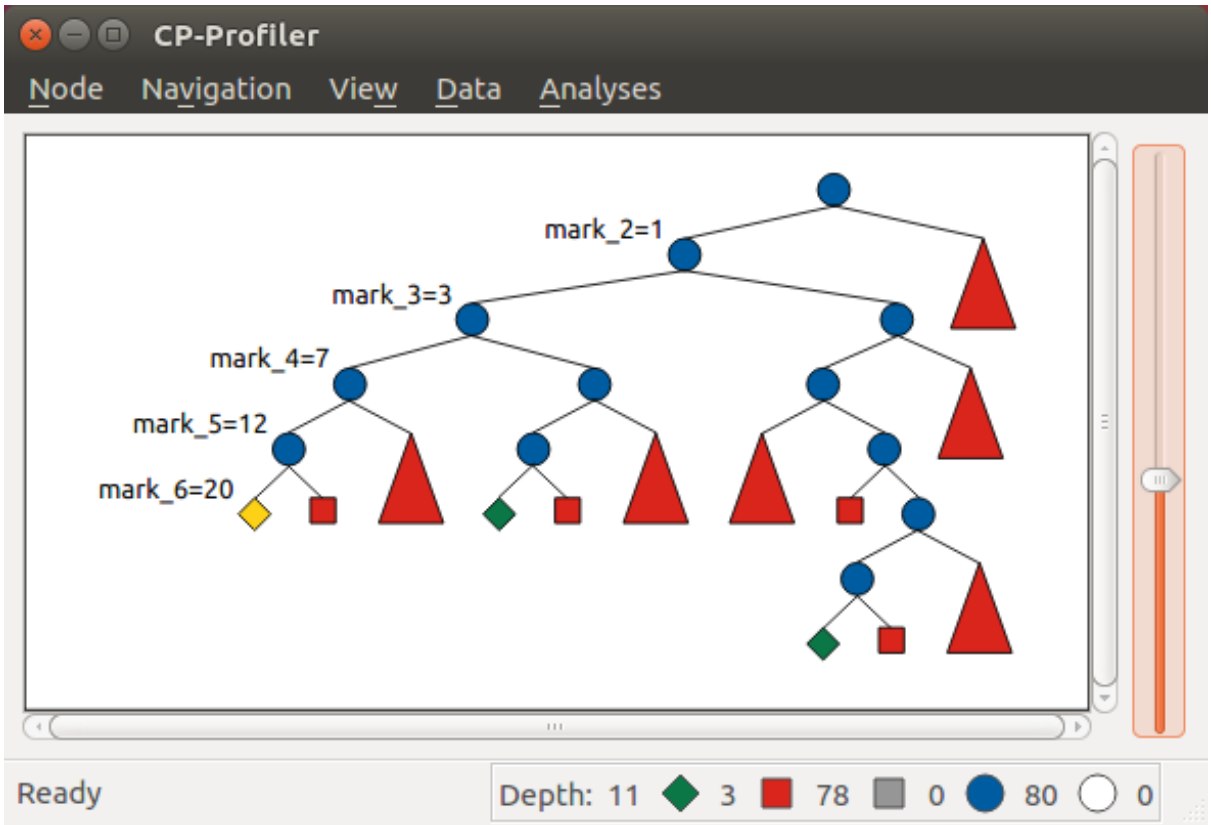
3.4.2 Traditional Tree Visualisation

The image below shows an example of a traditional (node-link) visualisation of the search tree. Different types of nodes are shown differently: branch (internal) nodes are shown as blue circles; nodes representing failures are shown as red squares; solution nodes are shown as green diamonds.

Note that the root of the tree is shown in gold colour indicating the currently selected node. Arrow keys on the keyboard allow the user to navigate the tree by changing which node is selected. `Down` navigates to the first child of the current node, `Shift+Down` – to its last child, `Up` – to its the parent, `Left` – to its next sibling on the left, `Right` – to its next sibling on the right. Additionally, pressing `R` will navigate to the root of the tree. The same actions are available under the **Navigation** menu.



If a subtree contains no solutions, it can be collapsed into a special single node displayed as a large red triangle. By default, the tree will be collapse failed subtrees automatically during its construction as a way to deal with large trees. The image below shows the same search tree as above, but with all failed subtrees collapsed.

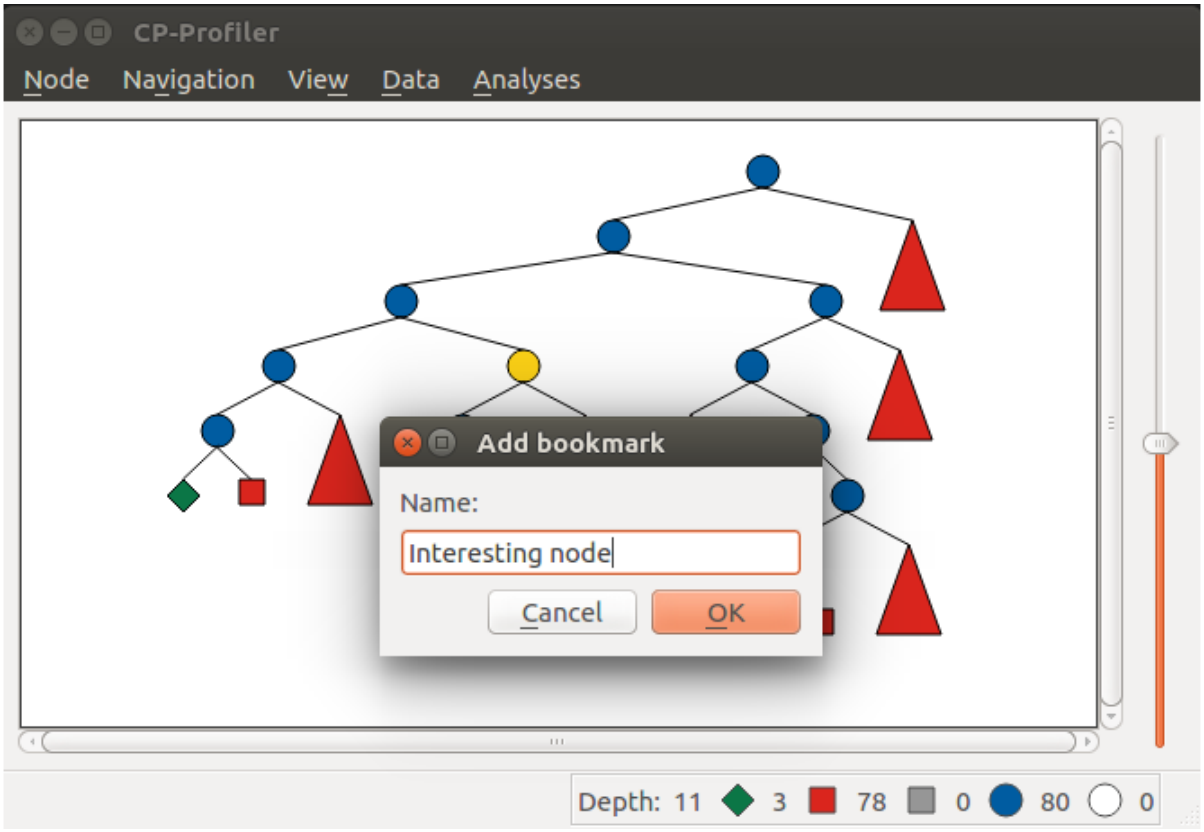


This view of the tree allows the user to show additional information for every node — its label, which usually represents the branching decision made by the solver to move from the parent node to its child. Pressing `L` on the keyboard will display labels for all descendants of the current node. `Shift+L` will display labels on the path to the current node. For example, the visualisation above shows branching decisions on the path from the first solution (shown as the new current node) to the root of the tree.

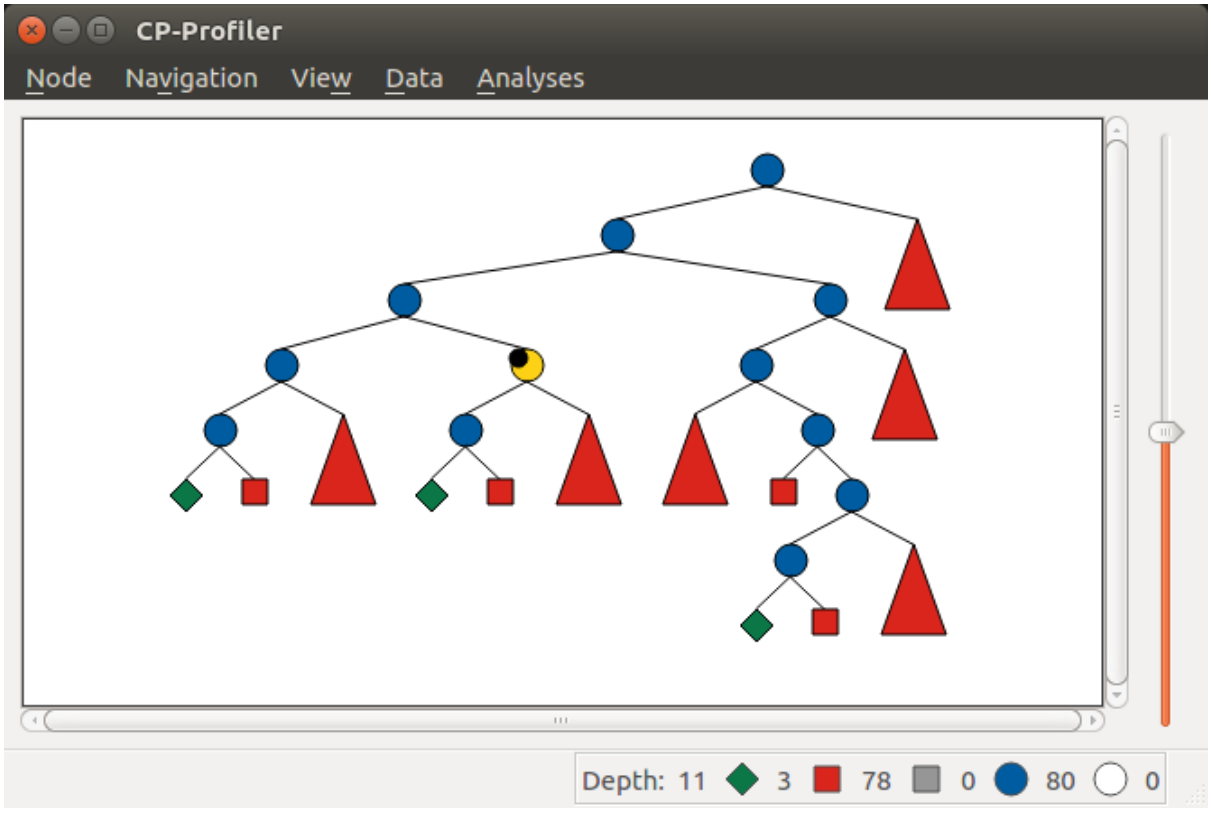
Status bar at the bottom of the window displays node statistics: the depth of the tree and the counts of different types of nodes. The scroll bar on the right lets the user to zoom in/out on the visualisation.

Adding Bookmarks

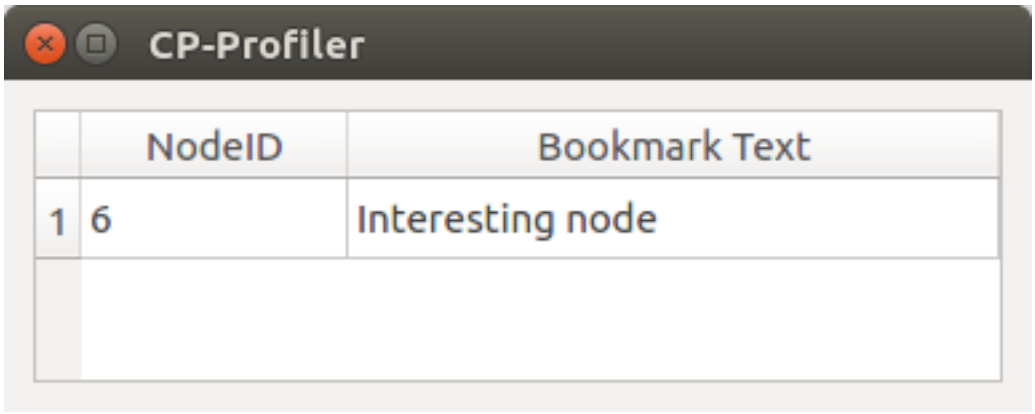
The user can *bookmark* any node on the tree associating it with an arbitrary text. This can be done by selecting the node of interest and clicking on **Add/remove bookmark** from the **Node** menu (shortcut: `Shift+B`). This will create a dialog as shown in the image below where the user can enter the bookmark text.



All bookmarked nodes are marked with a black circle on the traditional visualisation:



This message/node pair is also added to the bookmark list which can be retrieved from the **Data** menu under **Show bookmarks** action as shown below:



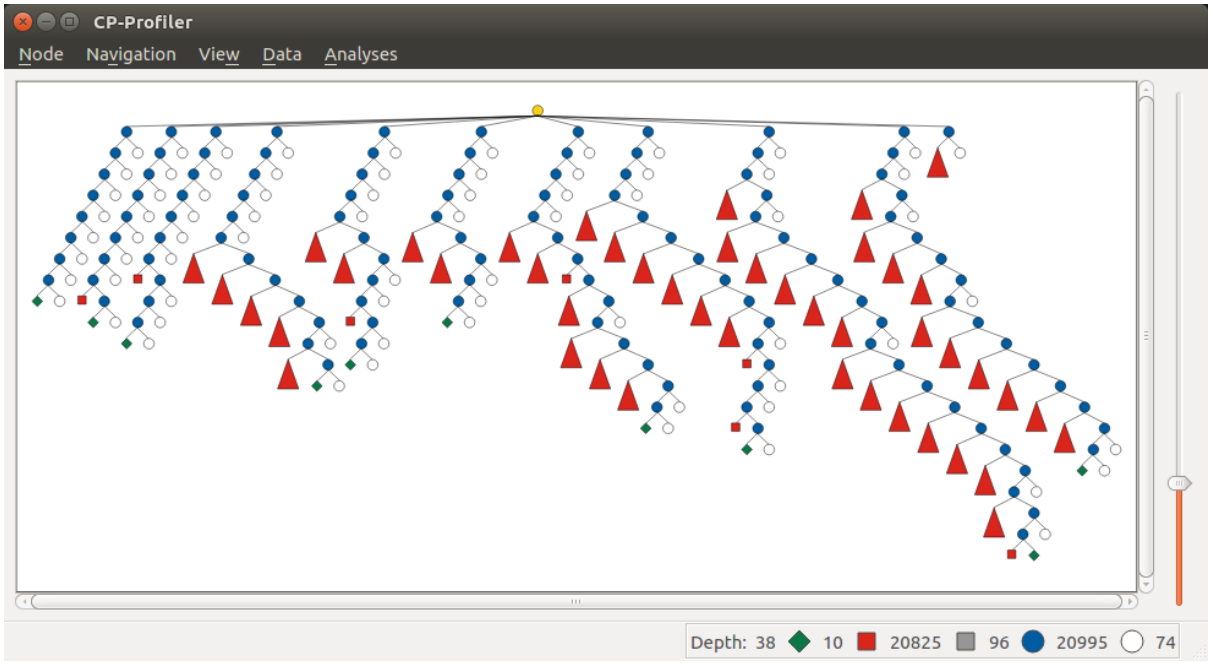
Note that each node is represented by a number – the node's unique identifier in CP-Profiler (it is likely to be different from the identifier sent by the solver). Clicking on a row will navigate the visualisation to the corresponding bookmarked node.

3.4.3 Alternative Search Tree Visualisations

Lantern Tree

One disadvantage of the traditional visualisation described above is the fact that when the nodes are collapsed (necessary for large trees) they become indistinguishable from one another. *Lantern Tree* visualisation addresses this limitation: it draws collapsed nodes in a way that indicate the size of the underlying subtrees.

Consider, for example, the traditional visualisation of a restart-based execution below:

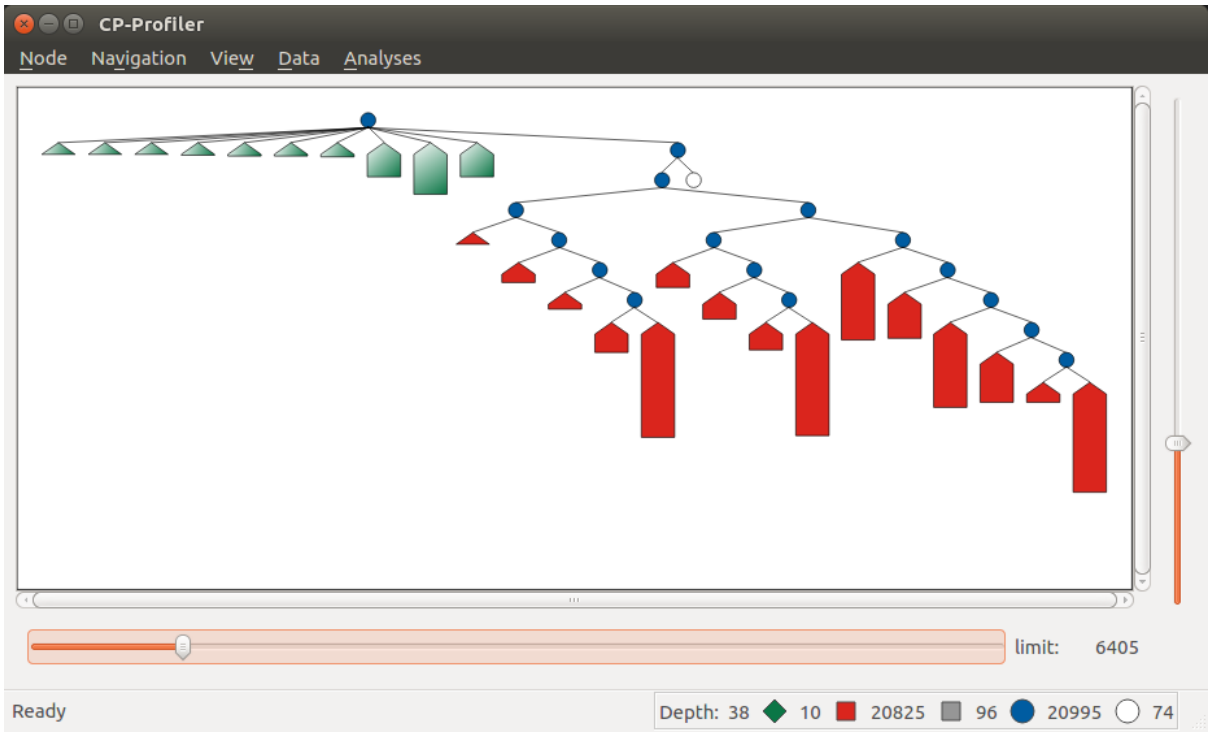


Each "restart" is represented by one of the subtrees attached to the "super" root node as a child. One can quickly determine that all but restarts terminated on finding a solution. The exception is the last restart in which no solution have been found (it corresponds to

the proof of optimality). However, not much can be concluded about the size of individual restarts, or the amount of search that they represent (at least not at a glance).

The following image shows the same restart tree using this alternative visualisation. All collapsed subtrees in this case are represented by "lantern" nodes, which cna be seen as a triangle extended with a rectangular section at the bottom.

The lantern tree visualisation is parameterised by N, which is the maximum number of nodes a lantern node can represent before splitting into several smaller lanterns. Note that is rarely possible to collapse subtrees such that each subtree contains exactly N nodes. For this reason, lantern nodes (their rectangular sections) are dynamic in size and grow linearly with the actual number of nodes n in the subtree ($1 < n <= N$).



In this example, the use of the lantern tree reveal additional insight: all but a few last restarts required very little search, and by far the greatest amount of work was done during the "proof of optimality" stage.

The lantern tree visualisation is available by enabling the **Lantern Tree View** option under the **View** menu (shortcut: `Ctrl+L`). Clicking on the same option again switches back to the traditional visualisation.

Pixel Tree

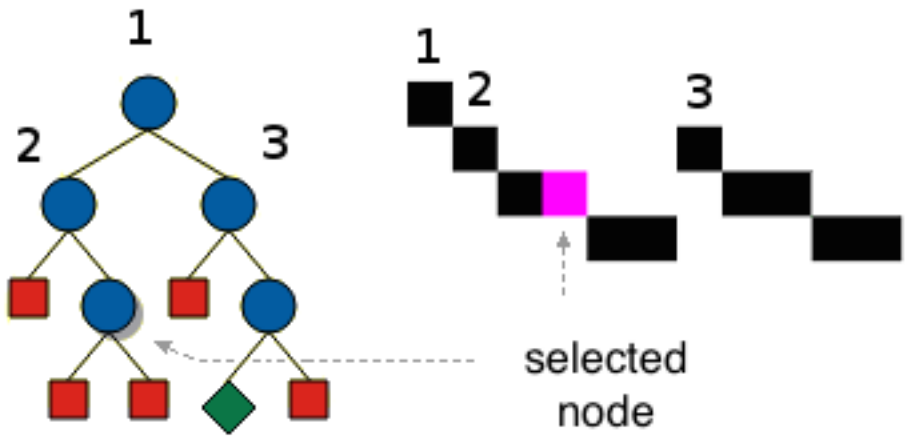
The pixel tree[1] and the icicle tree[2] visualisations are a quite different from the traditional visualisation in that they show links between nodes only implicitly, i.e. without drawing edges.

In the pixel tree view nodes are represented by squares (pixels), and edges only implicitly by the indentation between the squares. Parent nodes are placed immediately to the left of their subtree and the leaves of each subtree are grouped to the rightmost position.

The image below shows a correspondence between nodes in a traditional view and those

[1] Burch, M. et al., "Indented Pixel Tree Plots", 2010.
[2] Kruskal, J. B. et al., "Icicle Plots: Better Displays for Hierarchical Clustering", 1983.

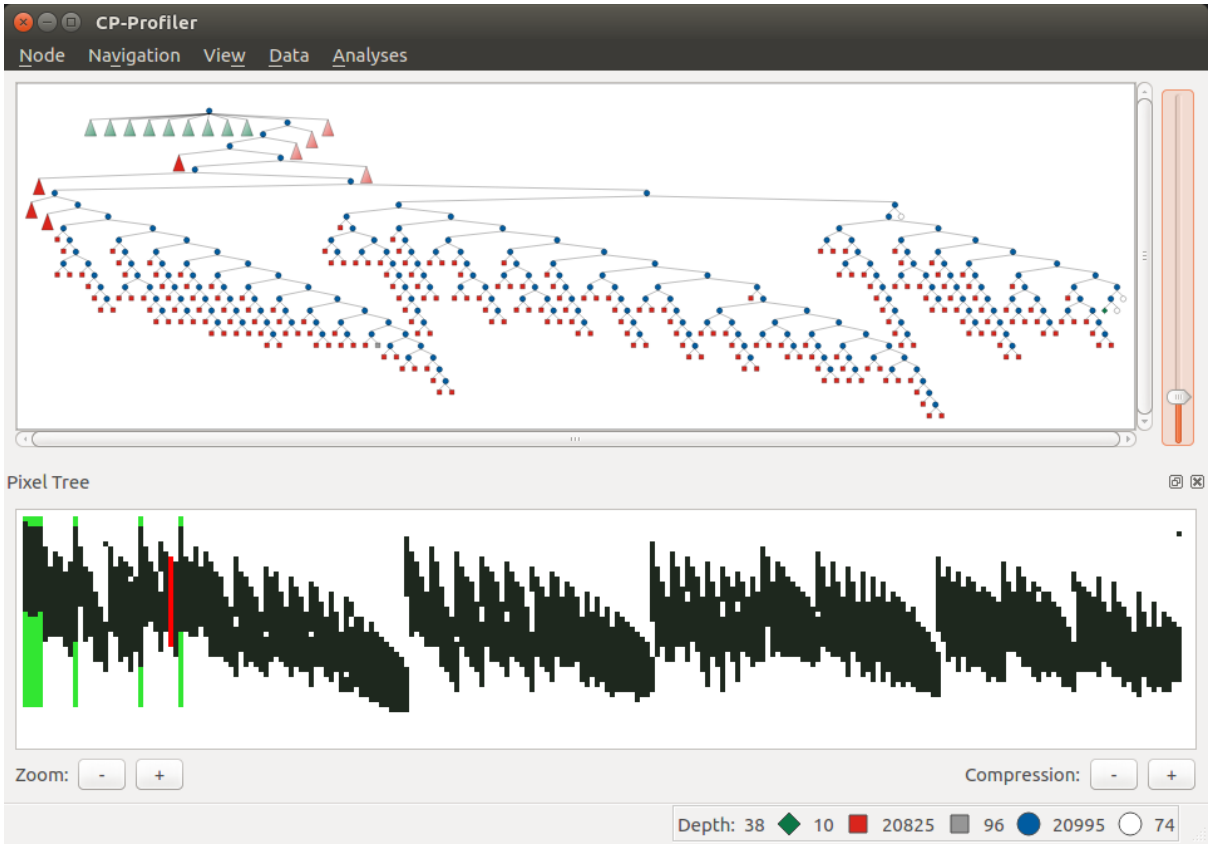
in the pixel tree view. Note that a green vertical line indicates a solution found at the corresponding node.



One of the main advantages of the pixel tree view is the ability to compress it while preserving large scale patterns and thus providing a good overview of the search. The compression is done by simply allowing multiple nodes on the same horizontal position.

In CP-Profiler the pixel tree can be shown alongside the traditional visualisation by enabling the **Pixel Tree View** option under the **View** menu (shortcut: `Shift+P`).

The following image shows the pixel tree view for the restart execution discussed earlier.



The pixel tree is compressed in this case to fit the entire tree in the window without any need for scrolling. Vertical green lines indicate that there is a solution node in the corresponding part of the tree. A vertical slices of the tree can be selected by clicking

and dragging with the left mouse key. Note that in this example a slice just before the final solution is selected (it is shown in red). Selecting a part of the tree in this way will update the traditional view to reveal the nodes that belong to that part, hiding the rest.

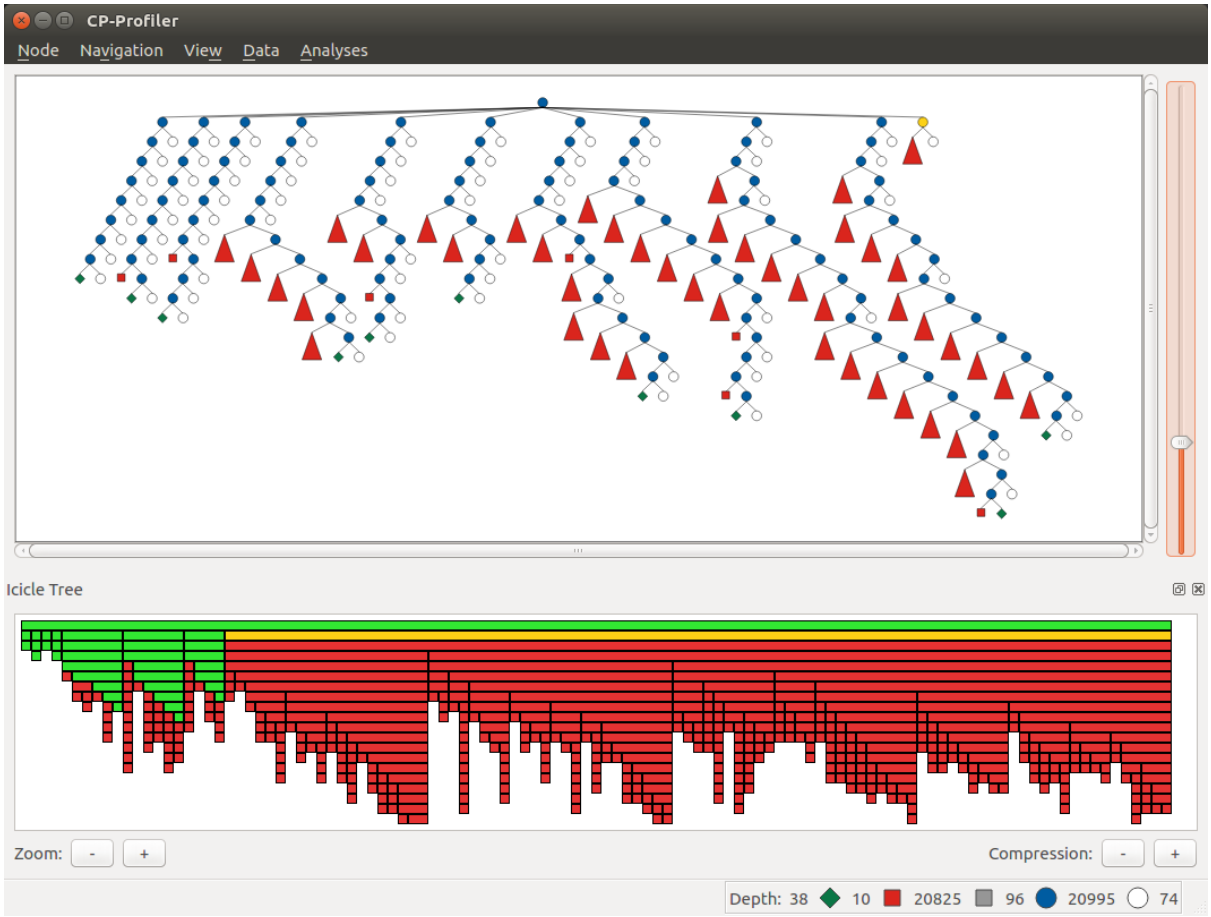
Icicle Tree

In the icicle tree parent nodes are positioned strictly above their children, i.e. they span horizontally over the same space as their children nodes. This property allows the user to quickly find a node's ancestry and determine the size of any subtree at a glance.

Icicle trees are compressed by cutting off leaf nodes, so that the corresponding parent nodes become the new leaf nodes. Doing so in a binary tree, for example, reduces the display area required for the visualisation by a factor of two. This action is repeated until the visualisation is of the desired size.

In CP-Profiler the icicle tree can be shown alongside the traditional visualisation by enabling the **Icicle Tree View** option under the **View** menu (shortcut: `Shift+I`).

The following image shows the icicle tree view for the restart execution from before.



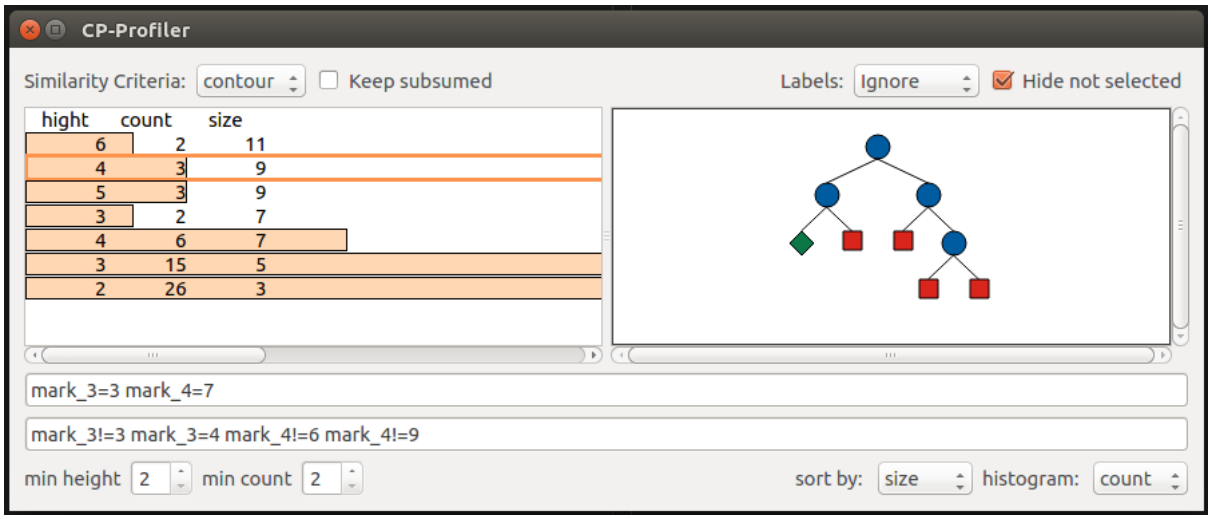
Again, the entire tree have been compressed to fit in the window. The following colour scheme is used: each icicle element that represents a subtree containing a solution is shown in green colour. All other nodes are shown in red as they represent entirely failed subtrees. Additionally elements of the icicle tree can be selected (shown in yellow colour) to show the corresponding subtree (node) in the traditional visualisation. Note that this relationship is two-way bound: selecting a different node in the traditional visualisation will update the icicle tree view correspondingly.

3.4.4 Similar Subtree Analysis

This analysis allows users to find similarities within a single search tree.

The analysis can be initiated by selecting **`Similar Subtrees`** from the menu **`Analyses`** (shortcut: `Shift+S`). The image below shows the result of running the analysis on the search tree above. Horizontal bars on the left lists all similarities (patterns) found in the tree. Here, the lengths of the bars are configured to indicate how many subtrees belong to a particular pattern (*count*). Additionally the bars are sorted so that the patterns with subtrees of larger *size* appear at the top. Another property of a pattern is its *height*, which indicates the height/depth of subtrees that the pattern represent.

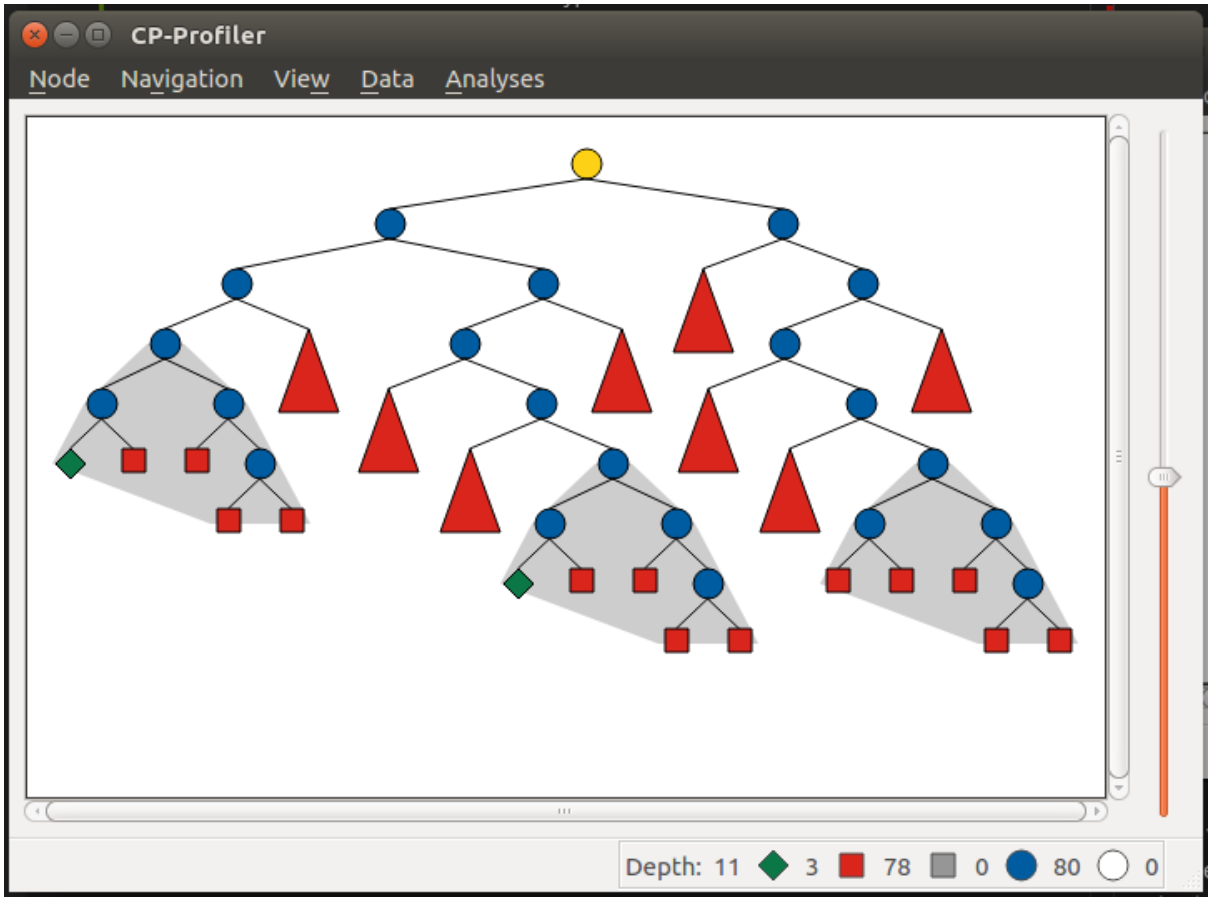
Note that in this example the second (from the top) pattern is currently selected (shown with orange outline). The view on the right shows a "preview" (traditional visualisation) of one of the subtrees representing the selected pattern. The two rows below the show the result of computing the difference in labels on the path from the root to two of the subtrees representing the pattern (in this case it is the first two subtrees encountered when the tree is traversed in the depth-first-search order).



TODO: discuss the two criteria

Changing the configuration menu at the bottom of the window, the user can filter the list of patterns based on their *count* and *height* values. They way the length of horizontal bars is determined and the sorting criteria can also be specified there.

Whenever a pattern on the left hand side is selected, the corresponding subtrees will be highlighted on the traditional visualisation by drawing their outlines in grey. Additionally, if the option *Hide not selected* is enabled (top of the window), everything but the subtrees of the current pattern will be collapsed as shown in this example.



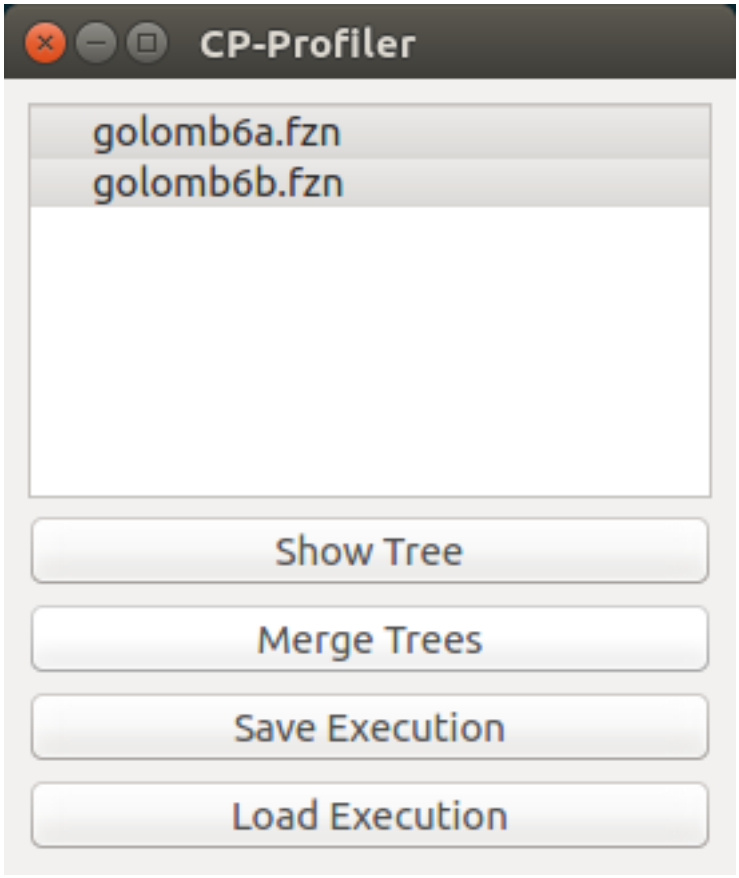
Elimination of Subsumed Patterns

A pattern *P* is said to be subsumed by one or more other patterns if subtrees of those patterns contain all of the subtrees of *P* as descendants. By default, the analysis is configured to eliminate all subsumed patterns. Enabling the *Keep subsumed* option (top of the window) will display the subsumed patterns along with non-subsumed patterns.

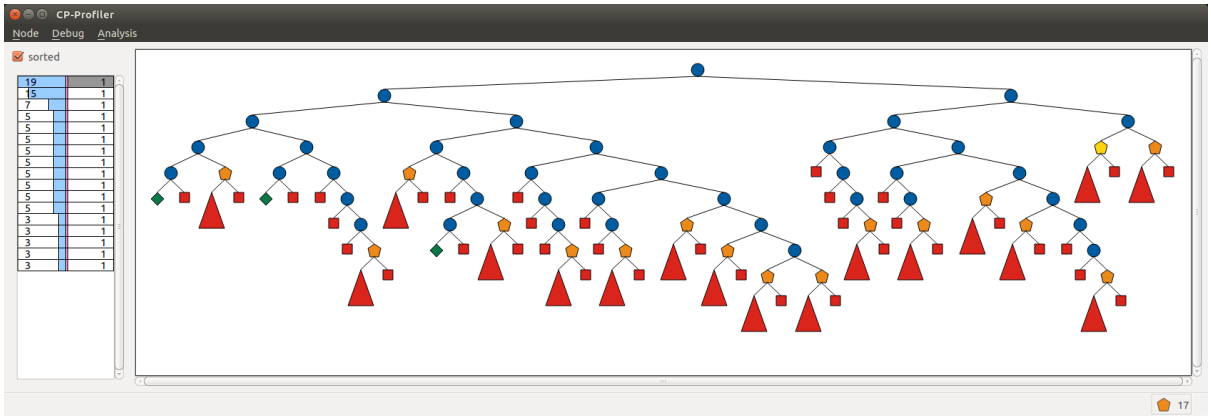
3.4.5 Execution Comparison by Merging

CP-Profiler allows users to compare two executions by merging the corresponding search trees. This is accomplished traversing both trees in lockstep in a depth-first-search manner, starting at the root nodes and comparing corresponding node at each step. A node is created on the resulting merged tree for every pair of nodes that are deemed identical. Once a difference is found, a special *pentagon* node is created, whose two children are the roots of the corresponding different subtrees.

This comparison of two execution can be done in CP-Profiler by selecting the two executions of interest from the list of executions and clicking the *Merge Trees* button.



The image below shows an example of the resulting merged tree. As indicated in the status bar, this tree contains 17 (orange) pentagons. The panel on the left lists all pentagons displaying the number of nodes on corresponding left/right subtrees. By default, the list is sorted by the absolute difference between the number of nodes. Note that in this case the top pentagon from the list (19 – 1) is selected, which automatically selects the corresponding node in the visualisation (second last pentagon node, highlighted in yellow).



3.4.6 Replaying Search

The two main factors that affect a solver execution are propagation and search. It can be useful to consider propagation and search in isolation (e.g. to aid merging), which can be done using the technique called *search replaying*. The idea is to force the same search

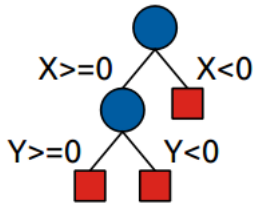
decisions by first performing one execution normally and recording the tree structure and branching decisions (labels) to form a *search log*. Then the *search log* log is used to drive the search in another execution.

The protocol for replaying

The following information (*entry*) is recorded in a *search log* per node: node's unique identifier, the number of children, and the search decisions associated with each child node. Each entry, in turn, is divided into *tokens* separated by a whitespace character. The first two tokens are the node's identifier and the number of children (0 for a leaf node). Entries of branch nodes are extended by a pair of tokens per child, containing the child's identifier and its branching decision.

Example

Consider the following basic search tree:



The corresponding *search log* is as follows:

```
0 2 1 X>=0 2 X<0
1 2 3 Y>=0 4 Y<0
3 0
4 0
2 0
```

3.4.7 Nogood Analysis

This analysis is based on the idea of comparing (*merging*) two execution of the same model (instance) using the same search (via *search replaying*), one with nogood learning and the other one – without (resulting in *learning* and *non-learning* executions). Since the only difference between the two executions will be in the higher propagation strength in the learning execution due to no-goods, the resulting merged tree will only have pentagons of the form *1/n* (one failure node corresponding to a subtree of *n* nodes in non-learning execution). *n* in this case is called *search reduction* and it is attributed to all previously generated no-goods that contributed to this early failure. The analysis aggregates this information throughout the execution, which results in a *nogood ranking* based on how much search reduction each nogood contributed to.

The following steps can be performed to execute the nogood analysis:

1. An instance is executed with a learning solver with profiling on.

For example: `chuffed pizza.fzn --cpprofiler`.

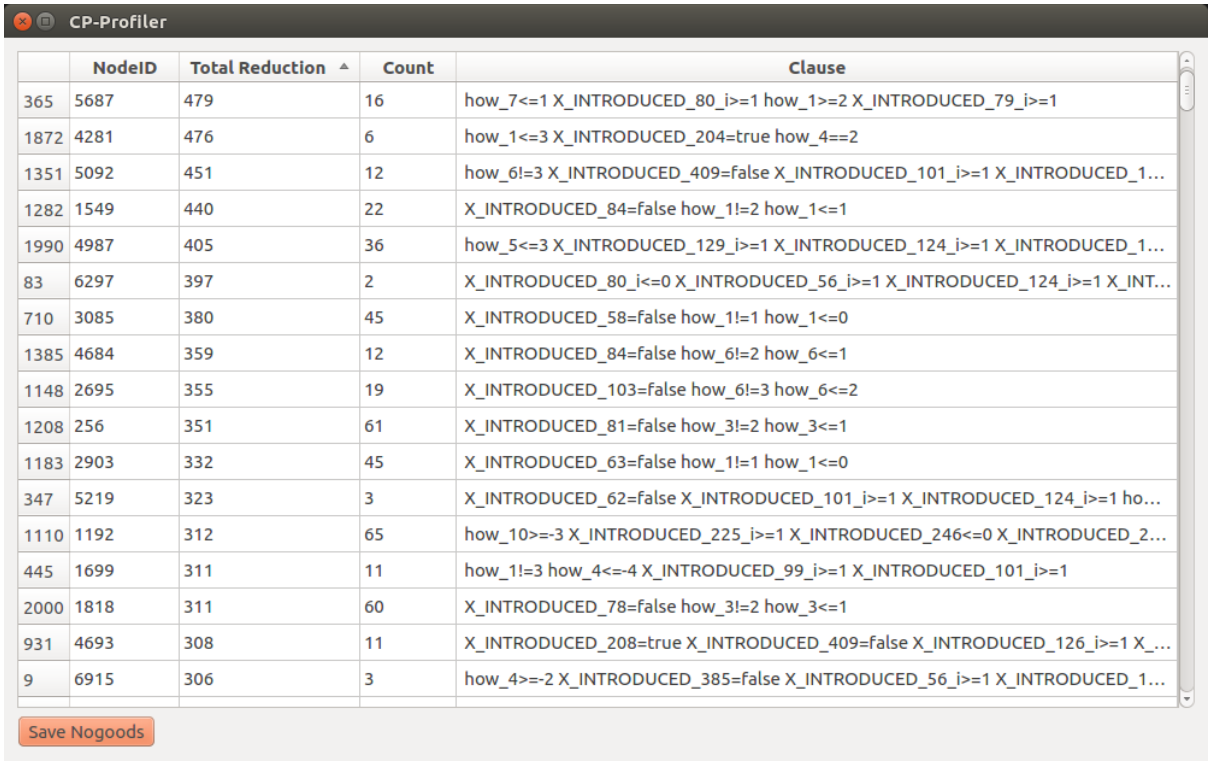
2. The resulting search is saved to a file using CP-Profiler.

3. The same instance is executed without learning (preferably by the same solver, but a solver with similar propagation will produce similar result).

For example: `gecode --cpprofiler_replay pizza.search pizza.fzn`

4. The two resulting trees are merged using CP-Profiler.
5. **Nogood analysis** option is selected from the **Analyses** menu in CP-Profiler.

The image below shows a possible result window, in which the ranking is displayed in a tabular form with one row per nogood:



The first column identifies the (failed) node at which the nogood has been generated. The next column shows search reduction (in the number of nodes) attributed to this nogood. This is followed by the column that shows the number of times the nogood contributed to search reduction, that is, the number of *1/n* pentagons that were partially or solely responsible for the early failure. Finally, the last column shows the nogood itself in a form of a clause (disjunction of literals) where its literals are separated by a whitespace character.

Note that dy default the entries are sorted based on search reduction. The sorting criteria can be changed by clicking on corresponding column headers.

This result can be saved to a file (by clicking the **Save Nogoods** button) for further examination. Each row in the resulting file will correspond to one nogood, and the tab character (`\t`) will separate different columns.

3.4.8 The protocol (high level)

The following describes the protocol that a solver must implement to communicate with the profiler.

The protocol distinguishes between the following types of messages: **Start**, **Restart**, **Done**, and **Node**.

The **Start** message is sent at the beginning of the execution. The message has two optional parameters:

- `Name`: execution's descriptor to be presented to the user (e.g. the model's name)
- `Execution ID`: globally unique identifier used to distinguish between different executions.

The **Restart** message is sent whenever a solver performs a restart in case of a restart-based search.

The **Done** message is sent to the profiler at the end of the execution to indicate that no further nodes should be expected.

The **Node** message is sent whenever a new node is explored by the solver and contains information necessary for reconstructing the search tree. The required parameters are:

- `Node ID`: unique node identifier in the execution.
- `Parent ID`: the identifier (`Node ID`) of the parent node. A root node can have an identifier of *-1*.
- `Alternative`: the node's position relative to its siblings; for the left-most child it is `0`, for the second left-most it is `1` etc.
- `Number of Children`: the number of children nodes. If not known, can be set to `0` and the node will be extended with extra children later on if necessary.It is, however, advisable to specify the number of children the profiler should expect (for example, the yet to arrive nodes can be visualised to give a better idea about the search).
- `Status`**: allows to distinguish between different types of nodes. Supported values are:**
  - *BRANCH*: internal node in the tree;
  - *SOLUTION*: leaf node representing a solution;
  - *FAILURE*: leaf node representing a failure;
  - *SKIPPED*: leaf node representing unexplored search space due to back-jumping.

Example. The following sequence of nodes (excluding the *Start* and *Done* messages) produces the simple tree with three nodes:

Label	Node ID	Parent ID	Alternative	Number of Children	Status
Root	0	-1	-1	2	BRANCH
Failure	1	0	0	0	FAILED
Solution	2	0	1	0	SOLVED

3.4.9 The protocol (low level)

Each message starts with a four-byte integer encoding the size of the remainder of the message in bytes. This is followed by a single byte encoding the type of the message. The corresponding values are: `Node`: `0`, `Done`: `1`, `Start`: `2`, `Restart`: `3`.

Node message

In case the message is of the type `Node`, the following fields are added in order: `id`, `pid`, `alt`, `children` and `status`.

Node identifiers `id` and `pid` are represented using three four-byte integers: first identifies the identifier of the node within a thread, the second — the identifier of the restart (in a restart-based search), and the third — the identifier of the thread. The `alt` and `children` fields are represented by a single four byte integer each. The `status` field is represented by a single byte; its possible values are: *SOLVED*: 0, *FAILED*: 1, *BRANCH*: 2, *SKIPPED*: 3. All multi-byte integer values are encoded using the *two's compliment* notation in the *big-endian order*.

Additionally, each node message can contain the following optional fields:

- `label`: branching decision (or any arbitrary string to be drawn together with the node);
- `nogood`: string representation of a newly generated nogood in a learning solver;
- `info`: arbitrary information about the node.

Field identifiers and their sizes in bytes:

field name	field id	size (bytes)
`id`	n/a	12
`pid`	n/a	12
`alt`	n/a	4
`children`	n/a	4
`status`	n/a	1
`label`	0	any
`nogood`	1	any
`info`	2	any

Example. The following is a possible correspondence between a solver and the profiler that generates a trivial tree. The order in which different fields arrive is shown from top to bottom (rows are numbered for convenience).

Message 1:

Row	Bytes	Interpretation
1	`00 00 00 21`	message size (33)
2	`02`	message type (*START*)
3	`02`	field (*info*)
4	`00 00 00 1B`	string size (27)
5	`7b 22 6e 61 6d 65 22 3a 20 22 6d 69 6e 69 6d 61 6c 20 65 78 61 6d 70 6c 65 22 7d`	'{"name": "minimal example"}'

Message 2:

Row	Bytes	Interpretation
6	`00 00 00 2B`	message size (43)
7	`00`	message type (**NODE**)
8	`00 00 00 00`	node id (0)
9	`FF FF FF FF`	node restart id (-1)
10	`FF FF FF FF`	node thread id (-1)
11	`FF FF FF FF`	parent id (-1)
12	`FF FF FF FF`	parent restart id (-1)
13	`FF FF FF FF`	parent thread id (-1)
14	`FF FF FF FF`	alternative (-1)
15	`00 00 00 02`	children (2)
16	`02`	status (*BRANCH*)
17	`00`	field (label)
18	`00 00 00 04`	string size (4)
19	`52 6f 6f 74`	'Root'

Message 3:

Row	Bytes	Interpretation
20	`00 00 00 01`	message size (1)
21	`01`	message type (**DONE**)

CHAPTER 3.5

Solving Technologies and Solver Backends

The `minizinc` tool can use various solver backends for a given model. Some solvers are separate executables that are called by `minizinc`; other solvers are part of the `minizinc` binary (either hard-coded or loaded as a dynamic library or "plugin"). In the former case, a temporary FlatZinc file is created and passed to the solver binary; in the latter one, the flattened model is passed to the backend directly in memory. Some solvers are part of the binary MiniZinc distribution, others have to be installed separately. You can find instructions for installing these solvers from source code and integrating into the `minizinc` tool in *Installation from Source Code* (page 267). This chapter summarises usage options available for various target solvers.

The help text of `minizinc` shows a list of configured solver backends and their tags. You can see solver-specific command-line options by running

```
$ minizinc --help <solver-id or tag>
```

3.5.1 Constraint Programming Solvers

Constraint Programming is the 'native' paradigm of MiniZinc. Below we discuss most common CP solvers. For their performance, consult MiniZinc Challenges (https://www.minizinc.org/challenge/).

3.5.1.1 Gecode

Gecode is an open-source constraint programming system (see https://www.gecode.org). It supports many of MiniZinc's global constraints natively, and has support for integer, set and float variables.

Gecode supports a number of constraints and search annotations that are not part of the MiniZinc standard library. You can get access to these by adding `include "gecode.mzn";` to your model. The additional declarations are documented in Section 4.2.4.

3.5.1.2 Chuffed

Chuffed is a constraint solver based on *lazy clause generation* (see https://github.com/chuffed/chuffed). This type of solver adapts techniques from SAT solving, such as conflict clause learning, watched literal propagation and activity-based search heuristics, and can often be much faster than traditional CP solvers.

In order to take full advantage of Chuffed's performance, it is often useful to add a search annotation to the model (see *Search* (page 97)), but allow Chuffed to switch between this defined search and its activity-based search. In order to enable this behaviour, use the `-f` (free search) command line option or select *Free search* in the solver configuration pane of the MiniZinc IDE.

Chuffed supports a number of additional search annotations that are not part of the MiniZinc standard library. The additional declarations are documented in Section 4.2.5.

3.5.1.3 OR-Tools

OR-Tools is a powerful open-source CP/SAT/LP solver (see https://developers.google.com/optimization/). It supports many of MiniZinc's global constraints natively. It often performs better multi-threaded (option `-p`) so it can employ various solving technologies. A search annotation (see *Search* (page 97)) can be useful, however allowing OR-Tools to mix the prescribed strategy with its own (option `-f`) usually is best, analogously to Chuffed.

3.5.2 Mixed-Integer Programming Solvers

MiniZinc has built-in support for Mixed Integer Programming solvers. If you have any MIP solver installed (and MiniZinc was compiled with its support), you can run a model using MIP like this on the command line:

```
minizinc --solver mip -v -s -a model.mzn data.dzn
```

Of course, you can also select a particular solver, e.g. Gurobi (in case it is available):

```
minizinc --solver gurobi -v -s -a model.mzn data.dzn
```

3.5.2.1 MIP-Aware Modeling (But Mostly Useful for All Backends)

Avoid mixing positive and negative coefficients in the objective. Use 'complementing' variables to revert sense.

Avoid nested expressions which are hard to linearize (decompose for MIP). For example, instead of

```
constraint forall(s in TASKS)(exists([whentask[s]=0] ++
  [whentask[s]>= start[s]+(t*numslots) /\ whentask[s]<=stop[s]+(t*numslots)␣
↪| t in 0..nummachines-1]));
```

prefer the tight domain constraint

```
constraint forall(s in TASKS)(whentask[s] in
  {0} union array_union([ start[s]+(t*numslots) .. stop[s]+(t*numslots) | t␣
↪in 0..nummachines-1]));
```

To avoid **numerical issues**, make variable domains as tight as possible (compiler can deduce bounds in certain cases but explicit bounding can be stronger). Try to keep magnitude difference in each constraint below 1e4. Especially for variables involved in logical constraints, if you cannot reduce the domains to be in +/-1e4, consider indicator constraints (available for some solvers, see below), or use the following trick: instead of saying `b=1 -> x<=0` where x can become very big, use e.g. `b=1 -> 0.001*x<=0.0`. Especially for integer variables, the domain size of 1e4 should be an upper bound if possible – what is the value of integrality otherwise? Avoid large coefficients too, as well as large values in the objective function. See more on tolerances in the Solver Options section.

Example 1: *basic big-M constraint vs implication.* Instead of `<expr> <= 1000000*y` given `var 0..1: y` and where you use the 'big-M' value of 1000000 because you don't know a good upper bound on `<expr>`, prefer `y=0 -> <expr> <= 0` so that MiniZinc computes a possibly tighter bound, and consider the above trick: `y=0 -> 0.0001*<expr> <= 0.0` to reduce magnitudes.

Example 2: *cost-based choice.* Assume you want the model to make a certain decision, e.g., constructing a road, but then its cost should be minimal among some others, otherwise not considered. This can be modeled as follows:

```
var 0..1: c;                                                %% Whether we␣
↪construct the road
var int: cost_road = 286*c + 1000000*(1-c);
var int: cost_final = min( [ cost_road, cost1, cost2 ] );
```

Note the big coefficient in the definition of `cost_road`. It can lead to numerical issues and a wrong answer: when the solver's integrality tolerance is 1e-6, it can assume `c=0.999999` as equivalent to `c=1` leading to `cost_road=287` after rounding.

A better solution, given reasonable bounds on `cost1` and `cost2`, is to replace the definition as follows:

```
int: cost_others_ub = 1+2*ub_array( [cost1, cost2] );    %% Multiply by 2␣
↪for a stronger LP relaxation
var int: cost_road = 286*c + cost_others_ub*(1-c);
```

3.5.2.2 Useful Flattening Parameters

The following parameters can be given on the command line or modified in `share/minizinc/linear/options.mzn`:

```
-D nSECcuts=0/1/2                          %% Subtour Elimination␣
↪Constraints, see below
-D fMIPdomains=true/false                  %% The unified domains feature,␣
↪see below
-D float_EPS=1e-6                          %% Epsilon for floats' strict␣
```

```
↪comparison,
                                          %% used e.g. for the following␣
↪cases:
                                          %% x!=y, x<y, b -> x<y, b <-> x
↪<=y
-DfIndConstr=true -DfMIPdomains=false     %% Use solver's indicator␣
↪constraints, see below
-DMinMaxGeneral=true                      %% Send min/max constraints to␣
↪the solver (Gurobi only)
-DQuadrFloat=false -DQuadrInt=false       %% Not forward float/integer␣
↪multiplications for MIQCP backends, see below
-DUseCumulative=false                     %% Not forward cumulative with␣
↪fixed durations/resources (SCIP only)
-DUseOrbisack=false                       %% Not forward lex_lesseq for␣
↪binary/bool vectors (SCIP only)
-DOrbisackAlwaysModelConstraint=true      %% lex_lesseq ignores being in␣
↪symmetry_breaking_constraint() (SCIP only)
                                          %% Required for SCIP 7.0.2, or␣
↪use patch: http://listserv.zib.de/pipermail/scip/2021-February/004213.html
-DUseOrbitope=false                       %% Not forward lex_chain_lesseq␣
↪for binary/bool matrices (SCIP only)
--no-half-reifications                    %% Turn off half-reification␣
↪(full reification was until v2.2.3)
```

3.5.2.3 Some Solver Options and Changed Default Values

The following command-line options affect the backend or invoke extra functionality. Note that some of them have default values which may be different from the backend's ones. For example, tolerances have been tightened to enable more precise solving with integer variables and objective. This slightly deteriorates performance on average, so when your model has moderate constant and bound magnitudes, you may want to pass negative values to use solver's defaults.

```
-h <solver-tag>    full description of the backend options
--relGap <n>       relative gap |primal-dual|/<solver-dep> to stop. Default␣
↪1e-8, set <0 to use backend's default
--feasTol <n>      primal feasibility tolerance (Gurobi). Default 1e-8
--intTol <n>       integrality tolerance for a variable. Default 1e-8
--solver-time-limit-feas <n>, --solver-tlf <n>
                   stop after <n> milliseconds after the first feasible␣
↪solution (some backends)
--writeModel <file>
                   write model to <file> (.lp, .mps, .sav, ...). All solvers␣
↪support the MPS format
                   which is industry standard. Most support the LP format.␣
↪Some solvers have own formats,
                   for example, the CIP format of SCIP ("constraint integer␣
↪programming").
--readParam <file>
```

```
                    read backend-specific parameters from file (some backends)
--writeParam <file>
                    write backend-specific parameters to file (some backends)
--readConcurrentParam <file>
                    each of these commands specifies a parameter file of one␣
↪concurrent solve (Gurobi only)
--keep-paths        this standard flattening option annotates every item in␣
↪FlatZinc by its "flattening history".
                    For MIP solvers, it additionally assigns each constraint
↪'s name as the first 255 symbols of that.
--cbcArgs '-guess -cuts off -preprocess off -passc 1'
                    parameters for the COIN-OR CBC backend
```

All MIP solvers directly support multi-threading (option `-p`). For COIN-BC to use it, it needs to be configured with `--enable-cbc-parallel`.

3.5.2.4 Subtour Elimination Constraints

Optionally use the SEC cuts for the circuit global constraint. Currently only Gurobi, IBM ILOG CPLEX, and COIN-OR CBC (trunk as of Nov 2019). If compiling from source, this needs boost and cmake flag `-DCOMPILE_BOOST_MINCUT=ON` (or `#define` it in `lib/algorithms/min_cut.cpp`). Compile your model with the flag `-DnSECcuts=<n>` with the following possible `<n>`: 0,1: use MTZ formulation; 1,2: pass on circuit constraints to the SEC cut generator, so 1 would use both.

3.5.2.5 Unified Domains (MIPdomains)

The 'MIPdomains' feature of the Flattener aims at reducing the number of binary flags encoding linearized domain constraints, see the paper *Belov, Stuckey, Tack, Wallace. Improved Linearization of Constraint Programming Models. CP 2016.*

By default it is on. To turn it off which might be good for some models, add option `-D fMIPdomains=false` during flattening. Some parameters of the unification are available, run with `--help`.

3.5.2.6 Indicator Constraints

Some solvers (IBM ILOG CPLEX, Gurobi, SCIP) have indicator constrains with greater numerical stability than big-M decomposition. Moreover, they can be applied to decompose logical constraints on *unbounded variables*. However, for reified comparisons with reasonable big-M bounds they perform worse because solvers don't include them in the LP relaxation. Add command-line parameters `-D fIndConstr=true -D fMIPdomains=false` when flattening to use them.

3.5.2.7 Quadratic Constraints and Objectives (MIQCP)

Gurobi 9.0 and SCIP support MIQCP (invoking non-convex global optimizer because MiniZinc translates multiplication to equality with an intermediate variable: whenever the model uses an expression x*y it is converted to z with z==x*y which is non-convex).

While this is mostly advantageous for integer multiplication (which is linearly decomposed for other solvers), for float variables this is the only way to go. To switch off forwarding float/integer multiplications to the backend, run compiler with either or both of `-DQuadrFloat=false -DQuadrInt=false`.

3.5.2.8 Pools of User Cuts and Lazy Constraints

Some constraints in the model can be declared as user and/or lazy cuts and they will be added to the corresponding pools for the solvers supporting them. For that, apply annotations `::MIP_cut` and/or `::MIP_lazy` after a constraint. For Gurobi and IBM ILOG CPLEX, see `share/minizinc/linear/options.mzn` for their exact meaning.

3.5.2.9 Warm Starts

For general information of warm start annotations, see *Warm Starts* (page 106). Warm starts are currently implemented for Gurobi, IBM ILOG CPLEX, Xpress, and COIN-OR CBC.

3.5.3 Non-Linear Solvers via NL File Format

MiniZinc has experimental support for non-linear solvers that conform to the AMPL NL standard. There are a number of open-source solvers, such as Ipopt, Bonmin and Couenne, that can be interfaced to MiniZinc in this way.

You can download binaries of these solvers from AMPL (https://ampl.com/products/solvers/open-source/). In order to use them with MiniZinc, you need to create a solver configuration file. Future version of MiniZinc will make this easier, but for now you can follow these steps:

1. Download the solver binary. For this example, we assume you chose the Couenne solver, which supports non-linear, non-convex, mixed discrete and continuous problems.
2. Create a solver configuration file called `couenne.msc` in the `share/minizinc/solvers` directory of your MiniZinc installation, with the following contents:

   ```
   {
     "id" : "org.coin-or.couenne",
     "name" : "Couenne",
     "executable" : "/Users/tack/Downloads/couenne-osx/couenne",
     "version": "0.5.6",
     "supportsFzn":false,
     "supportsNL":true
   }
   ```

   You can adapt the `version` field if you downloaded a different version (it's only used for displaying).
3. Run `minizinc --solvers`. The Couenne solver should appear in the list of solvers now.

4. Run `minizinc --solver couenne model.mzn` on some MiniZinc model, or use Couenne from the MiniZinc IDE.

The AMPL NL support is currently experimental, and your MiniZinc model is translated to NL without regard for the capabilities of the target solver. For example, Ipopt only supports continuous variables, so translating a model with integer variables will result in a solver-level error message. There is currently no support for translating Boolean variables and constraints into 0/1 integer variables (as required by e.g. Couenne). You can experiment with the standard linearisation library, using the `-Glinear [-DQuadrFloat=true -DQuadrInt=true]` flag. However, this will either linearise all integer constraints, even the ones that solvers like Couenne may support natively, or use non-convex representation. We will ship dedicated solver libraries for some NL solvers with future versions of MiniZinc.

CHAPTER 3.6

Automatic Solution Checking, Model Validation, and Benchmarking

MiniZinc supports checking the correctness of solutions of a model in two ways. First, it is the *autonomous checking* where compiler substitutes solution values into the model and tries to find inconsistencies, possibly invoking a solver to a much smaller instance. Second, it is by using a *checker model*. This latter method can also be seen as model validation.

3.6.1 Autonomous Automatic Solution Checking and Benchmarking

For this method, compiler needs the values of as many variables as possible, ideally all variables without the right-hand side. For all top-level variables (not those inside let constructs) the values can be output using `--output-mode dzn` compiler option. For optimization problems, when no explicit objective variable exists, a variable named `_objective` is added by `--output-objective`. Then, each produced solution can be copied into a `.dzn` and compiled together with the original instance, adding `--allow-multiple-assignments` option if `--output-objective` was used. In the case that some variables inside let constructs need to be assigned, a solver has to be invoked for the smaller instance remaining after fixing the supplied values. For example, for the following model:

```
var int: a;
var float: b = a;
solve
  maximize a-b;
output
  [ "a+b is \(a+b)\n" ];
```

running it with `minizinc --solver gecode test_output_mode.mzn --output-mode dzn --output-objective` ignores the provided output annotation and prints

```
a = -2147483646;
_objective = 0.0;
----------
==========
```

which can be added as an extra `.dzn` file.

This process of re-compiling and re-solving with the output variables fixed is automated by mzn-bench[8], which can check the solutions of a benchmark run against the original models and detect inconsistent claims of optimality or unsatisfiability. It can be installed with `pip install mzn-bench`.

3.6.2 Model Validation: Automatic Solution Checking with a Checker Model

This approach has two main applications:

- Instructors can provide checker models for student assignments. This provides students with immediate, detailed feedback on their modelling attempts.
- A simplified checker model can be used to verify a complex model used for solving. This can be useful when experimenting with new decompositions of constraints, or for post-processing solutions if some constraints cannot be added to the original model (e.g. in case the solver does not support certain constructs).

Running MiniZinc with solution checking is easy. On the command line, simply pass the name of the checker model in addition to the problem model:

```
minizinc model.mzn model.mzc.mzn data.dzn
```

Checker models can be pre-compiled in order to obfuscate their contents (e.g. if they contain clues to students how to model the problem):

```
minizinc --compile-solution-checker model.mzc.mzn
```

This will create the compiled checker model `model.mzc`, which can be used instead of the clear text version for checking:

```
minizinc model.mzn model.mzc data.dzn
```

The MiniZinc IDE has built-in support for solution checkers. Whenever the current project contains a file with the same file name as the current model but file extension `.mzc` or `.mzc.mzn`, the "Run" button turns into a "Run+Check" button.

The rest of this section describes how to implement checker models.

[8] https://github.com/MiniZinc/mzn-bench

3.6.2.1 Basic checker models

At its core, a checker model takes each solution that a solver produces as input, and outputs whether the solution is correct or not. Let's use the simple map colouring model from the tutorial as an example. Here is the model again:

Listing 6.1: A MiniZinc model `aust.mzn` for colouring the states and territories in Australia

```
% Colouring Australia using nc colours
int: nc = 3;

var 1..nc: wa;   var 1..nc: nt;  var 1..nc: sa;   var 1..nc: q;
var 1..nc: nsw;  var 1..nc: v;   var 1..nc: t;

constraint wa != nt;
constraint wa != sa;
constraint nt != sa;
constraint nt != q;
constraint sa != q;
constraint sa != nsw;
constraint sa != v;
constraint q != nsw;
constraint nsw != v;
solve satisfy;

output ["wa=\(wa)\t nt=\(nt)\t sa=\(sa)\n",
        "q=\(q)\t nsw=\(nsw)\t v=\(v)\n",
         "t=", show(t),  "\n"];
```

A checker model for this model requires the values of the variables `wa`, `nt`, and so on, for each solution, and then has to test whether all constraints hold. The output of the checker model should contain a line starting with `CORRECT` if the solution passes the test, or `INCORRECT` if it doesn't.

Since these values will be fixed in any solution, checker models simply declare parameters with the same name as the model variables:

Listing 6.2: A MiniZinc checker model `aust.mzc.mzn` for the map colouring problem

```
int: wa;
int: nt;
int: sa;
int: q;
int: nsw;
int: v;
int: t;

output [
  if wa!=nt /\ wa!=sa /\ nt!=sa /\ nt!=q /\ sa!=q /\
     sa!=nsw /\ sa!=v /\ q!=nsw /\ nsw!=v
```

```
  then "CORRECT\n"
  else "INCORRECT\n"
  endif
];
```

Running the model and the checker will produce output like this:

```
wa=3   nt=2    sa=1
q=3    nsw=2   v=3
t=3
% Solution checker report:
% CORRECT
----------
```

The solution checker report is embedded as comments in the original output.

3.6.2.2 Detailed feedback

The basic checker model above only reports whether the solutions satisfy the constraints, but it doesn't provide any insights into the nature of the error if a constraint is violated.

We can use standard MiniZinc functionality to provide much more detailed feedback. The following checker model introduces a helper function `check` which outputs a detailed error message if a constraint doesn't hold. The results of all the checks are combined, and if any of the constraints was violated, the output is `INCORRECT`, otherwise it is `CORRECT`.

Listing 6.3: A checker model `aust-2.mzc.mzn` for the map colouring problem with more detailed error messages

```
int: wa;
int: nt;
int: sa;
int: q;
int: nsw;
int: v;
int: t;

test check(bool: b,string: s) =
  if b then true else trace_stdout("ERROR: "++s++"\n",false) endif;

output [
  if check(wa!=nt, "wa and nt have the same colour")
  /\ check(wa!=sa, "wa and sa have the same colour")
  /\ check(nt!=sa, "nt and sa have the same colour")
  /\ check(nt!=q, "nt and q have the same colour")
  /\ check(sa!=q, "sa and q have the same colour")
  /\ check(sa!=nsw, "sa and nsw have the same colour")
  /\ check(sa!=v, "sa and v have the same colour")
  /\ check(q!=nsw, "q and nsw have the same colour")
```

```
  /\ check(nsw!=v, "nsw and v have the same colour")
  then "CORRECT: all constraints hold.\n"
  else "INCORRECT\n"
  endif
];
```

However, the checker model will only report the first violated constraint, since the conjunction operator short-circuits the evaluation when one of its arguments is false. For example, if we remove all constraints from the original model, the output would be:

```
wa=3   nt=3    sa=3
q=3    nsw=3   v=3
t=3
% Solution checker report:
% ERROR: wa and nt have the same colour
% INCORRECT
----------
```

In order to get all error messages, we can force the evaluation of all checkers by creating an auxiliary array of check results:

Listing 6.4: A checker model `aust-3.mzc.mzn` for the map colouring problem without short-circuit evaluation

```
int: wa;
int: nt;
int: sa;
int: q;
int: nsw;
int: v;
int: t;

test check(bool: b,string: s) =
  if b then true else trace_stdout("ERROR: "++s++"\n",false) endif;

array[int] of bool: checks = [
  check(wa!=nt, "wa and nt have the same colour"),
  check(wa!=sa, "wa and sa have the same colour"),
  check(nt!=sa, "nt and sa have the same colour"),
  check(nt!=q, "nt and q have the same colour"),
  check(sa!=q, "sa and q have the same colour"),
  check(sa!=nsw, "sa and nsw have the same colour"),
  check(sa!=v, "sa and v have the same colour"),
  check(q!=nsw, "q and nsw have the same colour"),
  check(nsw!=v, "nsw and v have the same colour")
];

output [
  if forall(checks)
  then "CORRECT: all constraints hold.\n"
```

```
  else "INCORRECT\n"
  endif
];
```

Now the output contains all error messages (for the case where the model has no constraints):

```
wa=3   nt=3    sa=3
q=3    nsw=3   v=3
t=3
% Solution checker report:
% ERROR: nsw and v have the same colour
% ERROR: q and nsw have the same colour
% ERROR: sa and v have the same colour
% ERROR: sa and nsw have the same colour
% ERROR: sa and q have the same colour
% ERROR: nt and q have the same colour
% ERROR: nt and sa have the same colour
% ERROR: wa and sa have the same colour
% ERROR: wa and nt have the same colour
% INCORRECT
----------
```

3.6.2.3 Instance data in checker models

The map colouring example was quite simple because the model did not contain any parameter declarations. For a model that is parameterised, the checker model simply contains the same parameter declarations. MiniZinc will then pass the actual parameters of the instance being solved to the checker model.

For example, the following checker model could be used for the n-Queens problem.

Listing 6.5: A checker model `nqueens.mzc.mzn` for the n-Queens problem

```
int: n; % number of queens
set of int: ROW = 1..n;
set of int: COL = 1..n;
array[int] of int: q; % col of queen in each row

test check(bool: b,string: s) =
  if b then true else trace_stdout("ERROR: "++s++"\n",false) endif;

output [
  if check(index_set(q)=1..n, "ERROR: array q should have index set 1..\(n)")
  /\ forall(i in 1..n)(check(q[i] in 1..n, "ERROR: q[\(i)] should have a␣
↪value in 1..\(n)"))
  /\ forall(r1, r2 in 1..n where r1 < r2)
          (check(q[r1] != q[r2],
           "queens in rows \(r1) and \(r2) are on same column\n")
           /\
```

```
            check(q[r1]+r1 != q[r2]+r2,
            "queens in rows \(r1) and \(r2) are on same up diagonal\n")
            /\
            check(q[r1]-r1 != q[r2]-r2,
            "queens in rows \(r1) and \(r2) are on same down diagonal\n")
           )
  then "CORRECT: All constraints hold"
  else "INCORRECT" endif ];
```

The checker model first makes sure that the solution has the right dimensions (correct array index set, and each variable is assigned a value in the correct domain), and then uses standard MiniZinc constructs to check each constraint.

3.6.2.4 Checking optimisation problems

Optimisation problems pose a difficulty for automatic checking. When a solver claims to prove optimality, we cannot easily verify this claim without solving the problem again (using a different model that is known to be correct). At the moment, solution checking for optimisation problems is restricted to checking that the objective has been computed correctly.

To check that the objective value is calculated correctly, define a parameter called `_objective` of the appropriate type (`int` or `float`) in the checker. The example in the next section illustrates this.

3.6.2.5 Hidden variables

For many models, the decision variables which describe the solution may not be the variables that are natural to describe the constraints. There may be other internal variables which are functionally defined by the decision variables, and which are likely to be used in the model for building constraints and/or are much more natural for describing the correctness of a candidate solution.

Consider the problem of lining up n people numbered from 1 to n for a photo in a single line. We want to make sure that there are no more than two people of the same gender (male, female or other) in sequence in the line. We want to minimize the total distance between the pairs of people who are numbered consecutively. The decisions are an array `pos` which for each person gives their position in the line. A correct model for this is given in Listing 6.6.

Listing 6.6: A model `photo.mzn` for the photo lineup problem

```
int: n;                           % number of people
set of int: PERSON = 1..n;        % set of people
enum GENDER = { M, F, O };        % set of genders
array[PERSON] of GENDER: g;       % the gender of each person
set of int: POSN = 1..n;          % set of positions

array[PERSON] of var POSN: pos;   % decisions: a position for each person

array[POSN] of var PERSON: who;   % view: a person for each position
```

```
include "inverse.mzn";
constraint inverse(pos,who);     % channel from decisions to view
constraint forall(i in 1..n-2)
                 (g[who[i]] != g[who[i+1]] \/ g[who[i+1]] != g[who[i+2]]);

solve minimize sum(i in 1..n-1)(abs(pos[i] - pos[i+1]));
```

A critical part of learning how to model this problem is to realise that it is worth introducing the inverse of the `pos` variables, called `who` in this model. The `who` array makes it much easier to specify that no more than two people of the same gender can be adjacent. The checker model **should not** include the `who` variables, because that would effectively give away the key to the solution.

The use of hidden variables (such as `who`) makes checking harder in two ways. First, we may need to define these in the checker model as decision variables (rather than parameters), and MiniZinc will then need to invoke a solver to generate the values of these variables in order to check a given solution. Second, given an incorrect candidate solution, there may be no possible value for these hidden variables, hence we have to guard the checking to ensure we do not assume that there is some value for the hidden variables.

Consider this data file for the photo problem:

Listing 6.7: Example data for the `photo.mzn` photo lineup problem (`photo.dzn`)

```
n = 9;
g = [M,M,M,M,F,F,F,M,M];
```

```
n = 9;
g = [M,M,M,M,F,F,F,M,M];
```

and assume that an incorrect model returns the erroneous candidate solution `pos = [1,2,3,4,5,6,7,2,9]; _objective=25;`. If we simply assert `inverse(pos,who)` in the checker model, then this constraint will fail since there is no inverse of this position array, because persons 2 and 8 are both at position 2. Hence the checker must guard the inverse constraint, and only when that succeeds use the computed values of the `who` variables for checking the solution. A checking model for the photo problem might look like this:

Listing 6.8: A checker model `photo.mzc.mzn` for the photo lineup problem

```
int: n;
set of int: PERSON = 1..n;
enum GENDER = { M, F, O };
array[PERSON] of GENDER: g;
set of int: POSN = 1..n;
array[int] of int: pos;

array[POSN] of var PERSON: who;
int: _objective;
```

```
test alldifferent(array[int] of int: x) =
                forall(i,j in index_set(x) where i < j)(x[i] != x[j]);

include "inverse.mzn";
constraint if alldifferent(pos) then inverse(pos,who)
           else forall(i in 1..n)(who[i] = 1) endif;

output [if
  check_array(pos, n, POSN, "pos") /\
  check_alldifferent(pos,"pos") /\
  check_array(fix(pos), n, PERSON, "who") /\
  forall(i in 1..n-2)
        (check(g[fix(who[i])] != g[fix(who[i+1])] \/
               g[fix(who[i+1])] != g[fix(who[i+2])],
               "three people of the same gender \(g[fix(who[i])]) in␣
↪positions \(i)..\(i+2)\n")) /\
  let { int: obj = sum(i in 1..n-1)(abs(pos[i] - pos[i+1])); } in
    check(obj = _objective, "calculated objective \(obj) does not agree with␣
↪objective from the model (\(_objective))\n")
  then
    "CORRECT: All constraints hold"
  else
    "INCORRECT"
  endif];

test check(bool: b,string: s) =
  if b then true else trace_stdout("ERROR: "++s++"\n",false) endif;

test check_alldifferent(array[int] of int: x, string: name) =
  forall(i, j in index_set(x) where i < j)
        (check(x[i] != x[j], name ++ "[\(i)] = \(x[i]) = " ++
         name ++ "[\(j)] " ++
         "when they should be different\n"));

test check_int(int: x, set of int: vals, string: name) =
  check(x in vals, "integer \(name) is not in values \(vals)\n");

function bool: check_array(array[int] of int: x, int: length, set of int:␣
↪vals, string: name) =
  check(length(x) = length, "array \(name) is not of length \(length)\n") /\
  forall(i in index_set(x))
        (check(x[i] in vals, "element \(i) of array \(name), \(x[i]) is not␣
↪in values \(vals)\n"));
```

The checker model first tests whether the given `pos` array satisfies the `alldifferent` property (using a custom `test` for `alldifferent` on a par array). If it passes the test, the `inverse` constraint is applied. Otherwise, the `who` array is simply fixed to a list of ones.

The check for the `alldifferent` constraint is a good example for a checker that tries to give a detailed description of the error.

CHAPTER 3.7

Assumptions (experimental)

Note:

The assumption interface is **experimental**. Its modelling syntax, the `fzn_assume` solver interface and the `%%%mzn-core` output described below may change or be removed in future versions of MiniZinc.

Some solvers can solve a model *under assumptions*: a set of Boolean conditions that are assumed to hold during the solving process. If the model is unsatisfiable under those assumptions, such a solver can report an *unsatisfiable core*: a subset of the assumptions that is sufficient to cause the unsatisfiability. This is useful for explanation and debugging: rather than just learning that a model is unsatisfiable, you learn *which* of your assumptions are in conflict.

MiniZinc exposes this through the `assume` predicate. The assumptions you provide are recorded, and any core reported by the solver is translated back into the original MiniZinc expressions you wrote.

Only some solvers support this interface. If the selected solver does not, the assumptions are posted as ordinary hard constraints. The model is still solved correctly, but no unsatisfiable core is reported. We also issue a warning to that effect.

3.7.1 Using `assume`

The predicate lives in the experimental library in `experimental/assume.mzn` and must be included explicitly. Give it the conditions that you want to assume as a single root-context constraint:

```
include "experimental/assume.mzn";

var 1..10: x;
var 1..10: y;
```

```
constraint assume([x + y < 5, x > 8]);

solve satisfy;
```

Each element of the array is assumed to be `true` while solving. `assume` may only be used in a root context (i.e. as a top-level `constraint`); using it in a reified or negated position is an error, because assumptions cannot be conditionally imposed.

An assumption that is fixed to `false` makes the model trivially unsatisfiable. If this is detected when compiling the `assume` call then the offending assumption is immediately reported. This detection is best effort: an assumption that is only fixed to `false` by *other* parts of the model might instead be reported by the solver as a simple core containing only `false`.

3.7.2 Interpreting the core

When the solver reports a core, MiniZinc prints it in terms of the original assumption expressions. In the default output this is a comment line:

```
%%%mzn-core: [x + y < 5, x > 8]
=====UNSATISFIABLE=====
```

When solving an optimisation problem, a solver may also report a core to justify that the solution it found is optimal; the same `%%%mzn-core` line is used in that case.

Expressions that are not individual array elements are reported as an indexed reference into the argument. For example, if the assumptions come from a function call `asmp(x)` returning an array, the core is reported as `asmp(x)[1]`, `asmp(x)[2]`, and so on.

With the *machine-readable JSON output* (page 551) (`--json-stream`) the core is reported as a `core` message instead (see *Machine-readable JSON output format* (page 551)).

CHAPTER 3.8

Black-Box Propagators (experimental)

Warning:

Black-box propagators are an **experimental** feature. The modelling interface, the external interfaces described here, and the set of supporting solvers may change in future releases.

A *black-box propagator* is a constraint whose propagation is performed by an external implementation (e.g. an executable program or a dynamically loaded library) rather than by decomposing the constraint into solver-primitive constraints. This makes it possible to add problem-specific propagation without modifying a solver, and to couple a model with external components such as simulations or legacy code (hybrid white-box/black-box optimisation).

A supporting solver executes the black-box through a generic callback interface, so the model remains solver-independent. Two kinds of propagator are supported:

- a **value propagator** is scheduled once all of its input variables are fixed, and either checks the assignment (a *relational* propagator) or computes output values (a *functional* propagator);
- a **bounds propagator** is scheduled whenever an input bound changes, and computes tightened bounds for the variables.

Because external evaluations can be expensive, solvers schedule these propagators at low priority.

This chapter describes how to define black-box propagators in a model, and the two external interfaces a black-box can be implemented against: a *dynamically loaded library* (page 231) and an *executable subprocess* (page 232).

3.8.1 Defining a black-box propagator in MiniZinc

The interface is provided by an experimental library that must be included:

```
include "experimental/blackbox.mzn";
```

A black-box propagator is declared as a **predicate or function without a body**, carrying two annotations: a *kind* annotation that selects the type of propagator, and a *source* annotation (`blackbox_exec` or `blackbox_dll`, see *Selecting the external implementation* (page 229)) that says where the external implementation lives. The MiniZinc compiler generates the body of the declaration automatically.

The decision-variable arguments of a propagator are restricted to **at most one** `list of var int` and **at most one** `list of var float`. Any number of fixed (parameter) arguments may also be given; these are not automatically passed to the external function but may be used by the companion functions described below.

3.8.1.1 Value propagators

A **relational** value propagator is a predicate. It is evaluated only once all of its variables are fixed and reports whether the assignment satisfies the constraint:

```
predicate all_ok(list of var int: xs)
    ::minizinc_value_propagator
    ::blackbox_exec("./checker");

constraint all_ok(x);
```

A **functional** value propagator is a function returning `list of var int` or `list of var float`; the external function computes the output values rather than merely checking them. Because the length of the result cannot be known in general, a functional propagator must also provide a `par` overload of the same function which the compiler evaluates to determine the number of elements in the output:

```
% par overload: the propagation function, used to size the output
function list of int: transform(list of int: xs) =
    [ xs[i] + i | i in index_set(xs) ];

% the black-box declaration (no body)
function list of var int: transform(list of var int: xs)
    ::minizinc_value_propagator
    ::blackbox_dll("libtransform.so");

constraint y = transform(x);
```

3.8.1.2 Bounds propagators

A **bounds** propagator is a predicate that receives the current bounds of its variables and returns tightened bounds:

```
predicate my_bounds(list of var int: xs)
    ::minizinc_bounds_propagator
    ::blackbox_exec("./bounds");

constraint my_bounds(x);
```

Lazy clause generation solvers require a *reason* for every bound change. By default the propagator uses a conservative reason in which every input bound explains every output bound. Tighter explanations can be provided by defining a companion `_reason` function, evaluated once during compilation, that returns for each variable index the literals explaining its lower- and upper-bound changes (using `PR_LB` / `PR_UB` of the `PropBnd` enum):

```
function list of tuple(int,
                       list of tuple(int, PropBnd),
                       list of tuple(int, PropBnd)):
    my_bounds_reason(list of var int: xs) =
    [ (i, [(j, PR_UB) | j in index_set(xs) where j != i],
          [(j, PR_LB) | j in index_set(xs) where j != i])
    | i in index_set(xs) ];
```

The reason template is reused during search, so it must be a sound over-approximation for all reachable domains.

Besides explaining propagated changes, the `_reason` template also identifies exactly which input bounds the propagator inspects. A solver can use this to schedule the propagator more precisely, waking it only when one of those bounds changes rather than on every change to an input bound.

Note:

A propagator-kind annotation must be paired with a `blackbox_exec` or `blackbox_dll` source annotation that provides the external implementation.

3.8.2 Selecting the external implementation

Two annotations select how the black-box is executed:

```
annotation blackbox_exec(string: executable, list of string: args);
annotation blackbox_dll(string: library, list of string: args);
```

- `blackbox_exec` runs an external program (see *Executable (subprocess) mode* (page 232)). The program is launched directly (no shell); `args` are appended to its command line.

- `blackbox_dll` loads a dynamically loaded library (see *Dynamically loaded library (DLL) mode* (page 231)); `args` are passed to the library's `fzn_init` function.

For convenience a single-argument form is also available and is equivalent to passing an empty argument list:

```
predicate p(list of var int: xs)
    ::minizinc_value_propagator ::blackbox_exec("./prog");          % no args
predicate q(list of var int: xs)
    ::minizinc_value_propagator ::blackbox_exec("./prog", ["-v"]);  % with␣
↪args
```

3.8.2.1 Locating the executable or library

The model can refer to the executable or library by a plain, platform-independent name (without an extension and, for libraries, without the `lib` prefix). When compiling to FlatZinc, MiniZinc resolves that name to an **absolute path** on the compilation host and stores it in the generated annotation, so the FlatZinc is self-contained. Resolution applies the platform naming convention (`lib<name>.so` / `lib<name>.dylib` / `<name>.dll`) and searches, in order:

- the directory of the file that declares the propagator (so a helper shipped alongside a model or a reusable library is found);
- for executables, the `PATH`; for libraries, the dynamic-loader search path (`LD_LIBRARY_PATH` / `DYLD_LIBRARY_PATH` / `PATH`) and the usual system library directories.

A name that is already an absolute path, or that contains a directory separator, is used as given (resolved relative to the declaring file). If the target cannot be found, the name is left unchanged. The solver can still apply its own resolving at runtime when it loads the library or launches the process.

3.8.3 The external interface

Regardless of the mode, a black-box implementation is a single function mapping input values to output values. The meaning of the arrays depends on the kind of propagator:

Propagator	Input	Output
value, relational	the assigned value of each input variable	a single integer: `1` if the constraint holds, `0` if it is violated
value, functional	the assigned value of each input variable	the computed value of each output variable
bounds	two entries per variable, a lower then an upper bound (`lb0`, `ub0`, `lb1`, `ub1`, ...)	two entries per variable, the tightened lower and upper bounds

Integer variables use the integer channel and float variables the float channel. When a propagator has no variables of one type, that channel is empty.

Note:

Concurrency. By default, most solvers evaluate propagators from a single thread, but a solver running in parallel (for example with the `-p` flag) may evaluate a black box from several workers at once. The two modes handle this differently:

- In **subprocess mode** the solver starts a separate process for each worker, so each process only ever handles one request at a time and needs no internal synchronisation.
- In **dynamically loaded library mode** the solver gives each worker its own *instance* created using `fzn_clone` (see *below* (page 231)). `fzn_blackbox` is therefore only ever called on a given instance by one thread at a time, while different instances may run concurrently.

3.8.3.1 Dynamically loaded library (DLL) mode

When the black-box is a dynamically loaded library, the solver loads it (e.g. with `dlopen`) and resolves the symbol `fzn_blackbox`:

```
void fzn_blackbox(
  void          *instance,
  const int64_t *int_in,    size_t int_in_size,
  const double  *float_in,  size_t float_in_size,
  int64_t       *int_out,   size_t int_out_size,
  double        *float_out, size_t float_out_size
);
```

The function is called once per propagation: the solver populates the input arrays and the function writes its result into the (pre-sized) output arrays, following the encoding in the table above. `instance` identifies the per-use state the call belongs to (see below); a stateless library can ignore it.

On Windows the entry points use the `__stdcall` calling convention and must be exported explicitly (for example with `__declspec(dllexport)`, otherwise `cl /LD` exports no symbols); on other platforms the ordinary C calling convention is used. A mismatched convention corrupts the stack on 32-bit Windows. The templates below wrap this in a small macro (in Rust, the equivalent is `extern "system"`).

Instances

Each dynamically loaded library has a single global namespace, so per-use state cannot live in global memory. Distinct constraints, and concurrent usage would clobber one another. As such, we capture such state in an *instance*, for each district constraint concurrent usage, by exporting three optional functions:

```
void *fzn_init (const char **args, size_t args_size);
void *fzn_clone(void *instance);
void  fzn_free (void *instance);
```

- `fzn_init` is called once per black-box constraint, with the arguments from the `blackbox_dll` annotation (an `argv`-style list, possibly empty). It returns an opaque

instance pointer that is passed to every `fzn_blackbox` call for that constraint. It can, for example, be use it to parse the arguments, load a program, or allocate working state.

- `fzn_clone` returns an **independent** copy of an instance, behaving identically to its source. The solver calls it to give each parallel worker its own instance, so that concurrent `fzn_blackbox` calls never share one.
- `fzn_free` releases an instance. Every pointer returned by `fzn_init` or `fzn_clone` is freed exactly once.

If the library does not export `fzn_init` the instance is `NULL` and initialisation is skipped. A library that exports `fzn_init` must also export `fzn_clone`. A **thread-safe** library that keeps no mutable per-call state need not copy anything: it can **reference-count** the state across `fzn_init` / `fzn_clone` / `fzn_free` and simply return the *same* pointer from `fzn_clone` (bumping the count), freeing the state only when the last reference is released. All workers then share one instance with no duplication, while still using the instance-based interface.

Templates: `C`, `Rust`.

3.8.3.2 Executable (subprocess) mode

In subprocess mode the solver starts the program once (with the arguments from the annotation) and then communicates over the program's standard input and output streams, enabling language-agnostic implementations. Each propagation is a single request line followed by a single response line:

```
request : <int values, comma-separated>;<float values, comma-separated>\n
response: <int outputs, comma-separated>;<float outputs, comma-separated>\n
```

The integer values and floating-point values are comma-separated, the integer and floating-point segments are separated by a semicolon, and each message is terminated by a newline. A segment is empty when there are no values of that type. For example, sending the integers `5` and `-7` together with the floating-point values `2.5` and `1.125` is written as:

```
5,-7;2.5,1.125\n
```

For a bounds propagator each segment carries two entries per variable (a lower then an upper bound), following the encoding in the table above.

The number of output values in the stream must match the number of output variables in the constrain call. If the response does not contain exactly that many values, this should be considered a runtime error.

The program must:

- read one request line, compute its response, and write exactly one response line, in the same format;
- **flush** its standard output after each response;
- keep looping until standard input reaches end-of-file, then exit.

Anything the program writes to its standard error stream is ignored by the solver and may be used for logging.

Templates: `C`, `Rust`, `Python`.

CHAPTER 3.9

Globalizer

Globalizer[1] analyses a model and set of data to provide suggestions of global constraints that can replace or enhance a user's model.

3.9.1 Basic Usage

To use Globalizer simply execute it on a model and set of data files:

```
minizinc --solver globalizer model.mzn data-1.dzn data-2.dzn
```

Note: Globalizer may also be executed on a non-parameterised model with no data files.

The following demonstrates the basic usage of Globalizer. Below, we see a simple model for the car sequencing problem[2], `cars.mzn`.

Listing 9.1: Model for the car sequencing problem (`cars.mzn`).

```
% cars.mzn
include "globals.mzn";

int: n_cars; int: n_options; int: n_classes;

set of int: steps = 1..n_cars;
set of int: options = 1..n_options;
set of int: classes = 1..n_classes;

array [options] of int: max_per_block;
array [options] of int: block_size;
array [classes] of int: cars_in_class;
array [classes, options] of 0..1: need;
```

[1] Leo, K. et al., "Globalizing Constraint Models", 2013.
[2] http://www.csplib.org/Problems/prob001/

```
% The class of car being started at each step.
array [steps] of var classes: class;

% Which options are required by the car started at each step.
array [steps, options] of var 0..1: used;

% Block p must be used at step s if the class of the car to be
% produced at step s needs it.
constraint forall (s in steps, p in options) (used[s, p]=need[class[s], p]);

% For each option p with block size b and maximum limit m, no consecutive
% sequence of b cars contains more than m that require option p.
constraint
    forall (p in options, i in 1..(n_cars - (block_size[p] - 1))) (
        sum (j in 0..(block_size[p] - 1)) (used[i + j, p])
            <= max_per_block[p]);

% Require that the right number of cars in each class are produced.
constraint forall (c in classes) (count(class, c, cars_in_class[c]));

solve satisfy; % Find any solution.
```

And here we have a data file `cars_data.dzn`.

Listing 9.2: Data for the car sequencing problem (`cars_data.dzn`).

```
% cars_data.dzn
n_cars = 10;
n_options = 5;
n_classes = 6;
max_per_block = [1, 2, 1, 2, 1];
block_size =    [2, 3, 3, 5, 5];
cars_in_class = [1, 1, 2, 2, 2, 2];

need =  array2d(1..6, 1..5, [
    1, 0, 1, 1, 0,
    0, 0, 0, 1, 0,
    0, 1, 0, 0, 1,
    0, 1, 0, 1, 0,
    1, 0, 1, 0, 0,
    1, 1, 0, 0, 0
]);
```

Executing Globalizer on this model and data file we get the following output:

```
% minizinc --solver globalizer cars.mzn cars_data.dzn
cars.mzn|33|12|33|69 [  ] gcc(class,cars_in_class)
cars.mzn|33|35|33|68 [  ] count(class,c,cars_in_class[c])
cars.mzn|28|27|28|65;cars.mzn|29|9|30|32 [  ] sliding_sum(0,max_per_block[p],
↪block_size[p],used[_, p])
```

Each line of output is comprised of three elements.

1. Expression locations (separated by semicolon ';') that the constraint might be able to replace.
2. (Between square brackets '[' ']') the context under which the constraint might replace the expressions of 1.
3. The replacement constraint.

From the example above we see that a `gcc` constraint and a `sliding_sum` constraint can replace some constraints in the model. Taking the `sliding_sum` case, we see that it replaces the expression `cars.mzn|28|27|28|65` which corresponds to `i in 1..(n_cars - (block_size[p] - 1))` and `cars.mzn|29|27|28|65` which corresponds to the `<=` expression including the `sum`. The original constraint can be replaced with:

```
constraint forall (p in options) (
    sliding_sum(0, max_per_block[p], block_size[p], used[..,p]));
```

3.9.2 Caveats

MiniZinc syntax support. Globalizer was implemented with support for a subset of an early version of the MiniZinc 2.0 language. As a result there are some limitations. First, Globalizer does not support `set` variables or `enum` types. The array slicing syntax supported in MiniZinc was not decided upon when Globalizer was implemented so it uses an incompatible syntax. Globalizer uses _ instead of .. This can be seen above in the translation of `used[_,p]` to `used[..,p]`.

New constraint. A special two argument version of the `global_cardinality` constraint called `gcc` has been added which is not in the standard library. It is defined as follows:

```
predicate gcc(array[int] of var int: x, array[int] of var int: counts) =
  global_cardinality(x,
                     [ i | i in index_set(counts) ],
                     array1d(counts));
```

3.9.3 Supported Constraints

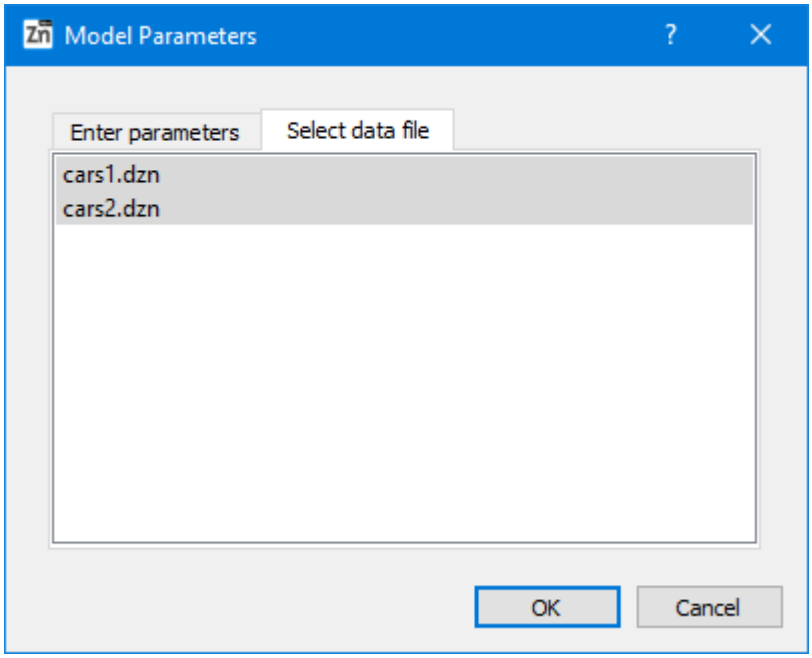
Globalizer currently supports detection of the following constraints:

```
alldifferent                alldifferent_except_0      all_equal_int
bin_packing                 bin_packing_capa           bin_packing_load
binaries_represent_int      binaries_represent_int_3A ␣
↪binaries_represent_int_3B
binaries_represent_int_3C   channel                    channelACB
count_geq                   cumulative_assert          decreasing
gcc                         global_cardinality         inverse
lex_lesseq_int_checking     lex2_checking              maximum_int_checking
minimum_int_checking        member                     nvalue
```

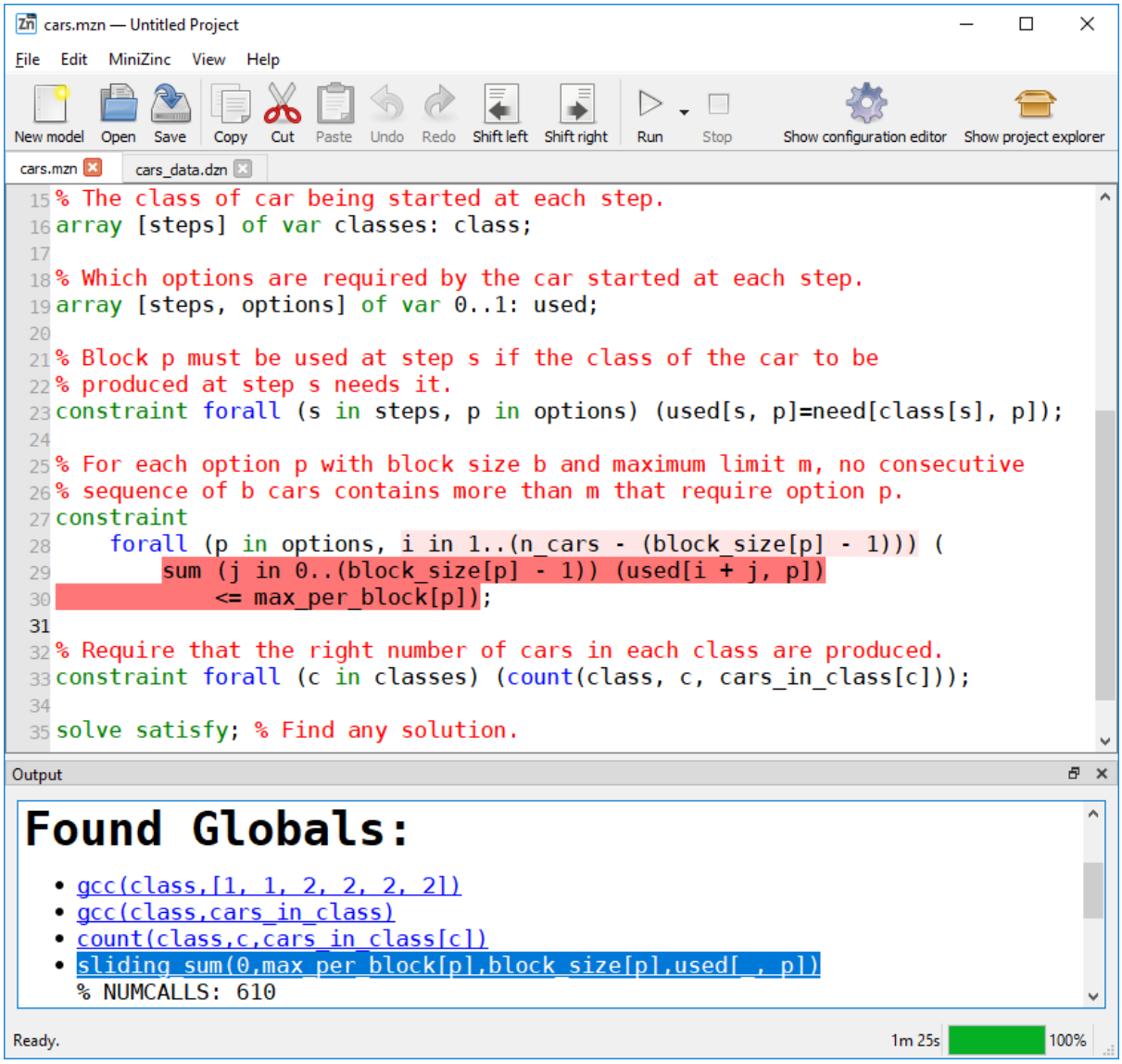
```
strict_lex2_checking        subcircuit_checking        true
atleast                     atmost                     bin_packing_load_ub
circuit_checking            count                      diffn
distribute                  element                    increasing
lex_less_int_checking       sliding_sum                sort_checking
unary                       value_precede_checking
```

3.9.4 Using Globalizer in the MiniZinc IDE

To use the Globalizer in the MiniZinc IDE, open a model, and several data files. Select `Globalizer` from the solver configuration dropdown menu. Click the solve button (play symbol). If you did not select any data file, a dialog should pop up that allows you to select data files. To select multiple data files hold the `Ctrl` or `Cmd` key while clicking on each data file.



Click run. While processing your model and data files the MiniZinc IDE will display a progress bar on the toolbar.



Any discovered global constraints will be listed in the output pane of the IDE. Clicking on one of these constraints will highlight in the model the expressions that might be replaceable or strengthened by the constraint.

3.9.5 How it works

A summary of the algorithm is presented here. For a more detailed exploration of the approach see the Globalizing Constraint Models paper[Page 235, 1].

- **Normalize.** In this step the model is transformed to make analysis easier.
  - Conjunctions of constraints are broken into individual constraints. For example: `C1 /\ C2;` becomes `C1; C2;`
  - Conjunctions under `forall` constraints are broken into individual `forall` constraints. For example: `forall(...) (C1 /\ C2);` becomes `forall(...) (C1); forall(...) (C2)`
- **Generate submodels.** In this step all subsets containing 1 or 2 constraints (this can be configured using `--numConstraints` argument) are enumerated.

- **Instantiate and unroll into groups.** Each submodel is instantiated with the provided data files. These instantiated submodels are further used to produce more submodels by unrolling loops. For example, a constraint such as `forall(i in 1..n)(c(i));` will be used to produce the constraints: `c(1)` and `c(n)` for each instantiation of n in the data files. All instantiated submodels are then grouped together.
- **Process groups.** For each submodel in a group a set of 30 random solutions are found. (configurable using the `--randomSolutions` argument). A template model with all of these random solutions is created.

  The different variables (including arrays and array accesses) and parameters used by a submodel are collected into a set of potential arguments along with the constant 0 and a special blank symbol representing an argument that can be inferred based on others used.

  The list of constraints above is filtered based on the arguments available. For example, the `alldifferent` constraint will be removed from the list of candidates if the submodel does not reference any arrays of variables.

  Finally the set of constraints are filtered by adding them to the template and solving it. If the template is satisfiable, the constraint accepts all of the random solutions.

  30 sample solutions for each candidate constraint are generated. (configurable using the `--sampleSolutions` argument). The candidate is then ranked based on how many of these solutions are also solutions to the original submodel. If its score is less than some threshold the candidate is removed from the set of candidates for this group and will not be tested on later submodels.
- **Report.** The remaining candidates are presented to the user.

3.9.6 Performance tips

Globalizing constraint models can be a time consuming task. If

- **Use small or relatively easy instances.** Globalizer solves many subproblems while processing your model and data files. Using easier instances can speed up the process considerably.
- **Disable the initial pass.** As discussed above, Globalizer performs two passes. The first pass tries to detect alternate viewpoints that can be added to your model. If you are confident that this will not be useful we recommend disabling this first pass using the `--no-initial-pass` argument.
- **Narrow search using filters.** Globalizer attempts to match a large selection of global constraints to subproblems in your model. If you wish to check if only a few specific constraints are present you can focus Globalizer using the `--constraintFilter` or `-f` arguments followed by a comma separated list of strings. Only global constraints where one of the strings is a substring will be included in the search.
- **Disable implies check.** The implication check can also be disabled to improve performance. This results in less accurate results but may still help a user to understand the structure of their model.
- **Free-search.** To improve accuracy and to avoid false positives, subproblems solved by Globalizer are solved using a random heuristic and restarts. If the sub-

problems are particularly difficult to solve a random heuristic may be prohibitive and the solver may not be able to produce enough samples within the solving time-limit. In these circumstances a user can either increase the solver timeout using the `-t` argument followed by the number of milliseconds a solver should be given to find samples. Alternatively the `--free-search` argument can be used to force Globalizer to use the solver's free search to find samples. This has the downside of reducing the diversity of samples but allows enough samples to be found to allow suggested globals to be found.

3.9.7 Limitations / Future work

- Globalizer supports only a subset of the MiniZinc language and as such cannot be executed on all MiniZinc models.
- There are some relatively cheap syntactic approaches that should be performed before Globalization that currently is not implemented. For example, there are several common formulations of an `alldifferent` constraint that can be detected syntactically. This would be much cheaper than using Globalizer.

CHAPTER 3.10

FindMUS

Version: 0.6.0

FindMUS[1] lists unsatisfiable subsets of constraints in your MiniZinc model. These subsets, called Minimal Unsatisfiable Subsets can help you to find faults in your unsatisfiable constraint model. FindMUS uses the hierarchical structure provided by the model to guide its search.

3.10.1 Basic Usage

To use FindMUS on the command line simply execute it on a model and set of data files by typing:

```
findMUS model.mzn data-1.dzn
```

This will perform a search for a single instance level MUS. To acquire a low level MUS the `--paramset fzn` argument can be used. To search for more than one MUS use the `-a` or `-n <count>` arguments. By default FindMUS will use `gecode_presolver` for checking the satisfiability of a subset of constraints. This can be changed using the `--subsolver` argument. For example, given a model that is better suited to a Mixed Integer Programming solver, the argument `--solver cbc` could be used to instruct FindMUS to flatten the model with the linear library and use the CBC solver for satisfiability checking.

Note: FindMUS requires a fully instantiated constraint model.

3.10.1.1 Commandline arguments

The FindMUS tool supports the following arguments:

Driver options

The driver creates the map solver and sub-solver and requests MUSes from FindMUS's enumeration algorithm HierMUS.

[1] Leo, K. et al., "Debugging Unsatisfiable Constraint Models", 2017.

`-n <count>` Stop after the *n* th MUS is found (Default: 1)

`-a` Find all MUSes (equivalent to `-n 0`)

`-t` or `--timeout <s&gt` Stop search after *s* seconds (Default: 1800)

`--paramset hint,mzn,fzn`
: Use a preset collection of parameters. (Default: mzn) The `mzn` and `fzn` options can acquire MUSes in more detail.

  `hint` –structure gen –depth mzn –shrink-alg map_lin

  `mzn` –structure normal –depth mzn –shrink-alg map_qx

  `fzn` –structure normal –depth fzn –shrink-alg map_qx

`--no-leftover`
: Do not print leftover candidate on timeout If FindMUS is exploring an unsatisfiable subset when it runs out of time, it behaves as if the set could not be reduced to a smaller set and prints it as an MUS. The `--no-leftover` setting disables this behaviour as these sets can sometimes be quite large.

`--frequent-stats` Print statistics after every MUS discovered.

`--no-progress` Disable `%%%mzn-progress` output

`--no-stats` Disable output of initial parsing stats and summary stats

`--output-{html, json, brief, human}` Changes output mode

  `html` HTML output mode for use with the MiniZinc IDE

  `json` Simple json output (each MUS should be parsed separately)

  `brief` Same as the default output mode but without outputting the traces (paths)

  `human` Tries to format the MUS in a more readable form grouping constraints by their constraint names

Compiler Options

`-g` or `--domains` Record domain changes during compilation (same as `minizinc -c -g`)

`--verbose-compile` Pass `--verbose` to MiniZinc compiler

Enumeration options

The enumeration algorithm (HierMUS) explores the constraint hierarchy provided in the user's model and proposes potential MUSes to the sub-solver. A `shrink` algorithm is used to reduce found UNSAT subsets to MUSes. There are currently four available shrink algorithms available with FindMUS.

`--shrink-alg lin,map_lin,qx,map_qx,qx2`
: Select shrink algorithm (Default: map_lin)

  `lin` simple linear shrink algorithm.

  `map_lin` alternative implementation of the linear algorithm that updates a global map of explored subsets while shrinking the subset.

`qx` uses the `QuickXplain`[4] algorithm to shrink the subset to a MUS.

`map_qx` is an alternative implementation of `QuickXplain` that uses the global map to split subsets, and to avoid repeating any SAT checks while shrinking

`qx2` a lightweight alternative to `map_qx` that only uses the map to avoid repeating SAT checks.

`--depth mzn,fzn,<n&gt`
How deep in the tree should search explore. (Default: mzn)

`mzn` expands the search as far as the point when the compiler leaves the MiniZinc model.

`fzn` expands search as far as the FlatZinc constraints.

`<n&gt` expand search to the *n* level of the hierarchy.

`--restarts` Enable restart to flat mode (plain Marco) if large run of SAT sets encountered

`--seed <n>` Set the random seed used for splitting with `map_qx`

Subsolver options

FindMUS can be used in conjunction with any FlatZinc solver. These options mimic the `minizinc` arguments `--solver` and `--fzn-flags`. The behavior of these arguments is likely to change in later versions of the tool.

`--subsolver <s&gt`
Use solver *s* for SAT checking. (Default: "gecode")

`--solver-flags <f&gt`
Pass flags *f* to sub-solver. (Default: empty)

`--solver-timelimit <ms&gt`
Set hard time limit for solver in milliseconds. (Default: 1100)

`--adapt-timelimit`
Automatically tighten solver time-limit based on performance during initial sanity check (prove background is SAT, background + entire foreground is UNSAT)

Filtering options

FindMUS can include or exclude constraints from its search based on the expression and constraint name annotations as well as properties of their paths (for example, line numbers). These filters are currently based on substrings but in future may support text spans and regular expressions.

`--soft-defines`
Consider functional constraints as part of MUSes

`--hard-domains`
Consider domain constraints (added by `-g` option) as background

`--named-only`
Only consider constraints annotated with string annotations

`--filter-mode fg,ex` Filtering mode

[4] Junker, U. et al., "QUICKXPLAIN: preferred explanations and relaxations for over-constrained problems", 2004.

`fg` Foreground: Excluded items go to the background

`ex` Exclusive: Excluded items are omitted entirely (Useful if they are part of MUSes you are not interested in)

`--filter-named <names>` `--filter-named-exclude <names>` Include/exclude constraints with names that match *sep* separated *names*

`--filter-path <paths>` `--filter-path-exclude <paths>` Include/exclude based on *paths*

`--filter-sep <sep>` Separator used for named and path filters

Structure options

The structure coming from a user's model can significantly impact the performance of a MUS enumeration algorithm. The hierarchy is constructed in two steps. The first takes the existing model structure and can modify it in several ways. The second step adds extra binary structure.

`--structure flat,gen,normal,mix`

Alters initial structure: (Default: `gen`)

`flat` Remove all structure

`gen` Remove instance specific structure (merges constraints)

`normal` No change

`mix` Apply `gen` before `normal`

`idx` Remove all location information

`idxmix` Apply `idx` before `normal`

`--no-binarize`

By default FindMUS will introduce extra binary structure to the hierarchy. This option disables this behaviour (This is likely to be the default in future since QX performs a type of binarization).

Verbosity options

`--verbose-{map,enum,subsolve} <n>` Set verbosity level for different components. Currently `n` can only take values 0, 1, and 2;

`--verbose` Set verbosity level of all components to 1

Misc options

`--dump-dot <dot>` Write tree in GraphViz format to file <dot>

3.10.1.2 Example

The following demonstrates the basic usage of FindMUS on a simple example. Below, we see a model for the latin squares puzzle[2] with some incompatible symmetry breaking constraints added.

[2] https://en.wikipedia.org/wiki/Latin_square

Listing 10.1: Faulty model for Latin Squares (`latin_squares.mzn`).

```
% latin_squares.mzn
include "globals.mzn";

int: n = 3;
set of int: N = 1..n;
array[N, N] of var N: X;

constraint :: "ADRows"
    forall (i in N)
        (alldifferent(X[i,..])           :: "AD(row \(i))"
        );
constraint :: "ADCols"
    forall (j in N)
        (alldifferent(X[..,j])           :: "AD(col \(j))"
        );

constraint :: "LLRows"
    forall (i in 1..n-1)
        (lex_less(X[i,..], X[i+1,..])    :: "LL(rows \(i) \(i+1))"
        );
constraint :: "LGCols"
    forall (j in 1..n-1)
        (lex_greater(X[..,j], X[..,j+1]) :: "LG(cols \(j) \(j+1))"
        );

solve satisfy;

output [ show2d(X) ];
```

Here we have used the new constraint and expression annotations added in MiniZinc 2.2.0. Note that these annotations are not necessary for FindMUS to work but may help with interpreting the output. The first two constraints: `ADRows` and `ADCols` define the `alldifferent` constraints on the respective rows and columns of the latin square. The next two constraints `LLRows` and `LGCols` post `lex` constraints that order the rows to be increasing and the columns to be increasing. Certain combinations of these constraints are going to be in conflict.

Executing the command `findMUS -n 2 latin_squares.mzn` returns the following output. Note that the `-n 2` argument requests that just the first two MUSes are found.

```
FznSubProblem:  hard cons: 0    soft cons: 10   leaves: 10      branches: 11␣
↪   Built tree in 0.02854 seconds.
MUS: 7 3 6 4 8
Brief: gecode_all_different_int;@{ADCols@AD(col 1)}:(j=1)
gecode_all_different_int;@{ADCols@AD(col 2)}:(j=2)
gecode_array_int_lt;@{LGCols@LG(cols 1 2)}:(j=1)
gecode_array_int_lt;@{LLRows@LL(rows 1 2)}:(i=1)
```

```
gecode_array_int_lt;@{LLRows@LL(rows 2 3)}:(i=2)

Traces:
latin_squares.mzn|18|5|20|9|ca|forall;latin_squares.mzn|18|5|20|9|ac;
↪latin_squares.mzn|18|13|18|13|i=2;latin_squares.mzn|19|10|19|37|ca|lex_less
latin_squares.mzn|13|5|15|9|ca|forall;latin_squares.mzn|13|5|15|9|ac;
↪latin_squares.mzn|13|13|13|13|j=1;latin_squares.
↪mzn|14|10|14|30|ca|alldifferent
latin_squares.mzn|18|5|20|9|ca|forall;latin_squares.mzn|18|5|20|9|ac;
↪latin_squares.mzn|18|13|18|13|i=1;latin_squares.mzn|19|10|19|37|ca|lex_less
latin_squares.mzn|13|5|15|9|ca|forall;latin_squares.mzn|13|5|15|9|ac;
↪latin_squares.mzn|13|13|13|13|j=2;latin_squares.
↪mzn|14|10|14|30|ca|alldifferent
latin_squares.mzn|22|5|24|9|ca|forall;latin_squares.mzn|22|5|24|9|ac;
↪latin_squares.mzn|22|13|22|13|j=1;latin_squares.
↪mzn|23|10|23|40|ca|lex_greater

MUS: 7 3 5 6 8 9
Brief: gecode_all_different_int;@{ADCols@AD(col 1)}:(j=1)
gecode_all_different_int;@{ADCols@AD(col 3)}:(j=3)
gecode_array_int_lt;@{LGCols@LG(cols 1 2)}:(j=1)
gecode_array_int_lt;@{LGCols@LG(cols 2 3)}:(j=2)
gecode_array_int_lt;@{LLRows@LL(rows 1 2)}:(i=1)
gecode_array_int_lt;@{LLRows@LL(rows 2 3)}:(i=2)

Traces:
latin_squares.mzn|18|5|20|9|ca|forall;latin_squares.mzn|18|5|20|9|ac;
↪latin_squares.mzn|18|13|18|13|i=2;latin_squares.mzn|19|10|19|37|ca|lex_less
latin_squares.mzn|13|5|15|9|ca|forall;latin_squares.mzn|13|5|15|9|ac;
↪latin_squares.mzn|13|13|13|13|j=1;latin_squares.
↪mzn|14|10|14|30|ca|alldifferent
latin_squares.mzn|13|5|15|9|ca|forall;latin_squares.mzn|13|5|15|9|ac;
↪latin_squares.mzn|13|13|13|13|j=3;latin_squares.
↪mzn|14|10|14|30|ca|alldifferent
latin_squares.mzn|18|5|20|9|ca|forall;latin_squares.mzn|18|5|20|9|ac;
↪latin_squares.mzn|18|13|18|13|i=1;latin_squares.mzn|19|10|19|37|ca|lex_less
latin_squares.mzn|22|5|24|9|ca|forall;latin_squares.mzn|22|5|24|9|ac;
↪latin_squares.mzn|22|13|22|13|j=1;latin_squares.
↪mzn|23|10|23|40|ca|lex_greater
latin_squares.mzn|22|5|24|9|ca|forall;latin_squares.mzn|22|5|24|9|ac;
↪latin_squares.mzn|22|13|22|13|j=2;latin_squares.
↪mzn|23|10|23|40|ca|lex_greater
Total Time: 0.39174     nmuses: 2       map:     40     sat:     21     ␣
↪total:     61
```

The first line, starting with `FznSubProblem:` provides some useful information for debugging the `findMUS` tool. Next we have the list of MUSes. Each MUS is described with three sections:

1. `MUS:` lists the indices of the FlatZinc constraints involved in this MUS.

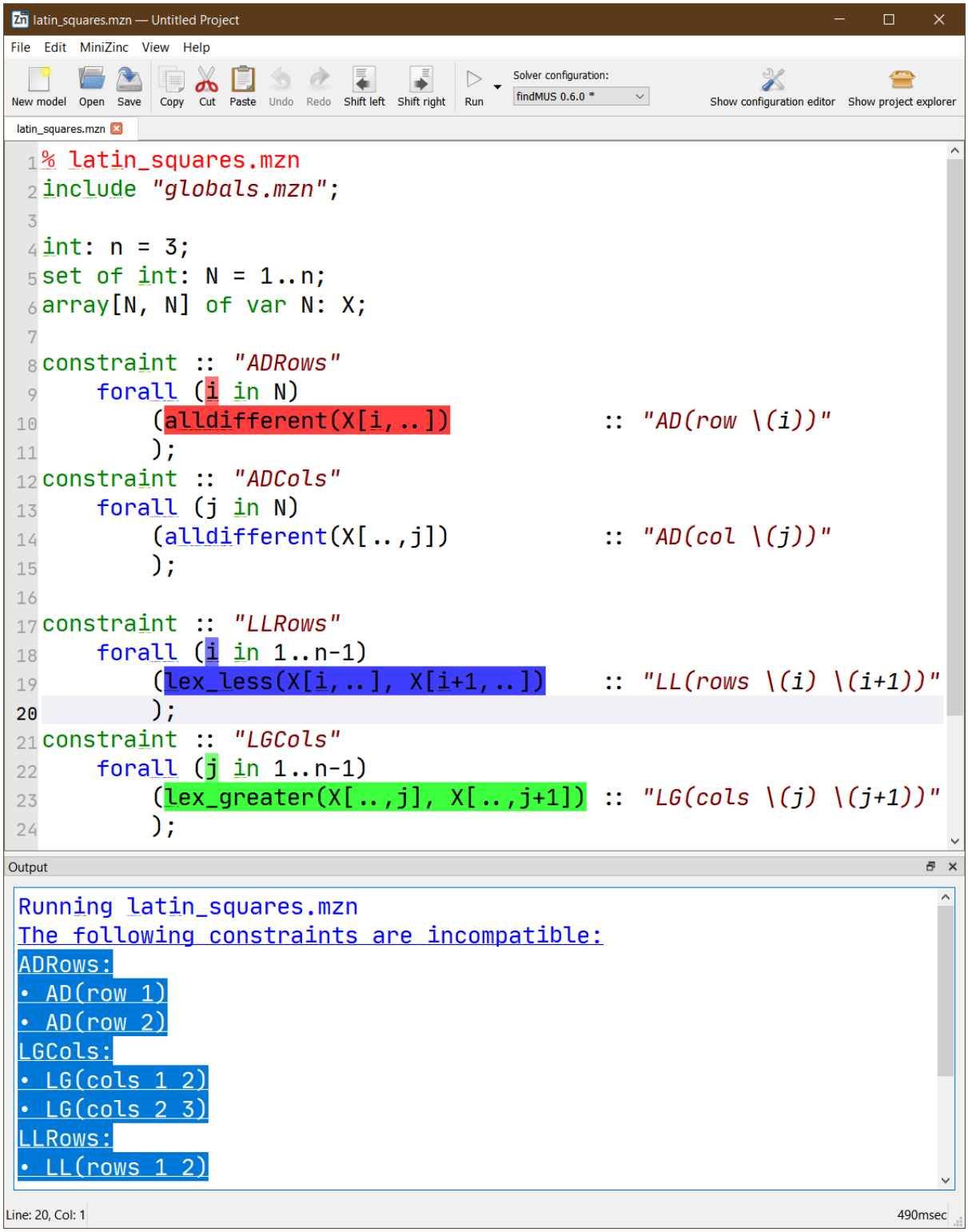
2. `Brief:` lists the FlatZinc constraint name, the expression name, the constraint name, and the loop index assignments for each involved FlatZinc constraint.
3. `Traces:` lists the MiniZinc paths corresponding to the constraints of the MUS. Each path typically contains a list of path elements separated by semi-colons `;`. Each element includes a file path, a start line, start column, end line and end column denoting a span of text from the listed file. And finally, a description of the element. In the examples above all paths point to calls to a forall on different lines of the model. (`ca|forall`)

The final line of output lists the `Total Time`, the number of MUSes found, and some statistics about the number of times the internal map solver `map` was called, and the number of times the subproblem solver was called `sat`.

Interpreting these two MUSes we see that the `lex` constraints are incompatible with the `alldifferent` constraints.

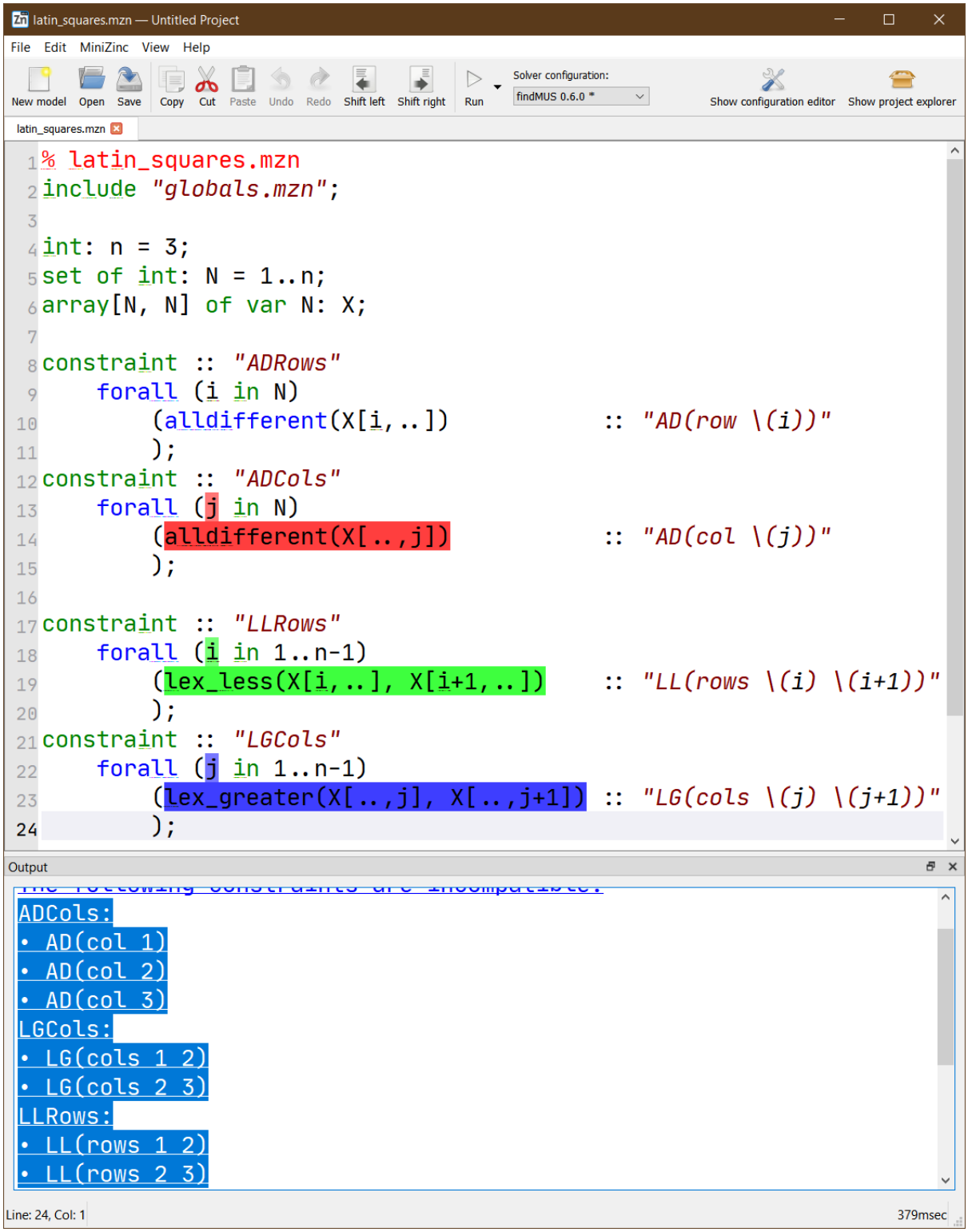
3.10.2 Using FindMUS in the MiniZinc IDE

To use FindMUS in the MiniZinc IDE, upon discovering that a model is unsatisfiable. Select `FindMUS` from the solver configuration dropdown menu and click the solve button (play symbol). This will look for MUSes using the default `--paramset mzn` argument.



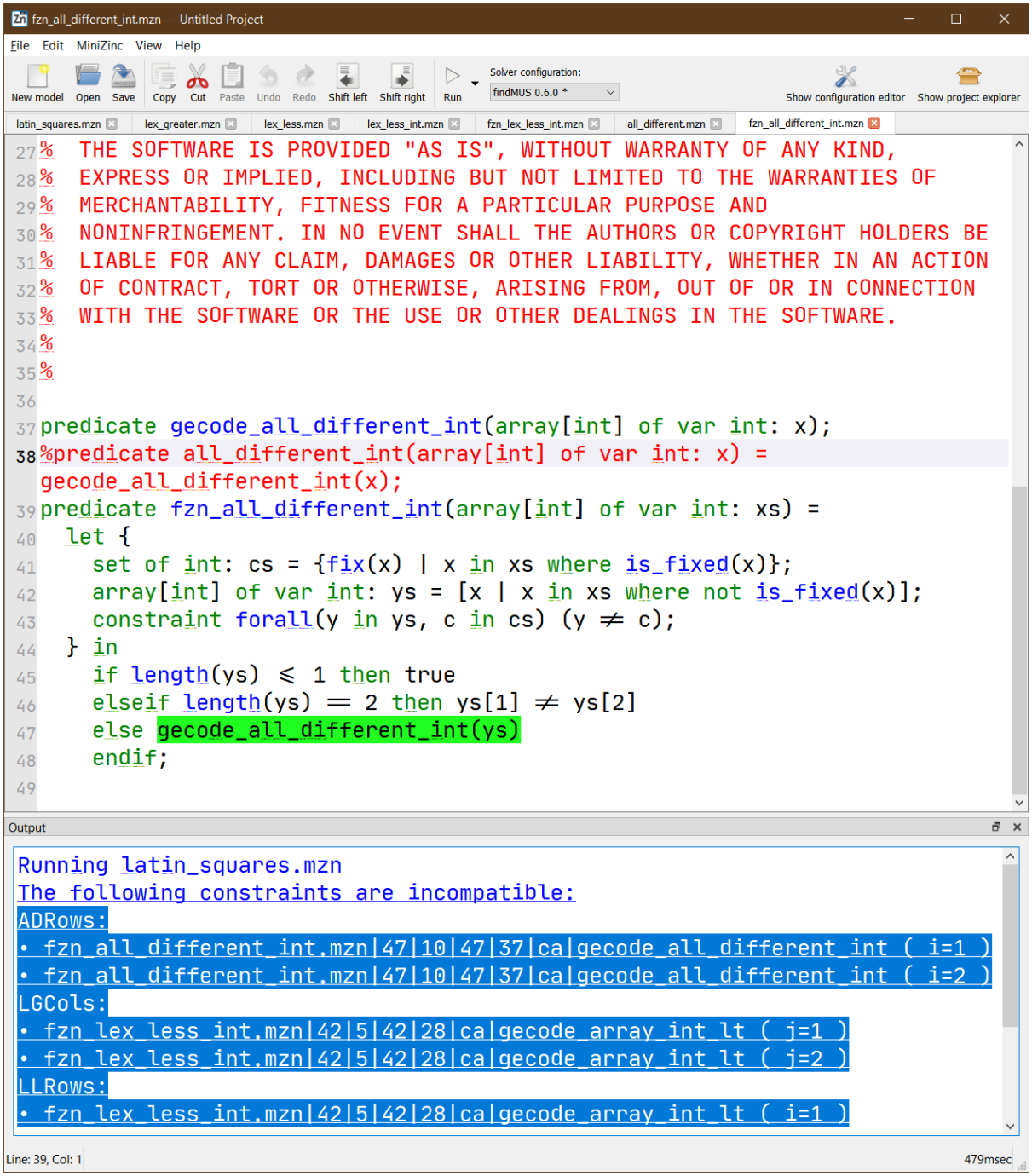
In this case we can see that the output pane shows the constraints involved in the MUS in terms of the expression annotations, grouped by the constraint annotations. Clicking on the MUS in the output pane will highlight the constraints in your model.

If finding MUSes with `--paramset mzn` is prohibitively slow, you can try running instead with `--paramset hint`. This merges many of the low level constraints into larger units and should be relatively fast and help locate the relevant constraint items. The following shows the result:



Selecting the returned MUS highlights three top level constraints as shown: `ADRows`, `LLRows` and `LGCols`.

Alternatively, in some cases you may wish to look at the MUSes of your model in terms of the flattened FlatZinc program. To display these MUSes run FindMUS with the `--paramset fzn` argument.



In this figure you can see that the MUSes are expressed in terms of FlatZinc constraints along with any assignments to relevant loop variables. Clicking on this MUS also opens and highlights the source of the constraints in the MiniZinc library (in this case in Gecode's custom redefinition of `fzn_all_different_int`.

3.10.3 How it works

A simple explanation of the algorithm is presented here. For a more detailed exploration of an earlier version of the approach see the Debugging Unsatisfiable Constraint Models paper[Page 243, 1].

The approach takes the full FlatZinc program and partitions the constraints into groups

based on the hierarchy provided in the user's model. To begin with (at depth '1') we search for MUSes in the set of top level constraint items using the MARCO[3] MUS enumeration algorithm. If we are not at the target depth we recursively select a found MUS, split its constituent constraints into lower level constraints based on the hierarchy and begin another search for MUSes underneath this high-level MUS. If any MUSes are found we know that the high-level MUS is not minimal and so it should not be reported. This process is repeated on any found unsatisfiable subsets until we reach the required depth at which point we will start to report MUSes. If in the recursive search we return to a high-level MUS without finding any sub-MUSes we can report this MUS as a real MUS. This recursive process is referred to as HierMUS. At each stage when we request the enumeration of a set of MUSes underneath a high-level MUS we can use one of several MUS enumeration algorithms.

3.10.4 Performance tips

If you are trying to find MUSes in a very large instance it is advised to make use of the filtering tools available. Use the default settings to find a very high-level MUS and then use the `--depth` option to find lower-level, more specific MUSes in conjunction with the `--filter-name` and `--filter-path` options to focus on finding specific sub-MUSes of a found high-level MUS.

When parameter tuning for FindMUS the oracle checker mode can speed up testing of different sets of parameters.

1. Run `findMUS -a --paramset fzn --output-brief ... > muses.log` to save the output log to a file.
2. Run subsequent sets of parameters with `findMUS -a --paramset fzn --output-brief --oracle-only muses.log ...`. The oracle mode will perform enumeration without making calls to the sub-solver, instead deciding SAT/UNSAT based on the MUS log (omit `--oracle-only` to call the sub-solver when the status of a subset is unknown)

3.10.5 Limitations / Future work

There are several features that we aim to include quite soon:

Solver core/MUS/IIS integration
: Set shrinking can be accelerated by using problem specific MUS extraction methods that some solvers provide.

Context-find
: Once a MUS has been found flip the foreground + background and narrow down the parts of the background that complete the MUS

Regular expression based filtering
: This will allow more complex filtering to be used.

Text span based filtering
: This will allow a user to simply click-and-drag a selection around the parts of a constraint model they wish to analyse.

[3] Liffiton, M. H. et al., "Fast, Flexible MUS Enumeration", 2016.

CHAPTER 3.11

Mzn-Analyse

`mzn-analyse` is an informal collection of experimental MiniZinc compiler "passes" that analyse and manipulate MiniZinc models. The passes pipeline structure allows relatively complex tasks to be performed on MiniZinc models by combining multiple passes together.

The tool provides the `get-diversity-anns` pass used by the `diverse_solutions()` solution diversity features of the MiniZinc python interface.

3.11.1 Basic Usage

```
mzn-analyse input.mzn [passes]
```

where `[passes]` is the list of passes and arguments for those passes (`cmd:arg1,arg2,...` with no spaces unless quoted)

An implicit `out:-` command (or `out_fzn:-` if reading FlatZinc) is appended at the end of the pipeline if no `out` commands occur in the sequence. An explicit final `out` must be added to the end if you wish to output models throughout the pipeline.

Typically, each pass takes a MiniZinc model, and outputs a MiniZinc model to the next pass. Optionally, a pass may store some information in a global json store which will be dumped to stdout by default. The contents of the json output can be inspected at any point by using the `json_out` pass. The json store can be cleared by using the `json_clear` pass.

3.11.1.1 Passes: read/write models & json

These passes control the reading and writing of MiniZinc models and the json store. In most cases, if no argument is passed to one of these passes, they will read/write to stdout.

`in:in.mzn` Read input file (no support for stdout)

`out:out.mzn` Write model to out.mzn (- for stdout)

`out_fzn:out.fzn` Write model to out.fzn (- for stdout)

`no_out` Disable automatic output insertion

`json_out:out.json` Write collected json output to out.json (- for stdout)

`json_clear` Clear collected json output

`no_json` Disable automatic json output

3.11.1.2 Passes: Manipulate Models

These passes are for making modifications to models or isolating/removing parts of models.

`inline-includes` Inline non-library includes (this allows a model to be relocated without needing to move models it includes)

`inline-all-includes` Inline all includes

`remove-anns:name1,[name2,...]` Remove Id and Call annotations matching names

`remove-includes:name1,[name2,...]` Remove includes matching names

`output-all` Add 'add_to_output' annotation to all VarDecls

`remove-stdlib` Remove stdlib includes

`get-items:idx1,[idx2,...]` Narrow to items indexed by idx1,...

`filter-items:iid1,[iid2,...]` Only keep items matching iids

`remove-items:iid1,[iid2,...]` Remove items matching iids

`filter-typeinst:{var|par}` Just show var/par parts of model

`replace-with-newvar:location1,location2` Replace expressions with 'let' expressions

`repeat-model:k` Produce `k` copies of the model (with unique variable names)

`get-solve-anns` Get information about the solving annotations used (useful if rewriting the original objective function)

3.11.1.3 Passes: Special Passes

These passes are needed to support other tools that are developed for MiniZinc. They are more specific to the projects where they are used.

`get-diversity-anns` Extract diversity information from the model

`annotate-data-deps` Annotate expressions with their data dependencies

`get-term-types:out.terms` Write .terms file with types of objective terms

`get-data-deps:out.cons` (FlatZinc only) Write .cons file with data dependencies of FlatZinc constraints

`get-exprs:location1,location2` Extract list of expressions occurring inside location location = path.mzn|sl|sc|el|ec

`get-ast:location1,location2` Build JSON representation of AST for whole model or just for expression matching location1 or location2, place in json_store

3.11.2 Examples

The following are a few examples of the usage of the `mzn-analyse` tool.

1. Remove the solve and output items from the model and write the model to solveless.mzn. It then inlines the local includes and outputs to stdout as "FlatZinc" (no line breaks while printing an item).

```
mzn-analyse in.mzn remove-items:solve,output out:solveless.mzn
inline-includes out_fzn
```

2. Remove all items except constraint items, picks out the 50th constraint, remove any annotations, then output to stdout. The implicit output will default to *out_fzn* since the input was fzn.

```
mzn-analyse in.fzn filter-items:constraint get-items:50 remove-anns
```

3. The following requests the data-deps information for constraints 60, 61, and 62 from a FlatZinc file. The *no_out* command disables the automatic insertion of *out_fzn*.

```
mzn-analyse rcpsp-wet-r0.annotated.fzn filter-items:constraint
get-items:60,61,62 get-data-deps no_out
```

Gives the following output:


```
{"constraint_info": [
  [
    ["in", "i", "Tasks"],
    ["in", "j", "suc[i]"],
    ["assign", "j", "24"],
    ["assign", "i", "2"],
    ["eq", "suc[i]", "23..24"],
    ["eq", "Tasks", "1..32"]],
  [
    ["in", "i", "Tasks"],
    ["in", "j", "suc[i]"],
    ["assign", "j", "5"],
    ["assign", "i", "3"],
    ["eq", "suc[i]", "{5,6,17}"],
    ["eq", "Tasks", "1..32"]],
  [
    ["in", "i", "Tasks"],
    ["in", "j", "suc[i]"],
    ["assign", "j", "6"],
    ["assign", "i", "3"],
    ["eq", "suc[i]", "{5,6,17}"],
    ["eq", "Tasks", "1..32"]]]}
```

3.11.3 Limitations / Future work

`mzn-analyse` is an experimental tool. The passes were implemented for very specific use cases and may not behave as expected in more general cases. The code in the repo is likely to make strong assumptions about the underlying models that will not always hold. The passes should be thought of more as proof-of-concept/AST manipulation sample code to be used as a starting point for implementing transformations.

CHAPTER 3.12

Using MiniZinc in Jupyter Notebooks

You can use MiniZinc inside a Jupyter / IPython notebook using the `iminizinc` Python module. The module provides a "cell magic" extension that lets you solve MiniZinc models.

The module requires an existing installation of MiniZinc.

3.12.1 Installation

You can install or upgrade this module via `pip`:

```
pip install -U iminizinc
```

Consult your Python documentation to find out if you need any extra options (e.g. you may want to use the `--user` flag to install only for the current user, or you may want to use virtual environments).

Make sure that the `minizinc` binary are on the `PATH` environment variable when you start the notebook server. The easiest way to do that is to get the "bundled installation" that includes the MiniZinc IDE and a few solvers, available from GitHub here: https://github.com/MiniZinc/MiniZincIDE/releases/latest You then need to change your `PATH` environment variable to include the MiniZinc installation.

3.12.2 Basic usage

After installing the module, you have to load the extension using `%load_ext iminizinc`. This will enable the cell magic `%%minizinc`, which lets you solve MiniZinc models. Here is a simple example:

```
In[1]:  %load_ext iminizinc

In[2]:  n=8

In[3]:  %%minizinc

        include "globals.mzn";
        int: n;
        array[1..n] of var 1..n: queens;
        constraint all_different(queens);
        constraint all_different([queens[i]+i | i in 1..n]);
        constraint all_different([queens[i]-i | i in 1..n]);
        solve satisfy;
Out[3]: {u'queens': [4, 2, 7, 3, 6, 8, 5, 1]}
```

As you can see, the model binds variables in the environment (in this case, `n`) to MiniZinc parameters, and returns an object with fields for all declared decision variables.

Alternatively, you can bind the decision variables to Python variables:

```
In[1]:  %load_ext iminizinc

In[2]:  n=8

In[3]:  %%minizinc -m bind

        include "globals.mzn";
        int: n;
        array[1..n] of var 1..n: queens;
        constraint all_different(queens);
        constraint all_different([queens[i]+i | i in 1..n]);
        constraint all_different([queens[i]-i | i in 1..n]);
        solve satisfy;

In[4]:  queens

Out[4]: [4, 2, 7, 3, 6, 8, 5, 1]
```

If you want to find all solutions of a satisfaction problem, or all intermediate solutions of an optimisation problem, you can use the `-a` flag:

```
In[1]:  %load_ext iminizinc

In[2]:  n=6

In[3]:  %%minizinc -a

        include "globals.mzn";
        int: n;
```

```
        array[1..n] of var 1..n: queens;
        constraint all_different(queens);
        constraint all_different([queens[i]+i | i in 1..n]);
        constraint all_different([queens[i]-i | i in 1..n]);
        solve satisfy;

Out[3]: [{u'queens': [5, 3, 1, 6, 4, 2]},
         {u'queens': [4, 1, 5, 2, 6, 3]},
         {u'queens': [3, 6, 2, 5, 1, 4]},
         {u'queens': [2, 4, 6, 1, 3, 5]}]
```

The magic supports a number of additional options, in particular loading MiniZinc models and data from files. Some of these may only work with the development version of MiniZinc (i.e., not the one that comes with the bundled binary releases). You can take a look at the help using

```
In[1]:  %%minizinc?
```

CHAPTER 3.13

Python Interface

MiniZinc can be directly used from Python using the MiniZinc Python package. This package provides an native python interface that interacts directly with the MiniZinc bundle installed on the machine. For installation instructions and more information about the package that provides the MiniZinc Python interface please visit https://python.minizinc.dev/.

Warning:

The development of MiniZinc Python is still in the very early stages. We appreciate any feedback for the package, but we would recommend refraining from using the package in production software. The package can drastically change and backwards compatible changes are not guaranteed.

CHAPTER 3.14

JavaScript Interface

MiniZinc can be used from JavaScript either in the browser using the WebAssembly build of MiniZinc, or in NodeJS using the MiniZinc bundle installed on the machine. For installation instructions and more information about the package providing the JavaScript interface, please visit https://js.minizinc.dev/.

CHAPTER 3.15

Installation from Source Code

The easiest way to install a complete version of MiniZinc is to use the binary packages we provide for several platforms, as explained in *Installation* (page 7). In the following, we describe how to install MiniZinc from source code.

All components of MiniZinc are free and open source software, and compilation should be straightforward if you have all the necessary build tools installed. However, third-party components, in particular the different solvers, may be more difficult to install correctly, and we cannot provide any support for these components.

The source code for MiniZinc can be downloaded from its GitHub repository at https://github.com/MiniZinc/libminizinc. The source code for the MiniZinc IDE is available from https://github.com/MiniZinc/MiniZincIDE. The `master` branch of each repository points to the latest release version, while the `develop` branch points to the current state of development (and may be buggy or not even compile sometimes). Installation details can be found in each project's `README.md` files.

You will also need to install additional solvers to use with MiniZinc. To get started, try Gecode (http://www.gecode.org) or Chuffed (https://github.com/chuffed/chuffed). We don't cover installation instructions for these solvers here.

Part 4

Reference Manual

CHAPTER 4.1

Specification of MiniZinc

4.1.1 Introduction

This document defines MiniZinc, a language for modelling constraint satisfaction and optimisation problems.

MiniZinc is a high-level, typed, mostly first-order, functional, modelling language. It provides:

- mathematical notation-like syntax (automatic coercions, overloading, iteration, sets, arrays);
- expressive constraints (finite domain, set, linear arithmetic, integer);
- support for different kinds of problems (satisfaction, explicit optimisation);
- separation of data from model;
- high-level data structures and data encapsulation (sets, arrays, enumerated types, constrained type-insts);
- extensibility (user-defined functions and predicates);
- reliability (type checking, instantiation checking, assertions);
- solver-independent modelling;
- simple, declarative semantics.

MiniZinc is similar to OPL and moves closer to CLP languages such as ECLiPSe.

This document has the following structure. *Notation* (page 272) introduces the syntax notation used throughout the specification. *Overview of a Model* (page 273) provides a high-level overview of MiniZinc models. *Syntax Overview* (page 275) covers syntax basics. *High-level Model Structure* (page 276) covers high-level structure: items, multi-file models, namespaces, and scopes. *Types and Type-insts* (page 278) introduces types and type-insts. *Expressions* (page 291) covers expressions. *Items* (page 310) describes the top-level items in detail. *Annotations* (page 320) describes annotations. *Partiality* (page 321)

describes how partiality is handled in various cases. *Built-in Operations* (page 324) describes the language built-ins. *Full grammar* (page 335) gives the MiniZinc grammar. *Content-types* (page 333) defines content-types used in this specification.

This document also provides some explanation of why certain design decisions were made. Such explanations are marked by the word *Rationale* and written in italics, and do not constitute part of the specification as such. *Rationale: These explanations are present because they are useful to both the designers and the users of MiniZinc.*

4.1.1.1 Original authors.

The original version of this document was prepared by Nicholas Nethercote, Kim Marriott, Reza Rafeh, Mark Wallace and Maria Garcia de la Banda. MiniZinc is evolving, however, and so is this document.

For a formally published paper on the MiniZinc language and the superset Zinc language, please see:

- N. Nethercote, P.J. Stuckey, R. Becket, S. Brand, G.J. Duck, and G. Tack. Minizinc: Towards a standard CP modelling language. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *LNCS*, pages 529–543. Springer-Verlag, 2007.
- K. Marriott, N. Nethercote, R. Rafeh, P.J. Stuckey, M. Garcia de la Banda, and M. Wallace. The Design of the Zinc Modelling Language. *Constraints*, 13(3):229-267, 2008.

4.1.2 Notation

The basics of the EBNF used in this specification are as follows.

- Non-terminals are written between angle brackets, `<item>`.
- Terminals are written in double quotes, e.g. `"constraint"`. A double quote terminal is written as a sequence of three double quotes: `"""`.
- Optional items are written in square brackets, e.g. `[ "var" ]`.
- Sequences of zero or more items are written with parentheses and a star, e.g. `( "," <ident> )*`.
- Sequences of one or more items are written with parentheses and a plus, e.g. `( <msg> )+`.
- Non-empty lists are written with an item, a separator/terminator terminal, and three dots. For example, this:

  ```
  <expr> "," ...
  ```

  is short for this:

  ```
  <expr> ( "," <expr> )* [ "," ]
  ```

  The final terminal is always optional in non-empty lists.

- Regular expressions are used in some productions, e.g. `[-+]?[0-9]+`.

MiniZinc's grammar is presented piece-by-piece throughout this document. It is also available as a whole in *Full grammar* (page 335). The output grammar also includes some details of the use of whitespace. The following conventions are used:

- A newline character or CRLF sequence is written `\n`.

4.1.3 Overview of a Model

Conceptually, a MiniZinc problem specification has two parts.

1. The *model*: the main part of the problem specification, which describes the structure of a particular class of problems.
2. The *data*: the input data for the model, which specifies one particular problem within this class of problems.

The pairing of a model with a particular data set is a *model instance* (sometimes abbreviated to *instance*).

The model and data may be separated, or the data may be "hard-wired" into the model. *Model Instance Files* (page 277) specifies how the model and data can be structured within files in a model instance.

There are two broad classes of problems: satisfaction and optimisation. In satisfaction problems all solutions are considered equally good, whereas in optimisation problems the solutions are ordered according to an objective and the aim is to find a solution whose objective is optimal. *Solve Items* (page 314) specifies how the class of problem is chosen.

4.1.3.1 Evaluation Phases

A MiniZinc model instance is evaluated in two distinct phases.

1. Instance-time: static checking of the model instance.
2. Run-time: evaluation of the instance (i.e., constraint solving).

The model instance may not compile due to a problem with the model and/or data, detected at instance-time. This could be caused by a syntax error, a type-inst error, the use of an unsupported feature or operation, etc. In this case the outcome of evaluation is a static error; this must be reported prior to run-time. The form of output for static errors is implementation-dependent, although such output should be easily recognisable as a static error.

An implementation may produce warnings during all evaluation phases. For example, an implementation may be able to determine that unsatisfiable constraints exist prior to run-time, and the resulting warning given to the user may be more helpful than if the unsatisfiability is detected at run-time.

An implementation must produce a warning if the objective for an optimisation problem is unbounded.

4.1.3.2 Run-time Outcomes

Assuming there are no static errors, the output from the run-time phase has the following abstract form:

```
<output> ::= <no-solutions> [ <warnings> ] <free-text>
           | ( <solution> )* [ <complete> ] [ <warnings> ] <free-text>
```

If a solution occurs in the output then it must be feasible. For optimisation problems, each solution must be strictly better than any preceding solution.

If there are no solutions in the output, the outcome must indicate that there are no solutions.

If the search is complete the output may state this after the solutions. The absence of the completeness message indicates that the search is incomplete.

Any warnings produced during run-time must be summarised after the statement of completeness. In particular, if there were any warnings at all during run-time then the summary must indicate this fact.

The implementation may produce text in any format after the warnings. For example, it may print a summary of benchmarking statistics or resources used.

4.1.3.3 Output

Implementations must be capable of producing output of content type `application/x-zinc-output`, which is described below and also in *Content-types* (page 333). Implementations may also produce output in alternative formats. Any output should conform to the abstract format from the previous section and must have the semantics described there.

Content type `application/x-zinc-output` extends the syntax from the previous section as follows:

```
<solution> ::= <solution-text> [ \n ] "----------" \n
```

The solution text for each solution must be as described in *Output Items* (page 315). A newline must be appended if the solution text does not end with a newline. *Rationale: This allows solutions to be extracted from output without necessarily knowing how the solutions are formatted.* Solutions end with a sequence of ten dashes followed by a newline.

```
<no-solutions> ::= "=====UNSATISFIABLE=====" \n
```

The completeness result is printed on a separate line. *Rationale: The strings are designed to clearly indicate the end of the solutions.*

```
<complete> ::= "==========" \n
```

If the search is complete, a statement corresponding to the outcome is printed. For an outcome of no solutions the statement is that the model instance is unsatisfiable, for

an outcome of no more solutions the statement is that the solution set is complete, and for an outcome of no better solutions the statement is that the last solution is optimal. *Rationale: These are the logical implications of a search being complete.*

```
<warnings> ::= ( <message> )+

<message>  ::= ( <line> )+
```

If the search is incomplete, one or more messages describing reasons for incompleteness may be printed. Likewise, if any warnings occurred during search they are repeated after the completeness message. Both kinds of message should have lines that start with `%` so they are recognized as comments by post-processing. *Rationale: This allows individual messages to be easily recognised.*

For example, the following may be output for an optimisation problem:

```
=====UNSATISFIABLE=====
% trentin.fzn:4: warning: model inconsistency detected before search.
```

Note that, as in this case, an unbounded objective is not regarded as a source of incompleteness.

4.1.4 Syntax Overview

4.1.4.1 Character Set

The input files to MiniZinc must be encoded as UTF-8.

MiniZinc is case sensitive. There are no places where upper-case or lower-case letters must be used.

MiniZinc has no layout restrictions, i.e., any single piece of whitespace (containing spaces, tabs and newlines) is equivalent to any other.

4.1.4.2 Comments

A `%` indicates that the rest of the line is a comment. MiniZinc also has block comments, using symbols `/*` and `*/` to mark the beginning and end of a comment.

4.1.4.3 Identifiers

Identifiers have the following syntax:

```
<ident> ::= _?[A-Za-z][A-Za-z0-9_]*       % excluding keywords
          | "'" [^'\xa\xd\x0]+ "'"
```

```
my_name_2
MyName2
'An arbitrary identifier'
```

A number of keywords are reserved and cannot be used as identifiers. The keywords are: `ann`, `annotation`, `any`, `array`, `bool`, `case`, `constraint`, `default`, `diff`, `div`, `else`, `elseif`, `endif`, `enum`, `false`, `float`, `function`, `if`, `in`, `include`, `infinity`, `int`, `intersect`, `let`, `list`, `maximize`, `minimize`, `mod`, `not`, `of`, `opt`, `output`, `par`, `predicate`, `record`, `satisfy`, `set`, `solve`, `string`, `subset`, `superset`, `symdiff`, `test`, `then`, `true`, `tuple`, `type`, `union`, `var`, `variant_record`, `where`, `xor`.

A number of identifiers are used for built-ins; see *Built-in Operations* (page 324) for details.

4.1.5 High-level Model Structure

A MiniZinc model consists of multiple *items*:

```
<model> ::= [ <item> ";" ... ]
```

Items can occur in any order; identifiers need not be declared before they are used. Items have the following top-level syntax:

```
<item>  ::= <include-item>
          | <var-decl-item>
          | <enum-item>
          | <type-inst-syn-item>
          | <assign-item>
          | <constraint-item>
          | <solve-item>
          | <output-item>
          | <predicate-item>
          | <test-item>
          | <function-item>
          | <annotation-item>

<ti-expr-and-id> ::= <ti-expr> ":" <ident>
```

Include items provide a way of combining multiple files into a single instance. This allows a model to be split into multiple files (*Include Items* (page 310)).

Variable declaration items introduce new global variables and possibly bind them to a value (*Variable Declaration Items* (page 310)).

Assignment items bind values to global variables (*Assignment Items* (page 314)).

Constraint items describe model constraints (*Constraint Items* (page 314)).

Solve items are the “starting point” of a model, and specify exactly what kind of solution is being looked for: plain satisfaction, or the minimization/maximization of an expression. Each model must have exactly one solve item (*Solve Items* (page 314)).

Output items are used for nicely presenting the result of a model execution (*Output Items* (page 315)).

Predicate items, test items (which are just a special type of predicate) and function items introduce new user-defined predicates and functions which can be called in expressions

(*User-defined Operations* (page 316)). Predicates, functions, and built-in operators are described collectively as *operations*.

Annotation items augment the ann type, values of which can specify non-declarative and/or solver-specific information in a model.

4.1.5.1 Model Instance Files

MiniZinc models can be constructed from multiple files using include items (see *Include Items* (page 310)). MiniZinc has no module system as such; all the included files are simply concatenated and processed as a whole, exactly as if they had all been part of a single file. *Rationale: We have not found much need for one so far. If bigger models become common and the single global namespace becomes a problem, this should be reconsidered.*

Each model may be paired with one or more data files. Data files are more restricted than model files. They may only contain variable assignments (see *Assignment Items* (page 314)).

Data files may not include calls to user-defined operations.

Models do not contain the names of data files; doing so would fix the data file used by the model and defeat the purpose of allowing separate data files. Instead, an implementation must allow one or more data files to be combined with a model for evaluation via a mechanism such as the command-line.

When checking a model with data, all global variables with fixed type-insts must be assigned, unless they are not used (in which case they can be removed from the model without effect).

A data file can only be checked for static errors in conjunction with a model, since the model contains the declarations that include the types of the variables assigned in the data file.

A single data file may be shared between multiple models, so long as the definitions are compatible with all the models.

4.1.5.2 Namespaces

All names declared at the top-level belong to a single namespace. It includes the following names.

1. All global variable names.
2. All function and predicate names, both built-in and user-defined.
3. All enumerated type names and enum case names.
4. All annotation names.

Because multi-file MiniZinc models are composed via concatenation (*Model Instance Files* (page 277)), all files share this top-level namespace. Therefore a variable x declared in one model file could not be declared with a different type in a different file, for example.

MiniZinc supports overloading of built-in and user-defined operations.

4.1.5.3 Scopes

Within the top-level namespace, there are several kinds of local scope that introduce local names:

- Comprehension expressions (*Set Comprehensions* (page 298)).
- Let expressions (*Let Expressions* (page 306)).
- Function and predicate argument lists and bodies (*User-defined Operations* (page 316)).

The listed sections specify these scopes in more detail. In each case, any names declared in the local scope overshadow identical global names.

4.1.6 Types and Type-insts

MiniZinc provides four scalar built-in types: Booleans, integers, floats, and strings; enumerated types; two compound built-in types: sets and multi-dimensional arrays; and the user extensible annotation type ann.

Each type has one or more possible *instantiations*. The instantiation of a variable or value indicates if it is fixed to a known value or not. A pairing of a type and instantiation is called a *type-inst*.

We begin by discussing some properties that apply to every type. We then introduce instantiations in more detail. We then cover each type individually, giving: an overview of the type and its possible instantiations, the syntax for its type-insts, whether it is a finite type (and if so, its domain), whether it is varifiable, the ordering and equality operations, whether its variables must be initialised at instance-time, and whether it can be involved in automatic coercions.

4.1.6.1 Properties of Types

The following list introduces some general properties of MiniZinc types.

- Currently all types are monotypes. In the future we may allow types which are polymorphic in other types and also the associated constraints.
- We distinguish types which are *finite types*. In MiniZinc, finite types include Booleans, enums, types defined via set expression type-insts such as range types (see *Set Expression Type-insts* (page 290)), as well as sets and arrays, composed of finite types. Types that are not finite types are unconstrained integers, unconstrained floats, unconstrained strings, and ann. Finite types are relevant to sets (*Sets* (page 282)) and array indices (*Arrays* (page 284)). Every finite type has a *domain*, which is a set value that holds all the possible values represented by the type.
- Every first-order type (this excludes ann) has a built-in total order and a built-in equality; >, <, ==/=, !=, <= and >= comparison operators can be applied to any pair of values of the same type. *Rationale: This facilitates the specification of symmetry breaking and of polymorphic predicates and functions.* Note that, as in most languages, using equality on floats or types that contain floats is generally not reliable due to their inexact representation. An implementation may choose to warn about the use of equality with floats or types that contain floats.

4.1.6.2 Instantiations

When a MiniZinc model is evaluated, the value of each variable may initially be unknown. As it runs, each variable's *domain* (the set of values it may take) may be reduced, possibly to a single value.

An *instantiation* (sometimes abbreviated to *inst*) describes how fixed or unfixed a variable is at instance-time. At the most basic level, the instantiation system distinguishes between two kinds of variables:

1. *Parameters*, whose values are fixed at instance-time (usually written just as "fixed").
2. *Decision variables* (often abbreviated to *variables*), whose values may be completely unfixed at instance-time, but may become fixed at run-time (indeed, the fixing of decision variables is the whole aim of constraint solving).

In MiniZinc decision variables can have the following types: Booleans, integers, floats, and sets of integers, and enums. Arrays and `ann` can contain decision variables.

4.1.6.3 Type-insts

Because each variable has both a type and an inst, they are often combined into a single *type-inst*. Type-insts are primarily what we deal with when writing models, rather than types.

A variable's type-inst *never changes*. This means a decision variable whose value becomes fixed during model evaluation still has its original type-inst (e.g. `var int`), because that was its type-inst at instance-time.

Some type-insts can be automatically coerced to another type-inst. For example, if a `par int` value is used in a context where a `var int` is expected, it is automatically coerced to a `var int`. We write this `par int` $\xrightarrow{c}$ `var int`. Also, any type-inst can be considered coercible to itself. MiniZinc allows coercions between some types as well.

Some type-insts can be *varified*, i.e., made unfixed at the top-level. For example, `par int` is varified to `var int`. We write this `par int` $\xrightarrow{v}$ `var int`.

Type-insts that are varifiable include the type-insts of the types that can be decision variables (Booleans, integers, floats, sets, enumerated types). Varification is relevant to type-inst synonyms and array accesses.

4.1.6.4 Type-inst expression overview

This section partly describes how to write type-insts in MiniZinc models. Further details are given for each type as they are described in the following sections.

A type-inst expression specifies a type-inst. Type-inst expressions may include type-inst constraints. Type-inst expressions appear in variable declarations (*Variable Declaration Items* (page 310)) and user-defined operation items (*User-defined Operations* (page 316)).

Type-inst expressions have this syntax:

```
<ti-expr> ::= <base-ti-expr>
            | <array-ti-expr>
            | <ti-expr> "++" <base-ti-expr>
```

```
<base-ti-expr> ::= <var-par> <opt-ti> <set-ti> <base-ti-expr-tail>
                 | "any" <ti-variable-expr-tail>

<var-par> ::= "var" | "par" |

<opt-ti> ::= "opt" |

<set-ti> ::= "set" [ "(" <expr> ")" ] "of" |

<base-type> ::= "bool"
              | "int"
              | "float"
              | "string"

<base-ti-expr-tail> ::= <ident>
                      | <base-type>
                      | <ti-variable-expr-tail>
                      | <tuple-ti-expr-tail>
                      | <record-ti-expr-tail>
                      | "ann"
                      | <expr>
```

(The final alternative, for range types, uses the numeric-specific `<num-expr>` non-terminal, defined in *Expressions Overview* (page 291), rather than the `<expr>` non-terminal. If this were not the case, the rule would never match because the `..` operator would always be matched by the first `<expr>`.)

This fully covers the type-inst expressions for scalar types. The compound type-inst expression syntax is covered in more detail in *Built-in Compound Types and Type-insts* (page 282).

The `par` and `var` keywords (or lack of them) determine the instantiation. The `par` annotation can be omitted; the following two type-inst expressions are equivalent:

```
par int
int
```

Rationale: The use of the explicit `var` *keyword allows an implementation to check that all parameters are initialised in the model or the instance. It also clearly documents which variables are parameters, and allows more precise type-inst checking.*

A type-inst is fixed if it does not contain `var`, with the exception of `ann`.

Note that several type-inst expressions that are syntactically expressible represent illegal type-insts. For example, although the grammar allows `var` in front of all these base type-inst expression tails, it is a type-inst error to have `var` in the front of a string or array expression.

4.1.6.5 Built-in Scalar Types and Type-insts

Booleans

Overview. Booleans represent truthhood or falsity. *Rationale: Boolean values are not represented by integers. Booleans can be explicit converted to integers with the* `bool2int` *function, which makes the user's intent clear.*

Allowed Insts. Booleans can be fixed or unfixed.

Syntax. Fixed Booleans are written `bool` or `par bool`. Unfixed Booleans are written as `var bool`.

Finite? Yes. The domain of a Boolean is `false, true`.

Varifiable? `par bool` $\xrightarrow{v}$ `var bool`, `var bool` $\xrightarrow{v}$ `var bool`.

Ordering. The value `false` is considered smaller than `true`.

Initialisation. A fixed Boolean variable must be initialised at instance-time; an unfixed Boolean variable need not be.

Coercions. `par bool` $\xrightarrow{c}$ `var bool`.

Also Booleans can be automatically coerced to integers; see *Integers* (page 281).

Integers

Overview. Integers represent integral numbers. Integer representations are implementation-defined. This means that the representable range of integers is implementation-defined. However, an implementation should abort at run-time if an integer operation overflows.

Allowed Insts. Integers can be fixed or unfixed.

Syntax. Fixed integers are written `int` or `par int`. Unfixed integers are written as `var int`.

Finite? Not unless constrained by a set expression (see *Set Expression Type-insts* (page 290)).

Varifiable? `par int` $\xrightarrow{v}$ `var int`, `var int` $\xrightarrow{v}$ `var int`.

Ordering. The ordering on integers is the standard one.

Initialisation. A fixed integer variable must be initialised at instance-time; an unfixed integer variable need not be.

Coercions. `par int` $\xrightarrow{c}$ `var int`, `par bool` $\xrightarrow{c}$ `par int`, `par bool` $\xrightarrow{c}$ `var int`, `var bool` $\xrightarrow{c}$ `var int`.

Also, integers can be automatically coerced to floats; see *Floats* (page 281).

Floats

Overview. Floats represent real numbers. Float representations are implementation-defined. This means that the representable range and precision of floats is implementation-defined. However, an implementation should abort at run-time on exceptional float operations (e.g., those that produce `NaN`, if using IEEE754 floats).

Allowed Insts. Floats can be fixed or unfixed.

Syntax. Fixed floats are written `float` or `par float`. Unfixed floats are written as `var float`.

Finite? Not unless constrained by a set expression (see *Set Expression Type-insts* (page 290)).

Varifiable? `par float` $\xrightarrow{v}$ `var float`, `var float` $\xrightarrow{v}$ `var float`.

Ordering. The ordering on floats is the standard one.

Initialisation. A fixed float variable must be initialised at instance-time; an unfixed float variable need not be.

Coercions. `par int` $\xrightarrow{c}$ `par float`, `par int` $\xrightarrow{c}$ `var float`, `var int` $\xrightarrow{c}$ `var float`, `par float` $\xrightarrow{c}$ `var float`.

Strings

Overview. Strings are primitive, i.e., they are not lists of characters.

String expressions are used in assertions, output items and annotations, and string literals are used in include items.

Allowed Insts. Strings must be fixed.

Syntax. Fixed strings are written `string` or `par string`.

Finite? Not unless constrained by a set expression (see *Set Expression Type-insts* (page 290)).

Varifiable? No.

Ordering. Strings are ordered lexicographically using the underlying character codes.

Initialisation. A string variable (which can only be fixed) must be initialised at instance-time.

Coercions. None automatic. However, any non-string value can be manually converted to a string using the built-in `show` function or using string interpolation (see *String Literals and String Interpolation* (page 297)).

4.1.6.6 Built-in Compound Types and Type-insts

Sets

Overview. A set is a collection with no duplicates.

Allowed Insts. The type-inst of a set's elements must be fixed. *Rationale: This is because current solvers are not powerful enough to handle sets containing decision variables.* Sets may contain any type, and may be fixed or unfixed. If a set is unfixed, its elements must be finite, unless it occurs in one of the following contexts:

- the argument of a predicate, function or annotation.
- the declaration of a variable or let local variable with an assigned value.

Syntax. A set base type-inst expression is a special case of the base type-inst rule:

```
<base-ti-expr> ::= <var-par> <opt-ti> "set" [ "(" <expr> ")" ] "of"
↪<base-ti-expr-tail>
```

Some example set type-inst expressions:

```
set of int
var set of bool
var set(2) of 1..5
var set(1..3) of 1..10
```

The optional (`<expr>`) is a *set cardinality declaration*: it constrains the cardinality of the declared set without requiring a separate `card` constraint. The cardinality expression `e` must have type `int`, `var int`, `set of int` or `var set of int` (also their enum-typed equivalents); any other type is a type error.

The declaration `var set(e) of T: s;` is equivalent to the plain declaration `var set of T: s;` together with an additional implicit constraint that depends on the type of `e`:

- if `e` has type `int` or `var int`, the implicit constraint is `card(s) = e`;
- if `e` has type `set of int` or `var set of int`, the implicit constraint is `card(s) in e`.

For example, `var set(2) of 1..5: a;` declares a variable set whose cardinality is fixed to two, while `var set(1..3) of 1..10: b;` declares a variable set whose cardinality is anywhere from one to three.

Set cardinality declarations are allowed in variable declaration items at the top level of a model and in `let` expressions, and may also appear on the set leaves of composite type-insts — tuples, records, and arrays (including arrays of arrays) of sets. In that case the cardinality constraint applies element-wise to every leaf set of the composite value. The syntax is *not* allowed in function or predicate parameter type-insts (neither directly nor nested inside composite parameter types). *Rationale: this restriction is reflected syntactically in the parameter grammar; implementations may choose to accept the looser form at parse time in order to report a clearer error during type-checking.*

Finite? Yes, if the set elements are finite types. Otherwise, no.

The domain of a set type that is a finite type is the power set of the domain of its element type. For example, the domain of `set of 1..2` is `powerset(1..2)`, which is `{}, {1}, {1,2}, {2}`.

Varifiable? `par set of TI` $\xrightarrow{v}$ `var set of TI`, `var set of TI` $\xrightarrow{v}$ `var set of TI`.

Ordering. The pre-defined ordering on sets is a lexicographic ordering of the *sorted set form*, where `{1,2}` is in sorted set form, for example, but `{2,1}` is not. This means, for instance, `{} < {1,3} < {2}`.

Initialisation. A fixed set variable must be initialised at instance-time; an unfixed set variable need not be.

Coercions. `par set of TI` $\xrightarrow{c}$ `par set of UI` and `par set of TI` $\xrightarrow{c}$ `var set of UI` and `var set of TI` $\xrightarrow{c}$ `var set of UI`, if `TI` $\xrightarrow{c}$ `UI`.

Arrays

Overview. MiniZinc arrays are maps from fixed integers to values. Values can be of any first-order type, including tuples, records and other arrays (so arrays of arrays are permitted). Arrays can be multi-dimensional.

MiniZinc arrays can be declared in two different ways.

- *Explicitly-indexed* arrays have index types in the declaration that are finite types. For example:

  ```
  array[1..3] of int: a1;
  array[0..3] of int: a2;
  array[1..5, 1..10] of var float: a5;
  ```

  For such arrays, the index type specifies exactly the indices that will be in the array - the array's index set is the *domain* of the index type - and if the indices of the value assigned do not match then it is a run-time error.

  For example, the following assignments cause run-time errors:

  ```
  a1 = [4,6,4,3,2];   % too many elements
  a2 = [3,2,6,5];     % index set mismatch
  a5 = [||];          % too few elements
  ```

  For `a2` above, the index set of the array literal `[3,2,6,5]` is (implicitly) `1..4`, so it cannot be assigned to an array declared with index set `0..3`, even though the length matches. A correct assignment would use the `array1d` function (see *Array Operations* (page 328)):

  ```
  a2 = array1d(0..3,[3,2,6,5]);     % correct
  ```

- *Implicitly-indexed* arrays have index types in the declaration that are not finite types. For example:

  ```
  array[int,int] of int: a6;
  ```

  No checking of indices occurs when these variables are assigned.

In MiniZinc all index sets of an array must be contiguous ranges of integers, or enumerated types. The expression used for initialisation of an array must have matching index sets. An array expression with an enum index set can be assigned to an array declared with an integer index set, but not the other way around. The exception are array literals, which can be assigned to arrays declared with enum index sets.

For example:

```
enum X = {A,B,C};
enum Y = {D,E,F};
array[X] of int: x = array1d(X, [5,6,7]); % correct
array[Y] of int: y = x;                   % index set mismatch: Y != X
```

```
array[int] of int: z = x;                % correct: assign X index set to␣
↪int
array[X] of int: x2 = [10,11,12];        % correct: automatic coercion for␣
↪array literals
```

The initialisation of an array can be done in a separate assignment statement, which may be present in the model or a separate data file.

Arrays can be accessed. See *Indexed Array Comprehensions* (page 302) for details.

Allowed Insts. An array's size must be fixed. Its indices must also have fixed type-insts. Its elements may be fixed or unfixed.

Syntax. An array base type-inst expression tail has this syntax:

```
<array-ti-expr> ::= "array" "[" <array-index> "," ... "]" "of" <base-ti-expr>
                  | "list" "of" <base-ti-expr>

<array-index> ::= <ti-expr>
                | <ident> "in" <ti-expr>
```

Some example array type-inst expressions:

```
array[1..10] of int
list of var int
```

Note that `list of <T>` is sugar for `array[1..infinity] of <T>`: a one-dimensional array whose index set starts at `1` but whose length is arbitrary. This is *not* the same as `array[int] of <T>`, which permits any integer index set. A value bound to a `list` — as a function argument, in a variable declaration, or as a function result — must have an index set that starts at `1`; otherwise it is an index set mismatch, and the value has to be rebased explicitly using `array1d`. This holds for every `list` position, including a `list` field of a tuple or record and a `list` nested in the elements of an array. Inside a function, the array is therefore guaranteed to be indexed from `1`. *Rationale: 1-based integer-indexed arrays are very common, and a predicate that iterates over a parameter using* `1..length(x)` *is only correct if the argument is 1-based. Implementing* `list` *as sugar for an unbounded index set avoids adding an extra type to the language while still expressing the 1-based guarantee. Rebasing is not applied implicitly, because silently renumbering the caller's array would make* `list` *parameters behave differently from* `array[int] of <T>` *parameters, which preserve the index set, in a way that is invisible at the call site.*

Note that most arrays are 1-based already, and so can be passed to a `list` without any rebasing: array literals, comprehensions, array slices, and enum-indexed arrays all have index sets that start at `1`.

An unbounded array index set is only permitted if its lower bound is `1` (that is, `1..infinity`, which is what `list of` produces); any other lower bound, such as `array[3..infinity] of int`, is a type error.

Because arrays must be fixed-size it is a type-inst error to precede an array type-inst expression with `var`.

Finite? Yes, if the index types and element type are all finite types. Otherwise, no.

The domain of an array type that is a finite array is the set of all distinct arrays whose index set equals the domain of the index type and whose elements are of the array element type.

Varifiable? No.

Ordering. Arrays are ordered lexicographically, taking absence of a value for a given key to be before any value for that key. For example, `[1, 1]` is less than `[1, 2]`, which is less than `[1, 2, 3]` and `array1d(2..4,[0, 0, 0])` is less than `[1, 2, 3]`.

Initialisation. An explicitly-indexed array variable must be initialised at instance-time only if its elements must be initialised at instance time. An implicitly-indexed array variable must be initialised at instance-time so that its length and index set is known.

Coercions. `array[TI0] of TI` $\xrightarrow{c}$ `array[UI0] of UI` if `TI0` $\xrightarrow{c}$ `UI0` and `TI` $\xrightarrow{c}$ `UI`.

Index-dependent array declarations

In an ordinary array declaration each index list entry is just a type-inst that specifies the index set of a dimension. An index list entry may also take the form

```
<ident> "in" <ti-expr>
```

which binds the identifier to the index of each element of that dimension and makes it available in the element type-inst. The element type then becomes an *index-dependent element type*: it can depend on the per-element index through one or more such binders. For example:

```
array[c in 1..n] of var 1..f(c): a;
```

declares an array whose element `a[c]` has the per-element domain `1..f(c)`. This is equivalent to declaring an array with element type `var int` together with an implicit constraint `forall(c in 1..n)(a[c] in 1..f(c))`.

Multiple dimensions may each introduce their own binder, and binders from outer dimensions are in scope in inner ones. Mixing binder entries with plain index-set entries within the same index list is allowed. Binders also nest through arrays of arrays. For example:

```
array[i in 1..3, j in 1..3] of var 1..(i+j): b;
array[i in 1..3] of array[j in 1..2] of var 1..h(i,j): aoa;
```

Each binder entry must consist of exactly one identifier in front of `in`, and `where` clauses are not allowed in the index list. *Rationale: index-dependent declarations introduce one implicit binder per dimension; multi-identifier generators and filters belong to comprehension syntax, not type-inst syntax.*

The static type of an element is determined from the *type* of the domain expression rather than from its syntactic form: for instance, `array[c in 1..n] of var dom_of(c): a;` where `dom_of` returns `set of float` declares an array of `var float` elements.

The binder is in scope in the element type-inst, which includes the leaves of composite type-insts. An index-dependent declaration therefore composes with tuple, record,

array-of-array element types, and with the *set cardinality syntax* (page 283):

```
array[c in 1..n] of tuple(var 1..f(c), int): t;
array[c in 1..n] of var set(f(c)) of 1..10: s;
```

Index-dependent array declarations are allowed in variable declaration items at the top level of a model and in `let` expressions. They are *not* allowed in function or predicate parameter type-insts. *Rationale: this restriction is reflected syntactically in the parameter grammar; implementations may choose to accept the looser form at parse time in order to report a clearer error during type-checking.*

Option Types

Overview. Option types defined using the `opt` type constructor, define types that may or may not be there. They are similar to `Maybe` types of Haskell implicitly adding a new value `<>` to the type.

Allowed Insts. The argument of an option type must be one of the base types `bool`, `int` or `float`, or a par *tuple* (page 288) or *record* (page 288) type. A `var opt` tuple or record is not supported; since a tuple or record containing any `var` field is itself `var`, an optional tuple or record may not have `var` fields either.

Syntax. The option type is written `opt <T>` where `<T>` if one of the three base types, or one of their constrained instances.

For a tuple or record, `opt` makes it optional as a *whole*: a value of type `opt T` is either a complete value of type `T`, or `<>`. The fields keep their declared types; `opt` does **not** distribute into them. An optional record and a record of optional fields are therefore two different types:

```
opt record(int: a, int: b): p = <>;          % the whole record is absent
record(opt int: a, opt int: b): q = (a: 1, b: <>);  % individual fields␣
↪absent
```

`p` has only the states "some `(a: .., b: ..)`" and "absent", so `absent(p)` holds exactly when `p = <>`. `q` additionally has the mixed states in which some fields are present and others are not. Neither type is a subtype of the other.

Accessing a field of an optional tuple or record yields an optional value: if the tuple or record is absent, so is the field. For example, if `r` has type `opt record(int: x)` then `r.x` has type `opt int`, and it is `<>` whenever `r` is.

An optional tuple or record may have array-typed fields, but such a field cannot be accessed this way, since there is no absent array to stand for it when the tuple or record is absent. Strip the optionality with `deopt` instead, guarding it with `occurs`:

```
opt record(array [1..2] of int: a): r;
array [int] of int: v = if occurs(r) then deopt(r).a else [] endif;
```

Finite? Yes if the underlying type is finite, otherwise no.

Varifiable? Yes.

Ordering. <> is always less than any other value in the type. But beware that overloading of operators like < is different for option types.

Initialisation. An `opt` type variable does not need to be initialised at instance-time. An uninitialised `opt` type variable is automatically initialised to <>.

Coercions. `TI` $\xrightarrow{c}$ `opt UI` if `TI` $\xrightarrow{c}$ `UI`..

Tuple Types

Overview. Tuples are fixed-size, heterogeneous collections. They must contain at least two elements; unary tuples are not allowed.

Allowed Insts. Tuples may contain unfixed elements. A par tuple may be optional; see *Option Types* (page 287).

Syntax. A tuple base type-inst expression tail has this syntax:

```
<tuple-ti-expr-tail> ::= "tuple" ( <ti-expr> "," ... )
```

An example tuple type-inst expression:

```
tuple(int, var float)
```

Finite? Yes, if all its constituent elements are finite types. Otherwise, no. The domain of a tuple type that is a finite type is the Cartesian product of the domains of the element types. For example, the domain of `tuple(1..2, {3,5})` is `{(1,3), (1,5), (2,3), (2,5)}`.

Varifiable? Yes, if all its constituent elements are varifiable.

Ordering. Tuples are ordered lexicographically.

Initialisation. A tuple variable must be initialised at instance-time if any of its constituent elements must be initialised at instance-time.

Coercions. `tuple(TI1, ..., TIn)` $\xrightarrow{c}$ `tuple(UI1, ..., UIn)` if `TI1` $\xrightarrow{c}$ `UI1`, ..., `TIn` $\xrightarrow{c}$ `UIn`.

Record Types

Overview. Records are fixed-size, heterogeneous collections. They are similar to tuples, but have named fields. Field names in different records can be identical, because each record's field names belong to a different namespace.

The order in which a record's field are specified is irrelevant; the following two record type-insts are equivalent:

```
record(var int: x, var int: y)
record(var int: y, var int: x)
```

Allowed Insts. Records may contain unfixed elements. A par record may be optional; see *Option Types* (page 287).

Syntax. A record base type-inst expression tail has this syntax:

```
<record-ti-expr-tail> ::= "record" ( <ti-expr-and-id> "," ... )
```

An example record type-inst expression:

```
record(int: x, int: y)
```

Finite? Yes, if all its constituent elements are finite types. Otherwise, no. The domain of a record type that is a finite type is the same as that of a tuple type, but with the fields included. For example, the domain of `record(1..2: x, {3,5}: y)` is `{(x: 1,y: 3), (x: 1,y: 5), (x: 2,y: 3), (x: 2,y: 5)}`.

Varifiable? Yes, if all its constituent elements are varifiable.

Ordering. Records are ordered lexicographically according to the values of the fields, where the fields are sorted in alphabetical order.

Initialisation. A record variable must be initialised at instance-time if any of its constituent elements must be initialised at instance-time.

Coercions. `record(TI1: x1, ..., TIn: xn)` $\xrightarrow{c}$ `record(UI1: x1, ..., UIn: xn)` if `TI1` $\xrightarrow{c}$ `UI1`, ..., `TIn` $\xrightarrow{c}$ `UIn`.

The Annotation Type

Overview. The annotation type, `ann`, can be used to represent arbitrary term structures. It is augmented by annotation items (*Annotation Items* (page 316)).

Allowed Insts. `ann` is always considered unfixed, because it may contain unfixed elements. It cannot be preceded by `var`.

Syntax. The annotation type is written `ann`.

Finite? No.

Varifiable? No.

Ordering. N/A. Annotation types do not have an ordering defined on them.

Initialisation. An `ann` variable must be initialised at instance-time.

Coercions. None.

4.1.6.7 User-defined Types and Type-insts

Enumerated Types

Overview. Enumerated types (or *enums* for short) provide a set of named alternatives. Each alternative is identified by its *case name*. Enumerated types, like in many other languages, can be used in the place of integer types to achieve stricter type checking.

Allowed Insts. Enums can be fixed or unfixed.

Syntax. Variables of an enumerated type named `X` are represented by the term `X` or `par X` if fixed, and `var X` if unfixed.

Finite? Yes.

The domain of an enum is the set containing all of its case names.

Varifiable? `par X` $\xrightarrow{v}$ `var X`, `var X` $\xrightarrow{v}$ `var X`.

Ordering. When two enum values with different case names are compared, the value with the case name that is declared first is considered smaller than the value with the case name that is declared second.

Initialisation. A fixed enum variable must be initialised at instance-time; an unfixed enum variable need not be.

Coercions. `par X` $\xrightarrow{c}$ `par int`, `var X` $\xrightarrow{c}$ `var int`.

Type-inst Synonyms

Overview. A type-inst synonym is an alternative name for a pre-existing type-inst which can be used interchangeably with the pre-existing type-inst. For example, if `MyFixedInt` is a synonym for `par int` then `MyFixedInt` can be used anywhere `par int` can, and vice versa.

Allowed Insts. Preceding a type-inst synonym with `var` varifies it, unless the type-inst is not varifiable, in which case it is a type-inst error. Preceding a type-inst synonym with `par` has the reverse effect, and the type is explicitly made `par`.

Syntax. A type-inst synonym named "X" is represented by the term X.

Finite? As for the pre-existing type-inst.

Varifiable? Yes, if the pre-existing type-inst is varifiable.

Ordering. As for the pre-existing type-inst.

Initialisation. As for the pre-existing type-inst.

Coercions. As for the pre-existing type-inst.

4.1.6.8 Constrained Type-insts

One powerful feature of MiniZinc is *constrained type-insts*. A constrained type-inst is a restricted version of a *base* type-inst, i.e., a type-inst with fewer values in its domain.

Set Expression Type-insts

Three kinds of expressions can be used in type-insts.

1. Integer ranges: e.g. `1..3`.
2. Set literals: e.g. `var {1,3,5}`.
3. Identifiers: the name of a set parameter (which can be global, let-local, the argument of a predicate or function, or a generator value) can serve as a type-inst.

In each case the base type is that of the set's elements, and the values within the set serve as the domain. For example, whereas a variable with type-inst `var int` can take any integer value, a variable with type-inst `var 1..3` can only take the value 1, 2 or 3.

All set expression type-insts are finite types. Their domain is equal to the set itself.

Float Range Type-insts

Float ranges can be used as type-insts, e.g. `1.0 .. 3.0`. These are treated similarly to integer range type-insts, although `1.0 .. 3.0` is not a valid expression whereas `1 .. 3` is.

Float ranges are not finite types.

4.1.7 Expressions

4.1.7.1 Expressions Overview

Expressions represent values. They occur in various kinds of items. They have the following syntax:

```
<expr> ::= <expr-atom> <expr-binop-tail>

<expr-atom> ::= <expr-atom-head> <expr-atom-tail> <annotations>

<expr-binop-tail> ::= [ <bin-op> <expr> ]

<expr-atom-head> ::= <builtin-un-op> <expr-atom>
                   | "(" <expr> ")"
                   | <ident-or-quoted-op>
                   | "_"
                   | <bool-literal>
                   | <int-literal>
                   | <float-literal>
                   | <string-literal>
                   | <set-literal>
                   | <set-comp>
                   | <array-literal>
                   | <array-literal-2d>
                   | <indexed-array-literal>
                   | <indexed-array-literal-2d>
                   | <tuple-literal>
                   | <record-literal>
                   | <array-comp>
                   | <indexed-array-comp>
                   | <ann-literal>
                   | <if-then-else-expr>
                   | <let-expr>
                   | <call-expr>
                   | <gen-call-expr>

<expr-atom-tail> ::=
                   | <array-access-tail> <expr-atom-tail>
                   | <field-access-tail> <expr-atom-tail>
```

Expressions can be composed from sub-expressions combined with operators. All operators (binary and unary) are described in *Operators* (page 293), including the prece-

dences of the binary operators. All unary operators bind more tightly than all binary operators.

Expressions can have one or more annotations. Annotations bind more tightly than unary and binary operator applications, but less tightly than access operations and non-operator applications. In some cases this binding is non-intuitive. For example, in the first three of the following lines, the annotation `a` binds to the identifier expression `x` rather than the operator application. However, the fourth line features a non-operator application (due to the single quotes around the `not`) and so the annotation binds to the whole application.

```
not x::a;
not (x)::a;
not(x)::a;
'not'(x)::a;
```

Annotations (page 320) has more on annotations.

Expressions can be contained within parentheses.

The array and field access operations all bind more tightly than unary and binary operators and annotations. They are described in more detail in *Indexed Array Comprehensions* (page 302).

The remaining kinds of expression atoms (from `<ident>` to `<gen-call-expr>`) are described in *Identifier Expressions and Quoted Operator Expressions* (page 295) to *Generator Call Expressions* (page 309).

We also distinguish syntactically valid numeric expressions. This allows range types to be parsed correctly.

```
<num-expr> ::= <num-expr-atom> <num-expr-binop-tail>

<num-expr-atom> ::= <num-expr-atom-head> <expr-atom-tail> <annotations>

<num-expr-binop-tail> ::= [ <num-bin-op> <num-expr> ]

<num-expr-atom-head> ::= <builtin-num-un-op> <num-expr-atom>
                       | "(" <num-expr> ")"
                       | <ident-or-quoted-op>
                       | <int-literal>
                       | <float-literal>
                       | <if-then-else-expr>
                       | <let-expr>
                       | <call-expr>
                       | <gen-call-expr>
```

4.1.7.2 Operators

Operators are functions that are distinguished by their syntax in one or two ways. First, some of them contain non-alphanumeric characters that normal functions do not (e.g. `+`). Second, their application is written in a manner different to normal functions.

We distinguish between binary operators, which can be applied in an infix manner (e.g. `3 + 4`), and unary operators, which can be applied in a prefix manner without parentheses (e.g. `not x`). We also distinguish between built-in operators and user-defined operators. The syntax is the following:

```
<builtin-op> ::= <builtin-bin-op> | <builtin-un-op>

<bin-op> ::= <builtin-bin-op> | `<ident>`

<builtin-bin-op> ::= "<->" | "->" | "<-" | "\/" | "xor" | "/\"
                   | "<" | ">" | "<=" | ">=" | "==" | "=" | "!="
                   | "~=" | "~!="
                   | "in" | "subset" | "superset" | "union" | "diff" |
↪"symdiff"
                   | ".." | "intersect" | "++" | "default" |
↪<builtin-num-bin-op>

<builtin-un-op> ::= "not" | <builtin-num-un-op>
```

Again, we syntactically distinguish numeric operators.

```
<num-bin-op> ::= <builtin-num-bin-op> | `<ident>`

<builtin-num-bin-op> ::= "+" | "-" | "*" | "/" | "div" | "mod" | "^"
                       | "~+" | "~-" | "~*" | "~/" | "~div"

<builtin-num-un-op> ::= "+" | "-"
```

Some operators can be written using their unicode symbols, which are listed in Table 1.1 (recall that MiniZinc input is UTF-8).

Table 1.1: Unicode equivalents of binary operators

Operator	Unicode symbol	UTF-8 code
`<->`	↔	E2 86 94
`->`	→	E2 86 92
`<-`	←	E2 86 90
`not`	¬	C2 AC
`\/`	∨	E2 88 A8
`/\`	∧	E2 88 A7
`!=`	≠	E2 89 A0
`<=`	≤	E2 89 A4
`>=`	≥	E2 89 A5
`in`	∈	E2 88 88
`subset`	⊆	E2 8A 86
`superset`	⊇	E2 8A 87
`union`	∪	E2 88 AA
`intersect`	∩	E2 88 A9
`^-1`	$^{-1}$	E2 81 BB C2 B9

The binary operators are listed in Table 1.2. A lower precedence number means tighter binding; for example, `1+2*3` is parsed as `1+(2*3)` because `*` binds tighter than `+`. Associativity indicates how chains of operators with equal precedences are handled; for example, `1+2+3` is parsed as `(1+2)+3` because `+` is left-associative, `a++b++c` is parsed as `a++(b++c)` because `++` is right-associative, and `1<x<2` is a syntax error because `<` is non-associative.

Note that the last entry in the table, `^-1`, is a combination of the binary power operator and the constant -1, which effectively turns it into a unary inverse operator. Other special cases (such as power of 2) are currently not supported yet.

Table 1.2: Binary infix operators

Symbol(s)	Assoc.	Prec.
`<->`	left	1600
`->`	left	1500
`<-`	left	1500
`\/`	left	1400
`xor`	left	1400
`/\`	left	1300
`<`	none	1200
`>`	none	1200
`<=`	none	1200
`>=`	none	1200
`==`	none	1100
`=`	none	1100
`!=`	none	1100
`in`	none	1000
`subset`	none	1000
`superset`	none	1000
`union`	left	900
`diff`	left	900
`symdiff`	left	900

continues on next page

Table 1.2 – continued from previous page

Symbol(s)	Assoc.	Prec.
`intersect`	left	800
`..`	none	700
`<..`	none	700
`..<`	none	700
`<..<`	none	700
`+`	left	600
`-`	left	600
`*`	left	500
`div`	left	500
`mod`	left	500
`/`	left	500
`^`	left	400
`++`	right	300
`default`	left	200
`` ` <ident> ` ``	left	100
`::`	left	0

A user-defined binary operator is created by backquoting a normal identifier, for example:

```
A `min2` B
```

This is a static error if the identifier is not the name of a binary function or predicate.

The unary operators are: `+`, `-` and `not`. Unary `+` and `-` have the same precedence as the binary multiplication operator `*`. The logic `not` operator has precedence 350, i.e., it binds more tightly than all other Boolean and numeric operators.

User-defined unary operators are not possible.

As *Identifiers* (page 275) explains, any built-in operator can be used as a normal function identifier by quoting it, e.g: `'+'(3, 4)` is equivalent to `3 + 4`.

The meaning of each operator is given in *Built-in Operations* (page 324).

4.1.7.3 Expression Atoms

Identifier Expressions and Quoted Operator Expressions

Identifier expressions and quoted operator expressions have the following syntax:

```
<ident-or-quoted-op> ::= <ident>
                       | '<builtin-op>'
```

Examples of identifiers were given in *Identifiers* (page 275). The following are examples of quoted operators:

```
'+'
'union'
```

In quoted operators, whitespace is not permitted between either quote and the operator. *Operators* (page 293) lists MiniZinc's built-in operators.

Syntactically, any identifier or quoted operator can serve as an expression. However, in a valid model any identifier or quoted operator serving as an expression must be the name of a variable.

Anonymous Decision Variables

There is a special identifier, _, that represents an unfixed, anonymous decision variable. It can take on any type that can be a decision variable. It is particularly useful for initialising decision variables within compound types. For example, in the following array the first and third elements are fixed to 1 and 3 respectively and the second and fourth elements are unfixed:

```
array[1..4] of var int: xs = [1, _, 3, _];
```

Any expression that does not contain _ and does not involve decision variables is fixed.

Boolean Literals

Boolean literals have this syntax:

```
<bool-literal> ::= "false" | "true"
```

Integer and Float Literals

There are three forms of integer literals - decimal, hexadecimal, and octal - with these respective forms:

```
<int-literal> ::= [0-9]+
                | 0x[0-9A-Fa-f]+
                | 0o[0-7]+
```

For example: `0`, `005`, `123`, `0x1b7`, `0o777`; but not `-1`.

Float literals have the following form:

```
<float-literal> ::= [0-9]+"."[0-9]+
                  | [0-9]+"."[0-9]+[Ee][-+]?[0-9]+
                  | [0-9]+[Ee][-+]?[0-9]+
                  | 0[xX]([0-9a-fA-F]*"."[0-9a-fA-F]+|[0-9a-fA-F]+"
↪")([pP][+-]?[0-9]+)
                  | (0[xX][0-9a-fA-F]+[pP][+-]?[0-9]+)
```

For example: `1.05`, `1.3e-5`, `1.3e+5`; but not `1.`, `.5`, `1.e5`, `.1e5`, `-1.0`, `-1E05`. A `-` symbol preceding an integer or float literal is parsed as a unary minus (regardless of intervening whitespace), not as part of the literal. This is because it is not possible in general to distinguish a `-` for a negative integer or float literal from a binary minus when lexing.

String Literals and String Interpolation

String literals are written as in C:

```
<string-contents> ::= ([^"\n\] |  \[0-7][0-7]?[0-7]? | \x[0-9a-fA-F][0-9a-fA-
↪F]? | \n | \t | \" | \\)*

<string-literal> ::= """ <string-contents> """
                   | """ <string-contents> "\(" <string-interpolate-tail>

<string-interpolate-tail> ::= <expr> ")"<string-contents>"""
                            | <expr> ")"<string-contents>"\("
↪<string-interpolate-tail>
```

This includes the following C-style escape sequences:

- `\"` for double quotes
- `\\` for backslash
- `\n` for newline
- `\t` for tab
- `\x[0-9a-fA-F][0-9a-fA-F]?` for an arbitrary 8-bit character specified by the given hexadecimal number
- `\[0-7][0-7]?[0-7]?` for an arbitrary 8-bit character specified by the given octal number

For example: `"Hello, world!\n"`.

String literals must fit on a single line.

All string literals are represented and interpreted as UTF-8 strings. This means that you can either include UTF-8 characters directly in the strings, or you can use the hexadecimal or octal notation to insert the UTF-8 byte sequences, for example: `"\xe4\xbd\xa0\xe5\xa5\xbd means hello"` is a valid string.

Long string literals can be split across multiple lines using string concatenation. For example:

```
string: s = "This is a string literal "
         ++ "split across two lines.";
```

A string expression can contain an arbitrary MiniZinc expression, which will be converted to a string similar to the builtin `show` function and inserted into the string. This is called *string interpolation*. You need to use the `\(e)` syntax to interpolate an expression *e* into a string.

For example:

```
var set of 1..10: q;
solve satisfy;
output [show("The value of q is \(q), and it has \(card(q)) elements.")];
```

Set Literals

Set literals have this syntax:

```
<set-literal> ::= "{" [ <expr> "," ... ] "}"
```

For example:

```
{ 1, 3, 5 }
{ }
{ 1, 2.0 }
```

The type-insts of all elements in a literal set must be the same, or coercible to the same type-inst (as in the last example above, where the integer `1` will be coerced to a `float`).

Set Comprehensions

Set comprehensions have this syntax:

```
<set-comp> ::= "{" <expr> "|" <comp-tail> "}"

<comp-tail> ::= <generator> [ "where" <expr> ] "," ...

<generator> ::= ( <ident> | "_" ) "," ... "in" <expr>
              | ( <ident> | "_" ) "=" <expr>
```

For example (with the literal equivalent on the right):

```
{ 2*i | i in 1..5 }     % { 2, 4, 6, 8, 10 }
{  1  | i in 1..5 }     % { 1 }   (no duplicates in sets)
```

The expression before the `|` is the *head expression*. The expression after the `in` is a *generator expression*. Generators can be restricted by a *where-expression*. For example:

```
{ i | i in 1..10 where (i mod 2 = 0) }     % { 2, 4, 6, 8, 10 }
```

When multiple generators are present, the right-most generator acts as the inner-most one. For example:

```
{ 3*i+j | i in 0..2, j in {0, 1} }    % { 0, 1, 3, 4, 6, 7 }
```

The scope of local generator variables is given by the following rules:

- They are visible within the head expression (before the `|`).
- They are visible within the where-expression of their own generator.
- They are visible within generator expressions and where-expressions in any subsequent generators.

The last of these rules means that the following set comprehension is allowed:

```
{ i+j | i in 1..3, j in 1..i }  % { 1+1, 2+1, 2+2, 3+1, 3+2, 3+3 }
```

Multiple where-expressions are allowed, as in the following example:

```
[f(i, j) | i in A1 where p(i), j in A2 where q(i,j)]
```

A generator expression must be an array or a fixed set.

Rationale: For set comprehensions, set generators would suffice, but for array comprehensions, array generators are required for full expressivity (e.g., to provide control over the order of the elements in the resulting array). Set comprehensions have array generators for consistency with array comprehensions, which makes implementations simpler.

The where-expression (if present) must be Boolean. It can be var, in which case the type of the comprehension is lifted to an optional type.

Simple Array Literals

Simple array literals have this syntax:

```
<array-literal> ::= "[" [ <expr> "," ... ] "]"
```

For example:

```
[1, 2, 3, 4]
[]
[1, _]
```

In an array literal all elements must have the same type-inst, or be coercible to the same type-inst (as in the last example above, where the fixed integer `1` will be coerced to a `var int`).

The indices of an array literal are implicitly `1..n`, where `n` is the length of the literal.

Indexed Array Literals

Indexed array literals have this syntax:

```
<indexed-array-literal> ::= "[" [ ( <index-tuple> ":" <expr> ) "," ... ] "]"
                          | "[" [ <index-tuple> ":" <expr> "," <expr> "," ...
↪ ] "]"
```

```
<index-tuple> ::= <expr>
                | "(" [ <expr> "," ... ] ")"
```

For example:

```
[ 1: 1, 2: 2, 3: 3, 4: 4, 5: 5]
[ A: 0, B: 3, C: 5]
[ (1,2): 1, (1,3): 2, (2,2): 3, (2,3): 4]
[ 1: 1, 4: 2, 5: 3, 3: 4, 2: 5]
[ 0: A, B, C ]
```

The expressions before the colons are the indexes (or keys), those after are the values.

The indexes must be of integer or enumerated type. The index set of the resulting array is, for each dimension, the union of the indexes of that dimension. For example, the index set of the first and fourth arrays above is `1..5`, the index set of the second array is `{A,B,C}`, and the index sets of the third array are `1..2` and `2..3`.

For one-dimensional arrays, it is possible to give only the starting index of the first element, as in the last example above. All following elements are indexed consecutively.

The index sets of indexed array literals must be contiguous and, for multi-dimensional arrays, "rectangular". For example, the following literals are not allowed:

```
% not contiguous, index 2 missing:
[ 1: 1, 3:2, 4:, 3]

% not contiguous, index 2 missing in dimension 2:
[ (1,1): 0, (1,3): 1, (2,1): 0, (2,3): 2]

% not rectangular, second row has fewer entries than first row:
[ (1,1): 0, (1,2): 0, (1,3): 0, (2,1): 1, (2,2): 2]
```

The indexes need not be specified in order (e.g. the last example above denotes the same array as the first one).

2d Array Literals

Simple 2d array literals have this syntax:

```
<array-literal-2d> ::= "[|" [ [ <expr> "," ... ] "|" ... ] "|]"
```

For example:

```
[| 1, 2, 3
 | 4, 5, 6
 | 7, 8, 9 |]       % array[1..3, 1..3]
[| x, y, z |]       % array[1..1, 1..3]
[| 1 | _ | _ |]     % array[1..3, 1..1]
```

In a 2d array literal, every sub-array must have the same length.

In a 2d array literal all elements must have the same type-inst, or be coercible to the same type-inst (as in the last example above, where the fixed integer `1` will be coerced to a `var int`).

The indices of a 2d array literal are implicitly `(1,1)..(m,n)`, where `m` and `n` are determined by the shape of the literal.

Indexed 2d Array Literals

Indexed 2d array literals have this syntax:

```
<indexed-array-literal-2d> ::= "[|" [ <expr> ":" ... ] [ [ <expr> ":" ]
↪<expr> "," ... ] "|]"
```

For example:

```
% only column index:
[| A: B: C:
 | 0, 0, 0
 | 1, 1, 1
 | 2, 2, 2 |];

% only row index:
[| A: 0, 0, 0
 | B: 1, 1, 1
 | C: 2, 2, 2 |];

% row and column index:
[|    A: B: C:
 | A: 0, 0, 0
 | B: 1, 1, 1
 | C: 2, 2, 2 |];
```

The index sets are either `1..n` for `n` rows/columns if no index is given, or the union of the given indexes. As for general indexed array literals, the union of the given indexes must be a contiguous set.

Simple Array Comprehensions

Simple array comprehensions have this syntax:

```
<array-comp> ::= "[" <expr> "|" <comp-tail> "]"
```

A generator expression (the part following the `in`) can be either a set or an array (multi-dimensional arrays are iterated over in row-major order).

For example (with the literal equivalents on the right):

```
[2*i | i in 1..5]          % [2, 4, 6, 8, 10]
[2*i | i in [3,5,7] ]      % [6, 10, 14]
[i   | i in [|1,2|3,4|] ] % [1, 2, 3, 4]
```

A generator variable in an array comprehension can also be the anonymous variable _. For instance, the following generates an array of `n*n` ones:

```
[ 1 | _, _ in 1..n ]
```

Array comprehensions have more flexible type and inst requirements than set comprehensions (see *Set Comprehensions* (page 298)).

Array comprehensions are allowed over a variable set with finite type, the result is an array of optional type, with length equal to the cardinality of the upper bound of the variable set. For example:

```
var set of 1..5: x;
array[int] of var opt int: y = [ i * i | i in x ];
```

The length of array will be 5.

Array comprehensions are allowed where the where-expression is a `var bool`. Again the resulting array is of optional type, and of length equal to that given by the generator expressions. For example:

```
var int x;
array[int] of var opt int: y = [ i | i in 1..10 where i != x ];
```

The length of the array will be 10.

The indices of an evaluated simple array comprehension are implicitly `1..n`, where `n` is the length of the evaluated comprehension.

Indexed Array Comprehensions

Indexed array comprehensions have this syntax:

```
<indexed-array-comp> ::= "[" <index-tuple> ":" <expr> "|" <comp-tail> "]"
```

For example:

```
% This equals [ 3:9, 4:12, 5:15 ]:
[ i: 3*i | i in 3..5 ]

% This generates a 2d array
% [|    1:  2:  3:
%  | 2:  7,  8,  9
%  | 3: 10, 11, 12
%  | 4: 13, 14, 15
```

```
%  |]:
[ (i,j): i*3+j | i in 2..4, j in 1..3]
```

The keys for indexed array comprehensions have the same requirements as for indexed array literals (see *Indexed Array Literals* (page 299)).

Array Access Expressions

Array elements are accessed using square brackets after an expression:

```
<array-access-tail> ::= "[" <expr> "," ... "]"
```

For example:

```
int: x = a1[1];
```

If all the indices used in an array access are fixed, the type-inst of the result is the same as the element type-inst. However, if any indices are not fixed, the type-inst of the result is the varified element type-inst. For example, if we have:

```
array[1..2] of int: a2 = [1, 2];
var int: i;
```

then the type-inst of `a2[i]` is `var int`. If the element type-inst is not varifiable, such an access causes a static error.

Multi dimensional arrays are accessed using comma separated indices.

```
array[1..3,1..3] of int: a3;
int: y = a3[1, 2];
```

Indices must match the index set type of the array. For example, an array declared with an enum index set can only be accessed using indices from that enum.

```
enum X = {A,B,C};
array[X] of int: a4 = [1,2,3];
int: y = a4[1];                  % wrong index type
int: z = a4[B];                  % correct
```

Array Slice Expressions

Arrays can be *sliced* in order to extract individual rows, columns or blocks. The syntax is that of an array access expression (see above), but where one or more of the expressions inside the square brackets are set-valued.

For example, the following extracts row 2 from a two-dimensional array:

```
array[1..n,4..8] of int: x;
array[int] of int: row_2_of_x = x[2,4..8];
```

Note that the resulting array `row_2_of_x` will have index set `4..8`.

A short-hand for all indices of a particular dimension is to use just dots:

```
array[1..n,4..8] of int: x;
array[int] of int: row_2_of_x = x[2,..];
```

You can also restrict the index set by giving a sub-set of the original index set as the slice:

```
array[1..n,4..8] of int: x;
array[int] of int: row_2_of_x = x[2,5..6];
```

The resulting array `row_2_of_x` will now have length 2 and index set `5..6`.

The dots notation also allows for partial bounds, for example:

```
array[1..n,4..8] of int: x;
array[int] of int: row_2_of_x = x[2,..6];
```

The resulting array will have length 3 and index set `4..6`. Of course `6..` would also be allowed and result in an array with index set `6..8`.

In addition to the `..` operator, you can also use the `<..`, `..<` and `<..<` operators (Section 4.1.11.4) to exclude the first, last or both elements from the slice. This also works if you leave one or both of the bounds:

```
enum X = {A, B, C, D};
array[X, 1..3] of var int: x;
% x1 = x[A..C, 2..3]
array[_, _] of var int: x1 = x[A..<, <..];
% x2 = x[B..C, 3..3]
array[_, _] of var int: x2 = x[<..<, 2<..];
```

Record and Tuple Field Access Expressions

Fields of records and tuples are accessed using a dot after an expression:

```
<field-access-tail> ::= "."<ident> | "."<int-literal>
```

For example:

```
record(int:a, string: n): r = (a: 3, n: "foo");
int: x = r.a;
string: y = r.n;
```

```
tuple(float, bool): t = (42.0, true);
float: f = t.1;
bool: b = t.2;
```

The type-inst of the result is the type-inst of the field.

Field access can be chained, for example:

```
record(record(int:a, string:n): r1, bool: b): r2 =
    (r1: (a: 3, n: "foo"), b: true);
int: x = r2.r1.a;
```

Field access syntax can be used to project fields from arrays of records or tuples. For example:

```
array[1..3] of record(int:a, string:n): r_array =
    [ (a: 1, n: "one"), (a: 2, n: "two"), (a: 3, n: "three") ];
array[1..3] of int: a_array = r_array.a;
array[1..3] of string: n_array = r_array.n;
```

Annotation Literals

Literals of the ann type have this syntax:

```
<ann-literal> ::= <ident> [ "(" <expr> "," ... ")" ]
```

For example:

```
foo
cons(1, cons(2, cons(3, nil)))
```

There is no way to inspect or deconstruct annotation literals in a MiniZinc model; they are intended to be inspected only by an implementation, e.g., to direct compilation.

If-then-else Expressions

MiniZinc provides if-then-else expressions, which provide selection from two alternatives based on a condition. They have this syntax:

```
<if-then-else-expr> ::= "if" <expr> "then" <expr> [ "elseif" <expr> "then"
↪<expr> ]* [ "else" <expr> ] "endif"
```

For example:

```
if x <= y then x else y endif
if x < 0 then -1 elseif x > 0 then 1 else 0 endif
```

The presence of the `endif` avoids possible ambiguity when an if-then-else expression is part of a larger expression.

The type-inst of the `if` expression must be `par bool` or `var bool`. The `then` and `else` expressions must have the same type-inst, or be coercible to the same type-inst, which is also the type-inst of the whole expression.

If the `if` expression is `var bool` then the type-inst of the `then` and `else` expressions must be varifiable.

If the `if` expression is `par bool` then evaluation of if-then-else expressions is lazy: the condition is evaluated, and then only one of the `then` and `else` branches are evaluated, depending on whether the condition succeeded or failed. This is not the case if it is `var bool`.

The `else` branch is optional if the type of the `then` branches is `bool`, `string`, `ann` or an `array` type. In that case, the `else` branch is taken to be the default value of the type:

- `false` for `bool`
- `""` for `string`
- `empty_annotation` for `ann`
- `[]` for `array`

Let Expressions

Let expressions provide a way of introducing local names for one or more expressions and local constraints that can be used within another expression. They are particularly useful in user-defined operations.

Let expressions have this syntax:

```
<let-expr> ::= "let" "{" <let-item> ( ";" | "," ) ... "}" "in" <expr>

<let-item> ::= <var-decl-item>
             | <constraint-item>
```

For example:

```
let { int: x = 3; int: y = 4; } in x + y;
let { var int: x;
      constraint x >= y /\ x >= -y /\ (x = y \/ x = -y); }
in x
```

The scope of a let local variable covers:

- The type-inst and initialisation expressions of any subsequent variables within the let expression (but not the variable's own initialisation expression).
- The expression after the `in`, which is parsed as greedily as possible.

A variable can only be declared once in a let expression.

Thus in the following examples the first is acceptable but the rest are not:

```
let { int: x = 3; int: y = x; } in x + y;  % ok
let { int: y = x; int: x = 3; } in x + y;  % x not visible in y's defn.
let { int: x = x; } in x;                  % x not visible in x's defn.
let { int: x = 3; int: x = 4; } in x;      % x declared twice
```

The initialiser for a let local variable can be omitted only if the variable is a decision variable. For example:

```
let { var int: x; } in ...;    % ok
let {     int: x; } in ...;    % illegal
```

For let local variables with initialisers, the type-inst can be omitted by using the `any` keyword, in which case the type-inst is inferred from the initialiser:

```
let { any: x = [5,6,7]; } in ...;    % x has type array[1..3] of int
```

The type-inst of the entire let expression is the type-inst of the expression after the `in` keyword.

There is a complication involving let expressions in negative contexts. A let expression occurs in a negative context if it occurs in an expression of the form `not X`, `X <-> Y}` or in the sub-expression `X` in `X -> Y` or `Y <- X`, or in a subexpression `bool2int(X)`.

If a let expression is used in a negative context, then any let-local decision variables must be defined only in terms of non-local variables and parameters. This is because local variables are implicitly existentially quantified, and if the let expression occurred in a negative context then the local variables would be effectively universally quantified which is not supported by MiniZinc.

Constraints in let expressions float to the nearest enclosing Boolean context. For example

```
constraint b -> x + let { var 0..2: y; constraint y != -1;} in y >= 4;
```

is equivalent to

```
var 0..2: y;
constraint b -> (x + y >= 4 /\ y != 1);
```

For backwards compatibility with older versions of MiniZinc, items inside the `let` can also be separated by commas instead of semicolons.

Call Expressions

Call expressions are used to call predicates and functions.

Call expressions have this syntax:

```
<call-expr> ::= <ident-or-quoted-op> [ "(" <call-arg> "," ... ")" ]

<call-arg> ::= <expr> | <ident> ":" <expr>
```

For example:

```
x = min(3, 5);
```

The type-insts of the expressions passed as arguments must match the argument types of the called predicate/function. The return type of the predicate/function must also be appropriate for the calling context.

Arguments may be passed by position, as above, or by name using the syntax `name: expr`, where `name` is a parameter name of the called operation. For example, given the user-defined operation `predicate foo(int: x, int: y) = ...`, the following calls are equivalent:

```
foo(1, 2);
foo(x: 1, y: 2);
foo(y: 2, x: 1);
```

Named arguments may be given in any order. Positional and named arguments can be mixed in a single call, but all positional arguments must come before any named argument; a positional argument binds to the parameter in the corresponding position, and the named arguments bind the remaining parameters. Each parameter must be bound exactly once, so a named argument may not refer to a parameter that has already been bound positionally, and no parameter name may appear more than once.

Named arguments are supported for user-defined operations and for the operations provided by the MiniZinc library. They are not yet supported for the low-level operations that a solver supports natively, that is, those declared without a body (see *User-defined Operations* (page 316)). Such an operation is positional-only, and supplying one of its arguments by name is a type error. *Rationale: the parameter names of a native operation become part of its public interface only once every solver library agrees on them. Until the solver ecosystem has converged on a common set of names, native operations are called positionally, so that a model resolves identically against every solver.*

Parameter names that start with an underscore (_) are not part of the public interface of an operation: they may be used inside the body but cannot be supplied as a named argument at a call site. This lets a declaration leave such parameter names unspecified for callers.

Because parameter names form part of the public interface, the MiniZinc library names them according to a consistent convention, so that named arguments read well and additions to the library stay predictable. When an argument's role is clear from the operation itself, a short generic name is used: a scalar is `x`, `y` or `z`; a set is `s`, then `t`; an array of decision variables is `xs`, then `ys`; an array of coefficients is `as`; and an array index is `idx`. Otherwise a descriptive name is chosen, as in `cumulative(start, duration, resource, capacity)`. For a descriptive array, the grammatical number reflects its role. When several parallel arrays together describe one collection of objects (one array per field, all indexed in lock-step) each array is named

in the *singular* after the field it holds, as if it were a member of the object; thus `disjunctive(start, duration)` passes the start and duration of every task as two arrays. An array that is itself a collection of values instead takes a *plural* name, such as `weights` or `tuples`. Two parameters of the same function or predicate are never distinguished only by the case of a letter, such as `c` and `C`, because the two are too easy to confuse at a named call site; at least one of them is spelled out in full (for example `capacity` and `cost`).

Note that a call to a function or predicate with no arguments is syntactically indistinguishable from the use of a variable, and so must be determined during type-inst checking.

Evaluation of the arguments in call expressions is strict: all arguments are evaluated before the call itself is evaluated. Note that this includes Boolean operations such as `/\`, `\/`, `->` and `<-` which could be lazy in one argument. The one exception is `assert`, which is lazy in its third argument (*Other Operations* (page 332)).

Rationale: Boolean operations are strict because: (a) this minimises exceptional cases; (b) in an expression like `A -> B` *where* `A` *is not fixed and* `B` *causes an abort, the appropriate behaviour is unclear if laziness is present; and (c) if a user needs laziness, an if-then-else can be used.*

The order of argument evaluation is not specified. *Rationale: Because MiniZinc is declarative, there is no obvious need to specify an evaluation order, and leaving it unspecified gives implementors some freedom.*

Generator Call Expressions

MiniZinc has special syntax for certain kinds of call expressions which makes models much more readable.

Generator call expressions have this syntax:

```
<gen-call-expr> ::= <ident-or-quoted-op> "(" <comp-tail> ")" "(" <expr> ")"
```

A generator call expression `P(Gs)(E)` is equivalent to the call expression `P([E | Gs])`. For example, the expression:

```
forall(i,j in Domain where i<j)
    (noattack(i, j, queens[i], queens[j]));
```

(in a model specifying the N-queens problem) is equivalent to:

```
forall( [ noattack(i, j, queens[i], queens[j])
        | i,j in Domain where i<j ] );
```

The parentheses around the latter expression are mandatory; this avoids possible confusion when the generator call expression is part of a larger expression.

The identifier must be the name of a unary predicate or function that takes an array argument.

The generators and where-expression (if present) have the same requirements as those in array comprehensions (*Simple Array Comprehensions* (page 301)).

4.1.8 Items

This section describes the top-level program items.

4.1.8.1 Include Items

Include items allow a model to be split across multiple files. They have this syntax:

```
<include-item> ::= "include" <string-literal>
```

For example:

```
include "foo.mzn";
```

includes the file `foo.mzn`.

Include items are particularly useful for accessing libraries or breaking up large models into small pieces. They are not, as *Model Instance Files* (page 277) explains, used for specifying data files.

If the given name is not a complete path then the file is searched for in an implementation-defined set of directories. The search directories must be able to be altered with a command line option.

4.1.8.2 Variable Declaration Items

Variable declarations have this syntax:

```
<var-decl-item> ::= <ti-expr-and-id> <annotations> [ "=" <expr> ]
                  | "any" ":" <ident> <annotations> [ "=" <expr> ]
```

For example:

```
int: A = 10;
```

It is a type-inst error if a variable is declared and/or defined more than once in a model.

A variable whose declaration does not include an assignment can be initialised by a separate assignment item (*Assignment Items* (page 314)). For example, the above item can be separated into the following two items:

```
int: A;
...
A = 10;
```

All variables that contain a parameter component must be defined at instance-time.

Variable declarations with a definition can omit the type-inst by using the `any` keyword:

```
any: x = [1, 2, 3];
any: y = x[2];
```

The type-inst of these variables is inferred by MiniZinc to be that of the definition. In the example above, `x` would have type *array[1..3] of int*, and `y` would have type *int*. Note that variables with type-inst `any` can only be defined in-place, not using separate assignment items.

Variables can have one or more annotations. *Annotations* (page 320) has more on annotations.

4.1.8.3 Enum Items

Enumerated type items have this syntax:

```
<enum-item>  ::= "enum" <ident> <annotations> [ "=" <enum-cases-list> ]

<enum-cases-list> ::= <enum-cases>
                    | <enum-cases-list> "++" <enum-cases>

<enum-cases> ::= "{" <ident> "," ... "}"
               | "_" "(" <expr> ")"
               | <ident> "(" <expr> ")"
```

An example of an enum:

```
enum country = {Australia, Canada, China, England, USA};
```

Each alternative is called an *enum case*. The identifier used to name each case (e.g. `Australia`) is called the *enum case name*.

Because enum case names all reside in the top-level namespace (*Namespaces* (page 277)), case names in different enums must be distinct.

Instead of defining explicit, named enum cases, it is also possible to define an *anonymous enum*. The special enum constructor `_` takes an integer set argument. For example,

```
enum Slot = _(1..n);
```

defines an enumerated type `Slot` with `n` cases. An alternative syntax for anonymous enums is `enum Slot = anon_enum(n);`, which defines the same enum as the code above.

Enumerated type items can also be defined in terms of other sets and enumerated types by using an *enumerated type constructor* and the `++` operator, as in the following example:

```
enum country_or_none = C(country) ++ {None};
```

The `country_or_none` type now contains all of the cases of the `country` type, plus the new case `None`. The two enumerated types are however completely separate: `country` is *not* a subtype of `country_or_none`. In order to coerce a value of type `country` to type

`country_or_none`, use the constructor. E.g., `C(Canada)` is of type `country_or_none`. Similarly, you can use the *inverse constructor* to coerce a value back:

```
country_or_none: c_o_n = C(England);
country: c = C ⁻¹(c_o_n);
```

The inverse operator can be written using unicode characters (as in the example) or using ASCII syntax, e.g. `C^-1(c_o_n)`.

The same enum constructor syntax also works with integer sets, for example

```
enum Node = Left(1..n) ++ Right(1..n);
```

declares an enum for nodes in a bipartite graph with `n` left nodes and `n` right nodes.

The enum constructor can also be used without arguments to return the set of values of the particular constructor. For example, given a `Node: n`, we can use the syntax *n in Left()* to test if `n` is in the `Left` set.

Enum constructors can be used to map non-contiguous sets to (contiguous) enumerated types. Consider the following example:

```
enum Person = { Amy, Bert, Celeste, Doug, Emely };
enum Staff = S({Amy, Doug});
enum Customers = C(Person diff S ⁻¹(Staff));
```

An enum constructor and its inverse can be applied to sets and arrays of enum values as well:

```
array[int] of Customers: custs = S([Bert])
array[int] of Person: custs_as_people = C ⁻¹(custs);
```

An enum can be declared but not defined, in which case it must be defined elsewhere within the model, or in a data file. For example, a model file could contain this:

```
enum Workers;
enum Shifts;
```

and the data file could contain this:

```
Workers = { welder, driller, stamper };
Shifts  = { idle, day, night };
```

Sometimes it is useful to be able to refer to one of the enum case names within the model. This can be achieved by using a variable. The model would read:

```
enum Shifts;
Shifts: idle;           % Variable representing the idle constant.
```

and the data file:

```
enum Shifts = { idle_const, day, night };
idle = idle_const;      % Assignment to the variable.
```

Although the constant `idle_const` cannot be mentioned in the model, the variable `idle` can be.

All enums must be defined at instance-time.

Enum items can be annotated. *Annotations* (page 320) has more details on annotations.

Each case name can be coerced automatically to the integer corresponding to its index in the type.

```
int: oz = Australia;  % oz = 1
```

For each enumerated type `T`, the following functions exist:

```
% Return next greater enum value of x in enum type X
function T: enum_next(T: x);
function var T: enum_next(var T: x);

% Return next smaller enum value of x in enum type X
function T: enum_prev(T: x);
function var T: enum_prev(var T: x);

% Return enum base set for value x in enum type x
function set of T: enum_of(T: x);
function set of T: enum_of(var T: x);
function set of T: enum_of(set of T: x);
function set of T: enum_of(var set of T: x);

% Convert enum type x to an integer
function int: enum2int(T: x);
function var int: enum2int(var T: x);

% Convert x to enum type X
function T: to_enum(set of T: X, int: x);
function var T: to_enum(set of T: X, var int: x);
```

4.1.8.4 Type-inst Synonym Items

Type-inst synonym items have this syntax:

```
<type-inst-syn-item> ::= "type" <ident> <annotations> "=" <ti-expr>
```

For example:

```
type MyInt     = int;
type Person    = record(string: name, int: height);
type Domain    = 1..n;
```

It is a type-inst error if a type-inst synonym is declared and/or defined more than once in a model. Type-inst synonym items can be annotated. Section (*Annotation Items* (page 316)) has more details on annotations. All type-inst synonyms must be defined at model-time.

4.1.8.5 Assignment Items

Assignments have this syntax:

```
<assign-item> ::= <ident> "=" <expr>
```

For example:

```
A = 10;
```

4.1.8.6 Constraint Items

Constraint items form the heart of a model. Any solutions found for a model will satisfy all of its constraints.

Constraint items have this syntax:

```
<constraint-item> ::= "constraint" [ <string-annotation> ] <expr>
```

For example:

```
constraint a*x < b;
```

The expression in a constraint item must have type-inst `par bool` or `var bool`; note however that constraints with fixed expressions are not very useful.

4.1.8.7 Solve Items

Every model must have exactly one or no solve item. Solve items have the following syntax:

```
<solve-item> ::= "solve" <annotations> "satisfy"
               | "solve" <annotations> "minimize" <expr>
               | "solve" <annotations> "maximize" <expr>
```

Example solve items:

```
solve satisfy;
solve maximize a*x + y - 3*z;
```

The solve item determines whether the model represents a constraint satisfaction problem or an optimisation problem. If there is no solve item, the model is assumed to be a satisfaction problem. For optimisation problems, the given expression is the one to be minimized/maximized.

The expression in a `minimize`/`maximize` solve item can have integer or float type.

Rationale: This is possible because all type-insts have a defined order. Note that having an expression with a fixed type-inst in a solve item is not very useful as it means that the model requires no optimisation.

Solve items can be annotated. *Annotations* (page 320) has more details on annotations.

4.1.8.8 Output Items

Output items are used to present the solutions of a model instance. They have the following syntax:

```
<output-item> ::= "output" [ "::" <output-annotation> ] <expr>

<output-annotation> ::= <string-literal>
                      | <ident> "(" [ <expr> "," ... ] ")"
                      | "(" <expr> ")"
```

For example:

```
output ["The value of x is ", show(x), "!\n"];
```

The expression must have type-inst `array[int] of par string`. It can be composed using the built-in operator `++` and the built-in functions `show`, `show_int`, and `show_float` (*Built-in Operations* (page 324)), as well as string interpolations (*String Literals and String Interpolation* (page 297)). The output is the concatenation of the elements of the array. If multiple output items exist, the output is the concatenation of all of their outputs, in the order in which they appear in the model.

If no output item is present, the implementation should print all the global variables and their values in a readable format.

Output items can optionally given a string annotation denoting a section name which the output is associated with. Sections can be turned on or off using the `--only-sections` and `--not-sections` command line arguments (*The MiniZinc Command Line Tool* (page 151)).

JSON Output Items

Output items can also be used to format an expression as JSON (see *JSON support* (page 333)) by annotating the output item with `json_section("<section name>")`:

```
output :: json_section("my_section") (a: 1, b: [2, 3]);
```

The output expression can have any type and is encoded using `showJSON` function. If there are multiple output items with the same JSON section name, only the first one is used.

4.1.8.9 Annotation Items

Annotation items are used to augment the `ann` type. They have the following syntax:

```
<annotation-item> ::= "annotation" <ident> <params> [ "=" <expr> ]
```

For example:

```
annotation solver(int: kind);
```

It is a type-inst error if an annotation is declared and/or defined more than once in a model.

The use of annotations is described in *Annotations* (page 320).

4.1.8.10 User-defined Operations

MiniZinc models can contain user-defined operations. They have this syntax:

```
<predicate-item> ::= "predicate" <operation-item-tail>

<test-item> ::= "test" <operation-item-tail>

<function-item> ::= "function" <ti-expr> ":" <operation-item-tail>

<operation-item-tail> ::= <ident> <params> <annotations> [ "=" <expr> ]

<params> ::= [ ( <param-ti-expr-and-id> [ "=" <expr> ] "," ... ) ]
```

The type-inst expressions can include type-inst variables in the function and predicate declaration.

For example, predicate `even` checks that its argument is an even number.

```
predicate even(var int: x) =
    x mod 2 = 0;
```

A predicate supported natively by the target solver is declared without a body; the solver's library supplies the definition, or leaves it body-less so that the constraint is passed straight to the solver. By convention these interface predicates carry the `fzn_`-prefix, and the user-facing global constraints are ordinary library predicates that call them. For example, native all-different support is provided through

```
predicate fzn_all_different_int(array [int] of var int: xs);
```

while `all_different` itself is a library predicate whose body calls `fzn_all_different_int`. A solver overrides the `fzn_` interface predicates, never the user-facing globals.

Predicate declarations that are natively supported in MiniZinc are restricted to using FlatZinc types (for instance, multi-dimensional and non-1-based arrays are forbidden). A body-less declaration also fixes the parameter names for its overload family, reserving them as the canonical interface; because such an operation is positional-only in the current release (see *Call Expressions* (page 307)), those names are not yet used at call sites.

A parameter can be given a default value using the syntax `ti: name = expr`. When the corresponding argument is omitted at a call site, the default expression is used in its place. For example:

```
function int: inc(int: x, int: step = 1) =
    x + step;
```

Here `inc(5)` evaluates to `6` and `inc(5, 10)` to `15`. A default expression is evaluated in the scope enclosing the declaration and must not refer to the operation's other parameters. A parameter without a default value may not follow one that has a default, so that the omitted arguments are always a suffix of the parameter list.

Declarations for user-defined operations can be annotated. *Annotations* (page 320) has more details on annotations.

Basic Properties

The term "predicate" is generally used to refer to both test items and predicate items. When the two kinds must be distinguished, the terms "test item" and "predicate item" can be used.

The return type-inst of a test item is implicitly `par bool`. The return type-inst of a predicate item is implicitly `var bool`.

Predicates and functions are allowed to be recursive. Termination of a recursive function call depends solely on its fixed arguments, i.e., recursive functions and predicates cannot be used to define recursively constrained variables.

Predicates and functions introduce their own local names, being those of the formal arguments. The scope of these names covers the predicate/function body. Argument names cannot be repeated within a predicate/function declaration.

Ad-hoc polymorphism

MiniZinc supports ad-hoc polymorphism via overloading. Functions and predicates (both built-in and user-defined) can be overloaded. A name can be overloaded as both a function and a predicate.

It is a type-inst error if a single version of an overloaded operation with a particular type-inst signature is defined more than once in a model. For example:

```
predicate p(1..5: x);
predicate p(1..5: x) = false;       % ok:     first definition
```

```
predicate p(1..5: x) = true;        % error:  repeated definition
```

Because arguments may be passed by name, overloadings are distinguished by the *names* of their parameters as well as by their types. Two overloadings of an operation may therefore share their parameter types provided they do not share their parameter names (this applies to operations with a body, since a call that selects between such overloadings by name is not available for the positional-only native operations):

```
predicate interval(var int: start, var int: end) = ...;
predicate interval(var int: start, var int: duration) = ...;   % ok
```

Conversely, it is a type-inst error to declare two overloadings that a call supplying all of its arguments by name could not tell apart, that is, two overloadings that give the same name and the same type to every parameter that must be supplied. Parameter order does not enter into this, and neither do parameters that have a default value, since a call need not supply them:

```
predicate q(var float: y, var int: x);
predicate q(var int: x, var float: y);      % error: q(x: a, y: b) matches␣
↪both

predicate r(int: x);
predicate r(int: x, int: z = 0);            % error: r(x: 1) matches both
```

This restriction does not apply to parameters whose names start with an underscore, as those cannot be supplied by name at all.

The combination of overloading and coercions can cause problems. Two overloadings of an operation are said to *overlap* if they could match the same arguments. For example, the following overloadings of `p` overlap, as they both match the call `p(3)`.

```
predicate p(par int: x);
predicate p(var int: x);
```

However, the following two predicates do not overlap because they cannot match the same argument:

```
predicate q(int:        x);
predicate q(set of int: x);
```

We avoid two potential overloading problems by placing some restrictions on overlapping overloadings of operations.

1. The first problem is ambiguity. Different placement of coercions in operation arguments may allow different choices for the overloaded function. For instance, if a MiniZinc function `f` is overloaded like this:

```
function int: f(int: x, float: y) = 0;
function int: f(float: x, int: y) = 1;
```

then `f(3,3)` could be either 0 or 1 depending on coercion/overloading choices.

To avoid this problem, any overlapping overloadings of an operation must be semantically equivalent with respect to coercion. For example, the two overloadings of the predicate `p` above must have bodies that are semantically equivalent with respect to overloading.

Currently, this requirement is not checked and the modeller must satisfy it manually. In the future, we may require the sharing of bodies among different versions of overloaded operations, which would provide automatic satisfaction of this requirement.

2. The second problem is that certain combinations of overloadings could require a MiniZinc implementation to perform combinatorial search in order to explore different choices of coercions and overloading. For example, if function `g` is overloaded like this:

```
function float: g(int: t1, float: t2) = t2;
function int  : g(float: t1, int: t2) = t1;
```

then how the overloading of `g(3,4)` is resolved depends upon its context:

```
float: s = g(3,4);
int: t = g(3,4);
```

In the definition of `s` the first overloaded definition must be used while in the definition of `t` the second must be used.

To avoid this problem, all overlapping overloadings of an operation must be closed under intersection of their input type-insts. That is, if overloaded versions have input type-inst $(S_1, ..., S_n)$ and $(T_1, ..., T_n)$ then there must be another overloaded version with input type-inst $(R_1, ..., R_n)$ where each R_i is the greatest lower bound (*glb*) of S_i and T_i.

Also, all overlapping overloadings of an operation must be monotonic. That is, if there are overloaded versions with input type-insts $(S_1, ..., S_n)$ and $(T_1, ..., T_n)$ and output type-inst S and T, respectively, then $S_i \preceq T_i$ for all i, implies $S \preceq T$. At call sites, the matching overloading that is lowest on the type-inst lattice is always used.

For `g` above, the type-inst intersection (or *glb*) of `(int,float)` and `(float,int)` is `(int,int)`. Thus, the overloaded versions are not closed under intersection and the user needs to provide another overloading for `g` with input type-inst `(int,int)`. The natural definition is:

```
function int: g(int: t1, int: t2) = t1;
```

Once `g` has been augmented with the third overloading, it satisfies the monotonicity requirement because the output type-inst of the third overloading is `int` which is less than the output type-inst of the original overloadings.

Monotonicity and closure under type-inst conjunction ensure that whenever an overloaded function or predicate is reached during type-inst checking, there is always a unique and safe "minimal" version to choose, and so the complexity of type-inst checking remains linear. Thus in our example `g(3,4)` is always resolved by choosing the new overloaded definition.

Local Variables

Local variables in operation bodies are introduced using let expressions. For example, the predicate `have_common_divisor` takes two integer values and checks whether they have a common divisor greater than one:

```
predicate have_common_divisor(int: A, int: B) =
    let {
        var 2..min(A,B): C;
    } in
        A mod C = 0 /\
        B mod C = 0;
```

However, as *Let Expressions* (page 306) explained, because `C` is not defined, this predicate cannot be called in a negative context. The following is a version that could be called in a negative context:

```
predicate have_common_divisor(int: A, int: B) =
    exists(C in 2..min(A,B))
        (A mod C = 0 /\ B mod C = 0);
```

4.1.9 Annotations

Annotations allow a modeller to specify non-declarative and solver-specific information that is beyond the core language. Annotations do not change the meaning of a model, however, only how it is solved.

Annotations can be attached to variables (on their declarations), constraints, expressions, type-inst synonyms, enum items, solve items and on user defined operations. They have the following syntax:

```
<annotations> ::= [ "::" <annotation> ]*

<annotation> ::= <expr-atom-head> <expr-atom-tail>

<string-annotation> ::= "::" <string-literal>
```

For example:

```
int: x::foo;
x = (3 + 4)::bar("a", 9)::baz("b");
solve :: blah(4)
```

```
    minimize x;
```

The types of the argument expressions must match the argument types of the declared annotation. Like user-defined predicates and functions, annotations can be overloaded.

Annotation signatures can contain type-inst variables.

The order and nesting of annotations do not matter. For the expression case it can be helpful to view the annotation connector `::` as an overloaded operator:

```
ann: '::'(var $T: e, ann: a);       % associative
ann: '::'(ann:    a, ann: b);       % associative + commutative
```

Both operators are associative, the second is commutative. This means that the following expressions are all equivalent:

```
e :: a :: b
e :: b :: a
(e :: a) :: b
(e :: b) :: a
e :: (a :: b)
e :: (b :: a)
```

This property also applies to annotations on solve items and variable declaration items.
Rationale: This property make things simple, as it allows all nested combinations of annotations to be treated as if they are flat, thus avoiding the need to determine what is the meaning of an annotated annotation. It also makes the MiniZinc abstract syntax tree simpler by avoiding the need to represent nesting.

Annotations have to be values of the `ann` type or string literals. The latter are used for *naming* constraints and expressions, for example

```
constraint ::"first constraint" alldifferent(x);
constraint ::"second constraint" alldifferent(y);
constraint forall (i in 1..n) (my_constraint(x[i],y[i])::"constraint \(i)");
```

Note that constraint items can *only* be annotated with string literals.

Rationale: Allowing arbitrary annotations on constraint items makes the grammar ambiguous, and seems unnecessary since we can just as well annotate the constraint expression.

4.1.10 Partiality

The presence of constrained type-insts in MiniZinc means that various operations are potentially *partial*, i.e., not clearly defined for all possible inputs. For example, what happens if a function expecting a positive argument is passed a negative argument? What happens if a variable is assigned a value that does not satisfy its type-inst constraints? What happens if an array index is out of bounds? This section describes what happens in all these cases.

In general, cases involving fixed values that do not satisfy constraints lead to run-time aborts. *Rationale: Our experience shows that if a fixed value fails a constraint, it is almost certainly due to a programming error. Furthermore, these cases are easy for an implementation to check.*

But cases involving unfixed values vary, as we will see. *Rationale: The best thing to do for unfixed values varies from case to case. Also, it is difficult to check constraints on unfixed values, particularly because during search a decision variable might become fixed and then backtracking will cause this value to be reverted, in which case aborting is a bad idea.*

4.1.10.1 Partial Assignments

The first operation involving partiality is assignment. There are four distinct cases for assignments.

- A value assigned to a fixed, constrained global variable is checked at run-time; if the assigned value does not satisfy its constraints, it is a run-time error. In other words, this:

  ```
  1..5: x = 3;
  ```

  is equivalent to this:

  ```
  int: x = 3;
  constraint assert(x in 1..5,
                    "assignment to global parameter 'x' failed")
  ```

- A value assigned to an unfixed, constrained global variable makes the assignment act like a constraint; if the assigned value does not satisfy the variable's constraints, it causes a run-time model failure. In other words, this:

  ```
  var 1..5: x = 3;
  ```

  is equivalent to this:

  ```
  var int: x = 3;
  constraint x in 1..5;
  ```

  Rationale: This behaviour is easy to understand and easy to implement.

- A value assigned to a fixed, constrained let-local variable is checked at run-time; if the assigned value does not satisfy its constraints, it is a run-time error. In other words, this:

  ```
  let { 1..5: x = 3; } in x+1
  ```

  is equivalent to this:

```
let { int: x = 3; } in
    assert(x in 1..5,
           "assignment to let parameter 'x' failed", x+1)
```

- A value assigned to an unfixed, constrained let-local variable makes the assignment act like a constraint; if the constraint fails at run-time, the failure "bubbles up" to the nearest enclosing Boolean scope, where it is interpreted as `false`.

  Rationale: This behaviour is consistent with assignments to global variables.

Note that in cases where a value is partly fixed and partly unfixed, e.g., some arrays, the different elements are checked according to the different cases, and fixed elements are checked before unfixed elements. For example:

```
u = [ let { var 1..5: x = 6} in x, let { par 1..5: y = 6; } in y) ];
```

This causes a run-time abort, because the second, fixed element is checked before the first, unfixed element. This ordering is true for the cases in the following sections as well. *Rationale: This ensures that failures cannot mask aborts, which seems desirable.*

4.1.10.2 Partial Predicate/Function and Annotation Arguments

The second kind of operation involving partiality is calls and annotations.

The semantics is similar to assignments: fixed arguments that fail their constraints will cause aborts, and unfixed arguments that fail their constraints will cause failure, which bubbles up to the nearest enclosing Boolean scope.

4.1.10.3 Partial Array Accesses

The third kind of operation involving partiality is array access. There are two distinct cases.

- A fixed value used as an array index is checked at run-time; if the index value is not in the index set of the array, it is a run-time error.
- An unfixed value used as an array index makes the access act like a constraint; if the access fails at run-time, the failure "bubbles up" to the nearest enclosing Boolean scope, where it is interpreted as `false`. For example:

  ```
  array[1..3] of int: a = [1,2,3];
  var int: i;
  constraint (a[i] + 3) > 10 \/ i = 99;
  ```

  Here the array access fails, so the failure bubbles up to the disjunction, and `i` is constrained to be 99. *Rationale: Unlike predicate/function calls, modellers in practice sometimes do use array accesses that can fail. In such cases, the "bubbling up" behaviour is a reasonable one.*

4.1.11 Built-in Operations

This appendix lists built-in operators, functions and predicates. They may be implemented as true built-ins, or in libraries that are automatically imported for all models. Many of them are overloaded.

Operator names are written within single quotes when used in type signatures, e.g. `bool: '\/'(bool, bool)`.

We use the syntax `TI: f(TI1,...,TIn)` to represent an operation named `f` that takes arguments with type-insts `TI,...,TIn` and returns a value with type-inst `TI`. This is slightly more compact than the usual MiniZinc syntax, in that it omits argument names.

4.1.11.1 Comparison Operations

Less than. Other comparisons are similar: greater than (`>`), less than or equal (`<=`), greater than or equal (`>=`), equality (`==`, `=`), and disequality (`!=`).

```
    bool: '<'(    $T,     $T)
var bool: '<'(var $T, var $T)
```

4.1.11.2 Arithmetic Operations

Addition. Other numeric operations are similar: subtraction (`-`), and multiplication (`*`).

```
    int:   '+'(    int,       int)
var int:   '+'(var int,   var int)
    float: '+'(    float,     float)
var float: '+'(var float, var float)
```

Unary minus. Unary plus (`+`) is similar.

```
    int:   '-'(    int)
var int:   '-'(var int)
    float: '-'(    float)
var float: '-'(var float)
```

Integer and floating-point division and modulo.

```
    int:   'div'(    int,       int)
var int:   'div'(var int,   var int)
    int:   'mod'(    int,       int)
var int:   'mod'(var int,   var int)
    float: '/'  (    float,     float)
var float: '/'  (var float, var float)
```

The result of the modulo operation, if non-zero, always has the same sign as its first operand. The integer division and modulo operations are connected by the following identity:

```
x = (x div y) * y + (x mod y)
```

Some illustrative examples:

```
 7 div  4 =  1        7 mod  4 =  3
-7 div  4 = -1       -7 mod  4 = -3
 7 div -4 = -1        7 mod -4 = 3
-7 div -4 =  1       -7 mod -4 = -3
```

Sum multiple numbers. Product (`product`) is similar. Note that the sum of an empty array is 0, and the product of an empty array is 1.

```
    int:   sum(array[$T]  of     int  )
var int:   sum(array[$T]  of var int  )
    float: sum(array[$T]  of     float)
var float: sum(array[$T]  of var float)
```

Minimum of two values; maximum (`max`) is similar.

```
$T:     min(    $T,     $T)
var $T: min(var $T, var $T)
```

Minimum of an array of values; maximum (`max`) is similar. Aborts if the array is empty.

```
$U:     min(array[$T]  of     $U)
var $U: min(array[$T]  of var $U)
```

Minimum of a fixed set; maximum (`max`) is similar. Aborts if the set is empty.

```
$T:    min(set of $T)
```

Absolute value of a number.

```
    int:   abs(    int)
var int:   abs(var int)
    float: abs(    float)
var float: abs(var float)
```

Square root of a float. Aborts if argument is negative.

```
    float: sqrt(    float)
var float: sqrt(var float)
```

Power operator. E.g. `pow(2, 5)` gives `32`.

```
  int: pow(int,       int)
float: pow(float,     float)
```

Natural exponent.

```
    float: exp(float)
var float: exp(var float)
```

Natural logarithm. Logarithm to base 10 (`log10`) and logarithm to base 2 (`log2`) are similar.

```
    float: ln(float)
var float: ln(var float)
```

General logarithm; the first argument is the base.

```
float: log(float, float)
```

Sine. Cosine (`cos`), tangent (`tan`), inverse sine (`asin`), inverse cosine (`acos`), inverse tangent (`atan`), hyperbolic sine (`sinh`), hyperbolic cosine (`cosh`), hyperbolic tangent (`tanh`), inverse hyperbolic sine (`asinh`), inverse hyperbolic cosine (`acosh`) and inverse hyperbolic tangent (`atanh`) are similar.

```
    float: sin(float)
var float: sin(var float)
```

4.1.11.3 Logical Operations

Conjunction. Other logical operations are similar: disjunction (`\/`) reverse implication (`<-`), forward implication (`->`), bi-implication (`<->`), exclusive disjunction (`xor`), logical negation (`not`).

Note that the implication operators are not written using `=>`, `<=` and `<=>` as is the case in some languages. This allows `<=` to instead represent "less than or equal".

```
    bool: '/\'(    bool,     bool)
var bool: '/\'(var bool, var bool)
```

Universal quantification. Existential quantification (`exists`) is similar. Note that, when applied to an empty list, `forall` returns `true`, and `exists` returns `false`.

```
    bool: forall(array[$T]  of     bool)
var bool: forall(array[$T]  of var bool)
```

N-ary exclusive disjunction. N-ary bi-implication (`iffall`) is similar, with `true` instead of `false`.

```
    bool: xorall(array[$T]  of     bool: bs) = foldl('xor', false, bs)
var bool: xorall(array[$T]  of var bool: bs) = foldl('xor', false, bs)
```

4.1.11.4 Set Operations

Set membership.

```
    bool: 'in'(     $T,       set of $T )
var bool: 'in'(var $$E,   var set of $$E)
```

Non-strict subset. Non-strict superset (`superset`) is similar.

```
    bool: 'subset'(    set of $T ,     set of $T )
var bool: 'subset'(var set of $$E, var set of $$E)
```

Set union. Other set operations are similar: intersection (`intersect`), difference (`diff`), symmetric difference (`symdiff`).

```
    set of  $T: 'union'(    set of  $T,     set of  $T )
var set of $$E: 'union'(var set of $$E, var set of $$E )
```

Set ranges. Constructs a set given a lower and an upper bound. Whether the operator constructs a set that includes or excludes the bounds depends on the placement of the < symbol in the operator. If the first argument is larger than the second (e.g. `1..0`), it returns the empty set. The arguments to the operators can be either integers or values of enumerated types (`l+1` and `u-1` in the comments mean `enum_next(l)` and `enum_prev(l)` in that case).

```
% Return set {l, ..., u}
set of $$E: '..'($$E: l, $$E: u)
% Return set {l+1, ..., u}
set of $$E: '<..'($$E: l, $$E: u)
% Return set {l, ..., u-1}
set of $$E: '..<'($$E: l, $$E: u)
% Return set {l+1, ..., u-1}
set of $$E: '<..<'($$E: l, $$E: u)
```

Note that set range operators can be used with only one of the bounds given. The minimum or maximum of the type of the bound is then used for the other bound:

```
% x = {l, ..., max(enum_of(l))}
set of X: x = l..;
% x = {min(enum_of(u)), ..., u}
set of X: x = ..u;
```

Cardinality of a set.

```
    int: card(    set of  $T)
var int: card(var set of $$E)
```

Union of an array of sets. Intersection of multiple sets (`array_intersect`) is similar.

```
    set of  $U:    array_union(array[$T]  of     set of  $U)
var set of $$E:    array_union(array[$T]  of var set of $$E)
```

4.1.11.5 Array Operations

Length of an array.

```
int: length(array[$T] of $U)
```

List concatenation. Returns the list (integer-indexed array) containing all elements of the first argument followed by all elements of the second argument, with elements occurring in the same order as in the arguments. The resulting indices are in the range `1..n`, where `n` is the sum of the lengths of the arguments. *Rationale: This allows list-like arrays to be concatenated naturally and avoids problems with overlapping indices. The resulting indices are consistent with those of implicitly indexed array literals.* Note that `'++'` also performs string concatenation.

```
array[int] of $T: '++'(array[int] of $T, array[int] of $T)
```

Index sets of arrays. If the argument is a literal, returns `1..n` where `n` is the (sub-)array length. Otherwise, returns the declared or inferred index set. This list is only partial, it extends in the obvious way, for arrays of higher dimensions.

```
set of $T:  index_set     (array[$T]      of $V)
set of $T:  index_set_1of2(array[$T, $U]  of $V)
set of $U:  index_set_2of2(array[$T, $U]  of $V)
...
```

Replace the indices of the array given by the last argument with the Cartesian product of the sets given by the previous arguments. Similar versions exist for arrays up to 6 dimensions.

```
array[$T1] of $V: array1d(set of $T1, array[$U] of $V)
array[$T1,$T2] of $V:
    array2d(set of $T1, set of $T2, array[$U] of $V)
array[$T1,$T2,$T3] of $V:
    array3d(set of $T1, set of $T2, set of $T3, array[$U] of $V)
```

4.1.11.6 Coercion Operations

Round a float towards $+\infty$, $-\infty$, and the nearest integer, respectively.

```
int: ceil (float)
int: floor(float)
int: round(float)
```

Explicit casts from one type-inst to another.

```
    int:          bool2int(    bool)
var int:          bool2int(var bool)
    float:        int2float(    int)
var float:        int2float(var int)
array[int] of $T: set2array(set of $T)
```

Capturing of undefinedness and optionality. These binary infix operators return `x` if it is both defined and not absent, and `y` otherwise.

```
                    $T: 'default'(opt $T:x, $T: y);
                opt $T: 'default'(opt $T:x, opt $T: y);
                var $T: 'default'(var opt $T:x, var $T: y);
            var opt $T: 'default'(var opt $T:x, var opt $T: y);
array[$U] of        $T: 'default'(array[$U] of $T:x, array[$U] of $T: y);
array[$U] of opt    $T: 'default'(array[$U] of opt $T:x, array[$U] of opt
↪$T: y);
array[$U] of var    $T: 'default'(array[$U] of var $T:x, array[$U] of var
↪$T: y);
array[$U] of var opt $T: 'default'(array[$U] of var opt $T:x, array[$U] of␣
↪var opt $T: y);
```

4.1.11.7 String Operations

To-string conversion. Converts any value to a string for output purposes. The exact form of the resulting string is implementation-dependent.

```
string: show($T)
```

Formatted to-string conversion for integers. Converts the integer given by the second argument into a string right justified by the number of characters given by the first argument, or left justified if that argument is negative. If the second argument is not fixed, the form of the string is implementation-dependent.

```
string: show_int(int, var int);
```

Formatted to-string conversion for floats. Converts the float given by the third argument into a string right justified by the number of characters given by the first argument, or left justified if that argument is negative. The number of digits to appear after the

decimal point is given by the second argument. It is a run-time error for the second argument to be negative. If the third argument is not fixed, the form of the string is implementation-dependent.

```
string: show_float(int, int, var float)
```

String concatenation. Note that `'++'` also performs array concatenation.

```
string: '++'(string, string)
```

Concatenate an array of strings. Equivalent to folding `'++'` over the array, but may be implemented more efficiently.

```
string: concat(array[$T] of string)
```

Concatenate an array of strings, putting a separator between adjacent strings. Returns the the empty string if the array is empty.

```
string: join(string, array[$T] of string)
```

4.1.11.8 Bound and Domain Operations

The bound operations `lb` and `ub` return fixed, correct lower/upper bounds to the expression. For numeric types, they return a lower/upper bound value, e.g. the lowest/highest value the expression can take. For set types, they return a subset/superset, e.g. the intersection/union of all possible values of the set expression.

The bound operations abort on expressions that have no corresponding finite bound. For example, this would be the case for a variable declared without bounds in an implementation that does not assign default bounds. (Set expressions always have a finite lower bound of course, namely `{}`, the empty set.)

Numeric lower/upper bound:

```
int:   lb(var int)
float: lb(var float)
int:   ub(var int)
float: ub(var float)
```

Set lower/upper bound:

```
set of int: lb(var set of int)
set of int: ub(var set of int)
```

Versions of the bound operations that operate on arrays are also available, they return a safe lower bound or upper bound for all members of the array - they abort if the array is empty:

```
int:        lb_array(array[$T] of var int)
float:      lb_array(array[$T] of var float)
set of int: lb_array(array[$T] of var set of int)
int:        ub_array(array[$T] of var int)
float:      ub_array(array[$T] of var float)
set of int: ub_array(array[$T] of var set of int)
```

Integer domain:

```
set of int: dom(var int)
```

The domain operation `dom` returns a fixed superset of the possible values of the expression.

Integer array domain, returns a superset of all possible values that may appear in the array - this aborts if the array is empty:

```
set of int: dom_array(array[$T] of var int)
```

Domain size for integers:

```
int: dom_size(var int)
```

The domain size operation `dom_size` is equivalent to `card(dom(x))`.

Note that these operations can produce different results depending on when they are evaluated and what form the argument takes. For example, consider the numeric lower bound operation.

- If the argument is a fixed expression, the result is the argument's value.
- If the argument is a decision variable, then the result depends on the context.
  - If the implementation can determine a lower bound for the variable, the result is that lower bound. The lower bound may be from the variable's declaration, or higher than that due to preprocessing, or lower than that if an implementation-defined lower bound is applied (e.g. if the variable was declared with no lower bound, but the implementation imposes a lowest possible bound).
  - If the implementation cannot determine a lower bound for the variable, the operation aborts.
- If the argument is any other kind of unfixed expression, the lower bound depends on the bounds of unfixed subexpressions and the connecting operators.

4.1.11.9 Option Type Operations

The option type value ($\top$) is written

```
opt $T:  '<>';
```

One can determine if an option type variable actually occurs or not using `occurs` and `absent`

```
par bool: occurs(par opt $T);
var bool: occurs(var opt $T);
par bool: absent(par opt $T);
var bool: absent(var opt $T);
```

One can return the non-optional value of an option type variable using the function `deopt`

```
par $T: deopt(par opt $T);
var $T: deopt(var opt $T);
```

4.1.11.10 Other Operations

Check a Boolean expression is true, and abort if not, printing the second argument as the error message. The first one returns the third argument, and is particularly useful for sanity-checking arguments to predicates and functions; importantly, its third argument is lazy, i.e. it is only evaluated if the condition succeeds. The second one returns `true` and is useful for global sanity-checks (e.g. of instance data) in constraint items.

```
$T:   assert(bool, string, s$T)
par bool: assert(bool, string)
```

Abort evaluation, printing the given string.

```
$T: abort(string)
```

Return true. As a side-effect, an implementation may print the first argument.

```
bool: trace(string)
```

Return the second argument. As a side-effect, an implementation may print the first argument.

```
$T: trace(string, $T)
```

Check if the argument's value is fixed at this point in evaluation. If not, abort; if so, return its value. This is most useful in output items when decision variables should be fixed: it allows them to be used in places where a fixed value is needed, such as if-then-else conditions.

```
$T: fix(var $T)
```

As above, but return `false` if the argument's value is not fixed.

```
par bool: is_fixed(var $T)
```

4.1.12 Content-types

The content-type `application/x-zinc-output` defines a text output format for Zinc. The format extends the abstract syntax and semantics given in *Run-time Outcomes* (page 274), and is discussed in detail in *Output* (page 274).

The full syntax is as follows:

```
% Output
<output> ::= <no-solutions> [ <warnings> ] <free-text>
           | ( <solution> )* [ <complete> ] [ <warnings> ] <free-text>

% Solutions
<solution> ::= <solution-text> [ \n ] "----------" \n

% Unsatisfiable
<no-solutions> ::= "=====UNSATISFIABLE=====" \n

% Complete
<complete> ::= "==========" \n

% Messages
<warnings> ::= ( <message> )+

<message>  ::= ( <line> )+
<line>     ::= "%" [^\n]* \n
```

The solution text for each solution must be as described in *Output Items* (page 315). A newline must be appended if the solution text does not end with a newline.

4.1.13 JSON support

MiniZinc can support reading input parameters and providing output formatted as JSON objects. A JSON input file needs to have the following structure:

- Consist of a single top-level object
- The members of the object (the key-value pairs) represent model parameters
- Each member key must be a valid MiniZinc identifier (and it supplies the value for the corresponding parameter of the model)
- Each member value can be one of the following:
  - A string (assigned to a MiniZinc string parameter)
  - A number (assigned to a MiniZinc int or float parameter)
  - The values `true` or `false` (assigned to a MiniZinc bool parameter)

  - An array of values. Arrays of arrays are supported only if all inner arrays are of the same length, so that they can be mapped to multi-dimensional MiniZinc arrays.
  - A set of values encoded as an object with a single member with key `"set"` and a list of values (the elements of the set). The list may additionally contain an element which is a sublist comprising a pair of values representing a minimum and maximum of a range in the set.
  - A value of an enumerated type encoded as an object with a single member with key `"e"` and a string value (the identifier of the enumerated value).
  - The value of an enumerated type constructor call encoded as an object with a member with key `"c"` and a string value (the identifier of the constructor call), and a member with key `"e"` and a enumerated type (or constructor) object value (the argument to the call).
  - A list of enum constructors used to define the members of an enumerated type. Each constructor is either a string value for an atomic enum value, or an object with members `"c"` and `"e"` as above for a constructor call.

MiniZinc supports coercion of some JSON input types for more convenient input:

- A list of values can be coerced to a set
- A string value can be coerced to an enumerated type value (but not a constructor call)

Assume a MiniZinc model declaring the following parameters:

```
int: n;
enum Customers;
array[1..n,Customers] of int: distances;
array[1..n] of set of int: patterns;
```

Here is an example of a JSON parameter file using all of the above features:

```
{
  "n" : 2,
  "Customers" : [ {"e" : "Customer A"}, {"e" : "Customer B"}, {"e" :
↪"Customer C"} ],
  "distances" : [ [1,2,3],
                  [4,5,6]],
  "patterns"  : [ {"set" : [1,3,5]}, {"set" : [2,4,6]} ]
}
```

The first parameter declares a simple integer `n`. The `Customers` parameter defines the members of an enumerated type. The `distances` parameter is a two-dimensional array; note that all inner arrays must be of the same size in order to map to a (rectangular) MiniZinc two-dimensional array. The `patterns` parameter is an array of sets of integers.

4.1.14 Full grammar

4.1.14.1 Items

```
% A MiniZinc model
<model> ::= [ <item> ";" ... ]

% Items
<item>  ::= <include-item>
          | <var-decl-item>
          | <enum-item>
          | <type-inst-syn-item>
          | <assign-item>
          | <constraint-item>
          | <solve-item>
          | <output-item>
          | <predicate-item>
          | <test-item>
          | <function-item>
          | <annotation-item>

<ti-expr-and-id> ::= <ti-expr> ":" <ident>

% Include items
<include-item> ::= "include" <string-literal>

% Variable declaration items
<var-decl-item> ::= <ti-expr-and-id> <annotations> [ "=" <expr> ]
                  | "any" ":" <ident> <annotations> [ "=" <expr> ]

% Enum items
<enum-item>  ::= "enum" <ident> <annotations> [ "=" <enum-cases-list> ]

<enum-cases-list> ::= <enum-cases>
                    | <enum-cases-list> "++" <enum-cases>

<enum-cases> ::= "{" <ident> "," ... "}"
               | "_" "(" <expr> ")"
               | <ident> "(" <expr> ")"

% Type-inst synonym items
<type-inst-syn-item> ::= "type" <ident> <annotations> "=" <ti-expr>

% Assign items
<assign-item> ::= <ident> "=" <expr>

% Constraint items
<constraint-item> ::= "constraint" [ <string-annotation> ] <expr>

% Solve item
```

```
<solve-item> ::= "solve" <annotations> "satisfy"
               | "solve" <annotations> "minimize" <expr>
               | "solve" <annotations> "maximize" <expr>

% Output items
<output-item> ::= "output" [ "::" <output-annotation> ] <expr>

<output-annotation> ::= <string-literal>
                      | <ident> "(" [ <expr> "," ... ] ")"
                      | "(" <expr> ")"

% Annotation items
<annotation-item> ::= "annotation" <ident> <params> [ "=" <expr> ]

% Predicate, test and function items
<predicate-item> ::= "predicate" <operation-item-tail>

<test-item> ::= "test" <operation-item-tail>

<function-item> ::= "function" <ti-expr> ":" <operation-item-tail>

<operation-item-tail> ::= <ident> <params> <annotations> [ "=" <expr> ]

<params> ::= [ ( <param-ti-expr-and-id> [ "=" <expr> ] "," ... ) ]

% Parameter type-insts: same as <ti-expr>, but the set-cardinality form
% (`set "(" <expr> ")" of ...`) and the index-dependent array form
% (`<ident> "in" <ti-expr>` in the index list) are not allowed.
<param-ti-expr-and-id> ::= <param-ti-expr> ":" <ident>

<param-ti-expr> ::= <param-base-ti-expr>
                  | <param-array-ti-expr>
                  | <param-ti-expr> "++" <param-base-ti-expr>

<param-base-ti-expr> ::= <var-par> <opt-ti> <param-set-ti>
↪<param-base-ti-expr-tail>
                       | "any" <ti-variable-expr-tail>

<param-set-ti> ::= "set" "of" |

<param-array-ti-expr> ::= "array" "[" <ti-expr> "," ... "]" "of"
↪<param-base-ti-expr>
                        | "list" "of" <param-base-ti-expr>

<param-base-ti-expr-tail> ::= <ident>
                            | <base-type>
                            | <ti-variable-expr-tail>
                            | "tuple" "(" <param-ti-expr> "," ... ")"
                            | "record" "(" <param-ti-expr-and-id> "," ... ")"
                            | "ann"
```

```
                                | <expr>
```

4.1.14.2 Type-Inst Expressions

```
<ti-expr> ::= <base-ti-expr>
            | <array-ti-expr>
            | <ti-expr> "++" <base-ti-expr>

<base-ti-expr> ::= <var-par> <opt-ti> <set-ti> <base-ti-expr-tail>
                 | "any" <ti-variable-expr-tail>

<var-par> ::= "var" | "par" |

<opt-ti> ::= "opt" |

<set-ti> ::= "set" [ "(" <expr> ")" ] "of" |

<base-type> ::= "bool"
              | "int"
              | "float"
              | "string"

<base-ti-expr-tail> ::= <ident>
                      | <base-type>
                      | <ti-variable-expr-tail>
                      | <tuple-ti-expr-tail>
                      | <record-ti-expr-tail>
                      | "ann"
                      | <expr>

% Type-inst variables
<ti-variable-expr-tail> ::= $[A-Za-z$][A-Za-z0-9_]*

% Array type-inst expressions
<array-ti-expr> ::= "array" "[" <array-index> "," ... "]" "of" <base-ti-expr>
                  | "list" "of" <base-ti-expr>

<array-index> ::= <ti-expr>
                | <ident> "in" <ti-expr>

% Tuple type-inst expressions
<tuple-ti-expr-tail> ::= "tuple" ( <ti-expr> "," ... )

% Record type-inst expressions
<record-ti-expr-tail> ::= "record" ( <ti-expr-and-id> "," ... )
```

4.1.14.3 Expressions

```
<expr> ::= <expr-atom> <expr-binop-tail>

<expr-atom> ::= <expr-atom-head> <expr-atom-tail> <annotations>

<expr-binop-tail> ::= [ <bin-op> <expr> ]

<expr-atom-head> ::= <builtin-un-op> <expr-atom>
                   | "(" <expr> ")"
                   | <ident-or-quoted-op>
                   | "_"
                   | <bool-literal>
                   | <int-literal>
                   | <float-literal>
                   | <string-literal>
                   | <set-literal>
                   | <set-comp>
                   | <array-literal>
                   | <array-literal-2d>
                   | <indexed-array-literal>
                   | <indexed-array-literal-2d>
                   | <tuple-literal>
                   | <record-literal>
                   | <array-comp>
                   | <indexed-array-comp>
                   | <ann-literal>
                   | <if-then-else-expr>
                   | <let-expr>
                   | <call-expr>
                   | <gen-call-expr>

<expr-atom-tail> ::=
                   | <array-access-tail> <expr-atom-tail>
                   | <field-access-tail> <expr-atom-tail>

% Numeric expressions
<num-expr> ::= <num-expr-atom> <num-expr-binop-tail>

<num-expr-atom> ::= <num-expr-atom-head> <expr-atom-tail> <annotations>

<num-expr-binop-tail> ::= [ <num-bin-op> <num-expr> ]

<num-expr-atom-head> ::= <builtin-num-un-op> <num-expr-atom>
                       | "(" <num-expr> ")"
                       | <ident-or-quoted-op>
                       | <int-literal>
                       | <float-literal>
                       | <if-then-else-expr>
                       | <let-expr>
```

```
                      | <call-expr>
                      | <gen-call-expr>

% Built-in operators
<builtin-op> ::= <builtin-bin-op> | <builtin-un-op>

<bin-op> ::= <builtin-bin-op> | `<ident>`

<builtin-bin-op> ::= "<->" | "->" | "<-" | "\/" | "xor" | "/\"
                   | "<" | ">" | "<=" | ">=" | "==" | "=" | "!="
                   | "~=" | "~!="
                   | "in" | "subset" | "superset" | "union" | "diff" |
↪"symdiff"
                   | ".." | "intersect" | "++" | "default" |
↪<builtin-num-bin-op>

<builtin-un-op> ::= "not" | <builtin-num-un-op>

% Built-in numeric operators
<num-bin-op> ::= <builtin-num-bin-op> | `<ident>`

<builtin-num-bin-op> ::= "+" | "-" | "*" | "/" | "div" | "mod" | "^"
                       | "~+" | "~-" | "~*" | "~/" | "~div"

<builtin-num-un-op> ::= "+" | "-"

% Boolean literals
<bool-literal> ::= "false" | "true"

% Integer literals
<int-literal> ::= [0-9]+
                | 0x[0-9A-Fa-f]+
                | 0o[0-7]+

% Float literals
<float-literal> ::= [0-9]+"."[0-9]+
                  | [0-9]+"."[0-9]+[Ee][-+]?[0-9]+
                  | [0-9]+[Ee][-+]?[0-9]+
                  | 0[xX]([0-9a-fA-F]*"."[0-9a-fA-F]+|[0-9a-fA-F]+".
↪")([pP][+-]?[0-9]+)
                  | (0[xX][0-9a-fA-F]+[pP][+-]?[0-9]+)

% String literals
<string-contents> ::= ([^"\n\] |  \[0-7][0-7]?[0-7]? | \x[0-9a-fA-F][0-9a-fA-
↪F]? | \n | \t | \" | \\)*

<string-literal> ::= """ <string-contents> """
                   | """ <string-contents> "\(" <string-interpolate-tail>

<string-interpolate-tail> ::= <expr> ")"<string-contents>"""
```

```
                            | <expr> ")"<string-contents>"\("
↪<string-interpolate-tail>

% Set literals
<set-literal> ::= "{" [ <expr> "," ... ] "}"

% Set comprehensions
<set-comp> ::= "{" <expr> "|" <comp-tail> "}"

<comp-tail> ::= <generator> [ "where" <expr> ] "," ...

<generator> ::= ( <ident> | "_" ) "," ... "in" <expr>
              | ( <ident> | "_" ) "=" <expr>

% Array literals
<array-literal> ::= "[" [ <expr> "," ... ] "]"

% 2D Array literals
<array-literal-2d> ::= "[|" [ [ <expr> "," ... ] "|" ... ] "|]"

% Indexed 2D Array literals
<indexed-array-literal-2d> ::= "[|" [ <expr> ":" ... ] [ [ <expr> ":" ]
↪<expr> "," ... ] "|]"

% Indexed array literals
<indexed-array-literal> ::= "[" [ ( <index-tuple> ":" <expr> ) "," ... ] "]"
                          | "[" [ <index-tuple> ":" <expr> "," <expr> "," ...
↪ ] "]"

<index-tuple> ::= <expr>
                | "(" [ <expr> "," ... ] ")"

% Tuple literals
<tuple-literal> ::= "(" <expr> "," [ <expr> "," ... ] ")"

% Record literals
<record-literal> ::= "(" <ident> ":" <expr> "," [ <ident> ":" <expr> "," ...␣
↪] ")"

% Array comprehensions
<array-comp> ::= "[" <expr> "|" <comp-tail> "]"

% Indexed array comprehensions
<indexed-array-comp> ::= "[" <index-tuple> ":" <expr> "|" <comp-tail> "]"

% Array access
<array-access-tail> ::= "[" <expr> "," ... "]"

% Field access
<field-access-tail> ::= "."<ident> | "."<int-literal>
```

```
% Annotation literals
<ann-literal> ::= <ident> [ "(" <expr> "," ... ")" ]

% If-then-else expressions
<if-then-else-expr> ::= "if" <expr> "then" <expr> [ "elseif" <expr> "then"
↪<expr> ]* [ "else" <expr> ] "endif"

% Call expressions
<call-expr> ::= <ident-or-quoted-op> [ "(" <call-arg> "," ... ")" ]

<call-arg> ::= <expr> | <ident> ":" <expr>

% Let expressions
<let-expr> ::= "let" "{" <let-item> ( ";" | "," ) ... "}" "in" <expr>

<let-item> ::= <var-decl-item>
             | <constraint-item>

% Generator call expressions
<gen-call-expr> ::= <ident-or-quoted-op> "(" <comp-tail> ")" "(" <expr> ")"
```

4.1.14.4 Miscellaneous Elements

```
% Identifiers
<ident> ::= _?[A-Za-z][A-Za-z0-9_]* | '[^'\xa\xd\x0]+'

% Identifiers and quoted operators
<ident-or-quoted-op> ::= <ident>
                       | '<builtin-op>'

% Annotations
<annotations> ::= [ "::" <annotation> ]*

<annotation> ::= <expr-atom-head> <expr-atom-tail>

<string-annotation> ::= "::" <string-literal>
```

4.2.1 Standard Library

These functions and predicates define built-in operations of the MiniZinc language.

4.2.1.1 Built-in functions and operators

These functions and operators provide the core of the MiniZinc expression language.

Comparison Builtins

These builtins implement comparison operations.

```
1.  test '!='($T: x, $T: y)

2.  test '!='(opt $T: x, opt $T: y)

3.  predicate '!='(any $T: x, any $T: y)

4.  test '!='(array [$U] of $T: x, array [$U] of $T: y)

5.  test '!='(array [$U] of opt $T: x, array [$U] of opt $T: y)

6.  predicate '!='(array [$U] of var $T: x, array [$U] of var $T: y)

7.  predicate '!='(array [$U] of var opt $T: x,
                   array [$U] of var opt $T: y)
```

1-3.

Return if `x` is not equal to `y`

Usage: `x != y`

4-7.
Return if array `x` is not equal to array `y`

Usage: `x != y`

```
1.  test '<'($T: x, $T: y)
2.  predicate '<'(var $T: x, var $T: y)
3.  predicate '<'($T: x, var $T: y)
4.  predicate '<'(var $T: x, $T: y)
5.  predicate '<'(var opt $$E: x, var opt $$E: y)
6.  test '<'(opt $$E: x, opt $$E: y)
7.  predicate '<'(var opt float: x, var opt float: y)
8.  test '<'(opt float: x, opt float: y)
9.  test '<'(array [$U] of $T: x, array [$U] of $T: y)
10. predicate '<'(array [$U] of var int: x, array [$U] of var int: y)
11. predicate '<'(array [$U] of var bool: x, array [$U] of var bool: y)
12. predicate '<'(array [$U] of var float: x, array [$U] of var float: y)
13. predicate '<'(array [$U] of var set of $$E: x,
                  array [$U] of var set of $$E: y)
```

1-4.
Return if `x` is less than `y`

Usage: `x < y`

5-8.
Weak comparison: true iff either `x` or `y` is absent, or both occur and the value of `x` is less than the value of `y`.

Usage: `x < y`

9-13.
Return if array `x` is lexicographically smaller than array `y`

Usage: `x < y`

```
1.  test '<='($T: x, $T: y)
```

```
 2.  predicate '<='(var $T: x, var $T: y)

 3.  predicate '<='($T: x, var $T: y)

 4.  predicate '<='(var $T: x, $T: y)

 5.  predicate '<='(var opt $$E: x, var opt $$E: y)

 6.  test '<='(opt $$E: x, opt $$E: y)

 7.  predicate '<='(var opt float: x, var opt float: y)

 8.  test '<='(opt float: x, opt float: y)

 9.  test '<='(array [$U] of $T: x, array [$U] of $T: y)

10.  predicate '<='(array [$U] of var int: x, array [$U] of var int: y)

11.  predicate '<='(array [$U] of var bool: x, array [$U] of var bool: y)

12.  predicate '<='(array [$U] of var float: x, array [$U] of var float: y)

13.  predicate '<='(array [$U] of var set of $$E: x,
                    array [$U] of var set of $$E: y)
```

1-4.
Return if `x` is less than or equal to `y`

Usage: `x <= y`

5-8.
Weak comparison: true iff either `x` or `y` is absent, or both occur and the value of `x` is less than or equal to the value of `y`.

Usage: `x <= y`

9-12.
Return if array `x` is lexicographically smaller than or equal to array `y`

Usage: `x <= y`

13. Return if array `x` is lexicographically $$E than or equal to array `y`

    Usage: `x <= y`

```
1.  test '='($T: x, $T: y)

2.  test '='(opt $T: x, opt $T: y)

3.  predicate '='(any $T: x, any $T: y)

4.  test '='(array [$U] of $T: x, array [$U] of $T: y)
```

```
5.  test '='(array [$U] of opt $T: x, array [$U] of opt $T: y)
6.  predicate '='(array [$U] of var $T: x, array [$U] of var $T: y)
7.  predicate '='(array [$U] of var opt $T: x, array [$U] of var opt $T: y)
```

1-3.
Return if `x` is equal to `y`

Usage: `x = y`

4-7.
Return if array `x` is equal to array `y`

Usage: `x = y`

```
1.  test '>'($T: x, $T: y)
2.  predicate '>'(var $T: x, var $T: y)
3.  predicate '>'($T: x, var $T: y)
4.  predicate '>'(var $T: x, $T: y)
5.  predicate '>'(var opt $$E: x, var opt $$E: y)
6.  test '>'(opt $$E: x, opt $$E: y)
7.  predicate '>'(var opt float: x, var opt float: y)
8.  test '>'(opt float: x, opt float: y)
9.  test '>'(array [$U] of $T: x, array [$U] of $T: y)
10. predicate '>'(array [$U] of var int: x, array [$U] of var int: y)
11. predicate '>'(array [$U] of var bool: x, array [$U] of var bool: y)
12. predicate '>'(array [$U] of var float: x, array [$U] of var float: y)
13. predicate '>'(array [$U] of var set of $$E: x,
                  array [$U] of var set of $$E: y)
```

1-4.
Return if `x` is greater than `y`

Usage: `x > y`

5-8.
Weak comparison: true iff either `x` or `y` is absent, or both occur and the value of `x`

is greater than the value of `y`.

Usage: `x > y`

9-13.

Return if array `x` is lexicographically greater than array `y`

Usage: `x > y`

```
1.  test '>='($T: x, $T: y)
2.  predicate '>='(var $T: x, var $T: y)
3.  predicate '>='($T: x, var $T: y)
4.  predicate '>='(var $T: x, $T: y)
5.  predicate '>='(var opt $$E: x, var opt $$E: y)
6.  test '>='(opt $$E: x, opt $$E: y)
7.  predicate '>='(var opt float: x, var opt float: y)
8.  test '>='(opt float: x, opt float: y)
9.  test '>='(array [$U] of $T: x, array [$U] of $T: y)
10. predicate '>='(array [$U] of var int: x, array [$U] of var int: y)
11. predicate '>='(array [$U] of var bool: x, array [$U] of var bool: y)
12. predicate '>='(array [$U] of var float: x, array [$U] of var float: y)
13. predicate '>='(array [$U] of var set of $$E: x,
                   array [$U] of var set of $$E: y)
```

1-4.

Return if `x` is greater than or equal to `y`

Usage: `x >= y`

5-8.

Weak comparison: true iff either `x` or `y` is absent, or both occur and the value of `x` is greater than or equal to the value of `y`.

Usage: `x >= y`

9-13.

Return if array `x` is lexicographically greater than or equal to array `y`

Usage: `x >= y`

```
predicate '~!='(var opt bool: x, var opt bool: y)
predicate '~!='(var opt $$E: x, var opt $$E: y)
predicate '~!='(var opt float: x, var opt float: y)
```

Weak disequality. True if either `x` or `y` are absent, or present and not equal.

```
predicate '~='(var opt bool: x, var opt bool: y)
predicate '~='(var opt $$E: x, var opt $$E: y)
predicate '~='(var opt float: x, var opt float: y)
```

Weak equality. True if either `x` or `y` are absent, or present and equal.

Arithmetic Builtins

These builtins implement arithmetic operations.

```
1.  function int: '*'(int: x, int: y)

2.  function var int: '*'(var int: x, var int: y)

3.  function var int: '*'(var opt int: x, var opt int: y)

4.  function int: '*'(opt int: x, opt int: y)

5.  function var float: '*'(var opt float: x, var opt float: y)

6.  function float: '*'(opt float: x, opt float: y)

7.  function float: '*'(float: x, float: y)

8.  function var float: '*'(var float: x, var float: y)
```

1, 2.

Return `x * y`

Usage: `x * y`

3-6.

Optional multiplication. Return product of `x` and `y`, with absent replaced by 1.

Usage: `x * y`

7, 8.

Return `x * y`

Usage: `x * y`

```
1.  function int: '+'(int: x, int: y)
2.  function var int: '+'(var int: x, var int: y)
3.  function float: '+'(float: x, float: y)
4.  function var float: '+'(var float: x, var float: y)
5.  function var int: '+'(var opt int: x, var opt int: y)
6.  function int: '+'(opt int: x, opt int: y)
7.  function var float: '+'(var opt float: x, var opt float: y)
8.  function float: '+'(opt float: x, opt float: y)
```

1-4.

Return `x + y`

Usage: `x + y`

5-8.

Optional addition. Return sum of `x` and `y`, with absent replaced by 0.

Usage: `x + y`

```
1.  function int: '-'(int: x, int: y)
2.  function var int: '-'(var int: x, var int: y)
3.  function float: '-'(float: x, float: y)
4.  function var float: '-'(var float: x, var float: y)
5.  function var opt int: '-'(var opt int: x, var opt int: y)
6.  function var opt float: '-'(var opt float: x, var opt float: y)
7.  function opt int: '-'(opt int: x, opt int: y)
8.  function opt float: '-'(opt float: x, opt float: y)
9.  function int: '-'(int: x)
10. function var int: '-'(var int: x)
11. function float: '-'(float: x)
12. function var float: '-'(var float: x)
```

```
13.  function opt int: '-'(opt int: x)

14.  function var opt int: '-'(var opt int: x)

15.  function opt float: '-'(opt float: x)

16.  function var opt float: '-'(var opt float: x)
```

1-4.
Return `x - y`

Usage: `x - y`

5-8.
Optional subtraction. Return absent if `x` is absent, `x` if `y` is absent, difference of `x` and `y` if both are present.

Usage: `x - y`

9-12.
Return negative `x`

Usage: `- x`

13-16.
Return negative `x` if it occurs, otherwise return absent

Usage: `- x`

```
1.  function float: '/'(float: x, float: y)

2.  function var float: '/'(var float: x, var float: y)

3.  function var opt float: '/'(var opt float: x, var opt float: y)

4.  function opt float: '/'(opt float: x, opt float: y)
```

1, 2.
Return result of floating point division `x / y`

Usage: `x / y`

3, 4.
Optional division. Return absent if `x` is absent, `x` if `y` is absent, `x` divided by `y` if both are present.

Usage: `x / y`

```
function int: '^'(int: x, int: y)
function var int: '^'(var int: x, var int: y)
function float: '^'(float: x, float: y)
```

```
function var float: '^'(var float: x, var float: y)
```

Return $\mathbf{x}^{\mathbf{y}}$

Usage: `x ^ y`

```
1.  function int: 'div'(int: x, int: y)
2.  function var int: 'div'(var int: x, var int: y)
3.  function var opt int: 'div'(var opt int: x, var opt int: y)
4.  function opt int: 'div'(opt int: x, opt int: y)
```

1, 2.
Return result of integer division `x / y`

Usage: `x div y`

3, 4.
Optional division. Return absent if `x` is absent, `x` if `y` is absent, `x` divided by `y` if both are present.

Usage: `x div y`

```
1.  function int: 'mod'(int: x, int: y)
2.  function var int: 'mod'(var int: x, var int: y)
3.  function var opt int: 'mod'(var opt int: x, var opt int: y)
4.  function opt int: 'mod'(opt int: x, opt int: y)
```

1, 2.
Return remainder of integer division `x % y`

Usage: `x mod y`

3, 4.
Optional modulo. Return absent if `x` or `y` is absent, `x` modulo `y` if both are present.

Usage: `x mod y`

```
1.  function int: abs(int: x)
2.  function var int: abs(var int: x)
3.  function opt int: abs(opt int: x)
```

```
4.  function var opt int: abs(var opt int: x)
5.  function float: abs(float: x)
6.  function var float: abs(var float: x)
7.  function opt float: abs(opt float: x)
8.  function var opt float: abs(var opt float: x)
```

1, 2.
Return absolute value of `x`

3, 4.
Return absolute value of `x` if it occurs, otherwise return absent

5, 6.
Return absolute value of `x`

7, 8.
Return absolute value of `x` if it occurs, otherwise return absent

---

```
function $$E: arg_max(array [$$E] of bool: xs)
function $$E: arg_max(array [$$E] of $$T: xs)
function $$E: arg_max(array [$$E] of float: xs)
```

Returns the index of the maximum value in the array `xs`. When breaking ties the least index is returned.

```
function $$E: arg_min(array [$$E] of bool: xs)
function $$E: arg_min(array [$$E] of $$T: xs)
function $$E: arg_min(array [$$E] of float: xs)
```

Returns the index of the minimum value in the array `xs`. When breaking ties the least index is returned.

```
function int: count(array [$T] of bool: xs)
function var int: count(array [$T] of var bool: xs)
function var int: count(array [$T] of var opt bool: xs)
```

Return number of true elements in array `xs`

```
1.  function $T: max($T: x, $T: y)
2.  function var float: max(var float: x, var float: y)
3.  function $T: max(array [$U] of $T: x)
4.  function var $$T: max(var $$T: x, var $$T: y)
5.  function $$T: max(array [$X] of opt $$T: x)
6.  function float: max(array [$X] of opt float: x)
7.  function var $$T: max(array [$U] of var $$T: x)
8.  function var float: max(array [$U] of var float: x)
9.  function var $$T: max(array [$X] of var opt $$T: x)
10. function var float: max(array [$X] of var opt float: x)
11. function $$E: max(set of $$E: s)
```

1, 2.
Return maximum of `x` and `y`

3. Return maximum of elements in array `x`
4. Return maximum of `x` and `y`

5, 6.
Return maximum of elements in `x` that are not absent. Result is undefined when all values are absent.

7, 8.
Return maximum of elements in array `x`

9, 10.
Return maximum of elements in `x` that are not absent. Result is undefined when all values are absent.

11. Return maximum of elements in set `s`

```
function opt $$T: max_weak(array [$X] of opt $$T: x)
function opt float: max_weak(array [$X] of opt float: x)
function var opt $$T: max_weak(array [$X] of var opt $$T: x)
function var opt float: max_weak(array [$X] of var opt float: x)
```

Return maximum of elements in `x` that are not absent. Result is absent when all values are absent.

```
1.  function $T: min($T: x, $T: y)
2.  function var $$T: min(var $$T: x, var $$T: y)
3.  function var float: min(var float: x, var float: y)
4.  function $T: min(array [$U] of $T: x)
5.  function $$T: min(array [$X] of opt $$T: x)
6.  function float: min(array [$X] of opt float: x)
7.  function var $$T: min(array [$U] of var $$T: x)
8.  function var float: min(array [$U] of var float: x)
9.  function var $$T: min(array [$X] of var opt $$T: x)
10. function var float: min(array [$X] of var opt float: x)
11. function $$E: min(set of $$E: s)
```

1-3.
Return minimum of `x` and `y`

4. Return minimum of elements in array `x`

5, 6.
Return minimum of elements in `x` that are not absent. Result is undefined when all values are absent.

7, 8.
Return minimum of elements in array `x`

9, 10.
Return minimum of elements in `x` that are not absent. Result is undefined when all values are absent.

11. Return minimum of elements in set `s`

```
1.  function opt $$T: min_weak(array [$X] of opt $$T: x)
2.  function opt float: min_weak(array [$X] of opt float: x)
3.  function var opt $$T: min_weak(array [$X] of var opt $$T: x)
4.  function var opt float: min_weak(array [$X] of var opt float: x)
```

1, 2.
Return maximum of elements in `x` that are not absent. Result is absent when all values are absent.

3, 4.
Return minimum of elements in `x` that are not absent. Result is absent when all values are absent.

```
1.  function int: pow(int: x, int: y)
2.  function var int: pow(var int: x, int: y)
3.  function var int: pow(var int: x, var int: y)
4.  function float: pow(float: x, float: y)
5.  function var float: pow(var float: x, var float: y)
```

1-3.
Return $\mathbf{x}^{\mathbf{y}}$, when $\mathbf{y} < 0$ it returns `1 div pow(x, abs(y))`.

4, 5.
Return $\mathbf{x}^{\mathbf{y}}$

```
1.  function int: product(array [$T] of int: x)
2.  function var int: product(array [$T] of var int: x)
3.  function float: product(array [$T] of float: x)
4.  function var float: product(array [$T] of var float: x)
5.  function int: product(array [$T] of opt int: x)
6.  function var int: product(array [$T] of var opt int: x)
7.  function float: product(array [$T] of opt float: x)
8.  function var float: product(array [$T] of var opt float: x)
```

1-4.
Return product of elements in array `x`

5-8.
Return product of non-absent elements of `x`.

```
1.  function float: sqrt(float: x)
2.  function var float: sqrt(var float: x)
```

```
3.  function opt float: sqrt(opt float: x)

4.  function var opt float: sqrt(var opt float: x)
```

1, 2.
Return $\sqrt{x}$

3, 4.
Return $\sqrt{x}$ if x occurs, otherwise return absent

```
1.  function int: sum(array [$T] of int: x)

2.  function var int: sum(array [$T] of var int: x)

3.  function float: sum(array [$T] of float: x)

4.  function var float: sum(array [$T] of var float: x)

5.  function var int: sum(array [int] of var opt int: x)

6.  function int: sum(array [int] of opt int: x)

7.  function var float: sum(array [int] of var opt float: x)

8.  function float: sum(array [int] of opt float: x)
```

1-4.
Return sum of elements in array x

5-8.
Return sum of non-absent elements of x.

```
function var opt int: '~*'(var opt int: x, var opt int: y)
function opt int: '~*'(opt int: x, opt int: y)
function var opt float: '~*'(var opt float: x, var opt float: y)
```

Weak multiplication. Return product of x and y if both are present, otherwise return absent.

```
function var opt int: '~+'(var opt int: x, var opt int: y)
function opt int: '~+'(opt int: x, opt int: y)
function var opt float: '~+'(var opt float: x, var opt float: y)
```

Weak addition. Return sum of x and y if both are present, otherwise return absent.

```
function var opt int: '~-'(var opt int: x, var opt int: y)
function opt int: '~-'(opt int: x, opt int: y)
function var opt float: '~-'(var opt float: x, var opt float: y)
```

Weak subtraction. Return difference of `x` and `y` if both are present, otherwise return absent.

```
function var opt float: '~/'(var opt float: x, var opt float: y)
function opt float: '~/'(opt float: x, opt float: y)
```

Weal optional division. Return absent if `x` or `y` is absent, `x` divided by `y` if both are present.

```
function var opt int: '~div'(var opt int: x, var opt int: y)
function opt int: '~div'(opt int: x, opt int: y)
```

Weak optional division. Return absent if `x` or `y` is absent, `x` divided by `y` if both are present.

Exponential and logarithmic builtins

These builtins implement exponential and logarithmic functions.

```
1. function float: exp(float: x)

2. function var float: exp(var float: x)

3. function opt float: exp(opt float: x)

4. function var opt float: exp(var opt float: x)
```

1, 2.
Return $e^{\mathbf{x}}$

3, 4.
Return $e^{\mathbf{x}}$ if `x` occurs, otherwise return absent

---

```
1. function float: ln(float: x)

2. function var float: ln(var float: x)

3. function opt float: ln(opt float: x)

4. function var opt float: ln(var opt float: x)
```

1, 2.
Return ln **x**

3, 4.
Return ln **x** if x occurs, otherwise return absent

```
function float: log(float: x, float: y)
```

Return $\log_{\mathbf{x}} \mathbf{y}$

```
1.  function float: log10(float: x)
2.  function var float: log10(var float: x)
3.  function opt float: log10(opt float: x)
4.  function var opt float: log10(var opt float: x)
```

1, 2.
Return $\log_{10} \mathbf{x}$

3, 4.
Return $\log_{10} \mathbf{x}$ if x occurs, otherwise return absent

---

```
1.  function float: log2(float: x)
2.  function var float: log2(var float: x)
3.  function opt float: log2(opt float: x)
4.  function var opt float: log2(var opt float: x)
```

1, 2.
Return $\log_2 \mathbf{x}$

3, 4.
Return $\log_2 \mathbf{x}$ if x occurs, otherwise return absent

---

Trigonometric functions

These builtins implement the standard trigonometric functions.

```
1.  function float: acos(float: x)
2.  function var float: acos(var float: x)
```

```
3.  function opt float: acos(opt float: x)

4.  function var opt float: acos(var opt float: x)
```

1, 2.
Return acos **x**

3, 4.
Return acos **x** if x occurs, otherwise return absent

```
1.  function float: acosh(float: x)

2.  function var float: acosh(var float: x)

3.  function opt float: acosh(opt float: x)

4.  function var opt float: acosh(var opt float: x)
```

1, 2.
Return acosh **x**

3, 4.
Return acosh **x** if x occurs, otherwise return absent

```
1.  function float: asin(float: x)

2.  function var float: asin(var float: x)

3.  function opt float: asin(opt float: x)

4.  function var opt float: asin(var opt float: x)
```

1, 2.
Return asin **x**

3, 4.
Return asin **x** if x occurs, otherwise return absent

```
1.  function float: asinh(float: x)

2.  function var float: asinh(var float: x)

3.  function opt float: asinh(opt float: x)

4.  function var opt float: asinh(var opt float: x)
```

1, 2.
Return asinh **x**

3, 4.
Return asinh **x** if x occurs, otherwise return absent

```
1.  function float: atan(float: x)

2.  function var float: atan(var float: x)

3.  function opt float: atan(opt float: x)

4.  function var opt float: atan(var opt float: x)
```

1, 2.
Return atan **x**

3, 4.
Return atan **x** if x occurs, otherwise return absent

```
1.  function float: atanh(float: x)

2.  function var float: atanh(var float: x)

3.  function opt float: atanh(opt float: x)

4.  function var opt float: atanh(var opt float: x)
```

1, 2.
Return atanh **x**

3, 4.
Return atanh **x** if x occurs, otherwise return absent

```
1.  function float: cos(float: x)

2.  function var float: cos(var float: x)

3.  function opt float: cos(opt float: x)

4.  function var opt float: cos(var opt float: x)
```

1, 2.
Return cos **x**

3, 4.
Return cos **x** if x occurs, otherwise return absent

```
1.  function float: cosh(float: x)
2.  function var float: cosh(var float: x)
3.  function opt float: cosh(opt float: x)
4.  function var opt float: cosh(var opt float: x)
```

1, 2.
Return cosh **x**

3, 4.
Return cosh **x** if x occurs, otherwise return absent

```
1.  function float: sin(float: x)
2.  function var float: sin(var float: x)
3.  function opt float: sin(opt float: x)
4.  function var opt float: sin(var opt float: x)
```

1, 2.
Return sin **x**

3, 4.
Return sin **x** if x occurs, otherwise return absent

```
1.  function float: sinh(float: x)
2.  function var float: sinh(var float: x)
3.  function opt float: sinh(opt float: x)
4.  function var opt float: sinh(var opt float: x)
```

1, 2.
Return sinh **x**

3, 4.
Return sinh **x** if x occurs, otherwise return absent

```
1.  function float: tan(float: x)
```

```
2.  function var float: tan(var float: x)

3.  function opt float: tan(opt float: x)

4.  function var opt float: tan(var opt float: x)
```

1, 2.
Return tan **x**

3, 4.
Return tan **x** if x occurs, otherwise return absent

```
1.  function float: tanh(float: x)

2.  function var float: tanh(var float: x)

3.  function opt float: tanh(opt float: x)

4.  function var opt float: tanh(var opt float: x)
```

1, 2.
Return tanh **x**

3, 4.
Return tanh **x** if x occurs, otherwise return absent

Coercions

These functions implement coercions, or channeling, between different types.

```
1.  function float: bool2float(bool: x)

2.  function opt float: bool2float(opt bool: x)

3.  function array [$T] of float: bool2float(array [$T] of bool: x)

4.  function array [$T] of var float: bool2float(array [$T] of var bool: x)

5.  function var float: bool2float(var bool: x)

6.  function var opt float: bool2float(var opt bool: x)

7.  function array [$T] of opt float: bool2float(array [$T] of opt bool: x)

8.  function array [$T] of var opt float: bool2float(array [$T] of var opt␣
↪bool: x)
```

1, 2.
Return Boolean x coerced to a float

3, 4.
Return array of Booleans x coerced to an array of floats

5. Return Boolean x coerced to a float
6. Return optional 0/1 float that is absent iff x is absent, and 1 iff x occurs and is true.

7, 8.
Return array of Booleans x coerced to an array of floats

```
1.  function int: bool2int(bool: x)

2.  function opt int: bool2int(opt bool: x)

3.  function var opt int: bool2int(var opt bool: x)

4.  function var int: bool2int(var bool: x)

5.  function array [$T] of var int: bool2int(array [$T] of var bool: x)

6.  function array [$T] of int: bool2int(array [$T] of bool: x)

7.  function array [$T] of set of int: bool2int(array [$T] of set of bool:␣
↪x)

8.  function array [$T] of var int: bool2int(array [$T] of var bool: x)

9.  function array [$T] of opt int: bool2int(array [$T] of opt bool: x)

10. function array [$T] of var opt int: bool2int(array [$T] of var opt␣
↪bool: x)
```

1, 2.
Return Boolean x coerced to an integer

3. Return optional 0/1 integer that is absent iff x is absent, and 1 iff x occurs and is true.
4. Return Boolean x coerced to an integer

5, 6.
Return array of Booleans x coerced to an array of integers

7. Return array of sets of Booleans x coerced to an array of sets of integers

8-10.
Return array of Booleans x coerced to an array of integers

```
function int: ceil(float: x)
function var int: ceil(var float: x)
```

Return $\lceil \mathbf{x} \rceil$

```
function int: enum2int($$E: x)
function var int: enum2int(var $$E: x)
function opt int: enum2int(opt $$E: x)
function var opt int: enum2int(var opt $$E: x)
function opt set of int: enum2int(opt set of $$E: x)
function set of int: enum2int(set of $$E: x)
function var set of int: enum2int(var set of $$E: x)
function array [$X] of int: enum2int(array [$X] of $$E: x)
function array [$X] of var int: enum2int(array [$X] of var $$E: x)
function array [$X] of opt int: enum2int(array [$X] of opt $$E: x)
function array [$X] of var opt int: enum2int(array [$X] of var opt $$E: x)
function array [$X] of opt set of int: enum2int(array [$X] of opt set of $
↪$E: x)
function array [$X] of set of int: enum2int(array [$X] of set of $$E: x)
function array [$X] of var set of int: enum2int(array [$X] of var set of $
↪$E: x)
```

Return the underlying integer representation of `x`

```
function int: floor(float: x)
function var int: floor(var float: x)
```

Return $\lfloor \mathbf{x} \rfloor$

```
1.  function array [int] of any $T: index2int(array [$$E] of any $T: x)

2.  function array [int,int] of any $T: index2int(array [$$E,$$F] of any $T:␣
↪x)

3.  function array [int,int,int] of any $T: index2int(array [$$E,$$F,$$G] of␣
↪any $T: x)

4.  function array [int,int,int,int] of any $T: index2int(array [$$E,$$F,$$G,
↪$$H] of any $T: x)

5.  function array [int,int,int,int,int] of any $T: index2int(array [$$E,$$F,
↪$$G,$$H,$$I] of any $T: x)

6.  function array [int,int,int,int,int,int] of any $T: index2int(array [$$E,
```

```
↪$$F,$$G,$$H,$$I,$$J] of any $T: x)
```

1. Return x with its index set coerced to integers

2-6.
Return x with its index sets coerced to integers

```
1.  function float: int2float(int: x)

2.  function opt float: int2float(opt int: x)

3.  function var float: int2float(var int: x)

4.  function var opt float: int2float(var opt int: x)

5.  function array [$T] of float: int2float(array [$T] of int: x)

6.  function array [$T] of opt float: int2float(array [$T] of opt int: x)

7.  function array [$T] of var float: int2float(array [$T] of var int: x)

8.  function array [$T] of var opt float: int2float(array [$T] of var opt␣
↪int: x)
```

1-3.
Return integer x coerced to a float

4. Return optional 0/1 integer that is absent iff x is absent, and 1 iff x occurs and is true.

5-7.
Return array of integers x coerced to an array of floats

8. Return array of optional integers x coerced to an array of optional floats

---

```
function int: round(float: x)
function var int: round(var float: x)
```

Return x rounded to nearest integer

```
function array [int] of $$E: set2array(set of $$E: x)
```

Return a set of integers x coerced to an array of integers

Array operations

These functions implement the basic operations on arrays.

Functions and Predicates

```
1.  function array [int] of any $T: '++'(array [$$X] of any $T: x,
                                         array [$$Y] of any $T: y)

2.  function array [int,$$E] of any $T: '++'(array [$$X,$$E] of any $T: x,
                                             array [$$Y,$$E] of any $T: y)

3.  function array [int,$$E,$$F] of any $T: '++'(array [$$X,$$E,$$F] of any
↪$T: x,
                                                 array [$$Y,$$E,$$F] of any
↪$T: y)

4.  function array [int,$$E,$$F,$$G] of any $T: '++'(array [$$X,$$E,$$F,$$G]␣
↪of any $T: x,
                                                     array [$$Y,$$E,$$F,$$G]␣
↪of any $T: y)

5.  function array [int,$$E,$$F,$$G,$$H] of any $T: '++'(array [$$X,$$E,$$F,$
↪$G,$$H] of any $T: x,
                                                         array [$$Y,$$E,$$F,$
↪$G,$$H] of any $T: y)

6.  function array [int,$$E,$$F,$$G,$$H,$$I] of any $T: '++'(array [$$X,$$E,$
↪$F,$$G,$$H,$$I] of any $T: x,
                                                             array [$$Y,$$E,$
↪$F,$$G,$$H,$$I] of any $T: y)
```

1. Return the concatenation of arrays `x` and `y`

   Usage: `x ++ y`

2-6.

Return the concatenation of the multidimensional arrays `x` and `y`

The index sets of the arrays must match except for the first index set.

Usage: `x ++ y`

```
test 'in'($X: x, array [$T] of $X: y)
test 'in'(opt $X: x, array [$T] of opt $X: y)
predicate 'in'(var $X: x, array [$T] of var $X: y)
predicate 'in'(var opt $X: x, array [$T] of var opt $X: y)
test 'in'(set of $X: x, array [$T] of set of $X: y)
predicate 'in'(var set of $$X: x, array [$T] of var set of $$X: y)
```

Return if `y` contains `x`

Usage: `x in y`

```
1.  function array [int] of any $V: array1d(array [$U] of any $V: x)

2.  function array [$$E] of any $V: array1d(set of $$E: S,
                                            array [$U] of any $V: x)
```

1. Return array `x` coerced to index set 1..length(x). Coercions are performed by considering the array `x` in row-major order.
2. Return array `x` coerced to one-dimensional array with index set `S`. Coercions are performed by considering the array `x` in row-major order.

```
function array [$$E,$$F] of any $V: array2d(set of $$E: S1,
                                            set of $$F: S2,
                                            array [$U] of any $V: x)
```

Return array `x` coerced to two-dimensional array with index sets `S1` and `S2`. Coercions are performed by considering the array `x` in row-major order.

```
1.  function var set of $$T: array2set(array [$X] of var $$T: x)

2.  function var set of $$T: array2set(array [$X] of var opt $$T: x)

3.  function set of $$T: array2set(array [int] of $$T: x)

4.  function set of $$T: array2set(array [int] of opt $$T: x)

5.  function set of bool: array2set(array [int] of bool: x)

6.  function set of float: array2set(array [int] of float: x)
```

1. Return the set containing the elements of `x`
2. Return the set containing the non-absent elements of `x`
3. Return the set containing the elements of `x`
4. Return the set containing the non-absent elements of `x`

5, 6.
Return the set containing the elements of `x`

```
function array [$$E,$$F,$$G] of any $V: array3d(set of $$E: S1,
                                                set of $$F: S2,
                                                set of $$G: S3,
                                                array [$U] of any $V: x)
```

Return array `x` coerced to three-dimensional array with index sets `S1`, `S2` and `S3`. Coercions are performed by considering the array `x` in row-major order.

```
function array [$$E,$$F,$$G,$$H] of any $V: array4d(set of $$E: S1,
                                                    set of $$F: S2,
                                                    set of $$G: S3,
                                                    set of $$H: S4,
                                                    array [$U] of any $V: x)
```

Return array `x` coerced to 4-dimensional array with index sets `S1`, `S2`, `S3` and `S4`. Coercions are performed by considering the array `x` in row-major order.

```
function array [$$E,$$F,$$G,$$H,$$I] of any $V: array5d(set of $$E: S1,
                                                        set of $$F: S2,
                                                        set of $$G: S3,
                                                        set of $$H: S4,
                                                        set of $$I: S5,
                                                        array [$U] of any
↪$V: x)
```

Return array `x` coerced to 5-dimensional array with index sets `S1`, `S2`, `S3`, `S4` and `S5`. Coercions are performed by considering the array `x` in row-major order.

```
function array [$$E,$$F,$$G,$$H,$$I,$$J] of any $V: array6d(set of $$E: S1,
                                                            set of $$F: S2,
                                                            set of $$G: S3,
                                                            set of $$H: S4,
                                                            set of $$I: S5,
                                                            set of $$J: S6,
                                                            array [$U] of␣
↪any $V: x)
```

Return array `x` coerced to 6-dimensional array with index sets `S1`, `S2`, `S3`, `S4`, `S5` and `S6`. Coercions are performed by considering the array `x` in row-major order.

```
function array [$T] of any $V: arrayXd(array [$T] of any $X: x,
                                       array [$U] of any $V: y)
```

Return array `y` coerced to array with same number of dimensions and same index sets as array `x`. Coercions are performed by considering the array `y` in row-major order.

```
1.  function array [$$E] of any $V: array_index_shift(set of $$E: idx,
                                                      array [$$F] of any $V:␣
↪y)

2.  function array [$$E,$$F] of any $V: array_index_shift(set of $$E: idx1,
                                                         set of $$F: idx2,
                                                         array [$$G,$$H] of␣
↪any $V: y)

3.  function array [$$E,$$F,$$G] of any $V: array_index_shift(set of $$E:␣
↪idx1,
                                                             set of $$F:␣
↪idx2,
                                                             set of $$G:␣
↪idx3,
                                                             array [$$H,$$I,
↪$$J] of any $V: y)

4.  function array [$$E,$$F,$$G,$$H] of any $V: array_index_shift(set of $
↪$E: idx1,
                                                                 set of $
↪$F: idx2,
                                                                 set of $
↪$G: idx3,
                                                                 set of $
↪$H: idx4,
                                                                 array [$$I,
↪$$J,$$K,$$L] of any $V: y)

5.  function array [$$E,$$F,$$G,$$H,$$I] of any $V: array_index_shift(set of
↪$$E: idx1,
                                                                     set of
↪$$F: idx2,
                                                                     set of
↪$$G: idx3,
                                                                     set of
↪$$H: idx4,
                                                                     set of
↪$$I: idx5,
                                                                     array [
↪$$J,$$K,$$L,$$M,$$N] of any $V: y)

6.  function array [$$E,$$F,$$G,$$H,$$I,$$J] of any $V:␣
↪array_index_shift(set of $$E: idx1,
                                                                        ␣
↪set of $$F: idx2,
```

```
                                                                          ␣
↪set of $$G: idx3,
                                                                          ␣
↪set of $$H: idx4,
                                                                          ␣
↪set of $$I: idx5,
                                                                          ␣
↪set of $$J: idx6,
                                                                          ␣
↪array [$$K,$$L,$$M,$$N,$$O,$$P] of any $V: y)
```

1. Return array `y` with its index set shifted to start at `idx`.
2. Return array `y` with each index set shifted to start at `idx1` and `idx2`.
3. Return array `y` with each index set shifted to start at `idx1`, `idx2` and `idx3`.
4. Return array `y` with each index set shifted to start at `idx1`, `idx2`, `idx3` and `idx4`.
5. Return array `y` with each index set shifted to start at `idx1`, `idx2`, `idx3`, `idx4` and `idx5`.
6. Return array `y` with each index set shifted to start at `idx1`, `idx2`, `idx3`, `idx4`, `idx5` and `idx6`.

```
function array [$$E] of any $T: col(array [$$E,$$F] of any $T: x,
                                    $$F: c)
```

Return column `c` of array `x`

```
function array [int] of any $T: concat_index_1(array [$$X11] of any $T: x,
                                               array [$$X12] of any $T: y)
function array [int,$$X2] of any $T: concat_index_1(array [$$X11,$$X2] of␣
↪any $T: x,
                                                    array [$$X12,$$X2] of␣
↪any $T: y)
function array [int,$$X2,$$X3] of any $T: concat_index_1(array [$$X11,$$X2,$
↪$X3] of any $T: x,
                                                         array [$$X12,$$X2,$
↪$X3] of any $T: y)
function array [int,$$X2,$$X3,$$X4] of any $T: concat_index_1(array [$$X11,$
↪$X2,$$X3,$$X4] of any $T: x,
                                                              array [$$X12,$
↪$X2,$$X3,$$X4] of any $T: y)
function array [int,$$X2,$$X3,$$X4,$$X5] of any $T: concat_index_1(array [$
↪$X11,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                   array [$
↪$X12,$$X2,$$X3,$$X4,$$X5] of any $T: y)
function array [int,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T:␣
↪concat_index_1(array [$$X11,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
```

```
                                                                    ␣
↪array [$$X12,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the first index.

```
function array [$$X1,int] of any $T: concat_index_2(array [$$X1,$$X21] of␣
↪any $T: x,
                                                    array [$$X1,$$X22] of␣
↪any $T: y)
function array [$$X1,int,$$X3] of any $T: concat_index_2(array [$$X1,$$X21,$
↪$X3] of any $T: x,
                                                        array [$$X1,$$X22,$
↪$X3] of any $T: y)
function array [$$X1,int,$$X3,$$X4] of any $T: concat_index_2(array [$$X1,$
↪$X21,$$X3,$$X4] of any $T: x,
                                                            array [$$X1,$
↪$X22,$$X3,$$X4] of any $T: y)
function array [$$X1,int,$$X3,$$X4,$$X5] of any $T: concat_index_2(array [$
↪$X1,$$X21,$$X3,$$X4,$$X5] of any $T: x,
                                                                array [$
↪$X1,$$X22,$$X3,$$X4,$$X5] of any $T: y)
function array [$$X1,int,$$X3,$$X4,$$X5,$$X6] of any $T:␣
↪concat_index_2(array [$$X1,$$X21,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                    ␣
↪array [$$X1,$$X22,$$X3,$$X4,$$X5,$$X6] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the second index.

```
function array [$$X1,$$X2,int] of any $T: concat_index_3(array [$$X1,$$X2,$
↪$X31] of any $T: x,
                                                    array [$$X1,$$X2,$
↪$X32] of any $T: y)
function array [$$X1,$$X2,int,$$X4] of any $T: concat_index_3(array [$$X1,$
↪$X2,$$X31,$$X4] of any $T: x,
                                                        array [$$X1,$
↪$X2,$$X32,$$X4] of any $T: y)
function array [$$X1,$$X2,int,$$X4,$$X5] of any $T: concat_index_3(array [$
↪$X1,$$X2,$$X31,$$X4,$$X5] of any $T: x,
                                                            array [$
↪$X1,$$X2,$$X32,$$X4,$$X5] of any $T: y)
function array [$$X1,$$X2,int,$$X4,$$X5,$$X6] of any $T:␣
↪concat_index_3(array [$$X1,$$X2,$$X31,$$X4,$$X5,$$X6] of any $T: x,
                                                                    ␣
↪array [$$X1,$$X2,$$X32,$$X4,$$X5,$$X6] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the third index.

```
function array [$$X1,$$X2,$$X3,int] of any $T: concat_index_4(array [$$X1,$
↪$X2,$$X3,$$X41] of any $T: x,
                                                              array [$$X1,$
↪$X2,$$X3,$$X42] of any $T: y)
function array [$$X1,$$X2,$$X3,int,$$X5] of any $T: concat_index_4(array [$
↪$X1,$$X2,$$X3,$$X41,$$X5] of any $T: x,
                                                                  array [$
↪$X1,$$X2,$$X3,$$X42,$$X5] of any $T: y)
function array [$$X1,$$X2,$$X3,int,$$X5,$$X6] of any $T:␣
↪concat_index_4(array [$$X1,$$X2,$$X3,$$X41,$$X5,$$X6] of any $T: x,
                                                                      ␣
↪array [$$X1,$$X2,$$X3,$$X42,$$X5,$$X6] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the fourth index.

```
function array [$$X1,$$X2,$$X3,$$X4,int] of any $T: concat_index_5(array [$
↪$X1,$$X2,$$X3,$$X4,$$X51] of any $T: x,
                                                                  array [$
↪$X1,$$X2,$$X3,$$X4,$$X52] of any $T: y)
function array [$$X1,$$X2,$$X3,$$X4,int,$$X6] of any $T:␣
↪concat_index_5(array [$$X1,$$X2,$$X3,$$X4,$$X51,$$X6] of any $T: x,
                                                                      ␣
↪array [$$X1,$$X2,$$X3,$$X4,$$X52,$$X6] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the fifth index.

```
function array [$$X1,$$X2,$$X3,$$X4,$$X5,int] of any $T:␣
↪concat_index_6(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X61] of any $T: x,
                                                                      ␣
↪array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X62] of any $T: y)
```

Return the concatenation of the multidimensional arrays `x` and `y` in the sixth index.

```
test has_element($T: e, array [int] of $T: x)
test has_element($T: e, array [int] of opt $T: x)
predicate has_element($T: e, array [$$E] of any $T: x)
```

Test if `e` is an element of array `x`

```
test has_index($$E: i, array [$$E] of any $T: x)
```

Test if `i` is in the index set of `x`

```
function set of $$E: index_set(array [$$E] of any $U: x)
```

Return index set of one-dimensional array `x`

```
function set of $$E: index_set_1of2(array [$$E,$$F] of any $U: x)
```

Return index set of first dimension of two-dimensional array `x`

```
function set of $$E: index_set_1of3(array [$$E,$$F,$$G] of any $U: x)
```

Return index set of first dimension of 3-dimensional array `x`

```
function set of $$E: index_set_1of4(array [$$E,$$F,$$G,$$H] of any $U: x)
```

Return index set of first dimension of 4-dimensional array `x`

```
function set of $$E: index_set_1of5(array [$$E,$$F,$$G,$$H,$$I] of any $U: x)
```

Return index set of first dimension of 5-dimensional array `x`

```
function set of $$E: index_set_1of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of first dimension of 6-dimensional array `x`

```
function set of $$F: index_set_2of2(array [$$E,$$F] of any $U: x)
```

Return index set of second dimension of two-dimensional array `x`

```
function set of $$F: index_set_2of3(array [$$E,$$F,$$G] of any $U: x)
```

Return index set of second dimension of 3-dimensional array `x`

```
function set of $$F: index_set_2of4(array [$$E,$$F,$$G,$$H] of any $U: x)
```

Return index set of second dimension of 4-dimensional array x

---

```
function set of $$F: index_set_2of5(array [$$E,$$F,$$G,$$H,$$I] of any $U: x)
```

Return index set of second dimension of 5-dimensional array x

---

```
function set of $$F: index_set_2of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of second dimension of 6-dimensional array x

---

```
function set of $$G: index_set_3of3(array [$$E,$$F,$$G] of any $U: x)
```

Return index set of third dimension of 3-dimensional array x

```
function set of $$G: index_set_3of4(array [$$E,$$F,$$G,$$H] of any $U: x)
```

Return index set of third dimension of 4-dimensional array x

```
function set of $$G: index_set_3of5(array [$$E,$$F,$$G,$$H,$$I] of any $U: x)
```

Return index set of third dimension of 5-dimensional array x

```
function set of $$G: index_set_3of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of third dimension of 6-dimensional array x

```
function set of $$H: index_set_4of4(array [$$E,$$F,$$G,$$H] of any $U: x)
```

Return index set of fourth dimension of 4-dimensional array x

---

```
function set of $$H: index_set_4of5(array [$$E,$$F,$$G,$$H,$$I] of any $U: x)
```

Return index set of fourth dimension of 5-dimensional array x

---

```
function set of $$H: index_set_4of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of fourth dimension of 6-dimensional array x

---

```
function set of $$I: index_set_5of5(array [$$E,$$F,$$G,$$H,$$I] of any $U: x)
```

Return index set of fifth dimension of 5-dimensional array x

```
function set of $$I: index_set_5of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of fifth dimension of 6-dimensional array x

```
function set of $$J: index_set_6of6(array [$$E,$$F,$$G,$$H,$$I,$$J] of any
↪$U: x)
```

Return index set of sixth dimension of 6-dimensional array x

```
test index_sets_agree(array [$T] of any $U: x, array [$T] of any $W: y)
```

Test if x and y have the same index sets

```
function int: length(array [$T] of any $U: x)
```

Return the length of array x

Note that the length is defined as the number of elements in the array, regardless of its dimensionality.

```
function array [$$E] of any $T: reverse(array [$$E] of any $T: x)
```

Return the array x in reverse order

The resulting array has the same index set as x.

```
function array [$$E] of any $T: row(array [$$F,$$E] of any $T: x,
                                    $$F: r)
```

Return row `r` of array `x`

```
function array [int] of any $T: select_from_index_1(array [$$X1] of any $T:␣
↪x,
                                                    set of $$X1: S)
function array [int,$$X2] of any $T: select_from_index_1(array [$$X1,$$X2]␣
↪of any $T: x,
                                                          set of $$X1: S)
function array [int,$$X2,$$X3] of any $T: select_from_index_1(array [$$X1,$
↪$X2,$$X3] of any $T: x,
                                                               set of $$X1: S)
function array [int,$$X2,$$X3,$$X4] of any $T: select_from_index_1(array [$
↪$X1,$$X2,$$X3,$$X4] of any $T: x,
                                                                    set of $
↪$X1: S)
function array [int,$$X2,$$X3,$$X4,$$X5] of any $T:␣
↪select_from_index_1(array [$$X1,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                         set␣
↪of $$X1: S)
function array [int,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T:␣
↪select_from_index_1(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                              ␣
↪set of $$X1: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the first index, specified by the set `S`. The result is undefined if `S` is not a subset of the first index set of `x`.

```
function array [$$X1,int] of any $T: select_from_index_2(array [$$X1,$$X2]␣
↪of any $T: x,
                                                          set of $$X2: S)
function array [$$X1,int,$$X3] of any $T: select_from_index_2(array [$$X1,$
↪$X2,$$X3] of any $T: x,
                                                               set of $$X2: S)
function array [$$X1,int,$$X3,$$X4] of any $T: select_from_index_2(array [$
↪$X1,$$X2,$$X3,$$X4] of any $T: x,
                                                                    set of $
↪$X2: S)
function array [$$X1,int,$$X3,$$X4,$$X5] of any $T:␣
↪select_from_index_2(array [$$X1,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                         set␣
↪of $$X2: S)
function array [$$X1,int,$$X3,$$X4,$$X5,$$X6] of any $T:␣
↪select_from_index_2(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
```

```
                                                                              ␣
↪set of $$X2: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the second index, specified by the set `S`. The result is undefined if `S` is not a subset of the second index set of `x`.

```
function array [$$X1,$$X2,int] of any $T: select_from_index_3(array [$$X1,$
↪$X2,$$X3] of any $T: x,
                                                              set of $$X3: S)
function array [$$X1,$$X2,int,$$X4] of any $T: select_from_index_3(array [$
↪$X1,$$X2,$$X3,$$X4] of any $T: x,
                                                                  set of $
↪$X3: S)
function array [$$X1,$$X2,int,$$X4,$$X5] of any $T:␣
↪select_from_index_3(array [$$X1,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                       set␣
↪of $$X3: S)
function array [$$X1,$$X2,int,$$X4,$$X5,$$X6] of any $T:␣
↪select_from_index_3(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                           ␣
↪set of $$X3: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the third index, specified by the set `S`. The result is undefined if `S` is not a subset of the third index set of `x`.

```
function array [$$X1,$$X2,$$X3,int] of any $T: select_from_index_4(array [$
↪$X1,$$X2,$$X3,$$X4] of any $T: x,
                                                                  set of $
↪$X4: S)
function array [$$X1,$$X2,$$X3,int,$$X5] of any $T:␣
↪select_from_index_4(array [$$X1,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                       set␣
↪of $$X4: S)
function array [$$X1,$$X2,$$X3,int,$$X5,$$X6] of any $T:␣
↪select_from_index_4(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                           ␣
↪set of $$X4: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the fourth index, specified by the set `S`. The result is undefined if `S` is not a subset of the fourth index set of `x`.

```
function array [$$X1,$$X2,$$X3,$$X4,int] of any $T:␣
↪select_from_index_5(array [$$X1,$$X2,$$X3,$$X4,$$X5] of any $T: x,
                                                                       set␣
↪of $$X5: S)
```

```
function array [$$X1,$$X2,$$X3,$$X4,int,$$X6] of any $T:␣
↪select_from_index_5(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                    ␣
↪set of $$X5: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the fifth index, specified by the set `S`. The result is undefined if `S` is not a subset of the fifth index set of `x`.

```
function array [$$X1,$$X2,$$X3,$$X4,$$X5,int] of any $T:␣
↪select_from_index_6(array [$$X1,$$X2,$$X3,$$X4,$$X5,$$X6] of any $T: x,
                                                                    ␣
↪set of $$X6: S)
```

Return a (potentially non-contiguous) slice of array `x`, in the sixth index, specified by the set `S`. The result is undefined if `S` is not a subset of the sixth index set of `x`.

```
function set of $$E: shift_index_set(set of $$E: idx, set of $$F: s)
```

Return an index set with the same cardinality as `s`, starting at the minimum value of `idx`.

```
function array [$$F] of any $T: slice_1d(array [$E] of any $T: x,
                                         array [int] of set of int: s,
                                         set of $$F: dims1)
```

Return slice of array `x` specified by sets `s`, coerced to new 1d array with index set `dims1`

```
function array [$$F,$$G] of any $T: slice_2d(array [$E] of any $T: x,
                                             array [int] of set of int: s,
                                             set of $$F: dims1,
                                             set of $$G: dims2)
```

Return slice of array `x` specified by sets `s`, coerced to new 2d array with index sets `dims1` and `dims2`

```
function array [$$F,$$G,$$H] of any $T: slice_3d(array [$E] of any $T: x,
                                                 array [int] of set of int:␣
↪s,
                                                 set of $$F: dims1,
                                                 set of $$G: dims2,
                                                 set of $$H: dims3)
```

Return slice of array `x` specified by sets `s`, coerced to new 3d array with index sets `dims1`, `dims2` and `dims3`

```
function array [$$F,$$G,$$H,$$I] of any $T: slice_4d(array [$E] of any $T: x,
                                                     array [int] of set of␣
↪int: s,
                                                     set of $$F: dims1,
                                                     set of $$G: dims2,
                                                     set of $$H: dims3,
                                                     set of $$I: dims4)
```

Return slice of array `x` specified by sets `s`, coerced to new 4d array with index sets `dims1`, `dims2`, `dims3`, `dims4`

```
function array [$$F,$$G,$$H,$$I,$$J] of any $T: slice_5d(array [$E] of any
↪$T: x,
                                                         array [int] of set␣
↪of int: s,
                                                         set of $$F: dims1,
                                                         set of $$G: dims2,
                                                         set of $$H: dims3,
                                                         set of $$I: dims4,
                                                         set of $$J: dims5)
```

Return slice of array `x` specified by sets `s`, coerced to new 5d array with index sets `dims1`, `dims2`, `dims3`, `dims4`, `dims5`

```
function array [$$F,$$G,$$H,$$I,$$J,$$K] of any $T: slice_6d(array [$E] of␣
↪any $T: x,
                                                             array [int] of␣
↪set of int: s,
                                                             set of $$F:␣
↪dims1,
                                                             set of $$G:␣
↪dims2,
                                                             set of $$H:␣
↪dims3,
                                                             set of $$I:␣
↪dims4,
                                                             set of $$J:␣
↪dims5,
                                                             set of $$K:␣
↪dims6)
```

Return slice of array `x` specified by sets `s`, coerced to new 6d array with index sets `dims1`, `dims2`, `dims3`, `dims4`, `dims5`, `dims6`

```
function tuple(array [$$E] of any $T, array [$$E] of any $U): unzip(array [$
↪$E] of tuple($T, $U): x)
```

Split array `x` of pairs into a pair of arrays.

The resulting arrays keep the index set of `x`.

```
function array [$$E] of tuple(any $T, any $U): zip(array [$$E] of any $T: x,
                                                   array [$$E] of any $U: y)
```

Pair elements of arrays `x` and `y` into tuples.

The arrays must have the same index set; the result has the same index set.

Annotations

```
annotation array_check_form
```

Forces a arrayNd call to throw an error if the new index sets are offsets of the current index sets.

Logical operations

Logical operations are the standard operators of Boolean logic.

```
test '->'(bool: x, bool: y)
test '->'(opt bool: x, opt bool: y)
predicate '->'(var bool: x, var bool: y)
predicate '->'(var opt bool: x, var opt bool: y)
```

Return truth value of `x` implies `y`

Usage: `x -> y`

```
1.  test '/\'(bool: x, bool: y)

2.  test '/\'(opt bool: x, opt bool: y)

3.  predicate '/\'(var bool: x, var bool: y)

4.  predicate '/\'(var opt bool: x, var opt bool: y)
```

1. Return truth value of x ∧ y

   Usage: `x /\ y`

2. Return truth value of x ∧ y with identity <> = True

   Usage: `x /\ y`

3. Return truth value of x ∧ y

   Usage: `x /\ y`

4. Return truth value of x ∧ y with identity <> = True

   Usage: `x /\ y`

```
test '<-'(bool: x, bool: y)
test '<-'(opt bool: x, opt bool: y)
predicate '<-'(var bool: x, var bool: y)
predicate '<-'(var opt bool: x, var opt bool: y)
```

Return truth value of y implies x

Usage: `x <- y`

```
test '<->'(bool: x, bool: y)
test '<->'(opt bool: x, opt bool: y)
predicate '<->'(var bool: x, var bool: y)
predicate '<->'(var opt bool: x, var opt bool: y)
```

Return truth value of x if-and-only-if y

Usage: `x <-> y`

```
1.  test '\/'(bool: x, bool: y)

2.  test '\/'(opt bool: x, opt bool: y)

3.  predicate '\/'(var bool: x, var bool: y)

4.  predicate '\/'(var opt bool: x, var opt bool: y)
```

1. Return truth value of x ∨ y

   Usage: `x \/ y`

2. Return truth value of x ∨ y with identity <> = False

   Usage: `x \/ y`

3. Return truth value of x ∨ y

   Usage: `x \/ y`

4. Return truth value of `x` ∨ `y` with identity <> = False

   Usage: `x \/ y`

```
1.  test 'not'(bool: x)

2.  function opt bool: 'not'(opt bool: x)

3.  predicate 'not'(var bool: x)

4.  function var opt bool: 'not'(var opt bool: x)
```

1. Return truth value of the negation of `x`

   Usage: `not x`

2. Negation of `x` if it occurs, otherwise absent

   Usage: `not x`

3. Return truth value of the negation of `x`

   Usage: `not x`

4. Negation of `x` if it occurs, otherwise absent

   Usage: `not x`

```
test 'xor'(bool: x, bool: y)
test 'xor'(opt bool: x, opt bool: y)
predicate 'xor'(var bool: x, var bool: y)
predicate 'xor'(var opt bool: x, var opt bool: y)
```

Return truth value of `x` xor `y`

Usage: `x xor y`

```
predicate bool_not(var bool: b)
```

Return negation of `b`

```
test clause(array [$T] of bool: x, array [$T] of bool: y)
test clause(array [$T] of opt bool: x, array [$T] of opt bool: y)
predicate clause(array [$T] of var bool: x, array [$T] of var bool: y)
predicate clause(array [$T] of var opt bool: x,
                 array [$T] of var opt bool: y)
```

Return truth value of $(\bigvee_i \mathbf{x}[i]) \vee (\bigvee_j \neg \mathbf{y}[j])$

```
1.  test exists(array [$T] of bool: x)

2.  predicate exists(array [$T] of var bool: x)

3.  predicate exists(array [int] of var opt bool: x)
```

1, 2.
Return truth value of $\bigvee_i \mathbf{x}[i]$

3. True iff for at least one `i`, `x`[i] occurs and is true

```
1.  test forall(array [$T] of bool: x)

2.  predicate forall(array [$T] of var bool: x)

3.  predicate forall(array [int] of var opt bool: x)
```

1, 2.
Return truth value of $\bigwedge_i \mathbf{x}[i]$

3. True iff for any `i`, `x`[i] is absent or true

```
test iffall(array [$T] of bool: x)
test iffall(array [$T] of opt bool: x)
predicate iffall(array [$T] of var bool: x)
predicate iffall(array [$T] of var opt bool: x)
```

Return truth value of true $\oplus$ $(\oplus_i \mathbf{x}[i])$

```
test xorall(array [$T] of bool: x)
test xorall(array [$T] of opt bool: x)
predicate xorall(array [$T] of var bool: x)
predicate xorall(array [$T] of var opt bool: x)
```

Return truth value of $\oplus_i \mathbf{x}[i]$

Set operations

These functions implement the basic operations on sets.

```
function set of $$E: '..'($$E: a, $$E: b)
function set of float: '..'(float: a, float: b)
function set of bool: '..'(bool: a, bool: b)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b}\}$

Usage: `a .. b`

```
function set of $$E: '..<'($$E: a, $$E: b)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b} - 1\}$

Usage: `a ..< b`

```
function set of $$E: '<..'($$E: a, $$E: b)
```

Return the set $\{\mathbf{a} + 1, \ldots, \mathbf{b}\}$

Usage: `a <.. b`

```
function set of $$E: '<..<'($$E: a, $$E: b)
```

Return the set $\{\mathbf{a} + 1, \ldots, \mathbf{b} - 1\}$

Usage: `a <..< b`

```
function set of $T: 'diff'(set of $T: x, set of $T: y)
function var set of $$T: 'diff'(var set of $$T: x, var set of $$T: y)
```

Return the set difference of sets $\mathbf{x} \setminus \mathbf{y}$

Usage: `x diff y`

```
1. test 'in'($$E: x, set of $$E: y)

2. predicate 'in'(var $$E: x, set of $$E: y)

3. predicate 'in'(var $$E: x, var set of $$E: y)

4. predicate 'in'(var opt $$E: x, set of $$E: y)
```

```
5.  test 'in'(opt $$E: x, set of $$E: y)
6.  predicate 'in'(var opt $$E: x, var set of $$E: y)
7.  test 'in'(float: x, set of float: y)
8.  predicate 'in'(var float: x, set of float: y)
9.  test 'in'(opt float: x, set of float: y)
10. predicate 'in'(var opt float: x, set of float: y)
```

1. Test if `x` is an element of the set `y`

   Usage: `x in y`

2, 3.

   `x` is an element of the set `y`

   Usage: `x in y`

4. `x` is an element of the set `y`. False if `x` is absent.

   Usage: `x in y`

5. Test if `x` is an element of the set `y`. False if `x` is absent.

   Usage: `x in y`

6. `x` is an element of the set `y`. False if `x` is absent.

   Usage: `x in y`

7, 8.

   Test if `x` is an element of the set `y`

   Usage: `x in y`

9. Test if `x` is an element of the set `y`. False if `x` is absent.

   Usage: `x in y`

10. `x` is an element of the set `y`. False if `x` is absent.

    Usage: `x in y`

```
function set of $T: 'intersect'(set of $T: x, set of $T: y)
function var set of $$T: 'intersect'(var set of $$T: x,
                                     var set of $$T: y)
```

Return the intersection of sets `x` and `y`

Usage: `x intersect y`

```
1.  test 'subset'(set of $T: x, set of $T: y)

2.  predicate 'subset'(var set of $$E: x, var set of $$E: y)
```

1. Test if `x` is a subset of `y`

   Usage: `x subset y`

2. `x` is a subset of `y`

   Usage: `x subset y`

```
1.  test 'superset'(set of $T: x, set of $T: y)

2.  predicate 'superset'(var set of $$E: x, var set of $$E: y)
```

1. Test if `x` is a superset of `y`

   Usage: `x superset y`

2. `x` is a superset of `y`

   Usage: `x superset y`

```
function set of $T: 'symdiff'(set of $T: x, set of $T: y)
function var set of $$T: 'symdiff'(var set of $$T: x,
                                   var set of $$T: y)
```

Return the symmetric set difference of sets `x` and `y`

Usage: `x symdiff y`

```
function set of $T: 'union'(set of $T: x, set of $T: y)
function var set of $$T: 'union'(var set of $$T: x, var set of $$T: y)
```

Return the union of sets `x` and `y`

Usage: `x union y`

```
function set of $$E: '..<o'($$E: a)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b} - 1\}$

```
function set of $$E: '..o'($$E: a)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b} - 1\}$

```
function set of $$E: '<..<o'($$E: a)
```

Return the set $\{\mathbf{a} + 1, \ldots, \mathbf{b} - 1\}$

```
function set of $$E: '<..o'($$E: a)
```

Return the set $\{\mathbf{a} + 1, \ldots, \mathbf{b}\}$

```
1.  function set of $U: array_intersect(array [$T] of set of $U: x)

2.  function var set of $$E: array_intersect(array [$T] of var set of $$E: x)
```

1. Return the intersection of the sets in array `x`
2. Return the intersection of the sets in array `x`. Undefined for empty `x`.

```
function set of $U: array_union(array [$T] of set of $U: x)
function var set of $$E: array_union(array [$T] of var set of $$E: x)
```

Return the union of the sets in array `x`

```
function int: card(set of $T: x)
function var int: card(var set of $$E: x)
```

Return the cardinality of the set `x`

```
function var $$E: max(var set of $$E: s)
```

Return the maximum of the set `s`

```
function var $$E: min(var set of $$E: s)
```

Return the minimum of the set `s`

```
function set of $$E: 'o..'($$E: a)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b}-1\}$

```
function set of $$E: 'o..<'($$E: a)
```

Return the set $\{\mathbf{a}, \ldots, \mathbf{b}-1\}$

```
function set of $$E: 'o<..'($$E: a)
```

Return the set $\{\mathbf{a}+1, \ldots, \mathbf{b}\}$

```
function set of $$E: 'o<..<'($$E: a)
```

Return the set $\{\mathbf{a}+1, \ldots, \mathbf{b}-1\}$

```
function array [int] of $$E: set_to_ranges(set of $$E: S)
function array [int] of float: set_to_ranges(set of float: S)
function array [int] of bool: set_to_ranges(set of bool: S)
```

Return a sorted list of the non-overlapping ranges in `S`

String operations

These functions implement operations on strings.

```
function string: '++'(string: s1, string: s2)
```

Return concatenation of `s1` and `s2`

Usage: `s1 ++ s2`

```
function string: concat(array [$T] of string: s)
```

Return concatenation of strings in array `s`

```
function string: file_path()
```

Return path of file where this function is called

```
1.  function string: format(any $T: x)

2.  function string: format(array [$U] of any $T: x)

3.  function string: format(int: w, int: p, any $T: x)

4.  function string: format(int: w, int: p, array [$U] of any $T: x)

5.  function string: format(int: w, any $T: x)

6.  function string: format(int: w, array [$U] of any $T: x)
```

1, 2.
Convert `x` into a string

3. Formatted to-string conversion

   Converts the value `x` into a string right justified by the number of characters given by `w`, or left justified if `w` is negative.

   The maximum length of the string representation of `x` is given by `p`, or the maximum number of digits after the decimal point for floating point numbers. It is a run-time error for `p` to be negative.

4. Formatted to-string conversion

   Converts the value `x` into a string right justified by the number of characters given by `w`, or left justified if `w` is negative.

   The maximum length of the string representation of `x` is given by `p`. It is a run-time error for `p` to be negative.

5, 6.
Formatted to-string conversion

Converts the value `x` into a string right justified by the number of characters given by `w`, or left justified if `w` is negative.

```
1.  function string: format_justify_string(int: w, string: x)

2.  function string: format_justify_string(int: w, int: p, string: x)
```

1. String justification

   Returns the string `x` right justified by the number of characters given by `w`, or left justified if `w` is negative.

2. String justification

   Returns the string `x` right justified by the number of characters given by `w`, or left justified if `w` is negative.

The maximum length of of `x` to be formatted is given by `p`. It is a run-time error for `p` to be negative.

```
function string: join(string: d, array [$T] of string: s)
```

Join string in array `s` using delimiter `d`

```
function string: json_array(array [int] of string: arr)
```

Create a JSON array from an array of JSON strings `arr`.

The elements are assumed to be strings already in JSON format (usually generated using showJSON)

```
function string: json_object(array [int,1..2] of string: obj)
```

Create a JSON object from an array of key-value pairs `obj`.

`obj[i, 1]` is the key name for the `i`-th entry `obj[i, 2]` is the JSON value for the o i-th entry (usually generated using showJSON)

```
1.  function array [int] of string: outputJSON()

2.  function array [int] of string: outputJSON(bool: b)
```

1. Return array for output of all variables in JSON format
2. Return array for output of all variables in JSON format, including objective if `b` is true

---

```
function array [int] of string: outputJSONParameters()
```

Return array for output of all parameters in JSON format

```
test output_to_json_section(string: s, any $T: o)
test output_to_json_section(string: s, array [$U] of any $T: o)
```

Output `o` to the JSON section `s`

```
test output_to_section(string: s, string: o)
```

Add `o` to the output section `s`

```
function string: show(any $T: x)
function string: show(array [$U] of any $T: x)
```

Convert `x` into a string

```
function string: show2d(array [$$E,$$F] of any $T: x)
```

Convert two-dimensional array `x` into a string

```
function string: show2d_indexed(array [int] of string: row_hdr,
                                array [int] of string: col_hdr,
                                array [int,int] of string: vs)
```

Convert two-dimensional array `vs` into a string with row and column headers `row_hdr` and `col_hdr`

```
function string: show3d(array [$$E,$$F,$$G] of any $T: x)
```

Convert three-dimensional array `x` into a string

```
function string: showJSON(any $T: x)
function string: showJSON(array [$U] of any $T: x)
```

Convert `x` into JSON string

```
function string: show_float(int: w, int: p, var float: x)
```

Formatted to-string conversion for floats.

Converts the float `x` into a string right justified by the number of characters given by `w`, or left justified if `w` is negative. The number of digits to appear after the decimal point is given by `p`. It is a run-time error for `p` to be negative.

```
function string: show_int(int: w, var int: x)
```

Formatted to-string conversion for integers

Converts the integer `x` into a string right justified by the number of characters given by `w`, or left justified if `w` is negative.

```
function int: string_length(string: s)
```

Return length of `s`

This function interprets the string as UTF-8 encoded. The length is the number of Unicode code points in the string.

```
function array [int] of string: string_split(string: s, string: sep)
```

Split `s` by separator `sep`

Returns an array of strings containing the substrings of `s` that are separated by `sep`. If `sep` is empty, the result is an array of characters of `s`.

This function interprets the string as UTF-8 encoded, so when splitting with an empty separator, the result is an array of Unicode code points.

Functions for enums

```
1.  function $$E: enum_next($$E: x)
2.  function opt $$E: enum_next(opt $$E: x)
3.  function var $$E: enum_next(var $$E: x)
4.  function var opt $$E: enum_next(var opt $$E: x)
5.  function $$E: enum_next(set of $$E: e, $$E: x)
6.  function opt $$E: enum_next(set of $$E: e, opt $$E: x)
7.  function var $$E: enum_next(set of $$E: e, var $$E: x)
8.  function var opt $$E: enum_next(set of $$E: e, var opt $$E: x)
```

1-4.
Return next greater enum value of `x`

5-8.
Return next greater enum value of `x` in enum `e`

```
function set of $$E: enum_of(var opt $$E: x)
function set of $$E: enum_of(var set of $$E: x)
function set of $$E: enum_of(array [$T] of var opt $$E: x)
function set of $$E: enum_of(array [$T] of var set of $$E: x)
```

Return enum set of `x`

```
1.  function $$E: enum_prev($$E: x)

2.  function opt $$E: enum_prev(opt $$E: x)

3.  function var $$E: enum_prev(var $$E: x)

4.  function var opt $$E: enum_prev(var opt $$E: x)

5.  function $$E: enum_prev(set of $$E: e, $$E: x)

6.  function opt $$E: enum_prev(set of $$E: e, opt $$E: x)

7.  function var $$E: enum_prev(set of $$E: e, var $$E: x)

8.  function var opt $$E: enum_prev(set of $$E: e, var opt $$E: x)
```

1-4.
Return next smaller enum value of `x`

5-8.
Return next smaller enum value of `x` in enum `e`

```
function $$E: to_enum(set of $$E: X, int: x)
function opt $$E: to_enum(set of $$E: X, opt int: x)
function var $$E: to_enum(set of $$E: X, var int: x)
function var opt $$E: to_enum(set of $$E: X, var opt int: x)
function array [$U] of $$E: to_enum(set of $$E: X,
                                    array [$U] of int: x)
function array [$U] of opt $$E: to_enum(set of $$E: X,
                                        array [$U] of opt int: x)
function array [$U] of var $$E: to_enum(set of $$E: X,
                                        array [$U] of var int: x)
function array [$U] of var opt $$E: to_enum(set of $$E: X,
                                            array [$U] of var opt int: x)
function set of $$E: to_enum(set of $$E: X, set of int: x)
function array [$U] of set of $$E: to_enum(set of $$E: X,
```

```
                                                array [$U] of set of int: x)
function array [$U] of var set of $$E: to_enum(set of $$E: X,
                                                array [$U] of var set of int:␣
↪x)
```

Convert `x` to enum type `X`

4.2.1.2 Annotations

These annotations control evaluation and solving behaviour.

General annotations

Functions and Predicates

```
annotation json_section(string: s)
```

Declare that the annotated output item should be output to the section s and formatted as JSON.

```
annotation mzn_output_section(string: s)
```

Used internally by the compiler to set the output section `s` for an output item. An output item can be annotated directly with a string literal to set its output section as in `output :: "my_section" ["hello, world\n"]` or using the syntax `output ["hello, world\n"] to "my_section`.

Annotations

```
annotation add_to_output
```

Declare that the annotated variable should be added to the output of the model. This annotation only has an effect when the model does not have an output item.

```
annotation annotated_expression
```

Mark annotation argument as annotated expression

```
annotation cache_result
```

Enables caching of the result of the annotated function declaration such that further calls with the same arguments yield the cached initial result.

Currently only supported for par functions with at least one parameter.

```
annotation computed_domain
```

Declare the annotated variable as having a domain computed for the convenience of the solver. The domain itself does not impose a constraint in the overall problem and can be ignored.

```
annotation constraint_name(string: s)
```

Used to attach a name `s` to a constraint and its decomposition. String annotations on constraint keywords are re-written as constraint_name annotations

---

```
1.  annotation defines_var(any $T: c)

2.  annotation defines_var(array [$U] of any $T: arr)
```

1. Declare variable: `c` as being functionally defined by the annotated constraint. This annotation is introduced into FlatZinc code by the compiler.
2. Declare an array of variables `arr` as being functionally defined by the annotated constraint. This annotation is introduced into FlatZinc code by the compiler.

```
annotation doc_comment(string: s)
```

Document the function or variable declaration item with the string `s`.

```
annotation domain_change_constraint
```

Marks a constraint as a recorded domain changing constraint (when mzn2fzn called with -g flag

---

```
annotation empty_annotation
```

Empty annotation, will be removed during compilation

```
annotation expression_name(string: s)
```

Used to attach a name `s` to an expression, this should also propagate to any sub-expressions or decomposition of the annotated expression. String annotations on expressions are re-written as expression_name annotations

---

```
annotation is_defined_var
```

Declare the annotated variable as being functionally defined. This annotation is introduced into FlatZinc code by the compiler.

```
annotation is_reverse_map
```

Declare that the annotated expression is used to map an expression back from FlatZinc to MiniZinc.

```
annotation maybe_partial
```

Declare that expression may have undefined result (to avoid warnings)

```
annotation mzn_add_annotated_expression(int: idx)
```

Used internally by the compiler

```
annotation mzn_check_enum_var(array [int] of set of int: x)
```

Declare that the annotated variable is required for checking solutions and has an enum type `x`.

```
annotation mzn_check_var
```

Declare that the annotated variable is required for checking solutions.

```
annotation mzn_constraint_name(string: n)
```

Declare a name `n` for the annotated constraint.

```
annotation mzn_deprecated(string: version, string: explanation)
```

State that a function is deprecated since MiniZinc version `version` with humand readable `explanation`.

```
annotation mzn_expression_name(string: n)
```

Declare a name `n` for the annotated expression.

```
annotation mzn_path(string: s)
```

Representation of the call-stack when the annotated item was introduced, as a string `s`. Can be used to uniquely identify variables and constraints across different compilations of a model that may have different names. This annotations is introduced into FlatZinc code by the compiler and is retained if –keep-paths argument is used.

```
annotation mzn_rhs_from_assignment
```

Used internally by the compiler

```
annotation no_cse
```

This annotation will prevent calls to the function annotated to be added to the Common Subexpression Elimination map.

WARNING: using this annotation might result in duplicated constraints when used incorrectly.

```
annotation no_output
```

Declare that the annotated variable should not be added to the output of the model. This annotation only has an effect when the model does not have an output item.

```
annotation 'output'(any $T: x)
annotation 'output'(array [$U] of any $T: x)
```

Declare that the annotated variable `x` should be added to the output of the model. This annotation only has an effect when the model does not have an output item.

```
annotation output_array(array [$U] of set of int: a)
```

Declare that the annotated array should be printed by the solver with the given index sets `a`. This annotation is introduced into FlatZinc code by the compiler.

```
annotation output_only
```

Declare that the annotated variable should be only used for output. This annotation can be used to define variables that are required for solution checkers, or that are necessary for the output item. The annotated variable must be par.

```
annotation output_var
```

Declare that the annotated variable should be printed by the solver. This annotation is introduced into FlatZinc code by the compiler.

```
annotation promise_commutative
```

Declare the function to be commutative, i.e., the function has the same result regardless of the order in which the function parameters are provided.

WARNING: a commutative function must have only arguments of a single type, or a single array as an argument.

```
annotation promise_total
```

Declare function as total, i.e. it does not put any constraints on its arguments.

```
annotation var_is_introduced
```

Declare a variable as being introduced by the compiler.

Propagation strength annotations

```
annotation bounds
```

Annotate a constraint to use bounds propagation

Note: This annotation will be deprecated. Use bounds_propagation instead.

```
annotation bounds_propagation
```

Annotate a constraint to use bounds propagation

```
annotation domain
```

Annotate a constraint to use domain propagation

Note: This annotation will be deprecated. Use domain_propagation instead.

```
annotation domain_propagation
```

Annotate a constraint to use domain propagation

```
annotation value_propagation
```

Annotate a constraint to use value propagation

Search annotations

Variable selection annotations

```
annotation anti_first_fail
```

Choose the variable with the largest domain

```
annotation dom_w_deg
```

Choose the variable with largest domain, divided by the number of attached constraints weighted by how often they have caused failure

---

```
annotation first_fail
```

Choose the variable with the smallest domain

```
annotation impact
```

Choose the variable with the highest impact so far during the search

---

```
annotation input_order
```

Search variables in the given order

```
annotation largest
```

Choose the variable with the largest value in its domain

```
annotation max_regret
```

Choose the variable with largest difference between the two smallest values in its domain

---

```
annotation most_constrained
```

Choose the variable with the smallest domain, breaking ties using the number of attached constraints

---

```
annotation occurrence
```

Choose the variable with the largest number of attached constraints

---

```
annotation smallest
```

Choose the variable with the smallest value in its domain

Value choice annotations

```
annotation indomain
```

Assign values in ascending order

```
annotation indomain_interval
```

If the domain consists of several contiguous intervals, reduce the domain to the first interval. Otherwise bisect the domain.

```
annotation indomain_max
```

Assign the largest value in the domain

```
annotation indomain_median
```

Assign the middle value in the domain, or the smaller of the two middle values in case of an even number of elements in the domain

---

```
annotation indomain_middle
```

Assign the value in the domain closest to the mean of its current bounds

---

```
annotation indomain_min
```

Assign the smallest value in the domain

```
annotation indomain_random
```

Assign a random value from the domain

```
annotation indomain_reverse_split
```

Bisect the domain, excluding the lower half first

```
annotation indomain_split
```

Bisect the domain, excluding the upper half first

```
annotation indomain_split_random
```

Bisect the domain, randomly selecting which half to exclude first

---

```
annotation outdomain_max
```

Exclude the largest value from the domain

```
annotation outdomain_median
```

Exclude the middle value from the domain

```
annotation outdomain_min
```

Exclude the smallest value from the domain

```
annotation outdomain_random
```

Exclude a random value from the domain

Exploration strategy annotations

```
annotation complete
```

Perform a complete search

Restart annotations

```
annotation restart_constant(int: scale)
```

Restart after constant number of nodes `scale`

```
annotation restart_geometric(float: base, int: scale)
```

Restart with geometric sequence with parameters `base` and `scale`

```
annotation restart_linear(int: scale)
```

Restart with linear sequence scaled by `scale`

```
annotation restart_luby(int: scale)
```

Restart with Luby sequence scaled by `scale`

```
annotation restart_none
```

Do not restart

Other declarations

Functions and Predicates

```
1.  annotation bool_search(array [$X] of var bool: x,
                           ann: select,
                           ann: choice,
                           ann: explore)

2.  annotation bool_search(array [$X] of var bool: x,
                           ann: select,
                           ann: choice)

3.  annotation bool_search(array [$X] of var opt bool: x,
                           ann: select,
                           ann: choice,
                           ann: explore)

4.  annotation bool_search(array [$X] of var opt bool: x,
                           ann: select,
                           ann: choice)
```

1. Specify search on variables `x`, with variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
2. Specify search on variables `x`, with variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
3. Search annotation for optional Boolean variables. Specify search on variables `x`, with variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
4. Search annotation for optional Boolean variables. Specify search on variables `x`, with variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).

```
1.  annotation float_search(array [$X] of var float: x,
                            float: prec,
                            ann: select,
                            ann: choice,
                            ann: explore)

2.  annotation float_search(array [$X] of var float: x,
                            float: prec,
                            ann: select,
                            ann: choice)

3.  annotation float_search(array [$X] of var opt float: x,
                            float: prec,
                            ann: select,
                            ann: choice,
                            ann: explore)

4.  annotation float_search(array [$X] of var opt float: x,
                            float: prec,
                            ann: select,
                            ann: choice)
```

1. Specify search on variables `x`, with precision `prec`, variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
2. Specify search on variables `x`, with precision `prec`, variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
3. Search annotation for optional float variables. Specify search on variables `x`, with precision `prec`, variable selection strategy `select`, value choice strategy `choice`, and

exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).

4. Search annotation for optional float variables. Specify search on variables `x`, with precision `prec`, variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).

```
1.  annotation int_search(array [$X] of var $$E: x,
                          ann: select,
                          ann: choice,
                          ann: explore)

2.  annotation int_search(array [$X] of var $$E: x,
                          ann: select,
                          ann: choice)

3.  annotation int_search(array [$X] of var opt $$E: x,
                          ann: select,
                          ann: choice,
                          ann: explore)

4.  annotation int_search(array [$X] of var opt $$E: x,
                          ann: select,
                          ann: choice)
```

1. Specify search on variables `x`, with variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
2. Specify search on variables `x`, with variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
3. Search annotation for optional integer variables. Specify search on variables `x`, with variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
4. Search annotation for optional integer variables. Specify search on variables `x`, with variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).

```
1.  annotation set_search(array [$X] of var set of $$E: x,
                          ann: select,
                          ann: choice,
                          ann: explore)
```

```
2.  annotation set_search(array [$X] of var set of $$E: x,
                          ann: select,
                          ann: choice)
```

1. Specify search on variables `x`, with variable selection strategy `select`, value choice strategy `choice`, and exploration strategy `explore`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).
2. Specify search on variables `x`, with variable selection strategy `select`, and value choice strategy `choice`. If `x` is a multi-dimensional array, it is coerced to one-dimensional in row-major order (as with the array1d function).

Annotations

```
annotation seq_search(array [int] of ann: s)
```

Sequentially perform the searches specified in array `s`

Warm start annotations

To be put on the solve item, similar to search annotations. A variable can be mentioned several times and in different annotations but only one of the values is taken

Warm start annotations with optional values

The value arrays can contain <> elements (absent values). The following decompositions eliminate those elements because FlatZinc 1.6 does not support optionals.

```
1.  annotation warm_start(array [int] of var bool: x,
                          array [int] of opt bool: v)

2.  annotation warm_start(array [int] of var int: x,
                          array [int] of opt int: v)

3.  annotation warm_start(array [int] of var float: x,
                          array [int] of opt float: v)

4.  annotation warm_start(array [int] of var set of int: x,
                          array [int] of opt set of int: v)
```

1. Specify warm start values `v` for an array of booleans `x`
2. Specify warm start values `v` for an array of integers `x`
3. Specify warm start values `v` for an array of floats `x`
4. Specify warm start values `v` for an array of sets `x`

Other declarations

```
1.  annotation warm_start(array [$X] of var bool: x, array [$X] of bool: v)

2.  annotation warm_start(array [int] of var bool: x,
                          array [int] of bool: v)

3.  annotation warm_start(array [$X] of var $$E: x, array [$X] of $$E: v)

4.  annotation warm_start(array [int] of var $$E: x, array [int] of $$E: v)

5.  annotation warm_start(array [$X] of var float: x,
                          array [$X] of float: v)

6.  annotation warm_start(array [int] of var float: x,
                          array [int] of float: v)

7.  annotation warm_start(array [$X] of var set of $$E: x,
                          array [$X] of set of $$E: v)

8.  annotation warm_start(array [int] of var set of int: x,
                          array [int] of set of int: v)
```

1, 2.
Specify warm start values `v` for an array of booleans `x`

3, 4.
Specify warm start values `v` for an array of integers `x`

5, 6.
Specify warm start values `v` for an array of floats `x`

7, 8.
Specify warm start values `v` for an array of sets `x`

```
annotation warm_start_array(array [int] of ann: w)
```

Specify an array `w` of warm_start annotations or other warm_start_array annotations. Can be useful to keep the annotation order in FlatZinc for manual updating.

Note: if you have search annotations as well, put warm_starts into seq_search in order to have precedence between both, which may matter.

Large Neighbourhood Search annotations

These annotations can be used on the solve item (similar to search annotations) to specify Large Neighbourhood Search meta-heuristics.

These annotations specify how the search is restricted to a neighbourhood when a solver restarts. The annotations define which variables will be fixed to their values from a previous solution in that case. All other variables will be reset to their initial domains by the restart.

The annotations can be used together with search annotations and restart annotations.

```
annotation relax_and_reconstruct(array [int] of var int: x, int: p)
```

Random neighbourhood over variables `x` with probability `p` percent

Upon restart, each variable in `x` is fixed to the value of the incumbent solution with a probability of `p` percent.

Context annotations

These are used internally by the compiler, and should not be used in models.

```
annotation ctx_mix
```

Mixed context

```
annotation ctx_neg
```

Negative context

```
annotation ctx_pos
```

Positive context

```
annotation ctx_root
```

Root context

Redundant and symmetry breaking constraints

These predicates allow users to mark constraints as e.g. symmetry breaking or redundant, so that solvers can choose to implement them differently.

We cannot easily use annotations for this purpose, since annotations are propagated to all constraints in a decomposition, which may be incorrect for redundant or symmetry breaking constraints in the presence of common subexpression elimination (CSE).

```
predicate implied_constraint(var bool: b)
```

Mark `b` as an implied constraint (synonym for redundant_constraint)

```
predicate redundant_constraint(var bool: b)
```

Mark `b` as a redundant constraint

```
predicate symmetry_breaking_constraint(var bool: b)
```

Mark `b` as a symmetry breaking constraint

4.2.1.3 Option type support

These functions and predicates implement the standard library for working with option types. Note that option type support is still incomplete.

Option type support for Booleans

```
predicate absent(var opt bool: x)
```

True iff `x` is absent

```
predicate deopt(var opt bool: x)
```

Return value of `x` (assumes that `x` is not absent)

```
predicate occurs(var opt bool: x)
```

True iff `x` is not absent

Option type support for integers

```
predicate absent(var opt $$E: x)
```

True iff x is absent

```
function var $$E: deopt(var opt $$E: x)
```

Return value of x (assumes that x is not absent)

```
test had_zero(var opt $$E: x)
test had_zero(opt $$E: x)
test had_zero(array [int] of var opt $$E: x)
```

True if x had zero in its original domain.

Returns true if absent zero representation is disabled or if it is possible that occurs$(x) \wedge$ deopt$(x) = 0$

```
predicate occurs(var opt $$E: x)
```

True iff x is not absent

Option type support for floats

```
predicate absent(var opt float: x)
```

True iff x is absent

```
function var float: deopt(var opt float: x)
```

Return value of x (assumes that x is not absent)

```
predicate occurs(var opt float: x)
```

True iff x is not absent

Other declarations

```
1.  test absent($T: x)

2.  test absent(var $T: x)

3.  test absent(set of $$T: x)

4.  test absent(var set of $$T: x)

5.  test absent(opt $T: x)
```

1-4.
Test if x is absent (always returns false)

5. Test if x is absent

```
 1.  function $$T: deopt(opt $$T: x)

 2.  function $T: deopt(opt $T: x)

 3.  function array [$$U] of $$T: deopt(array [$$U] of opt $$T: x)

 4.  function array [$$U] of $T: deopt(array [$$U] of opt $T: x)

 5.  function var $T: deopt(var $T: x)

 6.  function $T: deopt($T: x)

 7.  function array [$U] of var $T: deopt(array [$U] of var $T: x)

 8.  function array [$$U] of var bool: deopt(array [$$U] of var opt bool: x)

 9.  function array [$$U] of var $$E: deopt(array [$$U] of var opt $$E: x)

10.  function array [$$U] of var float: deopt(array [$$U] of var opt float:␣
↪x)
```

1. Return value of x if x is not absent. Returns undefined when evaluated on absent value.
2. Return value of x if x is not absent. Returns undefined when evaluated on absent value.

3, 4.
Return array of the value of x[i] where x[i] is not absent. Returns undefined when evaluated with an absent element.

5-7.
Return value x unchanged (since x is guaranteed to be non-optional).

8-10.
Return array of the value of x[i] (assumes that x[i] is not absent).

```
1.  test occurs($T: x)

2.  test occurs(var $T: x)

3.  test occurs(set of $$T: x)

4.  test occurs(var set of $$T: x)

5.  test occurs(opt $T: x)
```

1-4.
Test if x is not absent (always returns true)

5. Test if x is not absent

4.2.1.4 Array sorting operations

```
function array [int] of $$E: arg_sort(array [$$E] of $$T: xs)
function array [int] of $$E: arg_sort(array [$$E] of float: xs)
```

Returns the permutation `permutation` which causes `xs` to be in sorted order hence `xs[permutation[i]] <= xs[permutation[i+1]]`.

The permutation is the stable sort hence `xs[permutation[i]] = xs[permutation[i+1]]` → `permutation[i] < permutation[i+1]`.

```
function array [$$E] of $$F: sort(array [$$E] of $$F: xs)
function array [$$E] of float: sort(array [$$E] of float: xs)
function array [$$E] of bool: sort(array [$$E] of bool: xs)
```

Return values from array `xs` sorted in non-decreasing order

```
function array [$$E] of any $T: sort_by(array [$$E] of any $T: x,
                                        array [$$E] of int: y)
function array [$$E] of any $T: sort_by(array [$$E] of any $T: x,
                                        array [$$E] of float: y)
```

Return array x sorted by the values in y in non-decreasing order

The sort is stable, i.e. if `y[i] = y[j]` with `i < j`, then `x[i]` will appear in the output before `x[j]`.

4.2.1.5 Language information

These functions return information about the MiniZinc system.

Constants

```
opt int: mzn_max_version_required
```

If defined, this can be used to check that the MiniZinc compiler is not newer than the given version.

```
opt int: mzn_min_version_required
```

If defined, this can be used to check that the MiniZinc compiler supports all the features used in the model.

Functions and Predicates

```
function int: mzn_compiler_version()
```

Return MiniZinc version encoded as an integer (major*10000+minor*1000+patch).

```
function string: mzn_version_to_string(int: v)
```

Return string representation of `v` given an integer major*10000+minor*1000+patch

---

4.2.1.6 Compiler options

Constants

```
opt bool: mzn_absent_zero
```

Whether to use zero as the underlying representation for absent optional variables.

If not specified or set to true, zero is used as the underlying representation for absent optional variables, breaking symmetry. If set to false, the underlying representation of absent optional variables is left unconstrained.

```
opt bool: mzn_half_reify_clause
```

Whether to half reify bool_clause constraints

```
opt bool: mzn_ignore_redundant_constraints
```

Whether to ignore redundant constraints

If not specified or set to false, it depends on the solver library whether constraints that are wrapped in a redundant_constraint call are in fact compiled. If set to true, they are not compiled, independent of the solver library.

```
opt bool: mzn_ignore_symmetry_breaking_constraints
```

Whether to ignore symmetry breaking constraints

If not specified or set to false, it depends on the solver library whether constraints that are wrapped in a symmetry_breaking_constraint call are in fact compiled. If set to true, they are not compiled, independent of the solver library.

```
opt bool: mzn_opt_annotate_computed_domains
```

Whether to generate computed_domain annotations

```
opt bool: mzn_opt_annotate_defines_var
```

Whether to generate defines_var annotations

```
opt bool: mzn_opt_only_range_domains
```

Whether to only generate domains that are contiguous ranges

Functions and Predicates

```
test mzn_check_absent_zero()
```

Check whether to use zero as the underlying representation of absent optional variables

---

```
test mzn_check_annotate_computed_domains()
```

Check whether to generate computed_domain annotations

```
test mzn_check_annotate_defines_var()
```

Check whether to generate defines_var annotations

```
test mzn_check_half_reify_clause()
```

Check to half reify bool_clause constraints

```
test mzn_check_ignore_redundant_constraints()
```

Check whether to ignore redundant constraints

```
test mzn_check_ignore_symmetry_breaking_constraints()
```

Check whether to ignore symmetry breaking constraints

```
test mzn_check_only_range_domains()
```

Check whether to only generate domains that are contiguous ranges

---

4.2.1.7 Conditionals

These functions implement conditional (if-then-else-endif) constraints.

```
predicate if_then_else(array [int] of var bool: c,
                       array [int] of int: x,
                       var int: y)
predicate if_then_else(array [int] of var bool: c,
                       array [int] of var int: x,
                       var int: y)
predicate if_then_else(array [int] of var bool: c,
                       array [int] of opt int: x,
                       var opt int: y)
predicate if_then_else(array [int] of var bool: c,
```

```
                        array [int] of var opt int: x,
                        var opt int: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of bool: x,
                        var bool: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of var bool: x,
                        var bool: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of opt bool: x,
                        var opt bool: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of var opt bool: x,
                        var opt bool: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of float: x,
                        var float: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of var float: x,
                        var float: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of opt float: x,
                        var opt float: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of var opt float: x,
                        var opt float: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of set of int: x,
                        var set of int: y)
predicate if_then_else(array [int] of var bool: c,
                        array [int] of var set of int: x,
                        var set of int: y)
```

Conditional constraint $\{\mathbf{c}[i] \wedge \nexists \mathbf{c}[1..i-1] \rightarrow \mathbf{y} = \mathbf{x}[i]\}$

This constraint is generated by the compiler for if-then-else expressions. The last entry in the `c` array is always the constant true, corresponding to the else case.

```
predicate if_then_else_partiality(array [int] of var bool: c,
                                  array [int] of var bool: d,
                                  var bool: b)
```

Conditional partiality constraint

This constraint is generated by the compiler for if-then-else expressions with potentially undefined cases. The last entry in the `c` array is always the constant true, corresponding to the else case. The `d`[i] variable represents whether case `i` is defined. Constrains that if `b` is defined, then the selected case must be defined, and if the selected case is undefined, then `b` must be undefined.

4.2.1.8 Reflection operations

These functions return information about declared or inferred variable bounds and domains.

```
test annotate(any $T: x, ann: a)
```

Annotate declaration of `x` with annotation `a`

```
function set of $$E: dom(var $$E: x)
function set of $$E: dom(var opt $$E: x)
```

Return domain of `x`

```
function set of $$E: dom_array(array [$T] of var $$E: x)
function set of $$E: dom_array(array [$T] of var opt $$E: x)
```

Return union of all domains of the elements in array `x`

```
function set of $$E: dom_array_occurring(array [$T] of var opt $$E: x)
```

Return union of all domains of the elements in array `x`, ignoring absent elements

---

```
function set of $$E: dom_bounds_array(array [$T] of var $$E: x)
function set of $$E: dom_bounds_array(array [$T] of var opt $$E: x)
```

Return approximation of union of all domains of the elements in array `x`

```
function int: dom_size(var int: x)
```

Return cardinality of the domain of `x`

```
1.  function $T: fix(var $T: x)

2.  function opt $T: fix(var opt $T: x)

3.  function array [$U] of $T: fix(array [$U] of var $T: x)
```

```
4.  function array [$U] of opt $T: fix(array [$U] of var opt $T: x)
```

1, 2.
Check if the value of `x` is fixed at this point in evaluation. If it is fixed, return its value, otherwise abort.

3, 4.
Check if the value of every element of the array `x` is fixed at this point in evaluation. If all are fixed, return an array of their values, otherwise abort.

```
test has_ann(any $T: x, ann: a)
```

Test if `x` is annotated `a`

```
test has_bounds(var $$E: x)
test has_bounds(var float: x)
```

Test if variable `x` has declared, finite bounds

```
test has_ub_set(var set of $$E: x)
```

Test if variable `x` has a declared, finite upper bound

```
1.  test is_fixed(any $T: x)

2.  test is_fixed(array [$U] of any $T: x)
```

1. Test if `x` is fixed
2. Test if every element of array `x` is fixed

```
test is_same(any $T: x, any $U: y)
```

Test if `x` and `y` are the same variable

```
1.  function $$E: lb(var $$E: x)

2.  function $$E: lb(var opt $$E: x)
```

```
3.  function float: lb(var float: x)
4.  function float: lb(var opt float: x)
5.  function set of $$E: lb(var set of $$E: x)
6.  function array [$U] of $$E: lb(array [$U] of var $$E: x)
7.  function array [$U] of float: lb(array [$U] of var float: x)
8.  function array [$U] of set of $$E: lb(array [$U] of var set of $$E: x)
```

1-5.
Return lower bound of `x`

6-8.
Return array of lower bounds of the elements in array `x`

```
1.  function $$E: lb_array(array [$U] of var $$E: x)
2.  function $$E: lb_array(array [$U] of var opt $$E: x)
3.  function float: lb_array(array [$U] of var float: x)
4.  function float: lb_array(array [$U] of var opt float: x)
5.  function set of $$E: lb_array(array [$U] of var set of $$E: x)
```

1-4.
Return minimum of all lower bounds of the elements in array `x`

5. Return intersection of all lower bounds of the elements in array `x`

```
1.  function $$E: ub(var $$E: x)
2.  function $$E: ub(var opt $$E: x)
3.  function float: ub(var float: x)
4.  function float: ub(var opt float: x)
5.  function set of $$E: ub(var set of $$E: x)
6.  function array [$U] of $$E: ub(array [$U] of var $$E: x)
7.  function array [$U] of float: ub(array [$U] of var float: x)
```

```
8.  function array [$U] of set of $$E: ub(array [$U] of var set of $$E: x)
```

1-5.
Return upper bound of `x`

6-8.
Return array of upper bounds of the elements in array `x`

```
1.  function $$E: ub_array(array [$U] of var $$E: x)

2.  function $$E: ub_array(array [$U] of var opt $$E: x)

3.  function float: ub_array(array [$U] of var float: x)

4.  function float: ub_array(array [$U] of var opt float: x)

5.  function set of $$E: ub_array(array [$U] of var set of $$E: x)
```

1-4.
Return maximum of all upper bounds of the elements in array `x`

5. Return union of all upper bounds of the elements in array `x`

4.2.1.9 Assertions and debugging functions

These functions help debug models and check that input data conforms to the expectations.

Constants

```
bool: debug_mode
```

Set to true iff `--debug` flag is set.

Functions and Predicates

```
test abort(string: msg)
```

Abort evaluation and print message `msg`.

```
1.  function any $T: assert(bool: b, string: msg, any $T: x)
```

```
2.  function array [$U] of any $T: assert(bool: b,
                                          string: msg,
                                          array [$U] of any $T: x)

3.  test assert(bool: b, string: msg)
```

1, 2.
If `b` is true, return `x`, otherwise abort with message `msg`.

3. If `b` is true, return true, otherwise abort with message `msg`.

```
test assert_dbg(bool: b, string: msg)
```

Return true, when the `--debug` flag is set, check if `b` is true, otherwise abort with message `msg`.

```
 1.  function $T: 'default'(opt $T: x, $T: y)

 2.  function opt $T: 'default'(opt $T: x, opt $T: y)

 3.  function var $T: 'default'(var opt $T: x, var $T: y)

 4.  function var opt $T: 'default'(var opt $T: x, var opt $T: y)

 5.  function array [$U] of $T: 'default'(array [$U] of $T: x,
                                          array [$U] of $T: y)

 6.  function array [$U] of opt $T: 'default'(array [$U] of opt $T: x,
                                              array [$U] of opt $T: y)

 7.  function array [$U] of var $T: 'default'(array [$U] of var $T: x,
                                              array [$U] of var $T: y)

 8.  function array [$U] of var opt $T: 'default'(array [$U] of var opt $T:␣
↪x,
                                                  array [$U] of var opt $T:␣
↪y)

 9.  function set of $$E: 'default'(set of $$E: x, set of $$E: y)

10.  function var set of $$E: 'default'(var set of $$E: x,
                                        var set of $$E: y)

11.  function set of $T: 'default'(opt set of $T: x, set of $T: y)

12.  function opt set of $T: 'default'(opt set of $T: x, opt set of $T: y)
```

1-4.
If `x` is defined and not absent, return `x`, otherwise return `y`.

5-10.
If `x` is defined, return `x`, otherwise return `y`.

11, 12.
If `x` is define amd mpt absent, return `x`, otherwise return `y`.

```
annotation expression_name_dbg(string: s)
```

Used to attach a `expression_name` annotation with name `s` to an expression when –debug is enabled.

```
function string: logstream_to_string()
```

Return logging stream as string

```
test mzn_internal_check_debug_mode()
```

Returns true iff `--debug` flag is set.

```
1.  function any $T: trace(string: msg, any $T: x)

2.  function array [$U] of any $T: trace(string: msg,
                                         array [$U] of any $T: x)

3.  test trace(string: msg)
```

1, 2.
Return `x`, and print message `msg`.

3. Return true, and print message `msg`.

```
test trace_dbg(string: msg)
```

Return true, and when `--debug` flag is set print message `msg`.

```
function any $T: trace_exp(any $T: x)
function array [$U] of any $T: trace_exp(array [$U] of any $T: x)
```

Return `x`, and print a string representation of it.

```
1.  function any $T: trace_logstream(string: msg, any $T: x)

2.  function array [$U] of any $T: trace_logstream(string: msg,
                                                   array [$U] of any $T: x)

3.  test trace_logstream(string: msg)
```

1, 2.
Return `x`, and print message `msg` to logging stream.

3. Return true, and print message `msg` to logging stream.

```
1.  function any $T: trace_stdout(string: msg, any $T: x)

2.  function array [$U] of any $T: trace_stdout(string: msg,
                                                array [$U] of any $T: x)

3.  test trace_stdout(string: msg)
```

1, 2.
Return `x`, and print message `msg`.

3. Return true, and print message `msg`.

```
test trace_to_json_section(string: section, opt $T: x)
test trace_to_json_section(string: section, array [$U] of opt $T: x)
```

Return true, and print `x` to the JSON section `section`.

```
1.  function any $T: trace_to_section(string: section,
                                      string: msg,
                                      any $T: x)

2.  function array [$U] of any $T: trace_to_section(string: section,
                                                    string: msg,
                                                    array [$U] of any $T: x)

3.  test trace_to_section(string: section, string: msg)
```

1, 2.
Return `x`, and print message `msg` to section `section`.

3. Return true, and print message `msg` to section `section`.

Annotations

```
annotation mzn_break_here
```

With debug build of the MiniZinc compiler, call MiniZinc::mzn_break_here when flattening this expression to make debugging easier. This annotation is ignored by the release build.

4.2.1.10 Random Number Generator builtins

These functions implement random number generators from different probability distributions.

```
test bernoulli(float: p)
```

Return a boolean sample from the Bernoulli distribution defined by probability **p**

```
function int: binomial(int: t, float: p)
```

Return a sample from the binomial distribution defined by sample number `t` and probability `p`

```
function float: cauchy(float: mean, float: scale)
function float: cauchy(int: mean, float: scale)
```

Return a sample from the cauchy distribution defined by **mean**, **scale**

```
function float: chisquared(int: n)
function float: chisquared(float: n)
```

Return a sample from the chi-squared distribution defined by the degree of freedom **n**

```
function int: discrete_distribution(array [int] of int: weights)
```

Return a sample from the discrete distribution defined by the array of weights **weights** that assigns a weight to each integer starting from zero

---

```
function float: exponential(int: lambda)
function float: exponential(float: lambda)
```

Return a sample from the exponential distribution defined by **lambda**

```
function float: fdistribution(float: d1, float: d2)
function float: fdistribution(int: d1, int: d2)
```

Return a sample from the Fisher-Snedecor F-distribution defined by the degrees of freedom **d1**, **d2**

```
function float: gamma(float: alpha, float: beta)
function float: gamma(int: alpha, float: beta)
```

Return a sample from the gamma distribution defined by **alpha**, **beta**

```
function float: lognormal(float: mean, float: std)
function float: lognormal(int: mean, float: std)
```

Return a sample from the lognormal distribution defined by **mean**, **std**

```
function float: normal(float: mean, float: std)
function float: normal(int: mean, float: std)
```

Return a sample from the normal distribution defined by **mean**, **std**

```
1.  function int: poisson(float: mean)

2.  function int: poisson(int: mean)
```

1. Return a sample from the poisson distribution defined by `mean`
2. Return a sample from the poisson distribution defined by an integer `mean`

```
function float: tdistribution(float: n)
function float: tdistribution(int: n)
```

Return a sample from the student's t-distribution defined by the sample size **n**

```
1.  function float: uniform(float: lowerbound, float: upperbound)

2.  function int: uniform(int: lowerbound, int: upperbound)

3.  function int: uniform(set of int: S)
```

1, 2.
Return a sample from the uniform distribution defined by **lowerbound**, **upperbound**

3. Return a sample from the uniform distribution defined by `S`

```
function float: weibull(float: shape, float: scale)
function float: weibull(int: shape, float: scale)
```

Return a sample from the Weibull distribution defined by **shape**, **scale**

4.2.1.11 Structured Output

These functions provide structured output for common structures modelled in the MiniZinc language.

```
function string: show_array2d_bool(array [int,int] of var bool: x)
```

Create string to visualise a 2D boolean array `x` as a checkered board.

```
1.  function string: show_gantt(array [$$T] of var int: start,
                                array [$$T] of var int: dur,
                                array [$$T] of string: name)

2.  function string: show_gantt(array [$$T] of var int: start,
                                array [$$T] of var int: dur)
```

1. Create string Gantt chart from task `start` times, duration `dur`, and their names `name`.
2. Create string Gantt chart from task `start` times and duration `dur`, using the index set as name for the tasks.

4.2.2 Global constraints

These constraints represent high-level modelling abstractions. Many solvers implement dedicated, efficient inference algorithms for these constraints, or provide a MiniZinc

library decomposition that is better suited to the particular solving technology than the standard library decomposition.

4.2.2.1 All-Different and related constraints

```
1.  predicate all_different(array [$X] of var $$E: xs, set of $$E: except)

2.  predicate all_different(array [$X] of var set of $$E: xs)

3.  predicate all_different(array [$X] of var opt $$E: xs)
```

1. Constrain the elements in the array `xs` to be pairwise different, except for those that take a value in the set `except` (values in `except` may occur any number of times).
2. Constrain the elements in the array `xs` to be pairwise different.
3. Constrain the non-absent elements in the array `xs` to be pairwise different. The absent value <> is allowed to occur multiple times.

```
predicate all_different_except(array [$X] of var $$E: xs,
                               set of $$E: except)
```

Constrain the elements of the array `xs` to be pairwise different except for those values that appear in the set `except`.

This constraint is deprecated. Use all_different(`xs`, `except`) instead.

```
predicate all_different_except_0(array [$X] of var int: xs)
```

Constrain the elements of the array `xs` to be pairwise different except for those elements that are assigned the value 0.

This constraint is deprecated. Use all_different(`xs`, {0}) instead.

```
predicate all_disjoint(array [$X] of var set of $$E: xs)
```

Constrain the array of sets of integers `xs` to be pairwise disjoint.

```
predicate all_equal(array [$X] of var $$E: xs)
predicate all_equal(array [$X] of var set of $$E: xs)
```

Constrain the elements of the array `xs` to be all equal.

```
1.  predicate nvalue(var int: n, array [$X] of var $$E: xs)

2.  predicate nvalue(var int: n, array [$X] of var opt $$E: xs)

3.  function var int: nvalue(array [$X] of var $$E: xs)

4.  function var int: nvalue(array [$X] of var opt $$E: xs)
```

1, 2.
Requires that the number of distinct values in `xs` is `n`.

3, 4.
Returns the number of distinct values in `xs`.

```
predicate symmetric_all_different(array [int] of var int: xs)
```

Requires the array of integers `xs` to be all different, and for all `i`, `xs[i]=j` → `xs[j]=i`.

4.2.2.2 Lexicographic constraints

```
predicate lex2(array [$$E,$$F] of var $$T: xs)
```

Require adjacent rows and adjacent columns in the array `xs` to be lexicographically ordered. Adjacent rows and adjacent columns may be equal.

```
predicate lex2_strict(array [$$E,$$F] of var $$T: xs)
```

Require adjacent rows and adjacent columns in the array `xs` to be lexicographically ordered. Adjacent rows and adjacent columns cannot be equal.

```
predicate lex_chain(array [$$E,$$F] of var bool: xs)
predicate lex_chain(array [$$E,$$F] of var $$T: xs)
```

Requires that the columns of matrix `xs` are lexicographically sorted, non-decreasing.

```
predicate lex_chain_greater(array [$$E,$$F] of var bool: xs)
predicate lex_chain_greater(array [$$E,$$F] of var $$T: xs)
```

Requires that the columns of matrix `xs` are lexicographically sorted, strictly decreasing.

```
predicate lex_chain_greatereq(array [$$E,$$F] of var bool: xs)
predicate lex_chain_greatereq(array [$$E,$$F] of var $$T: xs)
```

Requires that the columns of matrix `xs` are lexicographically sorted, non-increasing.

```
predicate lex_chain_greatereq_orbitope(array [$$E,$$F] of var int: xs,
                                       int: _kind)
```

Requires that the columns of binary matrix `xs` are lexicographically sorted, non-increasing. Moreover, the second parameter `_kind` has the following meaning: 0: no further constraints, 1: set-partitioning orbitope, 2: set-packing orbitope

---

```
predicate lex_chain_less(array [$$E,$$F] of var bool: xs)
predicate lex_chain_less(array [$$E,$$F] of var $$T: xs)
```

Requires that the columns of matrix `xs` are lexicographically sorted, strictly increasing.

```
predicate lex_chain_lesseq(array [$$E,$$F] of var bool: xs)
predicate lex_chain_lesseq(array [$$E,$$F] of var $$T: xs)
```

Requires that the columns of matrix `xs` are lexicographically sorted, non-decreasing.

```
predicate lex_chain_lesseq_orbitope(array [$$E,$$F] of var int: xs,
                                    int: _kind)
```

Requires that the columns of binary matrix `xs` are lexicographically sorted, non-decreasing. Moreover, the second parameter `_kind` has the following meaning: 0: no further constraints, 1: set-partitioning orbitope, 2: set-packing orbitope

---

```
predicate lex_greater(array [$$E] of var bool: xs,
                      array [$$F] of var bool: ys)
predicate lex_greater(array [$$E] of var $$T: xs,
                      array [$$F] of var $$T: ys)
predicate lex_greater(array [$$E] of var float: xs,
                      array [$$F] of var float: ys)
predicate lex_greater(array [$$E] of var set of $$T: xs,
                      array [$$F] of var set of $$T: ys)
```

Requires that the array `xs` is strictly lexicographically greater than array `ys`. Compares them from first to last element, regardless of indices.

```
predicate lex_greatereq(array [$$E] of var bool: xs,
                        array [$$F] of var bool: ys)
predicate lex_greatereq(array [$$E] of var $$T: xs,
                        array [$$F] of var $$T: ys)
predicate lex_greatereq(array [$$E] of var float: xs,
                        array [$$F] of var float: ys)
predicate lex_greatereq(array [$$E] of var set of $$T: xs,
                        array [$$F] of var set of $$T: ys)
```

Requires that the array `xs` is lexicographically greater than or equal to array `ys`. Compares them from first to last element, regardless of indices.

```
predicate lex_less(array [$$E] of var bool: xs,
                   array [$$F] of var bool: ys)
predicate lex_less(array [$$E] of var $$T: xs,
                   array [$$F] of var $$T: ys)
predicate lex_less(array [$$E] of var float: xs,
                   array [$$F] of var float: ys)
predicate lex_less(array [$$E] of var set of $$T: xs,
                   array [$$F] of var set of $$T: ys)
```

Requires that the array `xs` is strictly lexicographically less than array `ys`. Compares them from first to last element, regardless of indices.

```
predicate lex_lesseq(array [$$E] of var bool: xs,
                     array [$$F] of var bool: ys)
predicate lex_lesseq(array [$$E] of var float: xs,
                     array [$$F] of var float: ys)
predicate lex_lesseq(array [$$E] of var $$T: xs,
                     array [$$F] of var $$T: ys)
predicate lex_lesseq(array [$$E] of var set of $$T: xs,
                     array [$$F] of var set of $$T: ys)
```

Requires that the array `xs` is lexicographically less than or equal to array `ys`. Compares them from first to last element, regardless of indices.

```
1.  predicate seq_precede_chain(array [int] of var int: xs)

2.  predicate seq_precede_chain(array [int] of var set of int: xs)
```

1. Requires that `i` precedes `i`+1 in the array `xs` for all positive `i`.
2. Requires that `i` appears in a set in array `xs` before `i`+1 for all positive `i`

```
predicate strict_lex2(array [$$E,$$F] of var $$T: xs)
```

Requires adjacent rows and adjacent columns in the array `xs` to be lexicographically ordered. Adjacent rows and adjacent columns cannot be equal.

```
1. predicate value_precede(
       $$E: before,
       $$E: after,
       array [$$X] of var $$E: xs)

2. predicate value_precede(
       $$E: before,
       $$E: after,
       array [$$X] of var opt $$E: xs)

3. predicate value_precede(
       $$E: before,
       $$E: after,
       array [$$X] of var set of $$E: xs)
```

1, 2.

Requires that `before` precede `after` in the array `xs`.

Precedence means that if any element of `xs` is equal to `after`, then another element of `xs` with a lower index is equal to `before`.

3. Requires that `before` precede `after` in the array `xs`.

   Precedence means that if an element of `xs` contains `after` but not `before`, then another element of `xs` with lower index contains `before` but not `after`.

```
1. predicate value_precede_chain(
       array [$$X] of $$E: order,
       array [$$Y] of var $$E: xs)

2. predicate value_precede_chain(
       array [$$X] of $$E: order,
       array [$$Y] of var opt $$E: xs)

3. predicate value_precede_chain(
       array [$$X] of $$E: order,
       array [$$Y] of var set of $$E: xs)
```

1, 2.

Requires that `order[i]` precedes `order[i +1]` in the array `xs`.

Precedence means that if any element of `xs` is equal to `order[i +1]`, then another element of `xs` with a lower index is equal to `order[i]`.

3. Requires that `order[i]` precedes `order[i +1]` in the array `xs`.

   Precedence means that if an element of `xs` contains `order[i +1]` but not `order[i]`, then another element of `xs` with lower index contains `order[i]` but not `order[i +1]`.

```
predicate var_perm_sym(array [$$X] of var $$Y: xs,
                       array [$$Z,$$X] of $$X: permutations)
```

Requires that the array `xs` is lex least under the given list of permutations `permutations`

```
predicate var_sqr_sym(array [$$X,$$X] of var $$Y: xs)
```

Requires that the square array `xs` is lex least under the 7 symmetries: 3 rotations and 4 reflections

---

4.2.2.3 Sorting constraints

```
1.  predicate arg_sort(array [$$E] of var $$T: xs,
                       array [int] of var $$E: permutation)

2.  predicate arg_sort(array [$$E] of var float: xs,
                       array [int] of var $$E: permutation)

3.  function array [int] of var $$E: arg_sort(array [$$E] of var $$T: xs)

4.  function array [int] of var $$E: arg_sort(array [$$E] of var float: xs)
```

1, 2.

Constrains `permutation` to be the permutation which causes `xs` to be in sorted order hence `xs[permutation[i]] <= xs[permutation[i+1]]`.

The permutation is the stable sort hence `xs[permutation[i]] = xs[permutation[i+1]]` → `permutation[i] < permutation[i+1]`.

3, 4.

Returns the permutation `permutation` which causes `xs` to be in sorted order hence `xs[permutation[i]] <= xs[permutation[i+1]]`.

The permutation is the stable sort hence `xs[permutation[i]] = xs[permutation[i+1]]` → `permutation[i] < permutation[i+1]`.

```
predicate decreasing(array [$X] of var bool: xs)
predicate decreasing(array [$X] of var opt float: xs)
predicate decreasing(array [$X] of var opt $$E: xs)
predicate decreasing(array [$X] of var set of $$E: xs)
```

Requires that the array `xs` is in decreasing order (duplicates are allowed).

```
predicate increasing(array [$X] of var bool: xs)
predicate increasing(array [$X] of var float: xs)
predicate increasing(array [$X] of var opt float: xs)
predicate increasing(array [$X] of var $$E: xs)
predicate increasing(array [$X] of var opt $$E: xs)
predicate increasing(array [$X] of var set of $$E: xs)
```

Requires that the array `xs` is in increasing order (duplicates are allowed).

```
1.  predicate sort(array [$$E] of var $$T: xs, array [$$F] of var $$T: ys)

2.  function array [$$E] of var $$T: sort(array [$$E] of var $$T: xs)
```

1. Requires that the multiset of values in `xs` are the same as the multiset of values in `ys` but `ys` is in sorted order.
2. Return a multiset of values that is the same as the multiset of values in `xs` but in sorted order.

```
predicate strictly_decreasing(array [$X] of var opt $$E: xs)
predicate strictly_decreasing(array [$X] of var opt float: xs)
predicate strictly_decreasing(array [$X] of var set of $$E: xs)
```

Requires that the array `xs` is in a strictly decreasing order (duplicates are *not* allowed).

```
1.  predicate strictly_increasing(array [$X] of var $$E: xs)

2.  predicate strictly_increasing(array [$X] of var opt $$E: xs)

3.  predicate strictly_increasing(array [$X] of var float: xs)

4.  predicate strictly_increasing(array [$X] of var opt float: xs)

5.  predicate strictly_increasing(array [$X] of var set of $$E: xs)
```

1, 2.
Requires that the array `xs` is in a strictly increasing order (duplicates are *not* allowed).

3-5.
Requires that the array `xs` is in increasing order (duplicates are allowed).

4.2.2.4 Channeling constraints

```
predicate int_set_channel(array [$$I] of var $$J: ints,
                          array [$$J] of var set of $$I: sets)
```

Requires that array of int variables `ints` and array of set variables `sets` are related such that (`ints`[`i`] = `j`) ↔ (`i` in `sets`[`j`]).

```
1.  predicate inverse(array [$$X] of var $$Y: f,
                      array [$$Y] of var $$X: invf)

2.  function array [$$E] of var $$F: inverse(array [$$F] of var $$E: f)

3.  function array [$$E] of var opt $$F: inverse(array [$$F] of var opt $$E:␣
↪f)

4.  function array [$$E] of $$F: inverse(array [$$F] of $$E: f)
```

1. Constrains two arrays of int variables, `f` and `invf`, to represent inverse functions. All the values in each array must be within the index set of the other array.

2-4.
Given a function `f` represented as an array, return the inverse function.

```
predicate inverse_in_range(array [$$A] of var $$B: xs,
                           array [$$B] of var $$A: ys)
```

If the `i` th variable of the collection `xs` is assigned to `j` and if `j` is in the index set of `ys` then the `j` th variable of the collection `ys` is assigned to `i`.

Conversely, if the `j` th variable of the collection `ys` is assigned to `i` and if `i` is in the index set of `xs` then the `i` th variable of the collection `xs` is assigned to `j`.

```
predicate inverse_set(array [$$X] of var set of $$Y: f,
                      array [$$Y] of var set of $$X: invf)
```

Constrains two arrays of set of int variables, `f` and `invf`, so that a `j` in f[`i`] iff `i` in invf[`j`]. All the values in each array's sets must be within the index set of the other array.

```
predicate link_set_to_booleans(var set of $$E: s,
                               array [$$E] of var bool: bs)
```

Constrains the array of Booleans `bs` to be a representation of the set `s`: `i` in `s` ↔ `bs`[`i`].

The index set of `bs` must be a superset of the possible values of `s`.

4.2.2.5 Counting constraints

These constraints count and restrict how many times certain values occur in an array of variables. MiniZinc will automatically generate the basic counting constraints below from expressions such as `count(i in x)(i=c) <= d`, so you can write models in this much more readable style instead of using these predicates. However, if your model contains multiple counting constraints over the same array, constraints like `distribute` or `global_cardinality` below may be useful.

```
1.  predicate among(var int: n,
                    array [$X] of var $$E: xs,
                    set of $$E: values)

2.  function var int: among(array [$X] of var $$E: xs, set of $$E: values)

3.  function int: among(array [$X] of $$E: xs, set of $$E: values)
```

1. Requires exactly `n` variables in `xs` to take one of the values in `values`.

2, 3.
Returns the number of variables in `xs` that take one of the values in `values`.

```
predicate at_least(int: n,
                   array [$X] of var set of $$E: xs,
                   set of $$E: value)
```

Requires at least `n` variables in `xs` to take the value `value`.

```
predicate at_most(int: n,
                  array [$X] of var set of $$E: xs,
                  set of $$E: value)
```

Requires at most `n` variables in `xs` to take the value `value`.

```
predicate at_most1(array [$X] of var set of $$E: xs)
```

Requires that each pair of sets in `xs` overlap in at most one element.

```
1.  function var int: count(array [$X] of var opt $$E: xs, var $$E: value)
```

```
2.  predicate count(array [$X] of var opt $$E: xs,
                    var $$E: value,
                    var int: n)
```

1. Returns the number of occurrences of `value` in `xs`.
2. Constrains `n` to be the number of occurrences of `value` in `xs`.

```
 1.  predicate count_eq(array [$X] of var $$E: xs,
                        var $$E: value,
                        var int: n)

 2.  predicate count_eq(array [$X] of var bool: xs,
                        var bool: value,
                        var int: n)

 3.  predicate count_eq(array [$X] of var float: xs,
                        var float: value,
                        var int: n)

 4.  predicate count_eq(array [$X] of var set of $$E: xs,
                        var set of $$E: value,
                        var int: n)

 5.  predicate count_eq(array [$X] of var opt $$E: xs,
                        var $$E: value,
                        var int: n)

 6.  predicate count_eq(array [$X] of var opt bool: xs,
                        var bool: value,
                        var int: n)

 7.  predicate count_eq(array [$X] of var $$E: xs, $$E: value, int: n)

 8.  predicate count_eq(array [$X] of var bool: xs, bool: value, int: n)

 9.  predicate count_eq(array [$X] of var float: xs, float: value, int: n)

10.  predicate count_eq(array [$X] of var set of $$E: xs,
                        set of $$E: value,
                        int: n)

11.  function var int: count_eq(array [$X] of var opt $$E: xs,
                                var $$E: value)
```

1-10.
Constrains `n` to be the number of occurrences of `value` in `xs`.

11. Returns the number of occurrences of `value` in `xs`.

```
1.  predicate count_geq(array [$X] of var $$E: xs,
                        var $$E: value,
                        var int: n)

2.  predicate count_geq(array [$X] of var bool: xs,
                        var bool: value,
                        var int: n)

3.  predicate count_geq(array [$X] of var float: xs,
                        var float: value,
                        var int: n)

4.  predicate count_geq(array [$X] of var set of $$E: xs,
                        var set of $$E: value,
                        var int: n)

5.  predicate count_geq(array [$X] of var opt $$E: xs,
                        var $$E: value,
                        var int: n)

6.  predicate count_geq(array [$X] of var $$E: xs, $$E: value, int: n)

7.  predicate count_geq(array [$X] of var bool: xs, bool: value, int: n)

8.  predicate count_geq(array [$X] of var float: xs, float: value, int: n)

9.  predicate count_geq(array [$X] of var set of $$E: xs,
                        set of $$E: value,
                        int: n)
```

1-4.
Constrains `n` to be greater than or equal to the number of occurrences of `value` in `xs`.

5. Constrains `n` to be strictly greater than the number of occurrences of `value` in `xs`.

6-9.
Constrains `n` to be greater than or equal to the number of occurrences of `value` in `xs`.

```
predicate count_gt(array [$X] of var $$E: xs,
                   var $$E: value,
                   var int: n)
predicate count_gt(array [$X] of var bool: xs,
                   var bool: value,
                   var int: n)
predicate count_gt(array [$X] of var float: xs,
                   var float: value,
```

```
                    var int: n)
predicate count_gt(array [$X] of var set of $$E: xs,
                    var set of $$E: value,
                    var int: n)
predicate count_gt(array [$X] of var opt $$E: xs,
                    var $$E: value,
                    var int: n)
predicate count_gt(array [$X] of var $$E: xs, $$E: value, int: n)
predicate count_gt(array [$X] of var bool: xs, bool: value, int: n)
predicate count_gt(array [$X] of var float: xs, float: value, int: n)
predicate count_gt(array [$X] of var set of $$E: xs,
                    set of $$E: value,
                    int: n)
```

Constrains `n` to be strictly greater than the number of occurrences of `value` in `xs`.

```
predicate count_leq(array [$X] of var $$E: xs,
                     var $$E: value,
                     var int: n)
predicate count_leq(array [$X] of var bool: xs,
                     var bool: value,
                     var int: n)
predicate count_leq(array [$X] of var float: xs,
                     var float: value,
                     var int: n)
predicate count_leq(array [$X] of var set of $$E: xs,
                     var set of $$E: value,
                     var int: n)
predicate count_leq(array [$X] of var opt $$E: xs,
                     var $$E: value,
                     var int: n)
predicate count_leq(array [$X] of var $$E: xs, $$E: value, int: n)
predicate count_leq(array [$X] of var bool: xs, bool: value, int: n)
predicate count_leq(array [$X] of var float: xs, float: value, int: n)
predicate count_leq(array [$X] of var set of $$E: xs,
                     set of $$E: value,
                     int: n)
```

Constrains `n` to be less than or equal to the number of occurrences of `value` in `xs`.

```
predicate count_lt(array [$X] of var $$E: xs,
                    var $$E: value,
                    var int: n)
predicate count_lt(array [$X] of var bool: xs,
                    var bool: value,
                    var int: n)
```

```
predicate count_lt(array [$X] of var float: xs,
                   var float: value,
                   var int: n)
predicate count_lt(array [$X] of var set of $$E: xs,
                   var set of $$E: value,
                   var int: n)
predicate count_lt(array [$X] of var opt $$E: xs,
                   var $$E: value,
                   var int: n)
predicate count_lt(array [$X] of var $$E: xs, $$E: value, int: n)
predicate count_lt(array [$X] of var bool: xs, bool: value, int: n)
predicate count_lt(array [$X] of var float: xs, float: value, int: n)
predicate count_lt(array [$X] of var set of $$E: xs,
                   set of $$E: value,
                   int: n)
```

Constrains `n` to be strictly less than the number of occurrences of `value` in `xs`.

```
predicate count_neq(array [$X] of var $$E: xs,
                    var $$E: value,
                    var int: n)
predicate count_neq(array [$X] of var bool: xs,
                    var bool: value,
                    var int: n)
predicate count_neq(array [$X] of var float: xs,
                    var float: value,
                    var int: n)
predicate count_neq(array [$X] of var set of $$E: xs,
                    var set of $$E: value,
                    var int: n)
predicate count_neq(array [$X] of var opt $$E: xs,
                    var $$E: value,
                    var int: n)
predicate count_neq(array [$X] of var $$E: xs, $$E: value, int: n)
predicate count_neq(array [$X] of var bool: xs, bool: value, int: n)
predicate count_neq(array [$X] of var float: xs, float: value, int: n)
predicate count_neq(array [$X] of var set of $$E: xs,
                    set of $$E: value,
                    int: n)
```

Constrains `n` to be not equal to the number of occurrences of `value` in `xs`.

```
1.  predicate distribute(array [$X] of var int: count,
                         array [$X] of var $$E: cover,
                         array [$Y] of var $$E: xs)
```

```
2.  function array [$X] of var int: distribute(array [$X] of var $$E: cover,
                                               array [$Y] of var $$E: xs)
```

1. Requires that `count[i]` is the number of occurrences of `cover[i]` in `xs`. The values in `cover` need not be distinct.
2. Returns an array of the number of occurrences of `cover[i]` in `xs`. The values in `cover` need not be distinct.

```
predicate exactly(int: n,
                  array [$X] of var set of $$E: xs,
                  set of $$E: value)
```

Requires exactly `n` variables in `xs` to take the value `value`.

```
1.  predicate global_cardinality(array [$X] of var $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of var int: count)

2.  predicate global_cardinality(array [$X] of var opt $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of var int: count)

3.  predicate global_cardinality(array [$X] of var $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of int: lower_bound,
                                 array [$Y] of int: upper_bound)

4.  predicate global_cardinality(array [$X] of var opt $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of int: lower_bound,
                                 array [$Y] of int: upper_bound)

5.  predicate global_cardinality(array [$X] of var set of $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of var int: count)

6.  predicate global_cardinality(array [$X] of var set of $$E: xs,
                                 array [$Y] of $$E: cover,
                                 array [$Y] of int: lower_bound,
                                 array [$Y] of int: upper_bound)

7.  function array [$Y] of var int: global_cardinality(array [$X] of var $
↪$E: xs,
                                                       array [$Y] of $$E:␣
↪cover)
```

```
8.  function array [$Y] of var int: global_cardinality(array [$X] of var opt
↪$$E: xs,
                                                       array [$Y] of $$E:␣
↪cover)
```

1, 2.
Requires that the number of occurrences of `cover[i]` in `xs` is `count[i]`.

3, 4.
Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `xs`.

5. Requires that the number of occurrences of `cover[i]` in `xs` is `count[i]`.

6. Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `xs`.

7, 8.
Returns the number of occurrences of `cover[i]` in `xs`.

```
1.  predicate global_cardinality_closed(array [$X] of var $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of var int: count)

2.  predicate global_cardinality_closed(array [$X] of var opt $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of var int: count)

3.  predicate global_cardinality_closed(array [$X] of var $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of int: lower_bound,
                                        array [$Y] of int: upper_bound)

4.  predicate global_cardinality_closed(array [$X] of var opt $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of int: lower_bound,
                                        array [$Y] of int: upper_bound)

5.  predicate global_cardinality_closed(array [$X] of var set of $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of var int: count)

6.  predicate global_cardinality_closed(array [$X] of var set of $$E: xs,
                                        array [$Y] of $$E: cover,
                                        array [$Y] of int: lower_bound,
                                        array [$Y] of int: upper_bound)

7.  function array [$Y] of var int: global_cardinality_closed(array [$X] of␣
↪var $$E: xs,
                                                             array [$Y] of $
```

```
↪$E: cover)

8.  function array [$Y] of var int: global_cardinality_closed(array [$X] of␣
↪var opt $$E: xs,
                                                              array [$Y] of $
↪$E: cover)
```

1. Requires that the number of occurrences of `cover[i]` in `xs` is `count[i]`.

   The elements of `xs` must take their values from `cover`.

2. Requires that the number of occurrences of `cover[i]` in `xs` is `count[i]`.

   The elements of `xs` must take their values from `cover` or be absent.

3. Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `xs`.

   The elements of `xs` must take their values from `cover`.

4. Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `xs`.

   The elements of `xs` must take their values from `cover` or be absent.

5. Requires that the number of occurrences of `cover[i]` in `xs` is `count[i]`.

   The elements of `xs` must take their values from `cover` or be absent.

6. Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `xs`.

   The elements of `xs` must take their values from `cover` or be absent.

7. Returns an array with number of occurrences of `cover[i]` in `xs`.

   The elements of `xs` must take their values from `cover`.

8. Returns an array with number of occurrences of `cover[i]` in `xs`.

   The elements of `xs` must take their values from `cover` or be absent.

```
1.  predicate sliding_among(int: low,
                            int: up,
                            int: window_size,
                            array [$X] of var $$E: xs,
                            set of $$E: values)

2.  predicate sliding_among(int: low,
                            int: up,
                            int: window_size,
                            array [$X] of var bool: xs,
                            bool: value)
```

1. Every subsequence of `xs` of length `window_size` has at least `low` and at most `up` occurrences of the values in `values`

2. Every subsequence of `xs` of length `window_size` has at least `low` and at most `up` occurrences of the value `value`

4.2.2.6 Array-related constraints

```
predicate element(var $$E: i, array [$$E] of var bool: x, var bool: y)
predicate element(var $$E: i,
                  array [$$E] of var float: x,
                  var float: y)
predicate element(var $$E: i, array [$$E] of var $$T: x, var $$T: y)
predicate element(var $$E: i,
                  array [$$E] of var set of $$T: x,
                  var set of $$T: y)
```

Constrains `i` to be the index of the element `y` in the array `x`.

```
1.  predicate member(array [$X] of var bool: x, var bool: y)

2.  predicate member(array [$X] of var float: x, var float: y)

3.  predicate member(array [$X] of var $$E: x, var $$E: y)

4.  predicate member(array [$X] of var set of $$E: x, var set of $$E: y)

5.  predicate member(var set of $$E: x, var $$E: y)
```

1-4.
Requires that `y` occurs in the array `x`.

5. Requires that `y` occurs in the set `x`.

```
1.  predicate write(array [$$E] of var $$T: I,
                    var $$E: i,
                    var $$T: v,
                    array [$$E] of var $$T: O)

2.  function array [$$E] of var $$T: write(array [$$E] of var $$T: I,
                                           var $$E: i,
                                           var $$T: v)
```

1. Creates a new array `O` from an input array `I` with a change at position `i` to take value `v`

   `I` is an array of values `O` is an array of values with same index set as `I` `i` is an index for `I` `v` is the value to write

2. Return the input array `I` with a change at position `i` to take value `v`

   `I` is an array of values `i` is an index for `I` `v` is the value to write

---

```
1.  predicate writes(array [$$X] of var $$T: I,
                     array [$$Y] of var $$X: P,
                     array [$$Y] of var $$T: V,
                     array [$$X] of var $$T: O)

2.  function array [$$X] of var $$T: writes(array [$$X] of var $$T: I,
                                            array [$$Y] of var $$X: P,
                                            array [$$Y] of var $$T: V)
```

1. Creates a new array `O` from an input array `I` with a simultaneous change at positions `P` to values `V`

   `I` is an array of input values `O` is an array of integers with same index set as `I` `P` is an array of index values in `I` `V` is an array of values to write

2. Returns a new array from an input array `I` with a simultaneous change at positions `P` to values `V`

   `I` is an array of input values `P` is an array of index values in `I` `V` is an array of values to write

---

```
1.  predicate writes_seq(array [$$X] of var $$T: I,
                         array [$$Y] of var $$X: P,
                         array [$$Y] of var $$T: V,
                         array [$$X] of var $$T: O)

2.  function array [$$X] of var $$T: writes_seq(array [$$X] of var $$T: I,
                                                array [$$Y] of var $$X: P,
                                                array [$$Y] of var $$T: V)
```

1. Creates a new array `O` from an input array `I` with a sequence of changes at positions `P` to take values `V`

   `I` is an array of input values `O` is an array of output values with same index set as `I` `P` is an array of indices in `I` `V` is an array of values to write

2. Return the input array `I` with a sequence of changes at positions `P` to take values `V`

   `I` is an array of input values `P` is an array of index values in `I` `V` is an array of values to write

---

4.2.2.7 Set-related constraints

```
predicate disjoint(var set of $$E: s, var set of $$E: t)
```

Requires that sets `s` and `t` do not intersect.

```
predicate partition_set(array [$X] of var set of $$E: xs,
                        set of $$E: universe)
```

Constrains the sets in array `xs` to partition the `universe`.

```
1.  predicate roots(array [$$X] of var $$Y: xs,
                    var set of $$X: preimage,
                    var set of $$Y: image)

2.  function var set of $$X: roots(array [$$X] of var $$Y: xs,
                                   var set of $$Y: image)
```

1. Requires that `xs[i]` in `image` for all `i` in `preimage`, i.e. `preimage` is the set of indices `i` for which `xs[i]` is in `image`.
2. Returns the set of indices `i` such that `xs[i]` is in `image`.

4.2.2.8 Mathematical constraints

```
1.  function var $$E: arg_max(array [$$E] of var $$T: xs)

2.  function var $$E: arg_max(array [$$E] of var bool: xs)

3.  function var $$E: arg_max(array [$$E] of var float: xs)

4.  function var $$E: arg_max(array [$$E] of var opt $$T: xs)

5.  function var $$E: arg_max(array [$$E] of var opt bool: xs)

6.  function var $$E: arg_max(array [$$E] of var opt float: xs)
```

1-3.
Returns the index of the maximum value in the array `xs`. When breaking ties the least index is returned.

4-6.
Returns the index of the maximum non-absent value in the array `xs`. When breaking ties the least index is returned.

```
function var opt $$E: arg_max_weak(array [$$E] of var opt $$T: xs)
function var opt $$E: arg_max_weak(array [$$E] of var opt bool: xs)
function var opt $$E: arg_max_weak(array [$$E] of var opt float: xs)
```

Returns the index of the maximum non-absent value in the array `xs`. When breaking ties the least index is returned. Returns absent when all elements are absent.

```
1.  function var $$E: arg_min(array [$$E] of var $$T: xs)

2.  function var $$E: arg_min(array [$$E] of var bool: xs)

3.  function var $$E: arg_min(array [$$E] of var float: xs)

4.  function var $$E: arg_min(array [$$E] of var opt $$T: xs)

5.  function var $$E: arg_min(array [$$E] of var opt bool: xs)

6.  function var $$E: arg_min(array [$$E] of var opt float: xs)
```

1-3.
Returns the index of the minimum value in the array `xs`. When breaking ties the least index is returned.

4-6.
Returns the index of the minimum non-absent value in the array `xs`. When breaking ties the least index is returned.

```
function var opt $$E: arg_min_weak(array [$$E] of var opt $$T: xs)
function var opt $$E: arg_min_weak(array [$$E] of var opt bool: xs)
function var opt $$E: arg_min_weak(array [$$E] of var opt float: xs)
```

Returns the index of the minimum non-absent value in the array `xs`. When breaking ties the least index is returned. Returns absent when all elements are absent.

```
function $$E: arg_val(array [$$E] of $T: xs, $T: value)
function var $$E: arg_val(array [$$E] of var bool: xs, var bool: value)
function var $$E: arg_val(array [$$E] of var opt bool: xs,
                          var opt bool: value)
function var $$E: arg_val(array [$$E] of var float: xs,
                          var float: value)
function var $$E: arg_val(array [$$E] of var opt float: xs,
                          var float: value)
function var $$E: arg_val(array [$$E] of var $$V: xs, var $$V: value)
```

```
function var $$E: arg_val(array [$$E] of var opt $$V: xs,
                          var opt $$V: value)
```

Returns the index of the value `value` in the array `xs`. When breaking ties the least index is returned. Note that this function forces the value `value` to occur in `xs`.

```
function opt $$E: arg_val_weak(array [$$E] of $T: xs, $T: value)
function var opt $$E: arg_val_weak(array [$$E] of var bool: xs,
                                   var bool: value)
function var opt $$E: arg_val_weak(array [$$E] of var opt bool: xs,
                                   var opt bool: value)
function var opt $$E: arg_val_weak(array [$$E] of var float: xs,
                                   var float: value)
function var opt $$E: arg_val_weak(array [$$E] of var opt float: xs,
                                   var opt float: value)
function var opt $$E: arg_val_weak(array [$$E] of var $$V: xs,
                                   var $$V: value)
function var opt $$E: arg_val_weak(array [$$E] of var opt $$V: xs,
                                   var opt $$V: value)
```

Returns the index of the value `value` in the array `xs` when `value`. When breaking ties the least index is returned. Returns absent when `value` does not occur in `xs`.

```
predicate maximum(var $$E: max_value, array [$X] of var $$E: xs)
predicate maximum(var float: max_value, array [$X] of var float: xs)
```

Constrains `max_value` to be the maximum of the values in `xs`.

Assumptions: `|xs|` > 0.

```
1.  predicate maximum_arg(array [$$E] of var $$T: xs, var $$E: idx)

2.  predicate maximum_arg(array [$$E] of var bool: xs, var $$E: idx)

3.  predicate maximum_arg(array [$$E] of var float: xs, var $$E: idx)
```

1, 2.
Constrain `idx` to be the index of the maximum value in the array `xs`. When breaking ties the least index is returned.

Assumption: `|xs|` > 0

3. Constrain `idx` to be the index of the maximum value in the array `xs`. When breaking ties the least index is returned.

   Assumption: `|xs|` > 0

```
predicate minimum(var float: min_value, array [$X] of var float: xs)
predicate minimum(var $$E: min_value, array [$X] of var $$E: xs)
```

Constrains `min_value` to be the minimum of the values in `xs`.

Assumptions: `|xs|` > 0.

```
predicate minimum_arg(array [$$E] of var $$T: xs, var $$E: idx)
predicate minimum_arg(array [$$E] of var bool: xs, var $$E: idx)
predicate minimum_arg(array [$$E] of var float: xs, var $$E: idx)
```

Constrain `idx` to be the index of the minimum value in the array `xs`. When breaking ties the least index is returned.

Assumption: `|xs|` > 0

```
1.  function var float: piecewise_linear(var float: x,
                                         array [int] of float: x_knot,
                                         array [int] of float: y_knot)

2.  predicate piecewise_linear(var float: x,
                               var float: y,
                               array [int] of float: x_knot,
                               array [int] of float: y_knot)

3.  function var float: piecewise_linear(var float: x,
                                         array [int] of float: x_start,
                                         array [int] of float: x_end,
                                         array [int] of float: y_start,
                                         array [int] of float: y_end)

4.  predicate piecewise_linear(var float: x,
                               var float: y,
                               array [int] of float: x_start,
                               array [int] of float: x_end,
                               array [int] of float: y_start,
                               array [int] of float: y_end)
```

1. Return the piecewise-linear function of `x` on the given point-value sequence. The array `x_knot` defines the points, and `y_knot` the corresponding values.
2. Constrains `y`(`x`) to be the piecewise-linear function on the provided point-value sequence. The array `x_knot` defines the points, and `y_knot` the corresponding values.
3. Return the piecewise-linear function of `x` on the given (possibly disconnected) intervals. Each interval `i` connects (`x_start`[`i`], `y_start`[`i`]) to (`x_end`[`i`], `y_end`[`i`]).

4. Constrains y(x) to be the piecewise-linear function on the given (possibly disconnected) intervals. Each interval i connects (x_start[i], y_start[i]) to (x_end[i], y_end[i]).

```
1.  predicate range(array [$$X] of var $$Y: xs,
                    var set of $$X: domain,
                    var set of $$Y: image)

2.  function var set of int: range(array [int] of var int: xs,
                                   var set of int: domain)
```

1. Requires that the image of function xs (represented as an array) on set of values domain is image. ub(domain) must be a subset of index_set(xs) otherwise an assertion failure will occur.
2. Returns the image of function xs (represented as an array) on set of values domain. ub(domain) must be a subset of index_set(xs) otherwise an assertion failure will occur.

```
predicate sliding_sum(int: low,
                      int: up,
                      int: window_size,
                      array [$$E] of var int: xs)
```

Requires that in each subsequence xs[i], ..., xs[i + window_size - 1] the sum of the values belongs to the interval [low, up].

```
predicate sum_pred(var $$X: idx,
                   array [$$X] of set of $$Y: sets,
                   array [$$Y] of int: coefficients,
                   var int: total)
```

Requires that the sum of coefficients[i1]..coefficients[iN] equals total, where i1..iN are the elements of the idx th set in sets.

Nb: not called 'sum' as in the constraints catalog because 'sum' is a MiniZinc built-in function.

```
predicate sum_set(array [$$X] of $$Y: values,
                  array [$$X] of int: weights,
                  var set of $$Y: selected,
                  var int: total)
```

Requires that the sum of the weights weights[i1]..weights[iN] equals total, where values[i1]..values[iN] are the elements appearing in set selected

4.2.2.9 Packing constraints

```
1. predicate bin_packing(int: capacity,
                         array [$$I] of var $$B: bin,
                         array [$$I] of int: weight)

2. predicate bin_packing(int: capacity,
                         array [$$I] of var opt $$B: bin,
                         array [$$I] of int: weight)
```

1. Requires that each item `i` with weight `weight[i]`, be put into `bin[i]` such that the sum of the weight of the items in each bin does not exceed the capacity `capacity`.

   Assumptions:

   - forall `i`, `weight[i]` >=0
   - `capacity` >=0

2. Requires that each item `i` with weight `weight[i]`, be put into `bin[i]` such that the sum of the weight of the items in each bin does not exceed the capacity `capacity`. An item may not be put into any bin, in which case `bin[i]` is absent.

   Assumptions:

   - forall `i`, `weight[i]` >=0
   - `capacity` >=0

```
1. predicate bin_packing_capa(array [$$B] of int: capacity,
                              array [$$I] of var $$B: bin,
                              array [$$I] of int: weight)

2. predicate bin_packing_capa(array [$$B] of int: capacity,
                              array [$$I] of var opt $$B: bin,
                              array [$$I] of int: weight)
```

1. Requires that each item `i` with weight `weight[i]`, be put into `bin[i]` such that the sum of the weight of the items in each bin `b` does not exceed the capacity `capacity[b]`.

   Assumptions:

   - forall `i`, `weight[i]` >=0
   - forall `b`, `capacity[b]` >=0

2. Requires that each item `i` with weight `weight[i]`, be put into `bin[i]` such that the sum of the weight of the items in each bin `b` does not exceed the capacity `capacity[b]`. An item may not be put into any bin, in which case `bin[i]` is absent.

   Assumptions:

   - forall `i`, `weight[i]` >=0

- forall b, capacity[b] >=0

```
1.  predicate bin_packing_load(array [$$B] of var int: load,
                               array [$$I] of var $$B: bin,
                               array [$$I] of int: weight)

2.  predicate bin_packing_load(array [$$B] of var int: load,
                               array [$$I] of var opt $$B: bin,
                               array [$$I] of int: weight)

3.  function array [$$B] of var int: bin_packing_load(array [$$I] of var $
↪$B: bin,
                                                      array [$$I] of int:␣
↪weight)

4.  function array [$$B] of var int: bin_packing_load(array [$$I] of var opt
↪$$B: bin,
                                                      array [$$I] of int:␣
↪weight)
```

1. Requires that each item i with weight weight[i], be put into bin[i] such that the sum of the weight of the items in each bin b is equal to load[b].

   Assumptions:

   - forall i, weight[i] >=0

2. Requires that each item i with weight weight[i], be put into bin[i] such that the sum of the weight of the items in each bin b is equal to load[b]. An item may not be put into any bin, in which case bin[i] is absent.

   Assumptions:

   - forall i, weight[i] >=0

3. Returns the load of each bin resulting from packing each item i with weight weight[i] into bin[i], where the load is defined as the sum of the weight of the items in each bin.

   Assumptions:

   - forall i, weight[i] >=0

4. Returns the load of each bin resulting from packing each item i with weight weight[i] into bin[i], where the load is defined as the sum of the weight of the items in each bin. An item may not be put into any bin, in which case bin[i] is absent.

   Assumptions:

   - forall i, weight[i] >=0

```
predicate diffn(array [$$E] of var int: x,
                array [$$E] of var int: y,
                array [$$E] of var int: dx,
                array [$$E] of var int: dy)
```

Constrains rectangles `i`, given by their origins (`x[i]`, `y[i]`) and sizes (`dx[i]`, `dy[i]`), to be non-overlapping. Zero-width rectangles can still not overlap with any other rectangle.

```
predicate diffn_k(array [$$E,$$F] of var int: box_posn,
                  array [$$E,$$F] of var int: box_size)
```

Constrains `k`-dimensional boxes to be non-overlapping. For each box `i` and dimension `j`, `box_posn[i, j]` is the base position of the box in dimension `j`, and `box_size[i, j]` is the size in that dimension. Boxes whose size is 0 in any dimension still cannot overlap with any other box.

```
predicate diffn_nonstrict(array [$$E] of var int: x,
                          array [$$E] of var int: y,
                          array [$$E] of var int: dx,
                          array [$$E] of var int: dy)
```

Constrains rectangles `i`, given by their origins (`x[i]`, `y[i]`) and sizes (`dx[i]`, `dy[i]`), to be non-overlapping. Zero-width rectangles can be packed anywhere.

```
predicate diffn_nonstrict_k(array [$$E,$$F] of var int: box_posn,
                            array [$$E,$$F] of var int: box_size)
```

Constrains `k`-dimensional boxes to be non-overlapping. For each box `i` and dimension `j`, `box_posn[i, j]` is the base position of the box in dimension `j`, and `box_size[i, j]` is the size in that dimension. Boxes whose size is 0 in at least one dimension can be packed anywhere.

```
1.  predicate geost(array [$$R,$$K] of int: rect_size,
                    array [$$R,$$K] of int: rect_offset,
                    array [$$S] of set of $$R: shape,
                    array [$$O,$$K] of var int: xs,
                    array [$$O] of var $$S: kind)

2.  predicate geost(int: k,
                    array [$$R,$$K] of int: rect_size,
                    array [$$R,$$K] of int: rect_offset,
                    array [$$S] of set of $$R: shape,
```

```
                    array [$$O,$$K] of var int: xs,
                    array [$$O] of var $$S: kind)
```

1. A global non-overlap constraint for `k` dimensional objects. It enforces that no two objects overlap.

   Parameters:

   - `rect_size`: the size of each box in `k` dimensions
   - `rect_offset`: the offset of each box from the base position in `k` dimensions
   - `shape`: the set of rectangles defining the `i`-th shape.
   - `xs`: the base position of each object. `xs[i,j]` is the position of object `i` in. dimension `j`.
   - `kind`: the shape used by each object.

2. A global non-overlap constraint for `k` dimensional objects. It enforces that no two objects overlap.

   Parameters:

   - `k`: the number of dimensions
   - `rect_size`: the size of each box in `k` dimensions
   - `rect_offset`: the offset of each box from the base position in `k` dimensions
   - `shape`: the set of rectangles defining the `i`-th shape.
   - `xs`: the base position of each object. `xs[i,j]` is the position of object `i` in. dimension `j`.
   - `kind`: the shape used by each object.

```
predicate geost_bb(array [$$R,$$K] of int: rect_size,
                   array [$$R,$$K] of int: rect_offset,
                   array [$$S] of set of $$R: shape,
                   array [$$O,$$K] of var int: xs,
                   array [$$O] of var $$S: kind,
                   array [$$K] of var int: lower_bound,
                   array [$$K] of var int: upper_bound)
predicate geost_bb(int: k,
                   array [$$R,$$K] of int: rect_size,
                   array [$$R,$$K] of int: rect_offset,
                   array [$$S] of set of $$R: shape,
                   array [$$O,$$K] of var int: xs,
                   array [$$O] of var $$S: kind,
                   array [$$K] of var int: lower_bound,
                   array [$$K] of var int: upper_bound)
```

A global non-overlap constraint for `k` dimensional objects. It enforces that no two objects overlap, and that all objects fit within a global `k` dimensional bounding box.

Parameters:

- `k`: the number of dimensions
- `rect_size`: the size of each box in `k` dimensions
- `rect_offset`: the offset of each box from the base position in `k` dimensions
- `shape`: the set of rectangles defining the `i`-th shape.
- `xs`: the base position of each object. `xs[i,j]` is the position of object `i` in dimension `j`.
- `kind`: the shape used by each object.
- `lower_bound`: is an array of lower bounds, `lower_bound[i]` is the minimum bounding box for all objects in dimension `i`.
- `upper_bound`: is an array of upper bounds, `upper_bound[i]` is the maximum bounding box for all objects in dimension `i`.

```
predicate geost_nonoverlap_k(array [$$K] of var int: x1,
                             array [$$K] of int: w1,
                             array [$$K] of var int: x2,
                             array [$$K] of int: w2)
```

A non-overlap constraint for two `k` dimensional objects. It enforces that there is at least one dimension where the objects occupy a different space.

Parameters:

- `x1`: coordinate of each dimension for the first object
- `w1`: width of each dimension for the first object
- `x2`: coordinate of each dimension for the second object
- `w2`: width of each dimension for the second object

```
1.  predicate geost_smallest_bb(array [$$R,$$K] of int: rect_size,
                                array [$$R,$$K] of int: rect_offset,
                                array [$$S] of set of $$R: shape,
                                array [$$O,$$K] of var int: xs,
                                array [$$O] of var $$S: kind,
                                array [$$K] of var int: lower_bound,
                                array [$$K] of var int: upper_bound)

2.  predicate geost_smallest_bb(int: k,
                                array [$$R,$$K] of int: rect_size,
                                array [$$R,$$K] of int: rect_offset,
                                array [$$S] of set of $$R: shape,
                                array [$$O,$$K] of var int: xs,
                                array [$$O] of var $$S: kind,
                                array [$$K] of var int: lower_bound,
```

```
                                array [$$K] of var int: upper_bound)
```

1. A global non-overlap constraint for `k` dimensional objects. It enforces that no two objects overlap, and that all objects fit within a global `k` dimensional bounding box. In addition, it enforces that the bounding box is the smallest one containing all objects, i.e., each of the `2k` boundaries is touched by at least by one object.

   Parameters:

   - `rect_size`: the size of each box in `k` dimensions
   - `rect_offset`: the offset of each box from the base position in `k` dimensions
   - `shape`: the set of rectangles defining the `i`-th shape.
   - `xs`: the base position of each object. `xs[i,j]` is the position of object `i` in dimension `j`.
   - `kind`: the shape used by each object.
   - `lower_bound`: is an array of lower bounds, `lower_bound[i]` is the minimum bounding box for all objects in dimension `i`.
   - `upper_bound`: is an array of upper bounds, `upper_bound[i]` is the maximum bounding box for all objects in dimension `i`.

2. A global non-overlap constraint for `k` dimensional objects. It enforces that no two objects overlap, and that all objects fit within a global `k` dimensional bounding box. In addition, it enforces that the bounding box is the smallest one containing all objects, i.e., each of the `2k` boundaries is touched by at least by one object.

   Parameters:

   - `k`: the number of dimensions
   - `rect_size`: the size of each box in `k` dimensions
   - `rect_offset`: the offset of each box from the base position in `k` dimensions
   - `shape`: the set of rectangles defining the `i`-th shape.
   - `xs`: the base position of each object. `xs[i,j]` is the position of object `i` in dimension `j`.
   - `kind`: the shape used by each object.
   - `lower_bound`: is an array of lower bounds, `lower_bound[i]` is the minimum bounding box for all objects in dimension `i`.
   - `upper_bound`: is an array of upper bounds, `upper_bound[i]` is the maximum bounding box for all objects in dimension `i`.

```
predicate knapsack(array [$$I] of int: weight,
                   array [$$I] of int: profit,
                   array [$$I] of var int: xs,
                   var int: total_weight,
                   var int: total_profit)
```

Requires that items are packed in a knapsack with certain weight and profit restrictions.

Assumptions:

- Weights `weight` and profit `profit` must be non-negative
- `weight`, `profit` and `xs` must have the same index sets

Parameters:

- `weight`: weight of each type of item
- `profit`: profit of each type of item
- `xs`: number of items of each type that are packed
- `total_weight`: sum of sizes of all items in the knapsack
- `total_profit`: sum of profit of all items in the knapsack

4.2.2.10 Scheduling constraints

```
predicate alternative(var opt int: span_start,
                      var int: span_duration,
                      array [int] of var opt int: start,
                      array [int] of var int: duration)
```

Alternative constraint for optional tasks. The task with start time `span_start` and duration `span_duration` spans the optional tasks with start times `start[i]` and duration `duration[i]`), and at most one of those tasks can occur.

```
1.  predicate cumulative(array [$$E] of var int: start,
                         array [$$E] of var int: duration,
                         array [$$E] of var int: resource,
                         var int: capacity)

2.  predicate cumulative(array [$$E] of var opt int: start,
                         array [$$E] of var int: duration,
                         array [$$E] of var int: resource,
                         var int: capacity)
```

1. Requires that a set of tasks given by start times `start`, duration `duration`, and resource requirements `resource`, never require more than a global resource bound `capacity` at any one time.

   Assumptions:

   - forall `i`, `duration[i]` >= 0 and `resource[i]` >= 0

2. Requires that a set of tasks given by start times `start`, duration `duration`, and resource requirements `resource`, never require more than a global resource bound `capacity` at any one time. Start times are optional variables, so that absent tasks do not need to be scheduled.

Assumptions:

- forall `i`, `duration`[`i`] >= 0 and `resource`[`i`] >= 0

```
1.  predicate cumulatives(array [$$E] of var int: start,
                          array [$$E] of var int: duration,
                          array [$$E] of var int: resource,
                          array [$$E] of var $$M: machine,
                          array [$$M] of var int: upper_bound)

2.  predicate cumulatives(array [$$E] of var int: start,
                          array [$$E] of var int: duration,
                          array [$$E] of var int: resource,
                          array [$$E] of var $$M: machine,
                          array [$$M] of var int: lower_bound)

3.  predicate cumulatives(array [$$E] of var opt int: start,
                          array [$$E] of var int: duration,
                          array [$$E] of var int: resource,
                          array [$$E] of var $$M: machine,
                          array [$$M] of var int: upper_bound)

4.  predicate cumulatives(array [$$E] of var opt int: start,
                          array [$$E] of var int: duration,
                          array [$$E] of var int: resource,
                          array [$$E] of var $$M: machine,
                          array [$$M] of var int: lower_bound)
```

1. We have a set of tasks given by start times `start`, duration `duration`, resource requirements `resource`, and machine `machine`. We also have a set of machine given by resource capacities `upper_bound`. The constraint requires that forall `i`, `machine`[`i`] is in the index set of `upper_bound` and that for each machine `j` and time instant `t`, either no task assigned to `j` executes at `t`, or the total resource requirement of such tasks is not greater than the given bound `upper_bound`[`j`]. Resource requirements can be positive (for consumption) or negative (for production).

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

2. We have a set of tasks given by start times `start`, duration `duration`, resource requirements `resource`, and machine `machine`. We also have a set of machine given by resource minima `lower_bound`. The constraint requires that forall `i`, `machine`[`i`] is in the index set of `lower_bound` and that for each machine `j` and time instant `t`, either no task assigned to `j` executes at `t`, or the total resource requirement of such tasks is not less than the given bound `lower_bound`[`j`]. Resource requirements can be positive (for consumption) or negative (for production).

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

3. We have a set of optional tasks given by start times `start`, duration `duration`, resource requirements `resource`, and machine `machine`. We also have a set of machine given by resource capacities `upper_bound`. The constraint requires that forall `i`, `machine`[`i`] is in the index set of `upper_bound` and that for each machine `j` and time instant `t`, either no task assigned to `j` executes at `t`, or the total resource requirement of such tasks is not greater than the given bound `upper_bound`[`j`]. Resource requirements can be positive (for consumption) or negative (for production).

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

4. We have a set of optional tasks given by start times `start`, duration `duration`, resource requirements `resource`, and machine `machine`. We also have a set of machine given by resource minima `lower_bound`. The constraint requires that forall `i`, `machine`[`i`] is in the index set of `lower_bound` and that for each machine `j` and time instant `t`, either no task assigned to `j` executes at `t`, or the total resource requirement of such tasks is not less than the given bound `lower_bound`[`j`]. Resource requirements can be positive (for consumption) or negative (for production).

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

```
1.  predicate disjunctive(array [$$T] of var int: start,
                          array [$$T] of var int: duration)

2.  predicate disjunctive(array [$$T] of var opt int: start,
                          array [$$T] of var int: duration)
```

1. Requires that a set of tasks given by start times `start` and duration `duration` do not overlap in time. Tasks with duration 0 can be scheduled at any time, even in the middle of other tasks.

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

2. Requires that a set of tasks given by start times `start` and duration `duration` do not overlap in time. Tasks with duration 0 can be scheduled at any time, even in the middle of other tasks. Start times are optional variables, so that absent tasks do not need to be scheduled.

   Assumptions:

   - forall `i`, `duration`[`i`] >= 0

```
1.  predicate disjunctive_strict(array [$$T] of var int: start,
                                 array [$$T] of var int: duration)

2.  predicate disjunctive_strict(array [$$T] of var opt int: start,
                                 array [$$T] of var int: duration)
```

1. Requires that a set of tasks given by start times `start` and duration `duration` do not overlap in time. Tasks with duration 0 CANNOT be scheduled at any time, but only when no other task is running.

   Assumptions:

   - forall `i`, `duration[i]` >= 0

2. Requires that a set of tasks given by start times `start` and duration `duration` do not overlap in time. Tasks with duration 0 CANNOT be scheduled at any time, but only when no other task is running. Start times are optional variables, so that absent tasks do not need to be scheduled.

   Assumptions:

   - forall `i`, `duration[i]` >= 0

```
predicate span(var opt int: span_start,
               var int: span_duration,
               array [$$E] of var opt int: start,
               array [$$E] of var int: duration)
```

Span constraint for optional tasks. Task (`span_start`,`span_duration`) spans the optional tasks (`start[i]`,`duration[i]`) in the array arguments.

4.2.2.11 Graph constraints

Many of these constraints operate on graphs that are represented as lists of edges. The constraints take a given (fixed) graph as arguments, which is represented using two arrays, `from` and `to`, representing the graph with edges `(from[i],to[i])`, where `from[i]` and `to[i]` are nodes of the graph.

```
1.  predicate bounded_dpath(int: N,
                            int: E,
                            array [int] of int: from,
                            array [int] of int: to,
                            array [int] of int: w,
                            var int: source,
                            var int: sink,
                            array [int] of var bool: ns,
                            array [int] of var bool: es,
                            var int: weight_bound)

2.  predicate bounded_dpath(array [int] of $$N: from,
                            array [int] of $$N: to,
                            array [int] of int: w,
                            var $$N: source,
                            var $$N: sink,
                            array [$$N] of var bool: ns,
```

```
                              array [int] of var bool: es,
                              var int: weight_bound)
```

1. Constrains the subgraph `ns` and `es` of a given directed graph to be a path from `source` to `sink` of weight `weight_bound`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `w`: the weight of each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph
   - `weight_bound`: the cost of the path

2. Constrains the subgraph `ns` and `es` of a given directed graph to be a path from `source` to `sink` of weight `weight_bound`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `w`: the weight of each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph
   - `weight_bound`: the cost of the path

```
1.  predicate bounded_path(int: N,
                           int: E,
                           array [int] of int: from,
                           array [int] of int: to,
                           array [int] of int: w,
                           var int: source,
                           var int: sink,
                           array [int] of var bool: ns,
                           array [int] of var bool: es,
                           var int: weight_bound)
```

```
2.  predicate bounded_path(array [int] of $$N: from,
                           array [int] of $$N: to,
                           array [int] of int: w,
                           var $$N: source,
                           var $$N: sink,
                           array [$$N] of var bool: ns,
                           array [int] of var bool: es,
                           var int: weight_bound)
```

1. Constrains the subgraph `ns` and `es` of a given undirected graph to be a path from `source` to `sink` of weight `weight_bound`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `w`: the weight of each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph
   - `weight_bound`: the cost of the path

2. Constrains the subgraph `ns` and `es` of a given undirected graph to be a path from `source` to `sink` of weight `weight_bound`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `w`: the weight of each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph
   - `weight_bound`: the cost of the path

```
1.  predicate circuit(array [$$E] of var $$E: xs)

2.  predicate circuit(array [$$E] of var opt $$E: xs)
```

1. Constrains the elements of `xs` to define a circuit where `xs[i] = j` means that `j` is the successor of `i`.
2. Constrains the elements of `xs` to define a circuit where `xs[i] = j` means that `j` is the successor of `i`. Absent elements do not take part in the circuit.

```
predicate connected(array [$$E] of $$N: from,
                    array [$$E] of $$N: to,
                    array [$$N] of var bool: ns,
                    array [$$E] of var bool: es)
```

Constrains the subgraph `ns` and `es` of a given undirected graph to be connected.

Parameters:

- `from`: is the leaving node for each edge
- `to`: is the entering node for each edge
- `ns`: is a Boolean for each node whether it is in the subgraph
- `es`: is a Boolean for each edge whether it is in the subgraph

---

```
1.  predicate d_weighted_spanning_tree(array [$$E] of $$N: from,
                                       array [$$E] of $$N: to,
                                       array [$$E] of int: w,
                                       var $$N: root,
                                       array [$$E] of var bool: es,
                                       var int: weight_bound)

2.  predicate d_weighted_spanning_tree(int: N,
                                       int: E,
                                       array [int] of int: from,
                                       array [int] of int: to,
                                       array [int] of int: w,
                                       var int: root,
                                       array [int] of var bool: es,
                                       var int: weight_bound)
```

1. Constrains the set of edges `es` of a given directed graph to be a weighted spanning tree rooted at `root` of weight `W`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node each edge
   - `w`: the weight of each edge
   - `root`: the root node (which may be variable)
   - `es`: a Boolean for each edge whether it is in the subgraph

- `weight_bound`: the weight of the tree

2. Constrains the set of edges `es` of a given directed graph to be a weighted spanning tree rooted at `root` of weight `W`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `w`: the weight of each edge
   - `root`: the root node (which may be variable)
   - `es`: a Boolean for each edge whether it is in the subgraph
   - `weight_bound`: the weight of the tree

```
predicate dag(array [$$E] of $$N: from,
              array [$$E] of $$N: to,
              array [$$N] of var bool: ns,
              array [$$E] of var bool: es)
```

Constrains the subgraph `ns` and `es` of a given directed graph to be a DAG.

Parameters:

- `from`: the leaving node for each edge
- `to`: the entering node for each edge
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

```
predicate dconnected(array [$$E] of $$N: from,
                     array [$$E] of $$N: to,
                     array [$$N] of var bool: ns,
                     array [$$E] of var bool: es)
```

Constrains the subgraph `ns` and `es` of a given directed graph to be connected.

Parameters:

- `from`: the leaving node for each edge
- `to`: the entering node for each edge
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

```
1.  predicate dpath(int: N,
                    int: E,
                    array [int] of int: from,
                    array [int] of int: to,
                    var int: source,
                    var int: sink,
                    array [int] of var bool: ns,
                    array [int] of var bool: es)

2.  predicate dpath(array [int] of $$N: from,
                    array [int] of $$N: to,
                    var $$N: source,
                    var $$N: sink,
                    array [$$N] of var bool: ns,
                    array [int] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given directed graph to be a path from `source` to `sink`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Constrains the subgraph `ns` and `es` of a given directed graph to be a path from `source` to `sink`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
1.  predicate dreachable(int: N,
                         int: E,
                         array [int] of int: from,
```

```
                          array [int] of int: to,
                          var int: root,
                          array [int] of var bool: ns,
                          array [int] of var bool: es)

2.  predicate dreachable(array [$$E] of $$N: from,
                          array [$$E] of $$N: to,
                          var $$N: root,
                          array [$$N] of var bool: ns,
                          array [$$E] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given directed graph to be reachable from `root`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Constrains the subgraph `ns` and `es` of a given directed graph to be reachable from `root`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
predicate dsteiner(int: N,
                   int: E,
                   array [int] of int: from,
                   array [int] of int: to,
                   array [int] of int: w,
                   var int: root,
                   array [int] of var bool: ns,
                   array [int] of var bool: es,
                   var int: weight_bound)
```

Constrains the subgraph `ns` and `es` of a given directed graph to be a weighted spanning tree rooted at `root` of weight `W`.

Parameters:

- `N`: the number of nodes in the given graph
- `E`: the number of edges in the given graph
- `from`: the leaving node 1..`N` for each edge
- `to`: the entering node 1..`N` for each edge
- `w`: the weight of each edge
- `root`: the root node (which may be variable)
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph
- `weight_bound`: the weight of the tree

```
1.  predicate dtree(int: N,
                    int: E,
                    array [int] of int: from,
                    array [int] of int: to,
                    var int: root,
                    array [int] of var bool: ns,
                    array [int] of var bool: es)

2.  predicate dtree(array [$$E] of $$N: from,
                    array [$$E] of $$N: to,
                    var $$N: root,
                    array [$$N] of var bool: ns,
                    array [$$E] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given directed graph to be a tree rooted at `root`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph
2. Constrains the subgraph `ns` and `es` of a given directed graph to be at tree rooted at `root`.

Parameters:

- `from`: the leaving node for each edge
- `to`: the entering node for each edge
- `root`: the root node (which may be variable)
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

```
predicate network_flow(array [$$E,1..2] of $$N: arcs,
                       array [$$N] of int: balance,
                       array [$$E] of var int: flow)
```

Defines a network flow constraint.

Parameters:

- `arcs`: the directed arcs of the flow network. Arc `i` connects node `arcs[i,1]` to node `arcs[i,2]`.
- `balance`: the amount of flow added or removed by each node. If the balance is positive then flow is added by the node, and it is removed when it is negative, i.e., `balance = output - input`.
- `flow`: the flow going through each arc.

```
predicate network_flow_cost(array [$$E,1..2] of $$N: arcs,
                            array [$$N] of int: balance,
                            array [$$E] of int: weight,
                            array [$$E] of var int: flow,
                            var int: total_cost)
```

Defines a network flow constraint with cost.

Parameters:

- `arcs`: the directed arcs of the flow network. Arc `i` connects node `arcs[i,1]` to node `arcs[i,2]`.
- `balance`: the amount of flow added or removed by each node. If the balance is positive then flow is added by the node, and it is removed when it is negative, i.e., `balance = output - input`.
- `weight`: the unit cost of the flow through the arc.
- `flow`: the flow going through each arc.
- `total_cost`: the overall cost of the flow.

```
1.  predicate path(int: N,
                   int: E,
                   array [int] of int: from,
                   array [int] of int: to,
                   var int: source,
                   var int: sink,
                   array [int] of var bool: ns,
                   array [int] of var bool: es)

2.  predicate path(array [int] of $$N: from,
                   array [int] of $$N: to,
                   var $$N: source,
                   var $$N: sink,
                   array [$$N] of var bool: ns,
                   array [int] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given undirected graph to be a path from `source` to `sink`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Constrains the subgraph `ns` and `es` of a given undirected graph to be a path from `source` to `sink`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `source`: the source node (which may be variable)
   - `sink`: the dest node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
1.  predicate reachable(int: N,
                        int: E,
                        array [int] of int: from,
```

```
                        array [int] of int: to,
                        var int: root,
                        array [int] of var bool: ns,
                        array [int] of var bool: es)

2.  predicate reachable(array [$$E] of $$N: from,
                        array [$$E] of $$N: to,
                        var $$N: root,
                        array [$$N] of var bool: ns,
                        array [$$E] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given undirected graph to be reachable from `root`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Constrains the subgraph `ns` and `es` of a given undirected graph to be reachable from `root`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
predicate steiner(int: N,
                  int: E,
                  array [int] of int: from,
                  array [int] of int: to,
                  array [int] of int: w,
                  array [int] of var bool: ns,
                  array [int] of var bool: es,
                  var int: weight_bound)
```

Constrains the set of edges `es` of a given undirected graph to be a weighted spanning tree of weight `W`.

-

Parameters:

- N: the number of nodes in the given graph
- E: the number of edges in the given graph
- from: the leaving node 1..N for each edge
- to: the entering node 1..N for each edge
- w: the weight of each edge
- ns: a Boolean for each node whether it is in the subgraph
- es: a Boolean for each edge whether it is in the subgraph
- weight_bound: the weight of the tree

```
predicate subcircuit(array [$$E] of var $$E: xs)
```

Constrains the elements of xs to define a subcircuit where xs[i] = j means that j is the successor of i and xs[i] = i means that i is not in the circuit.

```
1.  predicate subgraph(int: N,
                       int: E,
                       array [int] of int: from,
                       array [int] of int: to,
                       array [int] of var bool: ns,
                       array [int] of var bool: es)

2.  predicate subgraph(array [$$E] of $$N: from,
                       array [$$E] of $$N: to,
                       array [$$N] of var bool: ns,
                       array [$$E] of var bool: es)
```

1. Constrains that ns and es is a subgraph of a given directed graph.

   Parameters:

   - N: the number of nodes in the given graph
   - E: the number of edges in the given graph
   - from: the leaving node 1..N for each edge
   - to: the entering node 1..N for each edge
   - ns: a Boolean for each node whether it is in the subgraph
   - es: a Boolean for each edge whether it is in the subgraph

2. Constrains that ns and es is a subgraph of a given directed graph.

   Parameters:

- `from`: the leaving node for each edge
- `to`: the entering node for each edge
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

```
1.  predicate tree(int: N,
                   int: E,
                   array [int] of int: from,
                   array [int] of int: to,
                   var int: root,
                   array [int] of var bool: ns,
                   array [int] of var bool: es)

2.  predicate tree(array [$$E] of $$N: from,
                   array [$$E] of $$N: to,
                   var $$N: root,
                   array [$$N] of var bool: ns,
                   array [$$E] of var bool: es)
```

1. Constrains the subgraph `ns` and `es` of a given undirected graph to be a tree rooted at `root`.

   Parameters:

   - `N`: the number of nodes in the given graph
   - `E`: the number of edges in the given graph
   - `from`: the leaving node 1..`N` for each edge
   - `to`: the entering node 1..`N` for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Constrains the subgraph `ns` and `es` of a given undirected graph to be at tree rooted at `root`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `root`: the root node (which may be variable)
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
predicate weighted_spanning_tree(array [$$E] of int: from,
                                 array [$$E] of int: to,
                                 array [$$E] of int: w,
                                 array [$$E] of var bool: es,
                                 var int: weight_bound)
predicate weighted_spanning_tree(int: N,
                                 int: E,
                                 array [int] of int: from,
                                 array [int] of int: to,
                                 array [int] of int: w,
                                 array [int] of var bool: es,
                                 var int: weight_bound)
```

Constrains the set of edges `es` of a given undirected graph to be a weighted spanning tree of weight `W`.

Parameters:

- `N`: the number of nodes in the given graph
- `E`: the number of edges in the given graph
- `from`: the leaving node 1..`N` for each edge
- `to`: the entering node 1..`N` for each edge
- `w`: the weight of each edge
- `es`: a Boolean for each edge whether it is in the subgraph
- `weight_bound`: the weight of the tree

4.2.2.12 Extensional constraints (table, regular etc.)

```
predicate cost_mdd(array [int] of var int: xs,
                   int: N,
                   array [int] of int: level,
                   int: E,
                   array [int] of int: from,
                   array [int] of set of int: label,
                   array [int] of int: cost,
                   array [int] of int: to,
                   var int: total_cost)
```

Requires that `xs` defines a path in the cost MDD with total edge weight `total_cost`.

Parameters:

- `N`: the number of nodes, the root node is node 1
- `level`: the level of each node, the root is level 1, T is level `length`(xs)+1
- `E`: the number of edges

- `from`: the leaving node (1..`N`)for each edge
- `label`: the set of value of the xs variable for each edge
- `cost`: the cost for each edge
- `to`: the entering node for each edge, where 0 = T node
- `total_cost`: the total cost of the path defined by `xs`

```
predicate cost_regular(array [int] of var int: xs,
                       int: Q,
                       int: S,
                       array [int,int] of int: d,
                       int: q0,
                       set of int: F,
                       array [int,int] of int: cost,
                       var int: total_cost)
```

The sequence of values in array `xs` (which must all be in the range 1..`S`) is accepted by the DFA of `Q` states with input 1..`S` and transition function `d` (which maps (1..`Q`, 1..`S`) -> 0..`Q`)) and initial state `q0` (which must be in 1..`Q`) and accepting states `F` (which all must be in 1..`Q`). We reserve state 0 to be an always failing state. Each edge has an associated cost `cost`, and `total_cost` is the sum of cost taken on the accepting path for `xs`.

```
predicate mdd(array [int] of var int: xs,
              int: N,
              array [int] of int: level,
              int: E,
              array [int] of int: from,
              array [int] of set of int: label,
              array [int] of int: to)
```

Requires that `xs` defines a path from root to true node T through the (deterministic) Multivalued Decision Diagram (MDD) defined by the parameters.

The MDD must be deterministic, i.e., there cannot be two edges with the same label leaving the same node.

Parameters:

- `N`: the number of nodes, the root node is node 1
- `level`: the level of each node, the root is level 1, T is level `length`(xs)+1
- `E`: the number of edges
- `from`: the leaving node (1..`N`)for each edge
- `label`: the set of values of the `xs` variable for each edge
- `to`: the entering node for each edge, where 0 = T node

```
predicate mdd_nondet(array [int] of var int: xs,
                     int: N,
                     array [int] of int: level,
                     int: E,
                     array [int] of int: from,
                     array [int] of set of int: label,
                     array [int] of int: to)
```

Requires that `xs` defines a path from root to true node T through the (nondeterministic) Multivalued Decision Diagram (MDD) defined by the parameters.

The MDD can be nondeterministic, i.e., there can be two edges with the same label leaving the same node.

Parameters:

- `N`: the number of nodes, the root node is node 1
- `level`: the level of each node, the root is level 1, T is level `length`(xs)+1
- `E`: the number of edges
- `from`: the leaving node (1..`N`)for each edge
- `label`: the set of values of the `xs` variable for each edge
- `to`: the entering node for each edge, where 0 = T node

```
1.  predicate regular(array [$$X] of var $$Val: xs,
                      array [$$State,$$Val] of opt $$State: d,
                      $$State: q0,
                      set of $$State: F)

2.  predicate regular(array [int] of var int: xs,
                      int: Q,
                      int: S,
                      array [int,int] of int: d,
                      int: q0,
                      set of int: F)

3.  predicate regular(array [int] of var int: xs,
                      int: Q,
                      set of int: S,
                      array [int,int] of int: d,
                      int: q0,
                      set of int: F)

4.  predicate regular(array [int] of var $$E: xs, string: regexp)
```

1. The sequence of values in array `xs` is accepted by the DFA with transition function `d`, initial state `q0` and accepting states `F`. The transition function maps states (from

enumerated type `State`) and values (from type `Val`) to states, or the absent value if the transition is not possible.

2. The sequence of values in array `xs` (which must all be in the range 1..`S`) is accepted by the DFA of `Q` states with input 1..`S` and transition function `d` (which maps (1..`Q`, 1..`S`) -> 0..`Q`)) and initial state `q0` (which must be in 1..`Q`) and accepting states `F` (which all must be in 1..`Q`). We reserve state 0 to be an always failing state.

3. The sequence of values in array `xs` (which must all be in the set `S`) is accepted by the DFA of `Q` states with input `S` and transition function `d` (which maps (1..`Q`, `S`) -> 0..`Q`)) and initial state `q0` (which must be in 1..`Q`) and accepting states `F` (which all must be in 1..`Q`). We reserve state 0 to be an always failing state.

4. The sequence of values in array `xs` is accepted by the regular expression `regexp`. This constraint generates its DFA equivalent.

   Regular expressions can use the following syntax:

   - Selection:
     - Concatenation: “12 34”, 12 followed by 34. (Characters are assumed to be the part of the same number unless split by syntax or whitespace.)
     - Union: “7|11”, a 7 or 11.
     - Groups: “7(6|8)”, a 7 followed by a 6 or an 8.
     - Wildcard: “.”, any value within the domain.
     - Classes: “[3-6 7]”, a 3,4,5,6, or 7.
     - Negated classes: “[^3 5]”, any value within the domain except for a 3 or a 5.
   - Quantifiers:
     - Asterisk: “12*”, 0 or more times a 12.
     - Question mark: “5?”, 0 or 1 times a 5. (optional)
     - Plus sign: “42+”, 1 or more time a 42.
     - Exact: “1{3}”, exactly 3 times a 1.
     - At least: “9{5,}”, 5 or more times a 9.
     - Between: “7{3,5}”, at least 3 times, but at most 5 times a 7.

   Members of enumerated types can be used in place of any integer (e.g., “A B”, A followed by B). Enumerated identifiers still use whitespace for concatenation.

```
1.  predicate regular_nfa(array [$$X] of var $$Val: xs,
                          array [$$State,$$Val] of set of $$State: d,
                          $$State: q0,
                          set of $$State: F)

2.  predicate regular_nfa(array [int] of var int: xs,
                          int: Q,
                          int: S,
```

```
                          array [int,int] of set of int: d,
                          int: q0,
                          set of int: F)

3.  predicate regular_nfa(array [int] of var int: xs,
                          int: Q,
                          set of int: S,
                          array [int,int] of set of int: d,
                          int: q0,
                          set of int: F)
```

1. The sequence of values in array `xs` is accepted by the NFA with transition function `d`, initial state `q0` and accepting states `F`. The transition function maps states (from enumerated type `State`) and values (from type `Val`) to sets of states.
2. The sequence of values in array `xs` (which must all be in the range 1..`S`) is accepted by the NFA of `Q` states with input 1..`S` and transition function `d` (which maps (1..`Q`, 1..`S`) -> set of 1..`Q`)) and initial state `q0` (which must be in 1..`Q`) and accepting states `F` (which all must be in 1..`Q`).
3. The sequence of values in array `xs` (which must all be in the range `S`) is accepted by the NFA of `Q` states with input `S` and transition function `d` (which maps (1..`Q`, `S`) -> set of 1..`Q`)) and initial state `q0` (which must be in 1..`Q`) and accepting states `F` (which all must be in 1..`Q`).

```
1.  predicate table(array [$$E] of var bool: xs,
                    array [$$X,$$E] of bool: tuples)

2.  predicate table(array [$$E] of var int: xs,
                    array [$$X,$$E] of int: tuples)

3.  predicate table(array [$$E] of var opt int: xs,
                    array [$$X,$$E] of opt int: tuples)
```

1, 2.
Represents the constraint `xs` in `tuples` where we consider each row in `tuples` to be a tuple and `tuples` as a set of tuples.

3. Represents the constraint `xs` in `tuples` if the variable `x_i` and the value `t_i` occur where we consider each row in `tuples` to be a tuple and `tuples` as a set of tuples.

4.2.2.13 Machine learning constraints

```
predicate neural_net(array [int] of var float: inputs,
                     array [int] of int: input_id,
                     array [int] of var float: outputs,
                     array [int] of int: output_id,
```

```
                    array [int] of float: bias,
                    array [int] of float: edge_weight,
                    array [int] of int: edge_parent,
                    array [int] of int: first_edge,
                    NEURON_TYPE: neuron_type)
```

Constrain the output layer of a neural net to take the value defined by the input layer.

Parameters:

- `inputs`: an array of float variables
- `input_id`: array[int] of node
- `outputs`: an array of float variables
- `output_id`: array[int] of node
- `bias`: array[node] of float
- `edge_weight`: array[edge] of float
- `edge_parent`: array[edge] of neuron (start neuron for edge)
- `first_edge`: array[node] of 1..m+1
- `neuron_type`: { NT_RELU, NT_STEP, NT_LINEAR, NT_SOFTPLUS }

4.2.2.14 Deprecated constraints

```
predicate at_least(int: n, array [$X] of var $$E: xs, $$E: value)
```

Requires at least `n` variables in `xs` to take the value `value`.

This constraint is deprecated. Use count(i in xs)(i=value) >= n instead.

```
predicate at_most(int: n, array [$X] of var $$E: xs, $$E: value)
```

Requires at most `n` variables in `xs` to take the value `value`.

This constraint is deprecated. Use count(i in xs)(i=value) <= n instead.

```
predicate exactly(int: n, array [$X] of var $$E: xs, $$E: value)
```

Requires exactly `n` variables in `xs` to take the value `value`.

This constraint is deprecated. Use count(i in xs)(i=value) = n instead.

```
predicate global_cardinality_low_up(array [$X] of var int: x,
                                    array [$Y] of int: cover,
                                    array [$Y] of int: lower_bound,
                                    array [$Y] of int: upper_bound)
```

Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `x`.

This constraint is deprecated. Use global_cardinality(x, cover, lower_bound, upper_bound) instead.

```
predicate global_cardinality_low_up_closed(array [$X] of var int: x,
                                           array [$Y] of int: cover,
                                           array [$Y] of int: lower_bound,
                                           array [$Y] of int: upper_bound)
```

Requires that for all `i`, the value `cover[i]` appears at least `lower_bound[i]` and at most `upper_bound[i]` times in the array `x`.

The elements of `x` must take their values from `cover`.

This constraint is deprecated. Use global_cardinality_closed(x, cover, lower_bound, upper_bound) instead.

4.2.3 FlatZinc builtins

These are the standard constraints that need to be supported by FlatZinc solvers (or redefined in the redefinitions.mzn file).

4.2.3.1 Integer FlatZinc builtins

```
predicate array_int_element(var int: idx,
                            array [int] of int: xs,
                            var int: y)
```

Constrains `xs[idx] = y`

```
predicate array_var_int_element(var int: idx,
                                array [int] of var int: xs,
                                var int: y)
```

Constrains `xs[idx] = y`

```
predicate int_abs(var int: x, var int: y)
```

Constrains y to be the absolute value of x

```
predicate int_div(var int: x, var int: y, var int: z)
```

Constrains x / y = z, where z is rounded to the value closest to zero (i.e., truncated).

```
predicate int_eq(var int: x, var int: y)
```

Constrains x to be equal to y

```
predicate int_eq_reif(var int: x, var int: y, var bool: b)
```

Constrains (x=y) $\leftrightarrow$ b

```
predicate int_le(var int: x, var int: y)
```

Constrains x to be less than or equal to y

```
predicate int_le_reif(var int: x, var int: y, var bool: b)
```

Constrains (x □ y) $\leftrightarrow$ b

```
predicate int_lin_eq(array [int] of int: as,
                     array [int] of var int: xs,
                     int: y)
```

Constrains $\mathbf{y} = \sum_i \mathbf{as}[i] * \mathbf{xs}[i]$

```
predicate int_lin_eq_reif(array [int] of int: as,
                          array [int] of var int: xs,
                          int: y,
                          var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{y} = \sum_i \mathbf{as}[i] * \mathbf{xs}[i])$

```
predicate int_lin_le(array [int] of int: as,
                     array [int] of var int: xs,
                     int: y)
```

Constrains $\sum$ as[i]*xs[i] □ y

```
predicate int_lin_le_reif(array [int] of int: as,
                          array [int] of var int: xs,
                          int: y,
                          var bool: b)
```

Constrains b $\leftrightarrow$ ($\sum$ as[i]*xs[i] □ y)

```
predicate int_lin_ne(array [int] of int: as,
                     array [int] of var int: xs,
                     int: y)
```

Constrains $\mathbf{y} \neq \sum_i \mathbf{as}[i] * \mathbf{xs}[i]$

```
predicate int_lin_ne_reif(array [int] of int: as,
                          array [int] of var int: xs,
                          int: y,
                          var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{y} \neq \sum_i \mathbf{as}[i] * \mathbf{xs}[i])$

```
predicate int_lt(var int: x, var int: y)
```

Constrains x < y

```
predicate int_lt_reif(var int: x, var int: y, var bool: b)
```

Constrains b $\leftrightarrow$ (x < y)

```
predicate int_max(var int: x, var int: y, var int: z)
```

Constrains max(x, y) = z

```
predicate int_min(var int: x, var int: y, var int: z)
```

Constrains min(x, y) = z

```
predicate int_mod(var int: x, var int: y, var int: z)
```

Constrains x % y = z

```
predicate int_ne(var int: x, var int: y)
```

Constrains x ≠ y

```
predicate int_ne_reif(var int: x, var int: y, var bool: b)
```

b ↔ (x ≠ y)

```
predicate int_plus(var int: x, var int: y, var int: z)
```

Constrains x + y = z

```
predicate int_pow(var int: x, var int: y, var int: z)
```

Constrains z = $\mathbf{x}^{\mathbf{y}}$, {bf z} is constrained to `1 div pow(x, abs(y))` when $\mathbf{y} < 0$

```
predicate int_times(var int: x, var int: y, var int: z)
```

Constrains x * y = z

```
predicate set_in(var int: x, set of int: s)
```

Constrains x ∈ s

4.2.3.2 Bool FlatZinc builtins

```
predicate array_bool_and(array [int] of var bool: xs, var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow \bigwedge_i \mathbf{xs}[i]$

```
predicate array_bool_element(var int: idx,
                             array [int] of bool: xs,
                             var bool: y)
```

Constrains `xs[idx]` = `y`

```
predicate array_bool_xor(array [int] of var bool: xs)
```

Constrains $\oplus_i \mathbf{xs}[i]$

```
predicate array_var_bool_element(var int: idx,
                                 array [int] of var bool: xs,
                                 var bool: y)
```

Constrains `xs[idx]` = `y`

```
predicate bool2int(var bool: x, var int: y)
```

Constrains $\mathbf{y} \in \{0, 1\}$ and $\mathbf{x} \leftrightarrow \mathbf{y} = 1$

```
predicate bool_and(var bool: x, var bool: y, var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow \mathbf{x} \wedge \mathbf{y}$

```
predicate bool_clause(array [int] of var bool: xs,
                      array [int] of var bool: ys)
```

Constrains $\bigvee_i \mathbf{xs}[i] \vee \bigvee_j \neg\mathbf{ys}[j]$

```
predicate bool_eq(var bool: x, var bool: y)
```

Constrains x = y

```
predicate bool_eq_reif(var bool: x, var bool: y, var bool: b)
```

Constrains b ↔ (x = y)

```
predicate bool_le(var bool: x, var bool: y)
```

Constrains x □ y

```
predicate bool_le_reif(var bool: x, var bool: y, var bool: b)
```

Constrains b ↔ (x □ y)

```
predicate bool_lin_eq(array [int] of int: as,
                      array [int] of var bool: xs,
                      var int: y)
```

Constrains $\mathbf{y} = \sum_i \mathbf{as}[i] * \mathbf{xs}[i]$

```
predicate bool_lin_le(array [int] of int: as,
                      array [int] of var bool: xs,
                      int: y)
```

Constrains $\sum_i \mathbf{as}[i] * \mathbf{xs}[i] \leq \mathbf{y}$

```
predicate bool_lt(var bool: x, var bool: y)
```

Constrains x < y

```
predicate bool_lt_reif(var bool: x, var bool: y, var bool: b)
```

Constrains b ↔ (x < y)

```
predicate bool_not(var bool: x, var bool: y)
```

Constrains x ≠ y

```
predicate bool_or(var bool: x, var bool: y, var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow \mathbf{x} \vee \mathbf{y}$

```
predicate bool_xor(var bool: x, var bool: y, var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow \mathbf{x} \oplus \mathbf{y}$

4.2.3.3 Set FlatZinc builtins

```
predicate array_set_element(var int: idx,
                            array [int] of set of int: xs,
                            var set of int: y)
```

Constrains xs[idx] = y

```
predicate array_var_set_element(var int: idx,
                                array [int] of var set of int: xs,
                                var set of int: y)
```

Constrains xs[idx] = y

```
predicate set_card(var set of int: s, var int: x)
```

Constrains x = |s|

```
predicate set_diff(var set of int: s,
                   var set of int: t,
                   var set of int: r)
```

Constrains r = s \ t

```
predicate set_eq(var set of int: s, var set of int: t)
```

Constrains s = t

```
predicate set_eq_reif(var set of int: s,
                      var set of int: t,
                      var bool: b)
```

Constrains b ↔ (s = t)

```
predicate set_in(var int: x, var set of int: s)
```

Constrains x ∈ s

```
predicate set_in_reif(var int: x, set of int: s, var bool: b)
predicate set_in_reif(var int: x, var set of int: s, var bool: b)
```

Constrains **b** ↔ (**x** ∈ **s**)

```
predicate set_intersect(var set of int: s,
                        var set of int: t,
                        var set of int: r)
```

Constrains r = s ∩ t

```
predicate set_le(var set of int: s, var set of int: t)
```

Constrains s t (lexicographic order of the sorted lists of elements)

```
predicate set_le_reif(var set of int: s,
                      var set of int: t,
                      var bool: b)
```

Constrains **b** ↔ (**s** ≤ **t**) (lexicographic order of the sorted lists of elements)

```
predicate set_lt(var set of int: s, var set of int: t)
```

Constrains s < t (lexicographic order of the sorted lists of elements)

```
predicate set_lt_reif(var set of int: s,
                      var set of int: t,
                      var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{s} < \mathbf{t})$ (lexicographic order of the sorted lists of elements)

```
predicate set_ne(var set of int: s, var set of int: t)
```

Constrains $s \neq t$

```
predicate set_ne_reif(var set of int: s,
                      var set of int: t,
                      var bool: b)
```

Constrains $b \leftrightarrow (s \neq t)$

```
predicate set_subset(var set of int: s, var set of int: t)
```

Constrains $s \subseteq t$

```
predicate set_subset_reif(var set of int: s,
                          var set of int: t,
                          var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{s} \subseteq \mathbf{t})$

```
predicate set_superset(var set of int: s, var set of int: t)
```

Constrains $s \supseteq t$

```
predicate set_superset_reif(var set of int: s,
                            var set of int: t,
                            var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{s} \subseteq \mathbf{t})$

```
predicate set_symdiff(var set of int: s,
                      var set of int: t,
                      var set of int: r)
```

Constrains `r` to be the symmetric difference of `s` and `t`

```
predicate set_union(var set of int: s,
                    var set of int: t,
                    var set of int: r)
```

Constrains `r` = `s` ∪ `t`

4.2.3.4 Float FlatZinc builtins

```
predicate array_float_element(var int: idx,
                              array [int] of float: xs,
                              var float: y)
```

Constrains `xs[idx]` = `y`

```
predicate array_var_float_element(var int: idx,
                                  array [int] of var float: xs,
                                  var float: y)
```

Constrains `xs[idx]` = `y`

```
predicate float_abs(var float: x, var float: y)
```

Constrains `y` to be the absolute value of `x`

```
predicate float_acos(var float: x, var float: y)
```

Constrains `y` = acos(`x`)

```
predicate float_acosh(var float: x, var float: y)
```

Constrains `y` = acosh(`x`)

```
predicate float_asin(var float: x, var float: y)
```

Constrains y = asin(x)

```
predicate float_asinh(var float: x, var float: y)
```

Constrains y = asinh(x)

```
predicate float_atan(var float: x, var float: y)
```

Constrains y = atan(x)

```
predicate float_atanh(var float: x, var float: y)
```

Constrains y = atanh(x)

```
predicate float_cos(var float: x, var float: y)
```

Constrains y = cos(x)

```
predicate float_cosh(var float: x, var float: y)
```

Constrains y = cosh(x)

```
predicate float_div(var float: x, var float: y, var float: z)
```

Constrains x / y = z

```
predicate float_eq(var float: x, var float: y)
```

Constrains x = y

```
predicate float_eq_reif(var float: x, var float: y, var bool: b)
```

Constrains b ↔ (x = y)

```
predicate float_exp(var float: x, var float: y)
```

Constrains y = exp(x)

```
predicate float_le(var float: x, var float: y)
```

Constrains x □ y

```
predicate float_le_reif(var float: x, var float: y, var bool: b)
```

Constrains b ↔ (x □ y)

```
predicate float_lin_eq(array [int] of float: as,
                       array [int] of var float: xs,
                       float: y)
```

Constrains $\mathbf{y} = \sum_i \mathbf{as}[i] * \mathbf{xs}[i]$

```
predicate float_lin_eq_reif(array [int] of float: as,
                            array [int] of var float: xs,
                            float: y,
                            var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{y} = \sum_i \mathbf{as}[i] * \mathbf{xs}[i])$

```
predicate float_lin_le(array [int] of float: as,
                       array [int] of var float: xs,
                       float: y)
```

Constrains $\sum_i \mathbf{as}[i] * \mathbf{xs}[i] \leq \mathbf{y}$

```
predicate float_lin_le_reif(array [int] of float: as,
                            array [int] of var float: xs,
                            float: y,
                            var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\sum_i \mathbf{as}[i] * \mathbf{xs}[i] \leq \mathbf{y})$

```
predicate float_lin_lt(array [int] of float: as,
                       array [int] of var float: xs,
                       float: y)
```

Constrains $\sum_i \mathbf{as}[i] * \mathbf{xs}[i] < \mathbf{y}$

```
predicate float_lin_lt_reif(array [int] of float: as,
                            array [int] of var float: xs,
                            float: y,
                            var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\sum_i \mathbf{as}[i] * \mathbf{xs}[i] < \mathbf{y})$

```
predicate float_lin_ne(array [int] of float: as,
                       array [int] of var float: xs,
                       float: y)
```

Constrains $\mathbf{y} \neq \sum_i \mathbf{as}[i] * \mathbf{xs}[i]$

```
predicate float_lin_ne_reif(array [int] of float: as,
                            array [int] of var float: xs,
                            float: y,
                            var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow (\mathbf{y} \neq \sum_i \mathbf{as}[i] * \mathbf{xs}[i])$

```
predicate float_ln(var float: x, var float: y)
```

Constrains y = ln(x)

```
predicate float_log10(var float: x, var float: y)
```

Constrains y = log₁₀(x)

```
predicate float_log2(var float: x, var float: y)
```

Constrains y = log₂(x)

```
predicate float_lt(var float: x, var float: y)
```

Constrains x < y

```
predicate float_lt_reif(var float: x, var float: y, var bool: b)
```

Constrains b ↔ (x < y)

```
predicate float_max(var float: x, var float: y, var float: z)
```

Constrains max(x, y) = z

```
predicate float_min(var float: x, var float: y, var float: z)
```

Constrains min(x, y) = z

```
predicate float_ne(var float: x, var float: y)
```

Constrains x ≠ y

```
predicate float_ne_reif(var float: x, var float: y, var bool: b)
```

Constrains b ↔ (x ≠ y)

```
predicate float_plus(var float: x, var float: y, var float: z)
```

Constrains x + y = z

```
predicate float_pow(var float: x, var float: y, var float: z)
```

Constrains z = $\mathbf{x}^{\mathbf{y}}$

```
predicate float_sin(var float: x, var float: y)
```

Constrains y = sin(x)

```
predicate float_sinh(var float: x, var float: y)
```

Constrains y = sinh(x)

```
predicate float_sqrt(var float: x, var float: y)
```

Constrains $\mathbf{y} = \sqrt{\mathbf{x}}$

```
predicate float_tan(var float: x, var float: y)
```

Constrains y = tan(x)

```
predicate float_tanh(var float: x, var float: y)
```

Constrains y = tanh(x)

```
predicate float_times(var float: x, var float: y, var float: z)
```

Constrains x * y = z

```
predicate int2float(var int: x, var float: y)
```

Constrains y=x

4.2.3.5 FlatZinc builtins added in MiniZinc 2.0.0.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.0.0. Solvers that support these natively need to include a file called redefinitions-2.0.mzn in their solver library that redefines these predicates as builtins.

```
predicate array_float_maximum(var float: m,
                              array [int] of var float: xs)
```

Constrains `m` to be the maximum value in array `xs`.

```
predicate array_float_minimum(var float: m,
                              array [int] of var float: xs)
```

Constrains `m` to be the minimum value in array `xs`.

```
predicate array_int_maximum(var int: m, array [int] of var int: xs)
```

Constrains `m` to be the maximum value in array `xs`.

```
predicate array_int_minimum(var int: m, array [int] of var int: xs)
```

Constrains `m` to be the minimum value in array `xs`.

```
predicate bool_clause_reif(array [int] of var bool: xs,
                           array [int] of var bool: ys,
                           var bool: b)
```

Reified clause constraint. Constrains $\mathbf{b} \leftrightarrow \bigvee_i \mathbf{xs}[i] \vee \bigvee_j \neg \mathbf{ys}[j]$

4.2.3.6 FlatZinc builtins added in MiniZinc 2.0.2.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.0.2. Solvers that support these natively need to include a file called redefinitions-2.0.2.mzn in their solver library that redefines these predicates as builtins.

```
predicate array_var_bool_element_nonshifted(var int: idx,
                                            array [int] of var bool: xs,
                                            var bool: y)
```

Element constraint on array with MiniZinc index set, constrains `xs[idx] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_float_element_nonshifted(var int: idx,
                                             array [int] of var float: xs,
                                             var float: y)
```

Element constraint on array with MiniZinc index set, constrains `xs[idx] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_int_element_nonshifted(var int: idx,
                                           array [int] of var int: xs,
                                           var int: y)
```

Element constraint on array with MiniZinc index set, constrains `xs[idx] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_set_element_nonshifted(var int: idx,
                                           array [int] of var set of int: xs,
                                           var set of int: y)
```

Element constraint on array with MiniZinc index set, constrains `xs[idx] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

4.2.3.7 FlatZinc builtins added in MiniZinc 2.1.0.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.1.0. Solvers that support these natively need to include a file called redefinitions-2.1.0.mzn in their solver library that redefines these predicates as builtins.

```
predicate float_dom(var float: x, array [int] of float: as)
```

Constrains the domain of `x` using the values in `as`, using each pair of values `as[2*i-1]..as[2*i]` for `i` in 1..n/2 as a possible range

---

```
predicate float_in(var float: x, float: l, float: u)
```

Constrains $\mathbf{x} \in [\mathbf{l}, \mathbf{u}]$

4.2.3.8 FlatZinc builtins added in MiniZinc 2.1.1.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.1.1. Solvers that support these natively need to include a file called redefinitions-2.1.1.mzn in their solver library that redefines these predicates as builtins.

```
function var $$E: max(var set of $$E: s)
```

Returns variable constrained to be equal to the maximum of the set `s`. An alternative implementation can be found in the comments of the source code.

```
function var $$E: min(var set of $$E: s)
```

Returns variable constrained to be equal to the minimum of the set `s`. An alternative implementation can be found in the comments of the source code.

4.2.3.9 FlatZinc builtins added in MiniZinc 2.2.1.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.2.1. Solvers that support these natively need to include a file called redefinitions-2.2.1.mzn in their solver library that redefines these predicates as builtins.

```
predicate int_pow_fixed(var int: x, int: y, var int: z)
```

Constrains z = $\mathbf{x}^{\mathbf{y}}$, {bf z} is constrained to `1 div pow(x, abs(y))` when $\mathbf{y} < 0$ *

4.2.3.10 FlatZinc builtins added in MiniZinc 2.3.3.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.3.3. Solvers that support these natively need to include a file called redefinitions-2.3.3.mzn in their solver library that redefines these predicates as builtins.

```
predicate float_set_in(var float: x, set of float: s)
```

Constrains x $\in$ s

4.2.3.11 FlatZinc builtins added in MiniZinc 2.5.2.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.5.2. Solvers that support these natively need to include a file called redefinitions-2.5.2.mzn in their solver library that redefines these predicates as builtins.

```
predicate array_var_bool_element2d_nonshifted(var int: idx1,
                                               var int: idx2,
                                               array [int,int] of var bool:␣
```

```
↪xs,
                                                var bool: y)
```

Element constraint on 2d array with MiniZinc index set, constrains `xs[idx1,idx2] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_float_element2d_nonshifted(var int: idx1,
                                               var int: idx2,
                                               array [int,int] of var float:␣
↪xs,
                                               var float: y)
```

Element constraint on 2d array with MiniZinc index set, constrains `xs[idx1,idx2] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_int_element2d_nonshifted(var int: idx1,
                                             var int: idx2,
                                             array [int,int] of var int: xs,
                                             var int: y)
```

Element constraint on 2d array with MiniZinc index set, constrains `xs[idx1,idx2] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

```
predicate array_var_set_element2d_nonshifted(var int: idx1,
                                             var int: idx2,
                                             array [int,int] of var set of␣
↪int: xs,
                                             var set of int: y)
```

Element constraint on 2d array with MiniZinc index set, constrains `xs[idx1,idx2] = y` This can be overridden in a solver that can perform the index calculation more efficiently than using a MiniZinc decomposition.

4.2.3.12 FlatZinc builtins added in MiniZinc 2.7.1.

These functions and predicates define built-in operations of the MiniZinc language that have been added in MiniZinc 2.7.1. Solvers that support these natively need to include a file called redefinitions-2.7.1.mzn in their solver library that redefines these predicates as builtins.

```
predicate float_ceil(var float: x, var int: y)
```

Constrains $\mathbf{y} = \lceil \mathbf{x} \rceil$

```
predicate float_floor(var float: x, var int: y)
```

Constrains $\mathbf{y} = \lfloor \mathbf{x} \rfloor$

```
predicate float_round(var float: x, var int: y)
```

Constrains that `y` is the nearest integer to `x`

4.2.3.13 Deprecated FlatZinc builtins

These builtin constraints are no longer produced by the MiniZinc compiler, and they will be removed in future versions of MiniZinc.

```
predicate array_bool_or(array [int] of var bool: xs, var bool: b)
```

Constrains $\mathbf{b} \leftrightarrow \bigvee_i \mathbf{xs}[i]$

4.2.4 Additional declarations for Gecode

These annotations and predicates are available for the Gecode solver. In order to use them in a model, include the file "gecode.mzn".

4.2.4.1 Additional Gecode search annotations

```
annotation action_max
```

Select variable with largest action count

```
annotation action_min
```

Select variable with smallest action count

```
annotation action_size_max
```

Select variable with largest action count divided by domain size

---

```
annotation action_size_min
```

Select variable with smallest action count divided by domain size

---

```
annotation afc_max
```

Select variable with largest accumulated failure count

```
annotation afc_min
```

Select variable with smallest accumulated failure count

```
annotation afc_size_max
```

Select variable with largest accumulated failure count divided by domain size

---

```
annotation afc_size_min
```

Select variable with smallest accumulated failure count divided by domain size

---

```
annotation bool_default_search(ann: varsel, ann: valsel)
```

Specify default search strategy for Boolean variables to use variable selection strategy `varsel`, and value choice strategy `valsel`.

```
annotation float_default_search(ann: varsel, ann: valsel)
```

Specify default search strategy for float variables to use variable selection strategy `varsel`, and value choice strategy `valsel`.

```
annotation int_default_search(ann: varsel, ann: valsel)
```

Specify default search strategy for integer variables to use variable selection strategy `varsel`, and value choice strategy `valsel`.

```
annotation random
```

Select random variable

```
1.  annotation relax_and_reconstruct(array [int] of var int: x,
                                     int: percentage)

2.  annotation relax_and_reconstruct(array [int] of var int: x,
                                     int: percentage,
                                     array [int] of int: y)
```

1. Simple large neighbourhood search strategy: upon restart, for each variable in `x`, the probability of it being fixed to the previous solution is `percentage` (out of 100).
2. Simple large neighbourhood search strategy: upon restart, for each variable in `x`, the probability of it being fixed to the previous solution is `percentage` (out of 100). Start from an initial solution `y`.

```
annotation set_default_search(ann: varsel, ann: valsel)
```

Specify default search strategy for set variables to use variable selection strategy `varsel`, and value choice strategy `valsel`.

4.2.4.2 Additional Gecode constraints

```
1.  predicate among_seq(array [int] of var int: x,
                        set of int: S,
                        int: l,
                        int: m,
                        int: n)

2.  predicate among_seq(array [int] of var bool: x,
                        bool: b,
                        int: l,
                        int: m,
                        int: n)
```

1. Every subsequence of `x` of length `l` has at least `m` and at most `n` occurrences of the values in `S`
2. Every subsequence of `x` of length `l` has at least `m` and at most `n` occurrences of the value `b`

```
predicate circuit_cost(array [int] of int: c,
                       array [int] of var int: x,
                       var int: z)
```

Constrains the elements of x to define a circuit where x[i] = j means that j is the successor of i. Additionally, constrain z to be the cost of the circuit. Each edge cost is defined by array c.

```
predicate circuit_cost_array(array [int] of int: c,
                             array [int] of var int: x,
                             array [int] of var int: y,
                             var int: z)
```

Constrains the elements of x to define a circuit where x[i] = j means that j is the successor of i. Additionally, constrain z to be the cost of the circuit. Each edge cost is defined by array c. The variables y[i] are constrained to be the edge cost of the node x[i].

```
predicate gecode_array_set_element_intersect(var set of int: x,
                                             array [int] of var set of int:␣
↪y,
                                             var set of int: z)
```

Constrain z to be the intersection of all sets in y that are selected by x: $i \in \mathbf{z} \leftrightarrow \forall j \in \mathbf{x} : (i \in \mathbf{y}[j])$

```
predicate gecode_array_set_element_intersect_in(var set of int: x,
                                                array [int] of var set of␣
↪int: y,
                                                var set of int: z,
                                                set of int: u)
```

Constrain z to be a subset of u, and z to be the intersection of all sets in y that are selected by x: $i \in \mathbf{z} \leftrightarrow \forall j \in \mathbf{x} : (i \in \mathbf{y}[j])$

```
predicate gecode_array_set_element_partition(var set of int: x,
                                             array [int] of var set of int:␣
↪y,
                                             var set of int: z)
```

Constrain z to be the disjoint union of all sets in y that are selected by x: $i \in \mathbf{z} \leftrightarrow \exists j \in \mathbf{x} : (i \in \mathbf{y}[j])$ and $i \in \mathbf{x} \wedge j \in \mathbf{x} \wedge i \neq j \rightarrow \mathbf{y}[i] \cap \mathbf{y}[j] = \emptyset$

4.2.5 Additional declarations for Chuffed

These annotations and predicates are available for the Chuffed solver. In order to use them in a model, include the file "chuffed.mzn".

4.2.5.1 Additional Chuffed search annotations

```
annotation largest_smallest
```

Choose the variable with the largest smallest value in its domain

---

```
annotation priority_search(array [int] of var int: x,
                           array [int] of ann: search,
                           ann: select,
                           ann: explore)
```

Specify search on a group of searches `search`, for which variables `x` act as delegates for the search selection with variable selection strategy `select`, and exploration strategy `explore`.

```
annotation random_order
```

Choose a random variable

```
annotation smallest_largest
```

Choose the variable with the smallest largest value in its domain

---

4.2.5.2 Other declarations

```
predicate chuffed_minimal_spanning_tree(int: N,
                                        int: E,
                                        array [int] of int: from,
                                        array [int] of int: to,
                                        array [int] of int: w,
                                        array [int] of var bool: vs,
                                        array [int] of var bool: es,
                                        var int: K)
```

Constrains the subgraph `vs` and `es` of a given directed graph to be a minimal spanning tree with weight `K`.

Parameters:

- `N`: the number of nodes in the given graph
- `E`: the number of edges in the given graph
- `from`: the leaving node 1..`N` for each edge
- `to`: the entering node 1..`N` for each edge
- `w`: the weight of each edge
- `vs`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph
- `K`: the cost of the edges in the minimal spanning tree

4.2.6 MiniZincIDE tools

This library contains features designed for use from within the MiniZinc IDE. These are not guaranteed to work when run outside the IDE.

4.2.6.1 MiniZincIDE solution visualisation tools

Provides functionality for web-based solution visualisation.

All predefined visualisations are designed to used from `output` items:

```
var 1..10: x;
output vis_line(x, "x-value");
```

Multiple such output items can be used in a single model to open multiple visualisations concurrently.

Custom visualisations can be created using the `vis_server` function:

```
int: n;
var 1..10: x;
output :: vis_server("my-vis.html", (n: n)) (x: x);
```

The file `my-vis.html` will be opened and the MiniZinc IDE JavaScript API can be used to listen for events and retrieve solutions. For more information, see *Visualising solutions in the MiniZinc IDE* (page 179).

```
1.  annotation vis_bar(array [$$D] of var int: x)

2.  annotation vis_bar(array [$$D] of var float: x)

3.  annotation vis_bar(array [$$D] of var int: x,
                       array [$$D] of string: data_labels)
```

```
4.  annotation vis_bar(array [$$D] of var float: x,
                       array [$$D] of string: data_labels)

5.  annotation vis_bar(array [$$S,$$D] of var int: x)

6.  annotation vis_bar(array [$$S,$$D] of var float: x)

7.  annotation vis_bar(array [$$S,$$D] of var int: x,
                       array [$$S] of string: series_labels,
                       array [$$D] of string: data_labels)

8.  annotation vis_bar(array [$$S,$$D] of var float: x,
                       array [$$S] of string: series_labels,
                       array [$$D] of string: data_labels)
```

1, 2.
Visualise a bar chart of the values in `x` (labelled by index set)

3, 4.
Visualise a bar chart of the values in `x`. Chart values are labelled with `data_labels`.

5, 6.
Visualise a bar chart with grouped data (labelled by index set). `x[i, j]` is the value for `j` in data series `i`.

7, 8.
Visualise a bar chart with grouped data. `x[i, j]` is the value for `j` in data series `i`. Series are labelled with `series_labels`. Chart values are labelled with `data_labels`.

```
1.  annotation vis_column(array [$$D] of var int: x)

2.  annotation vis_column(array [$$D] of var float: x)

3.  annotation vis_column(array [$$D] of var int: x,
                          array [$$D] of string: data_labels)

4.  annotation vis_column(array [$$D] of var float: x,
                          array [$$D] of string: data_labels)

5.  annotation vis_column(array [$$S,$$D] of var int: x)

6.  annotation vis_column(array [$$S,$$D] of var float: x)

7.  annotation vis_column(array [$$S,$$D] of var int: x,
                          array [$$S] of string: series_labels,
                          array [$$D] of string: data_labels)

8.  annotation vis_column(array [$$S,$$D] of var float: x,
                          array [$$S] of string: series_labels,
                          array [$$D] of string: data_labels)
```

1, 2.
Visualise a column chart of the values in `x` (labelled by index set)

3, 4.
Visualise a column chart of the values in `x`. Chart values are labelled with `data_labels`.

5, 6.
Visualise a column chart with grouped data (labelled by index set). `x[i, j]` is the value for `j` in data series `i`.

7, 8.
Visualise a column chart with grouped data. `x[i, j]` is the value for `j` in data series `i`. Series are labelled with `series_labels`. Chart values are labelled with `data_labels`.

```
1.  annotation vis_digraph(array [$$E] of $$N: from,
                           array [$$E] of $$N: to,
                           array [$$E] of string: edge_labels,
                           array [$$N] of var bool: ns,
                           array [$$E] of var bool: es)

2.  annotation vis_digraph(array [$$E] of $$N: from,
                           array [$$E] of $$N: to,
                           array [$$N] of var bool: ns,
                           array [$$E] of var bool: es)

3.  annotation vis_digraph(array [$$N] of string: node_labels,
                           array [$$E] of $$N: from,
                           array [$$E] of $$N: to,
                           array [$$E] of string: edge_labels,
                           array [$$N] of var bool: ns,
                           array [$$E] of var bool: es)
```

1. Create a visualisation of the subgraph `ns` and `es` of a given directed graph. The nodes are labelled using the index set of `ns`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Create a visualisation of the subgraph `ns` and `es` of a given directed graph. The nodes are labelled using the index set of `ns` and the edges are labelled using the index set of `es`.

   Parameters:

   - `from`: the leaving node for each edge

- `to`: the entering node for each edge
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

3. Create a visualisation of the subgraph `ns` and `es` of a given directed graph.

   Parameters:

   - `node_labels`: the label for each node
   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
1.  annotation vis_digraph_highlight(array [$$E] of $$N: from,
                                     array [$$E] of $$N: to,
                                     array [$$E] of string: edge_labels,
                                     array [$$N] of var bool: ns,
                                     array [$$E] of var bool: es)

2.  annotation vis_digraph_highlight(array [$$E] of $$N: from,
                                     array [$$E] of $$N: to,
                                     array [$$N] of var bool: ns,
                                     array [$$E] of var bool: es)

3.  annotation vis_digraph_highlight(array [$$N] of string: node_labels,
                                     array [$$E] of $$N: from,
                                     array [$$E] of $$N: to,
                                     array [$$E] of string: edge_labels,
                                     array [$$N] of var bool: ns,
                                     array [$$E] of var bool: es)
```

1. Create a visualisation of the subgraph `ns` and `es` of a given directed graph. The nodes are labelled using the index set of `ns`. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Create a visualisation of the subgraph `ns` and `es` of a given directed graph. The nodes are labelled using the index set of `ns` and the edges are labelled using the index set of `es`. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

3. Create a visualisation of the subgraph `ns` and `es` of a given directed graph. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `node_labels`: the label for each node
   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

---

```
1.  annotation vis_gantt(array [$$E] of var int: start,
                         array [$$E] of var int: dur)

2.  annotation vis_gantt(array [$$E] of var int: start,
                         array [$$E] of var int: dur,
                         array [$$E] of string: labels)

3.  annotation vis_gantt(array [$$E] of var int: start,
                         array [$$E] of var int: dur,
                         array [$$E] of string: labels,
                         array [$$E] of string: colors)
```

1. Visualise a Gantt chart of tasks with start times `start` and durations `dur`. The tasks are labelled using the index set of `start`.
2. Visualise a Gantt chart of tasks with start times `start` and durations `dur`. The tasks are labelled using `labels`.
3. Visualise a Gantt chart of tasks with start times `start` and durations `dur`. The tasks are labelled using `labels` and given the colours `colors`.

```
1.  annotation vis_geost_2d(array [$$E] of int: rect_x,
                            array [$$E] of int: rect_y,
                            array [$$E] of int: rect_dx,
```

```
                            array [$$E] of int: rect_dy,
                            array [$$K] of set of $$E: shape,
                            array [$$T] of var int: x,
                            array [$$T] of var int: y,
                            array [$$T] of var $$K: kind)

2.  annotation vis_geost_2d(array [$$E,1..2] of int: rect_size,
                            array [$$E,1..2] of int: rect_offset,
                            array [$$K] of set of $$E: shape,
                            array [$$T,1..2] of var int: x,
                            array [$$T] of var $$K: kind)

3.  annotation vis_geost_2d(array [$$E] of tuple(int, int): rect_size,
                            array [$$E] of tuple(int, int): rect_offset,
                            array [$$K] of set of $$E: shape,
                            array [$$T] of tuple(var int, var int): x,
                            array [$$T] of var $$K: kind)
```

1. Visualise the positions of 2-dimensional objects which take a particular shape composed of rectangles.

   Parameters:

   - `rect_x`: x-position of each rectangle
   - `rect_y`: y-position of each rectangle
   - `rect_dx`: width of each rectangle
   - `rect_dy`: height of each rectangle
   - `shape`: the rectangles which comprise each shape
   - `x`: the x-position of each object
   - `y`: the y-position of each object
   - `kind`: which shape each object is

2, 3.

Visualise the positions of 2-dimensional objects which take a particular shape composed of rectangles.

Parameters:

- `rect_size`: the size of each rectangle
- `rect_offset`: the position of each rectangle
- `shape`: the rectangles which comprise each shape
- `x`: the position of each object
- `kind`: which shape each object is

```
1.  annotation vis_graph(array [$$E] of $$N: from,
                         array [$$E] of $$N: to,
                         array [$$E] of string: edge_labels,
                         array [$$N] of var bool: ns,
                         array [$$E] of var bool: es)

2.  annotation vis_graph(array [$$E] of $$N: from,
                         array [$$E] of $$N: to,
                         array [$$N] of var bool: ns,
                         array [$$E] of var bool: es)

3.  annotation vis_graph(array [$$N] of string: node_labels,
                         array [$$E] of $$N: from,
                         array [$$E] of $$N: to,
                         array [$$E] of string: edge_labels,
                         array [$$N] of var bool: ns,
                         array [$$E] of var bool: es)
```

1. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph. The nodes are labelled using the index set of `ns`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph. The nodes are labelled using the index set of `ns` and the edges are labelled using the index set of `es`.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

3. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph.

   Parameters:

   - `node_labels`: the label for each node
   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph

- `es`: a Boolean for each edge whether it is in the subgraph

```
1.  annotation vis_graph_highlight(array [$$E] of $$N: from,
                                   array [$$E] of $$N: to,
                                   array [$$E] of string: edge_labels,
                                   array [$$N] of var bool: ns,
                                   array [$$E] of var bool: es)

2.  annotation vis_graph_highlight(array [$$E] of $$N: from,
                                   array [$$E] of $$N: to,
                                   array [$$N] of var bool: ns,
                                   array [$$E] of var bool: es)

3.  annotation vis_graph_highlight(array [$$N] of string: node_labels,
                                   array [$$E] of $$N: from,
                                   array [$$E] of $$N: to,
                                   array [$$E] of string: edge_labels,
                                   array [$$N] of var bool: ns,
                                   array [$$E] of var bool: es)
```

1. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph. The nodes are labelled using the index set of `ns`. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `edge_labels`: labels to use for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

2. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph. The nodes are labelled using the index set of `ns` and the edges are labelled using the index set of `es`. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `from`: the leaving node for each edge
   - `to`: the entering node for each edge
   - `ns`: a Boolean for each node whether it is in the subgraph
   - `es`: a Boolean for each edge whether it is in the subgraph

3. Create a visualisation of the subgraph `ns` and `es` of a given undirected graph. The entire graph is drawn, with the given subgraph highlighted.

   Parameters:

   - `node_labels`: the label for each node

- `from`: the leaving node for each edge
- `to`: the entering node for each edge
- `edge_labels`: labels to use for each edge
- `ns`: a Boolean for each node whether it is in the subgraph
- `es`: a Boolean for each edge whether it is in the subgraph

```
1.  annotation vis_line(var int: x, string: label)

2.  annotation vis_line(var float: x, string: label)

3.  annotation vis_line(array [$$E] of var int: x)

4.  annotation vis_line(array [$$E] of var float: x)

5.  annotation vis_line(array [$$E] of var int: x,
                        array [$$E] of string: series_labels)

6.  annotation vis_line(array [$$E] of var float: x,
                        array [$$E] of string: series_labels)

7.  annotation vis_line(array [$$E] of var int: x,
                        array [$$E] of var int: y,
                        string: x_label,
                        string: y_label)

8.  annotation vis_line(array [$$E] of var float: x,
                        array [$$E] of var float: y,
                        string: x_label,
                        string: y_label)
```

1, 2.
Add a line graph of `x` with respect to solution time, labelling the trace `label`.

3, 4.
Add line graphs of the values in `x` with respect to solution time on the same axes, labelling the series using their index.

5, 6.
Add line graphs of the values in `x` with respect to solution time on the same axes, labelling the series with `series_labels`.

7, 8.
Add a line graph of `x` against `y`, labelling the axes `x_label` and `y_label` respectively.

```
annotation vis_scatter(array [$$E] of var int: x,
                       array [$$E] of var int: y,
                       string: x_label,
```

```
                        string: y_label)
annotation vis_scatter(array [$$E] of var float: x,
                        array [$$E] of var float: y,
                        string: x_label,
                        string: y_label)
```

Add a scatter plot of `x` against `y`, labelling the axes `x_label` and `y_label` respectively.

```
1.  annotation vis_scatter_cumulative(var int: x,
                                      var int: y,
                                      string: x_label,
                                      string: y_label,
                                      string: series_label)

2.  annotation vis_scatter_cumulative(var float: x,
                                      var float: y,
                                      string: x_label,
                                      string: y_label,
                                      string: series_label)

3.  annotation vis_scatter_cumulative(array [$$E] of var int: x,
                                      array [$$E] of var int: y,
                                      string: x_label,
                                      string: y_label)

4.  annotation vis_scatter_cumulative(array [$$E] of var float: x,
                                      array [$$E] of var float: y,
                                      string: x_label,
                                      string: y_label)

5.  annotation vis_scatter_cumulative(array [$$E] of var int: x,
                                      array [$$E] of var int: y,
                                      string: x_label,
                                      string: y_label,
                                      array [$$E] of string: series_labels)

6.  annotation vis_scatter_cumulative(array [$$E] of var float: x,
                                      array [$$E] of var float: y,
                                      string: x_label,
                                      string: y_label,
                                      array [$$E] of string: series_labels)
```

1, 2.

Create a scatter-plot visualisation. The point (`x`, `y`) is added to the plot for each solution.

Parameters:

- `x`: the x coordinate

- `y`: the y coordinate
- `x_label`: the x-axis label
- `y_label`: the y-axis label
- `series_label`: the series label

3, 4.

Create a scatter-plot visualisation with multiple series. The point (`x`[i], `y`[i]) for each series `i` is added to the plot for each solution. Series are labelled using the index set of `x` and `y`.

Parameters:

- `x`: the x coordinate for each series
- `y`: the y coordinate for each series
- `x_label`: the x-axis label
- `y_label`: the y-axis label

5, 6.

Create a scatter-plot visualisation with multiple series. The point (`x`[i], `y`[i]) for each series `i` is added to the plot for each solution.

Parameters:

- `x`: the x coordinate for each series
- `y`: the y coordinate for each series
- `x_label`: the x-axis label
- `y_label`: the y-axis label
- `series_labels`: the label for each series

```
1.  annotation vis_server(string: file)

2.  annotation vis_server(string: file, opt $T: user_data)

3.  annotation vis_server(string: file, array [$X] of opt $T: user_data)
```

1. Launch a visualisation server for a custom visualisation using the given `file`.

2, 3.

Launch a visualisation server for a custom visualisation using the given `file`. The `user_data` value can be retrieved in the page using `MiniZincIDE.getUserData()`.

4.2.7 MiniZinc definitions for the MiniZinc diversity extension.

These annotations and predicates can be used to produce diverse set of solutions.In order to use them in a model, include the file "diversity.mzn".

4.2.7.1 Annotations provided to guide the MiniZinc diversity extension.

Functions and Predicates

```
1.  annotation diverse_pairwise(array [int] of var int: xs,
                                string: distance_metric)

2.  annotation diverse_pairwise(array [$T] of var int: xs,
                                string: distance_metric)

3.  annotation diverse_pairwise(array [int] of var int: xs,
                                string: distance_metric,
                                int: min_dist,
                                int: max_dist,
                                int: weight)

4.  annotation diverse_pairwise(array [$T] of var int: xs,
                                string: distance_metric,
                                int: min_dist,
                                int: max_dist,
                                int: weight)

5.  annotation diverse_pairwise(array [int] of var float: xs,
                                string: distance_metric)

6.  annotation diverse_pairwise(array [$T] of var float: xs,
                                string: distance_metric)

7.  annotation diverse_pairwise(array [int] of var float: xs,
                                string: distance_metric,
                                float: min_dist,
                                float: max_dist,
                                float: weight)

8.  annotation diverse_pairwise(array [$T] of var float: xs,
                                string: distance_metric,
                                float: min_dist,
                                float: max_dist,
                                float: weight)
```

1, 2.

Use `distance_metric` on `xs` when computing distance to solutions. Here same index on different solutions are compared. For example, when obtaining the distance to k-th diverse solution using the incremental approach (diversity_incremental) each index of `xs` in k-th solution is compared to the corresponding index of all previously

obtained diverse solutions.

3, 4.
Use `distance_metric` on `xs` to compute the distance to solutions, and apply constraints using `min_dist` to enforce a minimum distance and `max_dist` to enforce a maximum distance. The `weight` parameter can be used to set relative priorities between multiple distance measures. Distance measures with higher weights are given greater priority. For example, when obtaining the distance to k-th diverse solution using the incremental approach (diversity_incremental) each index of `xs` in k-th solution is compared to the corresponding index of all previously obtained diverse solutions.

5, 6.
Use `distance_metric` on `xs` when computing distance to solutions. Here same index on different solutions are compared. For example, when obtaining the distance to k-th diverse solution using the incremental approach (diversity_incremental) each index of `xs` in k-th solution is compared to the corresponding index of all previously obtained diverse solutions.

7, 8.
Use `distance_metric` on `xs` to compute the distance to solutions, and apply constraints using `min_dist` to enforce a minimum distance and `max_dist` to enforce a maximum distance. The `weight` parameter can be used to set relative priorities between multiple distance measures. Distance measures with higher weights are given greater priority. For example, when obtaining the distance to k-th diverse solution using the incremental approach (diversity_incremental) each index of `xs` in k-th solution is compared to the corresponding index of all previously obtained diverse solutions.

Annotations

```
annotation diversity_aggregator(string)
```

An aggregator combines pairwise diversity measures from each pair of solutions into an overall diversity score to identify the k most diverse solutions, using an incremental approach (i.e., diversity_incremental). Currently, it supports three methods: sum, which sums up the pairwise diversity measures; min, which selects the minimum; and max, which selects the maximum.

```
annotation diversity_combinator(string)
```

Use to combine distance functions to diversity of a diverse solution. Currently, it supports three methods: sum, which sums up the distance measures; min, which selects the minimum; and max, which selects the maximum.

```
annotation diversity_incremental(int: k, float: gap)
```

Produce at most k number of diverse solutions with thier objective values bounded between optimal value and gap % from the optimal. Diverse solutions are obtained one after the other and distance to k-th diverse solution is sum of pairwise distance to all the previous diverse solutions

---

4.2.7.2 Distance functions provided in the MiniZinc diversity extension.

```
function int: hamming_distance(array [$A] of opt $T: xs,
                               array [$A] of opt $T: ys)
function var int: hamming_distance(array [$A] of var opt $T: xs,
                                   array [$A] of var opt $T: ys)
```

Returns the Hamming distance between xs and ys.

```
function int: manhattan_distance(array [$A] of $$T: xs,
                                 array [$A] of $$T: ys)
function var int: manhattan_distance(array [$A] of var $$T: xs,
                                     array [$A] of var $$T: ys)
function float: manhattan_distance(array [$A] of float: xs,
                                   array [$A] of float: ys)
function var float: manhattan_distance(array [$A] of var float: xs,
                                       array [$A] of var float: ys)
```

Returns the Manhattan distance between xs and ys.

4.2.8 Experimental Features

These features are experimental and might significantly change or be removed in future versions of MiniZinc.

4.2.8.1 Assumptions

An *assumption* interface for solvers that support solving under assumptions. The Boolean variables passed to **assume** are assumed to be true; a solver that supports assumptions may, when the model is unsatisfiable under these assumptions, report a subset of them as an unsatisfiable core. MiniZinc maps such a core back to the original expressions and prints it for inspection (look for the `Unsatisfiable core` comment in the solver output).

```
predicate assume(array [int] of var bool: b)
```

Assume that the Boolean variables in b hold.

A solver supporting assumptions treats these as assumptions rather than hard constraints, and may report a subset of them as an unsatisfiable core when the model is unsatisfiable under the assumptions. Fixed-true elements are dropped; a fixed-false element makes the model trivially unsatisfiable (and is reported to the user). If the selected solver has no native support, the assumptions are posted as hard constraints and a warning is issued.

4.2.8.2 On Restart

These annotations and functions provide the basic building blocks to implement metaheuristics triggered on restart and governed search with the use of solver internal values and functionality.

Constants

```
set of int: STATUS
```

Reports the status of the solver (before restarting).

Functions and Predicates

```
predicate basic_lns(var bool: nbh)
```

'basic_lns' provides LNS in its simplest form where one nbh is applied on every restart.

```
test complete()
```

When complete is set to 'true', then it marks the search as complete.

```
predicate last_val(var bool: x)
function var float: last_val(var float: x)
function var $$E: last_val(var $$E: x)
function var opt $$E: last_val(var opt $$E: x)
function var set of $$E: last_val(var set of $$E: x)
```

Provides access to last values taken by a model variables. The last_val functions are only safe to use when the STATUS has been SAT or OPT.

```
predicate round_robin(array [int] of var bool: nbhs)
```

round_robin provides a metaheuristic for LNS where each neighbourhood is chosen sequentially.

```
predicate sol(var bool: x)
function var float: sol(var float: x)
function var $$E: sol(var $$E: x)
function var opt $$E: sol(var opt $$E: x)
function var set of $$E: sol(var set of $$E: x)
function array [$$E] of var bool: sol(array [$$E] of var bool: x)
function array [$$E] of var float: sol(array [$$E] of var float: x)
function array [$$E] of var $$F: sol(array [$$E] of var $$F: x)
```

Provides access to solution values of model variables. The sol function are only safe to use when the STATUS has been SAT or OPT.

```
1.  function var $$E: uniform_on_restart($$E: low, $$E: high)

2.  function var $$E: uniform_on_restart(set of $$E: S)

3.  function var float: uniform_on_restart(float: low, float: high)
```

1. Returns a new random value between `low` and `high` chosen from a uniform distribution whenever the solver restarts.
2. Returns a new random value from the set `S` chosen from a uniform distribution whenever the solver restarts.
3. Returns a new random value between `low` and `high` chosen from a uniform distribution whenever the solver restarts.

Annotations

```
annotation on_restart(string: pred)
```

A search annotation that triggers a function to be executed on restart

---

CHAPTER 4.3

Interfacing Solvers to Flatzinc

This document describes the interface between the MiniZinc system and FlatZinc solvers. Interfacing a solver with MiniZinc usually requires three components:

1. The solver binary (or a script that runs the solver). It should support the command line options specified in Section 4.3.5, understand either the FlatZinc input language as defined in Section 4.3.1, or the FlatZinc-JSON input language as defined in Section 4.3.2. Solvers must produce output as defined in Section 4.3.3.
2. The solver's MiniZinc library. This library defines the global constraints that the solver supports, and redefines FlatZinc built-in constraints that the solver does not support (see Section 4.3.4).
3. The solver's configuration file. This file makes the solver known to the `minizinc` driver program and the MiniZinc IDE. Configuration files are discussed in Section 4.3.6.

We recommend to install solvers following something similar to the Linux Filesystem Hierarchy Standard: When installing a solver called `$SOLVERNAME` in a directory `$PREFIX`, its binary should be put in `$PREFIX/bin`, its MiniZinc library files should be in `$PREFIX/share/minizinc/$SOLVERNAME/`, and its solver configuration file should be `$PREFIX/share/minizinc/solvers/$SOLVERNAME.mzc`. The prefix can either be a global installation path (such as `/usr/local`) or any other path.

4.3.1 Specification of FlatZinc

FlatZinc is the target constraint modelling language into which MiniZinc models are translated. It is a very simple solver independent problem specification language, requiring minimal implementation effort to support.

Throughout this document: r_1, r_2 denote float literals; $x_1, x_2, \ldots, x_k, i, j, k$ denote int literals; $y_1, y_2, \ldots, y_k, y_i$ denote literal array elements.

4.3.1.1 Comments

Comments start with a percent sign `%` and extend to the end of the line. Comments can appear anywhere in a model.

4.3.1.2 Types

There are three varieties of types in FlatZinc.

- *Parameter* types apply to fixed values that are specified directly in the model.
- *Variable* types apply to values computed by the solver during search. Every parameter type has a corresponding variable type; the variable type being distinguished by a `var` keyword.
- *Annotations* and *strings*: annotations can appear on variable declarations, constraints, and on the solve goal. They provide information about how a variable or constraint should be treated by the solver (e.g., whether a variable should be output as part of the result or whether a particular constraint should implemented using domain consistency). Strings may appear as arguments to annotations, but nowhere else.

Parameter types

Parameters are fixed quantities explicitly specified in the model (see rule `<par-type>` in Section 4.3.7).

Type	Values
`bool`	`true` or `false`
`float`	float
`int`	int
`set of int`	subset of int
`array [1..` n `] of bool`	array of bools
`array [1..` n `] of float`	array of floats
`array [1..` n `] of int`	array of ints
`array [1..` n `] of set of int`	array of sets of ints

A parameter may be used where a variable is expected, but not vice versa.

In predicate declarations the following additional parameter types are allowed.

Type	Values
r_a `..` r_b	bounded float
x_a `..` x_b	int in range
`{` $x_a, x_b, \ldots, x_k$ `}`	int in set
`set of` x_a `..` x_b	subset of int range
`set of {` $x_a, x_b, \ldots, x_k$ `}`	subset of int set
`array [1..` n `] of` r_a `..` r_b	array of floats in range
`array [1..` n `] of` x_a `..` x_b	array of ints in range
`array [1..` n `] of set of` x_a `..` x_b	array of sets of ints in range
`array [1..` n `] of set of {` $x_a, x_b, \ldots, x_k$ `}`	array of subsets of set

A range x_a `..` x_b denotes a closed interval $\{x | x_a \leq x \leq x_b\}$ (same for float ranges).

An array type appearing in a predicate declaration may use just `int` instead of `1..` n for the array index range in cases where the array argument can be of any length.

Variable types

Variables are quantities decided by the solver (see rules `<basic-var-type>` and `<array-var-type>` in Section 4.3.7).

Variable type
`var bool`
`var float`
`var` r_a `..` r_b
`var int`
`var` x_a `..` x_b
`var {` $x_a, x_b, \ldots, x_k$ `}`
`var set of` x_a `..` x_b
`var set of {` $x_a, x_b, \ldots, x_k$ `}`
`array [1..` n `] of var bool`
`array [1..` n `] of var float`
`array [1..` n `] of var` r_a `..` r_b
`array [1..` n `] of var int`
`array [1..` n `] of var` x_a `..` x_b
`array [1..` n `] of var set of` x_a `..` x_b
`array [1..` n `] of var set of {` $x_a, x_b, \ldots, x_k$ `}`

In predicate declarations the following additional variable types are allowed.

Variable type
`var set of int`
`array [1..` n `] of var set of int`

An array type appearing in a predicate declaration may use just `int` instead of `1..` n for the array index range in cases where the array argument can be of any length.

The string type

String literals and literal arrays of string literals can appear as annotation arguments, but not elsewhere. Strings have the same syntax as in C programs (namely, they are delimited by double quotes and the backslash character is used for escape sequences).

Examples:

```
""                              % The empty string.
"Hello."
"Hello,\nWorld\t\"quoted!\""    % A string with an embedded newline, tab and␣
↪quotes.
```

4.3.1.3 Values and expressions

(See rule <expr> in Section 4.3.7)

Examples of literal values:

Type Literals `bool` `true`, `false` `float` `2.718`, `-1.0`, `3.0e8` `int` `-42`, `0`, `69` `set of int` `{}`, `{2, 3, 5}`, `1..10` `arrays` `[]`, [$y_a, \ldots, y_k$]

where each array element y_i is either: a non-array literal, or the name of a non-array parameter or variable, `v`. For example:

```
[1, 2, 3]             % Just literals
[x, y, z]             % x, y, and z are variables or parameters.
[x, 3]                % Mix of identifiers and literals
```

Section 4.3.7 gives the regular expressions specifying the syntax for float and int literals.

4.3.1.4 FlatZinc models

A FlatZinc model consists of:

1. zero or more external predicate declarations (i.e., a non-standard predicate that is supported directly by the target solver);
2. zero or more parameter declarations;
3. zero or more variable declarations;
4. zero or more constraints;
5. a solve goal

in that order.

FlatZinc uses the UTF-8 character set. Non-ASCII characters can only appear in string literals.

FlatZinc syntax is case sensitive (`foo` and `Foo` are different names). Identifiers start with a letter (`[A-Za-z]`) and are followed by any sequence of letters, digits, or underscores (`[A-Za-z0-9_]`). Additionally, identifiers of variable or parameter names may start with an underscore. Identifiers that correspond to the names of predicates, predicate parameters and annotations cannot have leading underscores.

The following keywords are reserved and cannot be used as identifiers: `annotation`, `any`, `array`, `bool`, `case`, `constraint`, `diff`, `div`, `else`, `elseif`, `endif`, `enum`, `false`, `float`, `function`, `if`, `in`, `include`, `int`, `intersect`, `let`, `list`, `maximize`, `minimize`, `mod`, `not`, `of`, `satisfy`, `subset`, `superset`, `output`, `par`, `predicate`, `record`, `set`, `solve`, `string`, `symdiff`, `test`, `then`, `true`, `tuple`, `union`, `type`, `var`, `where`, `xor`. Note that some of these keywords are not used in FlatZinc. They are reserved because they are keywords in Zinc and MiniZinc.

FlatZinc syntax is insensitive to whitespace.

4.3.1.5 Predicate declarations

(See rule <predicate-item> in Section 4.3.7)

Predicates used in the model that are not standard FlatZinc must be declared at the top of a FlatZinc model, before any other lexical items. Predicate declarations take the form

```
<predicate-item> ::= "predicate" <identifier> "(" [ <pred-param-type> ":"
↪<identifier> "," ... ] ")" ";"
```

Annotations are not permitted anywhere in predicate declarations.

It is illegal to supply more than one predicate declaration for a given <identifier>.

Examples:

```
    % m is the median value of {x, y, z}.
    %
predicate median_of_3(var int: x, var int: y, var int: z, var int: m);

    % all_different([x1, .., xn]) iff
    % for all i, j in 1..n: xi != xj.
    %
predicate all_different(array [int] of var int: xs);

    % exactly_one([x1, .., xn]) iff
    % there exists an i in 1..n: xi = true
    % and for all j in 1..n: j != i -> xj = false.
    %
predicate exactly_one(array [int] of var bool: xs);
```

4.3.1.6 Parameter declarations

(See rule `param_decl` in Section 4.3.7)

Parameters have fixed values and must be assigned values:

```
<par-decl-item> ::= "array" "[" <index-set> "]" "of" <basic-par-type> ":"
↪<var-par-identifier> "=" <par-array-literal> ";"
```

where <par-type> is a parameter type, <var-par-identifier> is an identifier, and <par-expr> is a literal value (either a basic integer, float or bool literal, or a set or array of such literals).

Annotations are not permitted anywhere in parameter declarations.

Examples:

```
float: pi = 3.141;
array [1..7] of int: fib = [1, 1, 2, 3, 5, 8, 13];
bool: beer_is_good = true;
```

4.3.1.7 Variable declarations

(See rule `var_decl` in Section 4.3.7)

Variables have variable types and can be declared with optional assignments. The assignment can fix a variable to a literal value, or create an alias to another variable. Arrays of variables always have an assignment, defining them in terms of an array literal that can contain identifiers of variables or constant literals. Variables may be declared with zero or more annotations.

```
<var-decl-item> ::= <basic-var-type> ":" <var-par-identifier> <annotations>␣
↪[ "=" <basic-expr> ] ";"
                  | <array-var-type> ":" <var-par-identifier> <annotations>
↪"=" <array-literal> ";"
```

where `<basic-var-type>` and `<array-var-type>` are variable types, `<var-par-identifier>` is an identifier, `<annotations>` is a (possibly empty) set of annotations, `<basic-expr>` is an identifier or a literal, and `<array-literal>` is a literal array value.

Examples:

```
var 0..9: digit;
var bool: b;
var set of 1..3: s;
var 0.0..1.0: x;
var int: y :: mip;        % 'mip' annotation: y should be a MIP variable.
array [1..3] of var 1..10: b = [y, 3, digit];
```

4.3.1.8 Constraints

(See rule `<constraint-item>` in Section 4.3.7)

Constraints take the following form and may include zero or more annotations:

```
<constraint-item> ::= "constraint" <identifier> "(" [ <expr> "," ... ] ")"
↪<annotations> ";"
```

The arguments expressions (`<expr>`) can be literal values or identifiers.

Examples:

```
constraint int_le(0, x);    % 0 <= x
constraint int_lt(x, y);    % x <  y
constraint int_le(y, 10);   % y <= 10
    % 'domain_propagation': use domain consistency for this constraint:
    % 2x + 3y = 10
constraint int_lin_eq([2, 3], [x, y], 10) :: domain_propagation;
```

4.3.1.9 Solve item

(See rule <solve-item> in Section 4.3.7)

A model finishes with a solve item, taking one of the following forms:

```
<solve-item> ::= "solve" <annotations> "satisfy" ";"
               | "solve" <annotations> "minimize" <basic-expr> ";"
               | "solve" <annotations> "maximize" <basic-expr> ";"
```

The first alternative searches for any satisfying assignment, the second one searches for an assignment minimizing the given expression, and the third one for an assignment maximizing the expression. The <basic-expr> can be either a variable identifier or a literal value (if the objective function is constant).

A solution consists of a complete assignment where all variables in the model have been given a fixed value.

Examples:

```
solve satisfy;      % Find any solution using the default strategy.

solve minimize w;   % Find a solution minimizing w, using the default␣
↪strategy.

    % First label the variables in xs in the order x[1], x[2], ...
    % trying values in ascending order.
solve :: int_search(xs, input_order, indomain_min, complete)
    satisfy;    % Find any solution.

    % First use first-fail on these variables, splitting domains
    % at each choice point.
solve :: int_search([x, y, z], first_fail, indomain_split, complete)
    maximize x; % Find a solution maximizing x.
```

4.3.1.10 Annotations

Annotations are optional suggestions to the solver concerning how individual variables and constraints should be handled (e.g., a particular solver may have multiple representations for int variables) and how search should proceed. An implementation is free to ignore any annotations it does not recognise, although it should print a warning on the standard error stream if it does so. Annotations are unordered and idempotent: annotations can be reordered and duplicates can be removed without changing the meaning of the annotations.

An annotation is prefixed by `::`, and either just an identifier or an expression that looks like a predicate call:

```
<annotations> ::= [ "::" <annotation> ]*

<annotation> ::= <identifier>
```

```
               | <identifier> "(" <ann-expr> "," ... ")"

<ann-expr>   := <basic-ann-expr>
              | "[" [ <basic-ann-expr> "," ... ] "]"

<basic-ann-expr>   := <basic-literal-expr>
                    | <var-par-identifier>
                    | <string-literal>
                    | <annotation>

<string-contents> ::= ([^"\n\] | \[^\n(])*

<string-literal> ::= """ <string-contents> """
```

The arguments of the second alternative can be any expression or other annotations (without the leading `::`).

Search annotations

While an implementation is free to ignore any or all annotations in a model, it is recommended that implementations at least recognise the following standard annotations for solve items.

```
seq_search([<searchannotation>, ...])
```

allows more than one search annotation to be specified in a particular order (otherwise annotations can be handled in any order).

A `<searchannotation>` is one of the following:

```
int_search(<vars>, <varchoiceannotation>, <assignmentannotation>,
↪<strategyannotation>)

bool_search(<vars>, <varchoiceannotation>, <assignmentannotation>,
↪<strategyannotation>)

set_search(<vars>, <varchoiceannotation>, <assignmentannotation>,
↪<strategyannotation>)
```

where `<vars>` is the identifier of an array variable or an array literal specifying the variables to be assigned (ints, bools, or sets respectively). Note that these arrays may contain literal values.

`<varchoiceannotation>` specifies how the next variable to be assigned is chosen at each choice point. Possible choices are as follows (it is recommended that implementations support the starred options):

input_order	⋆	Choose variables in the order they appear in vars.
first_fail	⋆	Choose the variable with the smallest domain.
anti_first_fail		Choose the variable with the largest domain.
smallest		Choose the variable with the smallest value in its domain.
largest		Choose the variable with the largest value in its domain.
occurrence		Choose the variable with the largest number of attached constraints.
most_constrained		Choose the variable with the smallest domain, breaking ties using the number of constraints.
max_regret		Choose the variable with the largest difference between the two smallest values in its domain.
dom_w_deg		Choose the variable with the smallest value of domain size divided by weighted degree, where the weighted degree is the number of times the variables been in a constraint which failed

<assignmentannotation> specifies how the chosen variable should be constrained. Possible choices are as follows (it is recommended that implementations support at least the starred options):

indomain_min	⋆	Assign the smallest value in the variable's domain.
indomain_max	⋆	Assign the largest value in the variable's domain.
indomain_middle		Assign the value in the variable's domain closest to the mean of its current bounds.
indomain_median		Assign the middle value in the variable's domain.
indomain		Nondeterministically assign values to the variable in ascending order.
indomain_random		Assign a random value from the variable's domain.
indomain_split		Bisect the variable's domain, excluding the upper half first.
indomain_reverse_split		Bisect the variable's domain, excluding the lower half first.
indomain_interval		If the variable's domain consists of several contiguous intervals, reduce the domain to the first interval. Otherwise just split the variable's domain.

Of course, not all assignment strategies make sense for all search annotations (e.g., bool_search and indomain_split).

Finally, <strategyannotation> specifies a search strategy; implementations should at least support complete (i.e., exhaustive search).

Output annotations

Model output is specified through variable annotations. Non-array output variables are annotated with `output_var`. Array output variables are annotated with `output_array([` $1 .. x$ `])` where $1 .. x$ is the index set range of the array. See Section 4.3.3 for details on the output format.

Variable definition annotations

To support solvers capable of exploiting functional relationships, a variable defined as a function of other variables may be annotated thus:

```
var int: x :: is_defined_var;

...

constraint int_plus(y, z, x) :: defines_var(x);
```

(The `defines_var` annotation should appear on exactly one constraint.) This allows a solver to represent `x` internally as a representation of `y+z` rather than as a separate constrained variable. The `is_defined_var` annotation on the declaration of `x` provides "early warning" to the solver that such an option is available.

Intermediate variables

Intermediate variables introduced during conversion of a MiniZinc model to FlatZinc may be annotated thus:

```
var int: X_INTRODUCED_3 :: var_is_introduced;
```

This information is potentially useful to the solver's search strategy.

Constraint annotations

Annotations can be placed on constraints advising the solver how the constraint should be implemented. Here are some constraint annotations supported by some solvers:

boun	**Use bounds propagation. The concrete propagation strength is not further specified.**
boun	Use integer bounds propagation.
boun	Use real bounds propagation.
boun	A tighter version of `boundsZ` where support for the bounds must exist.
doma	Use domain propagation.
valu	Use value propagation.
prop	
prio	where `k` is an integer constant indicating propagator priority.

4.3.2 Specification of FlatZinc-JSON

FlatZinc-JSON is an alternative syntax for FlatZinc that is compatible with the JSON data format. Solvers can indicate that they require FlatZinc output in their solver configuration files (see Section 4.3.6) using the *inputType* property.

The JSON representation of FlatZinc contains the same elements as a FlatZinc file: declarations of variables, arrays, constraints and a solve item. A full specification is given as a JSON schema at the end of this section. Here is an example of the cake optimisation model from Listing 1.2, compiled into FlatZinc-JSON:

```
{
  "variables": {
    "b" : { "type" : "int", "domain" : [[0, 3]] },
    "c" : { "type" : "int", "domain" : [[0, 6]] },
    "X_INTRODUCED_0_" : { "type" : "int", "domain" : [[0, 85000]], "defined"␣
↪: true }
  },
  "arrays": {
    "X_INTRODUCED_2_" : { "a": [250, 200] },
    "X_INTRODUCED_6_" : { "a": [75, 150] },
    "X_INTRODUCED_8_" : { "a": [100, 150] }
  },
  "constraints": [
    { "id" : "int_lin_le", "args" : ["X_INTRODUCED_2_", ["b", "c"], 4000]},
    { "id" : "int_lin_le", "args" : ["X_INTRODUCED_6_", ["b", "c"], 2000]},
    { "id" : "int_lin_le", "args" : ["X_INTRODUCED_8_", ["b", "c"], 500]},
    { "id" : "int_lin_eq", "args" : [[400, 450, -1], ["b", "c",
↪"X_INTRODUCED_0_"], 0],
      "ann" : ["ctx_pos"], "defines" : "X_INTRODUCED_0_"}
  ],
  "output": ["b", "c"],
  "solve": { "method" : "maximize", "objective" : "X_INTRODUCED_0_" },
  "version": "1.0"
}
```

There are a number of small differences compared to FlatZinc:

- Variables and arrays are declared separately. Each variable must have an identifier and a type, and can optionally have a domain, a flag indicating that the variable was introduced by compilation, a flag that the variable is functionally defined by a constraint, and a property `"rhs"` to define the "right hand side" of the variable, i.e., its value or alias to another variable.
- Domains are written as lists of contiguous, ordered, non-overlapping ranges. E.g., the set `{1,2,4,6,7}` would be written as `[[1,2],[4,4],[6,7]]`.
- Arrays are declared with an identifier and an object that has the property `"a"`, which maps to the actual array. Arrays can contain numeric literals or identifiers (written as strings).
- Constraints are declared as objects with a property `"id"` indicating the name of the constraint, a property `"args"` for the array of arguments to the constraint, and

an optional property `"defines"` that indicates the variable identifier that this constraint functionally defines.

- The `"output"` property is a list of all identifiers for which the solver must produce output for each solution. The output must follow the normal FlatZinc output rules as defined in Section 4.3.3.
- The `"solve"` property maps to an object with a property `"method"` that indicates whether the problem is a satisfaction problem (`"satisfy"`) or an optimisation problem (`"minimize"` or `"maximize"`), in which case the `"objective"` property must be present, which maps to an identifier.
- Variables, arrays, constraints and the solve item can have an optional list of annotations, indicated by the `"ann"` property. Atomic annotations (such as `indomain_min`) are represented simply as strings. Non-atomic annotations (such as `int_search`) follow the same syntax as constraints, i.e., an object with `"id"` and `"args"` properties.
- String literals can occur in FlatZinc as arguments to annotations or special solver-defined predicates. In FlatZinc-JSON, these string literals are represented as objects `{ "string" : "<string contents>" }`.

The full schema is listed below, and is available from https://www.minizinc.org/schemas/fznjson.

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "$id": "https://www.minizinc.org/schemas/fznjson",
  "title" : "FlatZincJSON",
  "description" : "A JSON representation of a FlatZinc model",

  "$defs" : {
    "identifier" : { "type" : "string", "pattern" : "[A-Za-z][A-Za-z0-9_]*" }
↪,
    "literal" : { "oneOf" : [
      { "type" : "number" },
      { "$ref" : "#/$defs/identifier" },
      { "type" : "boolean" },
      { "type" : "object",
        "properties" : {
          "set" : {
            "type" : "array",
            "items" : {
                        "type" : "array",
                        "items" : [
                          { "type" : "number" },
                          { "type" : "number" }
                        ]
                      }
          }
        },
        "required" : [ "set" ]
      },
      { "type" : "object",
```

```
        "properties" : {
          "string" : { "type" : "string" }
        },
        "required" : [ "string" ]
      }
    ]},
    "literals" : { "type" : "array", "items" : { "$ref" : "#/$defs/literal" }
↪ },
    "argument" : { "oneOf" : [
      { "$ref" : "#/$defs/literals"},
      { "$ref" : "#/$defs/literal" }
    ]},
    "annotation" : { "oneOf" : [
      { "$ref" : "#/$defs/annotationCall" },
      { "type" : "string" }
    ]},
    "annotations" : {
      "type" : "array",
      "items" : { "$ref" : "#/$defs/annotation" }
    },
    "annotationArgument" : { "oneOf" : [
      { "$ref" : "#/$defs/annotationLiterals"},
      { "$ref" : "#/$defs/annotationLiteral" }
    ]},
    "annotationCall" : {
      "type" : "object",
      "properties" : {
        "id": { "$ref": "#/$defs/identifier" },
        "args": {
          "type": "array",
          "items": { "$ref" : "#/$defs/annotationArgument" }
        }
      },
      "required": [
        "id",
        "args"
      ]
    },
    "annotationLiterals" : { "type" : "array", "items" : { "$ref" : "#/
↪$defs/annotationLiteral" } },
    "annotationLiteral" : { "oneOf" : [
      { "$ref" : "#/$defs/literal"},
      { "$ref" : "#/$defs/annotationCall" }
    ]},
    "domain" : {
      "type" : "array",
      "items" : {
        "type" : "array",
        "items" : [
          { "type" : "number" },
```

```
          { "type" : "number" }
        ]
      }
    }
  },

  "type": "object",
  "properties": {
    "version" : { "type" : "string" },
    "variables": {
      "type": "object",
      "patternProperties": {
        "[A-Za-z][A-Za-z0-9_]*": {
          "type": "object",
          "properties": {
            "type": {
              "enum": [
                "bool",
                "float",
                "int",
                "set of int"
               ]
            },
            "domain": { "$ref" : "#/$defs/domain" },
            "introduced": {
              "type": "boolean"
            },
            "defined": {
              "type": "boolean"
            },
            "ann" : { "$ref" : "#/$defs/annotations"}
          },
          "required": [
            "type"
          ]
        }
      }
    },
    "arrays": {
      "type": "object",
      "patternProperties": {
        "[A-Za-z][A-Za-z0-9_]*": {
          "type": "object",
          "properties": {
            "a": {
              "type": "array",
              "items": { "$ref" : "#/$defs/literal" }
            },
            "ann" : { "$ref" : "#/$defs/annotations"},
            "introduced": {
```

```
            "type": "boolean"
          },
          "defined": {
            "type": "boolean"
          }
        },
        "required": [
          "a"
        ]
      }
    }
  },
  "constraints": {
    "type": "array",
    "items": {
      "type" : "object",
      "properties" : {
        "id": { "$ref": "#/$defs/identifier" },
        "args": {
          "type": "array",
          "items": { "$ref" : "#/$defs/argument" }
        },
        "ann" : { "$ref" : "#/$defs/annotations"},
        "defines": { "$ref": "#/$defs/identifier" }
      },
      "required": [
        "id",
        "args"
      ]
    }
  },
  "output": {
    "type": "array",
    "items": { "$ref" : "#/$defs/identifier" }
  },
  "solve": {
    "type": "object",
    "properties": {
      "method": {
        "enum": [ "satisfy", "minimize", "maximize" ]
      },
      "objective": { "$ref" : "#/$defs/literal" },
      "ann" : { "$ref" : "#/$defs/annotations"}
  },
    "required": [
      "method"
    ]
  }
},
"required": [
```

```
    "version",
    "variables",
    "arrays",
    "output",
    "constraints",
    "solve"
  ]
}
```

4.3.3 Output

An implementation can produce three types of output: solutions, statistics, and errors.

4.3.3.1 Solution output

An implementation must output values for all and only the variables annotated with `output_var` or `output_array` (output annotations must not appear on parameters) in a FlatZinc model, or those variables and arrays whose identifiers are listed in the `"output"` property of a FlatZinc-JSON model. Output must be printed to the standard output stream.

For example:

```
var 1..10: x :: output_var;
var 1..10: y;       % y is not output.
array [1..4] of var int: zs :: output_array([1..4]);
```

All non-error output must be sent to the standard output stream.

Output must take the following form:

```
<var-par-identifier> = <basic-literal-expr> ;
```

or, for array variables,

```
<var-par-identifier> = [<y1>, <y2>, ... <yk>];
```

where `<y1>`, `<y2>`, `...` `<yk>` are literals of the element type.

Note: The argument to the `output_array` annotation is kept for compatibility with older versions of FlatZinc. Previous versions of the MiniZinc compiler generated multi-dimensional `output_array` annotations, and required the solver to output corresponding `arrayNd` calls (with `N` being the number of dimensions). Starting from MiniZinc version 2.7.8, all `output_array` annotations are guaranteed to be one-dimensional and 1-based, so that no `arrayNd` calls are required.

Using this format, the output of a FlatZinc model solution is suitable for input to a MiniZinc model as a data file (this is why parameters are not included in the output).

Implementations must ensure that *all* model variables (not just the output variables) have satisfying assignments before printing a solution.

The output for a solution must be terminated with ten consecutive minus signs on a separate line: `----------`.

Multiple solutions may be output, one after the other, as search proceeds. How many solutions should be output depends on the mode the solver is run in as controlled by the `-a` command line flag (see Section 4.3.5).

If at least one solution has been found and search then terminates having explored the whole search space, then ten consecutive equals signs should be printed on a separate line: `==========`.

If no solutions have been found and search terminates having explored the whole search space, then `=====UNSATISFIABLE=====` should be printed on a separate line.

If the objective of an optimization problem is unbounded, then `=====UNBOUNDED=====` should be printed on a separate line.

If no solutions have been found and search terminates having *not* explored the whole search space, then `=====UNKNOWN=====` should be printed on a separate line.

Implementations may output further information about the solution(s), or lack thereof, in the form of FlatZinc comments.

Examples:

Asking for a single solution to this model:

```
var 1..3: x :: output_var;
solve satisfy
```

might produce this output:

```
x = 1;
----------
```

Asking for all solutions to this model:

```
array [1..2] of var 1..3: xs :: output_array([1..2]);
constraint int_lt(xs[1], xs[2]);    % x[1] < x[2].
solve satisfy
```

might produce this output:

```
xs = array1d(1..2, [1, 2]);
----------
xs = array1d(1..2, [1, 3]);
----------
xs = array1d(1..2, [2, 3]);
----------
==========
```

Asking for a single solution to this model:

```
var 1..10: x :: output_var;
solve maximize x;
```

should produce this output:

```
x = 10;
----------
==========
```

The row of equals signs indicates that a complete search was performed and that the last result printed is the optimal solution.

Running a solver on this model with some termination condition (such as a very short time-out):

```
var 1..10: x :: output_var;
solve maximize x;
```

might produce this output:

```
x = 1;
----------
x = 2;
----------
x = 3;
----------
```

Because the output does not finish with ==========, search did not finish, hence these results must be interpreted as approximate solutions to the optimization problem.

Asking for a solution to this model:

```
var 1..3: x :: output_var;
var 4..6: y :: output_var;
constraint int_lt(y, x);    % y < x.
solve satisfy;
```

should produce this output:

```
=====UNSATISFIABLE=====
```

indicating that a complete search was performed and no solutions were found (i.e., the problem is unsatisfiable).

4.3.3.2 Statistics output

FlatZinc solvers can output statistics in a standard format so that it can be read by scripts, for example, in order to run experiments and automatically aggregate the results. Statistics should be printed to the standard output stream in the form of FlatZinc comments that follow a specific format. Statistics can be output at any time during the solving, i.e., before the first solution, between solutions, and after the search has finished. Statistics output corresponding to a solution should be the last one before its '----------' separator.

Each value should be output on a line of its own in the following format:

```
%%%mzn-stat: <name>=<value>
```

Each block of statistics is terminated by a line of its own with the following format:

```
%%%mzn-stat-end
```

Example

```
%%%mzn-stat: objective=1e+308
%%%mzn-stat: objectiveBound=0
%%%mzn-stat: nodes=0
%%%mzn-stat: solveTime=2.3567
%%%mzn-stat-end

(no feasible solution found yet but something can be printed...)

%%%mzn-stat: objective=12345
%%%mzn-stat: objectiveBound=122
%%%mzn-stat: nodes=35
%%%mzn-stat: solveTime=78.5799
%%%mzn-stat-end

(the corresponding feasible solution with value 12345 goes here
   or before its statistics but above the separator)
----------               (<- the solution separator)

%%%mzn-stat: objective=379
%%%mzn-stat: objectiveBound=379
%%%mzn-stat: nodes=4725
%%%mzn-stat: solveTime=178.5799
%%%mzn-stat-end

(the corr. optimal solution with value 379 goes here)
----------
==========               (<- the 'search complete' marker)

%%%mzn-stat: objective=379      (<- this is the concluding output)
%%%mzn-stat: objectiveBound=379
%%%mzn-stat: nodes=13456
```

```
%%%mzn-stat: solveTime=2378.5799
%%%mzn-stat-end
```

The <name> describes the kind of statistics gathered, and the <value> can be any value of a MiniZinc type. The following names are considered standard statistics:

Name	Type	Explanation
nodes	int	Number of search nodes
openNodes	int	Number of open search nodes
objective	float	Current objective value
objectiveBound	float	Dual bound on the objective value
failures	int	Number of leaf nodes that were failed
restarts	int	Number of times the solver restarted the search
variables	int	Number of variables
intVariables	int	Number of integer variables created
boolVariables	int	Number of bool variables created
floatVariables	int	Number of float variables created
setVariables	int	Number of set variables created
propagators	int	Number of propagators created
propagations	int	Number of propagator invocations
peakDepth	int	Peak depth of search tree
nogoods	int	Number of nogoods created
backjumps	int	Number of backjumps
peakMem	float	Peak memory (in Mbytes)
initTime	float	Initialisation time (in seconds)
solveTime	float	Solving time (in seconds)

4.3.3.3 Unsatisfiable core output (experimental)

Note:

Unsatisfiable core output is part of the **experimental** assumption interface (see Section 3.7) and may change in future versions of MiniZinc.

A solver that can solve under assumptions (see Section 3.7) can report an *unsatisfiable core*: a subset of the assumptions that is by itself sufficient to make the model unsatisfiable. Supporting this requires the solver to override the `fzn_assume` predicate in its solver-specific library. The core should be printed to the standard output stream as a FlatZinc comment of the following form:

```
%%%mzn-core: [<entry>, <entry>, ...]
```

The MiniZinc driver translates each <entry> back into the original MiniZinc expression before presenting the core to the user. For this translation to succeed, each <entry> must take one of the following two forms:

- An *identifier* that was passed as an element of the array argument of an `fzn_assume` call (i.e. one of the assumption variables).

- An expression of the form `<objective> <op> <const>`, where `<objective>` is the identifier of the objective variable, `<op>` is a comparison operator and `<const>` is a constant. This is used when optimising, to justify that the reported solution is optimal (the objective is bounded by `<const>`).

An entry of any other form (for example an arbitrary auxiliary variable, or a comparison over a non-objective variable) cannot be mapped back to a MiniZinc expression and is reported verbatim.

4.3.3.4 Error and warning output

Errors and warnings must be output to the standard error stream. When an error occurs, the implementation should exit with a non-zero exit code, signaling failure.

4.3.4 Solver-specific Libraries

Constraints in FlatZinc can call standard predicates as well as solver-specific predicates. Standard predicates are the ones that the MiniZinc compiler assumes to be present in all solvers. Without further customisation, the compiler will try to compile the entire model into a set of these standard predicates.

Solvers can use custom predicates and *redefine* standard predicates by supplying a *solver specific library* of predicate declarations. Examples of such libraries can be found in the binary distribution of MiniZinc, inside the `share/minizinc/gecode` and `share/minizinc/chuffed` directories.

The solver-specific library needs to be made available to the MiniZinc compiler by specifying its location in the solver's configuration file, see Section 4.3.6.

4.3.4.1 Standard predicates

FlatZinc solvers need to support the predicates listed as `FlatZinc builtins` in the library reference documentation, see Section 4.2.3.

Any standard predicate that is not supported by a solver needs to be *redefined*. This can be achieved by placing a file called `redefinitions.mzn` in the solver's MiniZinc library, which can contain alternative definitions of predicates, or define them as unsupported using the `abort` predicate.

Example for a `redefinitions.mzn`:

```
% Redefine float_sinh function in terms of exp
predicate float_sinh(var float: a, var float: b) =
    b == (exp(a)-exp(-a))/2.0;

% Mark float_tanh as unsupported
predicate float_tanh(var float: a, var float: b) =
    abort("The builtin float_tanh is not supported by this solver.");
```

The redefinition can use the full MiniZinc language. Note, however, that redefining builtin predicates in terms of MiniZinc expressions can lead to problems if the MiniZinc compiler translates the high-level expression back to the redefined builtin.

The reference documentation (Section 4.2.3) also contains sections on builtins that were added in later versions of MiniZinc. In order to maintain backwards compatibility with solvers that don't support these, they are organised in redefinition files with a version number attached, such as `redefinitions-2.0.mzn`. In order to declare support for these builtins, the solver-specific library must contain the corresponding redefinitions file, with the predicates either redefined in terms of other predicates, or declared as supported natively by the solver by providing a predicate declaration without a body.

Example for a `redefinitions-2.0.mzn` that declares native support for the predicates added in MiniZinc 2.0:

```
predicate bool_clause_reif(array[int] of var bool: as,
                           array[int] of var bool: bs,
                           var bool: b);
predicate array_int_maximum(var int: m, array[int] of var int: x);
predicate array_float_maximum(var float: m, array[int] of var float: x);
predicate array_int_minimum(var int: m, array[int] of var int: x);
predicate array_float_minimum(var float: m, array[int] of var float: x);
```

4.3.4.2 Solver-specific predicates

Many solvers have built-in support for some of the constraints in the MiniZinc standard library. But without declaring which constraints they support, MiniZinc will assume that they don't support any except for the standard FlatZinc builtins mentioned in the section above.

A solver can declare that it supports a non-standard constraint by overriding one of the files of the standard library in its own solver-specific library. For example, assume that a solver supports the `all_different` constraint on integer variables. In the standard library, this constraint is defined in the file `fzn_all_different_int.mzn`, with the following implementation:

```
predicate fzn_all_different_int(array[int] of var int: xs) =
  forall(i,j in index_set(xs) where i < j) ( xs[i] != xs[j] );
```

A solver, let's call it *OptiSolve*, that supports this constraint natively can place a file with the same name, `fzn_all_different_int.mzn`, in its library, and redefine it as follows:

```
predicate optisolve_alldifferent(array[int] of var int: xs);

predicate fzn_all_different_int(array[int] of var int: xs) =
  optisolve_alldifferent(xs);
```

When a MiniZinc model that contains the `all_different` constraint is now compiled with the *OptiSolve* library, the generated FlatZinc will contain calls to the newly defined predicate `optisolve_alldifferent`.

Note: The solver-specific library has been reorganised for MiniZinc version 2.3.0. Previously, a solver library would contain e.g. the file `bin_packing.mzn` in order to override the `bin_packing` constraint. With version 2.3.0, this is still possible (in order to maintain backwards compatibility). However, the predicate `bin_packing` from file `bin_packing.mzn`

now delegates to the predicate `fzn_bin_packing` in `fzn_bin_packing.mzn`. This enables the `bin_packing` predicate to check that the arguments are correct using assertions, before delegating to the solver-specific predicate. If your solver still uses the old library layout (i.e., overriding `bin_packing.mzn` instead of `fzn_bin_packing.mzn`), you should consider updating it to the new standard.

4.3.4.3 Parameter names in solver-specific libraries

The parameter names in the standard library's predicate declarations are part of the interface: they are the names by which a constraint's arguments may be given as named arguments (see *Call Expressions* (page 307)). When a solver redefines a standard predicate, its declaration should use the *same* parameter names as the standard-library declaration. The standard `fzn_all_different_int`, for instance, names its parameter `xs`, so a redefinition should too:

```
predicate fzn_all_different_int(array[int] of var int: xs);   % names match␣
↪the standard library
```

Using different names (for example `x` instead of `xs`) does not change how the predicate resolves — the compiler matches these low-level predicates by argument type, not by name, so a renamed library behaves exactly as before. It does, however, mean the constraint cannot be reached by those names once named arguments are enabled for native predicates.

To audit a library against the canonical names, compile any model with the `--warn-non-authoritative-names` option (Section 3.1). It reports every predicate declaration whose parameter names differ from the standard library's body-less declaration for the same signature.

Named arguments are being extended to native predicates in stages, so that existing solver libraries keep working while the ecosystem converges on a common set of names:

1. In the current release, native (body-less) predicates are positional-only. They resolve by argument type, named calls to them are rejected, and `--warn-non-authoritative-names` is off by default — so a library with divergent names still works unchanged.
2. In a later release, the warning will be on by default, prompting solver authors to align their libraries.
3. In a release after that, the divergence will become an error and named arguments will be enabled for native predicates.

Solver implementers are encouraged to adopt the standard-library parameter names now, so their libraries are ready when named arguments reach the native layer.

4.3.4.4 Reified and half-reified predicates

A reified constraint is a constraint that is not simply enforced, but whose truth value is bound to a Boolean variable. For example, a MiniZinc expression `var bool: b = all_different(x);` would constrain `b` to be true if and only if the variables `x` take pairwise different values.

If a predicate is called in such a reified context, the MiniZinc compiler will try to find a version of the predicate with `_reif` added to its identifier and an additional `var bool` argument. For the above example, the compiler will try to generate the following FlatZinc code:

```
var bool: b;
constraint all_different_reif(x, b);
```

If the `_reif` predicate does not exist, the compiler will try to use the definition of the original predicate. However, this may not be ideal: the original definition may make use of free variables in a `let` expression (which is not allowed in reified contexts), or it may lead to inefficient solving.

Solver libraries should therefore provide reified versions of constraints whenever possible. The library contains files `fzn_<constraintname>_reif.mzn` for this purpose.

When a reified constraint is used in a *positive context* (see Section 2.3.6), the MiniZinc compiler can use a special version, called a half-reified predicate and identified by an `_imp` suffix, instead of the `_reif` predicate. Half-reified predicates essentially represent constraints that are *implied* by a Boolean variable rather than being equivalent to one. This typically leads to simpler translations or more efficient propagation (e.g., a half-reified `all_different` only needs to *check* whether it is false, but it never has to implement the negation of the actual constraint).

For example, `constraint y=0 \/ all_different(x)` might be translated as follows:

```
var bool: X_INTRODUCED_1;
var bool: X_INTRODUCED_2;
constraint int_eq_imp(y,0,X_INTRODUCED_1);
constraint all_different_imp(x, X_INTRODUCED_2);
constraint bool_clause([X_INTRODUCED_1,X_INTRODUCED_2]);
```

MiniZinc will decide whether to use half-reification case by case based on the availability of the `_imp` predicate. As for reified constraints, it may be benefitial to provide specialised half-reified versions if the solver supports them.

4.3.5 Command-Line Interface and Standard Options

In order to work with the `minizinc` command line driver, a FlatZinc solver must be an executable (which can include e.g. shell scripts) that can be invoked as follows:

```
$ <executable-name> [options] model.fzn
```

where `<executable-name>` is the name of the executable. Solvers may support the following standard options:

`-a`

: Instructs the solver to report *all* solutions in the case of satisfaction problems, or print *intermediate* solutions of increasing quality in the case of optimisation problems.

`-n <i>`
: Instructs the solver to stop after reporting `i` solutions (only used with satisfaction problems).

`-i`
: Instructs the solver to print *intermediate* solutions of increasing quality (only used with optimisation problems). This option should be supported rather than `-a` for solvers which only support printing of intermediate solutions for optimisation problems but no reporting of all solutions for satisfaction problems.

`-f`
: Instructs the solver to conduct a "free search", i.e., ignore any search annotations. The solver is not *required* to ignore the annotations, but it is *allowed* to do so.

`-s`
: Print statistics during and/or after the search for solutions. Statistics should be printed as FlatZinc comments to the standard output stream. See Section 4.3.3.2 for the standard format for statistics.

`-v`
: Print log messages (verbose solving) to the standard error stream. If solvers choose to print to standard output instead, all messages must be valid comments (i.e., start with a `%` character).

`-p <i>`
: Run with `i` parallel threads (for multi-threaded solvers).

`-r <i>`
: Use `i` as the random seed (for any random number generators the solver may be using).

`-t <ms>`
: Wall time limit `ms` milliseconds.

4.3.6 Solver Configuration Files

In order for a solver to be available to MiniZinc, it has to be described in a *solver configuration file*. This is a simple file, in JSON or `.dzn` format, that contains some basic information such as the solver's name, version, where its library of global constraints can be found, and a path to its executable. Examples are given in Section 3.5.

A solver configuration file must have file extension `.msc` (for MiniZinc Solver Configuration), and can be placed in any of the following locations:

- In the `minizinc/solvers/` directory of the MiniZinc installation. If you install MiniZinc from the binary distribution, this directory can be found at `/usr/share/minizinc/solvers` on Linux systems, inside the MiniZincIDE application on macOS system, and in the `Program Files\MiniZinc IDE (bundled)` folder on Windows.
- In the directory `$HOME/.minizinc/solvers` on Linux and macOS systems, and the Application Data directory on Windows systems.
- In any directory listed on the `MZN_SOLVER_PATH` environment variable (directories are separated by `:` on Linux and macOS, and by `;` on Windows systems).

- In any directory listed in the `mzn_solver_path` option of the global or user-specific configuration file (see Section 3.1.4)
- Alternatively, you can use the MiniZinc IDE to create solver configuration files, see Section 3.2.5.2 for details.

You can also query the `minizinc` driver about these directories using the `--config-dirs` command line option.

Solver configuration files must be valid JSON or `.dzn` files. As a JSON file, it must be an object with certain fields. As a `.dzn` file, it must consist of assignment items.

For example, a simple solver configuration in JSON format could look like this:

```
{
  "name" : "My Solver",
  "version": "1.0",
  "id": "org.myorg.my_solver",
  "executable": "fzn-mysolver"
}
```

The same configuration in `.dzn` format would look like this:

```
name = "My Solver";
version = "1.0";
id = "org.myorg.my_solver";
executable = "fzn-mysolver";
```

Here is a list of all configuration options recognised by the configuration file parser. Any valid configuration file must at least contain the fields `name`, `version`, `id`, and `executable`.

- `name` (string, required): The name of the solver (displayed, together with the version, when you call `minizinc --solvers`, and in the MiniZinc IDE).
- `version` (string, required): The version of the solver.
- `id` (string, required): A unique identifier for the solver, "reverse domain name" notation.
- `executable` (string or list of strings, required): The executable for this solver that can run FlatZinc files. This can be just a file name (in which case the solver has to be on the current PATH), an absolute path to the executable, or a relative path (which is interpreted relative to the location of the configuration file). When a list of strings is provided, the first string is treated as the executable and consecutive strings are treated as arguments or flags that are always passed to the executable (i.e., arguments and flags that are not configurable by the solver user).
- `mznlib` (string, default `""`): The solver-specific library of global constraints and redefinitions. This should be the name of a directory (either an absolute path or a relative path, interpreted relative to the location of the configuration file). For solvers whose libraries are installed in the same location as the MiniZinc standard library, this can also take the form `-G<solverlib>`, e.g., `-Ggecode` (this is mostly the case for solvers that ship with the MiniZinc binary distribution).

- `tags` (list of strings, default empty): Each solver can have one or more tags that describe its features in an abstract way. Tags can be used for selecting a solver using the `--solver` option. There is no fixed list of tags, however we recommend using the following tags if they match the solver's behaviour:
  - `"cp"`: for Constraint Programming solvers
  - `"mip"`: for Mixed Integer Programming solvers
  - `"float"`: for solvers that support float variables
  - `"api"`: for solvers that use the internal C++ API
- `stdFlags` (list of strings, default empty): Which of the standard solver command line flags are supported by this solver. The standard flags are `-a`, `-n`, `-i`, `-s`, `-v`, `-p`, `-r`, `-f`, `-t`.
- `extraFlags` (list of list of strings, default empty): Extra command line flags supported by the solver. Each entry must be a list of four strings. The first string is the name of the option (e.g. `"--special-algorithm"`). The second string is a description that can be used to generate help output (e.g. `"which special algorithm to use"`). The third string specifies the type of the argument (`"int"`, `"bool"`, `"float"`, `"string"` or `"opt"`). The fourth string is the default value. The following types have an additional extended syntax:
  - `"int:n:m"` where `n` and `m` are integers, gives lower and upper bounds for the supported values
  - `"float:n:m"` where `n` and `m` are floating point numbers, gives lower and upper bounds for the supported values
  - `"bool:onstring:offstring"` specifies strings to add to the command line flag to turn it on (`onstring`) and off (`offstring`). E.g., `["-interrupt","whether to catch Ctrl-C","bool:yes:no","yes"]` specifies a command line option that can be called as `-interrupt yes` or `-interrupt no`. The standard behaviour (just `"bool"`) means that the option is either added to the command line or not.
  - `"opt:first option:second option:...:last option"` specifies a list of possible values for the option
- `inputType` (string, default `FZN`, options `MZN`, `FZN`, `NL`, `JSON`): The type of input expected by the solver.
- `supportsMzn` (bool, default `false`): Whether the solver can run MiniZinc directly (i.e., it implements its own compilation or interpretation of the model). [Deprecated, use `inputType`]
- `supportsFzn` (bool, default `true`): Whether the solver can run FlatZinc. This should be the case for most solvers. [Deprecated, use `inputType`]
- `supportsNL` (bool, default `false`): Whether the solver supports NL input. Used for non-linear solvers. [Deprecated, use `inputType`]
- `needsSolns2Out` (bool, default `true`): Whether the output of the solver needs to be passed through the MiniZinc output processor.
- `needsMznExecutable` (bool, default `false`): Whether the solver needs to know the location of the MiniZinc executable. If true, it will be passed to the solver using the `mzn-executable` option.

- `needsStdlibDir` (bool, default `false`): Whether the solver needs to know the location of the MiniZinc standard library directory. If true, it will be passed to the solver using the `stdlib-dir` option.
- `isGUIApplication` (bool, default `false`): Whether the solver has its own graphical user interface, which means that MiniZinc will detach from the process and not wait for it to finish or to produce any output.
- `isInteractive` (bool, default `false`): Whether the solver runs an interactive command-line session. When set, MiniZinc lets the solver read the user's terminal `stdin` directly (so the user can type commands to it) and echoes the solver's `stdout` back verbatim, *except* for solution blocks. A solution block is the text bracketed by the marker lines `%%%mzn-sol-begin` and `%%%mzn-sol-end`; the lines between those markers must be standard FlatZinc solution output (variable assignments terminated by `----------`) and are passed through the normal output processing (so they are mapped through the model's output item). The marker lines themselves are consumed. This is equivalent to passing `--fzn-interactive` to the generic FlatZinc driver. Because the solver's `stdout` is a pipe (not a terminal) in this mode, an interactive solver must flush its prompts and replies.

4.3.7 Grammar

This is the full grammar for FlatZinc. It is a proper subset of the MiniZinc grammar (see Section 4.1.14). However, instead of specifying all the cases in the MiniZinc grammar that do *not* apply to FlatZinc, the BNF syntax below contains only the relevant syntactic constructs. It uses the same notation as in Section 4.1.2.

```
% A FlatZinc model
<model> ::=
  [ <predicate-item> ]*
  [ <par-decl-item> ]*
  [ <var-decl-item> ]*
  [ <constraint-item> ]*
  <solve-item>

% Predicate items
<predicate-item> ::= "predicate" <identifier> "(" [ <pred-param-type> ":"
↪<identifier> "," ... ] ")" ";"

% Identifiers
<identifier> ::= [A-Za-z][A-Za-z0-9_]*

<basic-par-type> ::= "bool"
                   | "int"
                   | "float"
                   | "set of int"

<basic-var-type> ::= "var" <basic-par-type>
                   | "var" <int-literal> ".." <int-literal>
                   | "var" "{" <int-literal> "," ... "}"
                   | "var" <float-literal> ".." <float-literal>
```

```
                  | "var" "set" "of" <int-literal> ".." <int-literal>
                  | "var" "set" "of" "{" [ <int-literal> "," ... ] "}"

<array-var-type> ::= "array" "[" <index-set> "]" "of" <basic-var-type>

<index-set> ::= "1" ".." <int-literal>

<basic-pred-param-type> ::= <basic-par-type>
                          | <basic-var-type>
                          | <int-literal> ".." <int-literal>
                          | <float-literal> ".." <float-literal>
                          | "{" <int-literal> "," ... "}"
                          | "set" "of" <int-literal> .. <int-literal>
                          | "set" "of" "{" [  <int-literal> "," ... ] "}"

<pred-param-type> ::= <basic-pred-param-type>
                    | "array" "[" <pred-index-set> "]" "of"
↪<basic-pred-param-type>

<pred-index-set> ::= <index-set>
                   | "int"

<basic-literal-expr> ::= <bool-literal>
                       | <int-literal>
                       | <float-literal>
                       | <set-literal>

<basic-expr> ::= <basic-literal-expr>
               | <var-par-identifier>

<expr>       ::= <basic-expr>
               | <array-literal>

<var-par-identifier> ::= [A-Za-z_][A-Za-z0-9_]*

% Boolean literals
<bool-literal> ::= "false"
                 | "true"

% Integer literals
<int-literal> ::= [-]?[0-9]+
                | [-]?0x[0-9A-Fa-f]+
                | [-]?0o[0-7]+

% Float literals
<float-literal> ::= [-]?[0-9]+.[0-9]+
                  | [-]?[0-9]+.[0-9]+[Ee][-+]?[0-9]+
                  | [-]?[0-9]+[Ee][-+]?[0-9]+

% Set literals
```

```
<set-literal> ::= "{" [ <int-literal> "," ... ] "}"
                | <int-literal> ".." <int-literal>
                | "{" [ <float-literal> "," ... ] "}"
                | <float-literal> ".." <float-literal>

<array-literal> ::= "[" [ <basic-expr> "," ... ] "]"

<par-array-literal> ::= "[" [ <basic-literal-expr> "," ... ] "]"

% Parameter declarations

<par-decl-item> ::= "array" "[" <index-set> "]" "of" <basic-par-type> ":"
↪<var-par-identifier> "=" <par-array-literal> ";"

% Variable declarations

<var-decl-item> ::= <basic-var-type> ":" <var-par-identifier> <annotations>␣
↪[ "=" <basic-expr> ] ";"
                  | <array-var-type> ":" <var-par-identifier> <annotations>
↪"=" <array-literal> ";"

% Constraint items

<constraint-item> ::= "constraint" <identifier> "(" [ <expr> "," ... ] ")"
↪<annotations> ";"

% Solve item

<solve-item> ::= "solve" <annotations> "satisfy" ";"
               | "solve" <annotations> "minimize" <basic-expr> ";"
               | "solve" <annotations> "maximize" <basic-expr> ";"

% Annotations

<annotations> ::= [ "::" <annotation> ]*

<annotation> ::= <identifier>
               | <identifier> "(" <ann-expr> "," ... ")"

<ann-expr>   := <basic-ann-expr>
              | "[" [ <basic-ann-expr> "," ... ] "]"

<basic-ann-expr>   := <basic-literal-expr>
                    | <var-par-identifier>
                    | <string-literal>
                    | <annotation>

<string-contents> ::= ([^"\n\] | \[^\n(])*

<string-literal> ::= """ <string-contents> """
```

```
% End of FlatZinc grammar
```

CHAPTER 4.4

Machine-readable JSON output format

MiniZinc supports a machine-readable output format activated with the `--json-stream` (see *Options* (page 153)) option.

In this mode, MiniZinc outputs a line-break delimited stream of JSON objects (called messages). These are output to the standard output stream. The standard error stream is not affected by this mode. Each message contains the key `type` with a string value denoting the kind of message.

A note on the message format

In the following sections, messages are formatted on multiple lines for readability, however actual messages are always a single line.

4.4.1 Solution messages

```
{
  "type": "solution",
  "time": 1000,
  "output": {
    "foo": "foo output section",
    "bar": "bar output section"
  },
  "sections": ["foo", "bar"]
}
```

This message is produced whenever a solution is output (at the point where a `----------` would typically be produced).

The `time` field is optional and is only present if run with `--output-time`. It gives the elapsed time in milliseconds.

The `output` field contains an object mapping output section names to their contents. Output section contents can be either:

- A string containing the output value
- A JSON value if the section is the `json` section (generated when using `--output-mode json`), or if the section was specified using the `json_section` annotation.

The `sections` field contain a list of output section names (strings) to show the order that they were produced.

Note that all sections are included in the `output` key object. The resulting output for enabled sections is available in the `raw` section.

4.4.2 Checker messages

```
{
  "type": "checker",
  "messages": [
    {
      "type": "solution",
      "output": {
        "default": "foo output section"
      },
      "sections": ["default"]
    }
  ]
}
```

This message gives solution checker results and is produced directly preceding the solution it is associated with. The `messages` field gives a list of JSON stream messages produced by the solution checker (usually a `solution` message).

4.4.3 Status messages

```
{
  "type": "status",
  "status": "ALL_SOLUTIONS",
  "time": 1000
}
```

This message is produced when a final status message is received from a solver (at the point where a status line such as `==========` would be produced).

The `status` field contains the name of the solver status and can take one of the follow values:

- `"ALL_SOLUTIONS"`
- `"OPTIMAL_SOLUTION"`

- "UNSATISFIABLE"
- "UNBOUNDED"
- "UNSAT_OR_UNBOUNDED"
- "UNKNOWN"
- "ERROR"

The `time` field is optional and is only present if run with `--output-time`. It gives the elapsed time in milliseconds.

4.4.4 Statistics messages

```
{
  "type": "statistics",
  "statistics": {
    "method": "satisfy",
    "flatTime": 1000
  }
}
```

This message is produced whenever a set of statistics is output (at the point where `%%%mzn-stat-end` would typically be produced).

The `statistics` field contains an object mapping statistics names to their values.

4.4.5 Timestamp messages

```
{
  "type": "time",
  "time": 1000
}
```

This message is produced to indicate the current solve time in a standalone way. For example, when running with `--canonicalize` and `--output-time`, solution messages are printed at the end, however as solutions are produced, `time` messages are output.

The `time` field gives the elapsed time in milliseconds.

4.4.6 Comment messages

```
{
  "type": "comment",
  "comment": "% comment produced by solver\n"
}
```

This message is produced when a solver outputs a comment.

The `comment` field contains the comment as a string (including the leading `%` and trailing newline).

4.4.7 Trace messages

```
{
  "type": "trace",
  "section": "default",
  "message": "traced message\n"
}
```

This message is produced when a `trace_stdout`, `trace_to_section()` or similar call is evaluated.

The `section` field gives the section name as a string (`default` when using `trace_stdout`).

The `message` field gives the message contents as a string, or as a JSON value if appropriate (such as when using `trace_exp`).

Note that `trace()` calls output to standard error as normal.

4.4.8 Profiling messages

```
{
  "type": "profiling",
  "entries": [...]
}
```

This message is produced when `--output-detailed-timing`

The `entries` field contains a list of timing information entry objects.

Entries have the following format:

```
{
  "filename": "model.mzn",
  "line": 1,
  "time": 100
}
```

4.4.9 Paths messages

```
{
  "type": "paths",
  "paths": [...]
}
```

This message is produced when `--output-paths-to-stdout` is used.

The `paths` field contains a list of objects representing either a variable path or a constraint path.

Variable paths have the following format:

```
{
  "flatZincName": "X_INTRODUCED_0_",
  "niceName": "x[1]",
  "path": "model.mzn|1|27|1|27|id|x;|0|0|0|0|il|0;"
}
```

Constraint paths have the following format:

```
{
  "constraintIndex": 9,
  "path":
↪"model.mzn|3|12|3|59|ca|forall;model.mzn|3|12|3|59|ac;model.mzn|3|20|3|20|i=4;model.mzn|3|2
↪'!=';model.mzn|3|47|3|58|ca|int_lin_ne;"
}
```

4.4.10 Error messages

```
{
  "type": "error",
  "what": "type error",
  "location": {...},
  "message": "cannot determine coercion from type float to type var int"
}
```

This message is produced when an error occurs.

The `what` field contains the kind of error as a string.

The `message` field contains the error message as a string.

Error messages can also optionally contain one or both of:

- A `location` field containing a location object as described in *Locations* (page 556).

- A `stack` field containing a list of stack trace objects as described in *Stack traces* (page 556).

Some kinds of error messages have additional properties (depending on the `what` of the error message):

- The `syntax error` message can optionally contain the field `includedFrom` giving a list of file names.
- The `cyclic include error` message contains the field `cycle` giving a list of file names.

4.4.10.1 Locations

Locations in a file are represented as follows:

```
{
  "filename": "model.mzn",
  "firstLine": 1,
  "firstColumn": 1,
  "lastLine": 3,
  "lastColumn": 10
}
```

4.4.10.2 Stack traces

Stack traces are represented as a list of objects with the following format:

```
{
  "location": {...}
  "isCompIter": false,
  "description": "variable declaration"
}
```

4.4.11 Warning messages

```
{
  "type": "warning",
  "location": {...},
  "stack": [...],
  "message": "Warning message"
}
```

This message is produced when a warning is emitted.

The optional `location` field contains the relevant location (see *Locations* (page 556)) if present.

The optional `stack` field contains a stack trace (see *Stack traces* (page 556)) if present.

The `message` field contains the warning message as a string.

Note that if `-Werror` is used then `error` messages are produced instead.

4.4.12 Core messages

```
{
  "type": "core",
  "core": ["x + y < 5", "x > 8"]
}
```

This message is produced (in place of the `%%%mzn-core` comment line) when a solver reports an unsatisfiable core for a model that uses the experimental *assumption interface* (page 225). The `core` field contains the assumptions in the core, as strings, in terms of the original MiniZinc expressions.

A solver may also produce this message to justify that a solution is optimal.